

V93000

Digital User Training

7.4.0

TEXTBOOK

An abstract graphic consisting of numerous overlapping, semi-transparent blue folders or document icons. The folders are arranged in a dense, layered fashion, creating a sense of depth and organization. The colors range from light blue to a darker, more saturated blue. The graphic occupies the lower half of the page, starting below the 'TEXTBOOK' title and extending to the bottom edge.

Advantest V93000

Digital User Training

Rev. 7.4.x

Agenda

ADVANTEST[®]

Legal Notices

Notices

All rights reserved. All text and figures included in this publication are the exclusive property of Advantest Corporation. Reproduction of this publication in any manner without the written permission of Advantest Corporation is prohibited. Information in this document is subject to change without notice.

Trademarks and Registered Trademarks

- ADVANTEST is a trademark of Advantest Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Warranty

The material contained in this document is provided "as is," and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Advantest disclaims all warranties, either express or implied, with regard to this material and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Advantest shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this material or of any information contained herein. Should Advantest and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology Licenses

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

Restricted Rights Legend

If software is for use in the performance of a U.S. Government prime contract or subcontract, Software is delivered and licensed as "Commercial computer software" as defined in DFAR 252.227-7014 (June 1995), or as a "commercial item" as defined in FAR 2.101(a) or as "Restricted computer software" as defined in FAR 52.227-19 (June 1987) or any equivalent agency regulation or contract clause. Use, duplication or disclosure of Software is subject to Advantest's standard commercial license terms for 93000 systems and non-DOD Departments and Agencies of the U.S. Government will receive no greater than Restricted Rights as defined in FAR 52.227-19(c)(1-2) (June 1987). U.S. Government users will receive no greater than Limited Rights as defined in FAR 52.227-14 (June 1987) or DFAR 252.227-7015 (b)(2) (November 1995), as applicable in any technical data.

Safety Notices



A CAUTION notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a CAUTION notice until the indicated conditions are fully understood and met.



A WARNING notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in death or severe injury. Do not proceed beyond a WARNING notice until the indicated conditions are fully understood and met.

Agenda - Monday

Morning	00 Introduction/Agenda Lecture 01: ESD Lecture 02: Hardware Overview Lecture/Demo 03: Software Overview Lab 1: Software Overview Lab 2: Creating Device
Afternoon	Lecture 04: Digital Pins Lab 3: Dual Site Pin Configuration Lecture 05: Digital Levels Lab 4: Defining Levels

ADVANTEST[®]

Agenda - Tuesday

Morning	Lecture 06: Digital Timing Lab 5: Defining Timing Lecture 07: Pattern Debug Tool and Digital Vectors Lab 6: Vectors and Pattern Management
Afternoon	Lecture 08: Calibration Demo: Calibration (Loadboard Calibration) Lecture 09: Testflow Lab 7: Testflow (Flow Data/Flow Sequence)

ADVANTEST[®]

Agenda - Wednesday

Morning	Lecture 10: Test Method Introduction (UTM) and library Lecture 11: Test Method Debugging Lab 8: Test Method Parameters and Limits Lecture12 : Test Table Lecture 13 : DC Resource Programming Lecture 14: Datalogging
Afternoon	Lab 9: User Test Methods / Flex DC Lab 10: Datalogging & Test Time Measurement

ADVANTEST[®]

Agenda – Thursday

Morning	Lecture 15: SmartRDI Introduction and usage Lecture 16: DC and functional tests with SmartRDI Lab 11: RDI Test Methods Lab 12: DC Tests with SmartRDI
Afternoon	Lecture 17: Digital test Debugging (Timing Diagram, Error Map, Pattern Debug) Lab 13: Digital Debugging Lecture 18: Digital characterization (optional) Lab 14: Characterization (optional)

ADVANTEST[®]

Agenda - Friday

Morning	Lecture 19: Digital Shmoo Lab 15: Digital Shmoo Lecture 20: Test Program in Production Environment Lecture 21 : Integrated result Tool Lab 16: Integrated result tool workflow Lecture 22: Pin Margin Tool (optional)
Afternoon	Lab 17: Pin Margin Tool (optional) Lab 18: Test Program in production environment Lab 19: Advanced SmarTest features (optional) Lecture 23: X-Mode Introduction (optional) Lecture 24: Multiport Testing (optional) Lecture 25: Smartest Datalink (optional)

ADVANTEST[®]

ADVANTEST[®]

Advantest V93000

Digital User Training

ESD and System Safety

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Overview
- Prevention

ADVANTEST[®]

Learning Objective

- Understand the need to care about ESD (**e**lectro**s**tatic **d**ischarge)

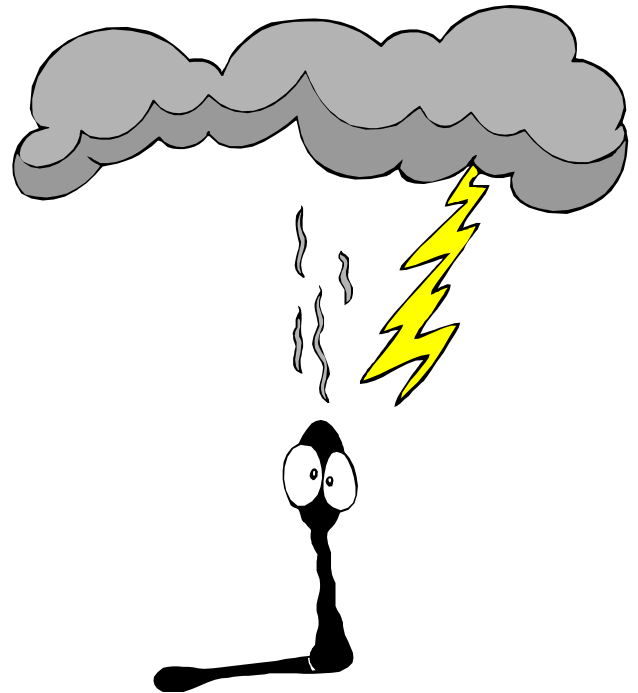
Relevance

- Understand that it is important to protect DUT, DUT board and Test System.

ESD Overview

What is ESD?

Electrostatic Discharge is a single-event, rapid transfer of electrostatic charge between two objects.



ADVANTEST

ESD Facts

- ESD is Electrostatic Discharge (lightning is a form of ESD)
- Static electricity fields of over 100 V/cm are able to damage sensitive electronic devices
- Static electricity greater than 10,000 V can be easily produced through rubbing your hands, feet, clothes, etc.



ADVANTEST

ESD

ESD Safe practices are required to maximize system uptime



- ... Ensure you are ESD protected!
- Wear ESD shoes
- Wear ESD coat
- Wear ESD wrist strap

The number one cause of ESD damage to the 93000 system is ESD Generators near the system

To verify an ESD safe environment an ESD Field Strength Meter must be used

Sources for ESD Field Strength Meter

<http://www.trifield.com/SurfaceDCVoltmeter.htm>
<http://www.absolutesci.com/shopexd.asp?id=22139>
<http://www.anti-staticmat.com/PrecisionElectrostaticLocatorMeter.htm>
<http://www.electrotechsystems.com/ProductInfo.asp?ProdID=73>



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

ESD and System Safety Unit 1 - 7

Problem Sources

Rubber gloves



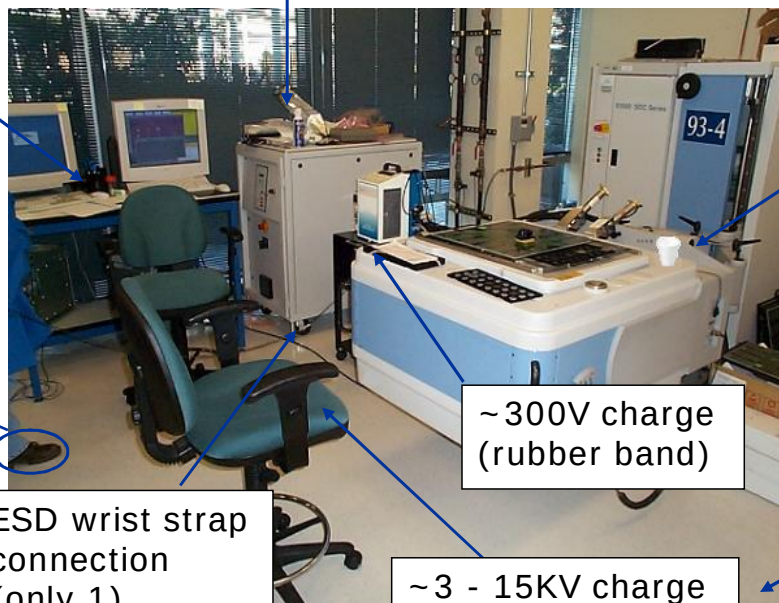
Non-ESD safe plastic bags



- ... Ensure you are ESD protected!
- Wear ESD shoes
- Wear ESD coat
- Wear ESD wrist strap

Tape dispenser

Non-ESD shoes or no ESD straps



ESD wrist strap connection (only 1)

~300V charge (rubber band)

~3 - 15KV charge (all 3 chairs in the area)

Styrofoam cups

DUT board not stored properly

Non-ESD floor wax

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

ESD and System Safety Unit 1 - 8

Problem Sources



- ... Ensure you are ESD protected!
- Wear ESD shoes
- Wear ESD coat
- Wear ESD wrist strap



ADVANTEST

April 2016

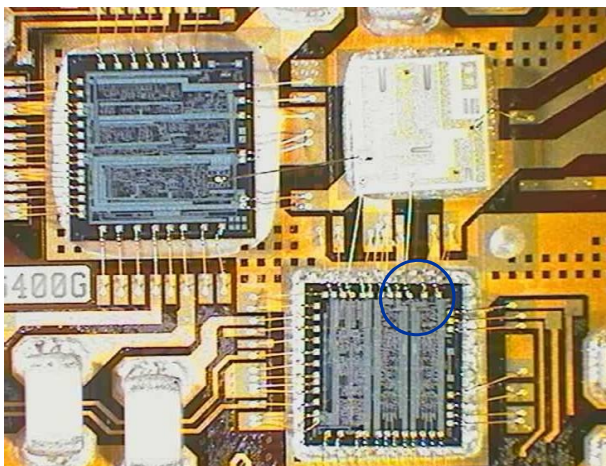
All Rights Reserved - ADVANTEST CORPORATION

ESD and System Safety Unit 1 - 9

Component Damage

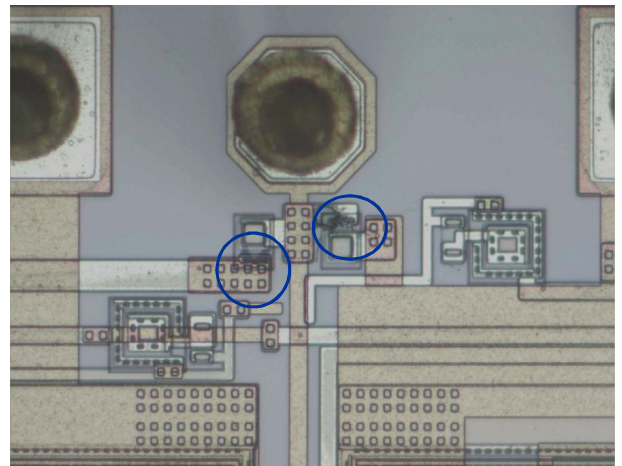
V93000 Channel Module ESD Damage

Digital Channel Driver /
Active Load



- ... Ensure you are ESD protected!
- Wear ESD shoes
- Wear ESD coat
- Wear ESD wrist strap

Digital Channel Active Load /
Comparator (x200)



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

ESD and System Safety Unit 1 - 10

Safety Symbols

The apparatus is marked with the following symbol when it is necessary for the user to refer to this manual in order to protect the apparatus from damage:



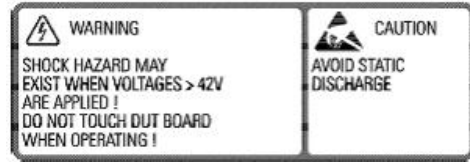
This symbol is used to mark Protective Earth terminals:



This symbol is used to mark high voltage terminals:



The test head is provided with the following label to denote a potential shock hazard, and to prevent damage by static discharge:



This symbol is used to mark areas inside the equipment where injury to the hands from moving parts could occur. Exercise particular care when components are pneumatically or motor driven:



ADVANTEST

Damage Prevention Documentation

In the V93000 Service Documentation, you can find the following:

[ESD Damage Prevention \(Topic 74387\)](#)

- ☐  ESD Damage Prevention
 -  General Information
 -  What to keep out of your ESD-protected area
 -  What to keep in your ESD-protected area
 -  What to wear in your ESD-protected area
 -  Precautions to take in your ESD-protected area

ADVANTEST

Advantest V93000

Digital User Training

Hardware Overview

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Scalable V93000 Platform
- Workstations
- Major Components
- Device Power Supplies
- Utility Line Block
- Sequencer controlled utility
- Digital Channel Card Architecture
- Parametric Measurement Units
- Analog Modules
- Port Scale RF Subsystem Components

ADVANTEST[®]

Learning Objectives

- Know the scalable architecture of the V93000 SOC test system
- Know the components of the V93000 SOC test system
- Know the characteristics of the device power supply, digital and analog boards of the V93000 SOC test system

Relevance

- Understand the structure of the scalable platform architecture

V93000 Scalable Platform



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 5

V93000 Scalable Platform

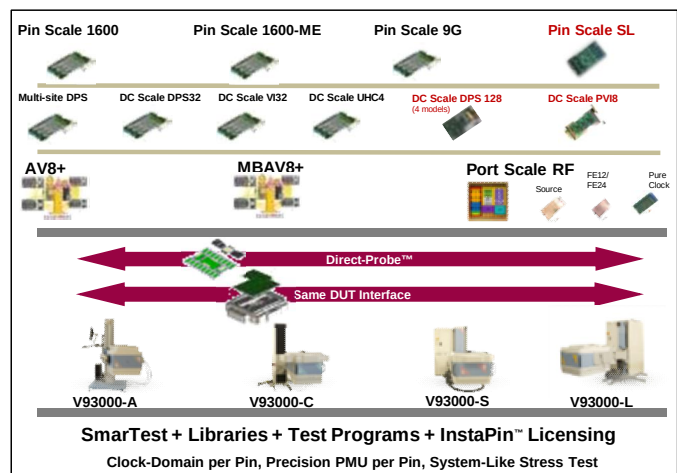
Fundamentals & Values

- All classes of V93000 are compatible
- The class determines the possible size of the configuration
- All features and performance points are available in all classes
- The classes allow V93000 to fit the class of the tested device

All instruments can be mixed and matched in all V93000 classes

The identical loadboard can be used in all V93000 classes
Direct Probe available in all classes

Same SmarTest System SW in all V93000 classes.
Same Test Programs, Licensing,
Same Test Methodology



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 6

V93000 Scalable Platform - Smart Scale

Pin Scale 1600

128 pins at 1.6G
1mV accuracy
Universal pin



Pin Scale 1600-ME

64 pins at 1.6G
1066Mbps data rate
Up to 512MByte emulated RAM



Pin Scale 9G

64 pins at >8G
Parallel and serial interfaces
PHY test



Pin Scale SL

Pattern + Loopback
Native protocol support (e.g. PCIe)
32 pins / 8 diff Tx/Rx lanes



Multi-site DPS



DC Scale DPS32



DC Scale VI32



DC Scale UHC4



DC Scale DPS128

(4 models)



DC Scale PVI8



AV8+



Consumer
Mixed Signal,
Superior Throughput

MBAV8+



LTE Advanced,
802.11ac, WiMAX,
Zero Overhead Upload

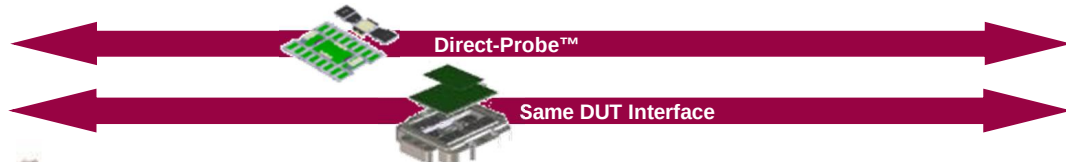
Port Scale RF



Source

FE12/
FE24

Pure
Clock



V93000-A



V93000-C



V93000-S



V93000-L

SmarTest + Libraries + Test Programs + InstaPin™ Licensing

Clock-Domain per Pin, Precision PMU per Pin, System-Like Stress Test

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 7

V93000 SOC Tester Platform

Support Rack
Compressed Air
Main power
Supply

Manipulator

OFF /Emergency

DUT
interface

Testhead with all pin
electronics

Workstation



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 8

V93000 SOC Host Workstation

LINUX PC

- Fiber-optic data transfer to V93000 test system
- Multi-core CPU
- Standardized to Red-Hat Enterprise OS, Z workstation series
- V93000 SW runs in offline mode for test-program development

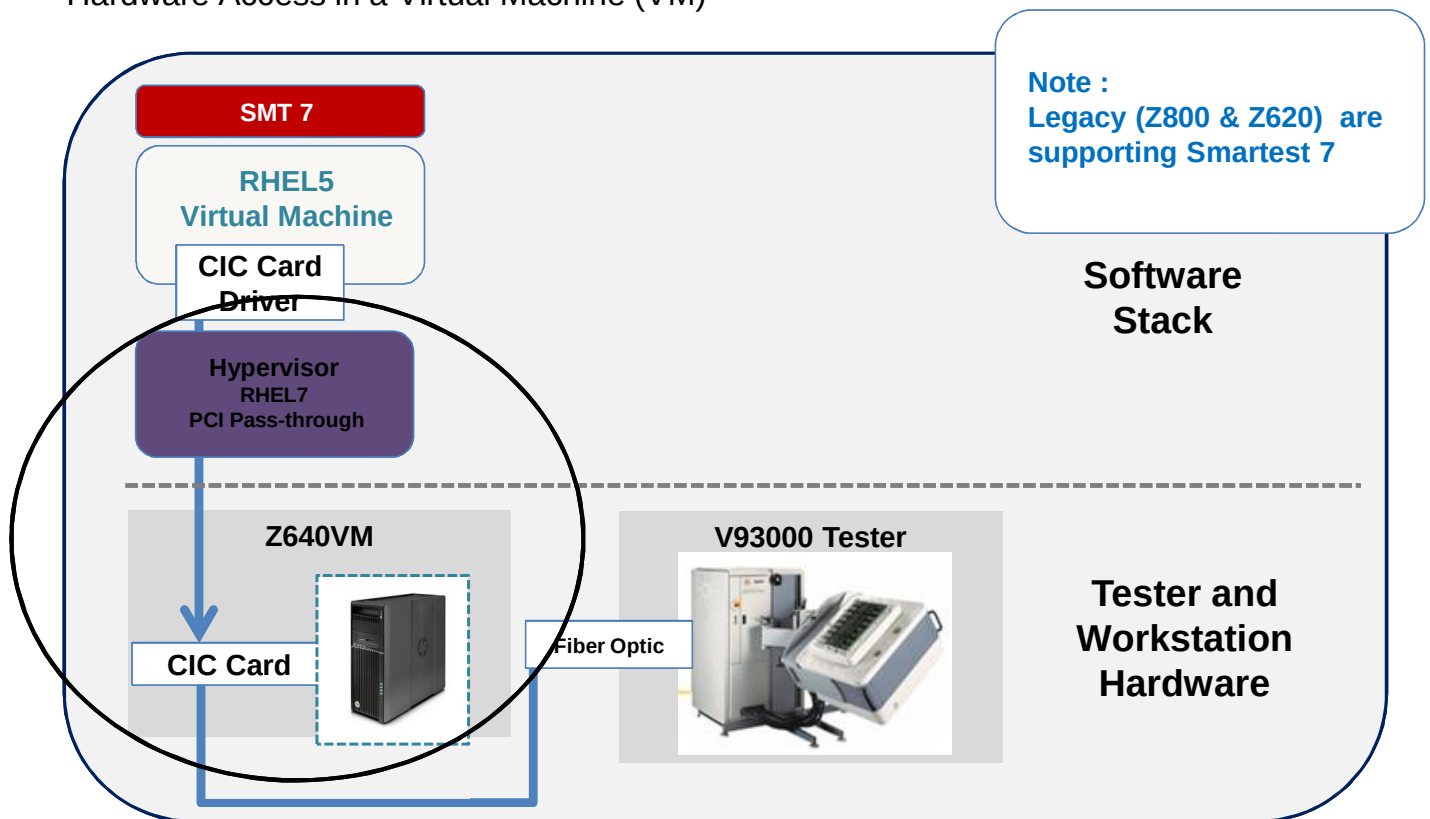


Workstation and Memory

Workstation	Memory as configured	Comment
Z800	16GB	
Z620	32GB	
Z640VM	64GB	•Virtual Machine

V93000 Virtualization with Z640VM

Hardware Access in a Virtual Machine (VM)



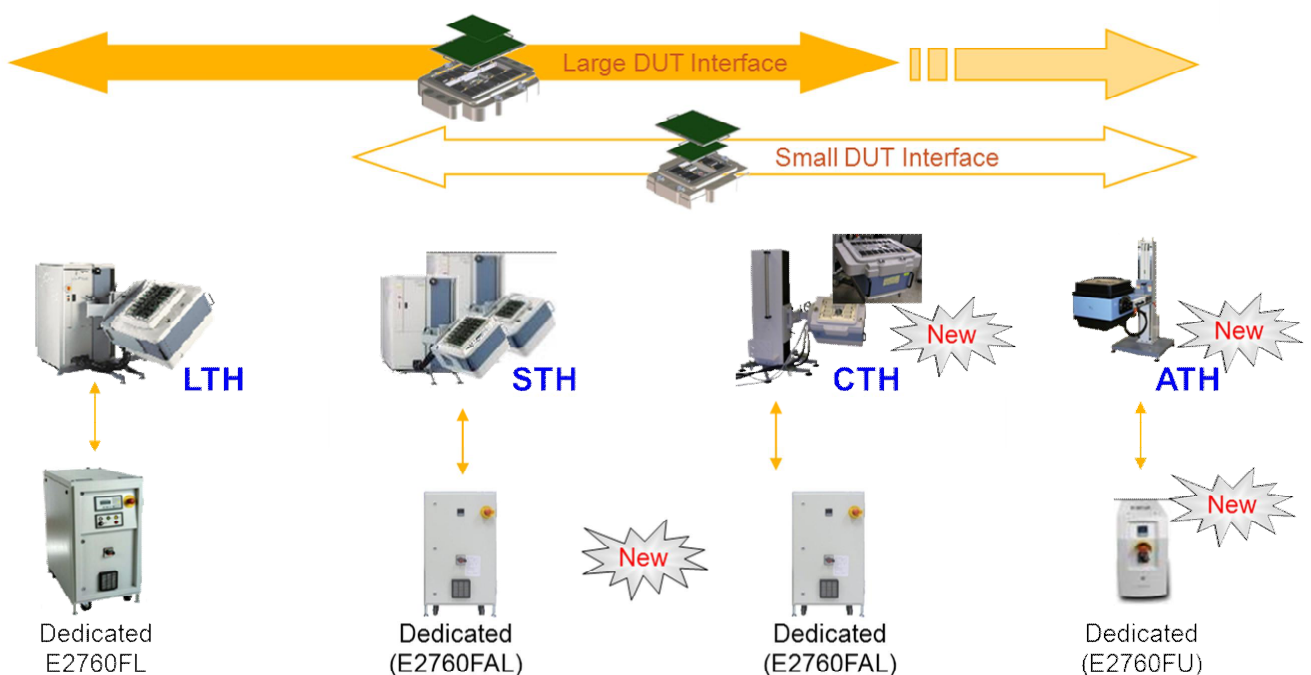
RHEL7 as the host operating system will be used in a minimal configuration to act as **HYPERVISOR** and just provide the workstation HW support and the KVM virtualization layer **ADVANTEST®**

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 11

Water Cooling technology



ADVANTEST®

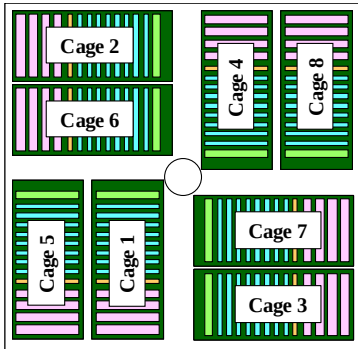
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 12

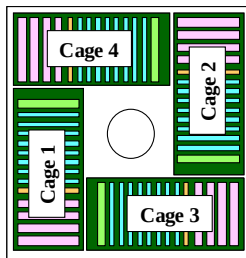
V93000 SOC Family Overview

L Testhead
64 Slots



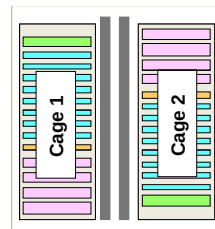
S Testhead

32 Slots



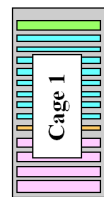
C Testhead

16 Slots



A Testhead

8 Slots

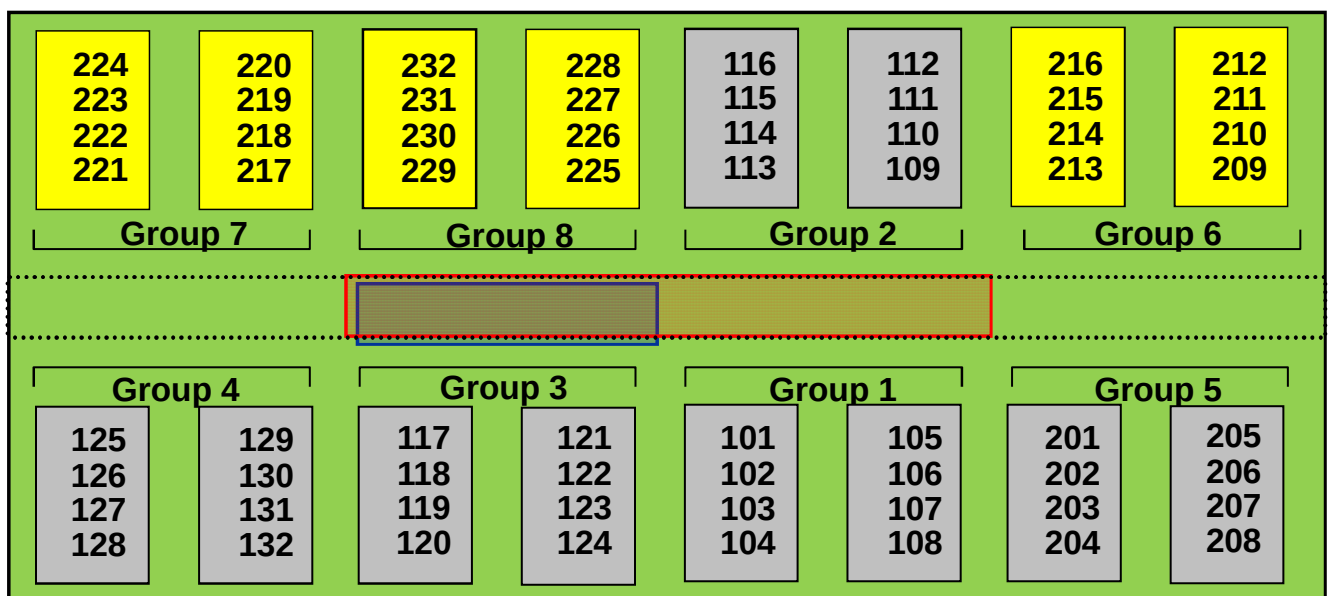
**ADVANTEST**®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 13

System Configuration & Loadboard compatibility



L Testhead

S Testhead

C Testhead*

* Flexible, in this example: C Testhead supports 2 groups among 1,2,3,8

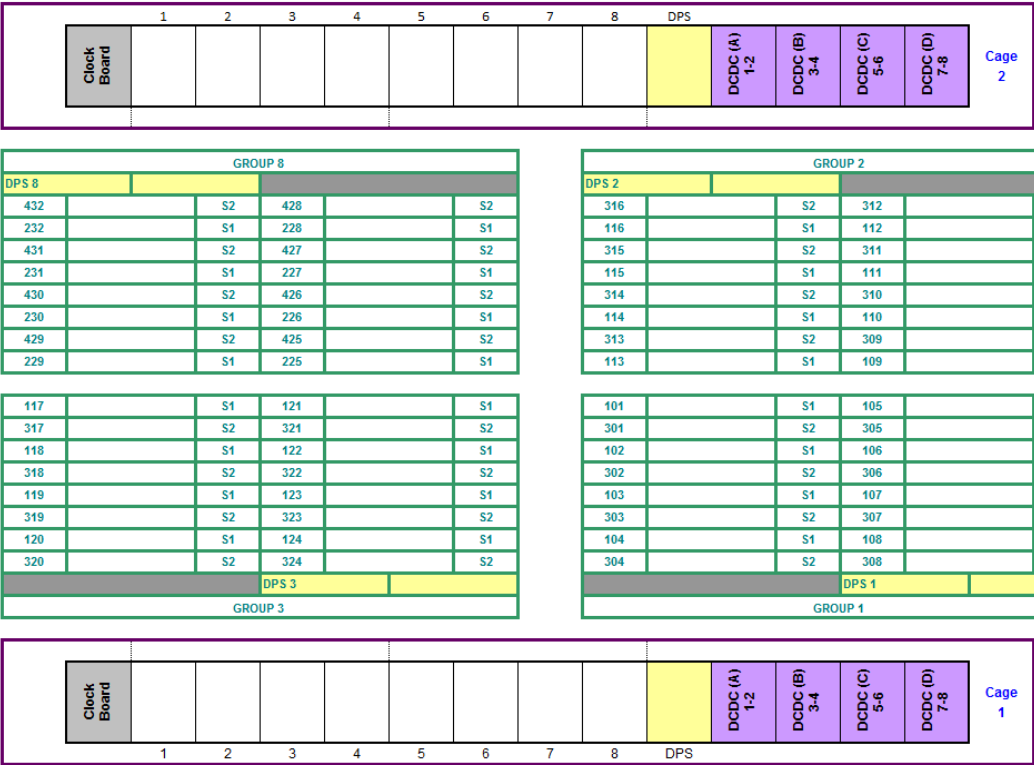
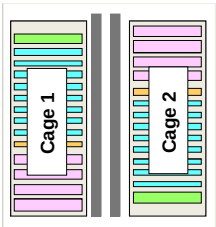
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 14

Test Head structure - Compact Test Head



ADVANTEST®

April 2016

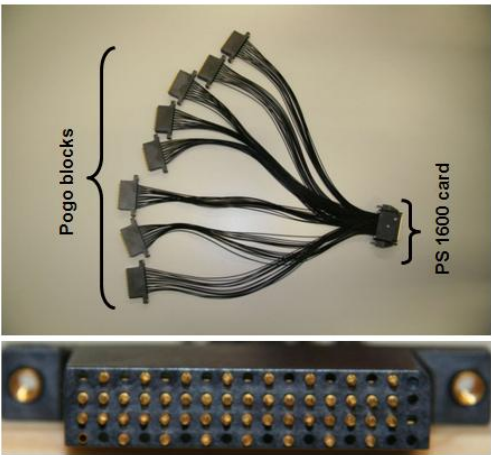
All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 15

Test Head Structure

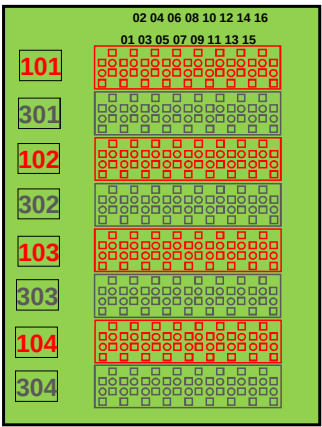
Example: Boards with 8 Pogo Connectors (e.g. PS1600) in Group 1/Card Cage 3

Pogo Cables



Channel Board for
101/102...108

Pogo Blocks on DUT Board



Compatible Pogo Blocks
16 Channels / Block

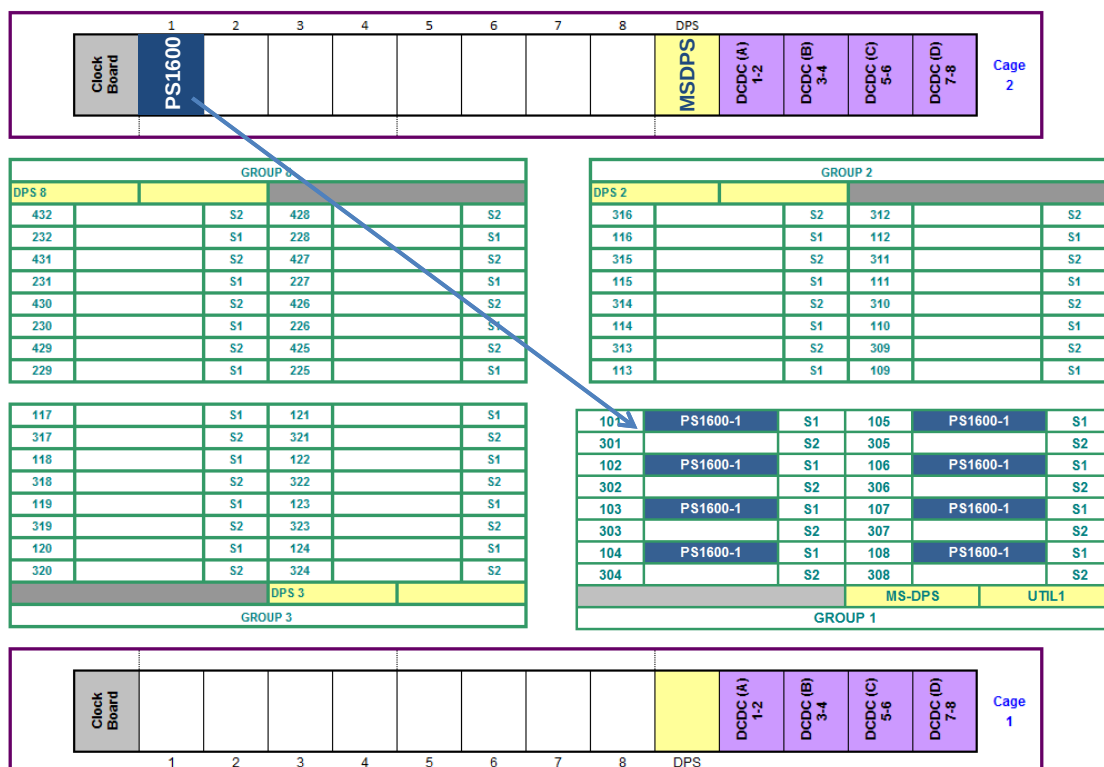
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 16

Test Head structure - Compact Test Head

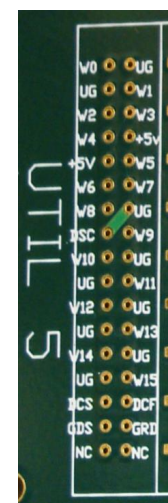
**ADVANTEST®**

April 2016

Hardware Overview Unit 2 - 17

Utility Pogo Block

Utility Read/Write Lines



Utility Power Supply/Ground



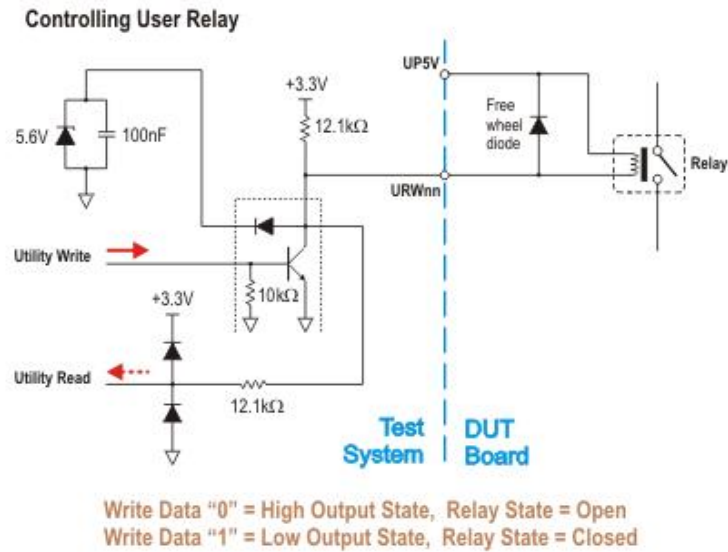
For details please see SmarTest User Documentation page 96536!

ADVANTEST®

April 2016

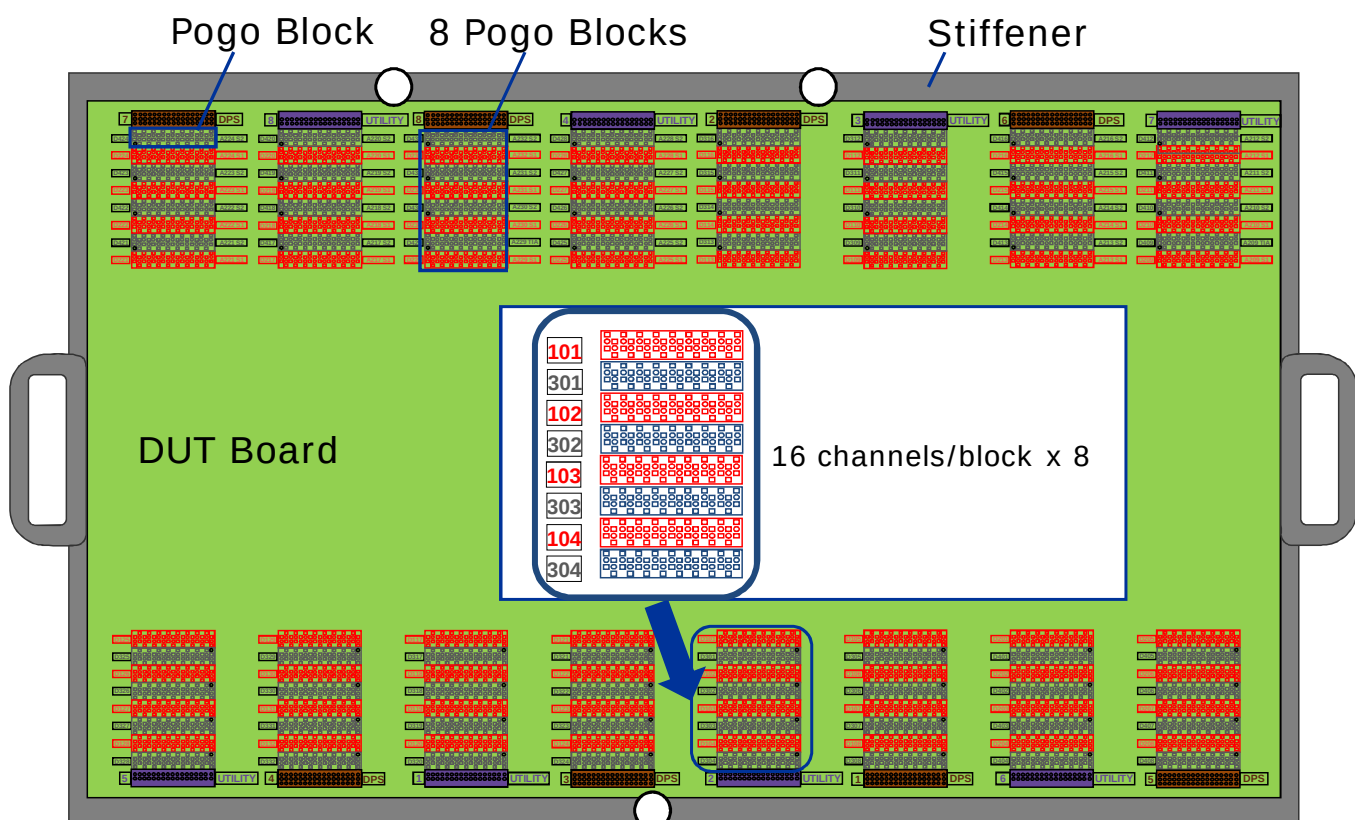
Hardware Overview Unit 2 - 18

Utility Line – Example Circuit



- For sequencer controlled utility lines, see topic 128329

DUT Board



Digital Pogo Block

Channel Numbering

- Pogo pins are assigned to channel resources with a 5-digit number

- Example: Channel **1 2 7 0 5**

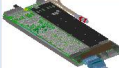
Pogo block on DUT board
[101..132, 201..232, 301..332, 401..432]

Channel number of pogo block
[01..16]



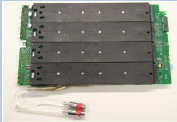
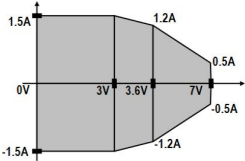
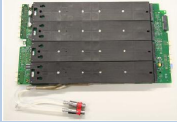
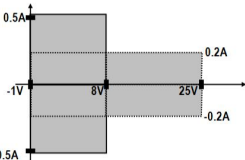
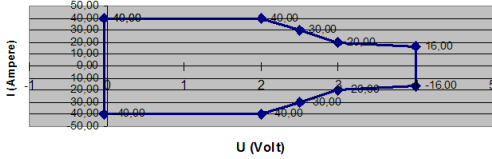
ADVANTEST

Device Power Supply Overview

Card name	Channel/board Channel type	Maximum voltage/board	Maximum current/board	Main features
DPS128 HC 	128 DC+Mixed	-2.5 V .. +7 V	±1.0 A @ 2.5 V	- High Parallelism - Fully pattern controlled - High Precision Differential Voltmeter
DPS128 HV 	128 DC+Mixed	-6 V .. +15 V	±0.2 A	
DPS64 HC	64 DC+Mixed	-2.5 V .. +7 V	±1.0 A @ 2.5 V	Similar specifications as DPS128 HC
AVI64 	64 DC (pulsed, floating) +Mixed +Digital +TMU	-40 ± ... +80 V	± 200mA@10 V ± 50mA@80 V ± 4A pulsed with High Current Unit	- Digital control loop regulation (V/I sampled at 1MHz) - Fully pattern controlled (range, force, measure) - Floating High Current Unit - Floating Differential Voltmeter - High Resolution AWG - Time measurement per channel
PVI8 	8 DC (pulsed, floating) +Mixed +TMU	-40 ± ... +80 V	±1A continuous; ±10A pulsed	- Fully Floating - Pattern controlled operation - Time measurement per channel

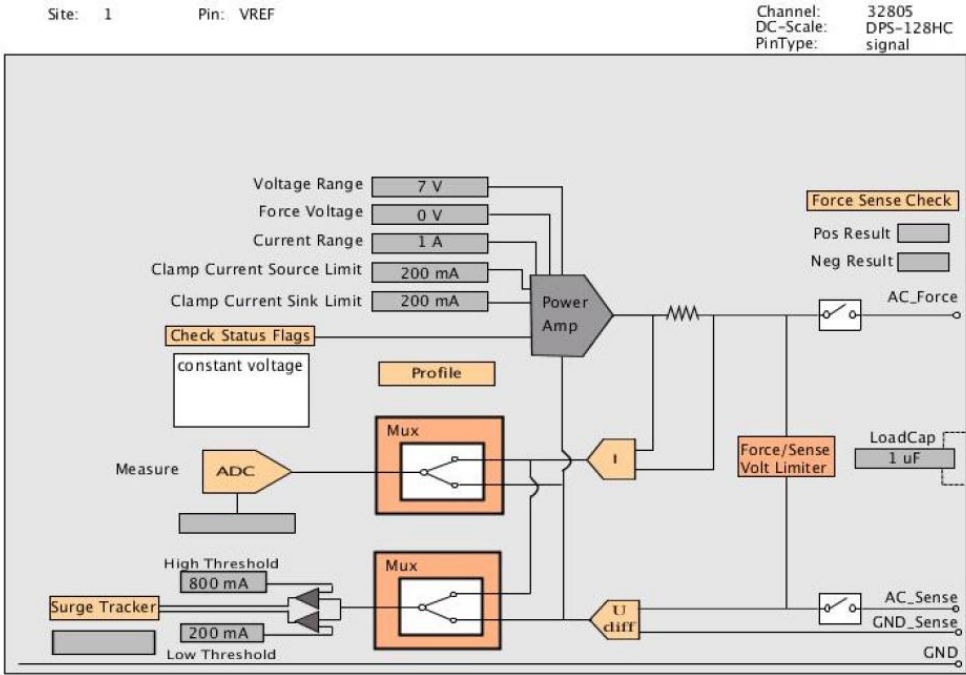
ADVANTEST

Other Power Supply Device

Card name	Channel/ board	Maximum voltage/board	Maximum current/board
DPS32 	32	0 .. + 7V	
VI32 	32	-1 V .. +25V	
UHC4/UHC4T 	4	0 V .. + 4V	40 A @ 2.0 V 
MSDPS	4/8	±8 V	8A/ 4A

ADVANTEST®

DPS128 Hardware Overview



DPS mode

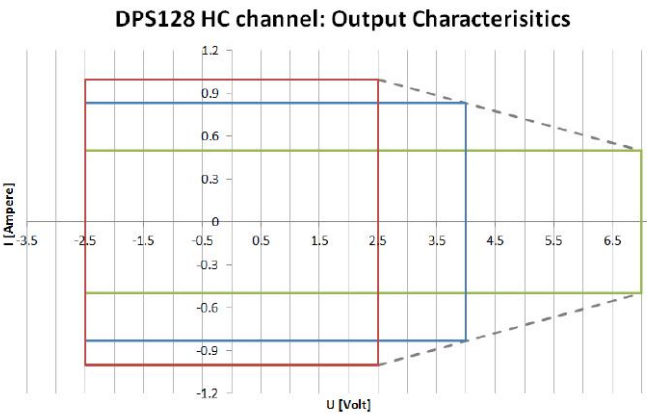
= use like a standard DPS, to supply devices with power during the execution of a testflow

PMU mode

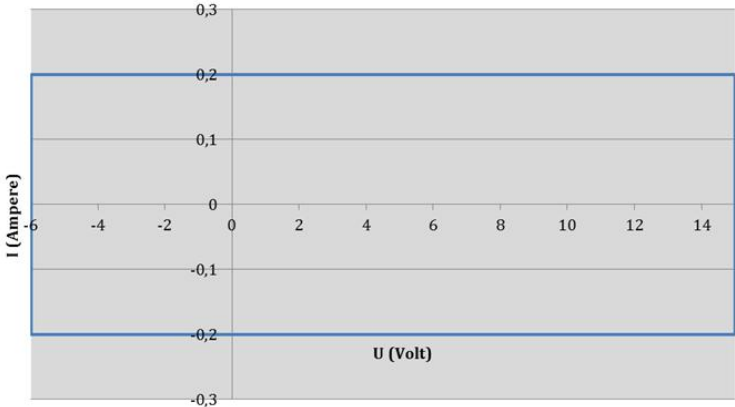
= use like a standard PMU to temporarily supply power for a single test suite or for a group of test suites.

ADVANTEST®

DPS128 Output characteristics



DPS128 HV Output characteristic: I versus U

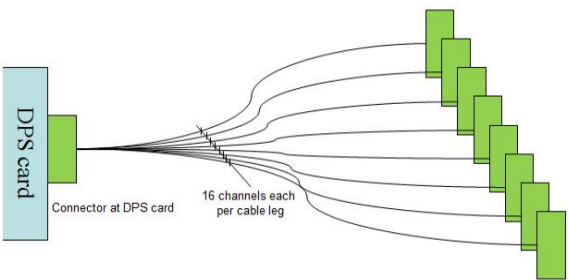


Best Accuracies:

Current measurement: $\pm (50 \text{ nA} + 0.2 \% \text{ of reading})$ for range $\pm 12.5 \text{ }\mu\text{A}$

Voltage measurement: $\pm 2 \text{ mV} / \pm 1 \text{ mV}$

E8023PA pogo assembly: 8 IO-pogo blocks (Standard)



Block A	Block B	Block C	Block D	Block E	Block F	Block G	Block H
A-1	B-1	C-1	D-1	E-1	F-1	G-1	H-1
A-2	B-2	C-2	D-2	E-2	F-2	G-2	H-2
A-3	B-3	C-3	D-3	E-3	F-3	G-3	H-3
A-4	B-4	C-4	D-4	E-4	F-4	G-4	H-4
A-5	B-5	C-5	D-5	E-5	F-5	G-5	H-5
A-6	B-6	C-6	D-6	E-6	F-6	G-6	H-6
A-7	B-7	C-7	D-7	E-7	F-7	G-7	H-7
A-8	B-8	C-8	D-8	E-8	F-8	G-8	H-8
A-9	B-9	C-9	D-9	E-9	F-9	G-9	H-9
A-10	B-10	C-10	D-10	E-10	F-10	G-10	H-10
A-11	B-11	C-11	D-11	E-11	F-11	G-11	H-11
A-12	B-12	C-12	D-12	E-12	F-12	G-12	H-12
A-13	B-13	C-13	D-13	E-13	F-13	G-13	H-13
A-14	B-14	C-14	D-14	E-14	F-14	G-14	H-14
A-15	B-15	C-15	D-15	E-15	F-15	G-15	H-15
A-16	B-16	C-16	D-16	E-16	F-16	G-16	H-16

Assignment				
Pin	a	b	c	d
X-1	GS2	P1	GND	PS1
X-2	PS2	GND	P2	GS1
X-3	GS4	P3	GND	PS3
X-4	PS4	GND	P4	GS3
X-5	GS6	P5	GND	PS5
X-6	PS6	GND	P6	GS5
X-7	GS8	P7	GND	PS7
X-8	PS8	GND	P8	GS7
X-9	GS10	P9	GND	PS9
X-10	PS10	GND	P10	GS9
X-11	GS12	P11	GND	PS11
X-12	PS12	GND	P12	GS11
X-13	GS14	P13	GND	PS13
X-14	PS14	GND	P14	GS13
X-15	GS16	P15	GND	PS15
X-16	PS16	GND	P16	GS15

X-n: Pin number per Block X (A to H)
Px: Power x
PSx: Power Sense x
GND: Ground (all grounds are connected at the relays board)
GSx: Ground Sense (separate line for each channel)

Digital Channels

V93000 Smart Scale Cards

Pin Scale 9G

64 high speed channels per card

>8.0Gbps data rate

Pattern and/or loopback



Pin Scale 1600

128 digital channels per card

1600Mbps data rate

High accuracy DC, HV PE

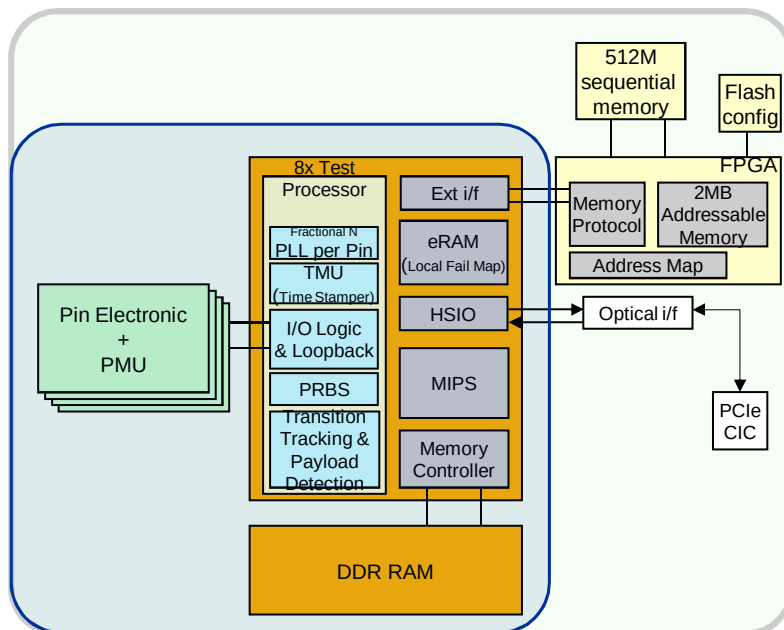


Pin Scale 1600 ME

64 digital channels per card

1600Mbps data rate

Memory Emulation HW



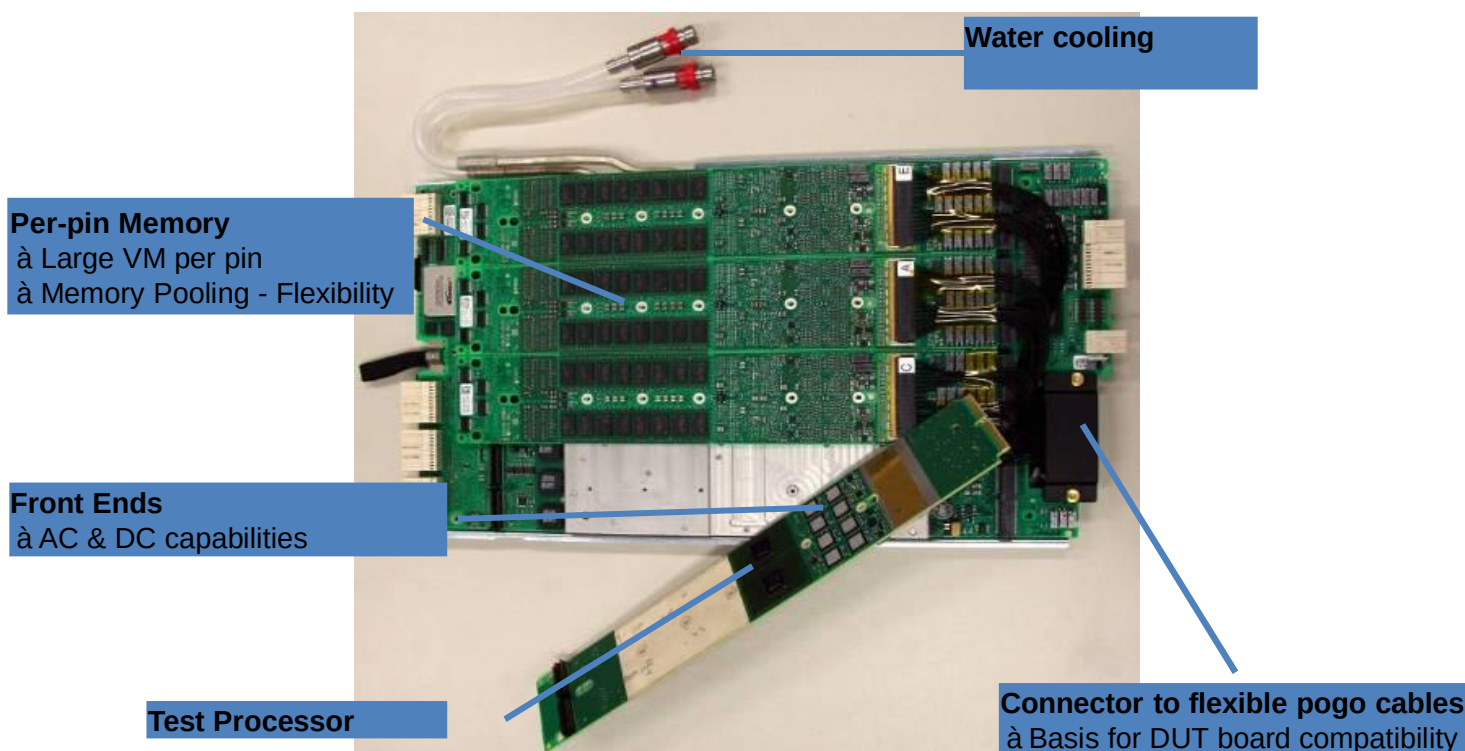
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 27

PS1600 Card



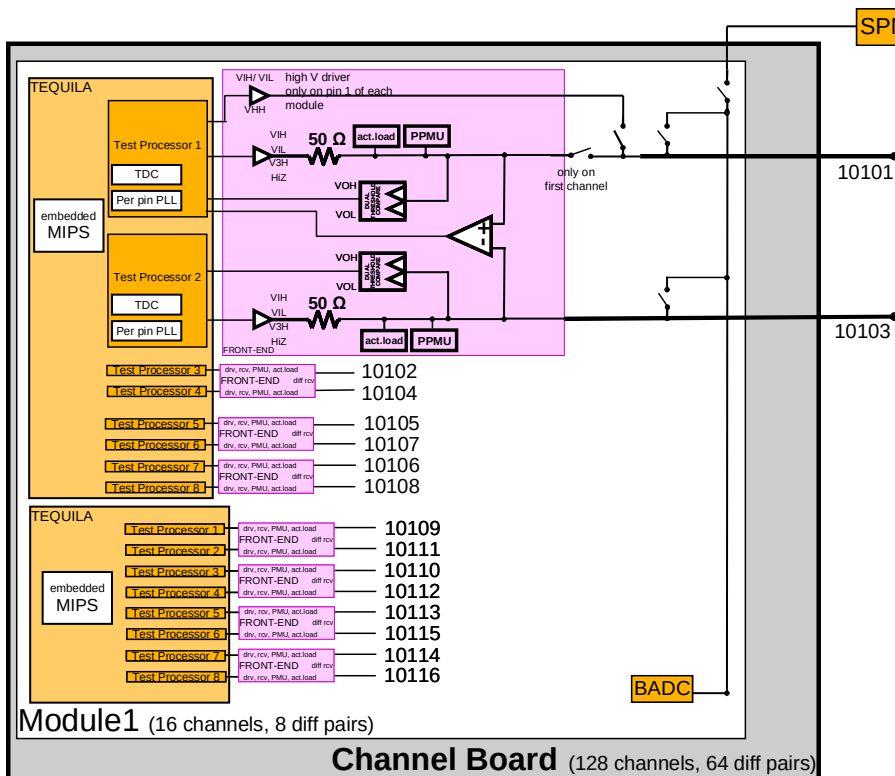
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 28

Pin Scale 1600



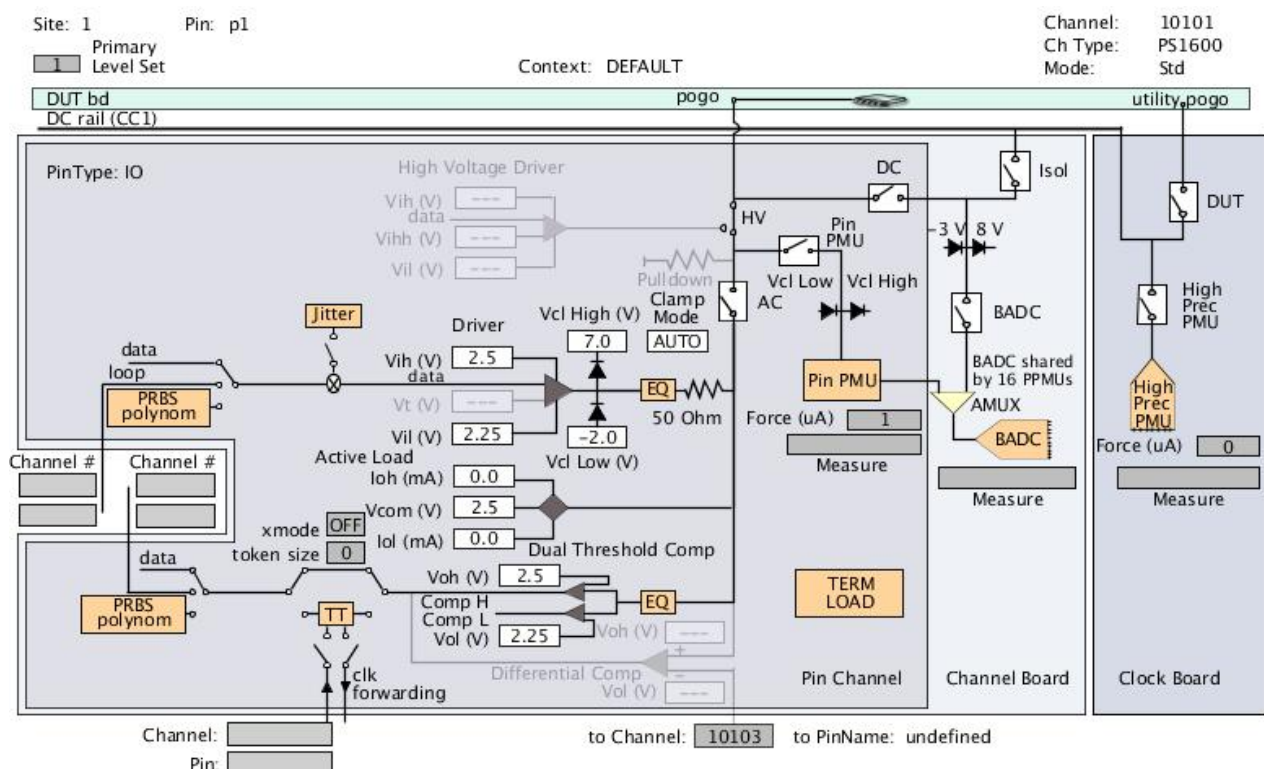
PS1600:

- 128 digital pins / card
- Clock Domain per Pin
- Single ended & differential signals
- 0..1600Mbps data rate
- OTA/EPA = $\pm 200/100$ ps(system), $\pm 160/80$ ps(board)
- 112MB/pin, 448MVec/pin (x4 mode)
- Driver: -1.5 V to 6.5 V
- High Voltage Driver (6V to 13.5V) per 16 pins
- Active Load with ≤ 25 mA
- Pin PMU
 - - 2.0 V to 6.50 V
 - ± 40 mA, ± 65 mA (with active load)
 - Sequencer controlled
- Board ADC per 16 pins
 - -3.0 V to 8.0 V
 - High impedance, accurate V-measure
 - Sequencer controlled
- Time Measurement Unit per Pin
- SmartLoop™ per Pin
- PRBS per Pin
- Protocol Engine per Pin – Tracking & Payload
- detection
- AWG and DGT capability

Full coverage of structural and functional test up to 1.6Gbps

ADVANTEST

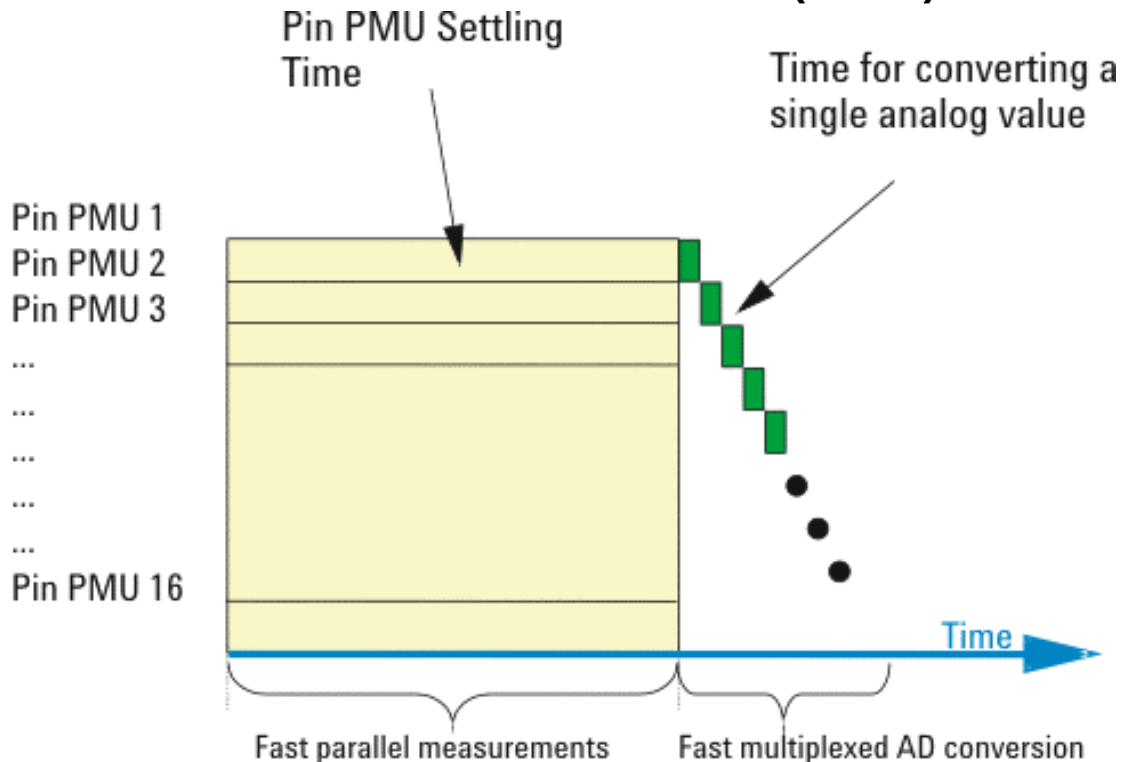
PS1600 Channel Module Overview



ADVANTEST

Digital Channels

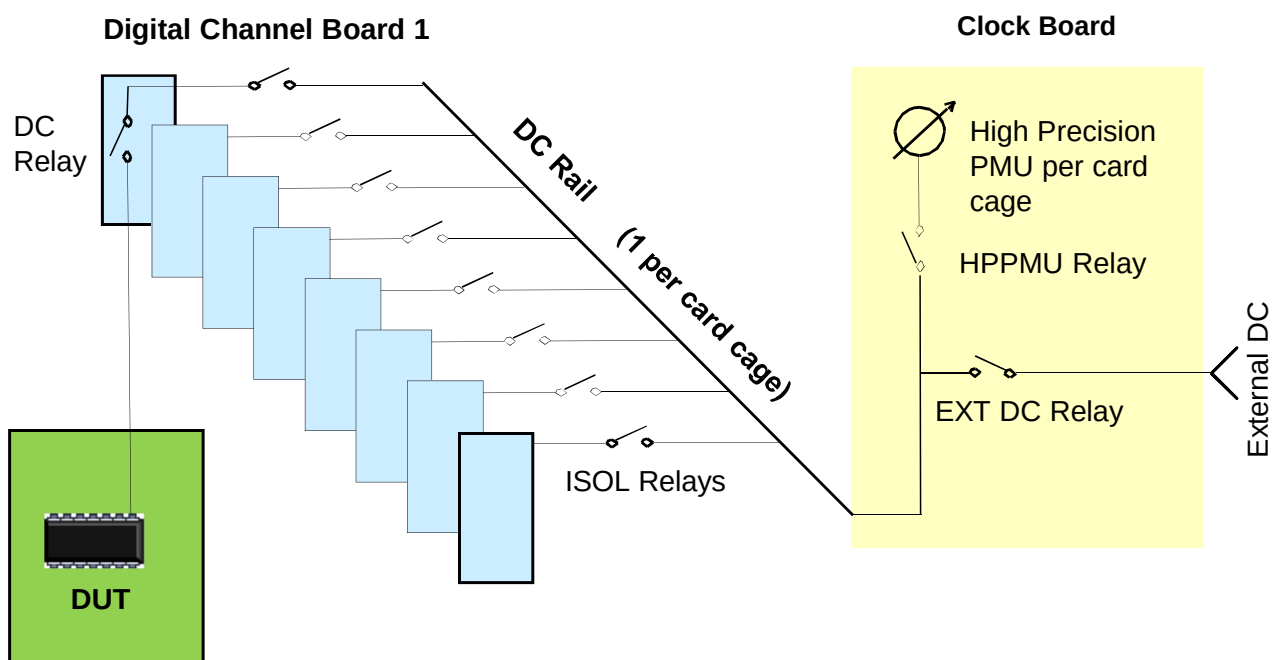
Pin Value Measurement with Board ADC (BADC)



ADVANTEST

Card Cage Resource

High Precision Parametric Measurement Unit (HPPMU)



ADVANTEST

Digital Channels

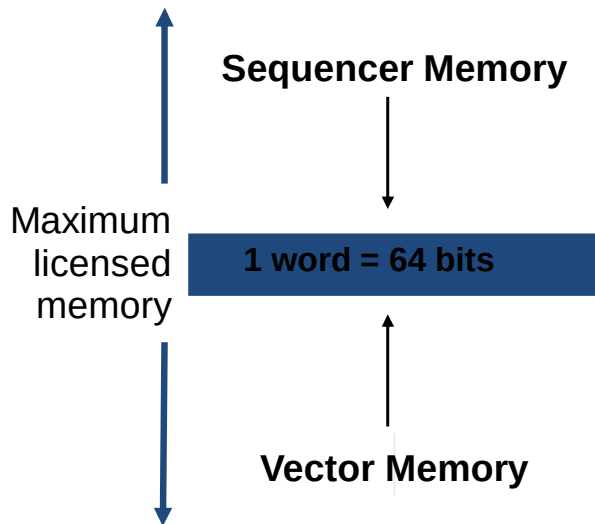
Per Pin Vector Memory (Unified Memory Mode)

Sequencer and Vectors share the same physical memory

8 bits/vector, 8 vectors/word

1 sequencer-instruction/word

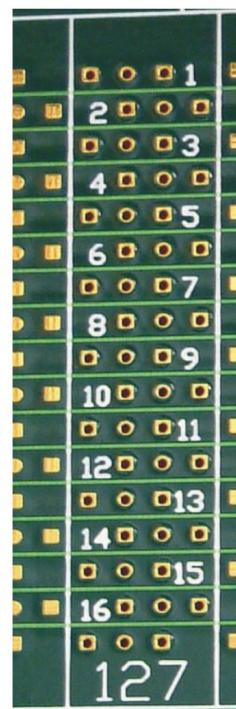
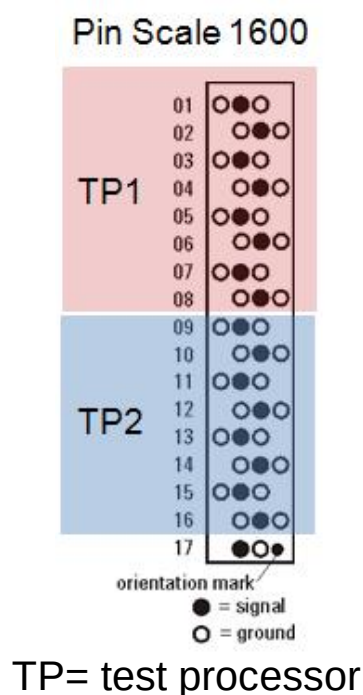
Memory license steps



Parallel vector memory on **PS1600**
16 MByte standard,
32 MByte optional,
64 MByte optional,
112 MByte optional

ADVANTEST

PS1600 – pogo mapping

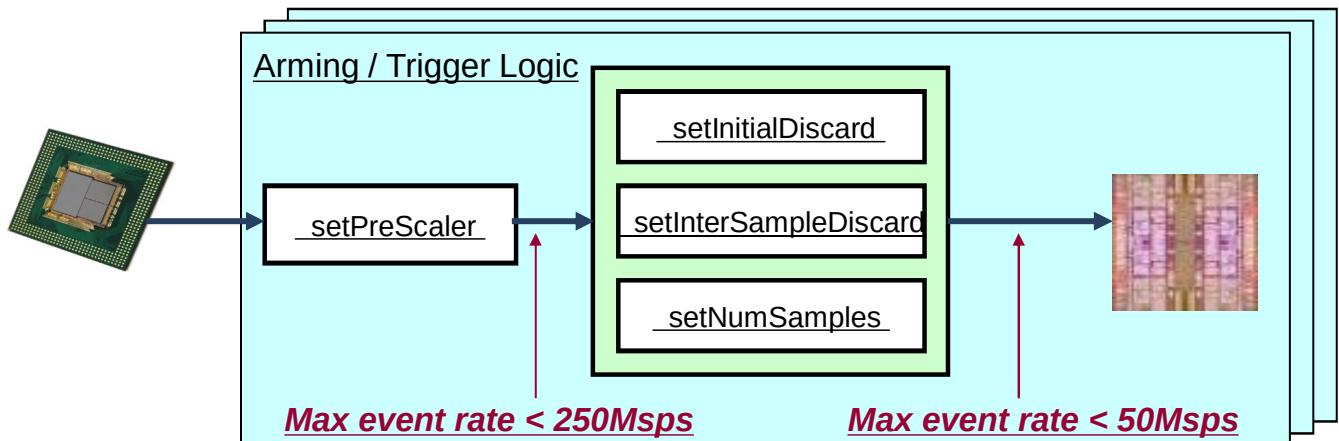


Pogo block example

ADVANTEST

TMU per Pin - Concepts

PS1600
PS9G



TMU specifics:

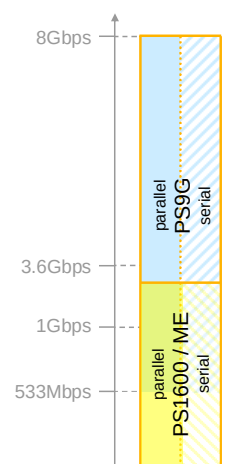
The incoming data rate has to be scaled to meet the limitations at the input stages. The user has the possibility to:

- Specify the incoming data rate and have the SW calculate the Pre-Scaler and Discard stages.
- Specify the Pre-Scaler and Discard stage manually

ADVANTEST

Optional - Pin Scale 9G - Overview

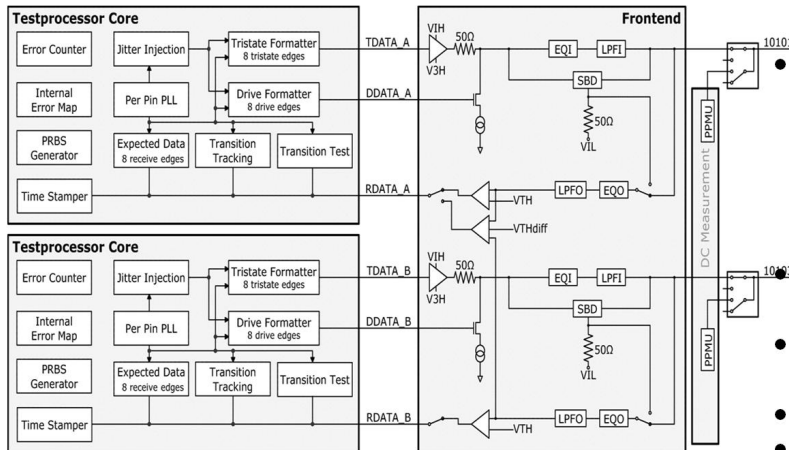
- Pin Scale 9G is the Smart Scale generation high speed digital card
- Unique concept of two instrument types in one card
 - Parallel test capabilities up to 6.8Gbps
 - Serial PHY test capabilities up to 8Gbps
- Pin Scale 1600 and Pin Scale 9G build a two-card lineup
- Test Program compatibility to existing Pin Scale digital cards
- Pin Scale 9G makes @speed testing affordable
 - Most flexible high-speed instrument in the industry
 - Pin Scale 9G addresses emerging test requirements such as
 - High speed parallel & serial I/F → Datarate up to 6.8 / 8Gbps
 - Non symmetrical I/F types (e.g. DP) → I/O mode flexibility
 - IPs with increasing number of cores → Clock Domain per Pin
 - System like stress test → Protocol Engine per Pin, SmartLoop™
 - Timing measurement requirements → TMU per Pin



ADVANTEST

Optional - Pin Scale 9G

- High speed parallel and serial interfaces
- Next generation computer, CE and memory interfaces
- Design verification, characterization and High Volume Manufacturing



Key Characteristics

- 64 ch. SE / 32 ch. diff ch. per card
- Uni- and Bi-directional SE & differential
- Data rate up to 8 Gbps
- EPA target: < 28ps
- 1ps edge resolution
- 112 Mbyte
- à 448 MVector per pin (x4 mode)
- System-Like Stress Test™
- à SmartLoop™
- à PRBS engine per Pin
- à Protocol Engine per Pin
- à Transitions Tracking
- à Payload Detection
- Timing measurement with TMU per Pin
- Clock Domain per Pin, any pin can run at any data rate
- Simultaneous Bi-Directional
- Flexible licensing options

ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 37

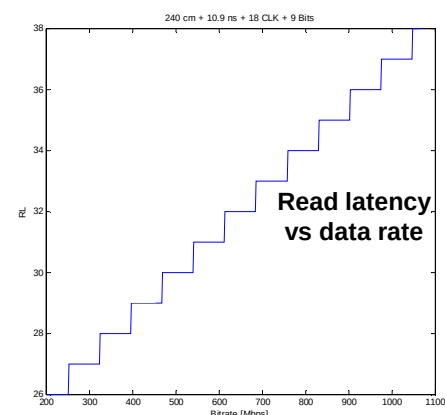
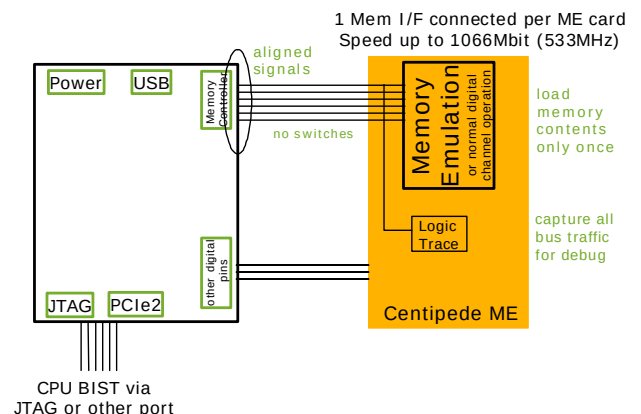
Optional - Pin Scale 1600-ME Memory Emulation

Objective:

- Enable DUT to perform R/W operations to an “emulated” external memory

Key Specifications:

- **64 channels per card**
 - Same capabilities as Pin Scale 1600
- Memory emulation port is completely independent of other ATE channels (clock, sequence, ...)
- Channels not used for ME, operate as a standard Pin Scale 1600
- Up to 1066 Mbit/s in ME mode (DDR)
- Supports LPDDR, DDR2, DDR3, Flash
- 2 MByte size of addressable emulated memory
- 500 MByte of sequentially accessed memory (for booting, memory compare, etc.)
- Assumes DFT in memory controller to tolerate CAS latency of up to 40 clocks
- Programmable timing and levels
- Logic Analyzer trace can capture all activity during emulation



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 38

Analog Modules

AWG

– MCB/MCE: Multi-site BB/VHF Analog Source & Measure (MB-AV8/MB-AV8+/AV8+)

• LF AWG	1.024 Msps	24-bit
• HF AWG	200 Msps	16-bit
• VHF AWG	500 Msps	16-bit

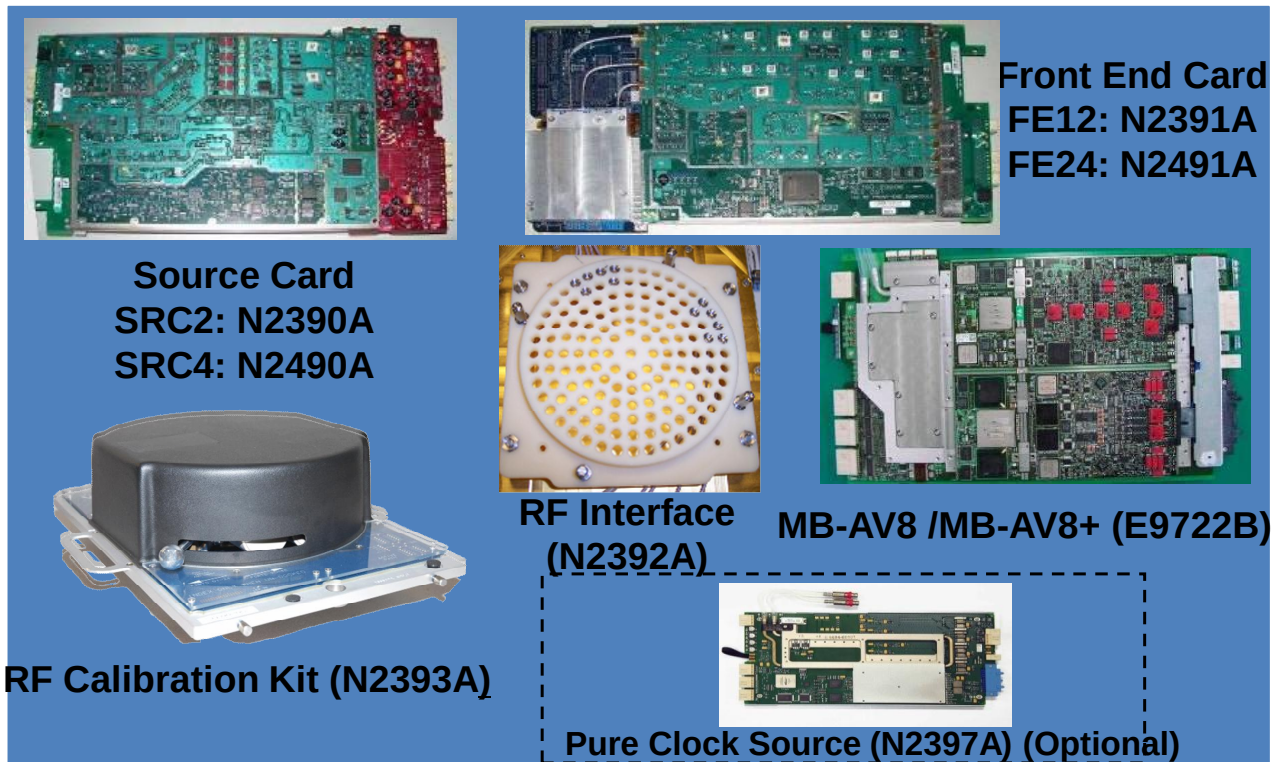
Analog Modules

Digitizer

– MCB/MCE:Multi-site BB/VHF Analog Source & Measure or
Multi-site BB/VHF Analog Measure (MBAV8/MBAV8+/AV8+)

• LF Digitizer	200 ksps	50 kHz BW	24-bit
• VHF Digitizer	110 Msps	100 MHz BW	16-bit

What Makes up the Port Scale RF Subsystem?



Plus a complete offering of load board & probe products

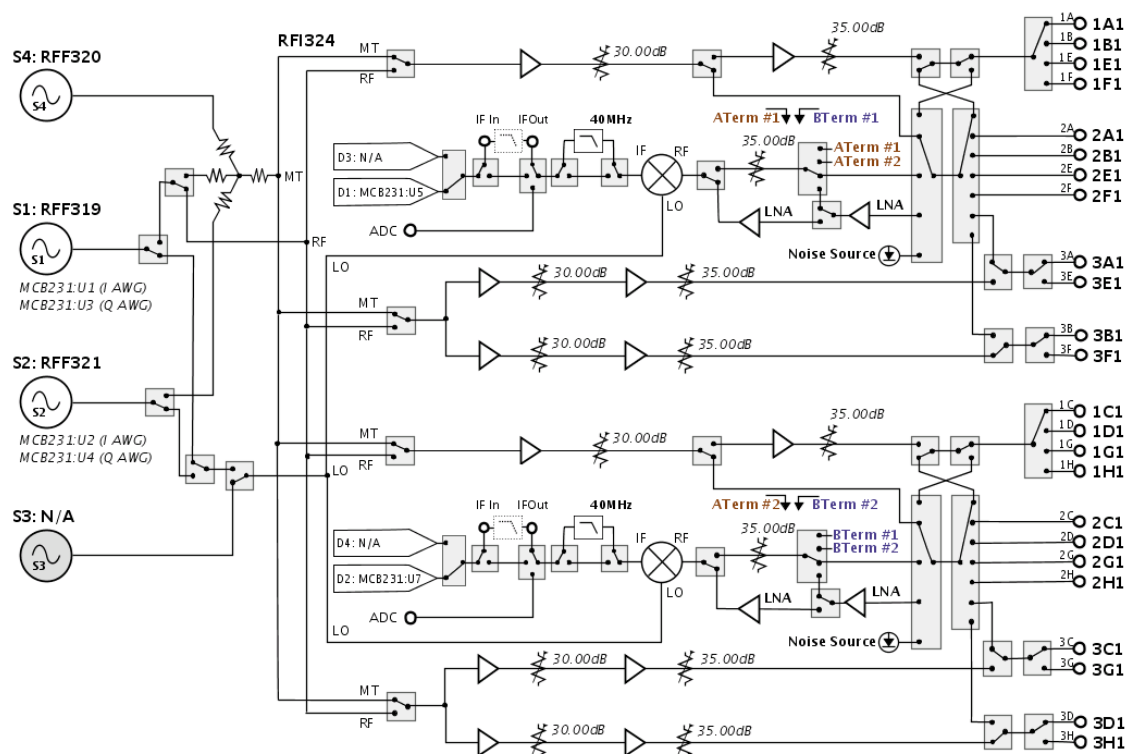
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 41

FE24: Block Diagram



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Hardware Overview Unit 2 - 42

References

- V93000 Technical Documentation Center (TDC):
<https://www.advantest.com/V93000/help/index.jsp>
- System Reference
- Specifications and Configuration Guidelines ([Topic 105099](#))

Advantest V93000

Digital User Training

Software Overview

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- SmarTest Overview & Structure
- Eclipse Workcenter
- Perspectives, Editors & Views
- SmarTest Device Directory
- .technology File
- Creating Device
- Model, Device License Files
- Technical Documentation Center (TDC)

ADVANTEST[®]

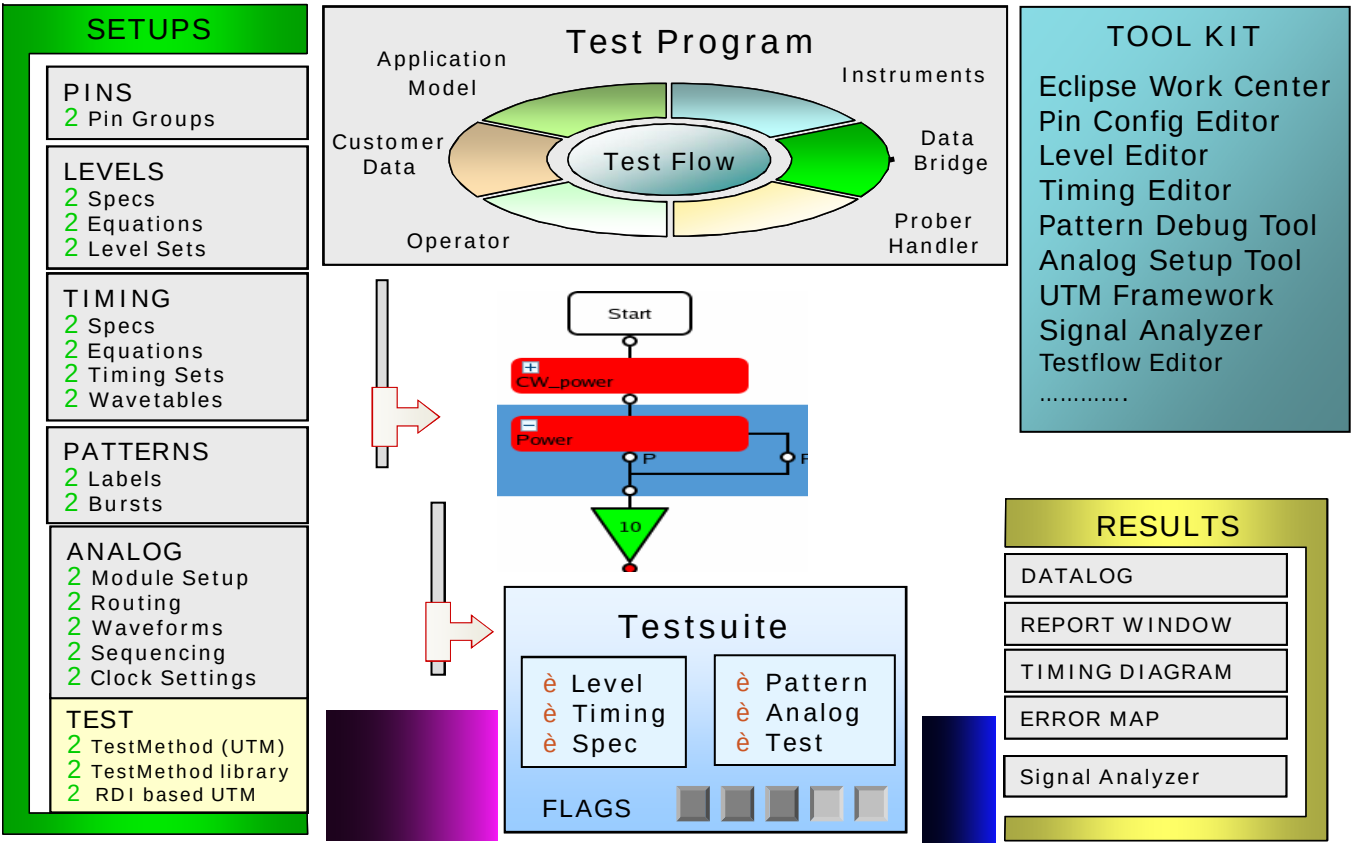
Learning Objectives

- Understand the SmarTest software concept
- Understand the SmarTest file structure
- Understand the purpose of the different tools and how they are linked together

Relevance

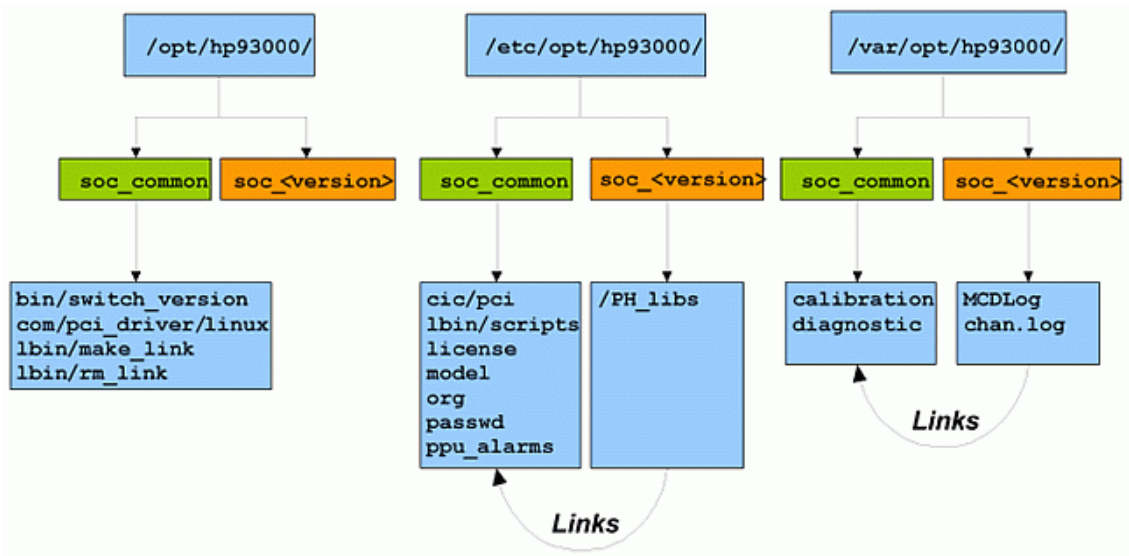
A basic understanding of the software helps to keep a high-level view when developing a test program

SmarTest Overview



ADVANTEST

SmarTest Path and File Structure

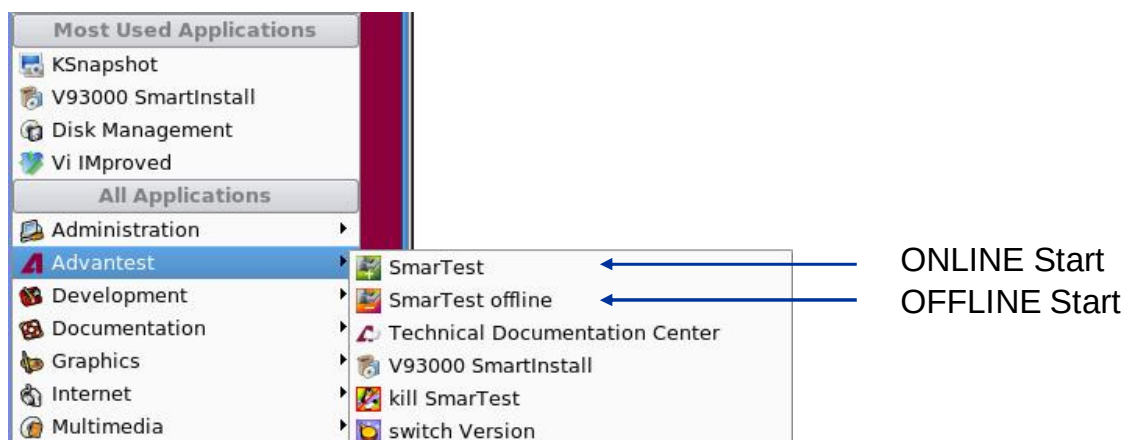


ADVANTEST

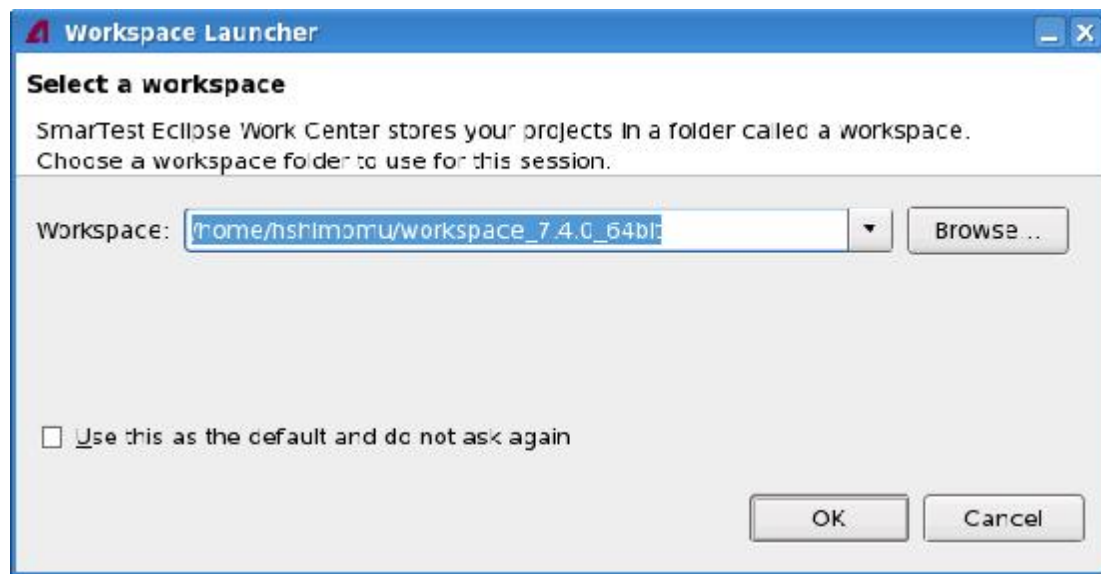
SmarTest 7.4.x features

- MUX cards definition in the model file
- Fast Sync on Smart Scale cards
- SmartRDI 2.4.0 enhancements
- DC Scale AVI64 card

Launching Smartest (ONLINE/OFFLINE)



Eclipse Workspace Selection at Startup



ADVANTEST

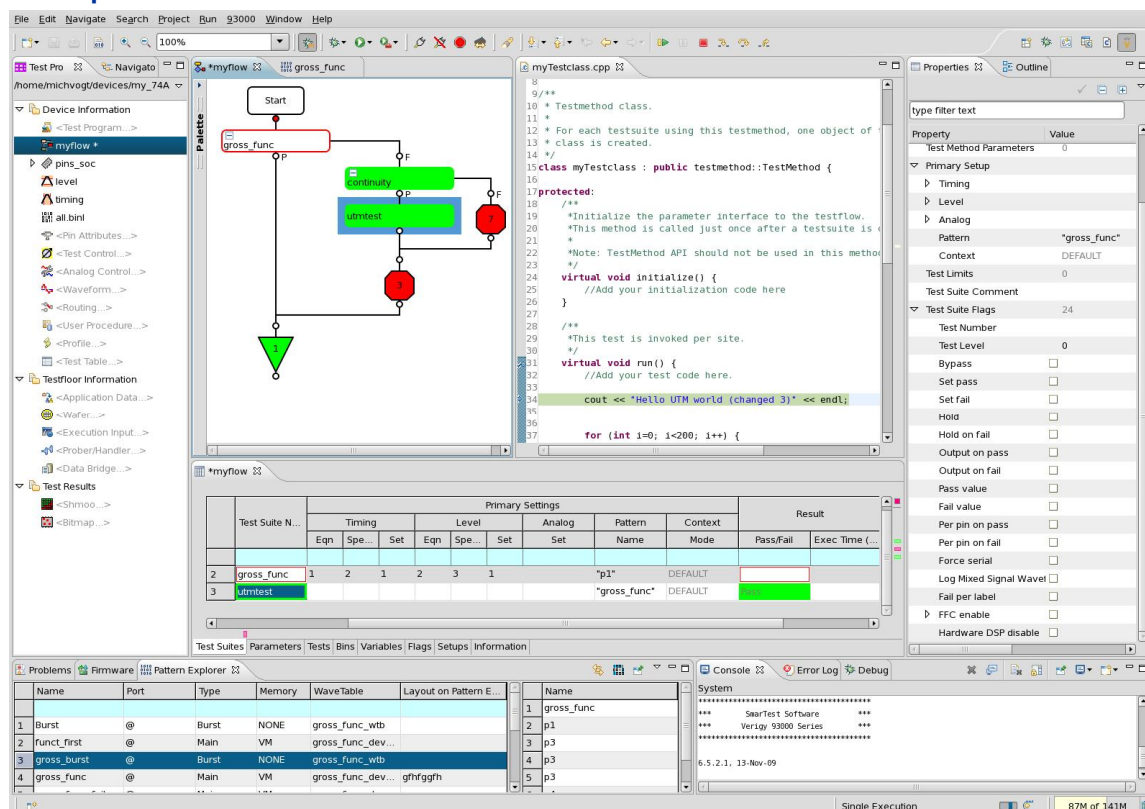
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 9

Integrated Environment

Eclipse Workcenter



ADVANTEST

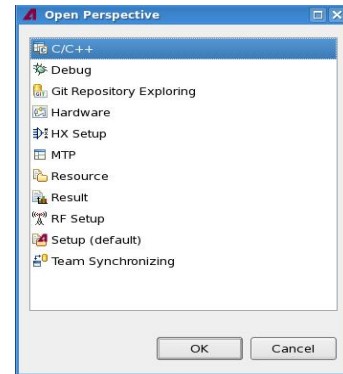
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 10

Eclipse Perspectives

- Perspectives are sets of window panes in your Workcenter
- Perspectives are specific arrangements of editors and views
- Predefined Perspectives:

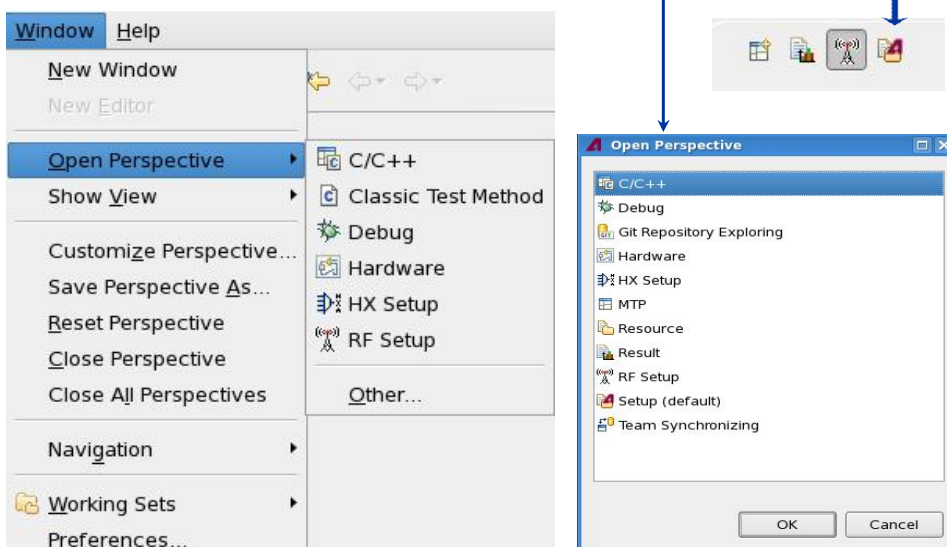


- You can change or create new perspectives which fulfill your specific needs
- Window -> Reset Perspective (restore default configuration)

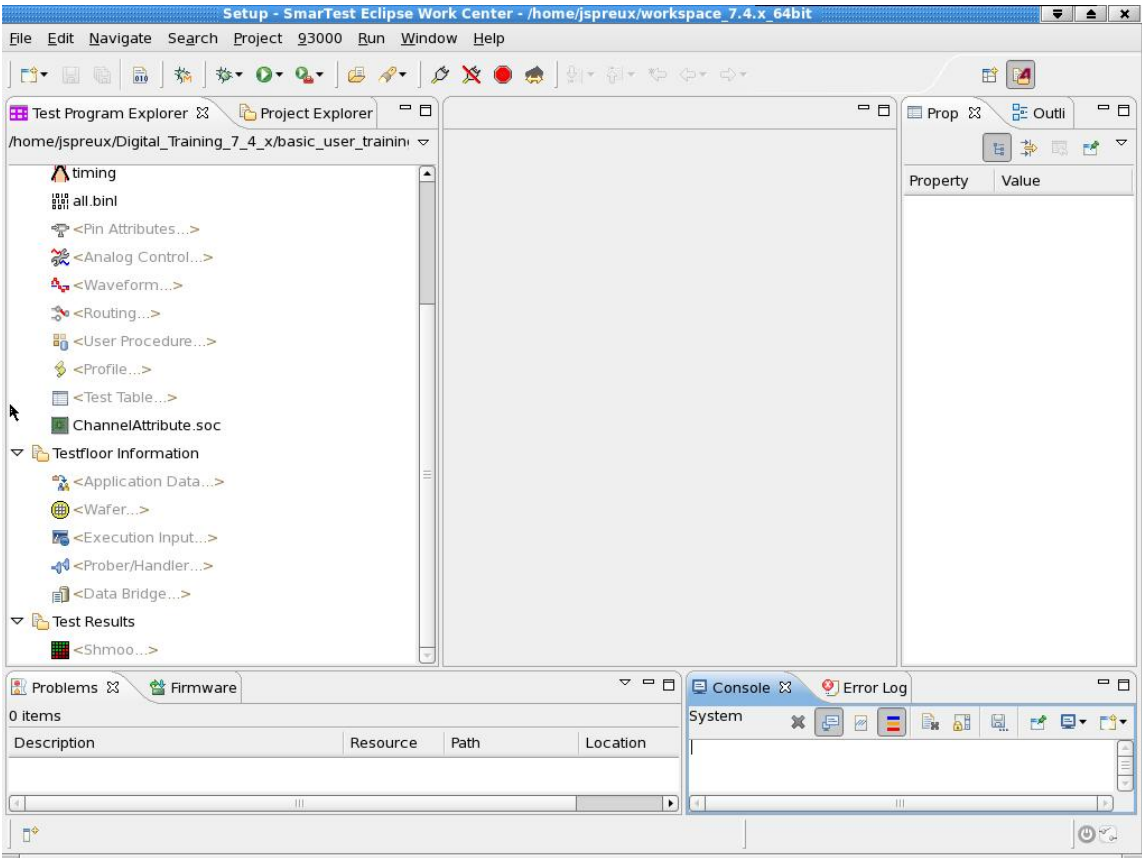
Eclipse Perspectives

Accessing perspectives

- Window => Open Perspective or via icons



Eclipse Perspectives



ADVANTEST

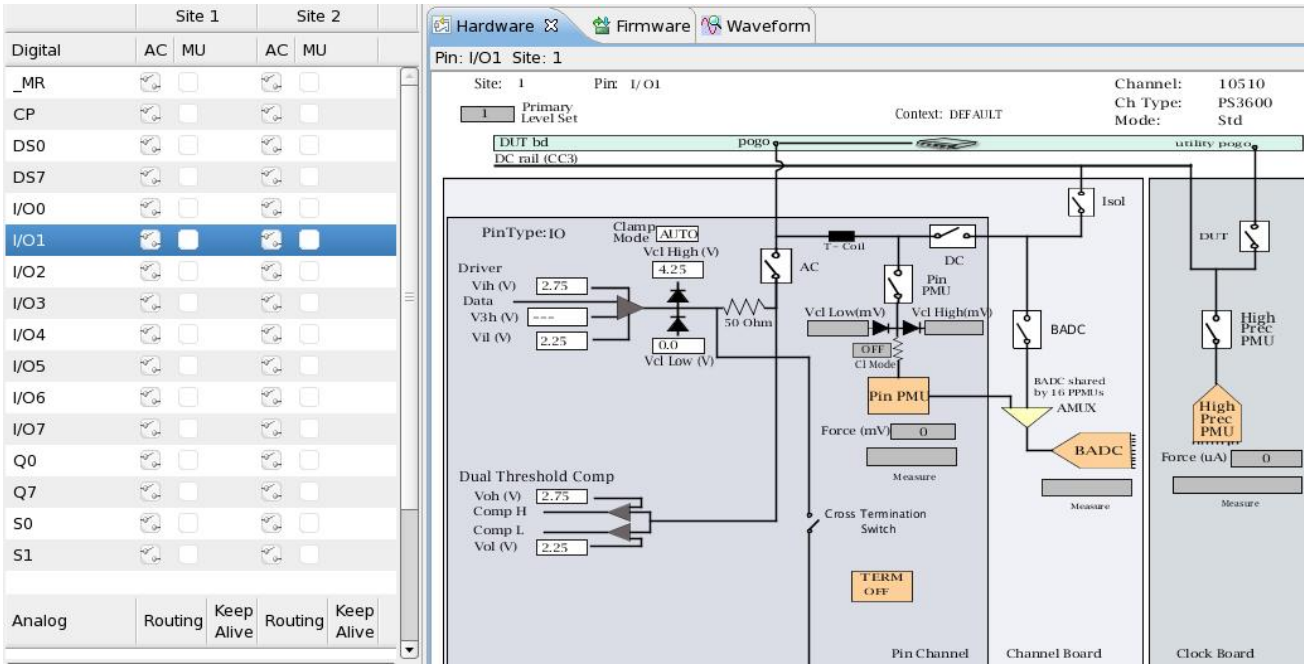
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 13

Eclipse Perspectives

Hardware Perspective / Monitoring Hardware Status



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

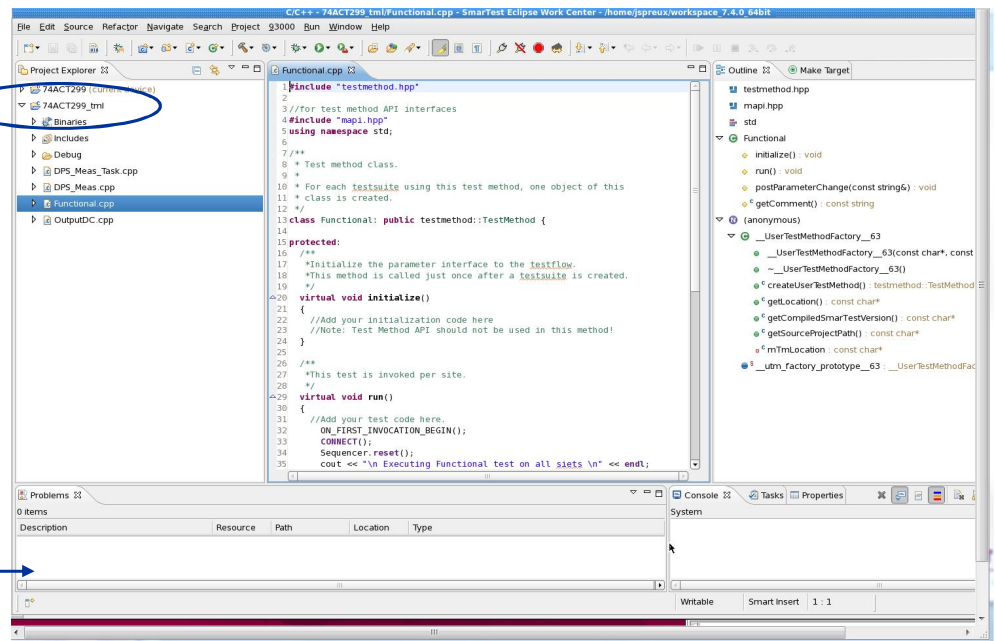
Software Overview Unit 3 - 14

Eclipse Perspectives

C/C++ Perspective

Click on the UTM project in the explorer frame and now you see the C-build console for the **UTM** project

Compilation errors and warnings will be shown here



ADVANTEST

April 2016

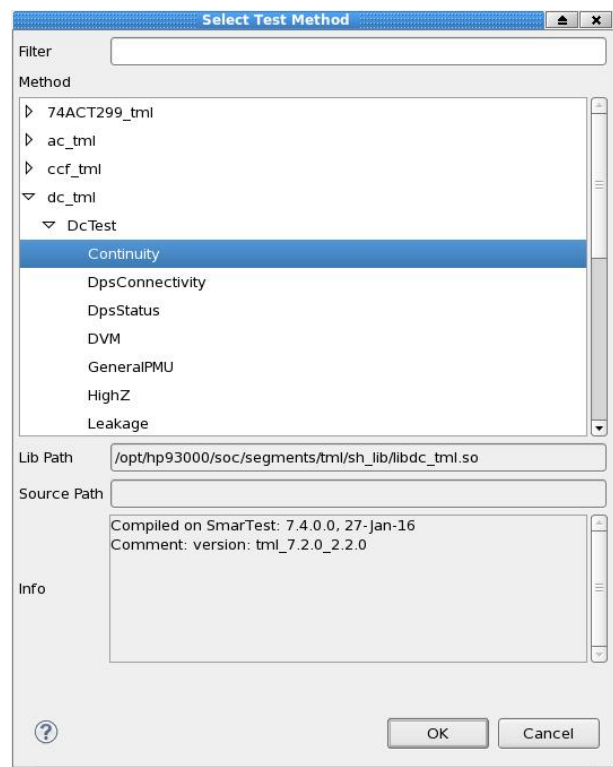
All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 15

C++ Test Method Templates

Universal Test Method (UTM) Libraries

- dc_tml
- ac_tml
- ccf_tml
- commandRunner
- multi_group_dc_tml
- miscellaneous_tml
- MTP_ExecutionEngine_tml
- pptia_tml
- scan_tml
- smartCalc
- tmu_tml



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 16

UTM based on SmartRDI

SmartRDI

In This Chapter



- [Installing and using SmartRDI](#)
- [RDI_Configure file](#)
- [Code structure with SmartRDI](#)
- [RDI_INFO_LEVEL](#)
- [Supported SmartTest Versions](#)
- [General SmartRDI functions](#)
- [Generating vector with SmartRDI](#)
- [Changing the level/timing set with SmartRDI](#)
- [Controlling the utility lines with SmartRDI](#)
- [Controlling relays in the digital channel with SmartRDI](#)
- [Setting up functional tests with SmartRDI](#)
- [Setting up digital capture tests with SmartRDI](#)
- [Setting up DC tests with SmartRDI](#)
- [Using a DC resources as an AWG/DGT with SmartRDI](#)
- [Setting up analog tests with SmartRDI](#)
- [Setting up RF tests with SmartRDI](#)
- [Setting up Power MUX with SmartRDI](#)
- [Setting up TMU tests with SmartRDI](#)
- [Pattern-based conditional jump](#)
- [Getting error map information from user patterns](#)
- [Setting up d2s-like protocol tests with SmartRDI](#)
- [Tool functions from SmartRDI](#)

ADVANTEST®

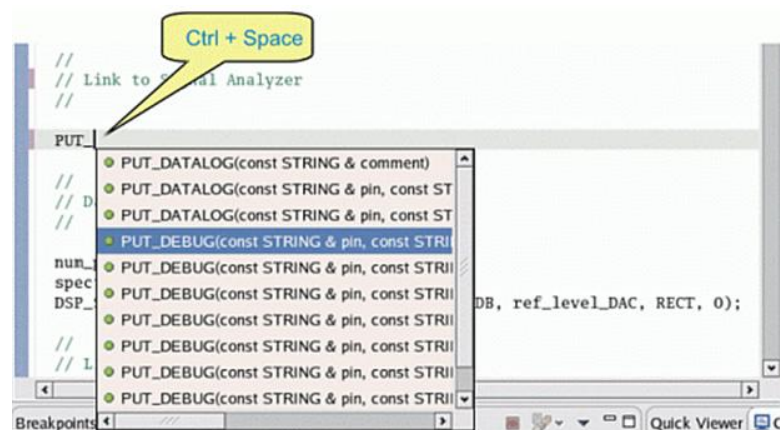
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 17

Editing via Content Assist

You can the Content Assist to enter the API statements into the Editor view or to Select entries in other context like Pattern or test setup data.

**ADVANTEST**®

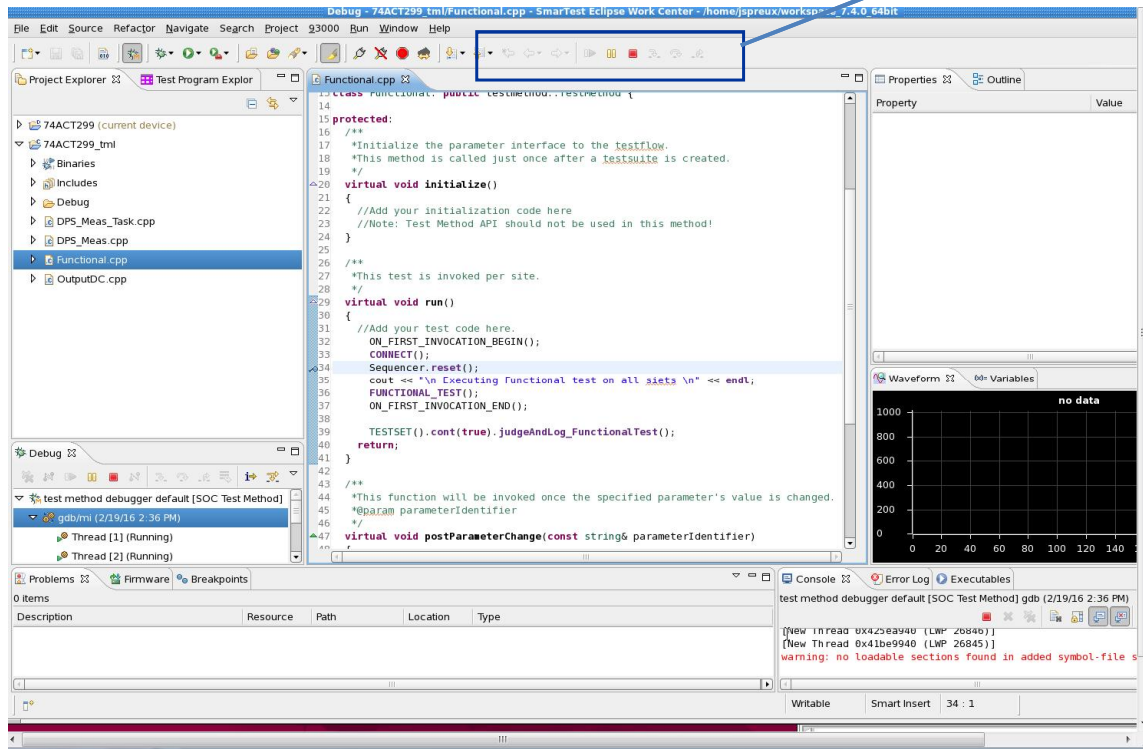
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 18

Eclipse Perspectives

Debug Perspective: Running a Debug Session



ADVANTEST

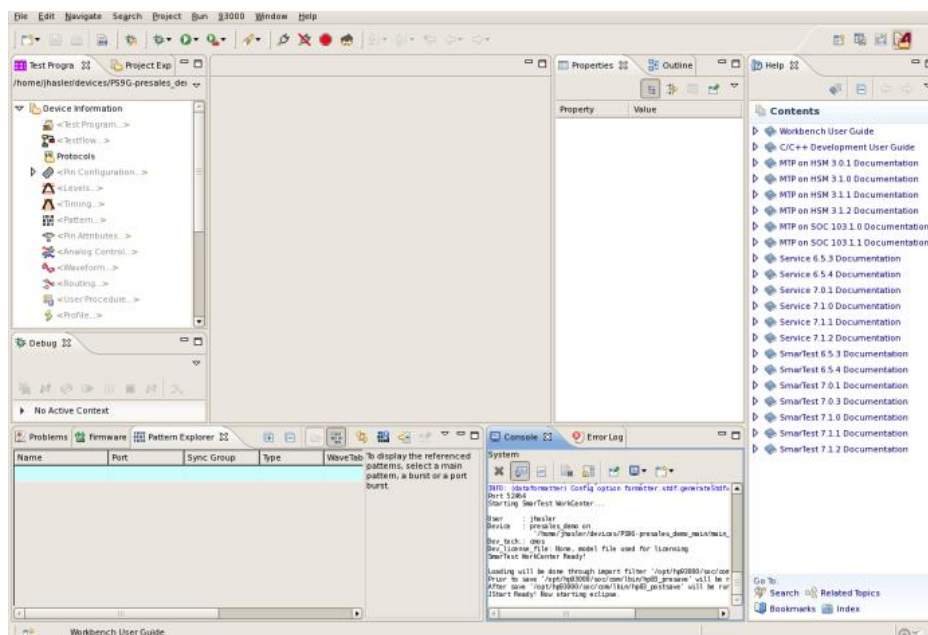
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 19

Eclipse Perspectives

Views and Perspectives



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

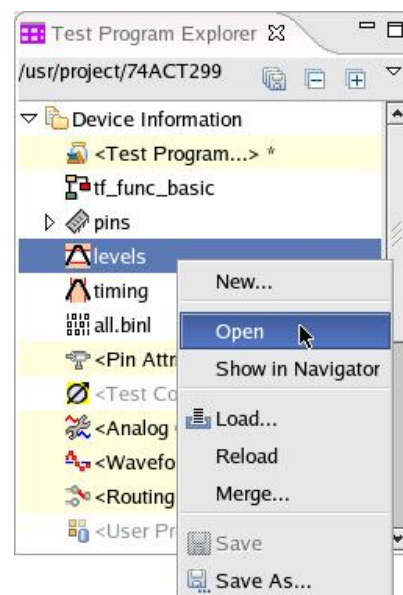
Software Overview Unit 3 - 20

Navigation

Test Program Explorer

Use the Test Program Explorer to

- Load, edit (launch editors), and save setup data
- Show logical content of test programs
- Visualize current context of execution



ADVANTEST

Integrated Testflow Tools

Flow Sequence Editor and Flow Data Editor

Flow Sequence Editor to Debug

Flow Data Editor to Edit

The screenshot displays the 'Test Program Explorer' on the left, the 'Flow Sequence Editor' in the center, and the 'Flow Data Editor' on the right. The Flow Sequence Editor shows a flowchart starting with 'Start', followed by 'CW_power' and 'Power' blocks, leading to a green triangle labeled '10'. The Flow Data Editor shows a table with test suite information.

Test Suite N...	Test Setup Name	Timing			Primary Se		
		Eqn	Spe...	Set	Eqn	Spe...	Set
1	CW_power	PSRF_Labs_project.LB_Labs	1	1	1	1	1
2	Power	PSRF_Labs_project.CW_Meas...	1	1	1	1	1

ADVANTEST

Pin Configuration Tool

Digital pins

CONTEXT: **DEFAULT**

	Name	No	Type	Mode	HW	Connection Set	Comment	Tester Channel	
								Site 1	Site 2
1	CP	12	i	std	PS1600			10509	10609
2	DS7	18	i	std	PS1600			10515	10615
3	DS0	11	i	std	PS1600			10508	10608
4	S1	19	i	std	PS1600			10516	10616
5	S0	1	i	std	PS1600			10507	10607
6	_MR	9	i	std	PS1600			10501	10601
7	Q7	17	o	std	PS1600			10514	10614
8	Q0	8	o	std	PS1600			10502	10602
9	I/O7	16	io	std	PS1600			10513	10613
10	I/O6	4	io	std	PS1600			10506	10606
11	I/O5	15	io	std	PS1600			10512	10612
12	I/O4	5	io	std	PS1600			10505	10605
13	I/O3	14	io	std	PS1600			10511	10611
14	I/O2	6	io	std	PS1600			10504	10604
15	I/O1	13	io	std	PS1600			10510	10610
16	I/O0	7	io	std	PS1600			10503	10603

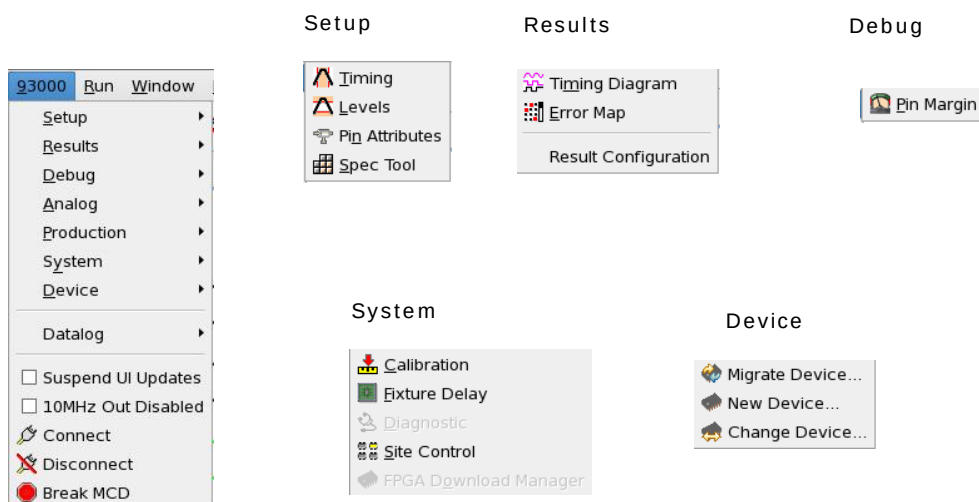
Analog or RF pins

	Name	No	Type	Instrument	Comment	Tester Channel			
						Site 1		Site 2	
						Pogo	Pad	Pogo	Pad
1	LB_src	1	io	RFI		324	1A1	324	1C1
2	LB_meas	2	io	RFI		324	2A1	324	2C1

ADVANTEST

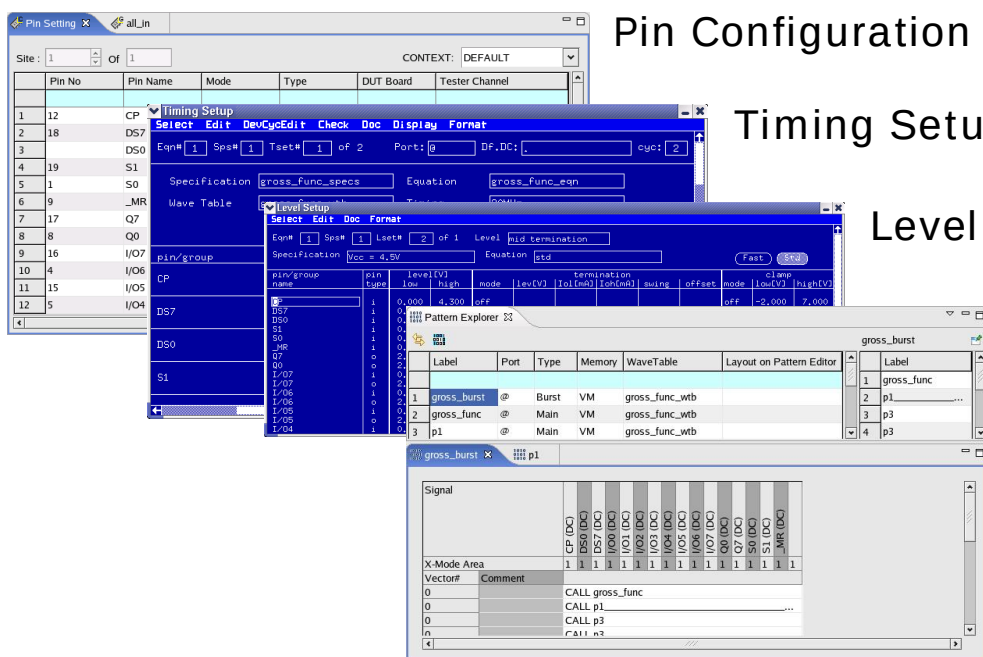
Accessing Setup Tools from the Eclipse Toolbar

The setup and debug tools can be accessed from the menu bar of the **Eclipse Work Center**



ADVANTEST

Setup Tools



Pin Configuration

Timing Setup

Level Setup

Pattern edit & debug

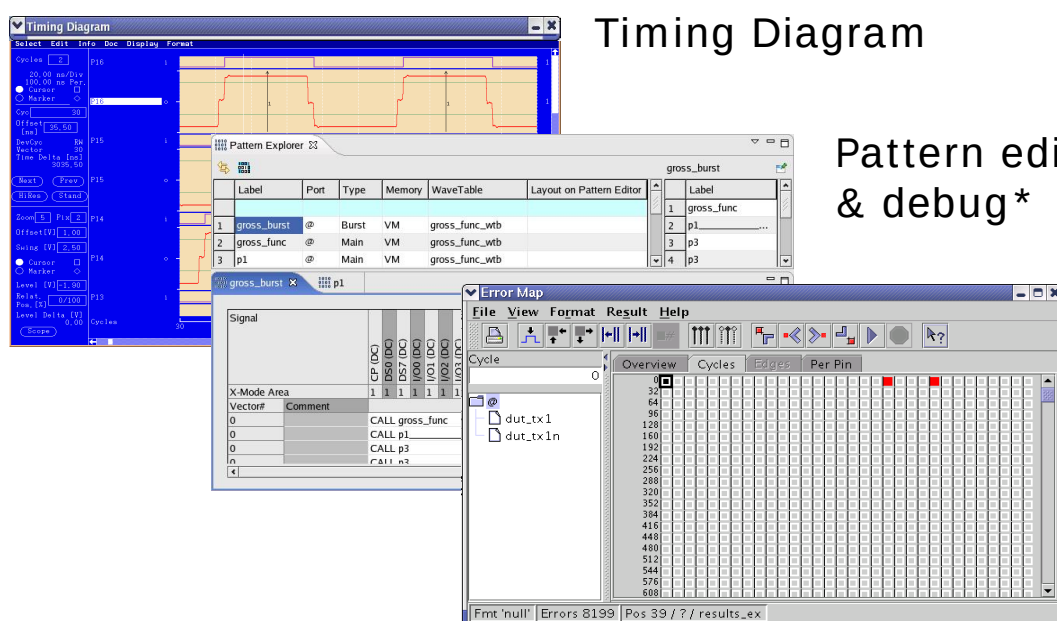
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 25

Functional debug Tools



Timing Diagram

Pattern edit & debug*

Error Map

* same GUI as in the setup tools

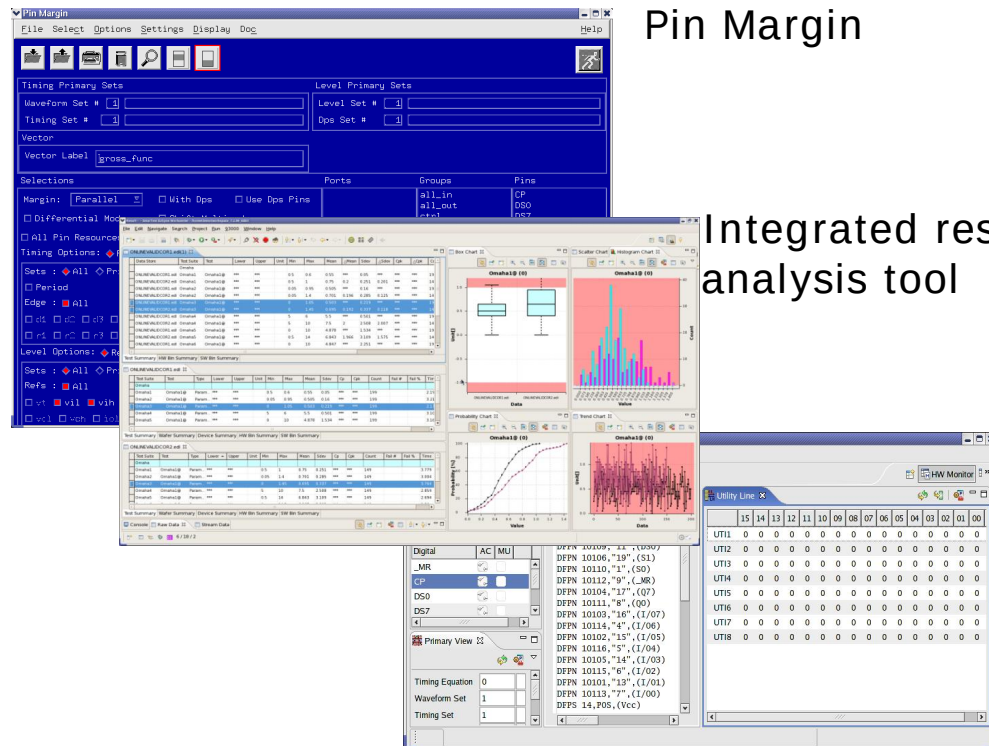
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Software Overview Unit 3 - 26

Result Analysis Tools



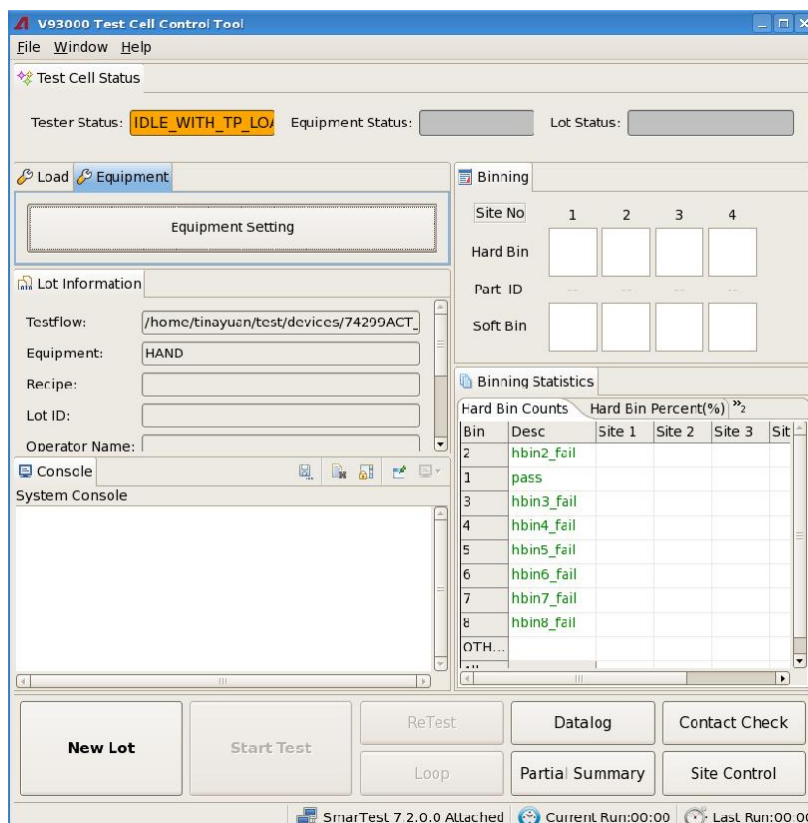
Pin Margin

Integrated result analysis tool

HW monitor

ADVANTEST

Test Cell Control Tool



ADVANTEST

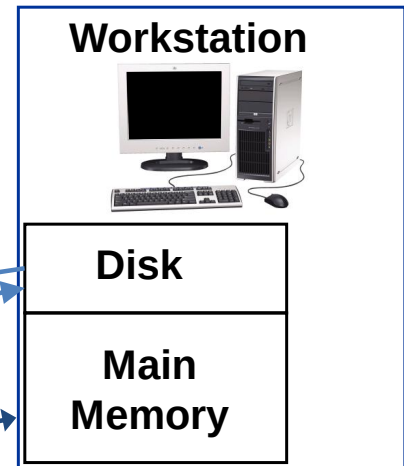
Data Handling

Load, Save, Apply

- Load – download from the setup files in the disk workstation to the V93000 hardware
- Save – from the V93000 hardware to the setup files in the disk workstation
- Upload – from the V93000 hardware to the workstation main memory



load
save
upload



ADVANTEST

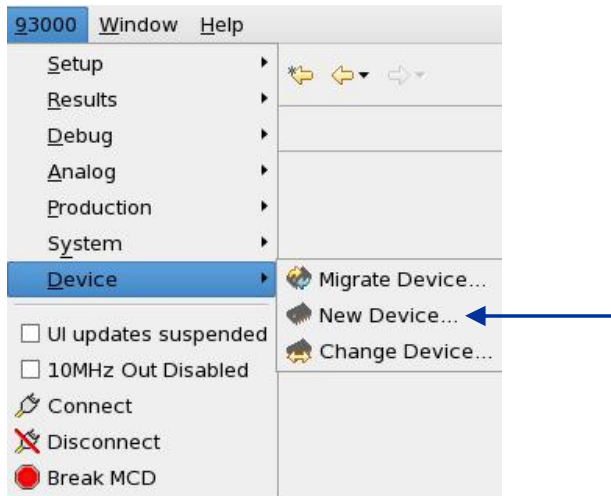
Data Handling

Device Directory

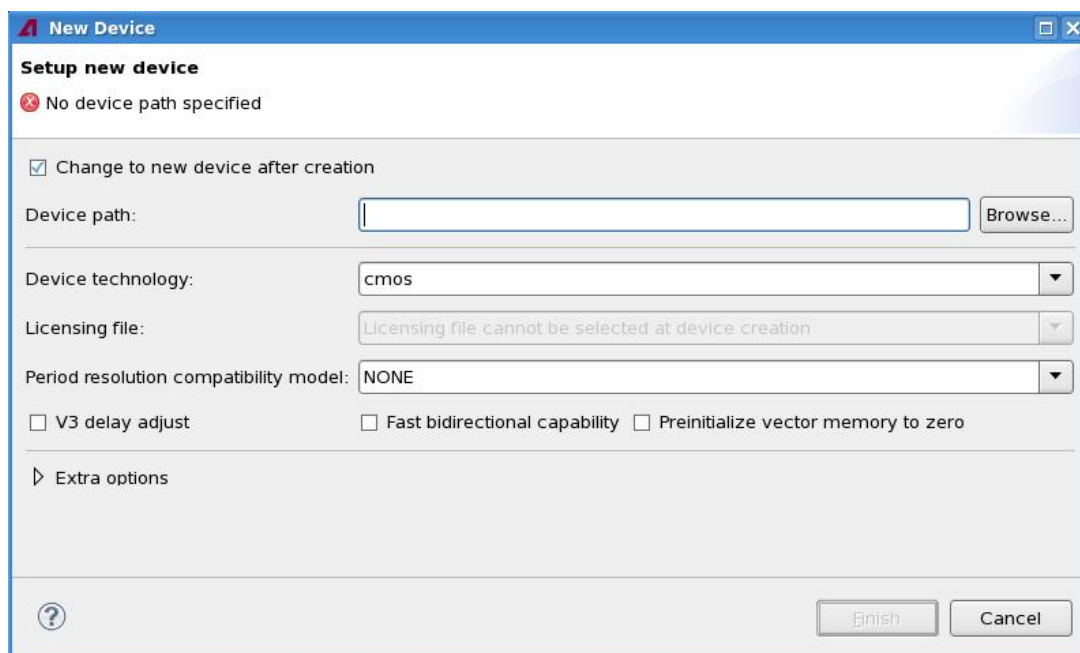
- A device directory is a structure that contains all basic test setup elements
- Each device configured on the V93000 SOC Series has its own **device directory**
- Each time you use the system, a file called **.device_inuse_soc** is stored in your home directory
- This file remembers which device you were using last, and sets a path to its device directory
- If a project directory is being used, a lock file is created in the device directory. This prevents other users from using that device software.

ADVANTEST

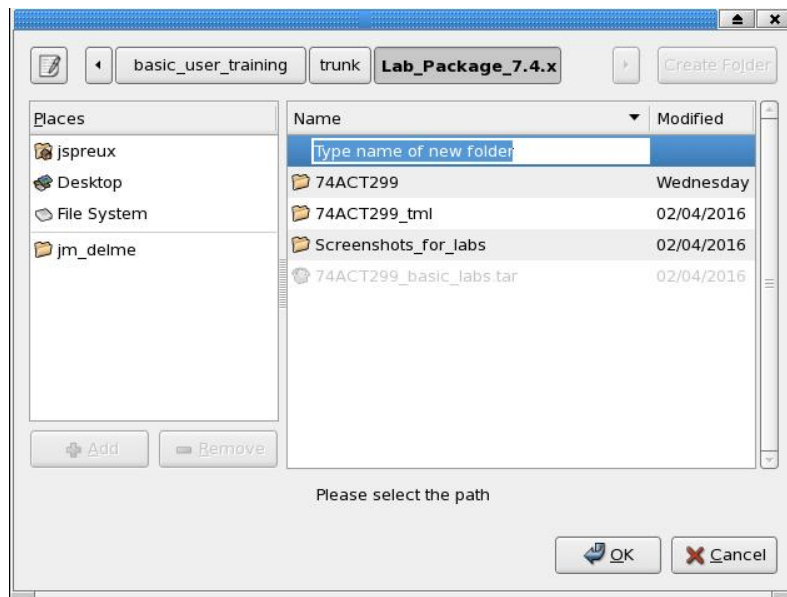
Creating a new Device Directory



Selecting Device Path



Adding Device Directory



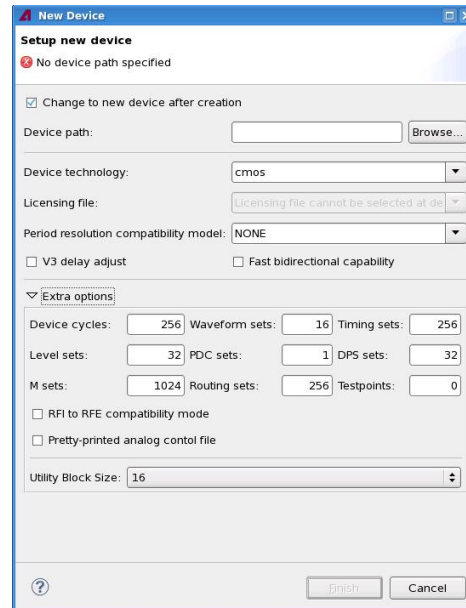
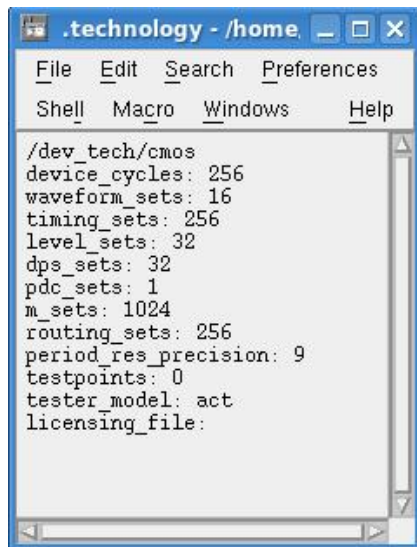
Device Directory Structure

- When a new device file (test program) is created, the subdirectories below are generated automatically

```
socbu173:/home/jspreux/Digital_Training_7_4_x/basic_user_training/trunk/Lab_Package_7.4.x/74
[34m/home/jspreux/Digital_Training_7_4_x/basic_user_training/trunk/Lab_Package_7.4.x/74ACT299 $ ls
analog_control  configuration  format      mtp          protocols    scan_res     testflow_vee
applicat        data_set      format.TS_DEFAULT_DC  mts          report       shmoo        testfunc
calibration     di_report     levels      pin_attributes  routing      state_list   testmethod
ch_attributes    error_map     license     profiles      scan         testflow     testprog
socbu173:/home/jspreux/Digital_Training_7_4_x/basic_user_training/trunk/Lab_Package_7.4.x/74ACT299 $
```

Storing Device Parameters

.technology File



- As extra options when creating a device

Located under *<device_directory>/.technology*

Model file

When working in OFFLINE mode, the model file is needed.

Model file entry for Digital Pins i.e. PS1600

```
IOCHANNEL
# Pin Scale System
# 10101-10216:HW = PS800, speed = 800 Mbps, smem = 64M, diff, ppTIA
# 31701-31816:HW = PS3600, speed = 3600 Mbps, smem = 64M, diff, ppTIA

# 10901-11216:HW = PS400, speed = 800 Mbps, smem = 112M, ppTIA
# 10901-10916:HW = PS400, speed = 800 Mbps, smem = 56M, ppTIA
# 11501-11516:HW = PS400, speed = 800 Mbps, smem = 112M, ppTIA
# 10101-13216:HW = PS9G, speed = 9000 Mbps, smem = 112M, diff, tmu, loopback = 9000Mbps

# 10101-10816:HW = PS1600, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps, tmu
# 20101-20216:HW = E8030, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps
# 40101-40216:HW = E8030, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps
# 60101-60216:HW = E8030, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps
# 80101-80216:HW = E8030, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps

10101-13216:HW = PS1600, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps, tmu
20101-22416:HW = PS1600, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps, tmu
32501-33216:HW = PS1600, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps, tmu
40101-42416:HW = PS1600, speed = 1600 Mbps, smem = 112M, diff, loopback = 1600Mbps, tmu
```

License

The license scheme includes three licenses

1. License for Smartest

Smartest Online License

2. License for S&S Calibration converter

SW Online Value Package

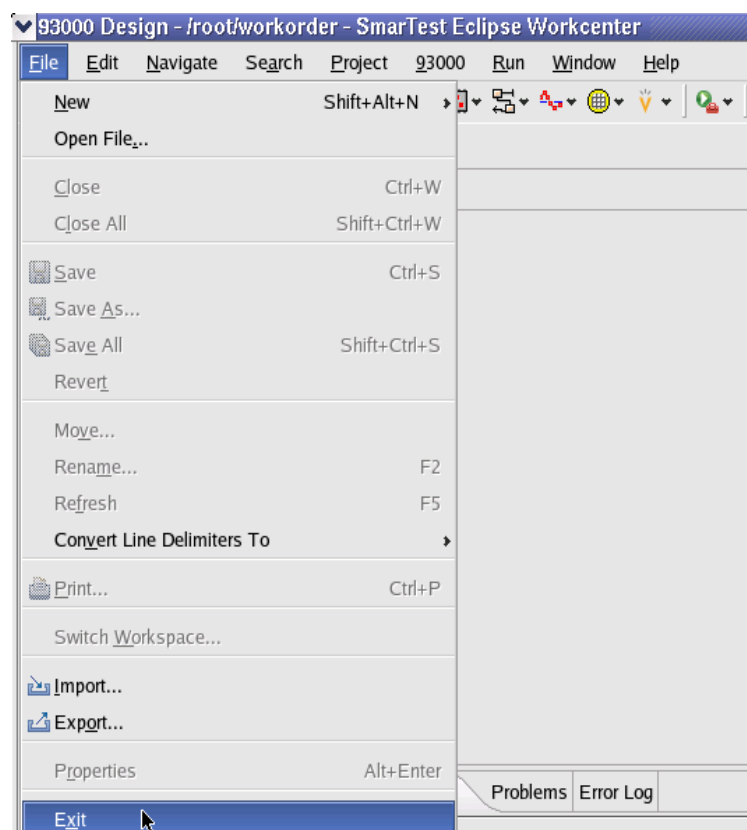
3. License for S&S MIGRATION FRAMEWORK

SW Offline Tool package

Quitting SmarTest

To exit SmarTest click

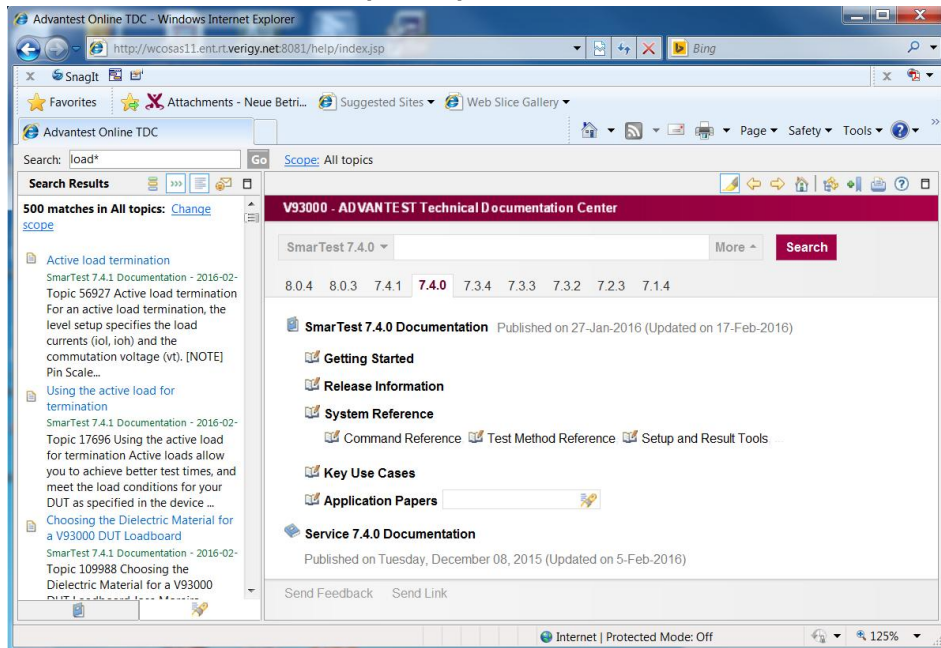
File > Exit



SmarTest Documentation TDC

Access: <https://www.advantest.com/V93000/help/index.jsp>

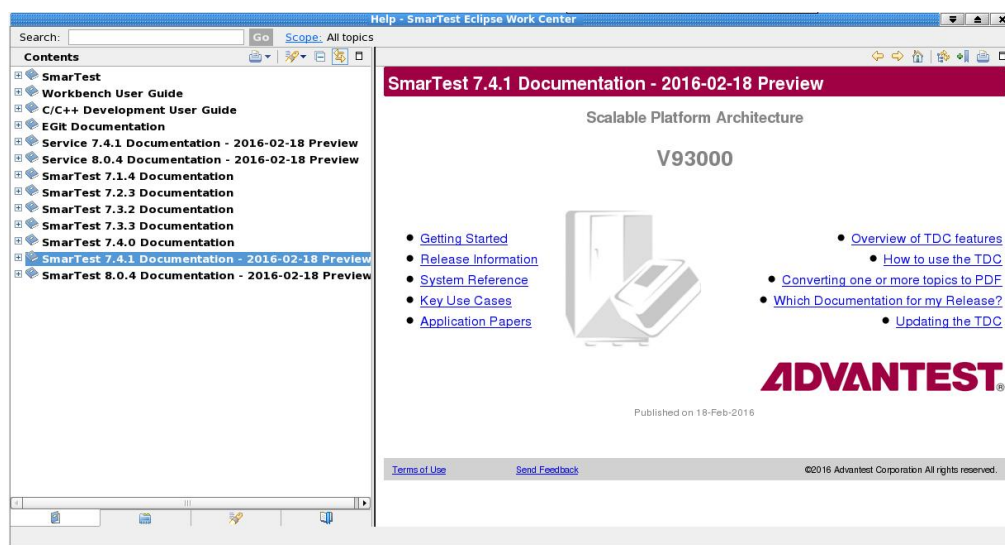
The SmarTest documentation is provided in the V93000 Technical Documentation Center (TDC)



ADVANTEST

SmarTest Help

When running SmarTest you can launch SmarTest Help by choosing Help > Help Contents



ADVANTEST

SmarTest Dynamic Help

1. Load all the setup files of your device
2. In the Flow Data Editor, select any test method, right click and choose “Modify Test Method... F3”
3. Open your test method in the Test Method editor
4. In SmarTest click Help > Dynamic Help
5. Highlight an API; the related information from TDC pops up on the right side

Advantest V93000

Digital User Training

Pin Configuration

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Purpose
- Allocation of Tester Resources
- Pin Configuration User Interface
- Allocating Digital Channels, Supplies & Utility Lines
- Grouping Pins

ADVANTEST[®]

Learning Objectives

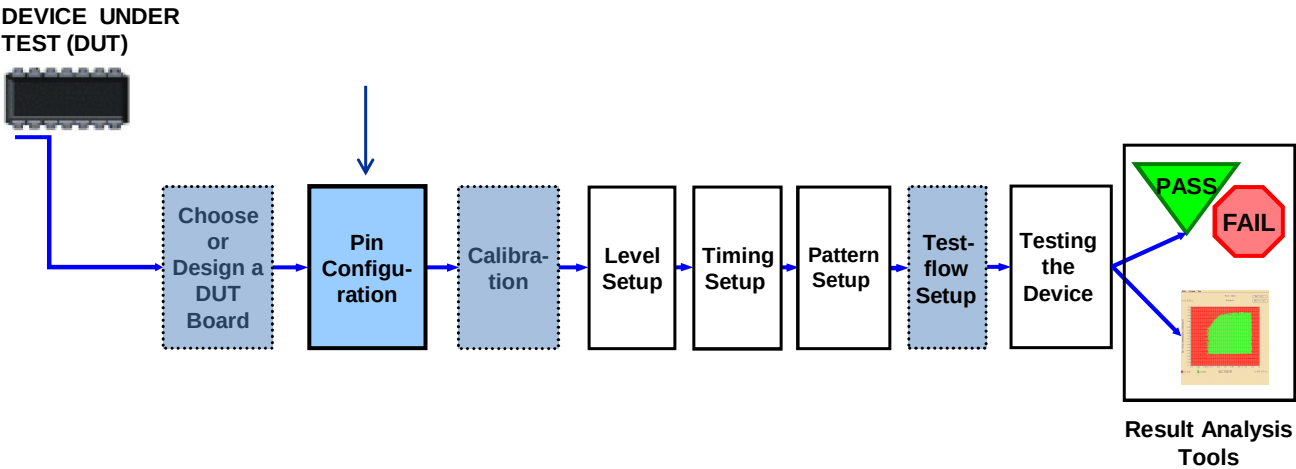
- Know the pin configuration tool and how to use it during the device setup

Relevance

- DUT pin to tester channel allocation is one of the first setups during the development of a device

Test Program Flow Chart

Defining the Pins



ADVANTEST®

April 2016

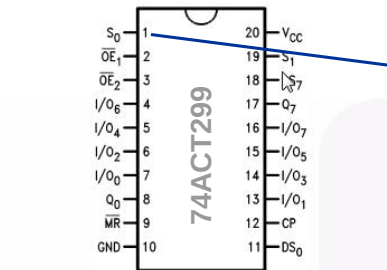
All Rights Reserved - ADVANTEST CORPORATION

Pin Configuration Unit 4 - 5

Pin Configuration Purpose

DUT data sheet excerpt PCB Pin list: 4 x74ACT299

DUT board allocation



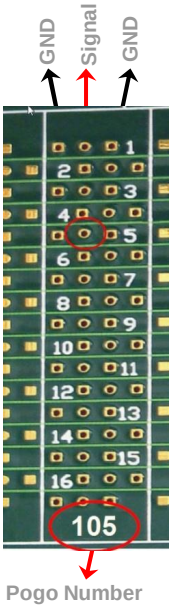
	Name	DUT1	DUT2	DUT3	DUT4
1	S0	D10507	D10607	D11015	D30104
2	OE1	D30209	D30211	D30205	D30208
3	OE2	D30210	D30212	D30206	D30207
4	I/O_6	D10506	D10606	D11013	D30103
5	I/O_4	D10505	D10605	D11011	D30105
6	I/O_2	D10504	D10604	D11009	D30102

Pin Description

Pin Names	Description
CP	Clock Pulse Input
DS ₀	Serial Data Input for Right Shift
DS ₇	Serial Data Input for Left Shift
S ₀ , S ₁	Mode Select Inputs
MR	Asynchronous Master Reset
OE ₁ , OE ₂	3-STATE Output Enable Inputs
I/O ₀ -I/O ₇	Parallel Data Inputs or 3-STATE Parallel Outputs
Q ₀ , Q ₇	Serial Outputs

SmarTest Pin Configuration Tool

	Name	No	Type	Mode	HW	Connection Set	Comment	Tester Channel Site 1	Site 2
1	CP	12	i	std	PS1600			10509	10609
2	DS7	18	i	std	PS1600			10515	10615
3	DS0	11	i	std	PS1600			10508	10608
4	S1	19	i	std	PS1600			10516	10616
5	S0	1	i	std	PS1600			10507	10607
6	MR	9	i	std	PS1600			10501	10601
7	Q7	17	o	std	PS1600			10514	10614
8	Q0	8	o	std	PS1600			10502	10602
9	I/O7	16	io	std	PS1600			10513	10613
10	I/O6	4	io	std	PS1600			10506	10606
11	I/O5	15	io	std	PS1600			10512	10612
12	I/O4	5	io	std	PS1600			10505	10605
13	I/O3	14	io	std	PS1600			10511	10611
14	I/O2	6	io	std	PS1600			10504	10604
15	I/O1	13	io	std	PS1600			10510	10610
16	I/O0	7	io	std	PS1600			10503	10603



Trace on interface-PCB from Pogo-block to device pin

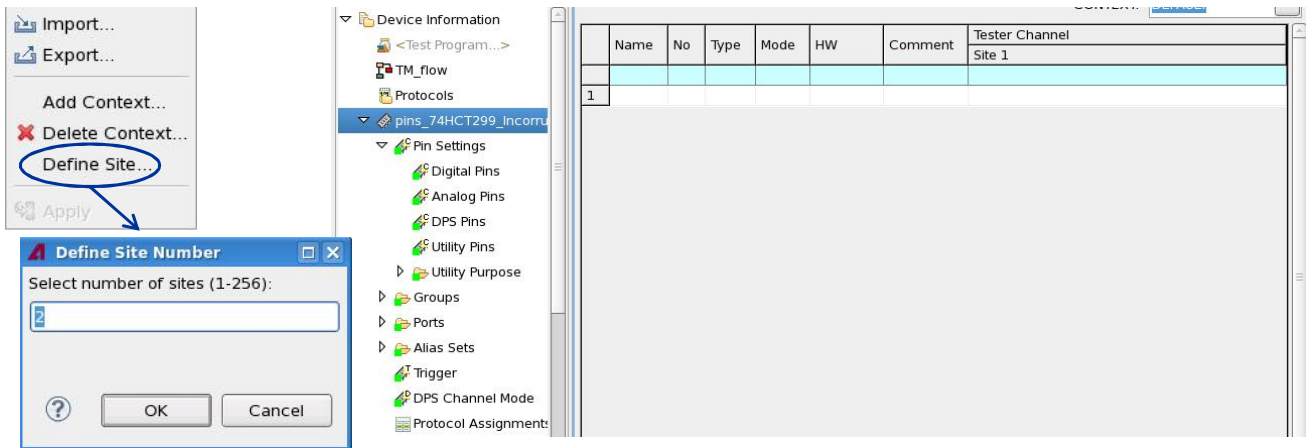
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Pin Configuration Unit 4 - 6

Basic Steps in Pin Configuration Tool



1. Select Eclipse Setup Perspective
2. Select 'Test Program Explorer'; this is a central place where you find all loaded setups in a hierarchically structured way
3. Unfold <Pin Configuration> (common folder for pin-relevant definitions)
4. Unfold 'Pin_Setting' for basic pin definitions, right-click for defining number of sites
5. Start with 'Digital Pins', continue with 'DPS Pins'

ADVANTEST

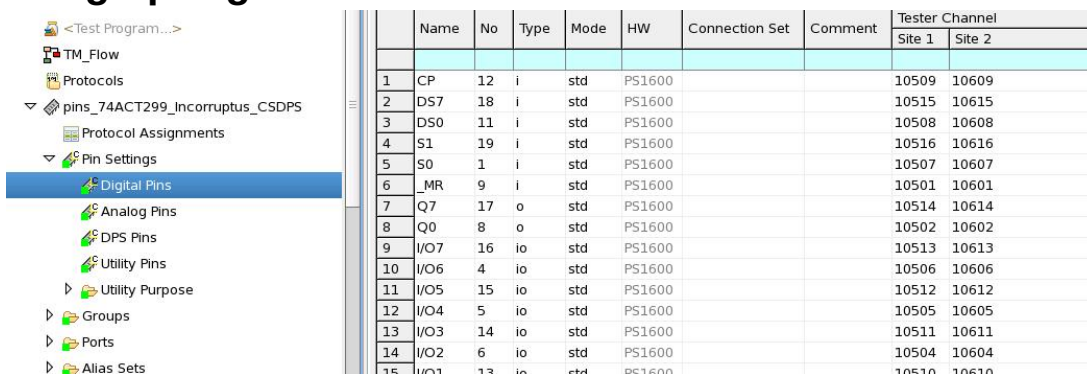
April 2016

All Rights Reserved - ADVANTEST CORPORATION

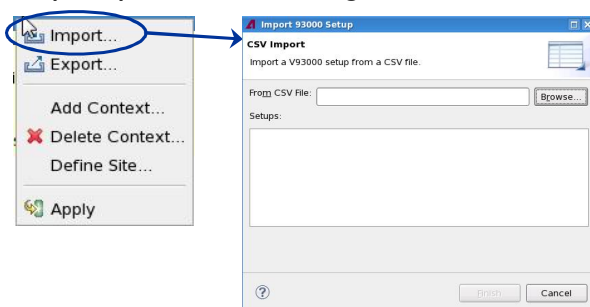
Pin Configuration Unit 4 - 7

Digital Pins Configuration

Filling up Digital Pins Section



Import pin-list out of a given ".csv"-file



“.csv”-file sample:

```

$, "Digital Pins"
>, "Site Number", 1
>, Name, No, Type, Mode, "Series Resistor
    [Ohm]", Instrument, Comment, "Tester Channel 1"
, CP, 12, i, std, 0.0, PS800, 10509
, DS7, 18, i, std, 0.0, PS800, 10515
, DS0, 11, i, std, 0.0, PS800, 10508
, S1, 19, i, std, 0.0, PS800, 10516
    
```

ADVANTEST

April 2016

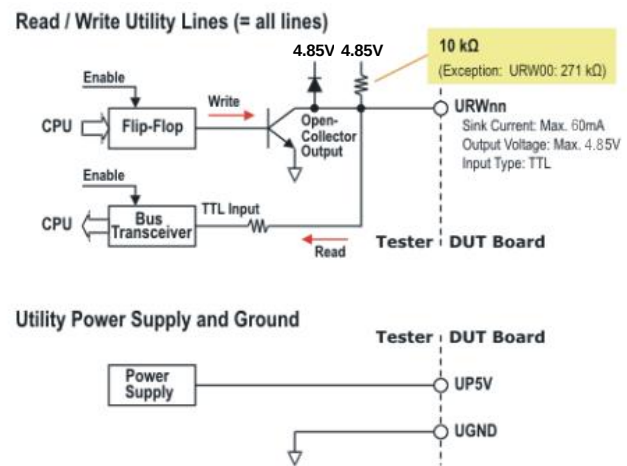
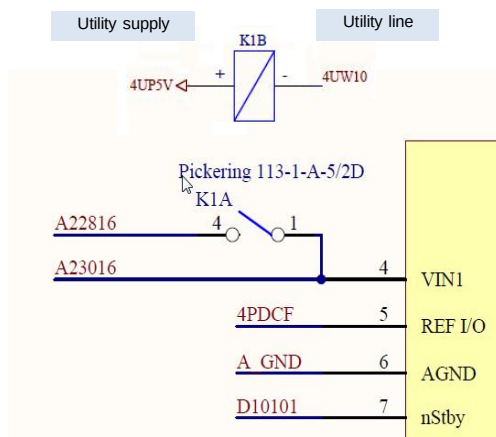
All Rights Reserved - ADVANTEST CORPORATION

Pin Configuration Unit 4 - 8

Utility Pins Configuration

Utility Pins Basics

.. allow you to control external devices such as relays, indicators (LEDs).
The lines are implemented as open collector outputs (16 lines per block).
Components are sourced by utility supply (Pin Scale: 4.85 V +1% / - 5%).

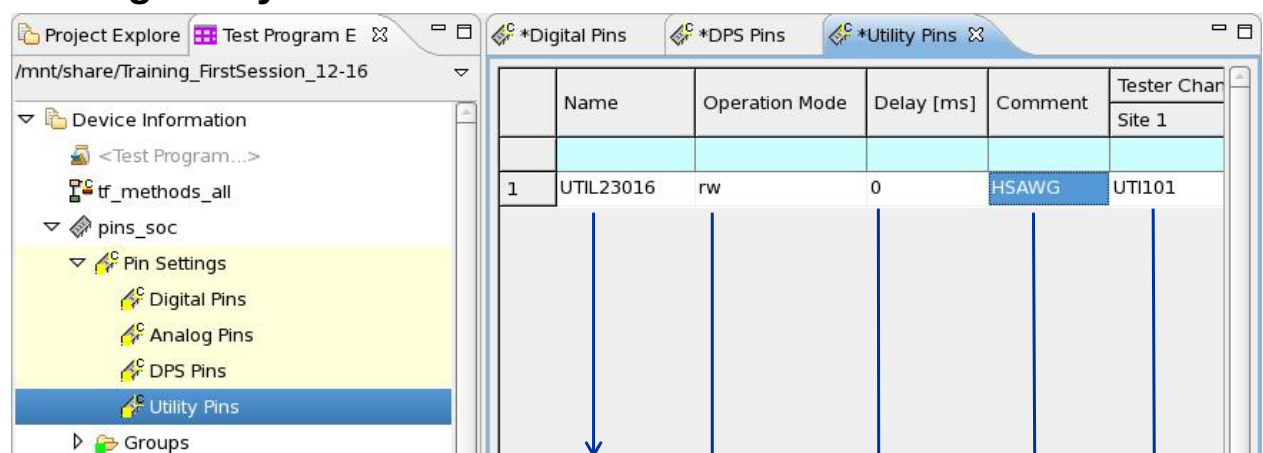


To make setups and program-code more readable, names are given to utility lines.

ADVANTEST

Utility Pins Configuration

Defining Utility Pins



Unique name

Read/Write capability

Settling time

Comment

Hardware resource

ADVANTEST

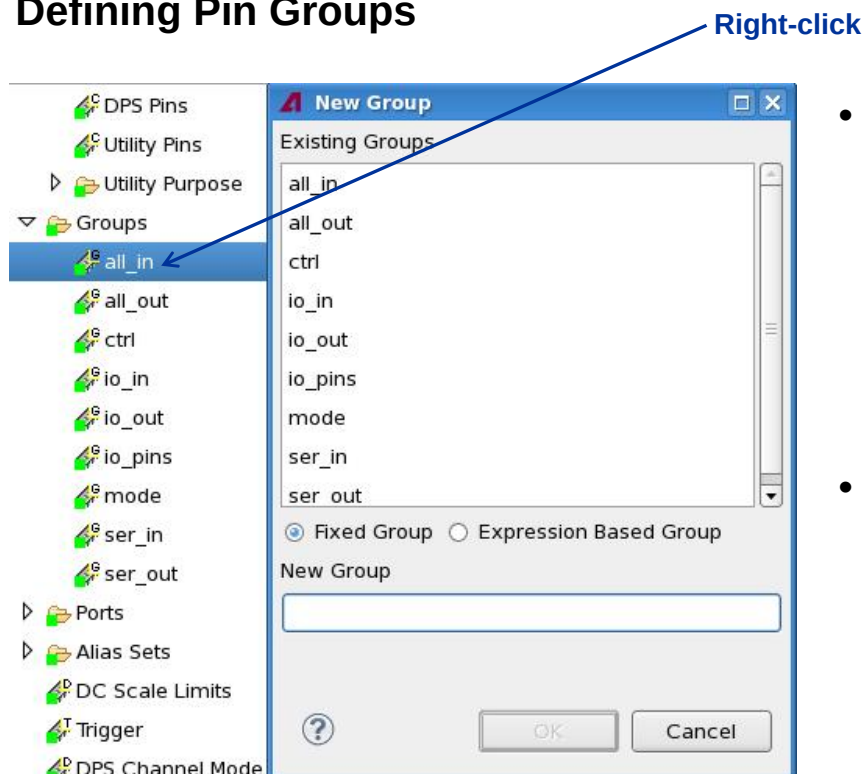
Digital Pins Configuration

Pin Groups Basics

- Pins with same specification (levels, timing, ...)
- Pins with same functionality (e.g. data-bus, address-bus, MC-ports) are grouped to ease further setups
 - Definition done to a group will affect all members
 - Logical state of a bus is defined through a value

Digital Pins Configuration

Defining Pin Groups



- **Fixed Group:**
Subset of input and/or output pins as defined in Pin Configuration Tool
- **Expression Based Group:**
Consists of Digital Pins/Pin Groups and group arithmetic operators

Digital Pins Configuration

Defining Fixed Groups

Group in focus

MSB

LSB

I/O pins appear separately as input and output

Pin is member of current group

ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Pin Configuration Unit 4 - 15

Digital Pins Configuration

Defining Expression-based Groups

Available operators:
Add
Subtract
Common Subset

Shows result of graphical operation

Edit expression in this editor field and evaluate

ADVANTEST®

April 2016

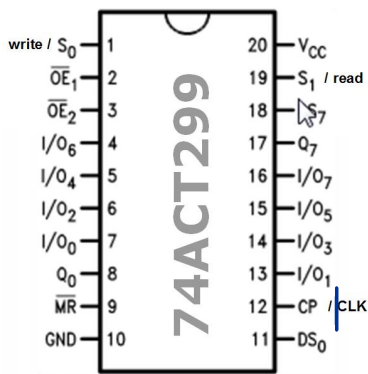
All Rights Reserved - ADVANTEST CORPORATION

Pin Configuration Unit 4 - 16

Digital Pins Configuration

Pin Alias

Different names for single pins are required for e.g. multi-core devices (pins are multiplexed)



Open "Alias Sets Editor" through Test Program Explorer

Alias set name

Alias

	Alias	Pin
1	CLK	CP
2	write	S0
3	read	S1

	Trans_IO::CLK	CP (PW)
Comment		
reset	1	1
hold	1	1
par load 0x80	1	1
shift right	1	1
	1	1

After defining alias, its name is available in all setups, e.g. can be seen in Pattern Tool

ADVANTEST

Content of Pin Configuration File

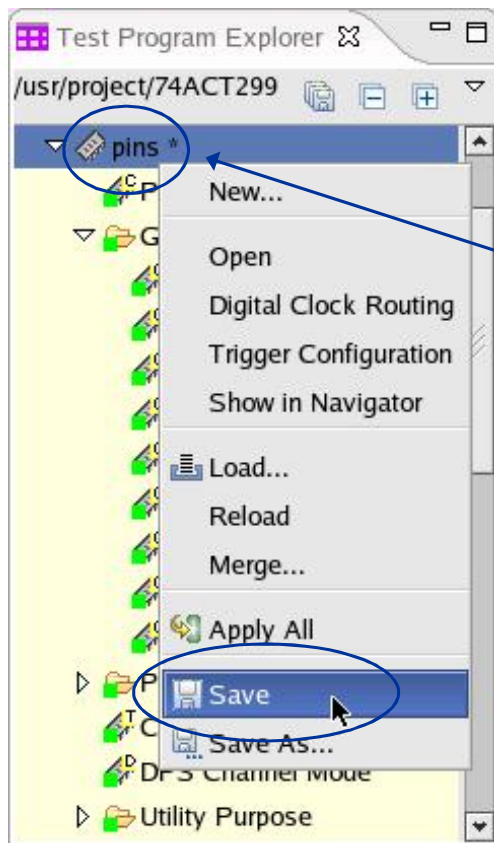
- With the exception of patterns, all the SmarTest files are readable with ASCII editors
- SmarTest files may be also written by proper conversion tools or scripts

Example of a pin configuration file:

```
hp93000,config,0.1
DFPN 10509,"12",(CP)
DFPN 10515,"18",(DS7)
DFPS 14,POS,(Vcc)
PALS 2,10609,,(CP)
PALS 3,11002,,(CP)
PALS 2,10615,,(DS7)
PALS 3,11016,,(DS7)
PALS 2,18,,(Vcc)
PALS 3,13,,(Vcc)
PSTE 3
CONF I,F160,(CP,DS7,DS0,S1,S0,_MR)
CONF O,F160,(Q7,Q0)
CONF IO,F160,(I/O7,I/O6,I/O5,I/O4,I/O3,I/O2,I/O1,I/O0)
DFGP I,(I/O7,I/O6,I/O5,I/O4,I/O3,I/O2,I/O1,I/O0),(io_in)
DFGP O,(I/O7,I/O6,I/O5,I/O4,I/O3,I/O2,I/O1,I/O0),(io_out)
DFGP I,(S1,S0),(mode)
DFGP I,(DS7,DS0),(ser_in)
DFGP O,(Q7,Q0),(ser_out)
DFGE IO,"io_in + io_out",(io_pins)
RDIV
0,(CP,DS7,DS0,S1,S0,_MR,Q7,Q0,I/O7,I/O6,I/O5,I/O4,I/O3,I/O2,I/O1,I/O0)
DDSL 0,(Vcc)
PSLC 2.3,(Vcc)
PSVR 500,(Vcc)
NOOP "7.1.4",,,
```

ADVANTEST

Saving the Pin Configuration File (1)



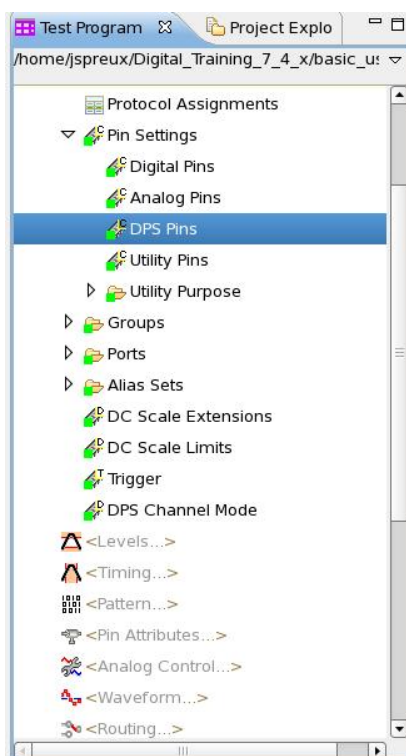
After entering your pin details and applying the changes to the tester hardware, the pins icon will appear with an asterisk: *

The asterisk * indicates that the data in the hardware no longer matches the data which is stored in the pin configuration file

Save your file as necessary

ADVANTEST

Saving the Pin Configuration File (2)



- When you edit the pin configuration and download it (apply), you must **reload** the other setups (don't save them!)
- *Saving* operation will overwrite your original setup files

ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):
<https://www.advantest.com/V93000/help/index.jsp>

Advantest V93000

Digital User Training

Levels

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Level Resources
- Equation Based Setup Concept
- Specification
- Verification

ADVANTEST[®]

Learning Objectives

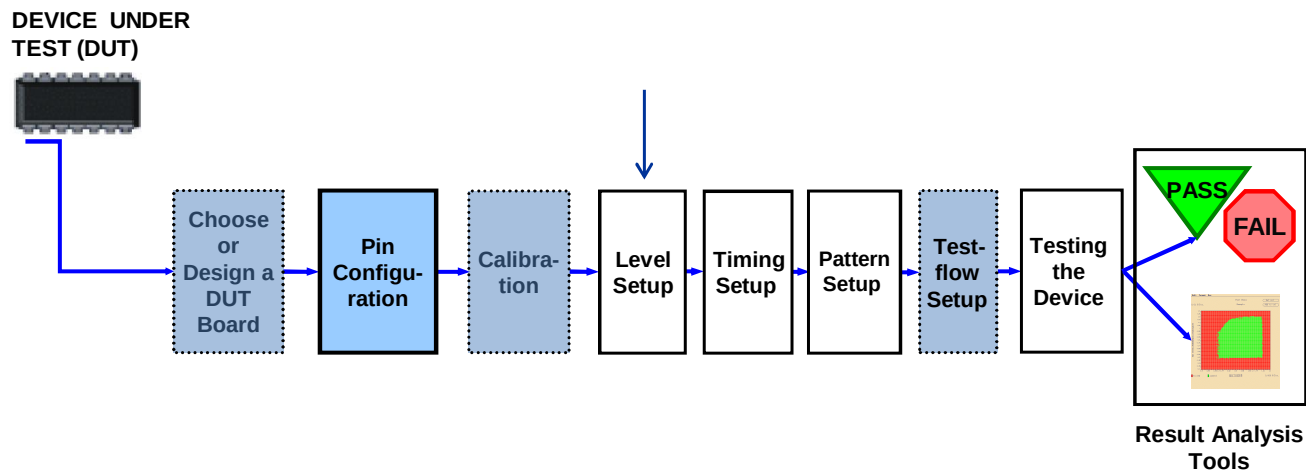
- Know the resources available to perform a level setup
- Know the tools used to perform a level setup
- Know how to translate a datasheet into the level setup files

Relevance

Level setup is the step where you consider the DC electrical characteristics of a DUT versus the pin electronic of the V93000 system.

Test Program Flow Chart

Defining the Levels



ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 5

Level Setup Purpose

Device requirements

V93000 levels realization

DC Electrical Characteristics for ACT

Symbol	Parameter	V _{CC} (V)	T _A = 25°C		T _A = -40°C to +85°C		Units	Conditions
			Typ	Guaranteed Limits				
V _{IH}	Minimum HIGH Level Input Voltage	4.5	1.5	2.0	2.0	V	V _{OUT} = 0.1V or V _{CC} - 0.1V	
		5.5	1.5	2.0	2.0			
V _{IL}	Maximum LOW Level Input Voltage	3.0	1.5	0.8	0.8	V	V _{OUT} = 0.1V or V _{CC} - 0.1V	
		4.5	1.5	0.8	0.8			
V _{OH}	Minimum HIGH Level Output Voltage	4.5	4.49	4.4	4.4	V	I _{OUT} = -50 μA	
		5.5	5.49	5.4	5.4			
		4.5				V	V _{IN} = V _{IL} or V _{IH} I _{OH} = -24 mA I _{OH} = -24 mA (Note 4)	
		5.5	0.0001	3.86	3.76			
		4.5				V	I _{OH} = -24 mA (Note 4)	
		5.5	0.0001	4.86	4.76			
V _{OL}	Maximum LOW Level Output Voltage	4.5	0.001	0.1	0.1	V	I _{OUT} = 50 μA	
		5.5	0.001	0.1	0.1			
		4.5				V	V _{IN} = V _{IL} or V _{IH} I _{OL} = 24 mA I _{OL} = 24 mA (Note 5)	
		5.5		0.36	0.44			
I _{IN}	Maximum Input Leakage Current	4.5		0.36	0.44	V	V _I = V _{CC} , GND V _I = V _{CC} - 2.1V	
		5.5		0.36	0.44			
I _{CC1}	Maximum I _{CC} /Input	5.5	0.6		1.5	mA	V _I = V _{CC} , GND	
I _{OLD}	Minimum Dynamic Output Current (Note 6)	5.5			75	mA	V _{CE} = 1.65V Max	
I _{OHD}	Output Current (Note 6)	5.5			-75	mA	V _{OH} = 3.85V Min	
I _{CC}	Maximum Quiescent Supply Current	5.5		4.0	40.0	μA	V _{IN} = V _{CC} or GND	
I _{OZT}	Maximum I/O Leakage Current	5.5		±0.3	±3.0	μA	V _I (OE) = V _{IL} , V _{IH} V _I = V _{CC} , GND V _O = V _{CC} , GND	

Note 5: All outputs loaded; thresholds on input associated with output under test.

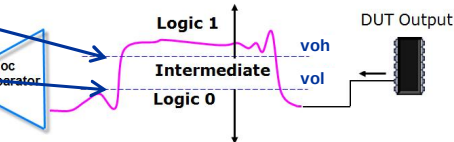
Note 6: Maximum test duration 2.0 ms, one output loaded at a time.

Equation based definition

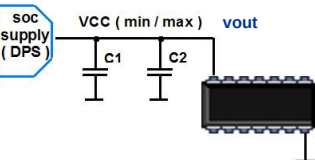
Input levels



Output levels



Supply levels



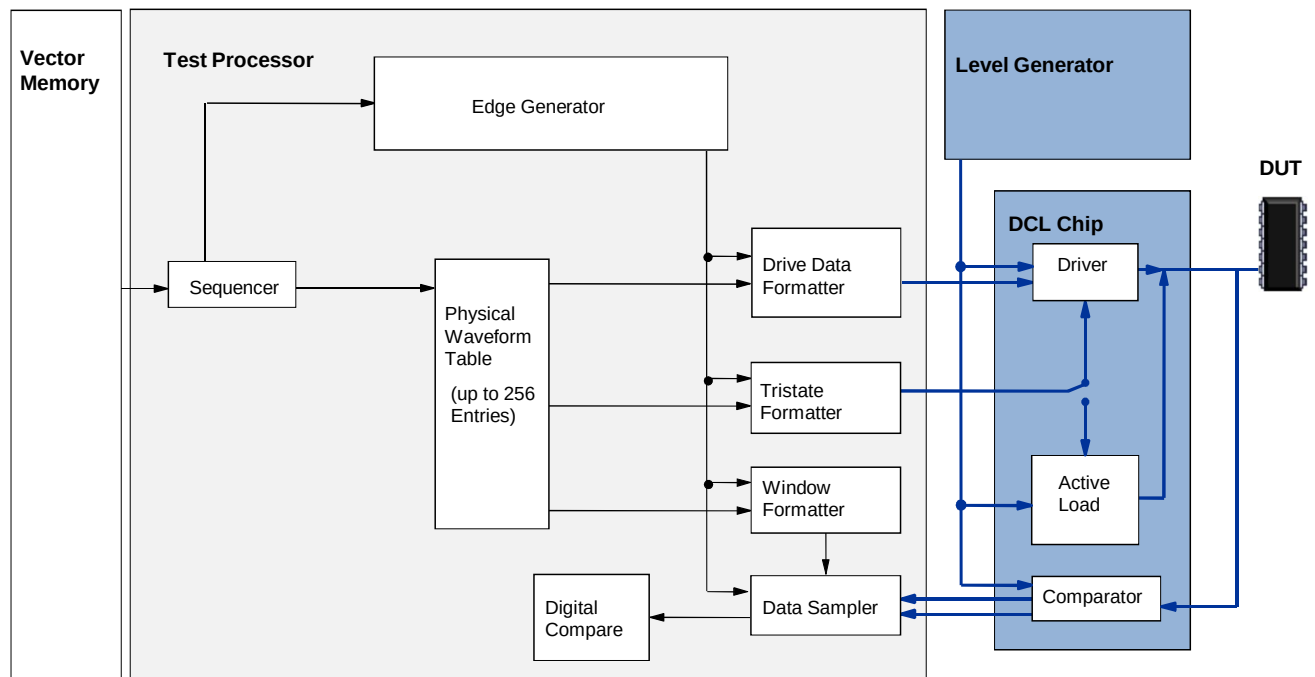
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 6

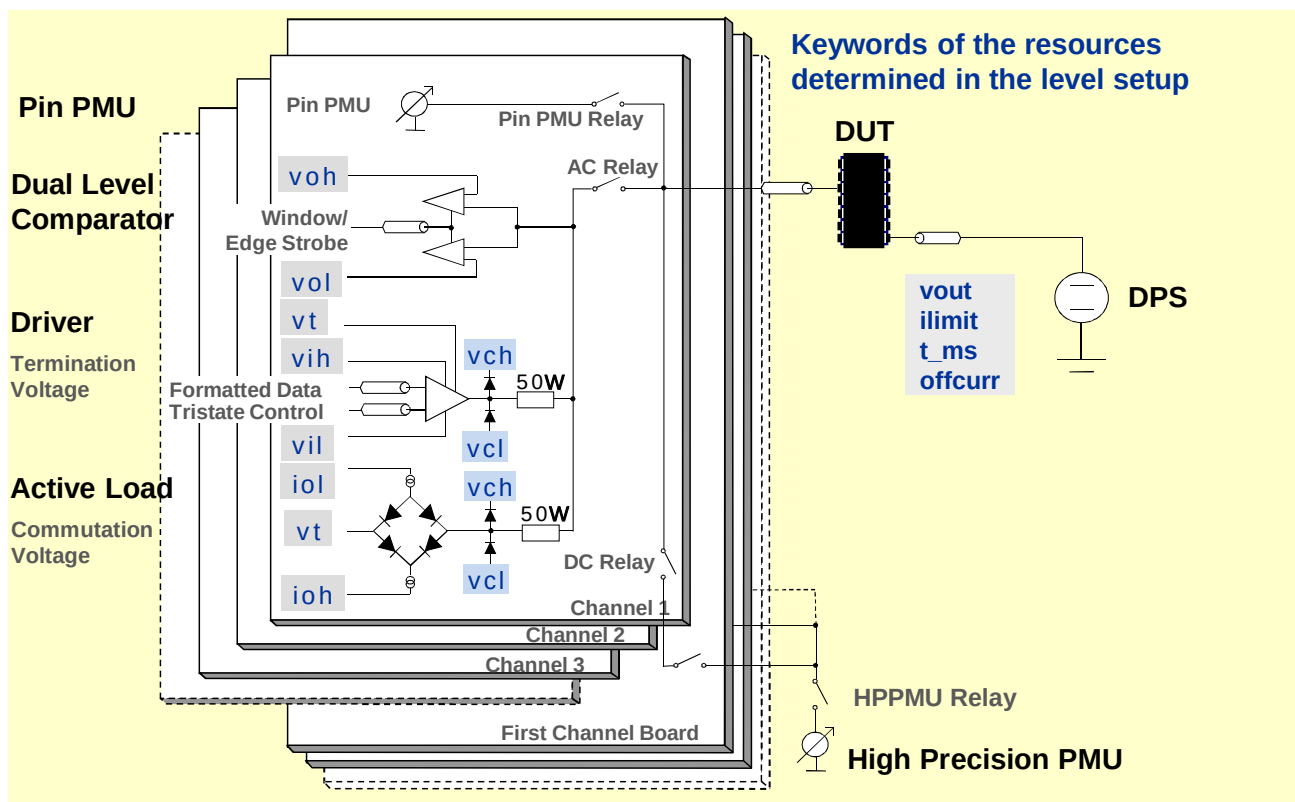
Level Generation within I/O Channel Pin Electronics



Resources are available per pin

ADVANTEST

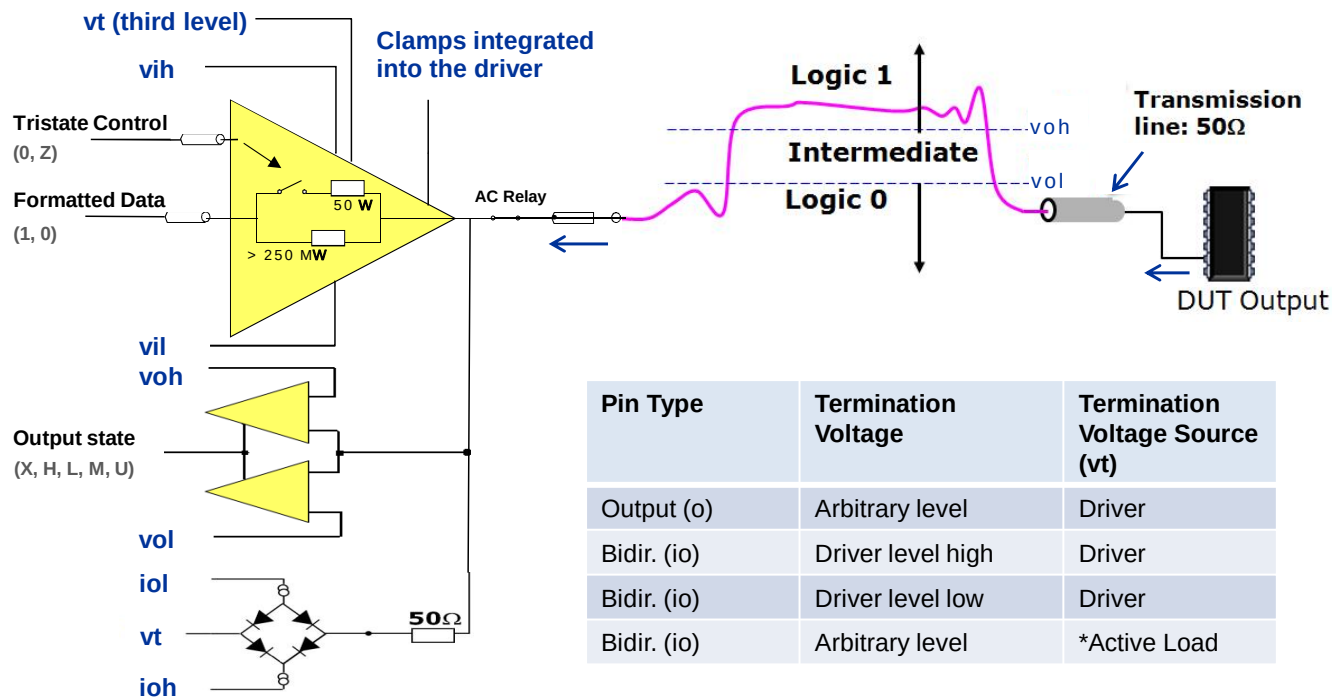
PinScale I/O Level Resources



ADVANTEST

Termination and Active Load

- Tristating the driver is coupled with switching the active load (on/off)*
- 'vt' defined only (no 'iol'/'ioh') = termination voltage (from driver or from active load module)



ADVANTEST®

Termination and Active Load

Termination modes for PS1600 in standard mode

Termination mode	Description	Parameters to declare
T	Driver termination with specified voltage	voh, vol, vch, vcl, vt
H	Driver high termination	vt = vih
L	Driver low termination	vt = vil
A	Active load	voh, vol, vch, vcl, vt, ioh, iol
C	Clamp	voh, vol, vch, vcl

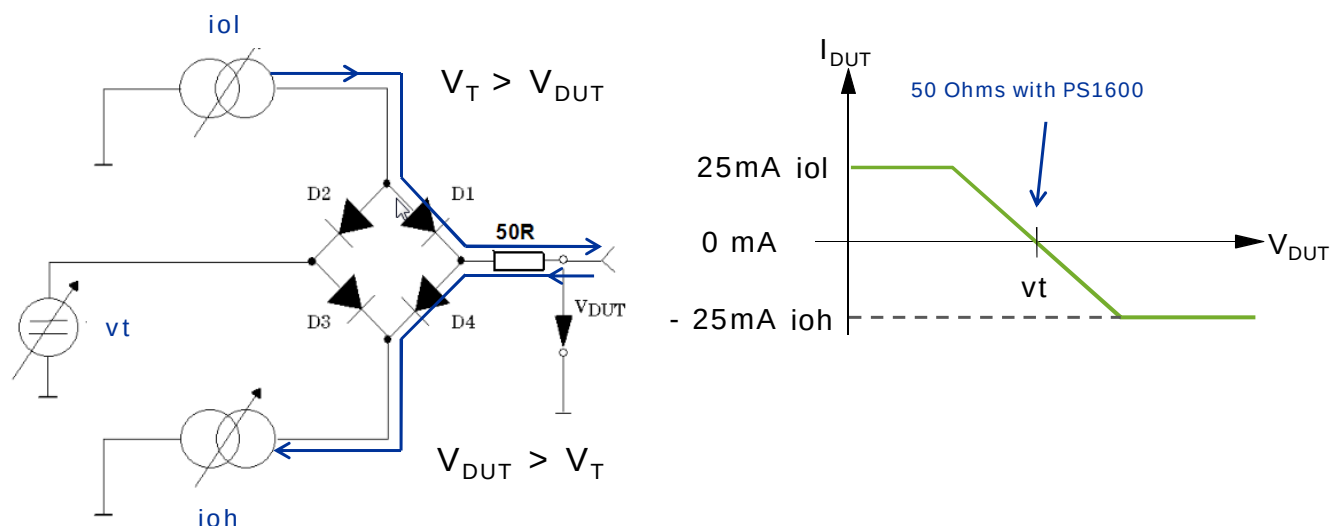
...

The termination mode is not specified directly.
It is determined from the presence (or absence) of the vch, vcl, ioh, iol and vt group of resources.

ADVANTEST®

Active Load

i_{ol} : source load current for low output levels
 i_{oh} : sink load current for high output levels
 v_t : commutation voltage



ADVANTEST

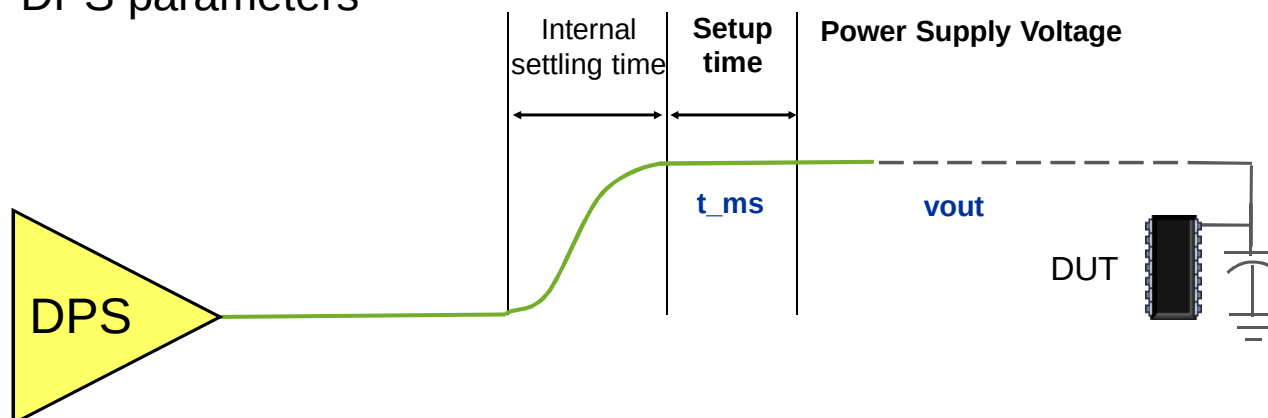
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 11

Device Power Supply (DPS) V/I Definition

DPS parameters



Connect Current Limit: i_{limit} (A) Prevents destruction of the DUT

Disconnect State:

- $offcurr = min$ High impedance
- $offcurr = act$ Drive 0V up to i_{limit}
- $offcurr = open$ Output relay off

For DPS128:

i_{limit_source} , i_{limit_sink} ,
 $v_{out_frc_rng}$, $i_{out_clamp_rng}$
 ...

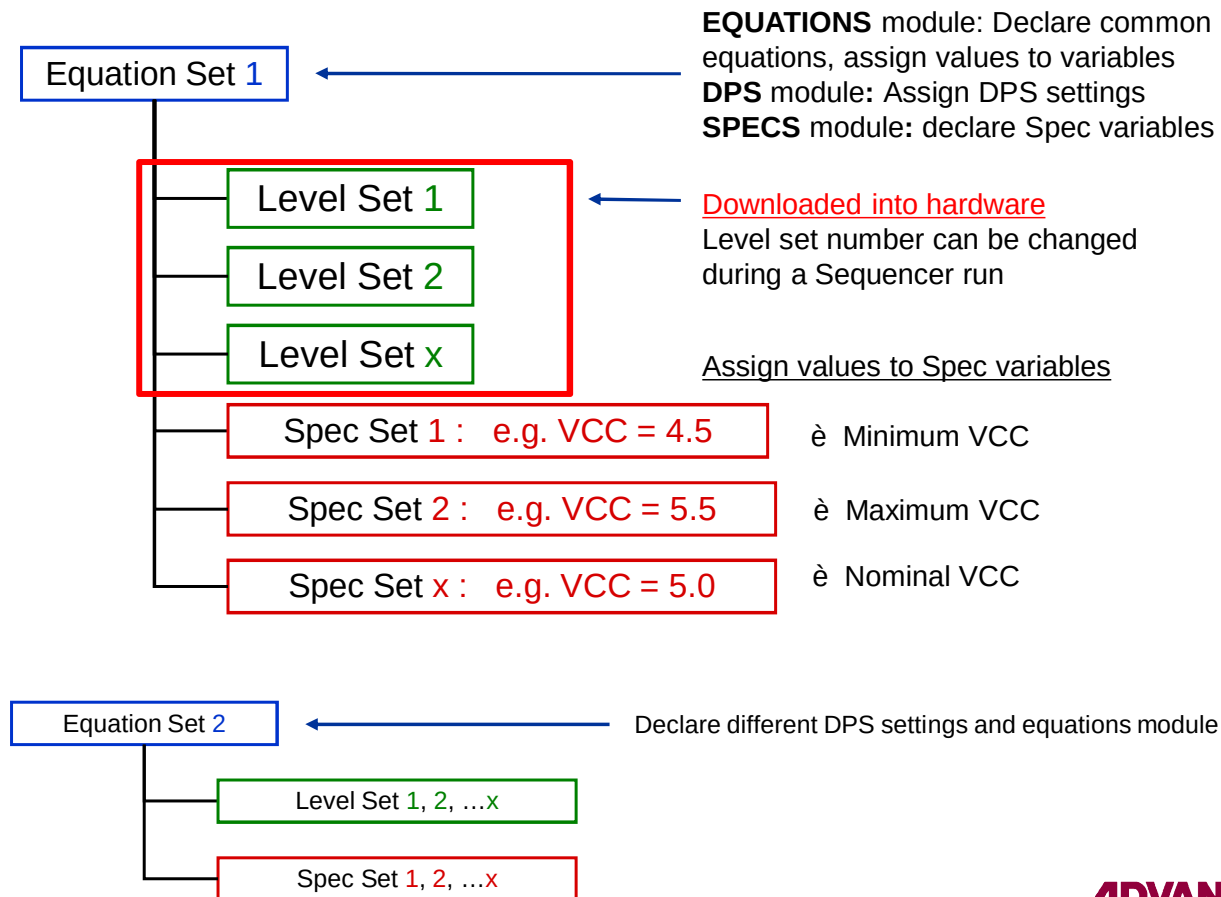
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 12

Level Equations Hierarchy



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 13

Equation-, Level- & Spec-sets

Level Equation Editor

Level Spec Tool

Several Equation Sets can exist

Spec Variables are initiated

Fixed variables or Equations

Power Supply pin settings

Several Levelsets for digital pins within EQNSET can exist

Possible to use pin-group names

```

/var/opt/hp93000/soc/tmp/93k.mfwH26/edit_lvleqn
File Edit Search Preferences Shell Macro Windows
/var/opt/hp93000/soc/tmp/93k.mfwH26/edit_lvleqn byte 343 of 434

1 EQNSET 2 "specs"
2
3
4 SPECS
5 VCC [V] #supply voltage
6 VIL [V] #low level input voltage
7 VIH [V] #high level input voltage
8
9
10 EQUATIONS
11 VOL = 2.2 #low level output voltage
12 VOH = VCC*0.48 #high level output voltage
13
14
15 DPSPINS Vcc
16 vout = VCC
17 ilimit = 1000
18 t_ms = 4
19 offcurr = act
20
21
22 LEVELSET 1 "no termination"
23
24 PINS ctrl ser_in
25 vil = VIL
26 vih = VIH
27
28 PINS ser_out
29
30 vol = VOL
31 voh = VOH
32
33 PINS io_pins
34
35 vil = VIL
36 vih = VIH
37 vol = VOL
38 voh = VOH
39
    
```

SpecTool

File Select Change Edit Sort Display Doc

Levels

Specification 2 1 spec_name_VCC=4.5

Equation 2 specs

Name	Actual	Minimum	Maximum	Measured	Unit
VCC	4.5				V
VIL	0				V
VIH	4.25				V

Values are assigned in the Spec tool



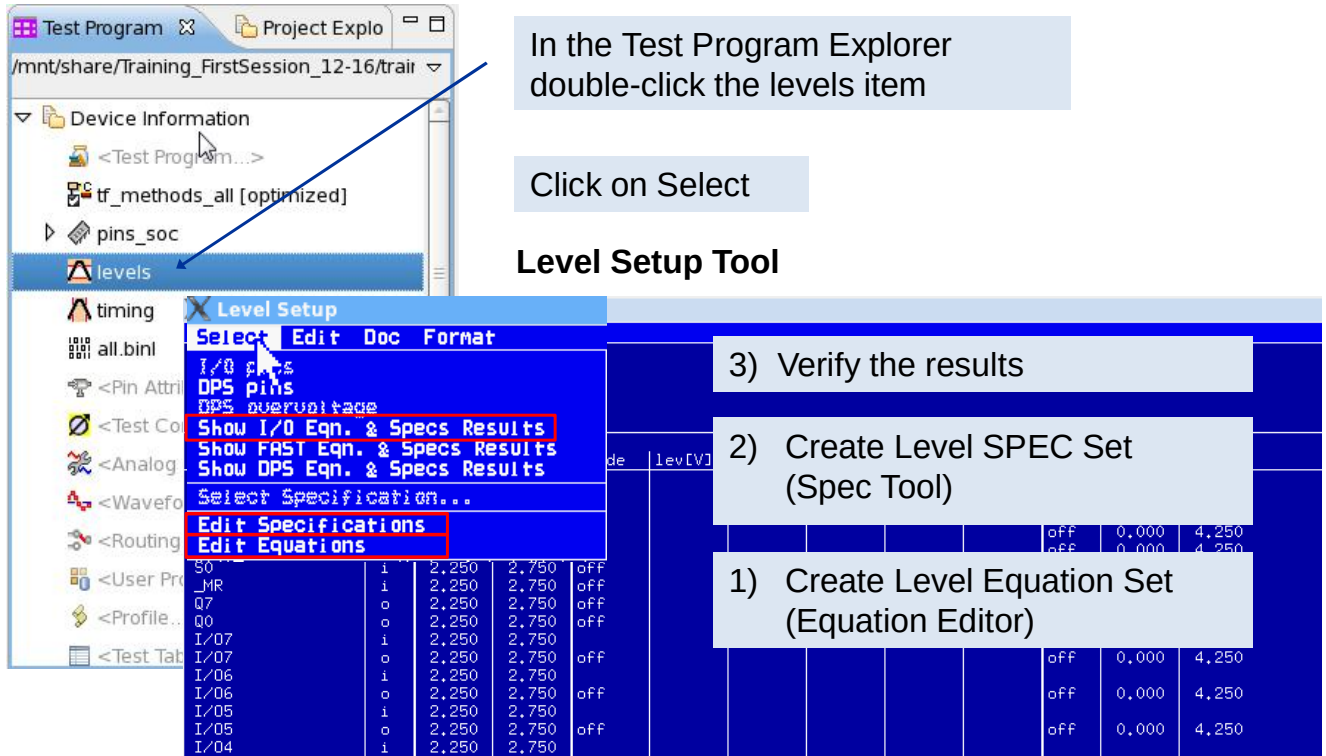
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 14

Equation based Levels Setup Steps

**ADVANTEST®**

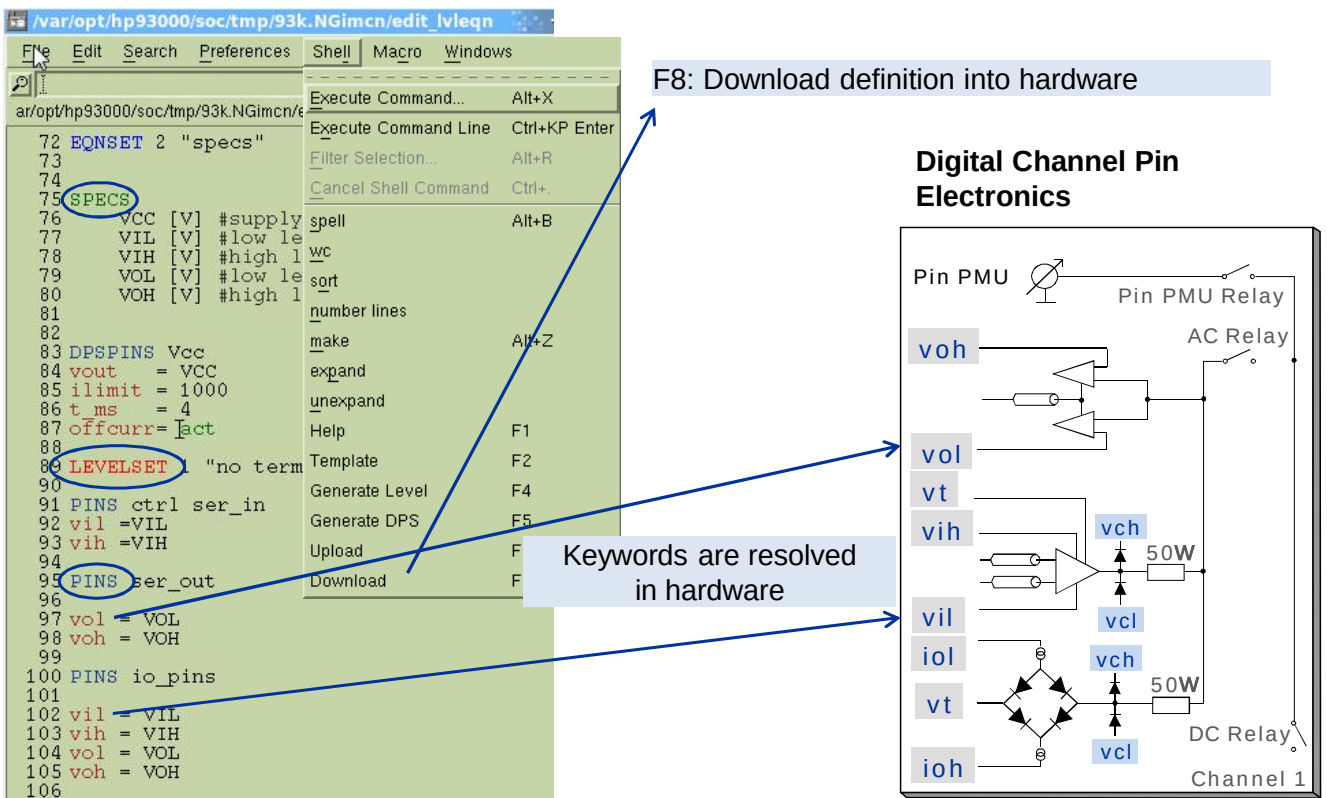
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 15

Step 1: Edit Equation Sets

Level Equation Editor

**ADVANTEST®**

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 16

Equation Sets

Without Specs
Without Equations module

EQNSET 1 "Without specs & equa."

DPSPINS Vcc
vout = 5.0 # in V

LEVELSET 1 "The only levelset"

PINS CP ser_in mode IMR
vil = 0
vih = 4.8

PINS Q0 Q7
vol = 2.0
voh = 3.0

PINS io_pins
vil = 0
vih = 4.8
vol = 2.0
voh = 3.0
vcl = 0
vch = 4.8

Without Specs
With Equations module

EQNSET 2 "Without specs"
EQUATIONS

vcc_level = 5.0
vil_level = 0
vih_level = vcc_level * 0.9
vol_level = 2.0
voh_level = 3.0

DPSPINS Vcc
vout = vcc_level
LEVELSET 1 "The only levelset"

PINS CP ser_in mode IMR
vil = vil_level
vih = vih_level
PINS Q0 Q7
vol = vol_level
voh = voh_level
PINS io_pins
vil = vil_level
vih = vih_level
vol = vol_level
voh = voh_level
vcl = vil_level
vch = vih_level

With Specs
With Equations module

EQNSET 3 "specs and equations"
SPECS

vcc_level [V]
vol_level [V]
voh_level [V]

EQUATIONS

vil_level = 0
vih_level = vcc_level * 0.9

DPSPINS Vcc

vout = vcc_level

LEVELSET 1 "The only levelset"

PINS CP ser_in mode IMR

vil = vil_level
vih = vih_level

PINS Q0 Q7

vol = vol_level
voh = voh_level

PINS io_pins

vil = vil_level
vih = vih_level
vol = vol_level
voh = voh_level
vcl = vil_level
vch = vih_level

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

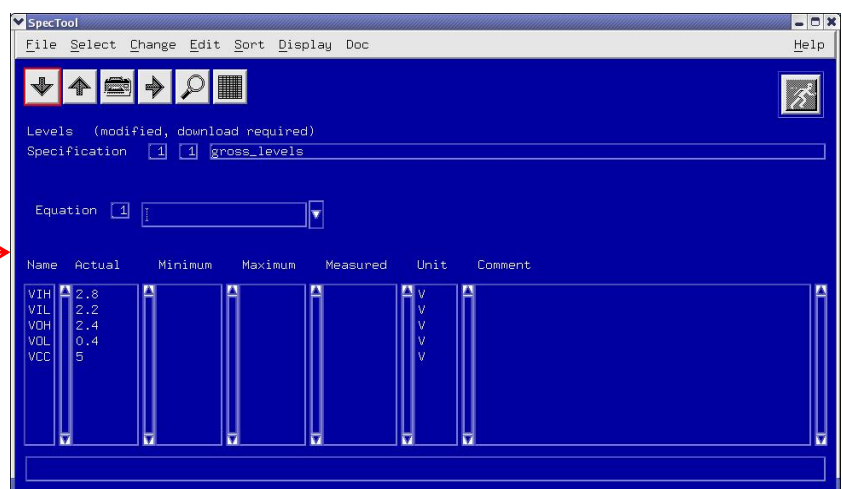
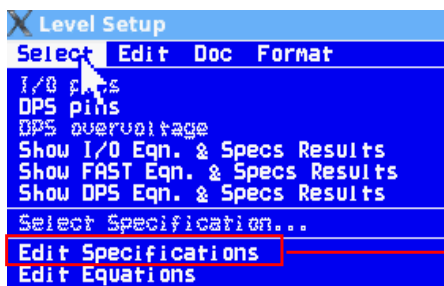
Levels Unit 5 - 17

Step 2: Create a Specification

Level Setup Tool

Level Spec Tool

Click "Select" > "Edit Specifications"



ADVANTEST

April 2016

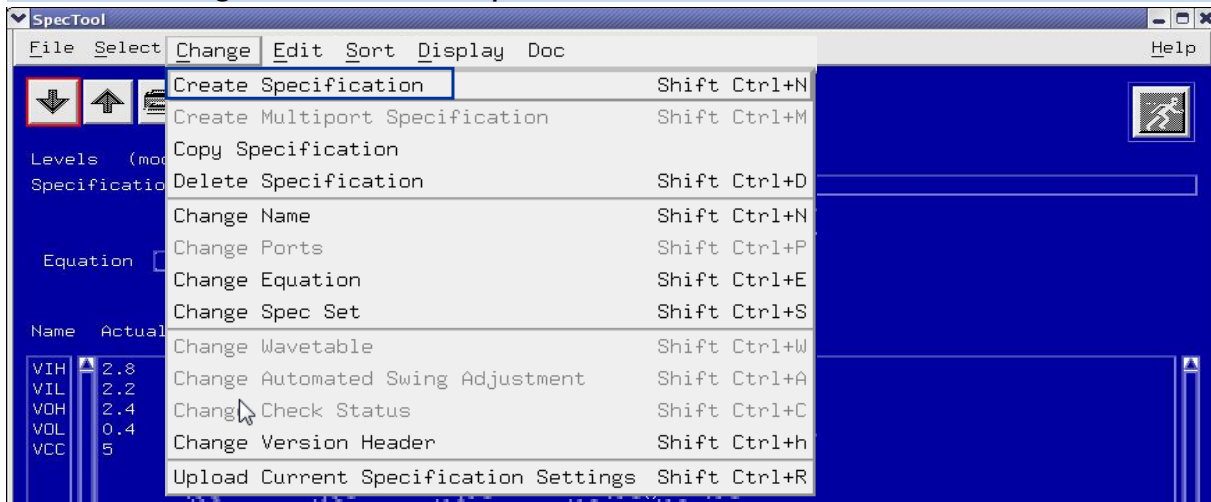
All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 18

Creating a Specification

Level Spec Tool

Click "Change" => "Create Specification"



Create a LEVEL SPECIFICATION

1. Select the EQUATION SET for which you want to define a SPEC SET
2. Specify spec values for the specs of the SPEC SET

ADVANTEST

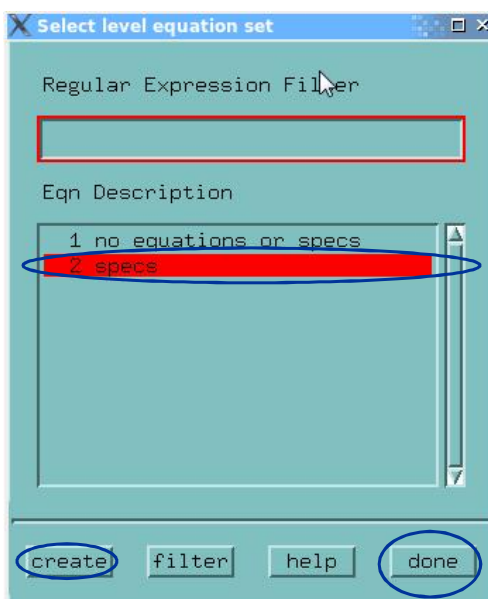
April 2016

All Rights Reserved - ADVANTEST CORPORATION

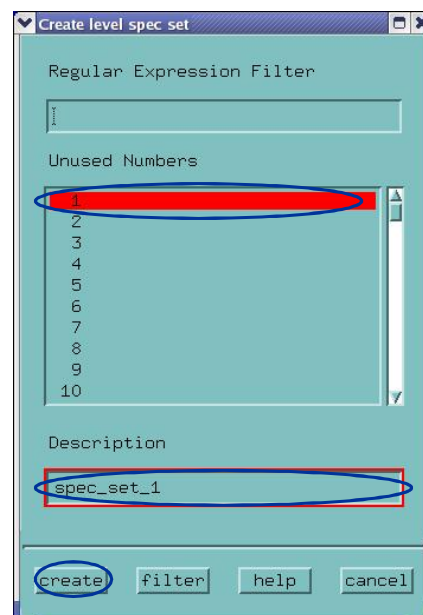
Levels Unit 5 - 19

Selecting Equation and Spec Set Number

1. Select the Equation Set, click **create**



2. a. Select the Spec Set number
b. Enter the Specification Description
c. Click **create**



3. Click **done** to close the equation set window

ADVANTEST

April 2016

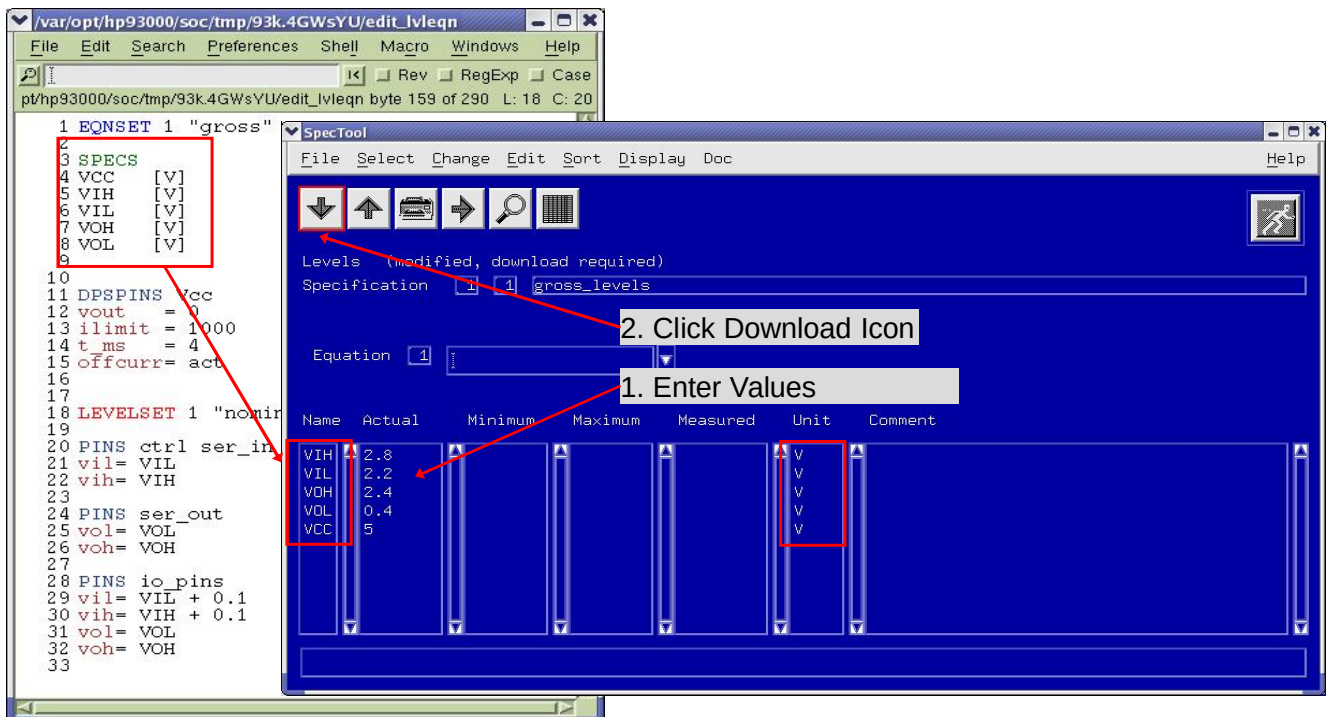
All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 20

Assigning Spec Values in SpecTool

Level Equation Editor

Level Spec Tool



ADVANTEST®

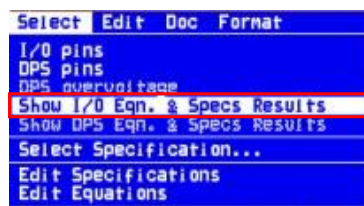
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 21

Step 3: Verification of Level Setups

Level Setup Tool



Spec Set

Level Set

Equation Set

pin/group name	pin type	level[V]	low	high	mode	lev[V]	Iol[mA]	Ioh[mA]	swing	offset	mode	low[V]	high[V]
CP	i	0.200	4.000	off							auto	-0.600	4.800
DS7	i	0.200	4.000	off							auto	-0.600	4.800
DS0	i	0.200	4.000	off							auto	-0.600	4.800
S1	i	0.200	4.000	off							auto	-0.600	4.800
S0	i	0.200	4.000	off							auto	-0.600	4.800
_MR	i	0.200	4.000	off							auto	-0.600	4.800
_Q7	o	0.400	2.400	off							auto	-1.300	4.100
Q0	o	0.400	2.400	off							auto	-1.300	4.100
I/O7	i	0.300	4.100										
I/O7	o	0.400	2.400	off							auto	-1.300	4.100
I/O6	i	0.300	4.100										
I/O6	o	0.400	2.400	off							auto	-1.300	4.100
I/O5	i	0.300	4.100										
I/O5	o	0.400	2.400	off							auto	-1.300	4.100
I/O4	i	0.300	4.100										

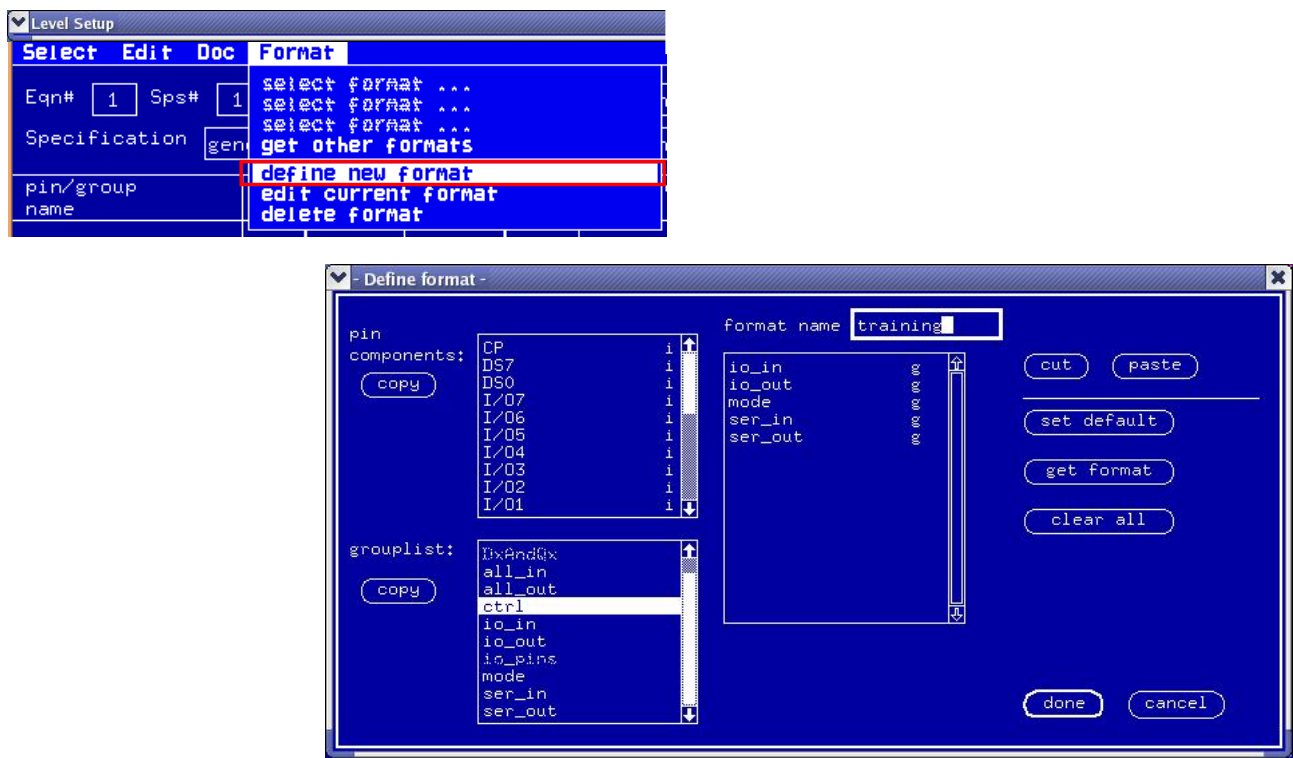
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 22

Using Formats



Overwriting Equation-based Levels

After changing a value in this section, the header appears in a different color (black). To keep changes, go to setup tools.

Level Setup															
Select Edit Doc Format															
Eqn#		Sps#		Lset#		of 3		Level		with termination					
Specification		Vcc = 4.2V		Equation		specs		EDITED		Diff		Fast		Std	
pin/group name	pin type	level [V/mA]		mode	lev/v3h	Iol [mA]	Ioh [mA]	swing	offset	pulldown	H R mode	mode	clamp		range [V/mA]
		low	high										low [V]	high [V]	
CP	i	0.000	4.000	off						none		auto	-2.000	7.000	
DS7	i	0.000	4.000	off						none		auto	-2.000	7.000	
DS0	i	0.000	4.000	off						none		auto	-2.000	7.000	
S1	i	0.000	4.000	off						none		auto	-2.000	7.000	
S0	i	0.000	4.000	off						none		auto	-2.000	7.000	
_MR	i	0.000	4.000	off						none		auto	-2.000	7.000	
Q7	o	2.400	2.500	term	2.500							off	-2.000	7.000	
Q0	o	2.400	2.600	term	2.500							off			
I/07	i	0.000	4.000												
I/07	o	2.400	2.600	term	2.500							off			
I/06	i	0.000	4.000												
I/06	o	2.400	2.600	term	2.500							off			
I/05	i	0.000	4.000												
I/05	o	2.400	2.600	term	2.500							off			
I/04	i	0.000	4.000												

Going back:
Edit à Restore Specification



Reference

V93000 Technical Documentation Center (TDC):
<https://www.advantest.com/V93000/help/index.jsp>
[Topic 17694](#)

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 25

74ACT299 Levels: Reading from Datasheet

DC Electrical Characteristics for ACT

Symbol	Parameter	V _{CC} (V)	T _A = 25°C		T _A = -40°C to +85°C	Units	Conditions
			Typ	Guaranteed Limits			
V _{IH}	Minimum HIGH Level Input Voltage	4.5	1.5	2.0	2.0	V	V _{OUT} = 0.1V or V _{CC} - 0.1V
		5.5	1.5	2.0	2.0		
V _{IL}	Maximum LOW Level Input Voltage	3.0	1.5	0.8	0.8	V	V _{OUT} = 0.1V or V _{CC} - 0.1V
		4.5	1.5	0.8	0.8		
V _{OH}	Minimum HIGH Level	4.5	4.49	4.4	4.4	V	I _{OUT} = -50 μA
		5.5	5.49	5.4	5.4		
		4.5	0.0001	3.86	3.76	V	V _{IN} = V _{IL} or V _{IH} I _{OH} = -24 mA I _{OH} = -24 mA (Note 5)
		5.5		4.86	4.76		
V _{OL}	Maximum LOW Level Output Voltage	4.5	0.001	0.1	0.1	V	I _{OUT} = 50 μA
		5.5	0.001	0.1	0.1		
		4.5		0.36	0.44	V	V _{IN} = V _{IL} or V _{IH} I _{OL} = 24 mA I _{OL} = 24 mA (Note 5)
		5.5		0.36	0.44		
I _{IN}	Maximum Input Leakage Current	5.5		± 0.1	± 1.0	μA	V _I = V _{CC} , GND
I _{CCT}	Maximum I _{CC} /Input	5.5	0.6		1.5	mA	V _I = V _{CC} - 2.1V
I _{OLD}	Minimum Dynamic Output Current (Note 6)	5.5			75	mA	V _{OLD} = 1.65V Max
I _{OHD}		5.5			-75	mA	V _{OHD} = 3.85V Min
I _{CC}	Maximum Quiescent Supply Current	5.5		4.0	40.0	μA	V _{IN} = V _{CC} or GND
I _{OZT}	Maximum I/O Leakage Current	5.5		±0.3	±3.0	μA	V _I (OE) = V _{IL} , V _{IH} V _I = V _{CC} , GND V _O = V _{CC} , GND

Note 5: All outputs loaded; thresholds on input associated with output under test.

Note 6: Maximum test duration 2.0 ms, one output loaded at a time.

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Levels Unit 5 - 26

Lab Exercise

Levels

Advantest V93000

Digital User Training

Timing

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- V93000 approach to timing
- Equation Based Timing
- Wavetables
- Equations
- Specifications
- Timing Verification

ADVANTEST[®]

Learning Objectives

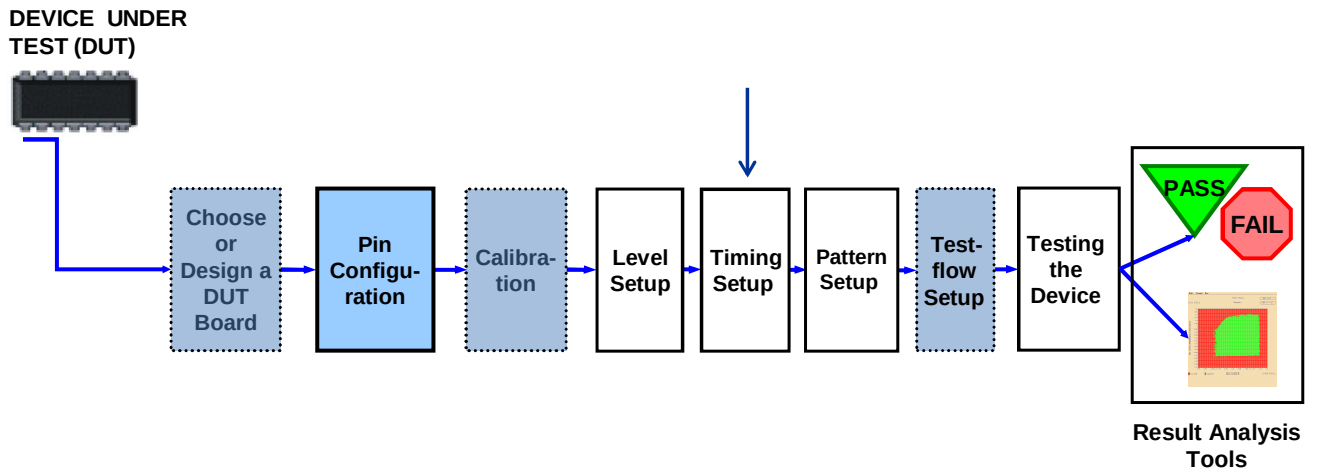
- Know the resources available to perform a timing setup
- Know the different tools involved in the timing setup
- Know the concept of the equation based setup

Relevance

- The timing is a key differentiator between the V93000 and other ATE systems. One of the milestones of this training is to understand the timing.

Test Program Flow Chart

Timing Setup



ADVANTEST®

April 2016

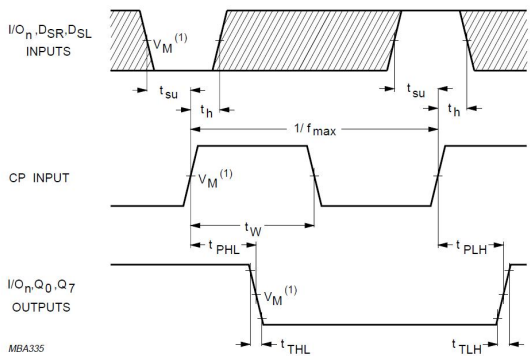
All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 5

Timing Setup Purpose

Device requirements

→ V93000 timing realization

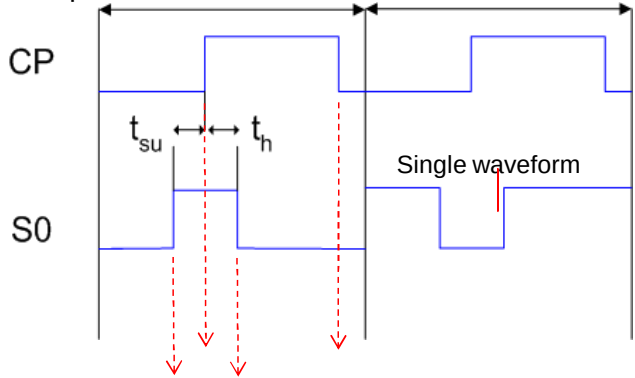


AC CHARACTERISTICS FOR 74HCT

GND = 0 V; $t_r = t_f = 6$ ns; $C_L = 50$ pF

SYMBOL	PARAMETER	T _{amb} (°C)						UNIT	
		74HCT							
		-25			-40 to +85		-40 to +125		
		min.	typ.	max.	min.	max.	min.		max.
t _{PHL}	propagation delay CP to I/O _n		27	46		58		69	ns
t _{THL} /t _{TLH}	output transition time standard (Q ₀ , Q ₇)		7	15		19		22	ns
t _W	clock pulse width HIGH or LOW	20	10		25		30		ns
t _{su}	set-up time S ₀ , S ₁ to CP	32	18		40		48		ns
t _h	hold time S ₀ , S ₁ to CP	0	-17		0		0		ns
f _{max}	maximum clock pulse frequency	25	42		20		17		MHz

1. Tester period



2. Position of edges defined through equation set

3. Sequence of actions concatenated to a waveform

4. All waveforms are assembled in the wave-table

5. Sequence of waveform is controlled by vector memory

Resources are available per pin

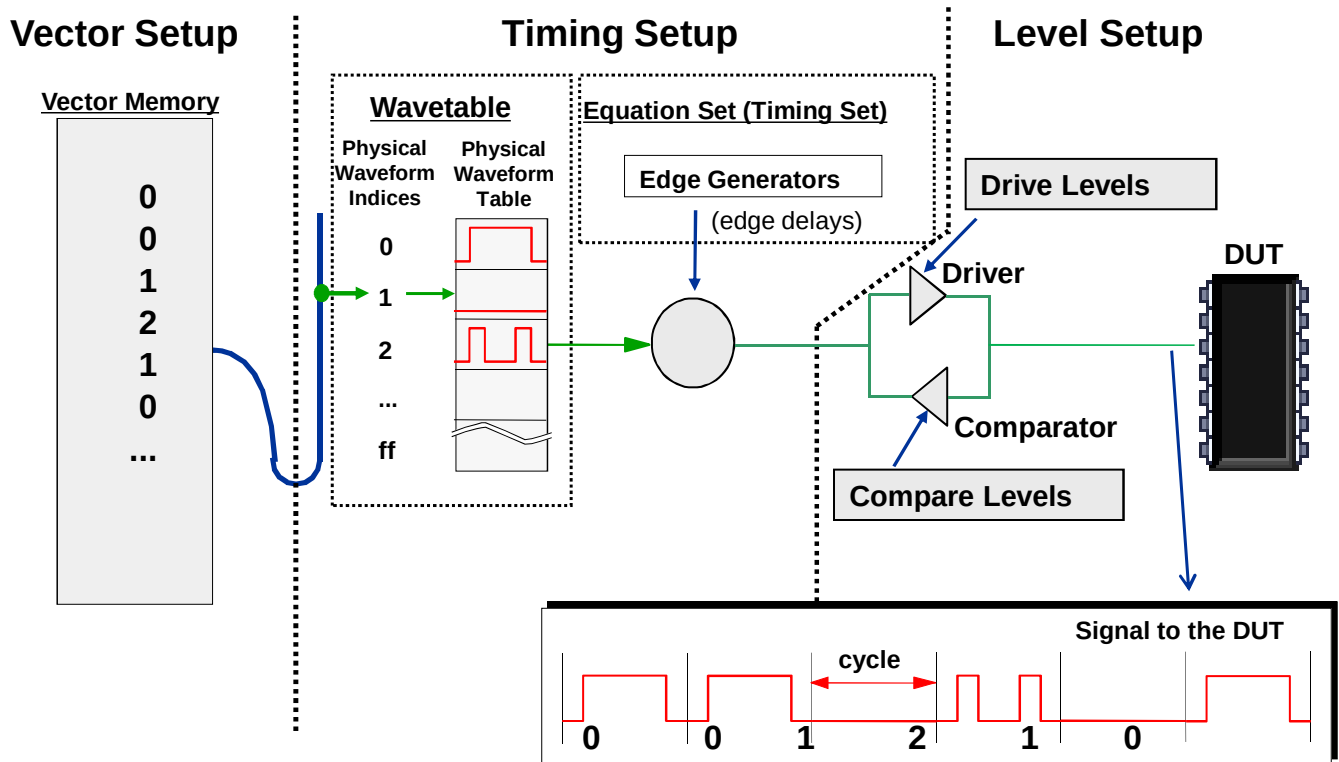
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 6

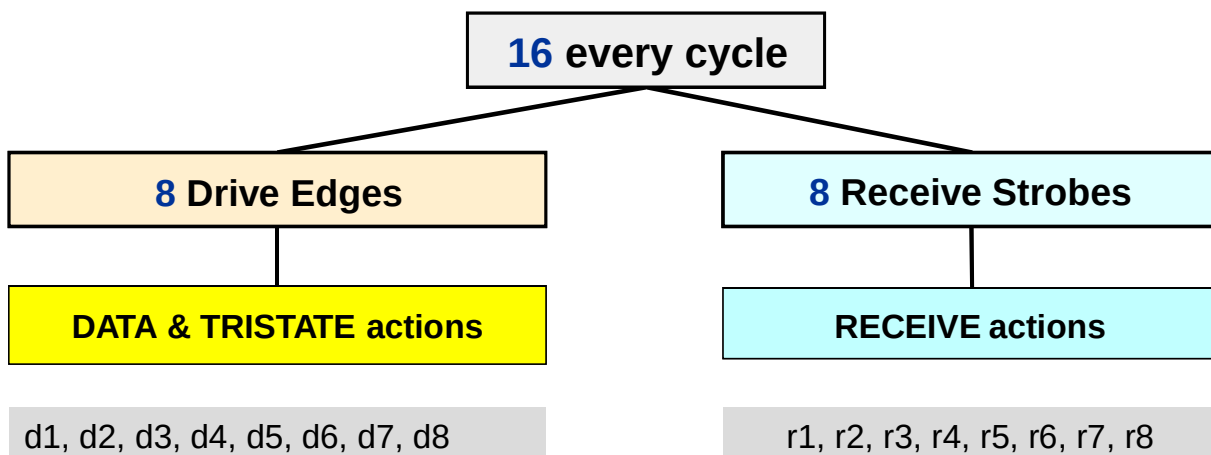
Linking V93000 Setups



ADVANTEST

V93000 Timing Resources: Edges and Strobes

- Edges and Strobes are used to position waveform transition in horizontal direction, that is the delay vs. start of tester cycle

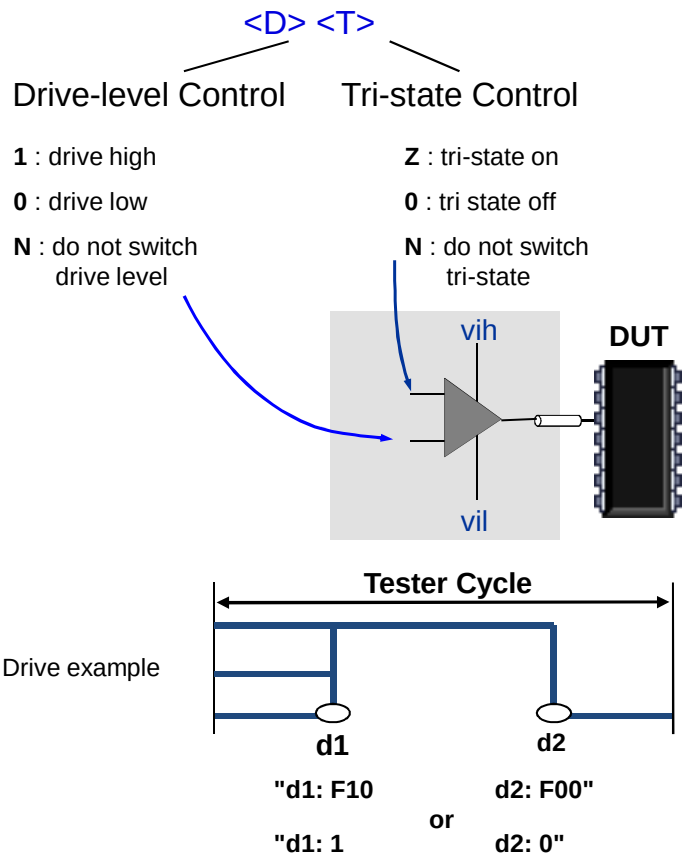


ADVANTEST

Drive Syntax

List of Drive Actions

FNN		
F0N		
F1N		
FN0	or	!Z
FNZ	or	Z
F00	or	0
F0Z		
F10	or	1
F1Z		



ADVANTEST®

Receive Syntax

Edge Compare

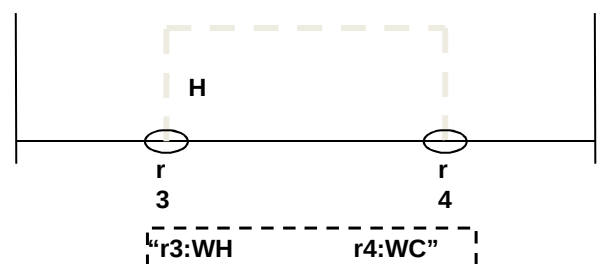
L	compare to low
H	compare to high
M	compare to intermediate
X	don't care (mask)

Receive Examples



Window Compare

WL	compare to low
WH	compare to high
WM	compare to intermediate
WX	don't care (mask)
WU	compare to unstable
WC	close Window



ADVANTEST®

Wavetable Introduction

WAVETBL gross_func_wtb

Wavetable name

PINS CP

0 "d1: 0 d2: 0"

1 "d1: 1 d2: 0"

Single Waveform

Physical Waveform indices (0x00 – 0xFF)

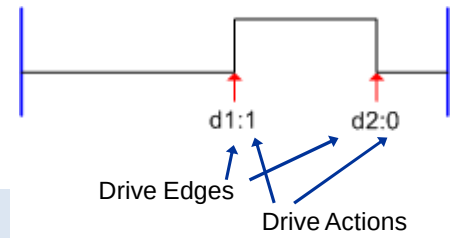
PINS ser_out

0 "r1: L"

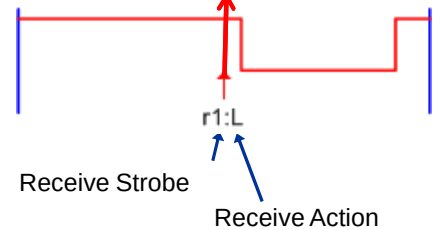
1 "r1: H"

2 "r1: X"

Input Waveform



Output signal / Fail



ADVANTEST®

Equations Introduction

EQNSET 1 "gross_func_eqn"

Equation set

TIMINGSET 1 "20 MHz"

Timing set

period = 50

PINS CP

d1 = 25

d2 = 40

Drive edge number and edge delay

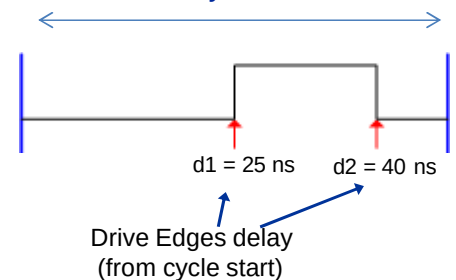
PINS ser_out

r1 = 25

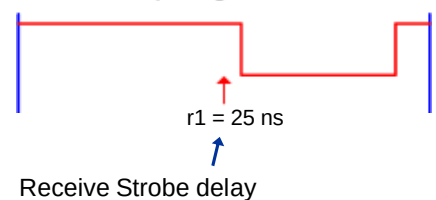
Receive edge number and edge delay

Input signal

Tester cycle: 50 ns



Output signal / Fail



ADVANTEST®

Edge Positions Defined through Equation-, Timing- & Spec-Sets

Timing Equation Editor

Several EQNSETS can exist

SPEC-Variables are initiated

Equation variables are assigned for better readability

Timing set initial

Tester-Period defined

Position of drive-edges (d1 – d8) defined per pin or pin-group

```
EQNSET 3 "spec"

SPECS

f [MHz]
tw_CP [ns]

EQUATIONS
p = 1/f*1000
CP_ref = p/2

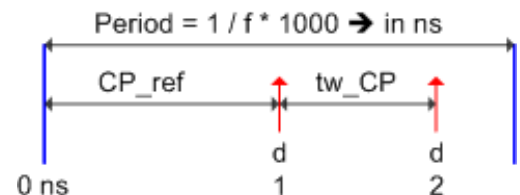
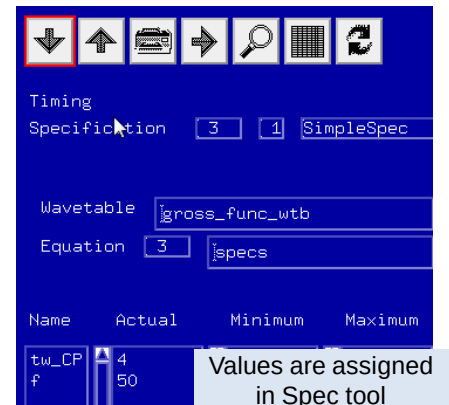
TIMINGSET 1 "std"

period = p

PINS CP
d1 = CP_ref
d2 = CP_ref + tw_CP
```

Final Timing at DUT-input

Timing Spec Tool



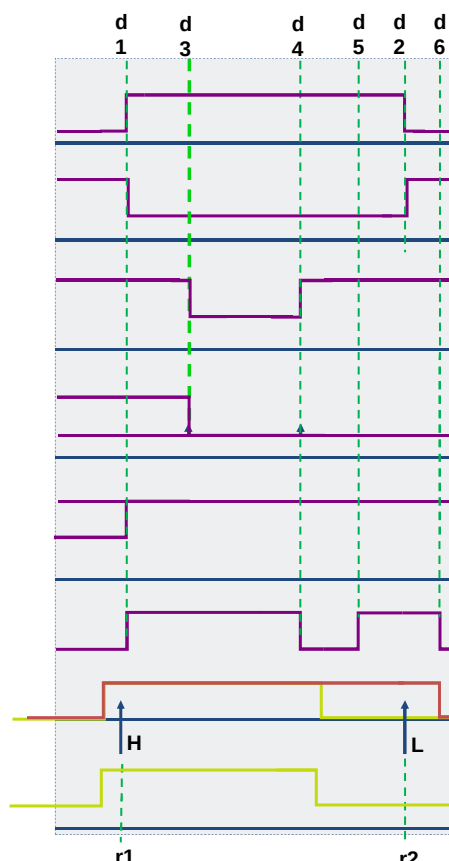
ADVANTEST

More Complex Wavetable

(0x00 - 0xFF ð 256 Waveforms)

Physical
Waveform
Indices

0	d1:1	d2:0
1	d1:0	d2:1
2	d3:0	d4:1
3	d3:0	d4:0
4	d1:1	
5	d1:1, d4:0, d5:1, d6:0	
6	r1:H	r2:L
7	r1:WU	r2:WC



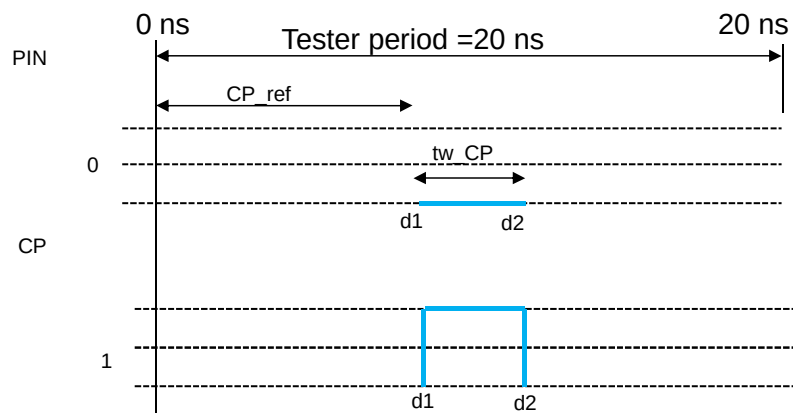
Input signals

Output fail

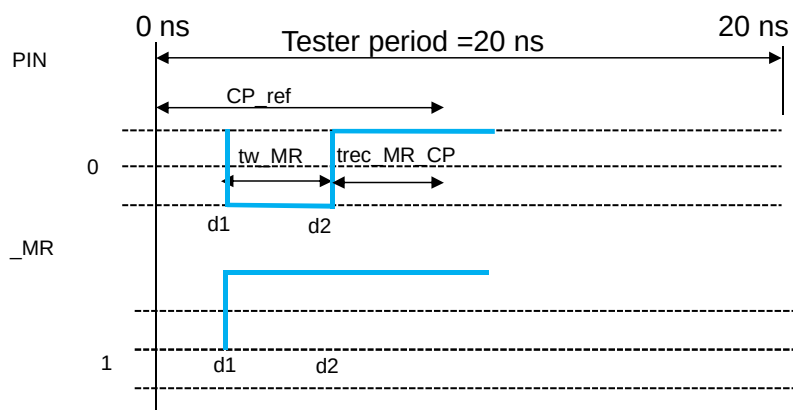
Output pass

ADVANTEST

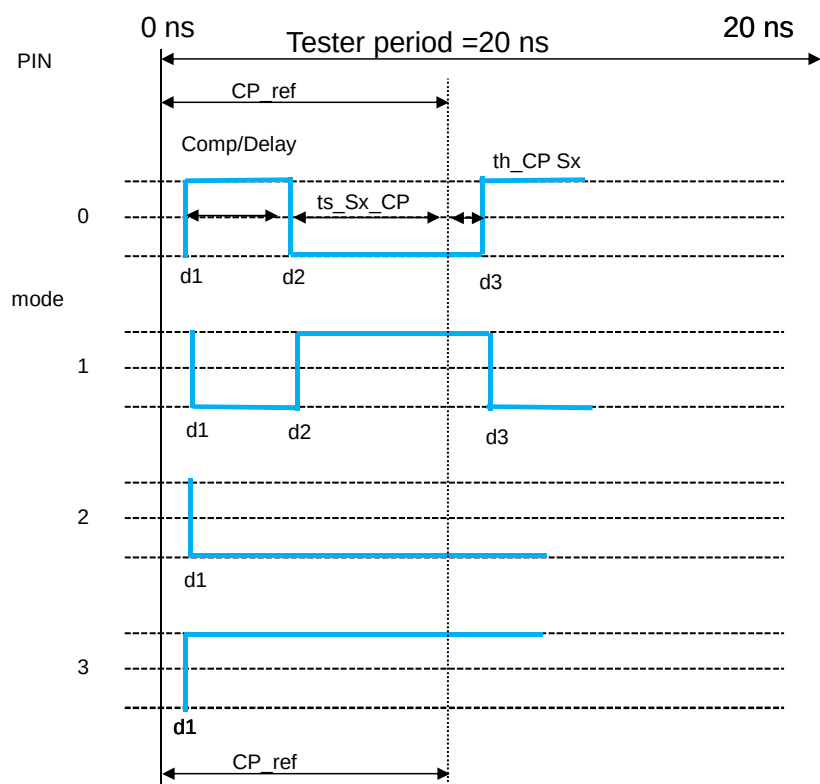
Physical waveform indices



Physical waveform indices

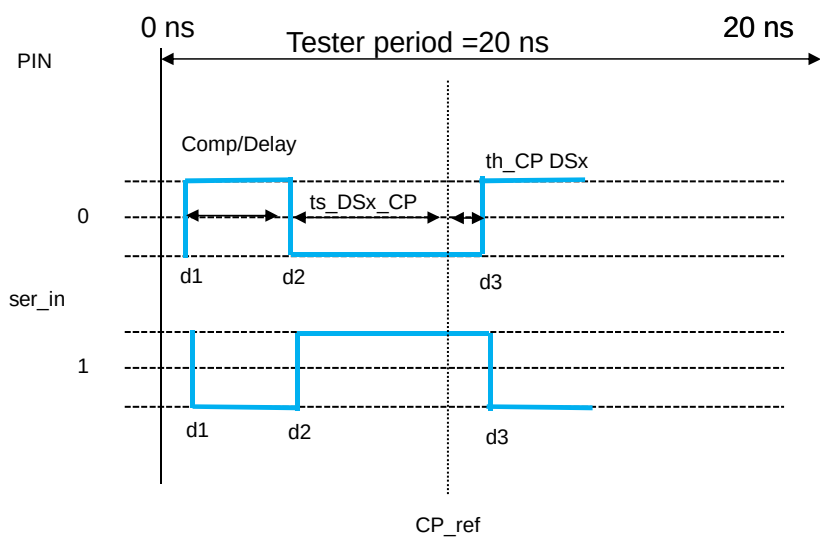


Physical waveform indices



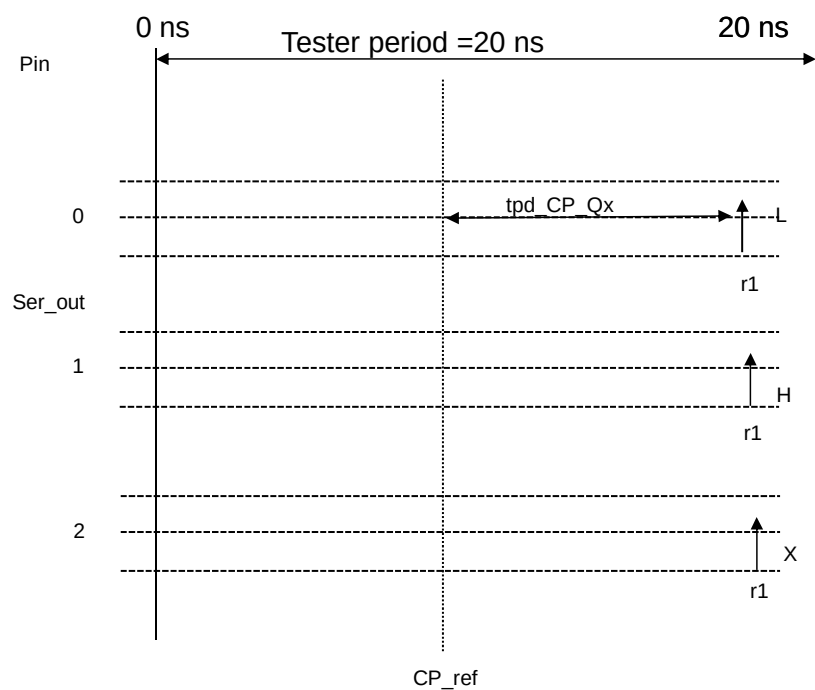
ADVANTEST

Physical waveform indices



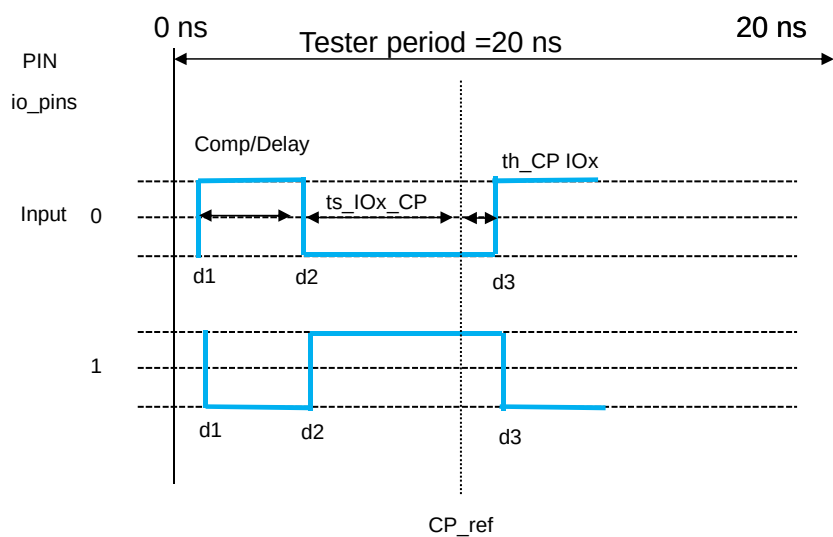
ADVANTEST

Physical waveform indices



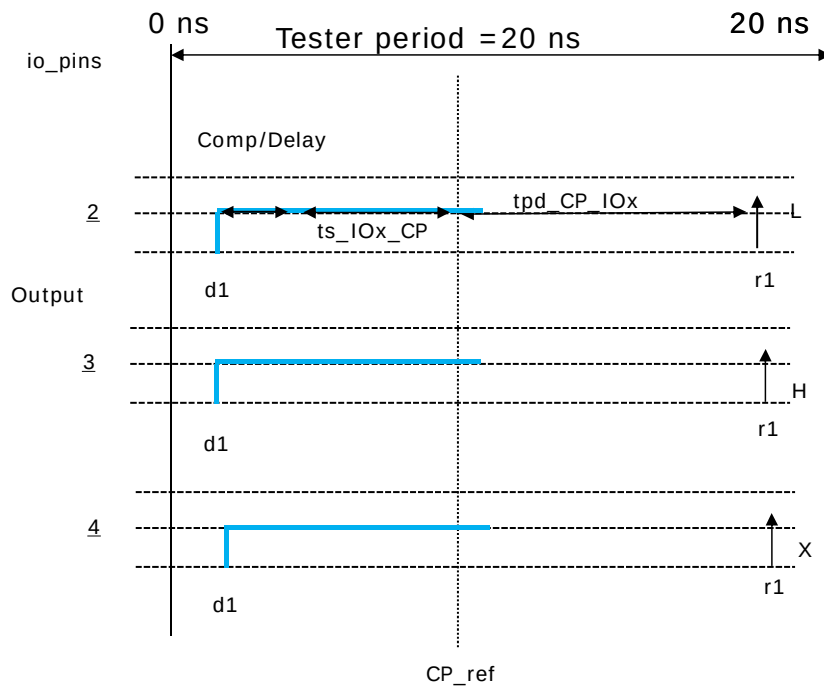
ADVANTEST

Physical waveform indices



ADVANTEST

Physical waveform indices



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 21

V93000 Approach to Timing

Device Requirements	Action (vertical)	Physical Wave Table Shape (horizontal)	Pot. Spec variables # of Edges / Strobes
CP	RZ	Drive 1 / Drive 0	2
io_pins (in)	SBC	Drive 1 / Drive 0 / Drive 1 Drive 0 / Drive 1 / Drive 0	3
io_pins (out)	Edge Compare	Strobe 1 Strobe 0	1
input	DNRZ	Drive 1 Drive 0	1
output	Window Compare	Strobe 1 / close Strobe Strobe 0 / close Strobe	2

ADVANTEST

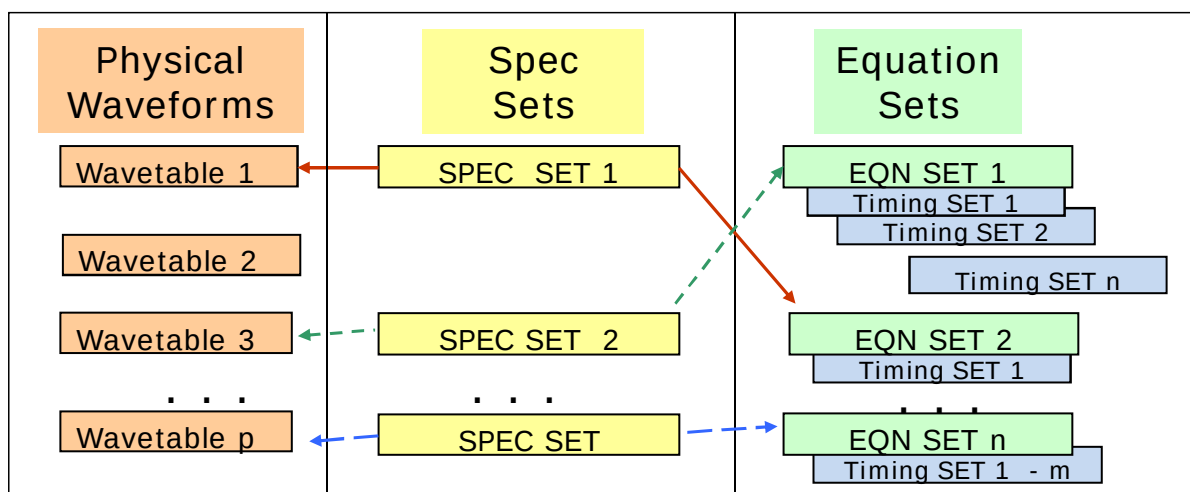
April 2016

All Rights Reserved - ADVANTEST CORPORATION

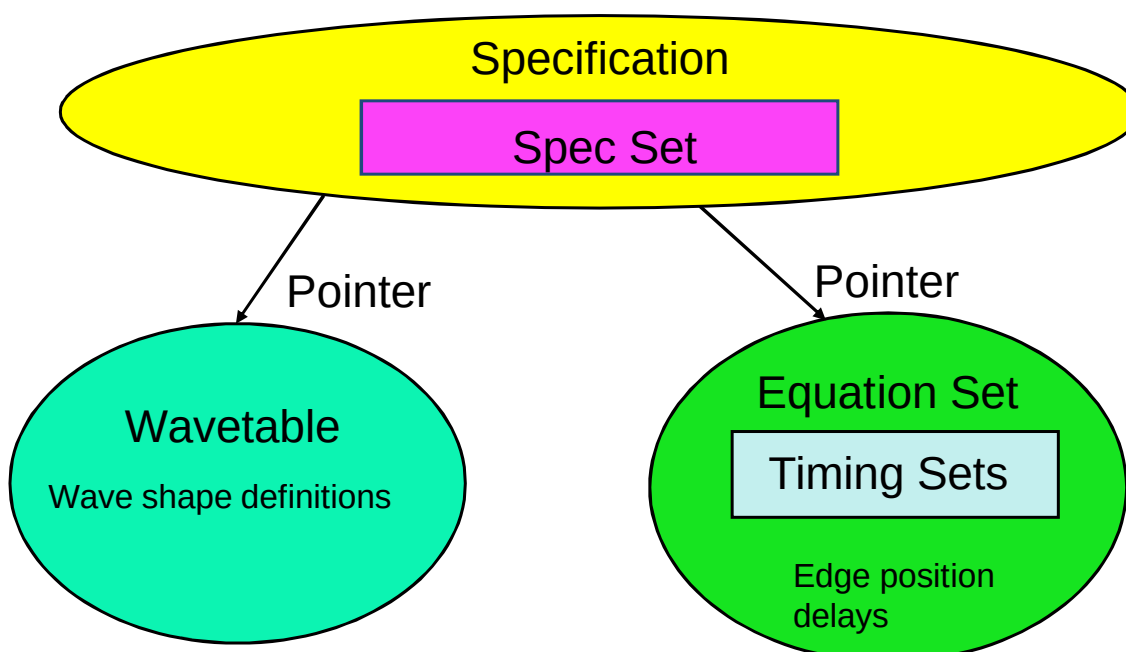
Timing Unit 6- 22

Assigning Wavetables to Equation Sets

- The specification links wavetable and equation set together and defines values for the spec variables



Specification Set



Equation based Timing Setup Steps



3) Create a SPEC SET
(Spec Tool)

2) Create an EQUATION
SET (Equation Editor)

1) Create a WAVETABLE
(Wavetable Editor)

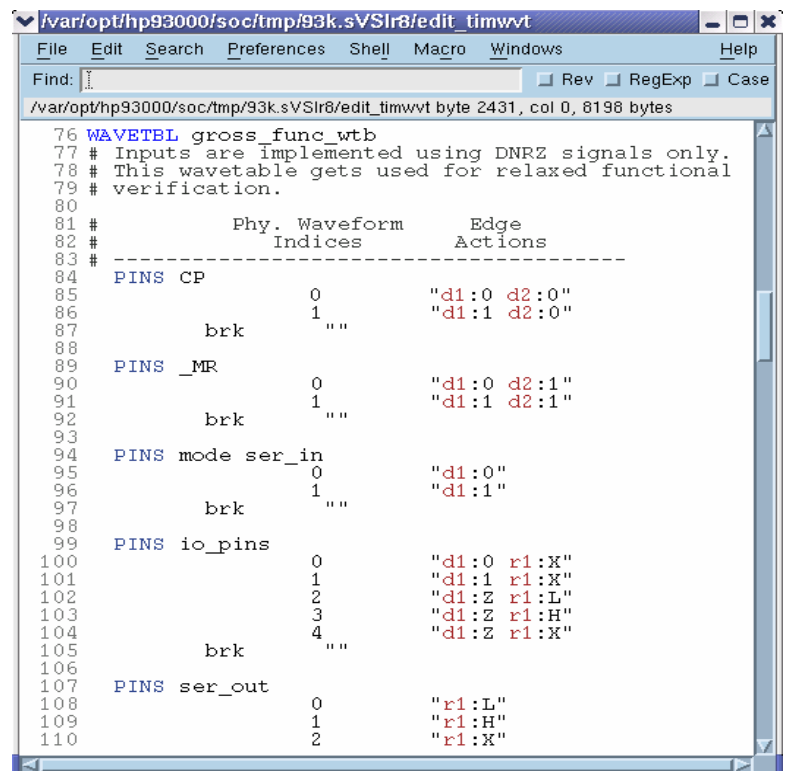
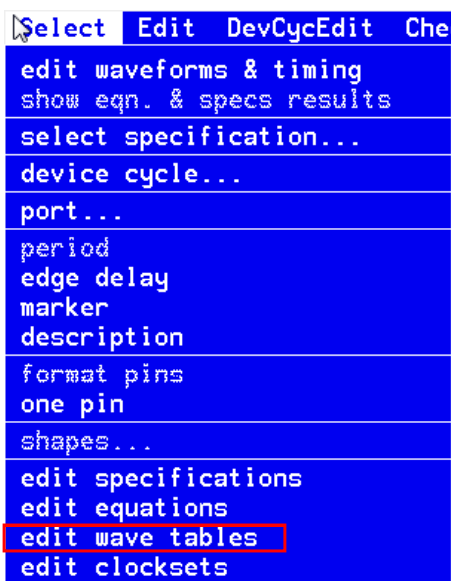
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 25

Step 1: Edit Wavetables



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 26

Dealing with IO pins

Driver set to high Z mode for IO pins

```
PINS io_pins
0      "d1:1 d2:0 d3:1"      0
1      "d1:0 d2:1 d3:0"      1
2      "d1:Z r1:L"           L
3      "d1:Z r1:H"           H
4      "d1:Z r1:X"           X
```

- i: tristate the driver or set it to high impedance
- O: perform the measurement

Break Waveform

The break waveform is used

- between testsuites
 - when changing level / timing sets
 - to fill data queues (advanced)
-
- If not defined, the default is *hold* (holds the previous state of drive and tri-state).

Why?

WAVETBL spec_search_wtb

PINS CP

```
0 "d1: 0 d2: 0"
1 "d1: 1 d2: 0"
brk "d1:0"
```

PINS _MR

```
0 "d1: 0 d2: 1"
1 "d1: 1 d2: 1"
brk "d1:1"
```

PINS mode

```
0 "d1: 0"
1 "d1: 1"
brk "d1:0"
```

Device Cycle Names (DVC)

WAVETBL gross_func_devcyc_wtb

PINS	CP		
0	"d1: 0 d2: 0"		0
1	"d1: 1 d2: 0"		1
PINS	mode ser_in		
0	"d1: 0"	0	
1	"d1: 1"	1	
PINS	ser_out		
0	"r1: L"	L	
1	"r1: H"	H	
2	"r1: X"	X	

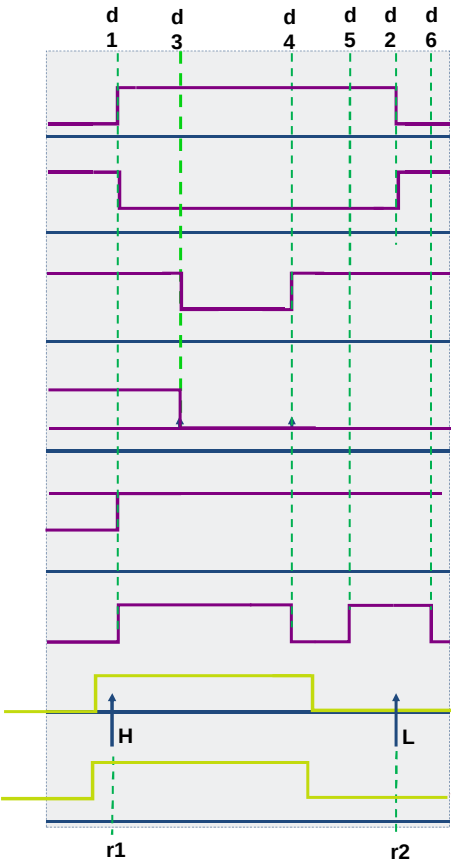
Device
Cycle
Names (DVC)

More Complex Wavetable

Physical Waveform

Indices
(PW)
↓

0	d1:1	d2:0
1	d1:0	d2:1
2	d3:0	d4:1
3	d3:0	d4:0
4	d1:1	
5	d1:1, d4:0, d5:1, d6:0	
6	r1:H	r2:L
7	r1:WX	r2:WC



Device
Cycle
Names (DVC)

CLK
0-1_1
0-1_2
0-0
1
1-0-1-0
H-L
WU

Defined by user,
can be any name

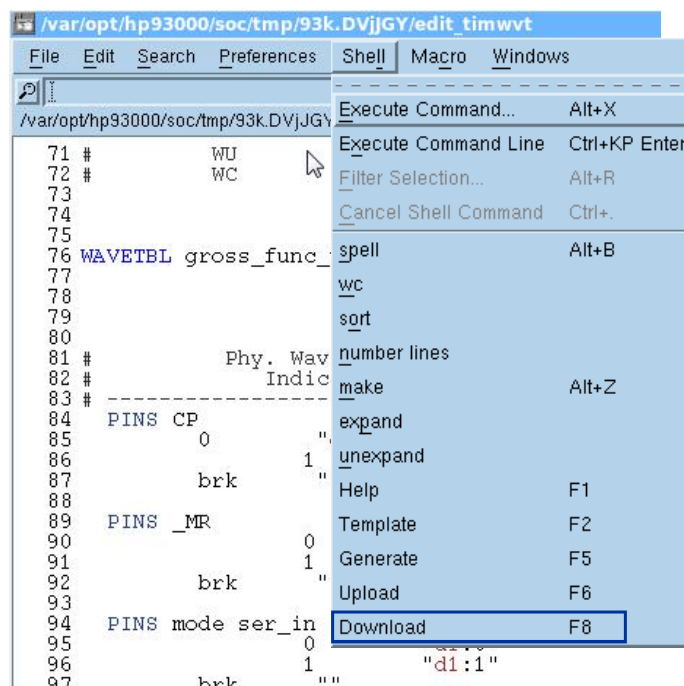
Download & Save Wavetable

To make changes available in hardware

- Open Shell
- Download

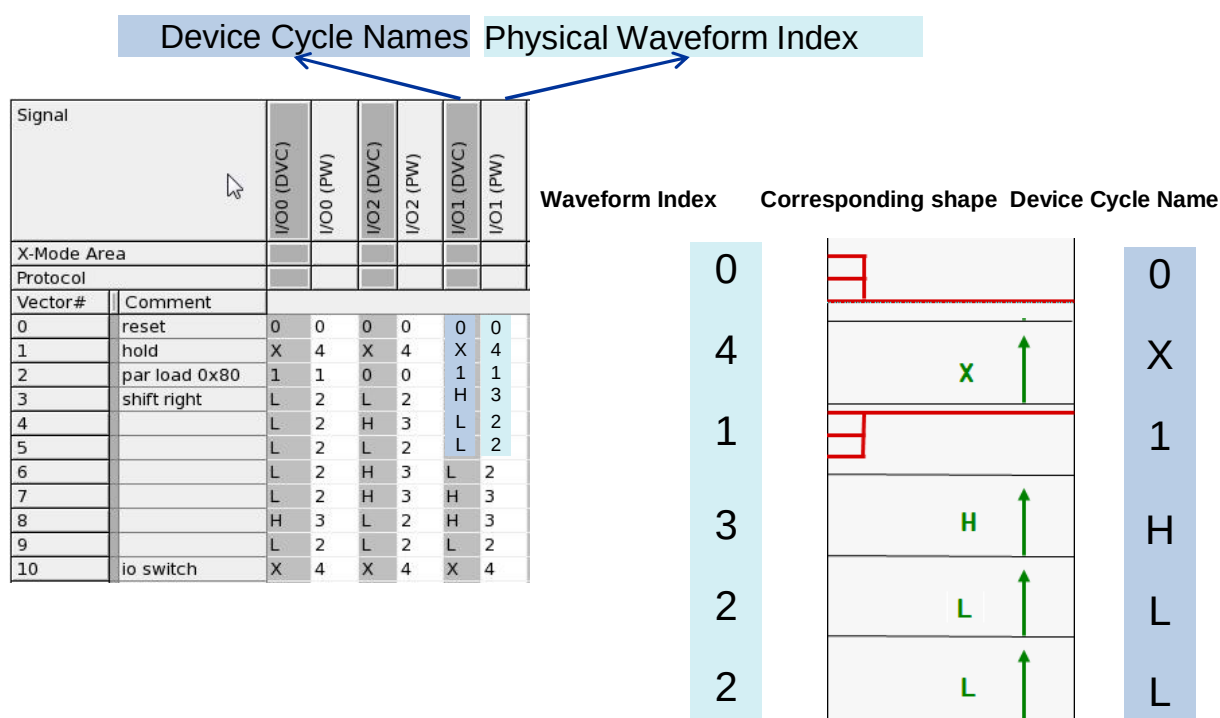
Or use Shortcut-key **"F8"**

Changes are from now on reflected in hardware.



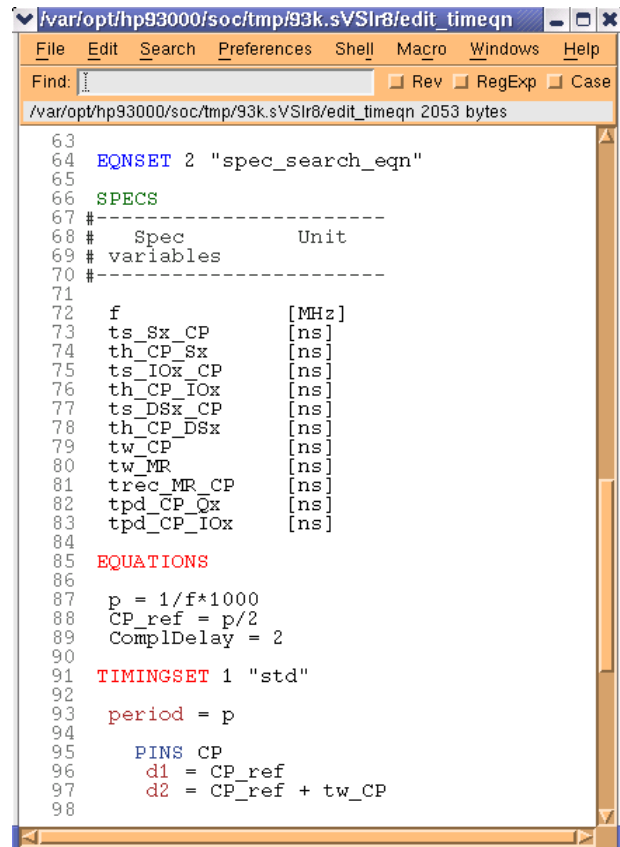
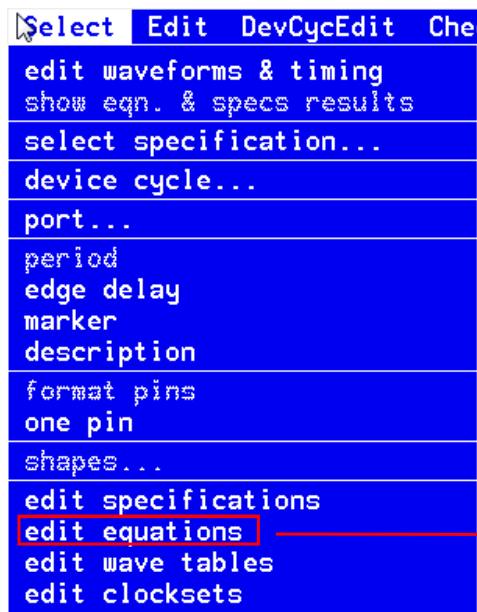
ADVANTEST

Verify Device Cycle Name in Pattern Debug Tool



ADVANTEST

Step 2: Edit Equations



ADVANTEST

Equation Set and Timing Set Syntax Using Fixed Values

EQNSET 1 "gross_func_eqn"

Equation Set Number & Name

TIMINGSET 1 "20 MHz"

Timing Set Number & Name

period = 50

PINS CP

d1 = 20

d2 = 45

PINS mode

d1 = 0

d2 = 30

PINS ser_out

r1 = 45

Drive Edge Number and
Edge Delay

Receive Edge Number and
Edge Delay

ADVANTEST

Specs & Equations in an Equation Set

EQNSET 2 "spec_search_eqn"

SPECS

f [MHz]
tw_CP [ns]
tpd_CP_Qx [ns]
...

EQUATIONS

p = $1/f * 1000$
CP_ref = $p/2$
ComplDelay = 2

TIMINGSET

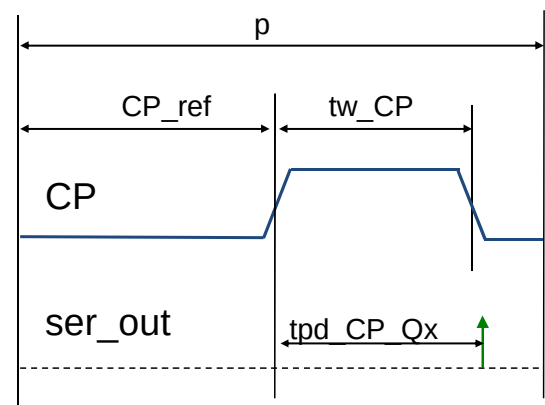
 1 "std"

period = p
PINS CP
d1 = CP_ref
d2 = CP_ref + tw_CP
PINS ser_out
r1 = CP_ref + tpd_CP_Qx

Initialize Spec Variables

Equation Variables:
Available in equation editor only

Resulting Waveform



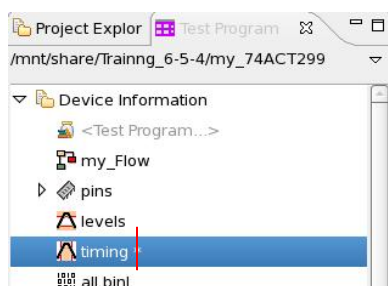
ADVANTEST

Download & Save Equations

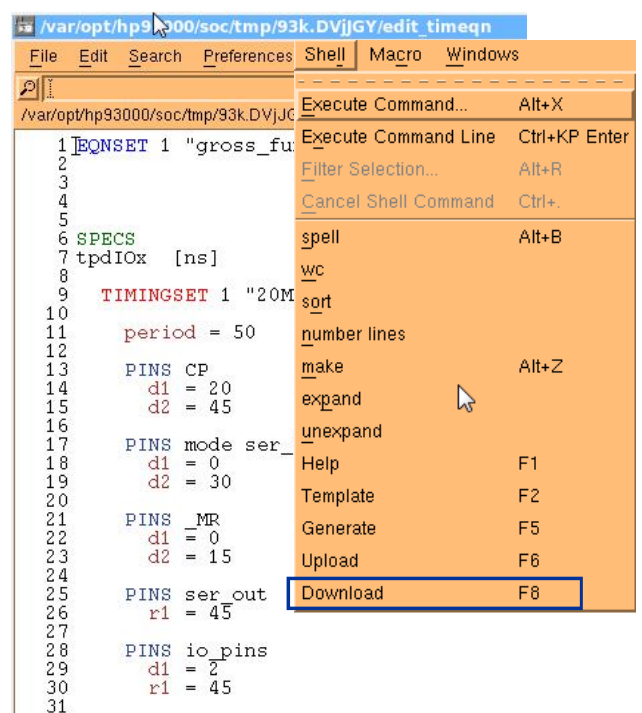
To make changes available in hardware:

- Open shell
- Download

Or use Shortcut-key "**F8**"
Changes are from now on
reflected in hardware



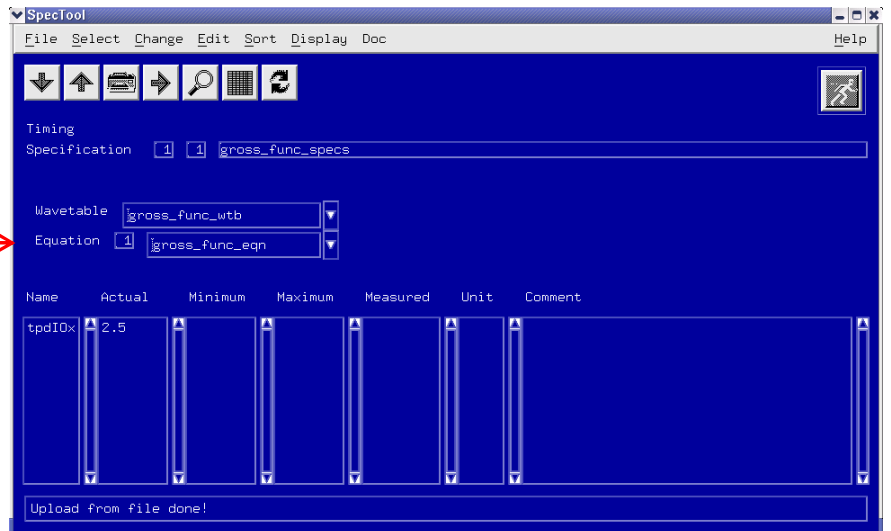
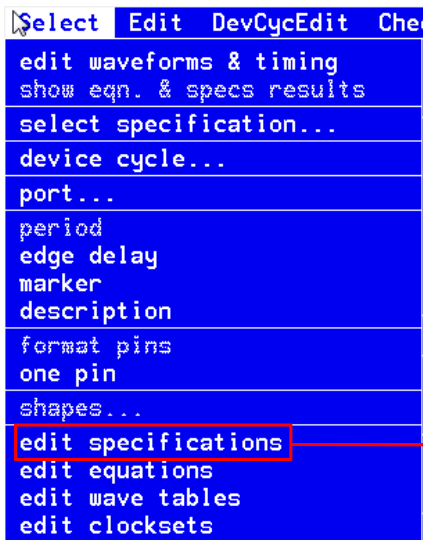
Timing-file is not up-to-date, save it



ADVANTEST

Step 3: Create a Specification

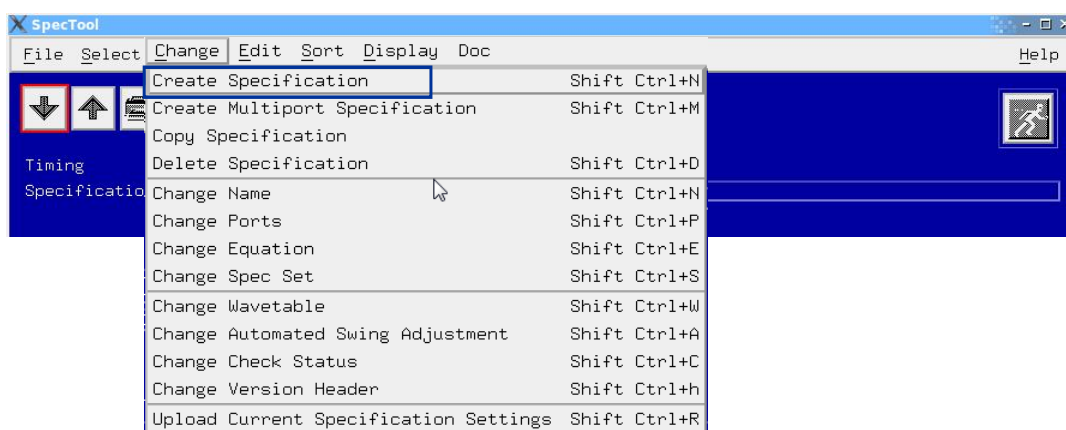
In Timing Setup window click “Select” > “edit specifications”



ADVANTEST

Creating a Specification

In Spec Tool, click “Change” => “Create Specification”



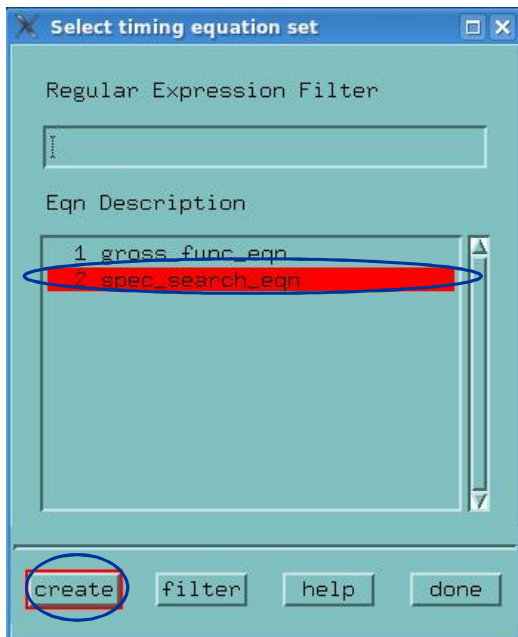
Create a TIMING SPECIFICATION

1. Select the EQUATION SET for the SPECIFICATION
2. Select the SPEC SET number and enter a description
3. Specify values for the spec variables
4. Select a WAVETABLE

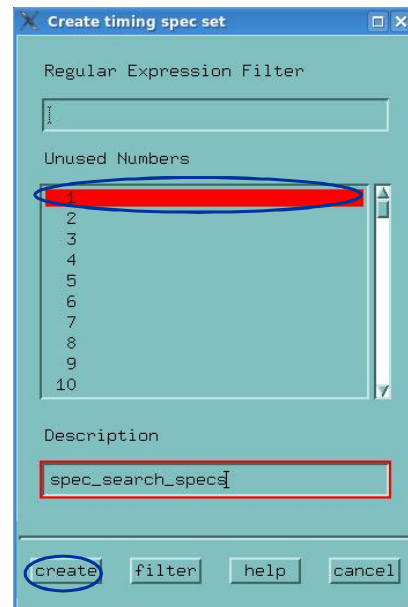
ADVANTEST

Selecting Equation and Spec Set Number

1. Select the Equation Set, click **create**



2. a. Select the Spec Set number
b. Enter the Specification Description
c. Click **create**

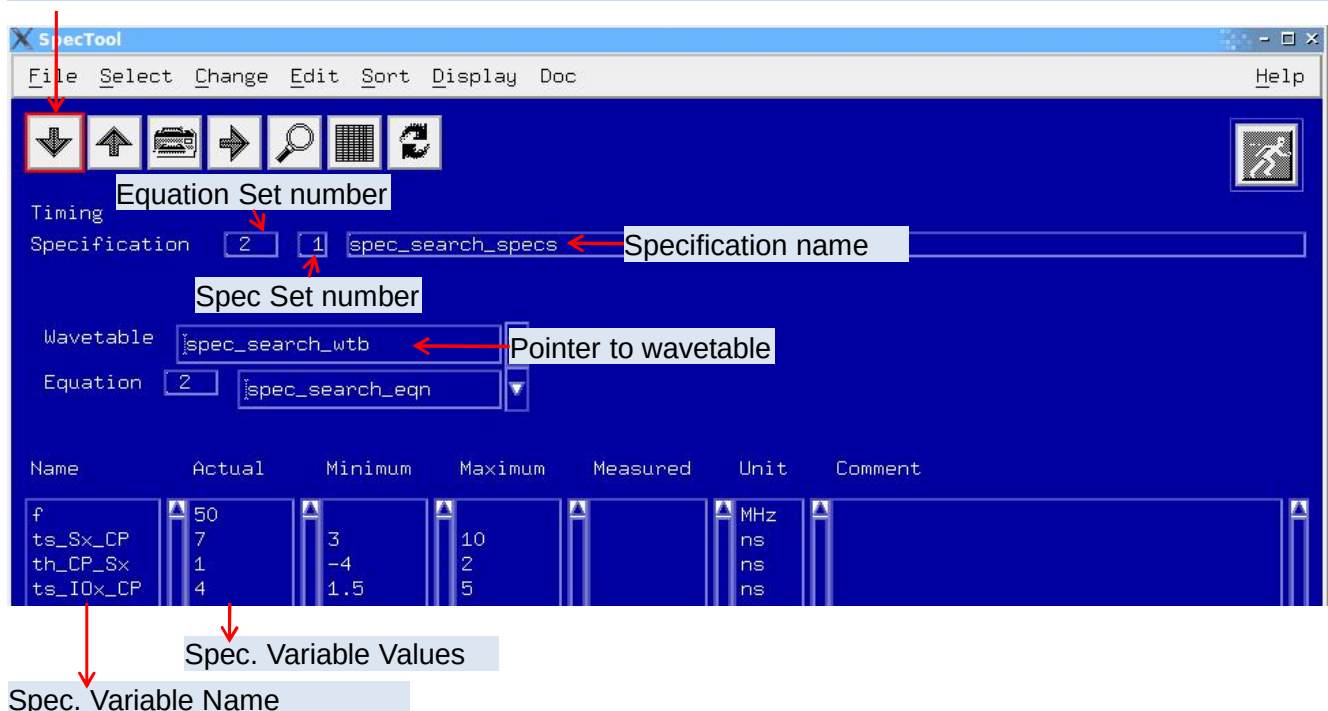


3. Click **done** to close the equation set window

ADVANTEST

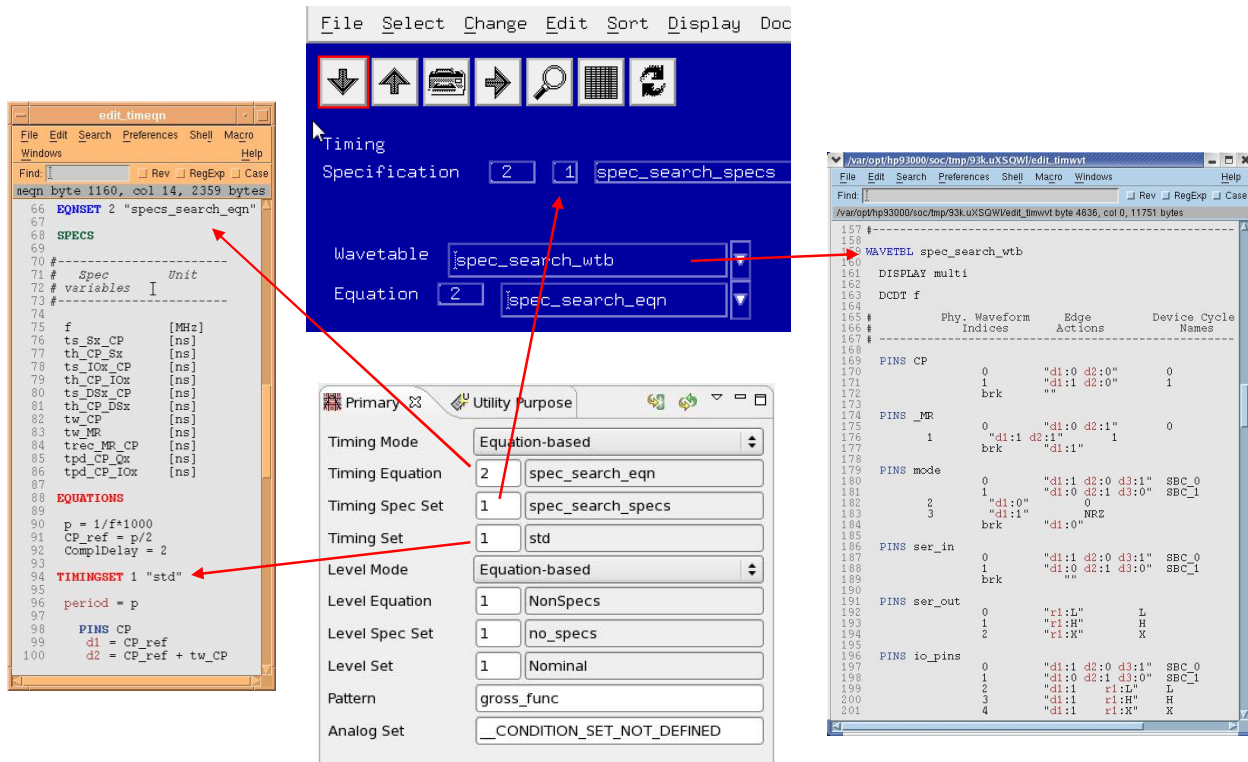
A Complete Timing Specification

After editing values in SpecTool, download them into the hardware. Save timing file afterwards.



ADVANTEST

Relationship of Sets



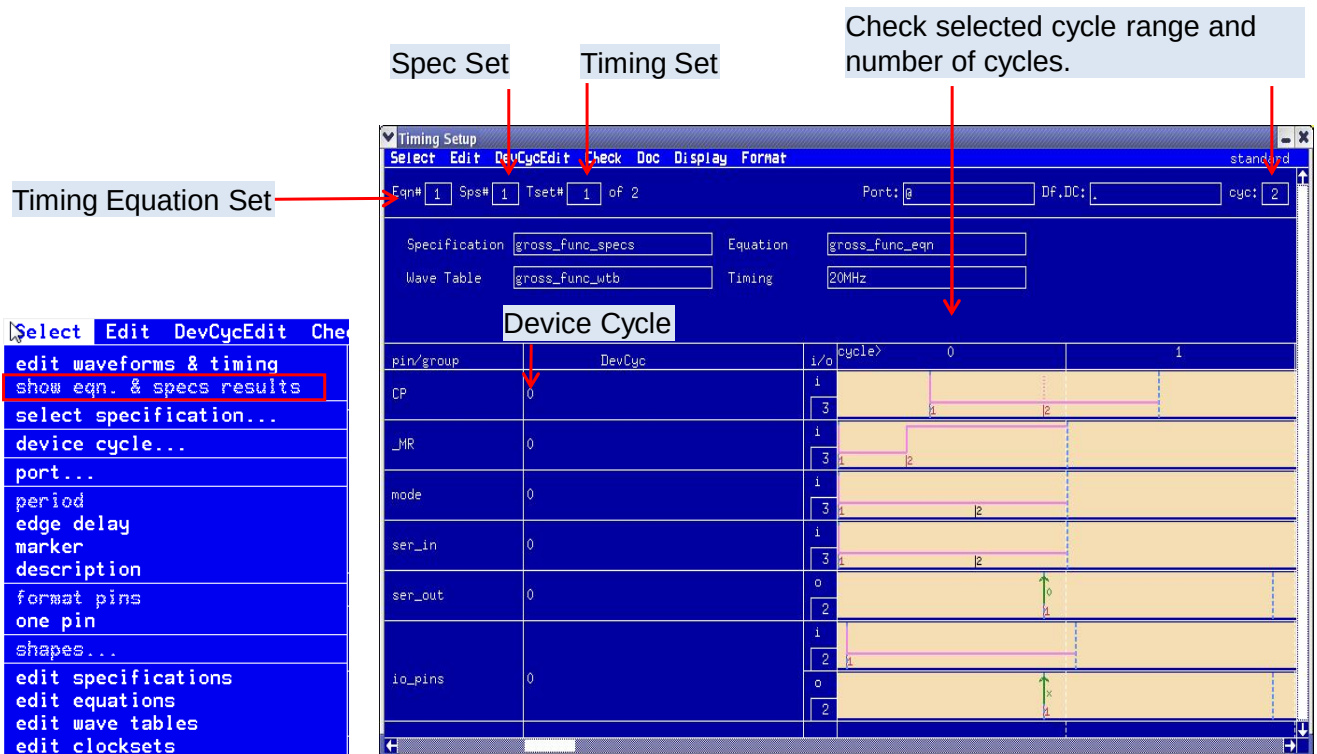
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 41

Step 4: Verification of Timing Setup (1)



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 42

Step 4: Verification of Timing Setup (2)

pin CP with device cycle "0"

The same Device Cycle name for each pin

Define format -

format name: standard

pinlist:

- CP
- DS7
- DS0
- I/07
- I/06
- I/05
- I/04
- I/03
- I/02
- I/01

grouplist:

- CPandMR
- DxAndOx
- all_in
- all_out
- ctrl
- io_in
- io_out
- io_pins
- mode
- ser_in

show same device cycle for all pins ☒

show prev/next device cycle column ☐

show free resource column ☐

show waveform numbers ☐

Buttons: copy, paste, delete, set default, get format, clear all, save, cancel

ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 43

Overwriting Equation Based Timing

After changing a value in this section, the header appears in a different color. To keep changes, go to setup tools.

Timing Setup

Eqn# 1 Sps# 1 Tset# 1 of 2 EDITED Port: a DF,DC: F cyc: 2

edge delays [ns]:

edge	1	2	3	4	5	6	7	8
D	10,000	45,000	0,000	0,000	0,000	0,000	0,000	0,000
R	0,000	0,000	0,000	0,000	0,000	0,000	0,000	0,000

Tester period STD-MODE: 50,000 ns (20,00 MHz)

Restore Specification

Reg Exp Filter :

Specifications :

Eqn	Sps	Description
1	2	gross_func_devcyc_specs

Buttons: restore, cancel

Going back:
Edit à Restore Specification

ADVANTEST®

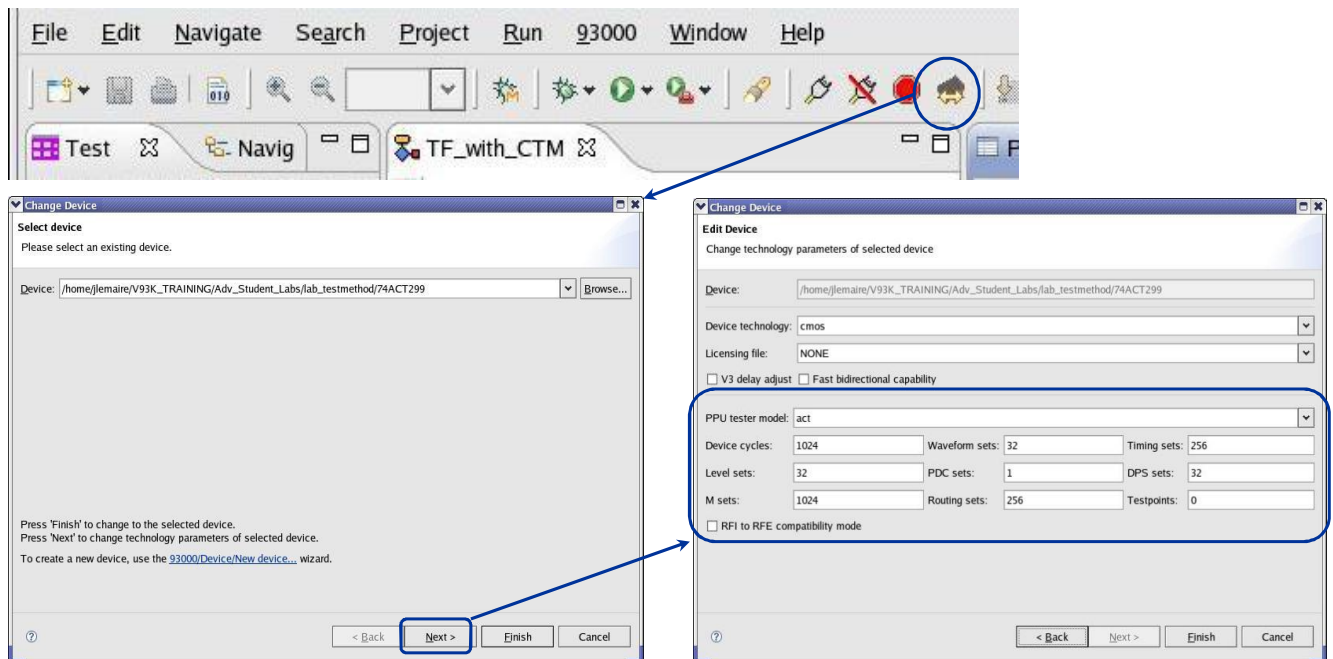
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Timing Unit 6- 44

Other Timing Topics: Change Number of Sets

- Reserves space in Sequencer Memory for sets
- May limit # of waveforms per wavetable



ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):
[Topic 109172](#)

ADVANTEST

Advantest V93000

Digital User Training

Vectors/Pattern

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Vectors and Patterns
- Pattern Editing
- Sequencer Instructions
- Exporting/Importing ASCII files
- Vector-burst
- DUT oriented view: xN and x1/N mode
- Pattern Master File
- Result Analysis: Displaying & Analyzing Failure Data

ADVANTEST[®]

Learning Objectives

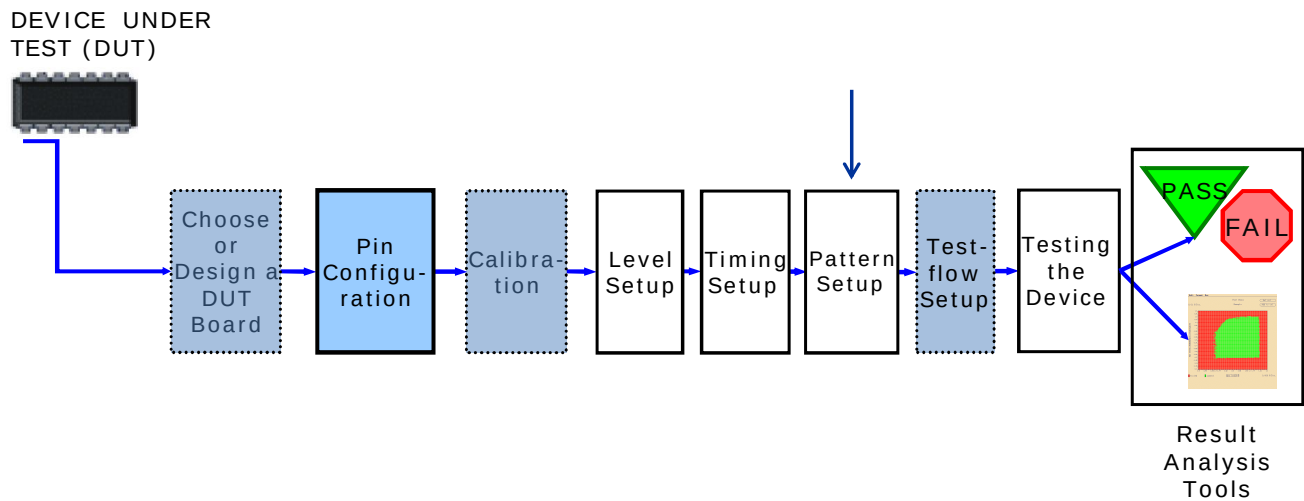
- Understand the terminologies: vector, pattern and label
- Know the structure of a vector file
- Know link between a vector file and a wavetable
- Know how to use the pattern explorer and the pattern debug tool

Relevance

- Vector/Pattern setup is one of the major task during test program development. In debug and characterization phase, test engineers need to be able to change and modify patterns.

Test Program Flow Chart

Defining the Vectors



ADVANTEST

April 2016

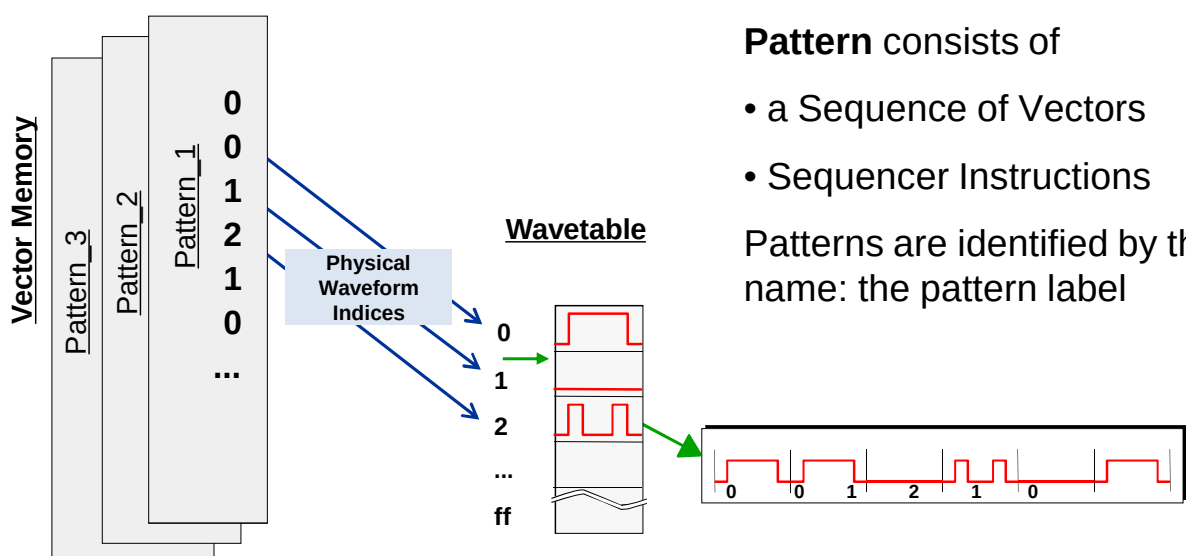
All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 5

Terms & Definitions

Single Vector is a set of driver actions and compare actions at one tester cycle; the vectors are pointers to waveforms

Vector setup determines the sequence of waveforms sent to the DUT during test execution



Pattern consists of

- a Sequence of Vectors
- Sequencer Instructions

Patterns are identified by their name: the pattern label

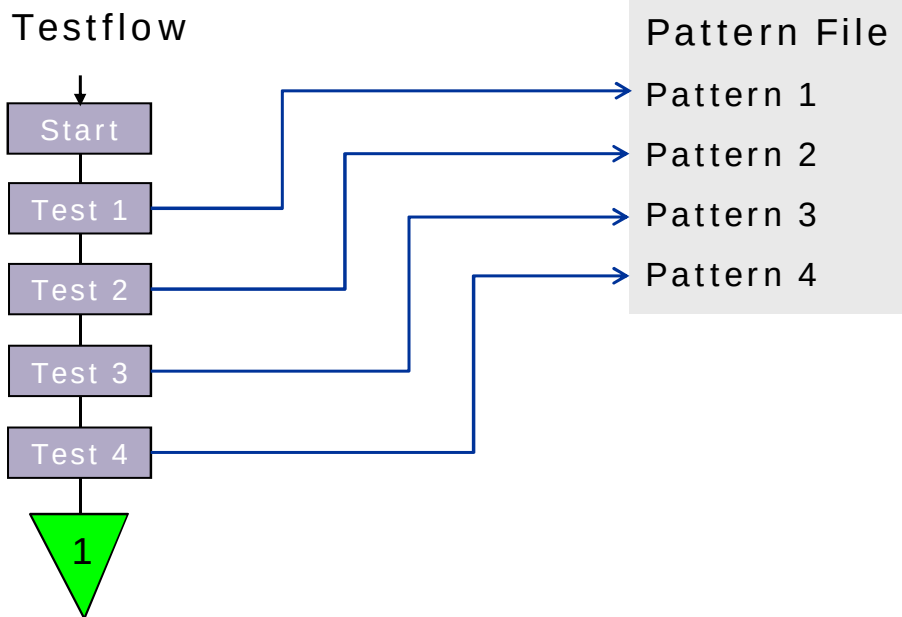
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 6

Typical Pattern Usage in Testflow



ADVANTEST

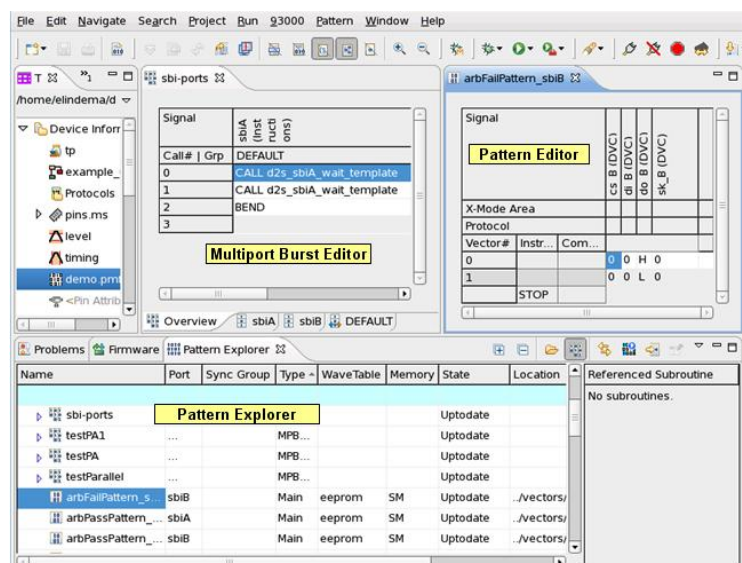
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 7

Pattern Tools

- Pattern Explorer
- Pattern Editor
- Multiport Burst Editor



ADVANTEST

April 2016

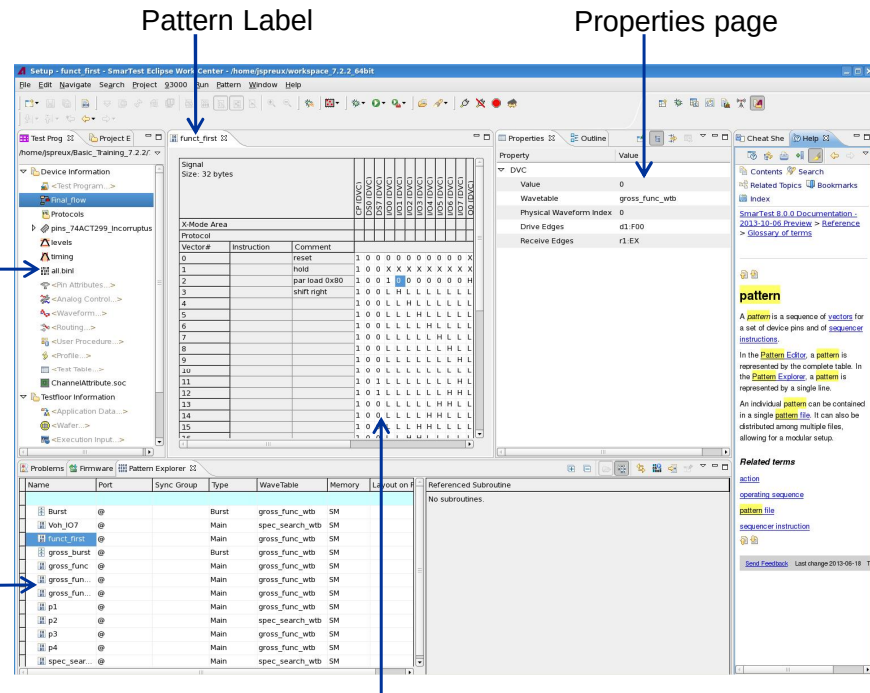
All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 8

Open the Pattern Editor/Debug Tools

1. In the Test Program Explorer double-click the pattern item

2. In the Pattern Explorer double-click the pattern



Pattern Editor /
Debug Tool
consists of two
windows:

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 9

Overview of the Pattern Explorer

Pattern Explorer tasks

- Explore the patterns, sorting and filtering
- Open a pattern, create new pattern, delete pattern
- Open and edit bursts
- Copy, edit, rename a pattern or burst
- Associate layout and wavetable

Name	Port	Sync ...	Type	WaveTable	Memory	Layout on Pattern ...	Call
Burst	@		Burst	gross_func_wtb	SM		gross_func
Burst_test	@		Burst	gross_func_wtb	NONE		p1
Voh_IO7	@		Main	spec_search_wtb	SM		p3
funcnt_first	@		Main	gross_func_wtb	SM		p3
gross_burst	@		Burst	gross_func_wtb	SM		p4
gross_func	@		Main	gross_func_wtb	SM		gross_func
gross_fun...	@		Main	spec_search_wtb	SM		
p1	@		Main	gross_func_wtb	SM		
p2	@		Main	spec_search_wtb	SM		
p3	@		Main	gross_func_wtb	SM		
p4	@		Main	gross_func_wtb	SM		
spec_sear...	@		Main	spec_search_wtb	SM		

Displays all patterns of a pattern file

Displays the contents
of a burst

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 10

Pattern Explorer Navigation

Name	Port	Type	Memory	WaveTable	Layout on Pattern Editor	Name
------	------	------	--------	-----------	--------------------------	------



Link with Editor

Synchronize label in Pattern-Explorer and Pattern-Editor



Show Last Executed Pattern

Select label of primary set / current label for test-execution



Go to Row

Navigate to line in Pattern-Explorer



Lock Detailed Area

Freeze detailed content, independent of selected label; required for defining burst-labels

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 11

Overview of the Pattern Editor

Pattern Editor tasks

- Display the content of patterns
- Edit Patterns
- Analyze and debug pattern failures

Pattern editor Icons
Properties view

Name	Port	Sync ...	Type	WaveTable	Memory	Layout on Pattern Editor
Burst	@		Burst	gross_func_wtb	SM	
Burst_test	@		Burst	gross_func_wtb	NONE	
Voh_IO7	@		Main	spec_search_wtb	SM	
func_t_first	@		Main	gross_func_wtb	SM	my_layout

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 12

Pattern Editor menu items



Switch to expanded/collapsed mode (*)



Refresh result display: Show failure around selected vector area



Switch to horizontal pattern view



Align display: Synchronize other analysis tools (e.g. timing diagram) with current vector cycle (*)



Show/Hide all instructions in current editor



Show/Hide all vectors in current editor



Show/Hide all protocols in current editor



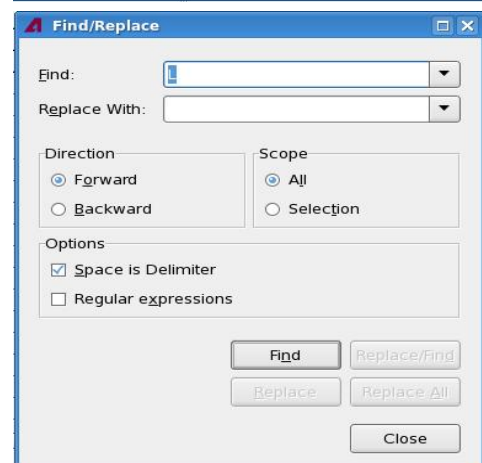
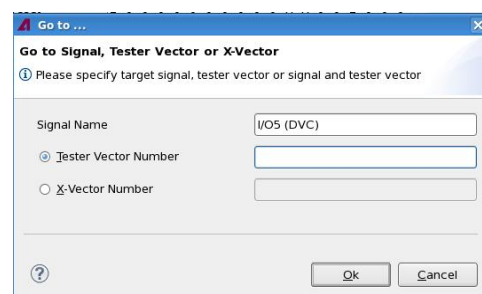
Show/Hide all comments in current editor

(*) details on next slides

ADVANTEST

Pattern Editor Navigating the Pattern

- Go To
In the main menu select Navigate > Go To > and then: Signal, Tester Vector or X-Vector
- Find/Replace
In the Pattern Editor right-click and select Find/Replace...



ADVANTEST

Signal Representations: DVC, PW, SC...

- **PW**: Physical Waveform Indexes
- **DVC**: Device cycle names
- **SC**: State characters
- ...

Signal Size: 32 bytes			CP (DVC)	CP (PW)	CP (SC)	DS0 (DVC)	DS0 (PW)	DS0 (SC)	DS7 (DVC)	DS7 (PW)	DS7 (SC)	I/O1 (DVC)	I/O1 (PW)	I/O1 (SC)	I/O6 (PW)	I/O6 (SC)	I/O7 (DVC)
X-Mode Area																	
Protocol																	
Vector#	Instruction	Comment															
0			1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
1			1	1	1	0	0	0	0	0	0	X	4	X	4	X	X
2			1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
3			1	1	1	0	0	0	0	0	0	H	3	H	2	L	L
4			1	1	1	0	0	0	0	0	0	L	2	L	2	L	L
5			1	1	1	0	0	0	0	0	0	L	2	L	2	L	L
6			1	1	1	0	0	0	0	0	0	L	2	L	2	L	L
7			1	1	1	0	0	0	0	0	0	L	2	L	2	L	L
8			1	1	1	0	0	0	0	0	0	L	2	L	3	H	L
9			1	1	1	0	0	0	0	0	0	L	2	L	2	L	H
10			1	1	1	0	0	0	0	0	0	L	2	L	2	L	L
11			1	1	1	0	0	0	1	1	1	L	2	L	2	L	H
12			1	1	1	0	0	0	1	1	1	L	2	L	3	H	H

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

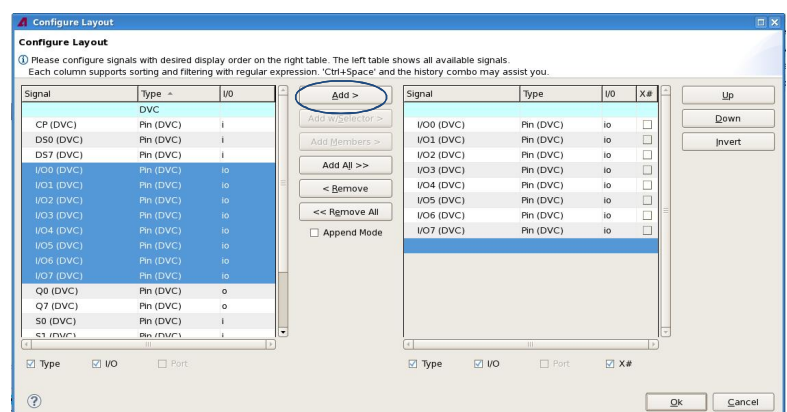
Vectors / Pattern Unit 7 - 15

Layout

- Layouts define which signals are displayed and in which format they are displayed
- Define several layouts in Pattern Editor and store under different names
- In the Pattern Explorer you can associate a layout to a pattern

To create a layout in the Pattern Editor

- Right-click and select **Configure Layout...**
- Select the signals
- Right-click and select **Save Layout As...**



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 16

Layout (example)

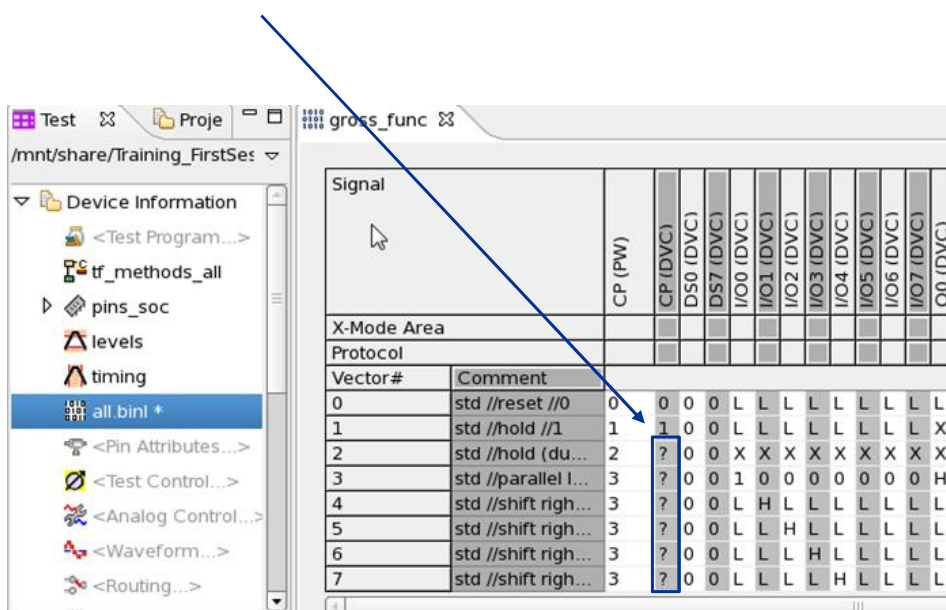
Signal Size: 32 bytes			I/O0 (DVC)	I/O1 (DVC)	I/O2 (DVC)	I/O3 (DVC)	I/O4 (DVC)	I/O5 (DVC)	I/O6 (DVC)	I/O7 (DVC)
X-Mode Area										
Protocol										
Vector#	Instruction	Comment								
0			0	0	0	0	0	0	0	0
1			X	X	X	X	X	X	X	X
2			1	0	0	0	0	0	0	0
3			L	H	L	L	L	L	L	L
4			L	L	H	L	L	L	L	L
5			L	L	L	H	L	L	L	L
6			L	L	L	L	H	L	L	L
7			L	L	L	L	L	H	L	L
8			L	L	L	L	L	L	H	L
9			L	L	L	L	L	L	L	H
10			L	L	L	L	L	L	L	L
11			L	L	L	L	L	L	L	H
12			L	L	L	L	L	L	H	H
13			L	L	L	L	L	H	H	L
14			L	L	L	L	H	H	L	L
15			L	L	L	H	H	L	L	L
16			L	L	H	H	L	L	L	L
17			L	H	H	L	L	L	L	L
18			H	H	L	L	L	L	L	L
19			H	H	L	L	L	L	L	L
20			H	H	L	L	L	L	L	L

Example of Layout:

Display only I/O pins with DVC (Device Cycle Name)

Unresolved Pointers

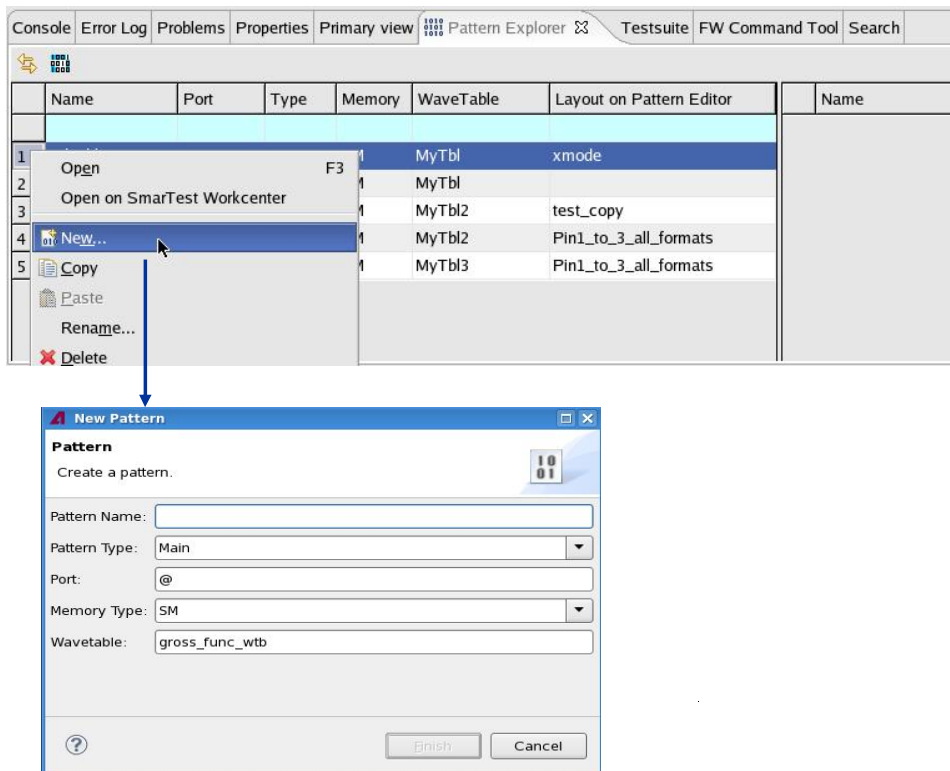
- Question marks denote unresolved pointers in the vector memory



Signal			CP (PW)	CP (DVC)	DS0 (DVC)	DS7 (DVC)	I/O0 (DVC)	I/O1 (DVC)	I/O2 (DVC)	I/O3 (DVC)	I/O4 (DVC)	I/O5 (DVC)	I/O6 (DVC)	I/O7 (DVC)	O0 (DVC)
X-Mode Area															
Protocol															
Vector#	Comment														
0	std //reset //0		0	0	0	L	L	L	L	L	L	L	L	L	L
1	std //hold //1		1	1	0	L	L	L	L	L	L	L	L	L	X
2	std //hold (du...		?	0	0	X	X	X	X	X	X	X	X	X	X
3	std //parallel l...		?	0	0	1	0	0	0	0	0	0	0	0	H
4	std //shift righ...		?	0	0	L	H	L	L	L	L	L	L	L	L
5	std //shift righ...		?	0	0	L	L	H	L	L	L	L	L	L	L
6	std //shift righ...		?	0	0	L	L	L	H	L	L	L	L	L	L
7	std //shift righ...		?	0	0	L	L	L	L	H	L	L	L	L	L

Pattern Explorer

Creating a Pattern



ADVANTEST

April 2016

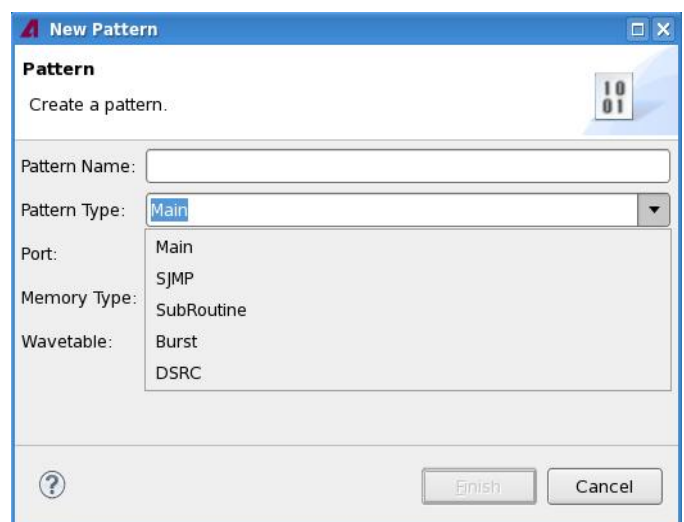
All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 19

Pattern Explorer

Pattern Types

- **Main:** Regular pattern, most commonly used
- **SubRoutine:** Sub-pattern, jump from a MAIN pattern
- **Burst:** A group of MAIN patterns in series without any break
- **DSRC:** Pattern for Digital SouRCe Memory



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 20

Pattern Editor

Quick Reference

- **<ctrl><space>**: Activates the content assist (also in Pattern Explorer)
- **<ctrl><space>** - **<space>**: Activates the content assist and opens pull-down menu
- **<ctrl><backspace>**: Activates the content assist and deletes the current entry
- **<ctrl><backspace>** - **<backspace>**: Activates the content assist, deletes the current entry and shows the complete pull-down menu
- **<shift>Left-click**: Selects a range in a table. To select a range in a table click the left upper cell, press Shift and click the right lower cell

Signal	CP (PW)	CP (SC)	DS0 (SC)	DS7 (SC)	IO0 (SC)	IO1 (SC)	IO2 (SC)	IO3 (SC)	IO4 (SC)	IO5 (SC)	IO6 (SC)
0	1	1	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0	0
3	1	1	0	0	0	0	0	0	0	0	0
4	1	1	0	0	0	0	0	0	0	0	0
5	1	1	0	0	0	0	0	0	0	0	0
6	1	1	0	0	0	0	0	0	0	0	0
7	1	1	0	0	0	0	0	0	0	0	0
8	1	1	0	0	0	0	0	0	0	0	0
9	1	1	0	0	0	0	0	0	0	0	0
10	1	1	0	0	0	0	0	0	0	0	0
11	1	1	0	0	0	0	0	0	0	0	0
12	1	1	0	0	0	0	0	0	0	0	0
13	1	1	0	0	0	0	0	0	0	0	0
14	1	1	0	0	0	0	0	0	0	0	0
15	1	1	0	0	0	0	0	0	0	0	0
16	1	1	0	0	0	0	0	0	0	0	0
17	1	1	0	0	0	0	0	0	0	0	0
18	1	1	0	0	0	0	0	0	0	0	0

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 21

Pattern Editor

Copy and Paste Vectors

- Standard controls are available like working a spread sheet:
- **<ctrl><C>**: copy selected
- **<ctrl><V>**: paste copied
- **<ctrl><Z>**: undo

Signal	CP (PW)	CP (SC)	DS0 (SC)	DS7 (SC)	IO0 (SC)	IO1 (SC)	IO2 (SC)	IO3 (SC)	IO4 (SC)	IO5 (SC)	IO6 (SC)
0	1	1	0	0	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0	0
3	1	1	0	0	0	0	0	0	0	0	0
4	1	1	0	0	0	0	0	0	0	0	0
5	1	1	0	0	0	0	0	0	0	0	0
6	1	1	0	0	0	0	0	0	0	0	0
7	1	1	0	0	0	0	0	0	0	0	0
8	1	1	0	0	0	0	0	0	0	0	0
9	1	1	0	0	0	0	0	0	0	0	0
10	1	1	0	0	0	0	0	0	0	0	0
11	1	1	0	0	0	0	0	0	0	0	0

Copy block of vectors:

<ctrl><C>

<ctrl><V>

- If the target vectors lines are an exact multiple of the copied lines, the copied lines will be pasted multiple times
- To insert new vector line(s): Highlight the line following the insertion point, right-click and select **Insert Vector** or **Insert Vector Special**

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 22

Pattern Editor

Expanded & Collapsed Mode

The  button toggles between collapsed and expanded mode

Collapsed mode:

- Displays all instructions, but vectors contained in loops and other subroutines are not shown individually

Expanded mode:

- Displays all instructions and vectors (including vectors contained in loops and other routines)
- Cycle # are displayed

*Expanded mode is only available after execution

Signal				CP (PW)	CP (SC)	DS0 (SC)	DS7 (SC)	I/O0 (SC)	I/O1 (SC)	I/O2 (SC)
X-Mode Area										
Protocol										
Vector#	Instruction	Comment								
0		init		1	1	0	0	0	0	0
1	RPTV 1 3	reset		1	1	0	0	0	0	0
2		hold		1	1	0	0	X	X	X
3		par load 0x80		1	1	0	0	1	0	0
4		shift right		1	1	0	0	L	H	L
5				1	1	0	0	L	L	H
6				1	1	0	0	L	L	L

Signal				CP (PW)	CP (SC)	DS0 (SC)	DS7 (SC)	I/O0 (SC)	I/O1 (SC)	I/O2 (SC)
X-Mode Area										
Protocol										
Cycle#	Vector#	Instruction	Comment							
0	0	GENV 1	init	1	1	0	0	0	0	0
1	1	RPTV 1 1/3	reset	1	1	0	0	0	0	0
2	1	RPTV 1 2/3	reset	1	1	0	0	0	0	0
3	1	RPTV 1 3/3	reset	1	1	0	0	0	0	0
4	2	GENV 28	hold	1	1	0	0	X	X	X
5	3		par load 0x80	1	1	0	0	1	0	0
6	4		shift right	1	1	0	0	L	H	L
7	5			1	1	0	0	L	L	H
8	6			1	1	0	0	L	L	L

ADVANTEST

Pattern Editor

Inserting Sequencer Instructions

- Highlight the line at which you want to insert the sequencer instruction, if you want to insert a loop (LOOP, MATCH), highlight all lines that shall be included in the loop
- Right-click and select **Insert Instruction**
- From the list box select the sequencer instruction and double-click
- Type the parameters if required

Signal				DS0 (DC)	DS7 (DC)	I/O0 (DC)	I/O1 (DC)	I/O2 (DC)	I/O3 (DC)	I/O4 (DC)	I/O5 (DC)	I/O6 (DC)	I/O7 (DC)	Q0 (DC)	Q7 (DC)	S0 (DC)	S1 (DC)	_MR (DC)
X-Mode Area				1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Vector#				Comment														
0				std //reset //0														
1				std //hold //1														
2				std //hold (dumm...														
3				0 0 0 0 0 H L 1 1 1														
4				L L L L L L L 1 0 1														
5				L L L L L L L 1 0 1														
6				L L L L L L L 1 0 1														
7				H L L L L L L 1 0 1														
8				L L L L L L L 1 0 1														
9				L L L L L L L 1 0 1														
10				L L L L L L L 1 0 1														
11				L L L L L L L 1 0 1														

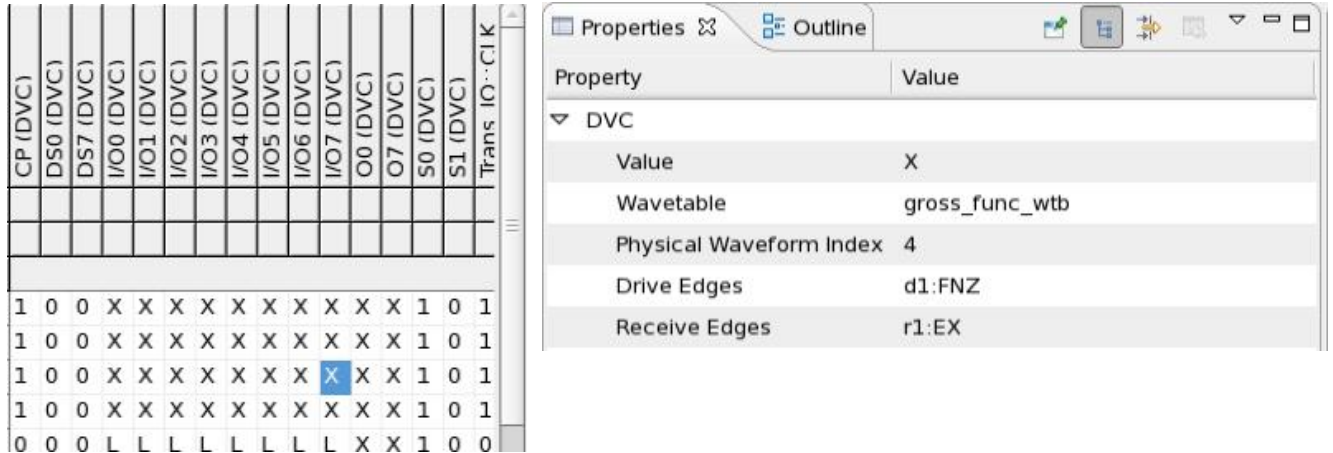
Signal				DS0 (DC)	DS7 (DC)	I/O0 (DC)	I/O1 (DC)	I/O2 (DC)	I/O3 (DC)	I/O4 (DC)	I/O5 (DC)	I/O6 (DC)	I/O7 (DC)	Q0 (DC)	Q7 (DC)	S0 (DC)	S1 (DC)	_MR (DC)
X-Mode Area				1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
Vector#				Comment														
0				std //reset //0														
1				std //hold //1														
2				std //hold (dumm...														
3				std //parallel loa...														
4				std //shift right //4														
5				std //shift right //5														
6				std //shift right //6														
7				std //shift right //7														
8				std //shift right //8														
9				std //shift right //9														
10				std //shift right //10														
11				std //shift right //11														
12				std //shift left ...														
13				std //shift left ...														
14				std //shift left //14														

ADVANTEST

Pattern Editor

Properties view

- Highlight a device cycle in the Pattern Editor to view its properties



ADVANTEST

April 2016

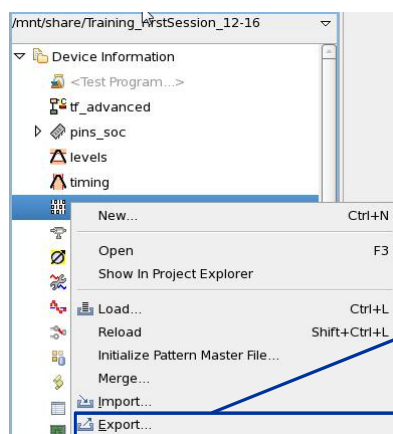
All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 25

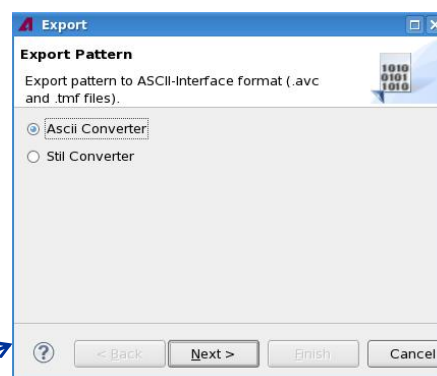
Export/Import Vectors into ASCII-file

- In SmarTest, Pattern are stored in binary format.
- By means of export/import feature within vector-tools, text-files (non binary) can be saved/loaded.

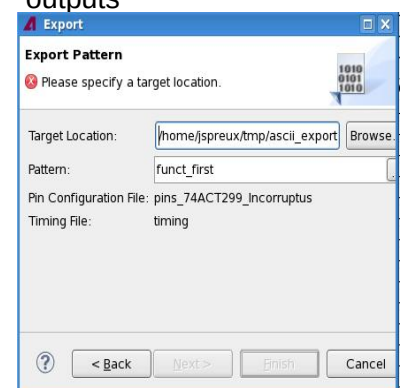
1. Start exporting via Test Program Explorer



2. Select output format



3. Define filename and location of outputs



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

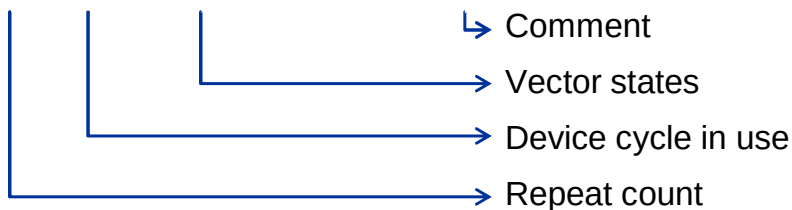
Vectors / Pattern Unit 7 - 26

Vector-file in ASCII-format

Vector ASCII file

```
FORMAT CP DS0 DS7 I/O0 I/O1 I/O2 I/O3 I/O4 I/O5 I/O6 I/O7 Q0 Q7 S0 S1 _MR ;
```

```
R1 dvc_1 100000000000XX000      reset;  
R1 dvc_1 100XXXXXXXXXX001      hold;  
R1 dvc_1 10010000000HL111      par load 0x80;  
R1 dvc_1 100LHLLLLLLLLL101      shift right;  
R1 dvc_1 100LLHLLLLLLLLL101;  
R1 dvc_1 100LLLHLLLLLLLLL101;  
R1 dvc_1 100LLLLHLLLLLLLLL101;  
R1 dvc_1 100LLLLLHLLLLLLLLL101;  
R1 dvc_1 100LLLLLLHLLLLLLLLL101;  
R1 dvc_1 100LLLLLLLHLLLLLLLLL101;  
R1 dvc_1 100LLLLLLLLHLLLLLLLLL101;  
R1 dvc_1 100XXXXXXXL001      io switch;  
R1 dvc_1 10010101010HL111      par load 0xaa;  
R1 dvc_1 100HLHLHLHLHL001      hold;  
R1 dvc_1 100LHLHLHLHL011      shift left;  
R1 dvc_1 000XXXXXXXXXX000      reset;  
R1 dvc_1 100XXXXXXXXXX001      hold;
```



Timing file

Wavetable expanded by STATEMAP section

```
PINS io_pins  
0 "d1:0 r1:X" 0  
1 "d1:1 r1:X" 1  
2 "d1:Z r1:L" L  
3 "d1:Z r1:H" H  
4 "d1:Z r1:X" X  
brk ""
```

```
STATEMAP  
# index state_Char x_mode  
0 0 x1  
1 1 x1  
2 L x1  
3 H x1  
4 X x1
```

value of state-character in ASCII-file

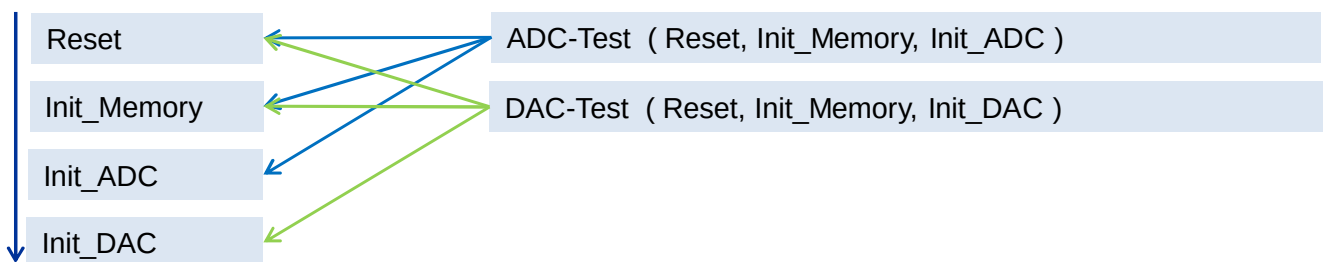
ADVANTEST

Burst Label

... is made up of several other main-labels

Main Labels

Burst Labels



- Reusing same pattern in various combination with other patterns, e.g. more complex reset-routine combined with different initialization vectors
- Patterns with same test requirement (primary) can be concatenated to a burst
- Due to a bit of time-overhead for each execution, this will save test-time

ADVANTEST

Creating a Burst-Label

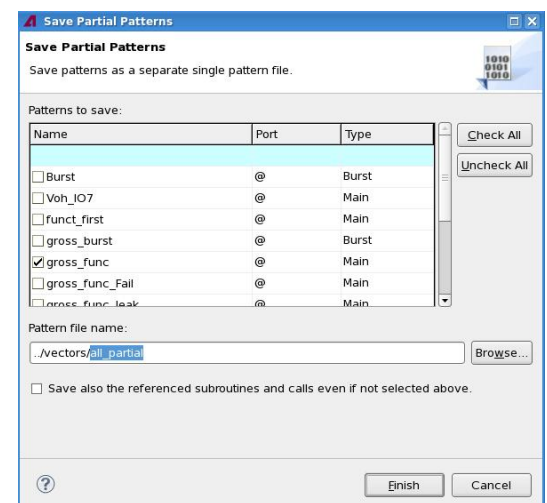
1. Create a new pattern with pattern type burst (see slide “Creating a pattern”)
2. In the main area highlight the burst
3. Click the button  (Lock view)
4. In the main area highlight the pattern you want to add to your burst, right-click and select **Copy**
5. In the burst instruction area right-click and select **Insert** (add pattern) or **Paste** (replace pattern)
6. To change the sequence: In the details area highlight the line you want to shift, right-click and select **Up** or **Down**

Signal	m_burst (instr uctions)
Call#	
0	CALL p1
1	CALL Voh_IO7
2	BEND

Splitting up Pattern Files (Save Partial)

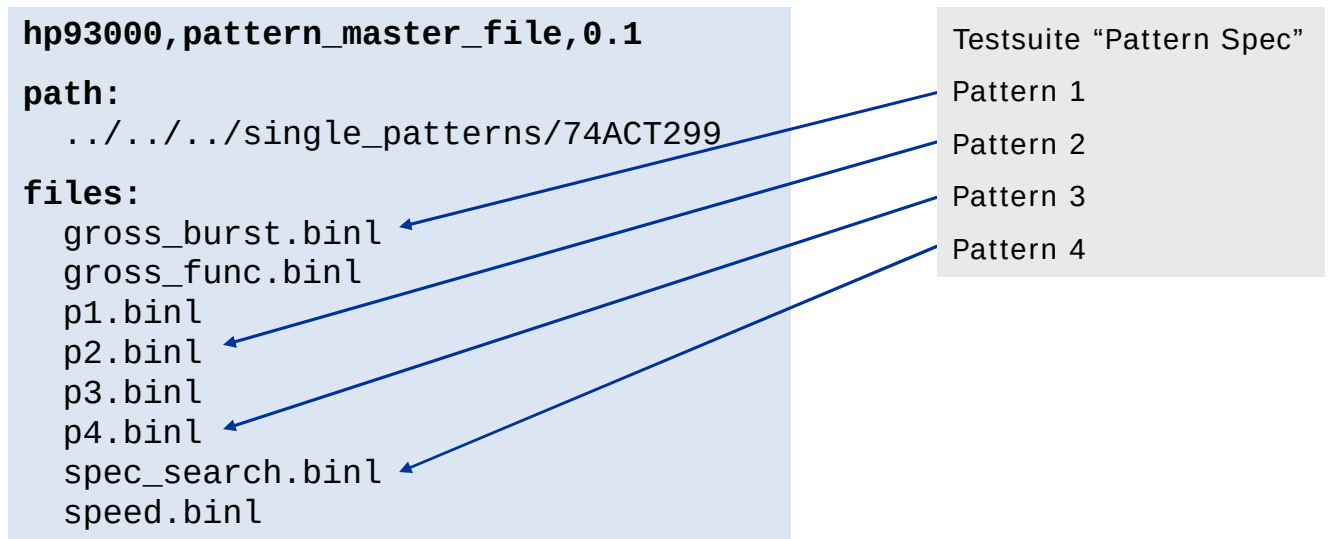
You can split up large pattern files in order to simplify your pattern management.

1. In the Test Program Explorer highlight the pattern item, right-click and select **Save Partial**
2. Select or type a file name for your new pattern file and click **OK**
3. In the Pattern to Save window select the patterns which shall be a part of your new vectors file and click **Finish**



Pattern Master File

- SmarTest needs to “see” one single pattern file on the controller disk: this can also be a “master file”, pointing to many pattern files
- Each pattern is downloaded separately to the hardware



ADVANTEST

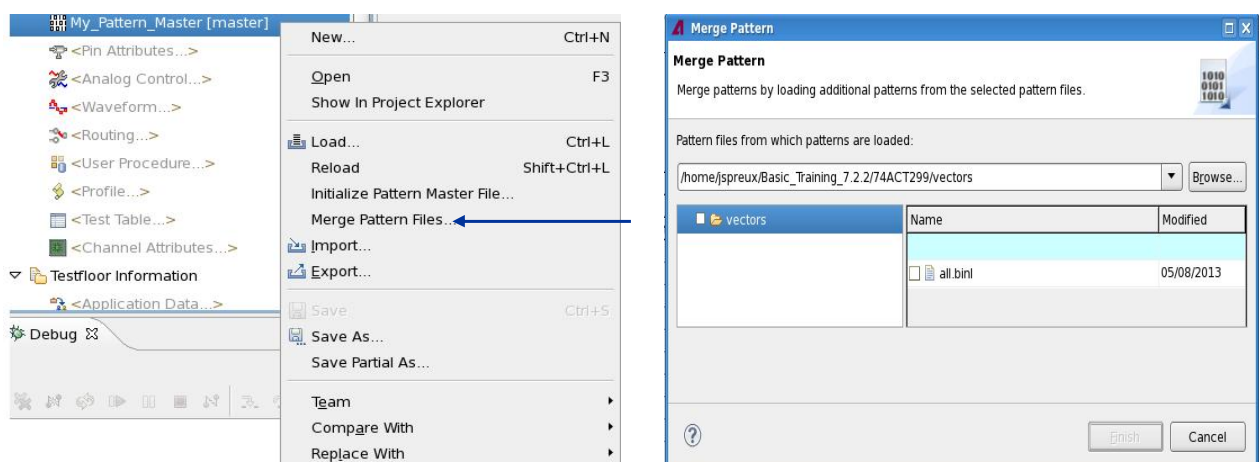
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 31

Merging Pattern Files

1. Create a Pattern Master file from the test program explorer
2. Select and add pattern files to the Pattern Master file



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Vectors / Pattern Unit 7 - 32

Initialize Pattern Master File

Pattern Manager

you can load single pattern as needed in debug scenario.

1. From the Test Program Explorer highlight the pattern item,
2. right-click and select Initialize Pattern Master file....
3. Select your Pattern Master File and click OK.

The Pattern Explorer shows the content of the Pattern Master file.

Pattern name							Pattern file location		
Name	Port	Sync ...	Type	WaveTable	Memory	State	Location	Layout on	
▼  ../vectors (4)									
 p1	@		Main	gross_func_wtb	SM	Uptodate	../vectors/p1		
 p2	@		Main	spec_search_wtb	SM	Uptodate	../vectors/p2		
 p3	@		Main	gross_func_wtb	SM	Uptodate	../vectors/p3		
 p4	@		Main	gross_func_wtb	SM	Uptodate	../vectors/p4		

ADVANTEST[®]

Pattern Editor functions as result analysis tool

- Pattern Editor must be in expanded mode
- Errors are highlighted in red
- Horizontal annotations indicate failing cycles
- Vertical annotations indicate failing signals (pins/pin groups)

ADVANTEST[®]

Result Analysis/Displaying Failure Data

Vertical Annotation

Global Fail Annotation

Vector Number

Cycle Number

Horizontal Annotation

Horizontal Overview Annotation

Failing Cycles

Vertical Overview Annotation

Signal	CP (DVC)	D50 (DVC)	D57 (DVC)	I/O0 (DVC)	I/O1 (DVC)	I/O2 (DVC)	I/O3 (DVC)	I/O4 (DVC)	I/O5 (DVC)	I/O6 (DVC)	I/O7 (DVC)	O0 (DVC)	O7 (DVC)	S0 (DVC)	S1 (DVC)
0	1	0	0	L	L	L	L	L	L	L	L	L	L	0	0
1	1	0	0	L	L	L	L	L	L	L	L	X	X	0	0
2	1	0	0	X	X	X	X	X	X	X	X	X	X	0	0
3	1	0	0	1	0	0	0	0	0	0	0	0	H	L	1
4	1	0	0	L	H	L	L	L	L	L	L	L	L	1	0
5	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
6	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
7	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
8	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
9	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
10	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
11	1	0	0	L	L	L	L	L	L	L	L	L	L	1	0
12	1	0	1	L	L	L	L	L	L	L	L	L	H	0	1
13	1	0	1	L	L	L	L	L	L	L	L	L	H	0	1
14	1	0	0	L	L	L	L	L	L	L	L	L	L	0	1
15	1	0	0	L	L	L	L	L	L	L	L	L	L	0	1
16	1	0	0	L	L	L	L	L	L	L	L	L	L	0	1
17	1	0	0	L	L	L	L	L	L	L	L	L	L	0	1
18	1	0	0	L	H	H	L	L	L	L	L	L	L	0	1
19	1	0	0	H	H	L	L	L	L	L	L	L	H	0	1
20	1	0	0	H	L	L	L	L	L	L	L	L	H	0	1
21	1	0	0	H	L	L	L	L	L	L	L	X	X	0	0
22	1	0	0	H	L	L	L	L	L	L	L	X	X	0	0
23	1	0	0	H	L	L	L	L	L	L	L	X	X	0	0

ADVANTEST

Navigate through Failure Data

Move forward to next and backward to previous annotation

Failure analysis requires expanded mode

Refresh the current window area with failure information

Filter Annotation

Errors

Info

Point Markers

Warnings

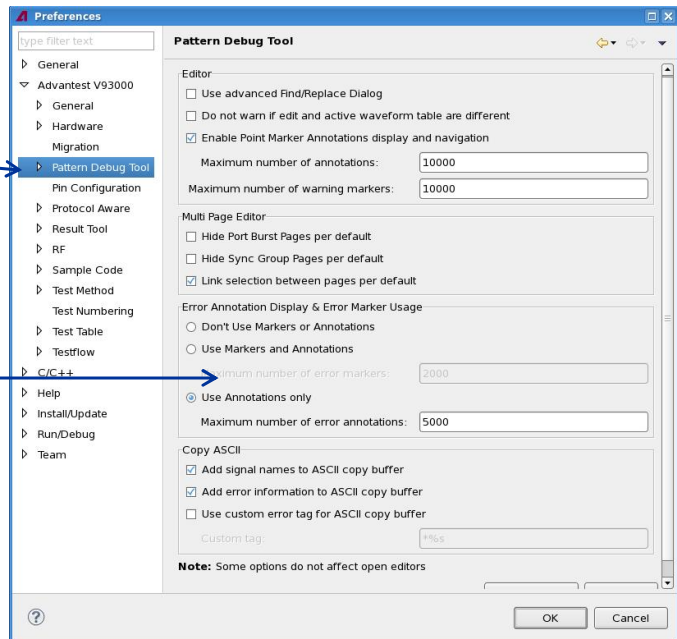
ADVANTEST

Customizing SmarTest-Editors

"Window" > "Preferences" will open Preferences GUI

Select GUI you want to change preferences for

Adjust GUI-specific setups



ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):
[Topic 100084](#)

ADVANTEST

Lab Exercise

Vectors/Patterns

Advantest V93000 SOC Digital User Training

Loadboard Calibration

ADVANTEST[®]

Learning Objectives

- Understand the need for loadboard calibration
- Know how to perform loadboard calibration

ADVANTEST[®]

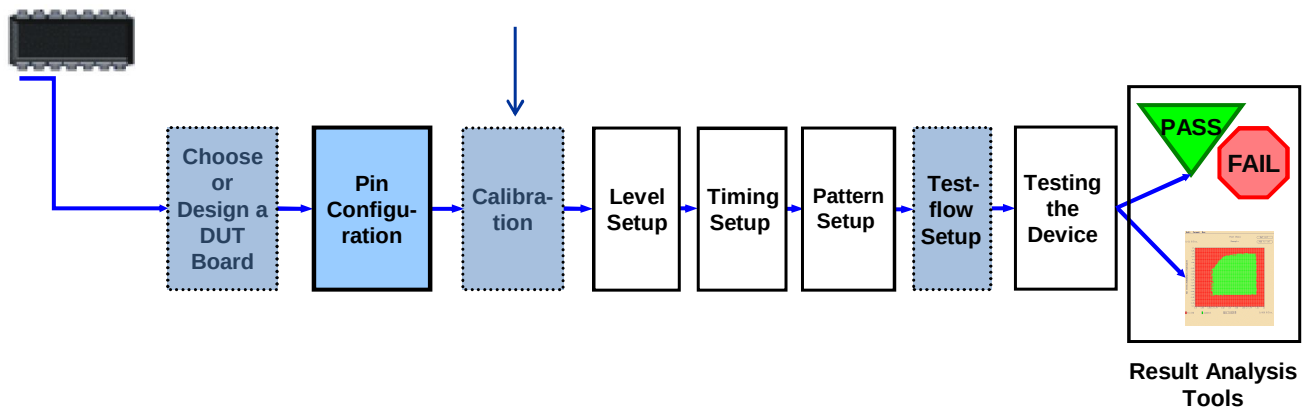
Relevance

- The loadboard calibration is based on time domain reflectometry
- This is a test engineer's or product engineer's task

Test Program Flow Chart

Loadboard Calibration

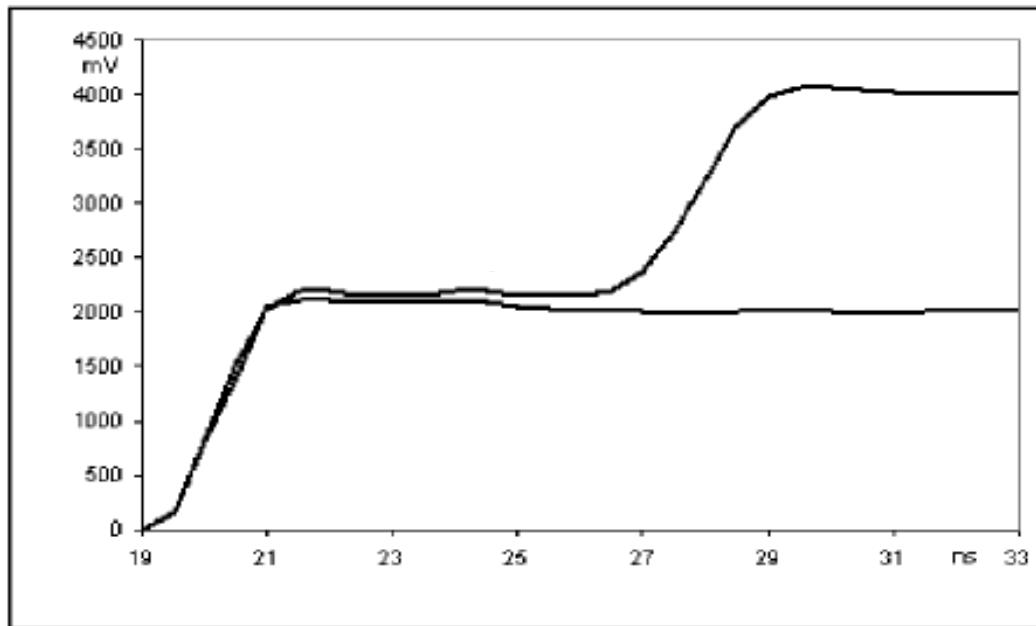
DEVICE UNDER
TEST (DUT)



Signal Path Delay Calibration

TDR (Time Domain Reflectometry)

Signal Reflection in a Transmission Line



Open Circuit

Terminated

ADVANTEST

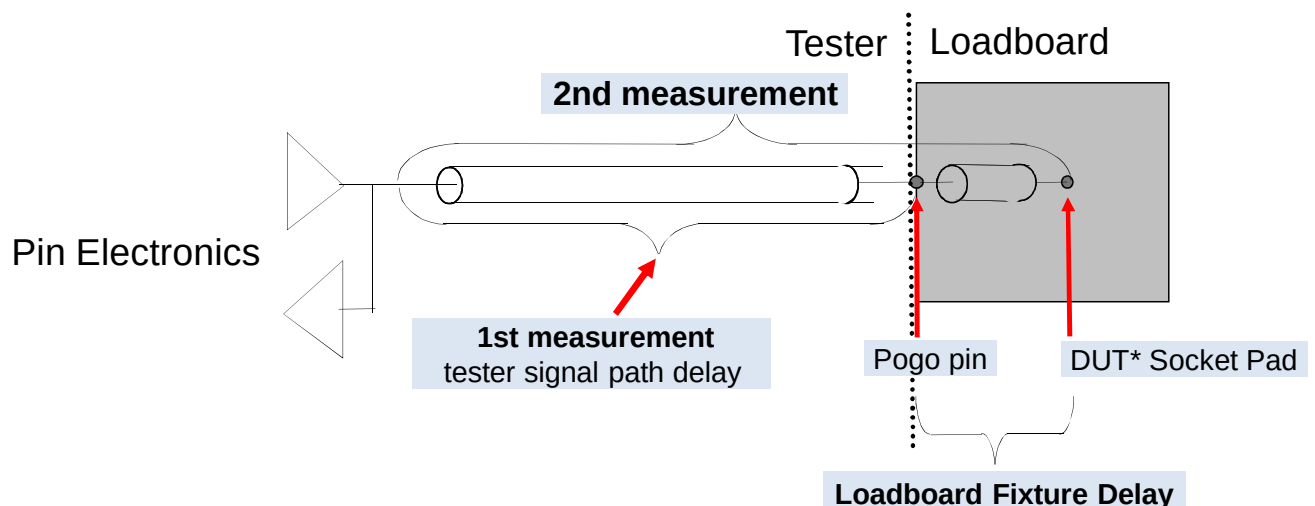
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Loadboard Calibration Unit 8- 5

Signal Path Delay Calibration

2-Step TDR Measurement



* DUT = Device Under Test

Difference between 2. measurement and 1. measurement is
Loadboard Fixture Delay

ADVANTEST

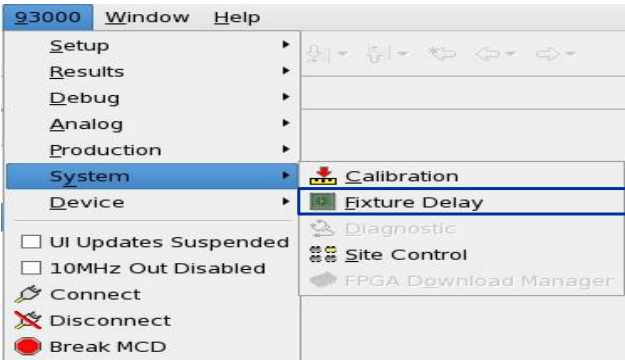
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Loadboard Calibration Unit 8- 6

Fixture Delay Calibration Tool

1. Click 93000 > System > Fixture Delay



The screenshot shows the 'Installed channels view' with a table of configured channels. The table has columns for Pin Name, Pin Number, Channel, and drive (drv) for both Site 1 and Site 2. The Round Trip Delay is set to 0.000 ns. The table lists 32 channels, including CP, DS0, DS7, I/O0 through I/O7, Q0, Q7, S0, S1, and _MR.

Pin Name	Pin Number	Site 1		Site 2	
		Channel	drv	Channel	drv
CP	12	10509	0.817	10609	1.194
DS0	11	10508	0.763	10608	1.140
DS7	18	10515	0.867	10615	1.257
I/O0	7	10503	0.860	10603	1.237
I/O1	13	10510	0.821	10610	1.214
I/O2	6	10504	0.861	10604	1.249
I/O3	14	10511	0.839	10611	1.229
I/O4	5	10505	0.884	10605	1.265
I/O5	15	10512	0.836	10612	1.230
I/O6	4	10506	0.888	10606	1.268
I/O7	16	10513	0.847	10613	1.239
Q0	8	10502	0.840	10602	1.226
Q7	17	10514	0.853	10614	1.245
S0	1	10507	0.891	10607	1.266
S1	19	10516	0.874	10616	1.264
_MR	9	10501	0.865	10601	1.241

Configured Channels, 32 configured channel(s) selected for measurement.

Installed channels view: shows all channels installed in the system



DUT Interface Board Calibration Results

Optional: Save backup via File Drop Down Menu: **SAVE AS** or **SAVE**

The screenshot shows the 'Configured channels view' with a table of configured channels. The table has columns for Pin Name, Pin Number, Channel, and drive (drv) for both Site 1 and Site 2. The Round Trip Delay is set to 0.000 ns. The table lists 32 channels, including CP, DS0, DS7, I/O0 through I/O7, Q0, Q7, S0, S1, and _MR.

Pin Name	Pin Number	Site 1		Site 2	
		Channel	drv	Channel	drv
CP	12	10509	0.817	10609	1.194
DS0	11	10508	0.763	10608	1.140
DS7	18	10515	0.867	10615	1.257
I/O0	7	10503	0.860	10603	1.237
I/O1	13	10510	0.821	10610	1.214
I/O2	6	10504	0.861	10604	1.249
I/O3	14	10511	0.839	10611	1.229
I/O4	5	10505	0.884	10605	1.265
I/O5	15	10512	0.836	10612	1.230
I/O6	4	10506	0.888	10606	1.268
I/O7	16	10513	0.847	10613	1.239
Q0	8	10502	0.840	10602	1.226
Q7	17	10514	0.853	10614	1.245
S0	1	10507	0.891	10607	1.266
S1	19	10516	0.874	10616	1.264
_MR	9	10501	0.865	10601	1.241

Configured Channels, 32 configured channel(s) selected for measurement.

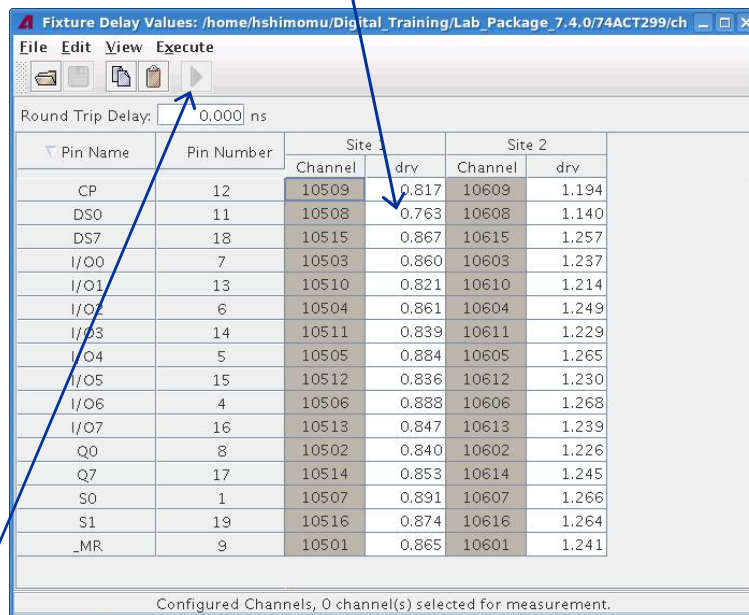
Configured channels view: shows all channels set up in the currently loaded pin configuration



Fixture Delay Calibration Tool

Pin Selection and Calibration Execution

2. Select Channels to calibrate, use the Edit menu and/or <Ctrl>-click on the channel(s)



3. Click the Measure Icon to Start the measurement process

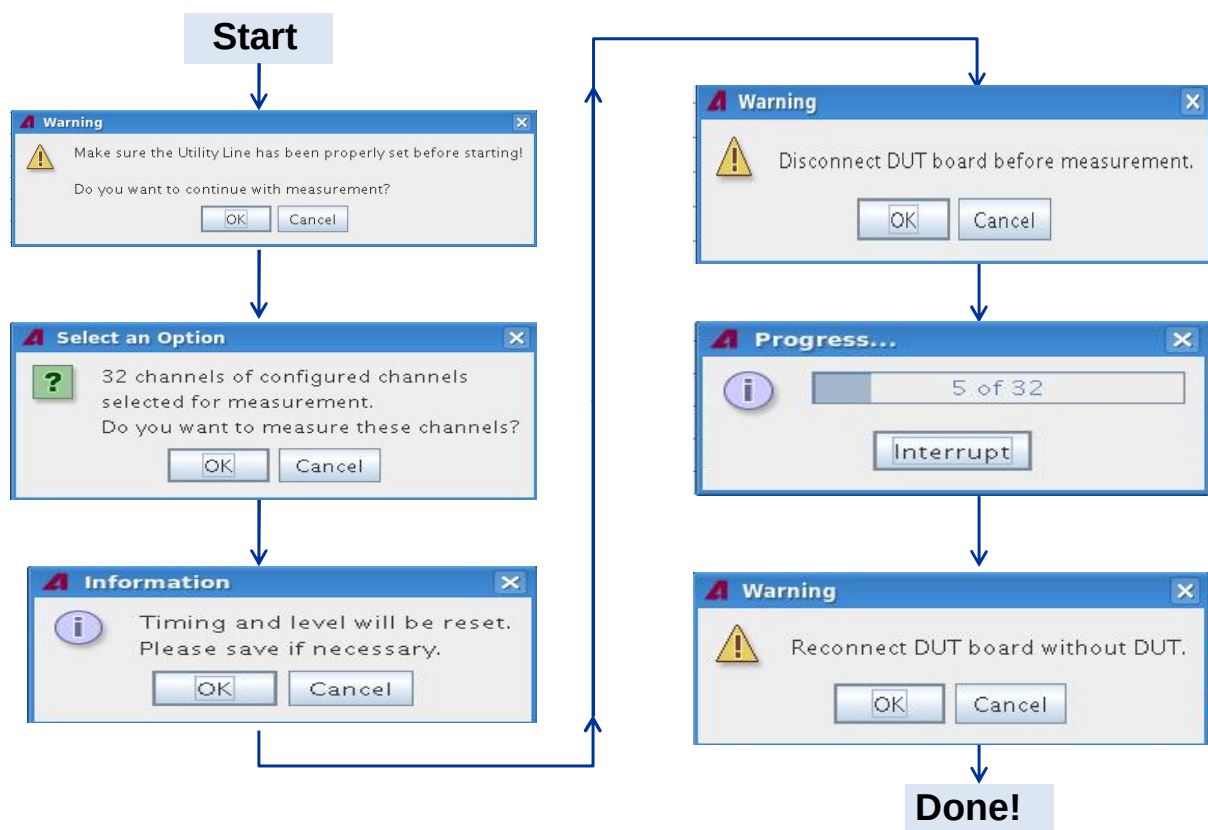
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Loadboard Calibration Unit 8- 9

Delay Calibration Process Dialog



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Loadboard Calibration Unit 8- 10

Fixture Delay Calibration Tool

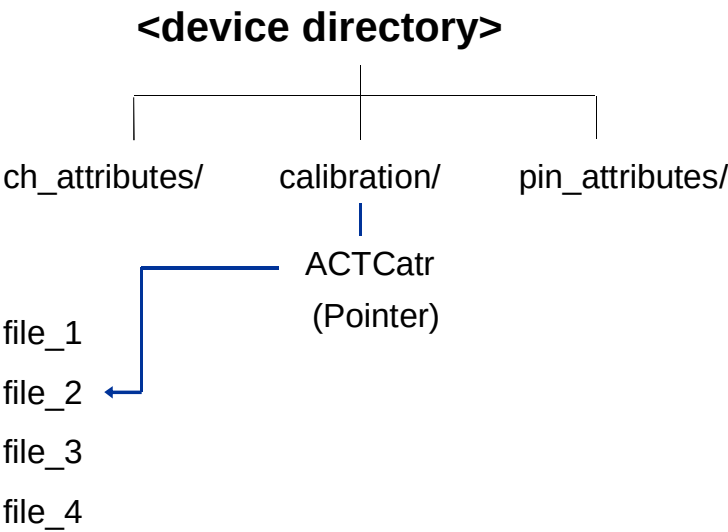
Saving the Results

4. File à **SAVE AS** or **SAVE**

Pin Name	Pin Number	Site 1		Site 2	
		Channel	drv	Channel	drv
CP	12	10509	0.817	10609	1.194
DS0	11	10508	0.763	10608	1.140
DS7	18	10515	0.867	10615	1.257
I/O0	7	10503	0.860	10603	1.237
I/O1	13	10510	0.821	10610	1.214
I/O2	6	10504	0.861	10604	1.249
I/O3	14	10511	0.839	10611	1.229
I/O4	5	10505	0.884	10605	1.265
I/O5	15	10512	0.836	10612	1.230
I/O6	4	10506	0.888	10606	1.268
I/O7	16	10513	0.847	10613	1.239
Q0	8	10502	0.840	10602	1.226
Q7	17	10514	0.853	10614	1.245
S0	1	10507	0.891	10607	1.266
S1	19	10516	0.874	10616	1.264
_MR	9	10501	0.865	10601	1.241

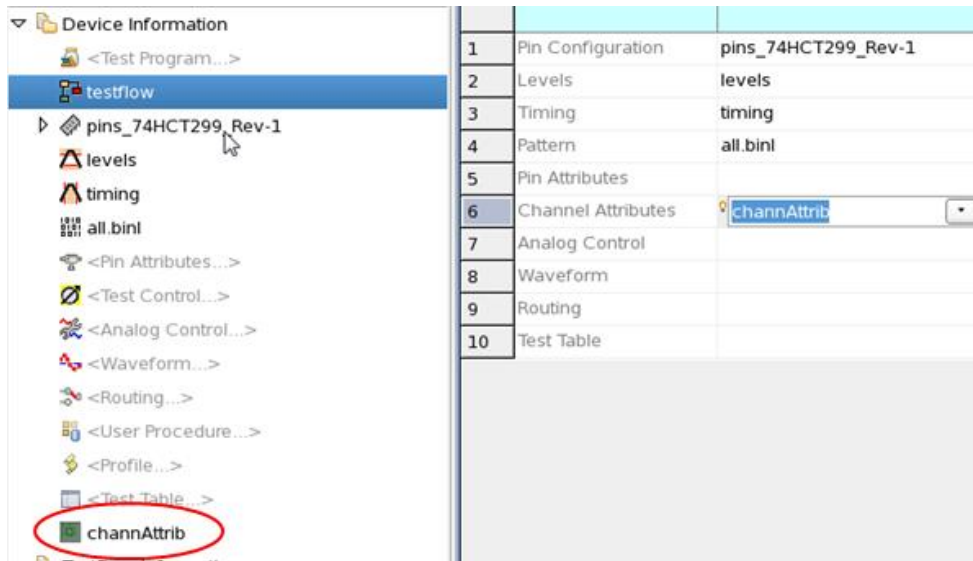
Fixture Delay Calibration Program Access

File Structure (Default)



Fixture Delay is part of Testflow Setup

5. On Setup Page of your Testflow, define Channel Attributes file



ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):
[Topic 56334](#)

ADVANTEST

Demo

Loadboard Calibration

Advantest V93000

Digital User Training

Testflow

ADVANTEST[®]

Learning Objectives

- Know how to build the DUT testing flow using the SmarTest Testflow GUI
- Know the features of the SmarTest Testflow GUI
- Know how to link setup elements to build a Testflow

ADVANTEST[®]

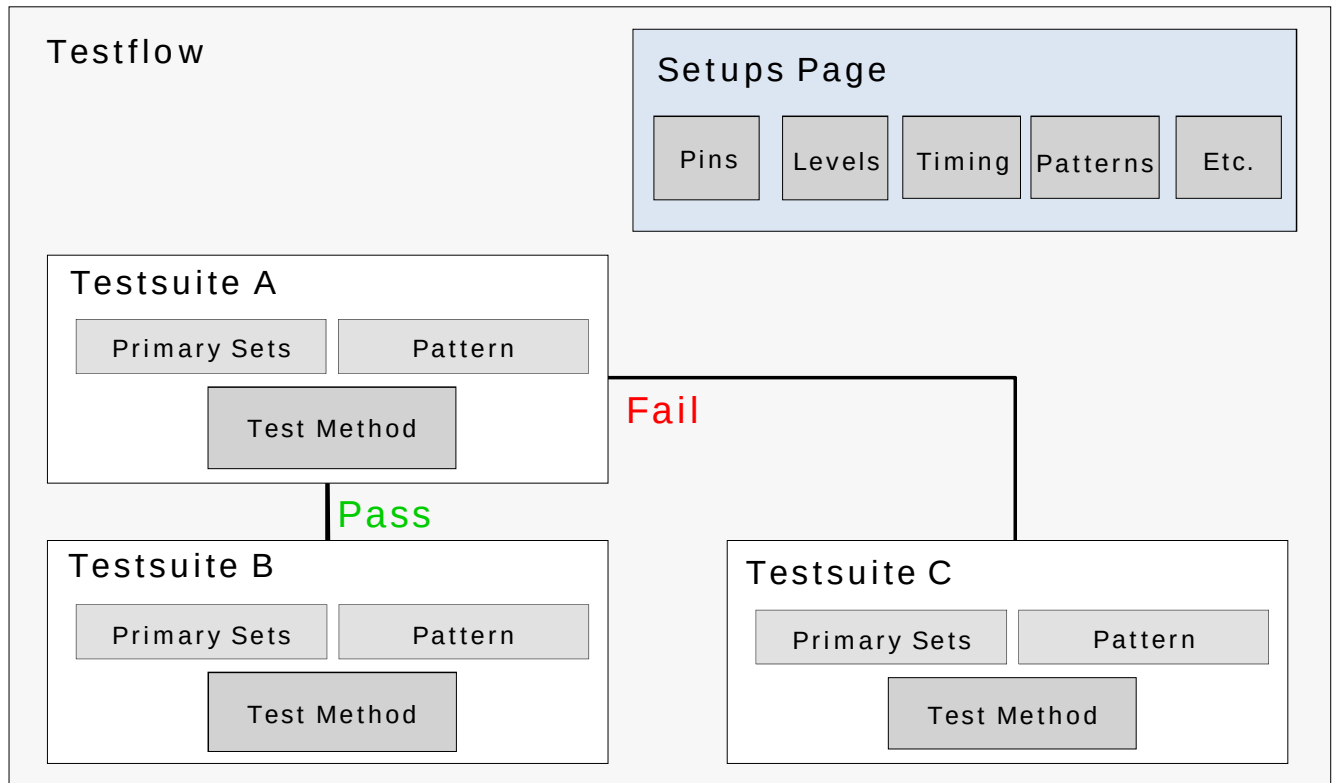
Relevance

- The testflow is the place within the SmarTest environment where all the setup steps converge and where the test is executed.
- A testflow is a collection of individual tests, each one testing a particular parameter or function. Together, all these tests form the complete test of your device.

Agenda

- Testflow Overview
- Testflow Elements
- Testflow Creation
- Testflow Execution
- Testflow Flags
- Working with Testflow
- Other Features

Testflow Overview: Top-Down Approach

**ADVANTEST®**

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 5

From Test Plan to Testflow

Testsuite Order	Functional Tests	DC Tests	AC Tests
1	gross_func	power_short	
2			
3		Continuity	
4		OutputDC	
5	spec_func		
6	gross_func _p2		
7			tpd_CP_IOx
8			tpd_CP_Qx
9		vol_voh	
10		leakage	
11		Iddq	

Flow

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

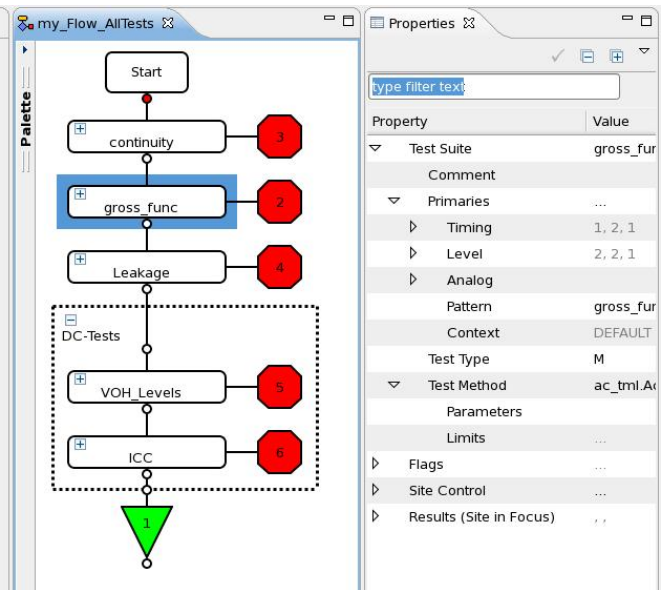
Testflow Unit 9 - 6

Testflow Interfaces

Flow Data Editor

	Test Suite Na...	Test Setup		Timing			Ec
		Type	Name	Eqn	Spe...	Set	
1	continuity	M	dc_tml.DcTest.Continuity	1	1	1	2
2	gross_func	M	ac_tml.AcTest.FunctionalTest	1	2	1	2
3	Leakage	M	dc_tml.DcTest.Leakage	1	1	1	2
4	VOH_Levels	M	dc_tml.DcTest.OutputDC				
5	ICC	M	dc_tml.DcTest.OperatingCurr...				

Flow Sequence Editor Properties page



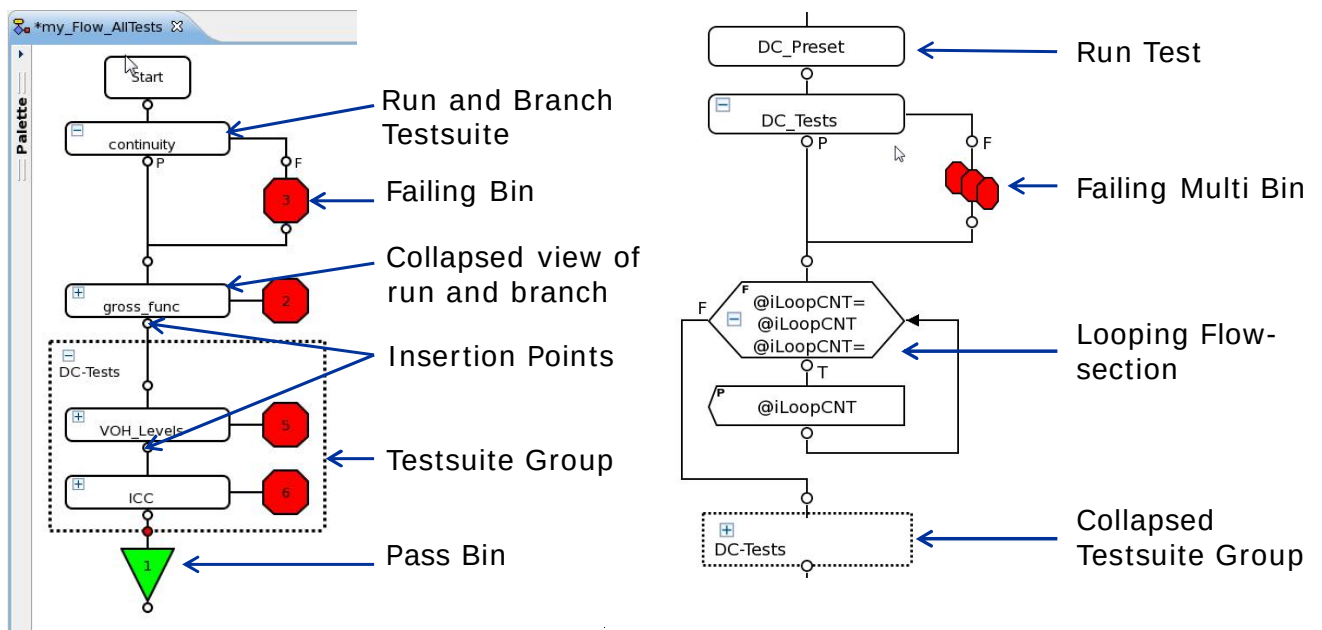
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 7

Flow Sequence Editor Testflow Elements



ADVANTEST

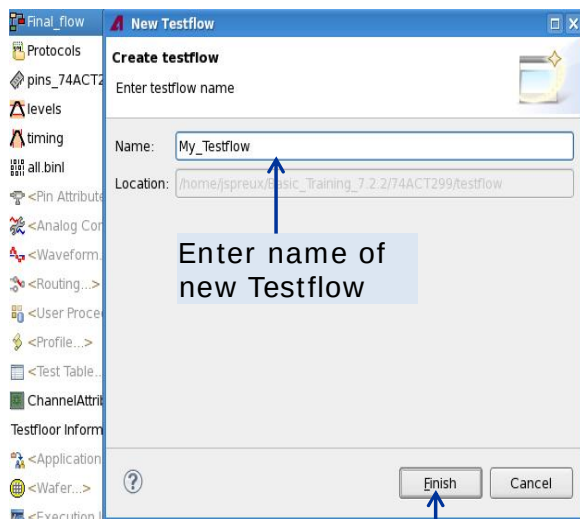
April 2016

All Rights Reserved - ADVANTEST CORPORATION

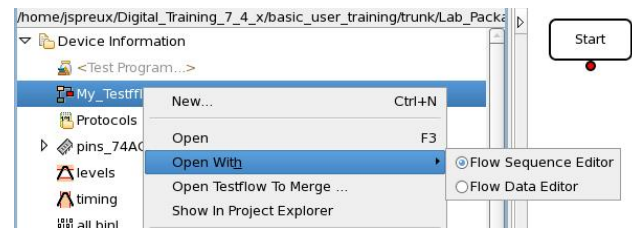
Testflow Unit 9 - 8

Create a new Testflow

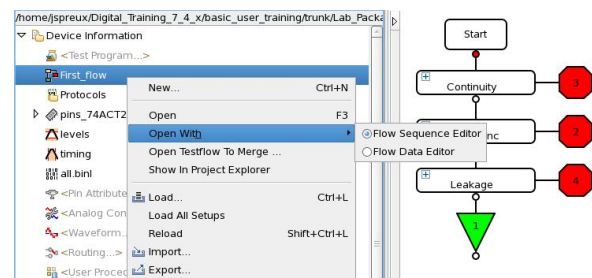
1. Right-click -> "New..." opens ...



2. "Flow Sequence Editor" opens empty flow



3. After flow is created, right-click -> "Open With" brings up existing flow...



ADVANTEST

April 2016

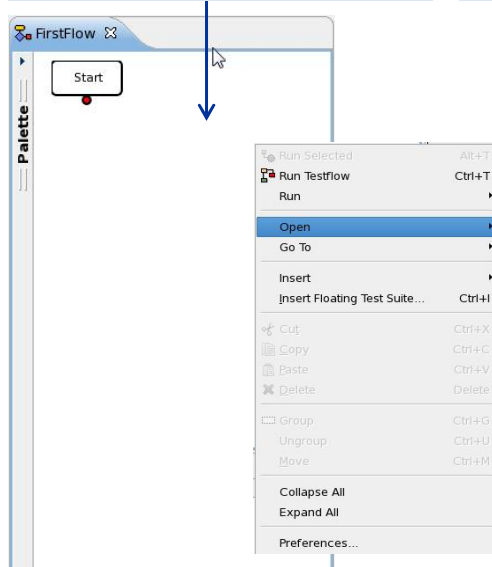
All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 9

Assign Setup Files to Testflow

- To make the setup files (timing, levels, ...) available within Testflow, we have to select them from the Setups Page

1. Right-click into empty "Flow Sequencer Editor"



2. Select "Open" and "Setup" page

Variables Page
Flags Page
Setups Page

3. On Setups-page of flow data editor we choose our setup files

	Type	Name
1	Pin Configuration	pins_74ACT299_Incorruptus_CSDPS
2	Protocols	
3	Levels	levels
4	Timing	timing
5	Pattern	all.binl
6	Pin Attributes	
7	Channel Attributes	ChannelAttribute.soc
8	Analog Control	
9	Waveform	
10	Routing	
11	Test Table	ttable

Use <Ctrl> + <Space> for content assist

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 10

Insert Testsuite Elements

1. Select insertion point

2. Open Palette and select "Run and Branch"

3. Mandatory Entries:

- Test Suite Name
- Test Method

4. Select Test Method

5. Click Finish

dc_tml.DcTest.Continuity

ADVANTEST

Insert a Bin into the Testflow

2. Open Palette

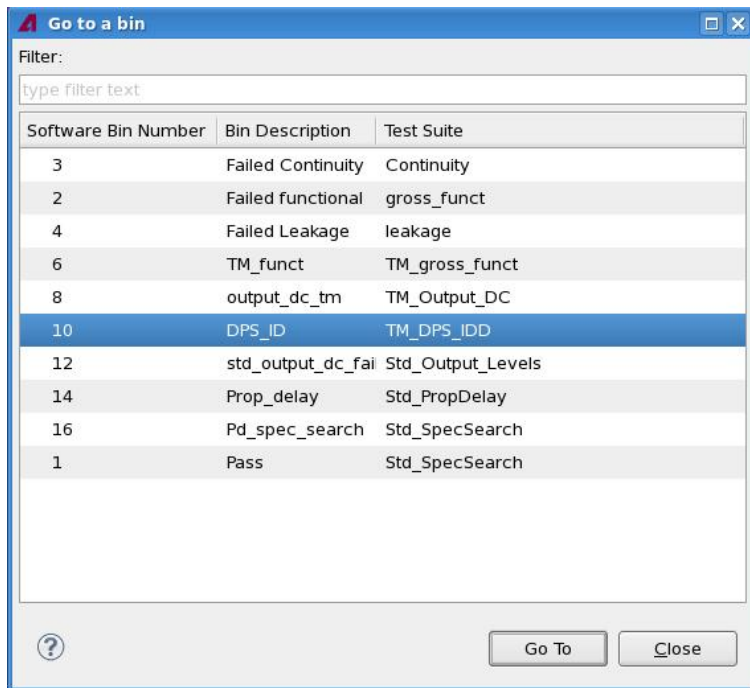
1. Select insertion point

3. Select Bad Bin

ADVANTEST

Overview of all Tests and Bins in a Testflow

1. Right Click-> Go to -> Bin



ADVANTEST

April 2016

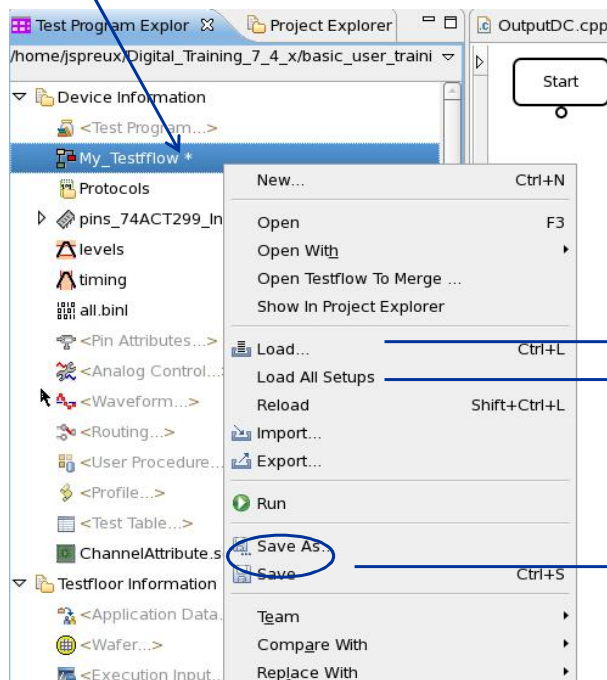
All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 13

Save and Load Testflow

1. * Changes done in Flow è Save required

2. Right-click in Test Program Explorer on Testflow



Loading Testflow in two steps:

- Load... will open "Select file to load"
- Load All Setups will fill tester-hardware with setup data

3. Latest after defining basic Testflow, do a save

ADVANTEST

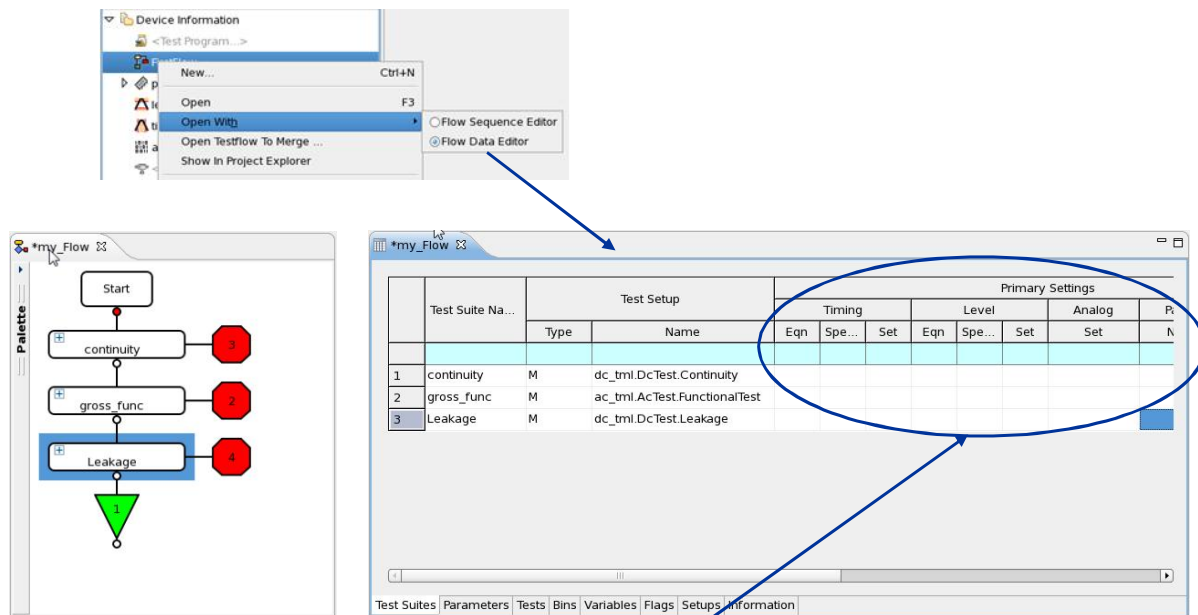
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 14

Top-Down Design Approach

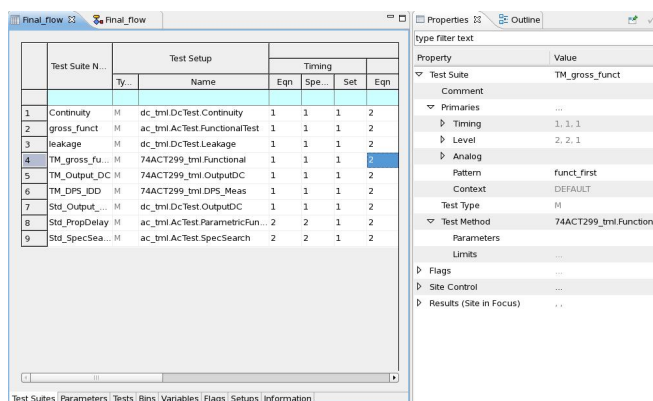
- After flow sequence is defined, open “Flow Data Editor”



- All Primaries are empty in new flow

ADVANTEST

Using Flow Data Editor for Primary Settings



- Sort and Filter functionality
- Click any field and check the properties view
- Navigate through Flow Data Editor like in a spreadsheet
- Select, copy & paste via windows default key function
 - copy <Ctrl> + <C>
 - paste <Ctrl> + <V>
 - undo <Ctrl> + <Z>
 - ...

ADVANTEST

Edit or assigning Primary Settings to Testsuite via Property Page

	Test Suite N...	Ty...	Test Setup Name	Timing			
				Eqn	Spe...	Set	Eqn
1	Continuity	M	dc_tml.DcTest.Continuity	1	1	1	2
2	gross_func	M	ac_tml.AcTest.FunctionalTest	1	1	1	2
3	leakage	M	dc_tml.DcTest.Leakage	1	1	1	2
4	TM_gross_fu...	M	74ACT299_tml.Functional	1	1	1	2
5	TM_Output_DC	M	74ACT299_tml.OutputDC	1	1	1	2
6	TM_DPS_IDD	M	74ACT299_tml.DPS_Meas	1	1	1	2
7	Std_Output_...	M	dc_tml.DcTest.OutputDC	1	1	1	2
8	Std_PropDelay	M	ac_tml.AcTest.ParametricFun...	2	2	1	2
9	Std_SpecSea...	M	ac_tml.AcTest.SpecSearch	2	2	1	2

Property	Value
Test Suite	TM_gross_func
Comment	
Primaries	...
Timing	1, 1, 1
Equation	1 [gross_func_eqn]
Spec/Wave	1 [gross_func_specs]
Set	1 [20MHz]
Level	2, 2, 1
Equation	2 [specs]
Spec/DPS	2 [Vcc=4.2V]
Set	1 [no termination]
Analog	
Pattern	funct_first
Context	DEFAULT
Test Type	M
Test Method	74ACT299_tml.Functional
Parameters	
Limits	
Flags	
Site Control	
Results (Site in Focus)	

- Primaries as property values

Pages of Flow Data Editor

If you need assistance:

Press <Shift> + <F1>

è TDC opens with related subjects

Primary Settings												
Test Suite Na...	Test Setup		Timing			Level			Analog	Pattern	Co	
	Type	Name	Eqn	Spe...	Set	Eqn	Spe...	Set	Set	Name	M	
1	Continuity	M	my_dc_classic_tm_project...	1	1	1	2	2	1		gross_func	DEFAU
2	gross_func	M	my_ac_classic_tm_project...	1	1	1	2	2	1		gross_func	DEFAU
3	leakage	M	my_dc_classic_tm_project...	1	1	1	2	2	1		gross_func	DEFAU
4	user_gross_func	M	my_user_project.user_digital...	1	1	1	2	2	1		gross_func	DEFAU
5	USer_DC_test	M	my_user_project.user_digital...	1	1	1	2	2	1		gross_func	DEFAU
6	USer DPS Meas	M	mv user project.user digital...	1	1	1	2	2	1		gross func	DEFAU

Test Suites | Parameters | Tests | Bins | Variables | Flags | Setups | Information

- Define Setup-Files e.g. Pin Config, Levels, Timing...
- Testflow Flags are sort of run-options during execution
- Define and edit Testflow Variables
- Result limit ranges for the tests defined in the test method
- Input parameters of test method
- Add, delete or edit test suites along with their Primary settings

Bin Pages of the Flow Data Editor

If you need assistance:

Press <Shift> + <F1>

è TDC opens with related subjects

	Software Bin	Software Bin Desc	Hardware Bin	Hardware Bin Desc	Type	Bin Color	Bin ReProbe	Bin OverOn
1	1	goodbin1	1	goodbin1	good	Green	no	yes
2	100	USB_Rx_Diff_HV_fail	3	Analog_fail	bad	Red	no	yes
3	101	DFR_X_HV_Fail	3	Analog_fail	bad	Red	no	yes
4	102	DFR_Y_HV_Fail	3	Analog_fail	bad	Red	no	yes
5	103	Z1_ADC_HV_Fail	3	Analog_fail	bad	Red	no	yes
6	104	Z2_ADC_HV_Fail	3	Analog_fail	bad	Red	no	yes

Software bin: The software bin number, which is a specific value that uniquely identifies the bin, not a count

Hardware bin: The hardware bin number. The hardware bin number is a positive integer (0...999), that is used by the Prober/Handler library for binning purposes after the test.

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 19

Viewing and Navigating the Properties Page

Double-click on Testsuite will open Properties view

Double-click on Testsuite will open Properties view

Instant Console Properties

type filter text

Property	Value
Test Suite	gross_func
Comment	
Primaries	...
Timing	1, 1, 1
Level	2, 2, 1
Analog	
Pattern	gross_func
Context	DEFAULT
Test Type	M
Test Method	my_ac_classic_tm_project.ClassAcTest.function
Parameters	
Limits	...
Flags	...
Site Control	...
Results (Site in Focus)	...

Signal

Vector #	Instruction	Comment	CP (PW)	CP (SC)	D50 (SC)	D57 (SC)	IO0 (SC)	IO1 (SC)	IO2 (SC)	IO3 (SC)	IO4 (SC)	IO5 (SC)	IO6 (SC)
0	reset		1	1	0	0	0	0	0	0	0	0	0
1	hold		1	1	0	0	X	X	X	X	X	X	X
2	par load 0x80		1	1	0	0	1	0	0	0	0	0	0
3	shift right		1	1	0	0	L	H	L	L	L	L	L
4			1	1	0	0	L	H	L	L	L	L	L
5			1	1	0	0	L	L	L	L	L	L	L
6			1	1	0	0	L	L	L	L	L	L	L
7			1	1	0	0	L	L	L	L	L	L	L
8			1	1	0	0	L	L	L	L	L	L	L
9			1	1	0	0	L	L	L	L	L	L	L
10			1	1	0	0	L	L	L	L	L	L	L
11			1	1	0	0	L	L	L	L	L	L	L
12			1	1	0	0	L	L	L	L	L	L	L

- Selecting value and pressing <F3>, will open associated editor
- E.g. Pattern Editor will open if pattern is selected

ADVANTEST

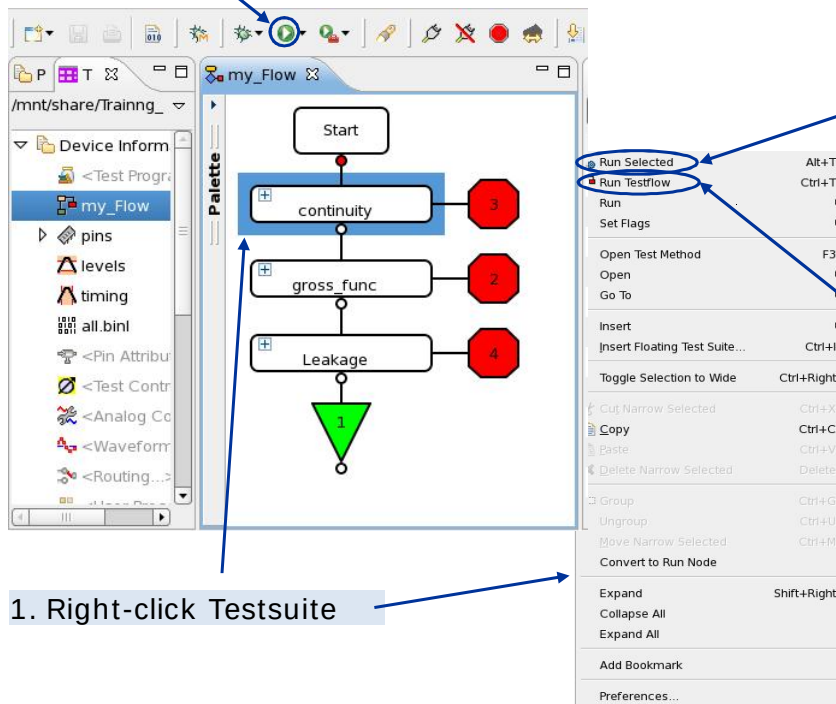
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 20

Executing Testflow and Single Testsuite

3. Will execute last executed run, even when Flow Sequence Editor is not in focus (e.g. in other Eclipse perspective)



2. Execute current Selection (e.g. currently selected Testsuites)

or

2. Execute overall Testflow

Using shortcuts and the run button make debugging most efficient!

ADVANTEST

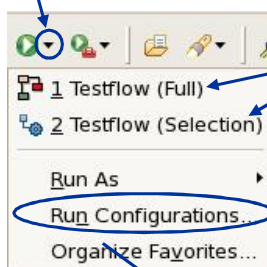
April 2016

All Rights Reserved - ADVANTEST CORPORATION

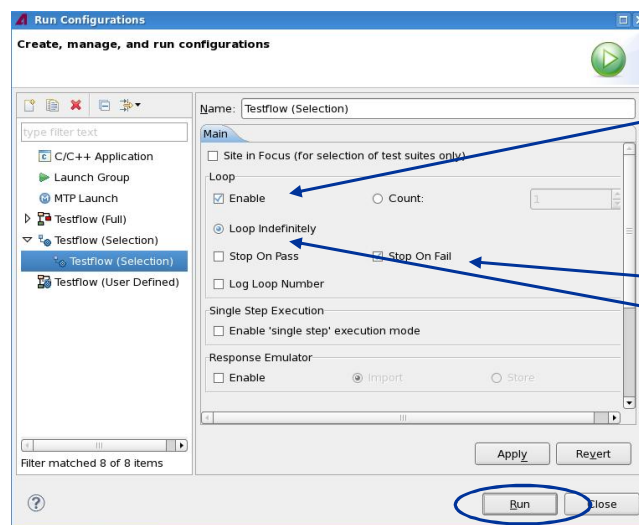
Testflow Unit 9 - 21

Testflow Run Options (Loops)

Open run-option



Alternative way of executing Tests (Full / Selection)



Looping tests

Loop until fail

Execute selected Flow (Full, Selection or User Defined) with options chosen above

ADVANTEST

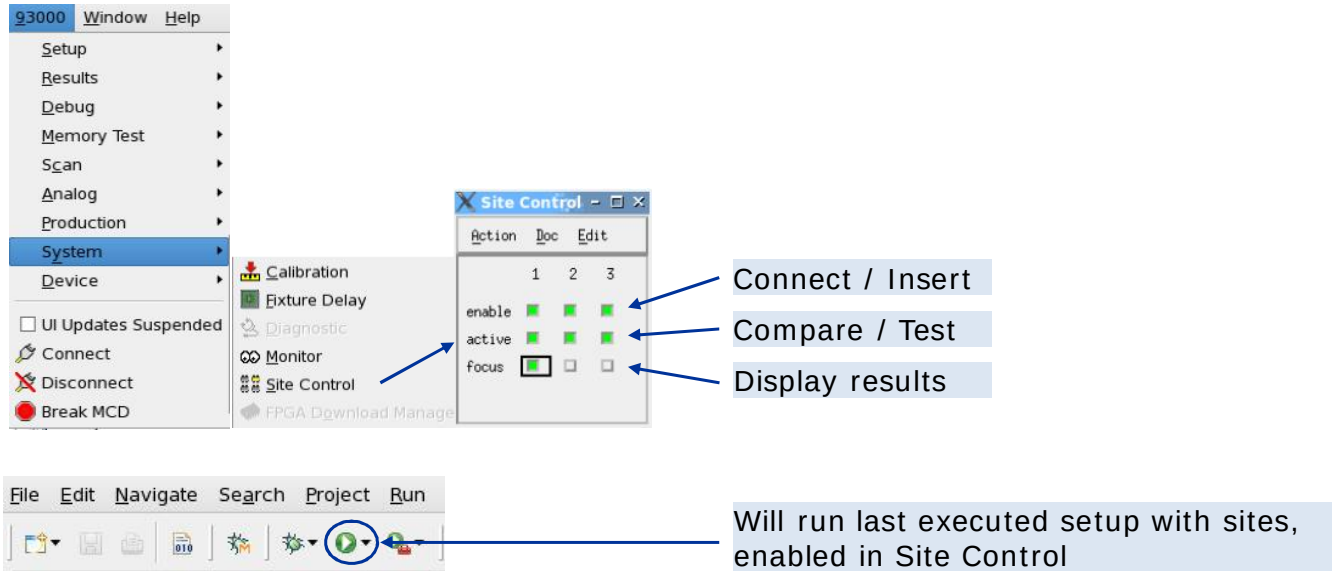
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 22

Multi-site and Single-site Execution

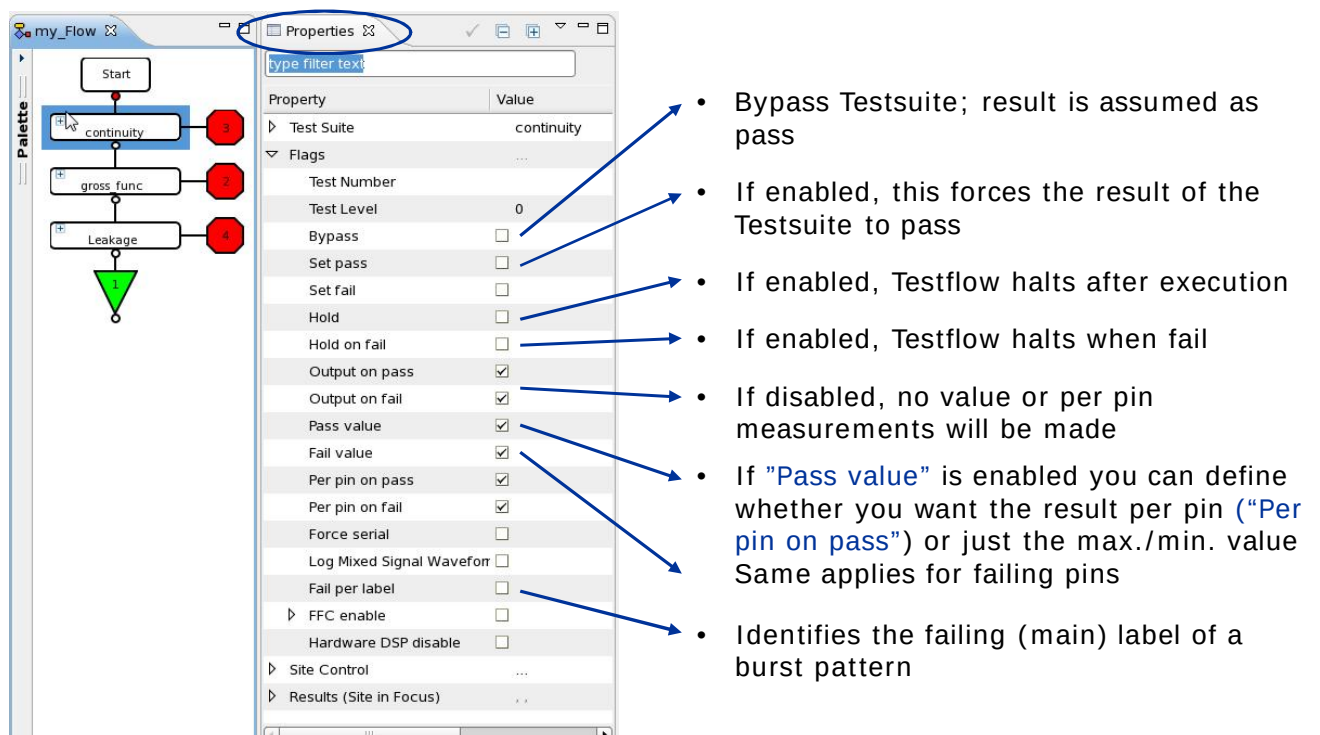
- For multi-site testing, use the Site Control window to select the sites to run your tests
- To get Site Control click the menu **93000 -> System -> Site Control**



ADVANTEST

Testsuite Flags

- ... are to control the test execution of a particular Testsuite
- ... are part of Properties section



ADVANTEST

Testflow Flags

... help you debugging and controlling the testflow

	Flags	Flag Type	Value
1	set_pass_level	TESTFLOW	0
2	set_fail_level	TESTFLOW	0
3	set_bypass_level	TESTFLOW	0
4	hold_on_fail	TESTFLOW	off
5	optimized_mode	TESTFLOW	on
6	global_hold	TESTFLOW	off
7	debug_mode	TESTFLOW	0 (Debug off)
8	debug_analog	TESTFLOW	off
9	parallel_mode	TESTFLOW	on
10	site_match_mode	TESTFLOW	auto(on)
11	global_overon	TESTFLOW	on
12	limits_enable	TESTFLOW	off
13	test_number_enable	TESTFLOW	on
14	test_number_inc	TESTFLOW	1
15	test_num_col_offset	TESTFLOW	1000000
16	test_number_manager	TESTFLOW	off
17	log_cycles_before	TESTFLOW	0
18	log_cycles_after	TESTFLOW	0
19	unburst_mode	TESTFLOW	off
20	sqst_mode	TESTFLOW	off
21	warn_as_fail	TESTFLOW	on
22	rf_cal_enable	TESTFLOW	on
23	rf_cal_interp	TESTFLOW	off
24	rf_dernbedCal_enable	TESTFLOW	off
25	use_hw_dsp	TESTFLOW	off
26	dsp_file_enable	TESTFLOW	off
27	buffer_testflow_Log	TESTFLOW	off
28	check_testmethod_api	TESTFLOW	off

Flow Data Editor, flags page sorted for TESTFLOW-flags

- Execute high-throughput versions of your Testflow e.g. Re-use DC-settings as in Flex-DC
- Switch between different levels of debug information
- Testflow continues after stop bins that are marked as overon bins

ADVANTEST

System Flags

... help you, control the Testflow

	Flags	Flag Type	Value
1	report_to_file	SYSTEM	on
2	report_to_printer	SYSTEM	off
3	datalog_to_file	SYSTEM	on
4	datalog_to_printer	SYSTEM	off
5	datalog_to_report_win	SYSTEM	off
6	datalog_formatter	SYSTEM	off
7	datalog_sample_size	SYSTEM	1
8	graphic_result_display	SYSTEM	on
9	state_display	SYSTEM	off
10	print_wafermap	SYSTEM	off
11	ink_wafer	SYSTEM	off
12	max_reprobes	SYSTEM	1
13	temp_monitor	SYSTEM	on
14	calib_age_monitor	SYSTEM	on
15	diag_monitor	SYSTEM	on
16	current_monitor	SYSTEM	on
17	log_events_enable	SYSTEM	on
18	stdf_generation	SYSTEM	on
19	tm_crash_as_fatal	SYSTEM	off

Flow Data Editor, flags page sorted for SYSTEM-flags

If you need assistance:
Press <Shift> + <F1>
è TDC opens with related subjects

- Datalog sampling setting: log every n-th device to save test time, for all remaining devices only binning information available
- Enable/Disable event logging for Event Datalogging
- Standard Test Data Formatter enabled / disabled

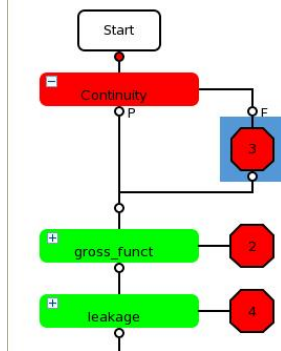
ADVANTEST

Example: How to continue the testflow execution after a fail

	Flags	Flag Type	Value
1	set_pass_level	TESTFLOW	0
2	set_fail_level	TESTFLOW	0
3	set_bypass_level	TESTFLOW	0
4	hold_on_fail	TESTFLOW	off
5	optimized_mode	TESTFLOW	on
6	global_hold	TESTFLOW	off
7	debug_mode	TESTFLOW	0 (Debug off)
8	debug_analog	TESTFLOW	off
9	parallel_mode	TESTFLOW	on
10	site_match_mode	TESTFLOW	auto(on)
11	global_overon	TESTFLOW	on
12	limits_enable	TESTFLOW	off
13	test_number_enable	TESTFLOW	on
14	test_number_inc	TESTFLOW	1
15	test_num_col_offset	TESTFLOW	1000000
16	test_number_manager	TESTFLOW	off
17	log_cycles_before	TESTFLOW	0
18	log_cycles_after	TESTFLOW	0
19	unburst_mode	TESTFLOW	off
20	sqst_mode	TESTFLOW	off
21	warn_as_fail	TESTFLOW	on
22	rf_cal_enable	TESTFLOW	on
23	rf_cal_interp	TESTFLOW	off
24	rf_dembedCal_enable	TESTFLOW	off
25	use_hw_dsp	TESTFLOW	off
26	dsp_file_enable	TESTFLOW	off
27	buffer_testflow_Log	TESTFLOW	off
28	check_testmethod_api	TESTFLOW	off

1. Set the testflow flag “Global Overon” to “on”

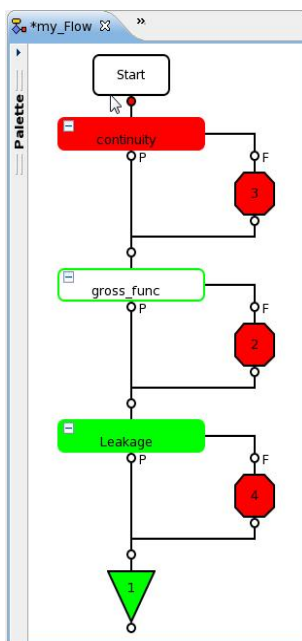
2. A local overon flag can be set for each testsuite



type filter text	
Property	Value
▼ Software	
Bin Number	3
Description	Failed Continuity
▼ Hardware	
Bin Number	3
Description	
Bin Type	bad
▼ Attribute	
Color	Red
Reprobe	☐
Overon	☒

ADVANTEST

Impact of Testflow Flags



• Testflow was started with <Ctrl> + <T> or Run Testflow

• Testsuite continuity failed
“global_overon” and bin 3 “Overon” flags were set, otherwise Testflow execution would have finished at the stop bin 3

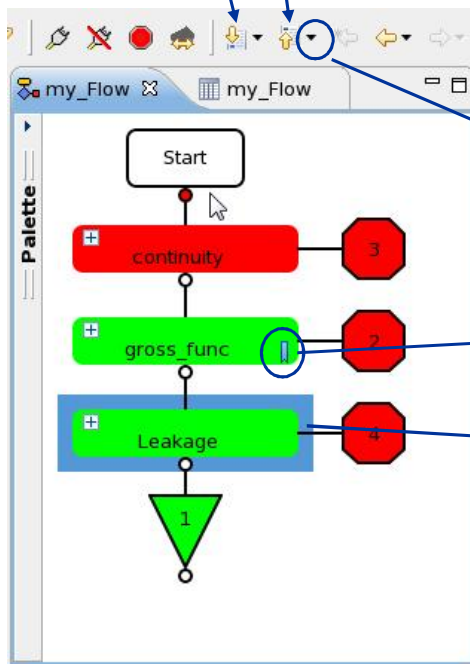
• Testsuite gross_func is set to pass, no test executed

• Testsuite Leakage passed

ADVANTEST

Navigating a Testflow

Jump forward/backward to next Annotation (defined stop-position)



Possible Annotations:

- ☒ Bookmarks
- ☒ Executed and Failed
- ☐ Executed and Passed
- ☐ Failed
- ☐ Invalid
- ☐ Linked
- ☐ Passed

Enable/disable annotation by selecting/unselecting the box

• Added Bookmark

Right-click on selected Testsuite:

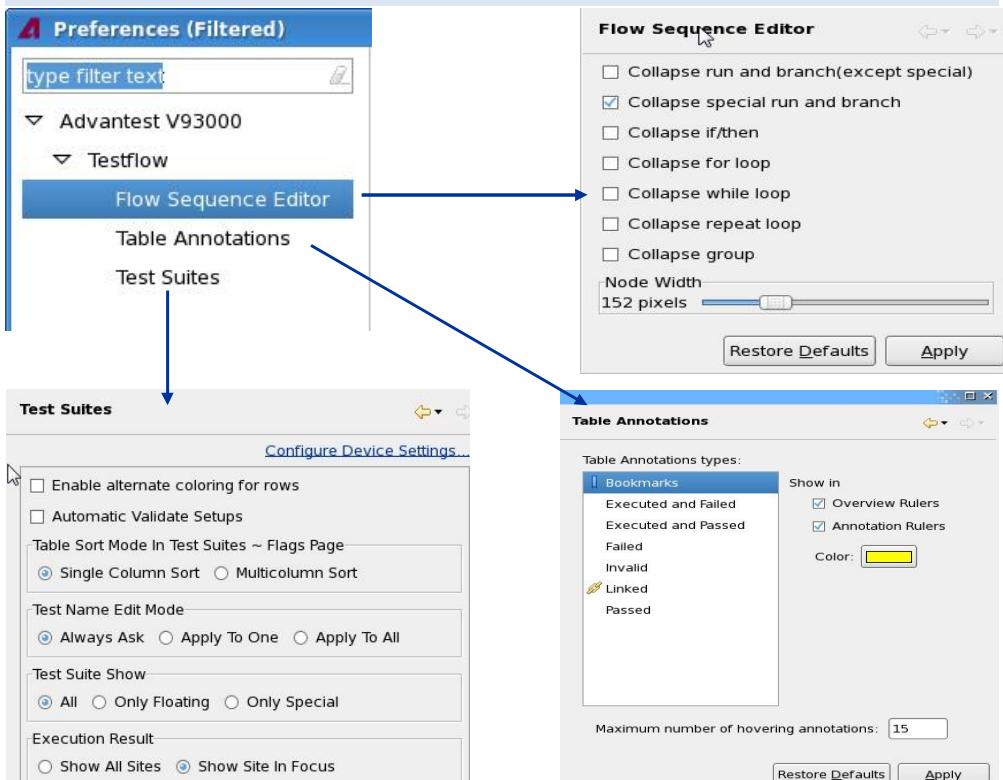
- Move Narrow Selected Ctrl+M
- Convert to Run Node
- Expand Shift+Right
- Collapse All
- Expand All
- Add Bookmark**
- Preferences...

• Add Bookmark

ADVANTEST

Customize Testflow Appearance

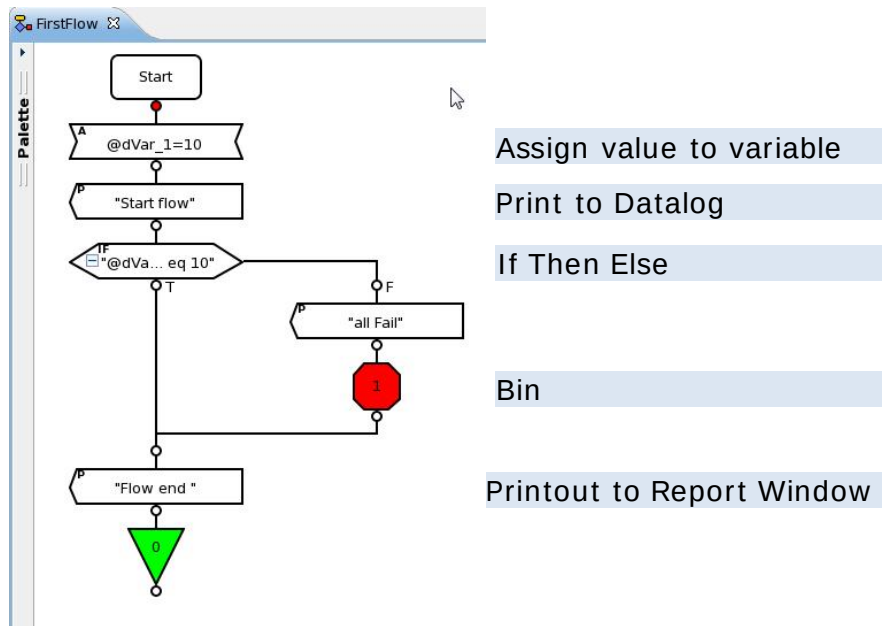
Right-click into Testflow-Editor and select "Preferences..."



ADVANTEST

Advanced Testflow Features

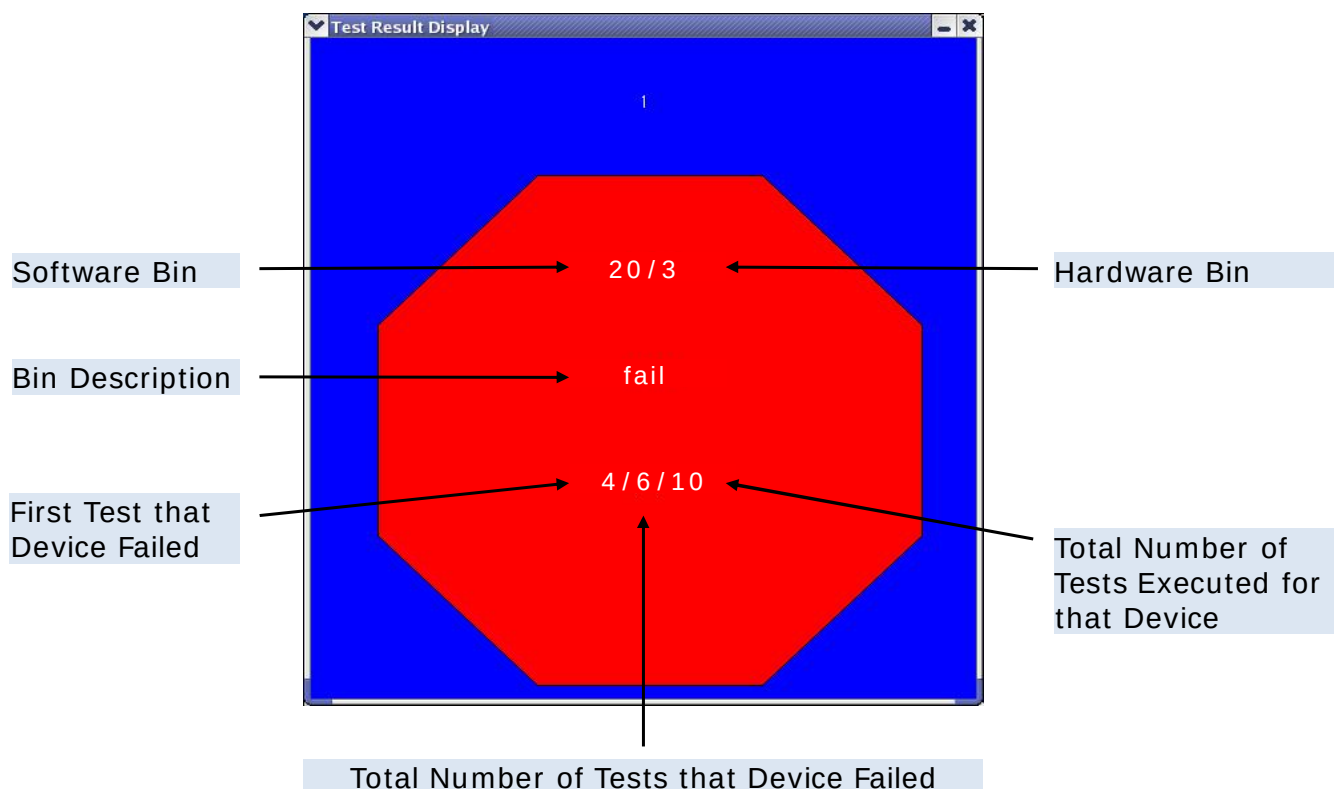
Using Testflow Variables



Testflow variables are site-dependent!

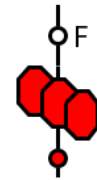
ADVANTEST

Binning Results Legend



ADVANTEST

Multibin



- Allow **multiple and flexible bin assignments**

Once a multibin is set, failing tests of following test suites with multibin nodes won't log to the multibin.

=> At the end of the Testflow, the bin displayed is the first multibin set.

- Two ways to set multibins :

SET_MULTIBIN API

Note :The bin set must exist as a normal or floating bin in the testflow.

Test Table

Depending of the failing site, the bin logged will be the one defined in the Test Table

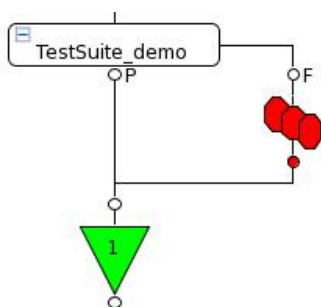
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 33

Multibin with Test Table



1 – Insert a Multibin in the flow

2 – Define which bin will be set in code :

Example :

```
ON FIRST_INVOCATION_BEGIN();  
...  
ON_FIRST_INVOCATION_END();
```

- 1 TestSet().cont(true).judgeAndLog_FunctionalTest("funcTest", tmLimits);
- 2 TestSet().cont(true).judgeAndLog_ParametricTest(pin1,"param1",tmLimits,value1);
- 3 TestSet().cont(true).judgeAndLog_ParametricTest(pin2,"param2",tmLimits,value2);

The **FIRST** failing TestSet will set the multibin according with Test Table.

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Testflow Unit 9 - 34

Multibin with Test Table

	Suite name	Test name	Bin_s_num	Bin_s_name	Bin_h_num	Bin_h_name
	Test mode					
1	TestSuite_demo	Functional	2		2	NOTOK
2	TestSuite_demo	param1	3		2	NOTOK
3	TestSuite_demo	param2	4		2	NOTOK

↓
Soft bin

↓
Hard bin

- The multibins value will be defined according the failing Test of the TestSuite_demo.
- The order of the TestSets is important.

1	2	3	
Functional	param1	param2	Multibins
Fail	Pass	Pass	2/2
Pass	Fail	Fail	3/2

ADVANTEST

Lab Exercise

Flow Sequence Editor lab

ADVANTEST

Reference

- V93000 Technical Documentation Center (TDC):
[Topic 20223](#)

Advantest V93000

Digital User Training

Universal Test Methods - Introduction

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Universal Test Method Overview
- Creating a Universal Test Method
- Using the Universal Test Method in a Testflow
- Multi Site Basics
- Test Method Libraries

ADVANTEST[®]

Learning Objectives

- Understand the concept of Test Methods
- Know how to create Project, Test Method Class using the creation wizard
- Know how some typical Test Method APIs for digital applications
- Know how to build Test Methods and use them in a Testflow
- Know how create multi-site test methods
- Know how to use test method libraries

Relevance

- Test Methods creation is one of the main tasks during test program development
- Every V93000 user must be able to create new Test Methods or use existing Test Methods

What are Universal Test Methods

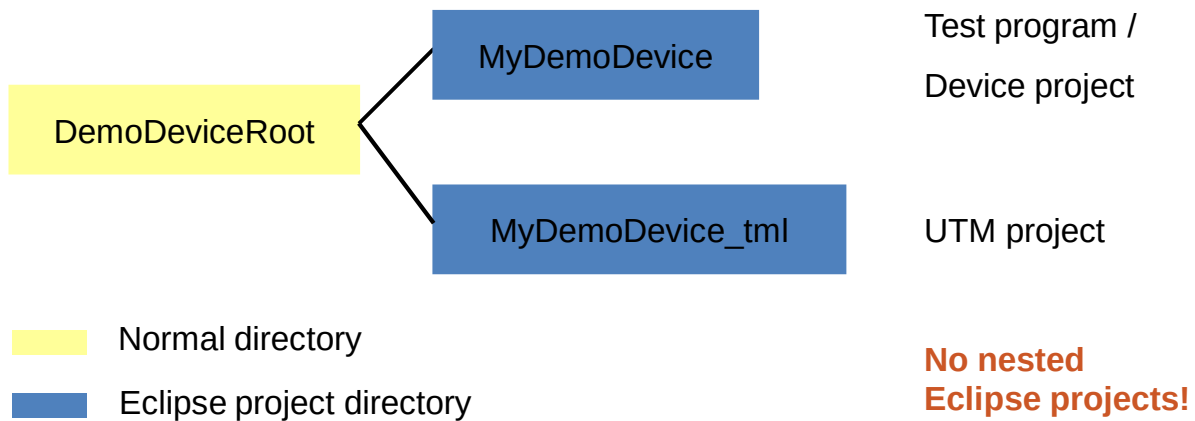
- Universal Test Methods (UTM) is a C++ based test method framework
- Each Test Method is one class
- The Test Method of each test suite is represented as one object
- For each site the void *run(void)* member function is called
- Test Method parameters are handled as member variables

UTM Class

- There is one Test Method per class
- Two member functions of this class required for a minimal UTM
 - Member function *run()* is called every time a Test Method is executed;
 - Member function *initialize()* is called when a new test suite is created; it sets up the parameter interface to the test flow

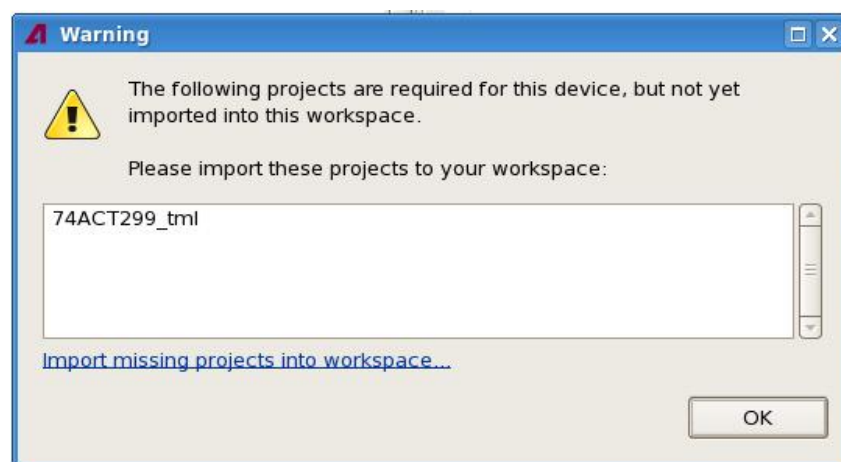
UTM Structure

- The device is treated as an Eclipse project in SmarTest
- UTMs are different Eclipse projects which have to be put outside of device folder
- Suggestion is to create a parent (non project directory) with a UTM directory and a device project directory



ADVANTEST®

UTM Structure



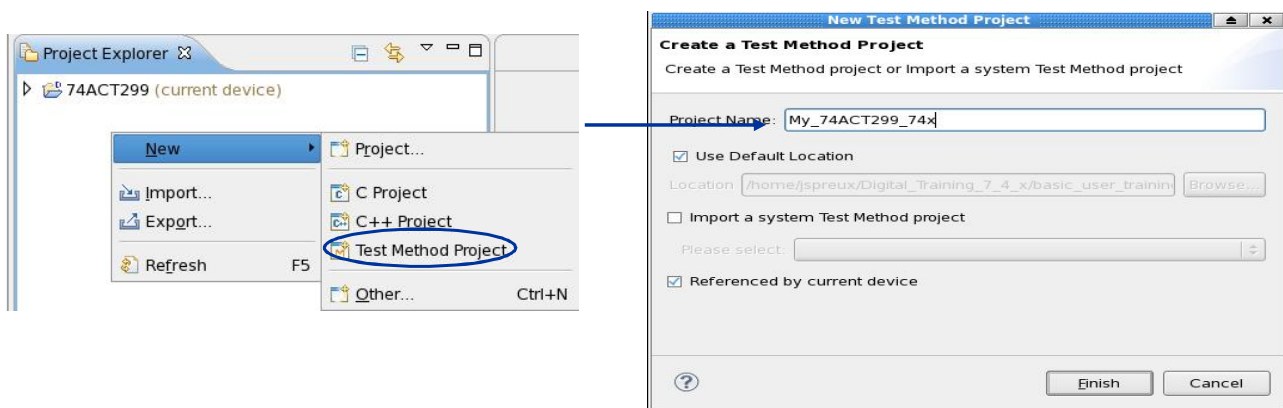
ADVANTEST®

UTM Creation Procedure

1. Create Universal Test Method Project
2. Create Universal Test Method Class
3. Set up the input parameters and test limits
4. Add code to the Test Method program
5. Build (compile, link and register) Shared Library
6. Add the Test Method to a Testflow

1. Creating a Universal Test Method Project

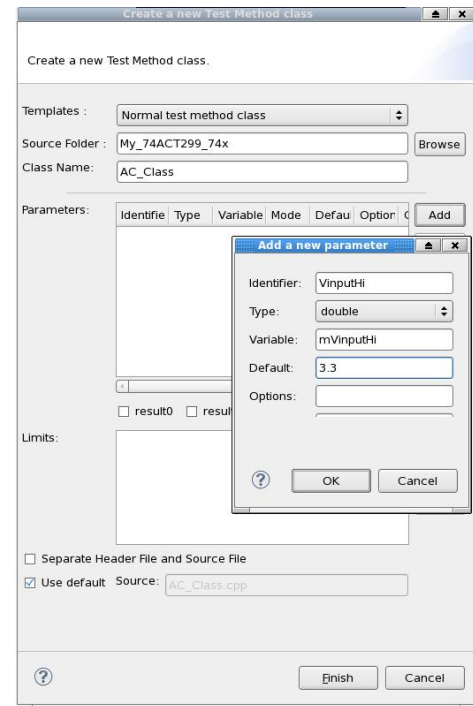
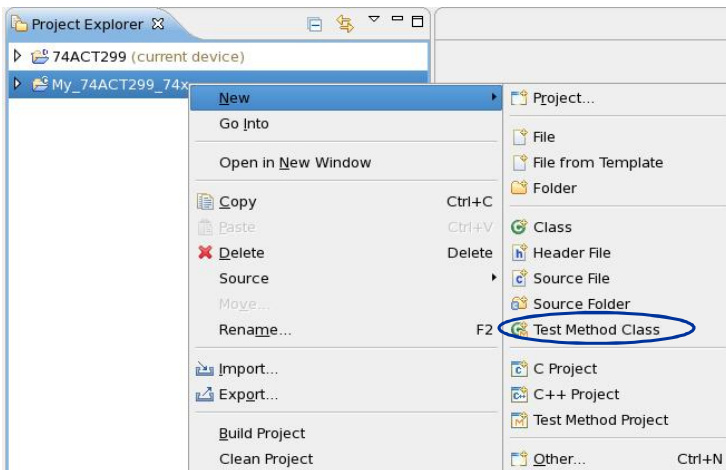
File -> New -> Test Method Project



- Default Location is on same Directory-level as Device
- Referenced by current device will make the library available within Testsuites immediately

2. Creating a Universal Test Method Class

New -> Test Method Class



ADVANTEST®

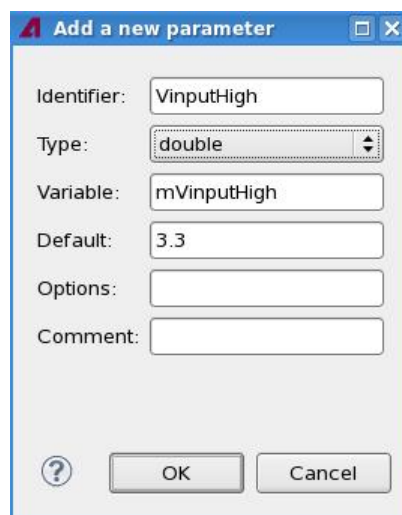
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 11

3. Universal Test Method Parameters

In Parameters section:



Possible Types



Parameter types:

- **PinString**: Pins, Pinlists and Pingroups can be entered; content assist available for selection
- **SpecVariable**: Level- or Timing- Spec Variables to be entered; content assist available
- **SpecValue**: is in the format of value[unit]
- **double, int & string**: is standard format

ADVANTEST®

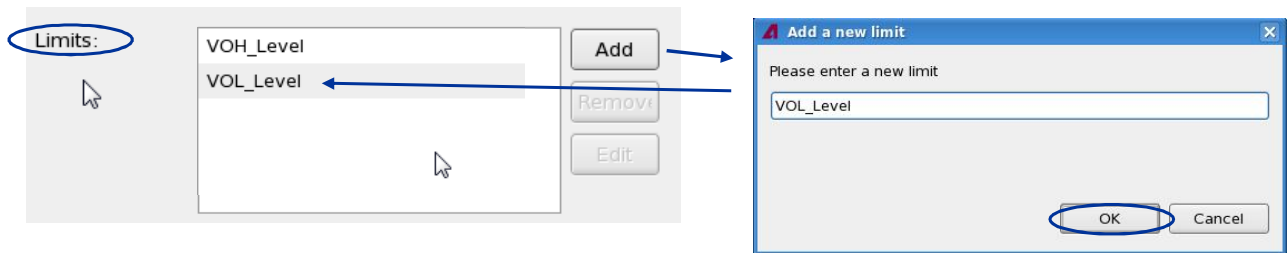
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 12

3. Universal Test Method Limits

In Limits section:



- To add Limits to the Test Method, click **Add** on the right side of the Limits area
- Only Limit names can be specified here
- Limit values are set and modified in the Flow Data Editor or Testsuite Properties View

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 13

4. Adding Code to the Test Method Program

- Upon clicking **Finish** in “Create a new Test Method class” a new source file is created and opened

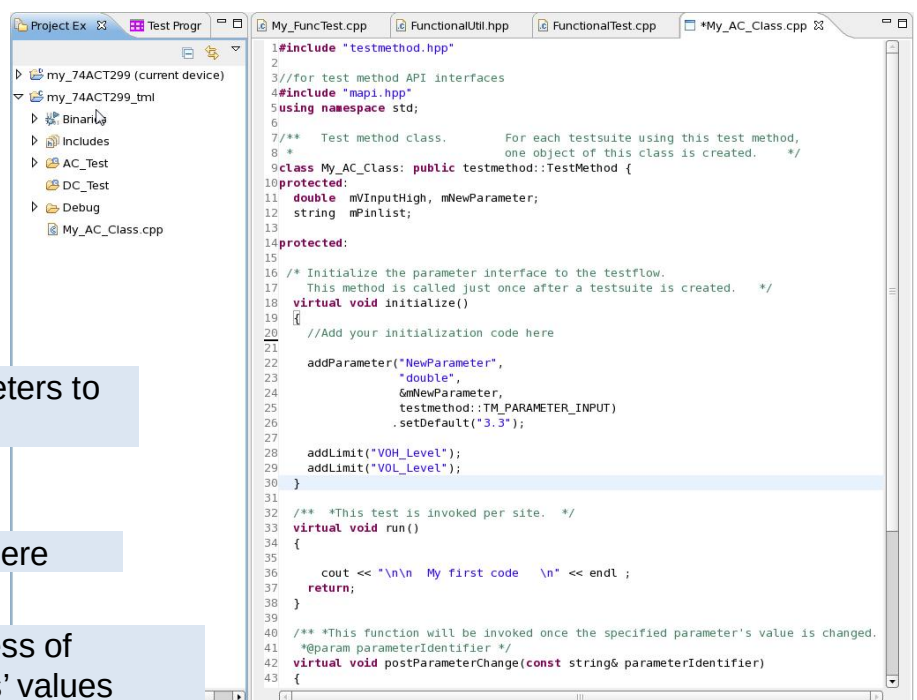
Test Method project

Class source file

This is how you add parameters to the properties view

Application TM code goes here

Optional: Enable post process of parameters and parameters' values



ADVANTEST

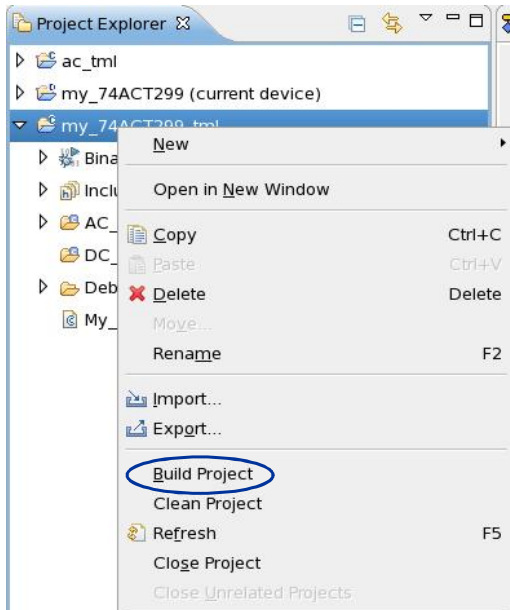
April 2016

All Rights Reserved - ADVANTEST CORPORATION

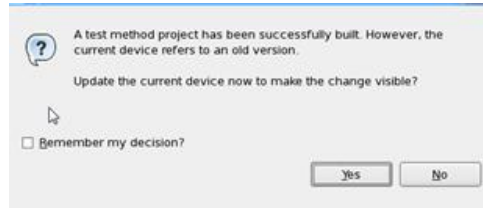
Universal Test Methods Introduction Unit 10 - 14

5. Building the UTM project

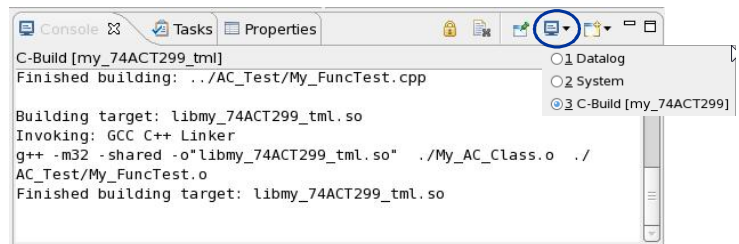
In Project Explorer,
right-click the Project



Update Device Project
with built TML Project



Check C-Build console for errors



Alternatively press <Ctrl>+

ADVANTEST

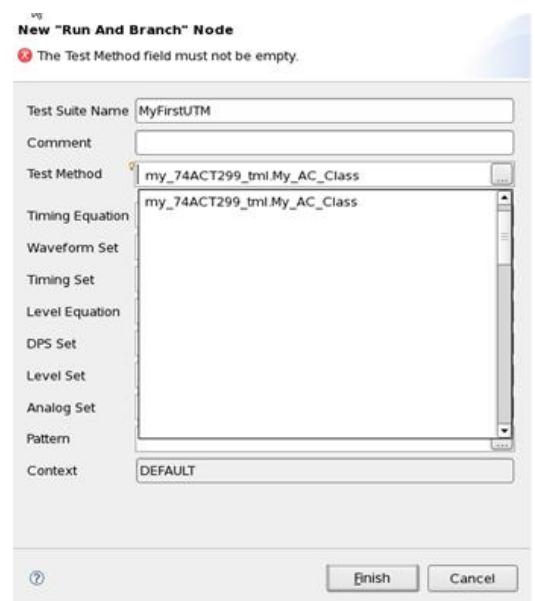
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 15

6. Add UTM to a Testflow

- In “Flow Sequence Editor” insert a new Testsuite
- <CTRL>+<Space> opens the list of available UTMs
- Select UTM
- Define primaries in here or through flow data editor
- Press **Finish** and run Testflow



ADVANTEST

April 2016

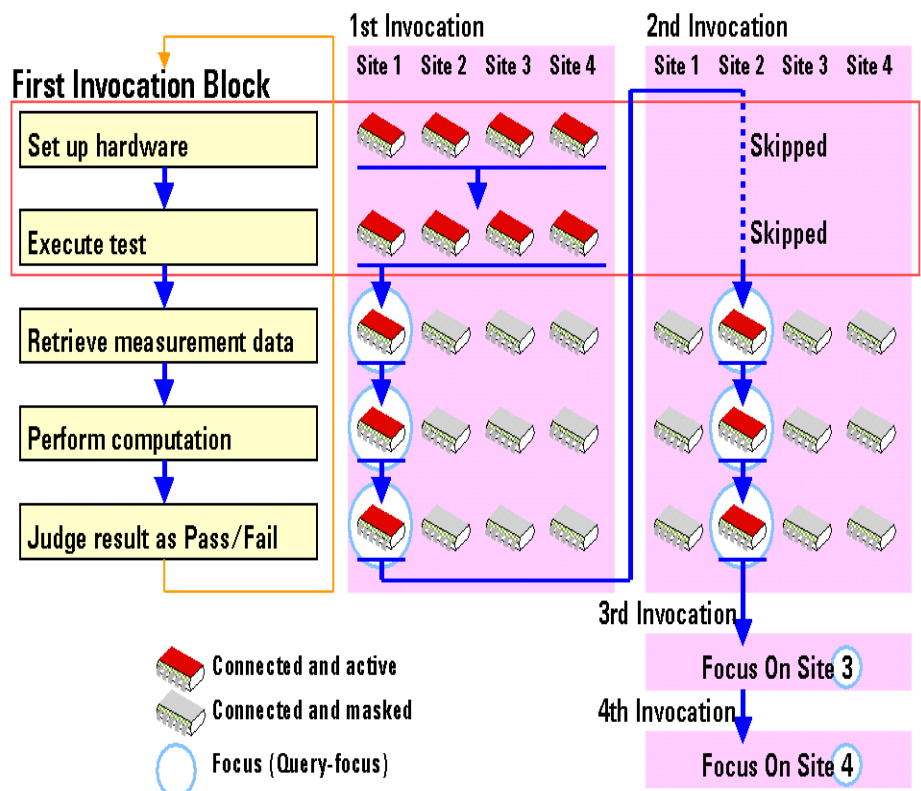
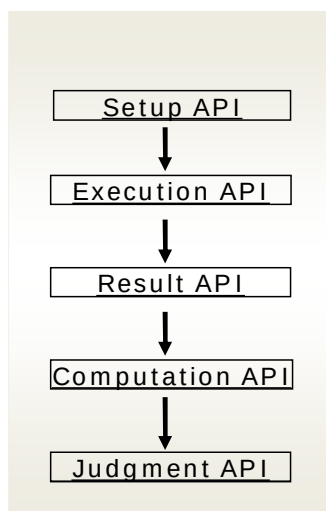
All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 16

Multi Site testing

Parallel Execution and API

Generic Test Method flow



TM Programming Strategy in Multisite

General TM programming strategy:

- Ø Setup
- Ø Execute
- Ø Get Result
- Ø Calculate
- Ø Judge

- The **Setup** and **Execute** steps involve hardware. These steps may be done in parallel when they are placed within these special multisite APIs:
 - **ON_FIRST_INVOCATION_BEGIN()**
 - **ON_FIRST_INVOCATION_END()**
- The Get Result, Calculate and Judge are performed on work station and may not be done in parallel.

ADVANTEST[®]

ON_FIRST_INVOCATION Blocks

An **ON_FIRST_INVOCATION** block is a sequence of instructions in your test method that will *only* be executed when the test method is invoked for the first site under test.

- Hardware setup and execution actions are executed in *parallel* when executed within an ON_FIRST_INVOCATION block.
- There may be more than one **ON_FIRST_INVOCATION** block within one test method.
- The test method APIs **ON_FIRST_INVOCATION_BEGIN()** and **ON_FIRST_INVOCATION_END()** are ignored in a single-site configuration.

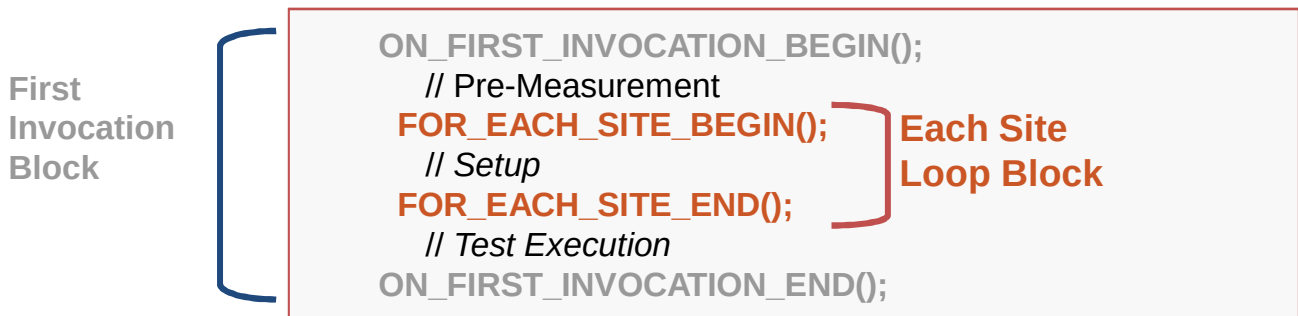
ADVANTEST[®]

FOR_EACH_SITE Block

Two APIs are used to create an “FOR_EACH_SITE” block :

FOR_EACH_SITE_BEGIN()
FOR_EACH_SITE_END()

EACH_SITE_LOOP blocks should always be located *within* ON_FIRST_INVOCATION blocks.



General Guideline

- Put Setup APIs executed for **all enabled** sites inside the ON_FIRST_INVOCATION Blocks
- Put CONNECT, DISCONNECT, and routing setup APIs executed for the **all enabled** sites inside the ON_FIRST_INVOCATION Blocks
- Put Execution APIs executed for **all enabled** sites inside the ON_FIRST_INVOCATION Blocks
- Put Setup APIs executed for **setup-focused** sites inside the FOR_EACH_SITE_BEGIN Blocks

Site Specific Setup APIs

- Level/Timing specification value
 - Primary.getLevelSpec().change(param)
 - Primary.getLevelSpec().restore()
- FlexDC APIs support different settings on different sites

Example – Site Specific Settings

```
ON_FIRST_INVOCATION_BEGIN();
setting.pin("pin1, pin2").iRange(100 uA);
setting.pin("pin1").iForce(10 uA).min(50 mV).max(100 mV);
setting.pin("pin2").vForce(1 mV).min(10 uA).max(20 uA);
meas.pin ( "pin1,pin2" ).execMode( TM::PVAL );

tasks.add( setting ).add( meas ).setup();

FOR_EACH_SITE_BEGIN();
DOUBLE refV;
GET_USER_DOUBLE("refV", &refV);
setting.pin("pin2").vForce(refV + 0.5 V);
FOR_EACH_SITE_END();

tasks.execute();
ON_FIRST_INVOCATION_END();

DOUBLE val1 = meas.getValue( "pin1" );
DOUBLE val2 = meas.getValue( "pin2" );

TEST( "pin1" , "testname", val1);
TEST( "pin2" , "testname", val2);
```

To set different value for each site:

FOR_EACH_SITE_BEGIN() and
FOR_EACH_SITE_END() are available

Download setting to HW. Use the setup() method*.
-> Fix MSET !
-> Do this *before* the site-specific settings.

Getting a site specific measurement result from a former test suite.

Setting the site specific measurement result as a force value.

- _Execute measurement once in parallel for all sites
- Execute value return and logging by site

*** Important :**
When using different Force Values by devices or sites, the 'SETUP' command must be used
(if not used the software will run out of MSETs as if will assign a new MSET for each new value)

Execution APIs for All Enabled Sites

EXECUTE_TEST()
START_TEST()/WAIT_TEST_DONE()
DIGITAL_CAPTURE_TEST()
FUNCTIONAL_TEST()
SEARCH_FUNC_TASK()
Sequencer.function()
TASK_LIST()
.....

Test Method Libraries

- Universal Test Method Libraries are collections of standard Test Methods that are provided with the SmarTest software
- These Test Methods can be used for specific tests and measurements with or without modification
- Pre-defined libraries are deployed under the directory
/opt/hp93000/soc/testmethod/sh_lib/
- Universal Test Method Library source files are under the directory
/opt/hp93000/soc/testmethod/library/

Library location in File System

```
-rw-r--r-- 1 bin bin 50405 Jan 27 22:10 SMC_scan_tml.zip
-rw-r--r-- 1 bin bin 229339 Jan 27 22:10 ac_tml.zip
-rw-r--r-- 1 bin bin 600648 Jan 27 22:10 ccf_tml.zip
-rw-r--r-- 1 bin bin 43383 Jan 27 22:10 d2s_tml.zip
-rw-r--r-- 1 bin bin 121349 Jan 27 22:10 dc_tml.zip
drwxr-xr-x 4 root root 4096 Feb 1 10:21 include
-rw-r--r-- 1 bin bin 37552 Jan 27 22:10 miscellaneous_tml.zip
-rw-r--r-- 1 bin bin 180443 Jan 27 22:10 multi_group_dc_tml.zip
-rw-r--r-- 1 bin bin 58331 Jan 27 22:10 pptia_tml.zip
-rw-r--r-- 1 bin bin 68 Jan 27 22:10 properties
-rw-r--r-- 1 bin bin 35534 Jan 27 22:10 scan_tml.zip
drwxr-xr-x 3 root root 4096 Feb 1 10:21 sh_lib
-rw-r--r-- 1 bin bin 13893 Jan 27 22:10 tci_tml.zip
-rw-r--r-- 1 bin bin 103799 Jan 27 22:10 testmethodSnippet.xml
-rw-r--r-- 1 bin bin 57089 Jan 27 22:10 tmu_tml.zip
socbu173:/opt/hp93000/soc/testmethod/library $ █

-r-xr-xr-x 1 bin bin 399472 Jan 27 22:10 libSMC_scan_tml.so
-r-xr-xr-x 1 bin bin 1713853 Jan 27 22:10 libac_tml.so
-r-xr-xr-x 1 bin bin 364659 Jan 27 22:10 libd2s_tml.so
-r-xr-xr-x 1 bin bin 813691 Jan 27 22:10 libdc_tml.so
-r-xr-xr-x 1 bin bin 240264 Jan 27 22:10 libmiscellaneous_tml.so
-r-xr-xr-x 1 bin bin 1879666 Jan 27 22:10 libmulti_group_dc_tml.so
-r-xr-xr-x 1 bin bin 246888 Jan 27 22:10 libpptia_tml.so
-r-xr-xr-x 1 bin bin 68747 Jan 27 22:10 libpretriggering_tml.so
-r-xr-xr-x 1 bin bin 189733 Jan 27 22:10 libscan_tml.so
-r-xr-xr-x 1 bin bin 26479 Jan 27 22:10 libsmartCalc.so
-r-xr-xr-x 1 bin bin 76036 Jan 27 22:10 libswitching_Tj_Tp_tml.so
-r-xr-xr-x 1 bin bin 359069 Jan 27 22:10 libtmu_tml.so
drwxr-xr-x 2 root root 4096 Feb 1 10:21 preload
socbu173:/opt/hp93000/soc/testmethod/library/sh_lib $ █
```

ADVANTEST

Pre-defined Test Method Libraries

dc_tml DC Methods list

Continuity:	checks for connectivity and short circuits on the DUT and the DUT interface
DpsConnectivity:	checks that the force and sense wires of the DPS are connected properly on the DUT board
DpsStatus:	provides status information about DPS pins
DVM:	measures the voltage at any pin
GeneralPMU:	allows direct control of the Parametric Measurement Unit (PMU) in order to force a voltage (and measure current) or force a current (and measure voltage)

ADVANTEST

Pre-defined Test Method Libraries

dc_tml DC Methods list continued

HighZ:	measures the leakage current of outputs in high-z condition
Leakage:	measures the leakage current on DUT pins
OperatingCurrent:	measures the active operating current of the DUT
ProductionIddq:	measures the quiescent power supply current of the DUT
StandbyCurrent:	measures the standby current of the DUT

Pre-defined Test Method Libraries

ac_tml AC Methods list

FunctionalTest:	examines the DUT at nominal level and timing settings
SpecSearch:	characterizes the device by shifting the specs defined in the equation-based test setup
Parametric FunctionalTest :	characterizes the DUT with a customized functional test
Frequency_by DigitalCapture:	measures frequency by digital capture on the designated DUT pins

Pre-defined Test Method Libraries

ac_tml AC Methods list continued

JitterHistogram:	measures the jitter on the output of a device
JitterSeparation:	measures the jitter on the device output and separates the random jitter and deterministic jitter
JitterStartdust:	measures the jitter on the output of a device and calculates the jitter frequency and spectrum
Shmoo:	performs a shmoo measurement in the specified range of X-axis parameter and Y-axis parameter
SetBasedSpecSearch:	performs spec search based on the pre-calculated level/timing set and pattern (burst)

Note: additional Test Methods available in ac_tml library

ADVANTEST[®]

Miscellaneous Test Methods

Disconnect:	Disconnects the test system from the DUT
Connect:	Connects the test system to the DUT
SetupLevelSet0:	Programs the level set of tri-state highZ state
TestState:	Returns the current sequencer state
StartSequencer:	Starts the vector sequencer
TestheadTemperature:	Reads the temperature of the test head
SetUtilityLines:	Sets the status of the utility lines
GetUtilityLines:	Gets the status of the utility lines
FwEscape:	Executes a firmware command
ExecuteLinuxCommand:	Executes a Linux shell command

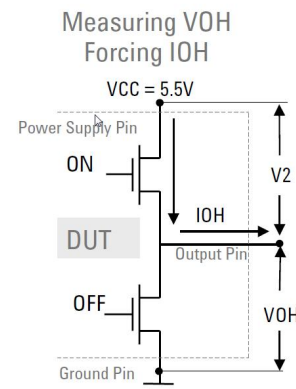
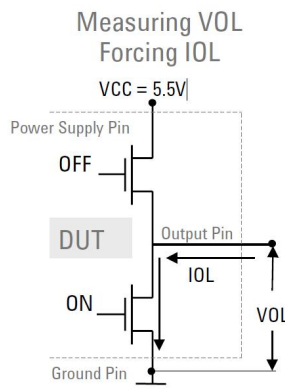
ADVANTEST[®]

Create Testsuite Using Standard Test Method Library

Device's Output Voltage Characteristics

Symbol	Parameter	V _{CC} (V)	T _A = 25°C		T _A = -40°C to +85°C		Units	Conditions
			Typ	Guaranteed Limits				
V _{OH}	Minimum HIGH Level	4.5	4.49	4.4	4.4		V	I _{OUT} = -50 μA
		5.5	5.49	5.4	5.4			
		4.5	0.0001	3.86	3.76		V	V _{IN} = V _{IL} or V _{IH}
		5.5		4.86	4.76			I _{OH} = -24 mA
V _{OL}	Maximum LOW Level Output Voltage	4.5	0.001	0.1	0.1		V	I _{OUT} = 50 μA
		5.5	0.001	0.1	0.1			
		4.5		0.36	0.44		V	V _{IN} = V _{IL} or V _{IH}
		5.5		0.36	0.44			I _{OL} = 24 mA
								I _{OL} = 24 mA (Note 5)

Use e.g. Test Method
Output DC Voltage of
dc_tml



ADVANTEST

April 2016

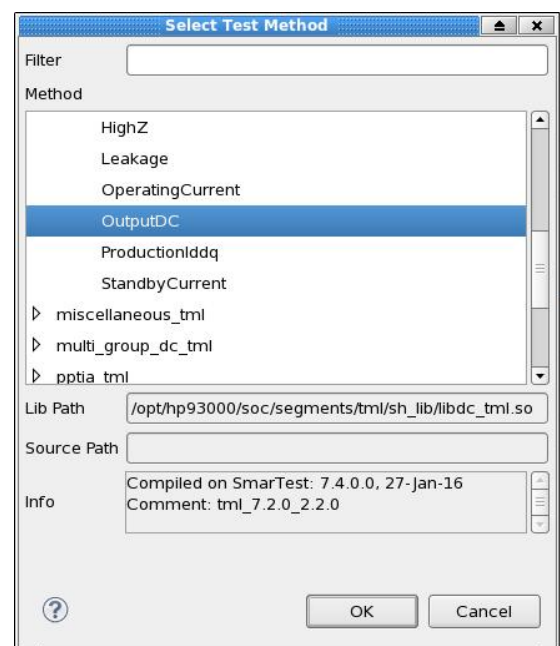
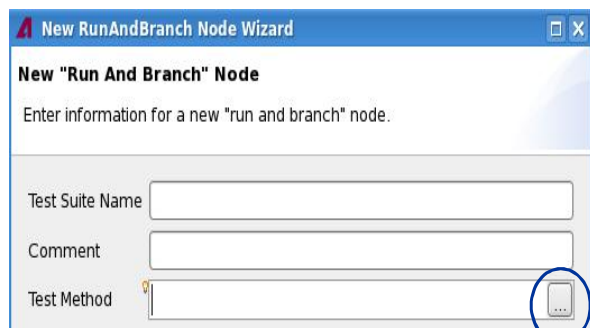
All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 33

Standard Test Method Called by Testsuite

Add Run and Branch Testsuite

While inserting a Test Method of a Standard Universal Test Method Library, the system provides you detailed information of library location



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 34

Test Method Parameter Definition

After selecting Test Method, define parameters through Properties GUI

Property	Value
Test Suite	Std_Output_Levels
Comment	
Primarys	...
Timing	1, 1, 1
Level	2, 2, 1
Analog	
Pattern	gross_func
Context	DEFAULT
Test Type	M
Test Method	dc_tml.DcTest.OutputDC
Parameters	all_out, PPMU, BOTH, 24[mA], -24[mA], 2[ms], 2[ms]
pinlist	all_out
mode	PPMU
measuredLevel	BOTH
forceCurrentLow	24[mA]
forceCurrentHigh	-24[mA]
settlingTimeLow	2[ms]
settlingTimeHigh	2[ms]
spmuClampVolta	0[V]
spmuClampVolta	0[V]
vectorRange	30
testName	(OutputDC_LowLevel[V],OutputDC_HighLevel[V])
output	ReportUI
Limits	...
Flags	...
Site Control	...
Results (Site in Focus)	...

Modes are part of the parameter list in Universal Test Method

PPMU
PPMU
SPMU
PPMUTerm
SPMUTerm
ProgramLoad

ADVANTEST®

Define Vector Range for OutputDC-Test

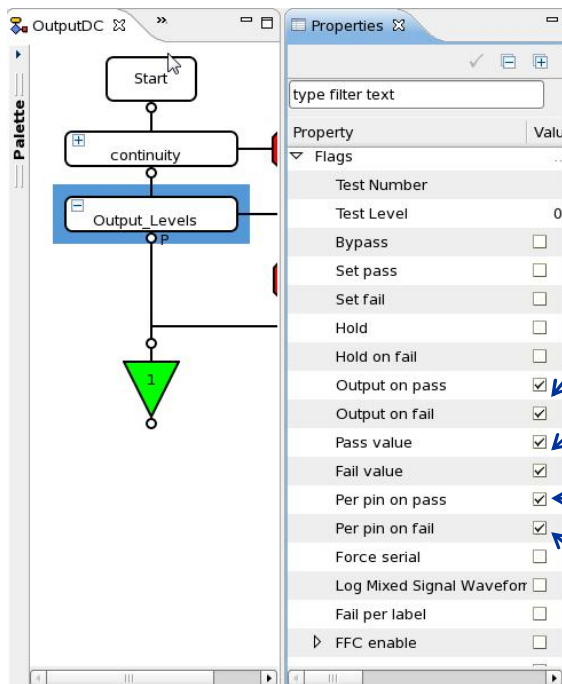
Signal	CP (DVC)	DS0 (DVC)	DS7 (DVC)	I/O0 (DVC)	I/O1 (DVC)	I/O2 (DVC)	I/O3 (DVC)	I/O4 (DVC)	I/O5 (DVC)	I/O6 (DVC)	I/O7 (DVC)	O0 (DVC)	O7 (DVC)	S0 (DVC)	S1 (DVC)	Trans IO::CLK	Trans IO::rea	Trans IO::wrt	MR (DVC)
X-Mode Area																			
Protocol																			
Vector#																			
5	1	0	0	L	L	H	L	L	L	L	L	L	1	0	1	0	1	1	
6	1	0	0	L	L	L	H	L	L	L	L	L	1	0	1	0	1	1	
7	1	0	0	L	L	L	L	H	L	L	L	L	1	0	1	0	1	1	
8	1	0	0	L	L	L	L	H	L	L	L	L	1	0	1	0	1	1	
9	1	0	0	L	L	L	L	L	H	L	L	L	1	0	1	0	1	1	
10	1	0	0	L	L	L	L	L	L	H	L	H	1	0	1	0	1	1	
11	1	0	0	L	L	L	L	L	L	L	L	L	1	0	1	0	1	1	
12	1	0	1	L	L	L	L	L	L	H	L	H	0	1	1	1	0	1	
13	1	0	1	L	L	L	L	L	L	H	H	L	H	0	1	1	1	0	1
14	1	0	0	L	L	L	L	L	H	H	L	L	L	0	1	1	1	0	1
15	1	0	0	L	L	L	L	H	H	L	L	L	L	0	1	1	1	0	1
16	1	0	0	L	L	L	H	H	L	L	L	L	L	0	1	1	1	0	1
17	1	0	0	L	L	H	H	L	L	L	L	L	L	0	1	1	1	0	1
18	1	0	0	L	H	H	L	L	L	L	L	L	L	0	1	1	1	0	1
19	1	0	0	H	H	L	L	L	L	L	L	H	L	0	1	1	1	0	1
20	1	0	0	H	L	L	L	L	L	L	L	H	L	0	1	1	1	0	1
21	1	0	0	H	L	L	L	L	L	L	L	X	X	0	0	1	0	0	1
22	1	0	0	H	L	L	L	L	L	L	L	X	X	0	0	1	0	0	1
23	1	0	0	H	L	L	L	L	L	L	L	X	X	0	0	1	0	0	1
24	1	0	0	L	L	L	L	L	L	L	L	X	X	0	0	1	0	0	1

- Functional test must pass
- Both low (L) and high (H) expected data states for the specified pins must be present in the specified vector range

ADVANTEST®

Testflow Flags

- The Test Method only performs a value measurement if required by the Testsuite flags



- Generate result output
- Return the maximum of VOL and minimum of VOH
- Return measurement result for all passed pins
- Return measurement result for all failed pins

ADVANTEST

Setting Testsuite Flags for a Passing Test

Output on pass	☒
Output on fail	☐
Pass value	☒
Fail value	☐
Per pin on pass	☐
Output on pass	☒
Output on fail	☐
Pass value	☒
Fail value	☐
Per pin on pass	☒
Output on pass	☒
Output on fail	☐
Pass value	☐
Fail value	☐
Per pin on pass	☒

- Returns the maximum value of VOL and the minimum value of VOH
- Returns the values of VOL and VOH per pin
- Returns Pass/Fail per pin

ADVANTEST

Import and Customize Test Method Library

In SmarTest you can easily import the SmarTest standard Test Method Libraries as source code and then adjust them to your needs

1. Create new Test Method Project and select Standard Test Method Library
2. Customize Test Method by changing code
3. Build customized Test Method Library
4. Use customized Test Method Library

ADVANTEST

April 2016

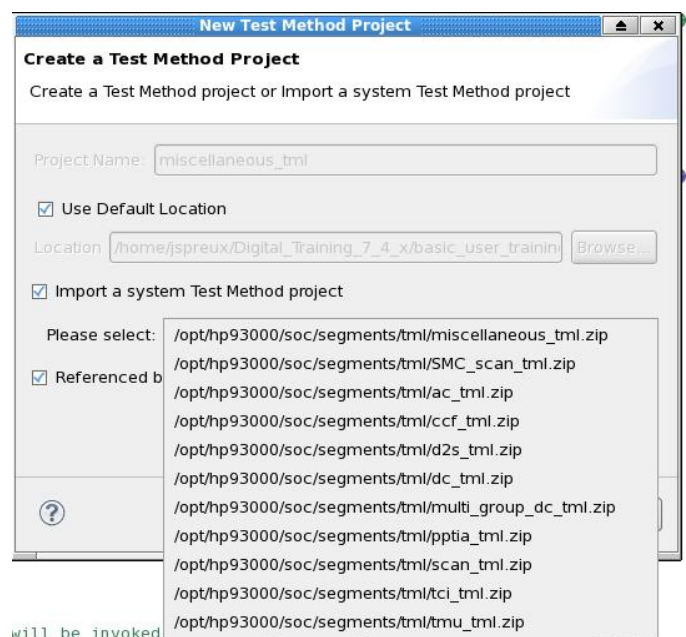
All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 39

1. Create new Test Method Project and Select Standard Test Method Library

Select "File" > "New" >
"Test Method Project"

Check



On pressing "Finish" the project directory is created (e.g. dc_tml/) beside of device directory; verify in "Project Explorer"

ADVANTEST

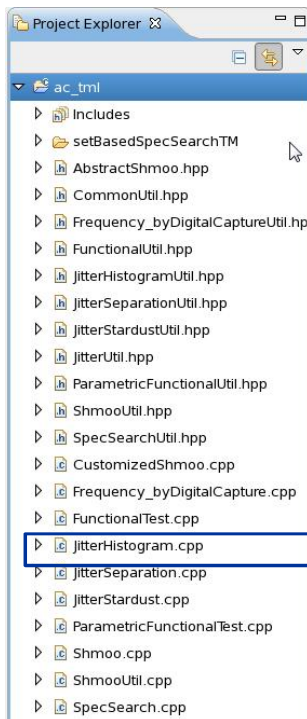
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 40

2. Customize Test Method by Changing Code

View the created project in Project Explorer in the C/C++ perspective



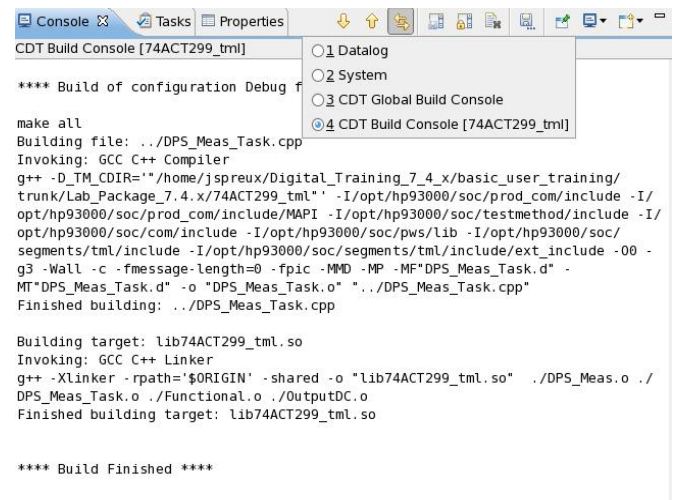
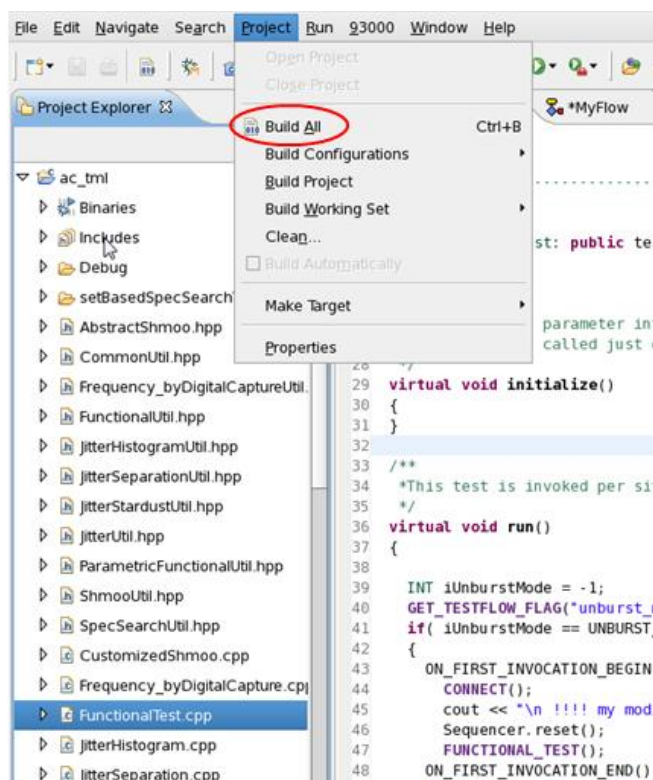
Double-click on the file will open the editor

Adjust the code to your needs

```
19 *.....
20 */
21
22class FunctionalTest: public testmethod::TestMethod
23{
24protected:
25 /**
26  *Initialize the parameter interface to the testflow.
27  *This method is called just once after a testsuite is created.
28  */
29 virtual void initialize()
30 {
31 }
32
33 /**
34  *This test is invoked per site.
35  */
36 virtual void run()
37 {
38
39     INT iUnburstMode = -1;
40     GET_TESTFLOW_FLAG("unburst_mode", &iUnburstMode);
41     if( iUnburstMode == UNBURST_OFF )
42     {
43         ON_FIRST_INVOCATION_BEGIN();
44         CONNECT();
45         cout << "\n !!!! my modified functional test !!!! \n" << endl;
46         Sequencer.reset();
47         FUNCTIONAL_TEST();
48         ON_FIRST_INVOCATION_END();
49         TESTSET().cont(true).judgeAndLog_FunctionalTest();
50     }
51     else
52     {
53         FunctionalUtil::unburstAndExecute(Primary.getLabel(), iUnburstMode);
54     }
55     return ;
56 }
57};
58
59REGISTER_TESTMETHOD("AcTest.FunctionalTest", FunctionalTest);
60
```

ADVANTEST

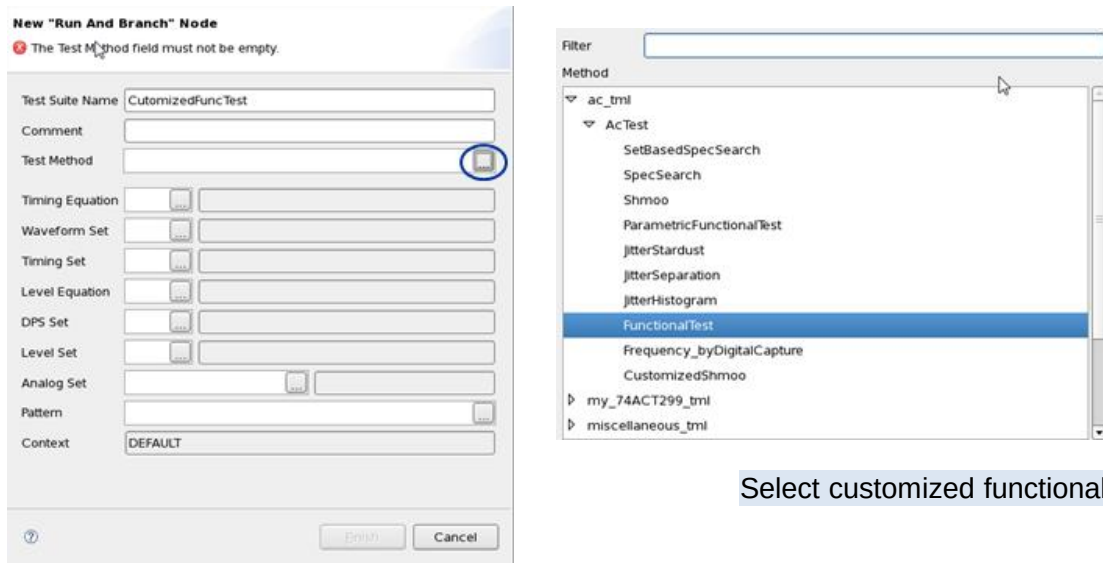
3. Build customized Test Method Library



ADVANTEST

4. Use customized Test Method Library

Insert new “Run and branch” Testsuite into Testflow



Select customized functional test

ADVANTEST

Test Method Snippets

SmarTest provides Test Method snippets you can easily use as starting point for code development and adjust to your needs

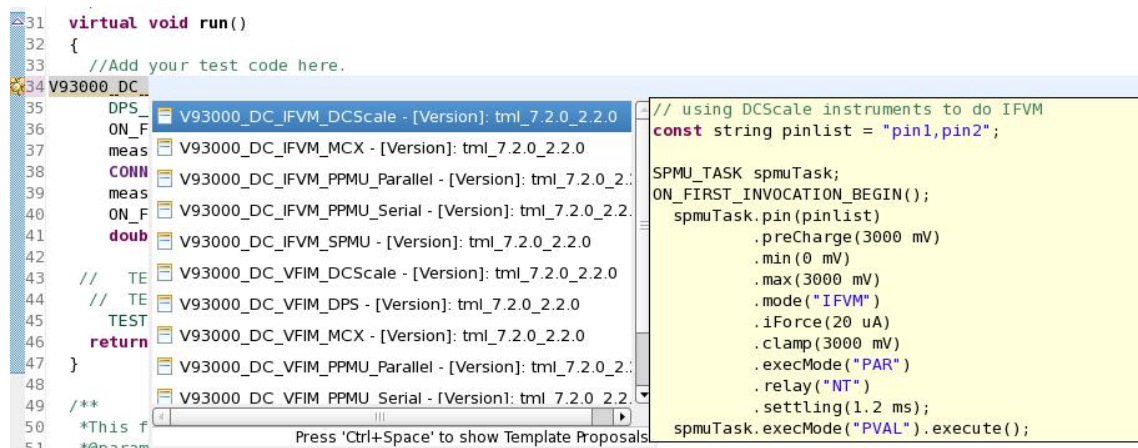
To use Test Method Snippets, proceed as follows:

1. Create a Test Method Project and add a Test Method Class to it
2. In the Test Method Editor enter the first characters
3. Hit <Ctrl>+<Space>
4. Highlight a snippet from the provided list
5. Double Click the snippet you want to use
6. Customize the code as needed

ADVANTEST

Test Method Snippet for DC Scale

Ctrl Space -> show templates proposals



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 45

Reference

- V93000 Technical Documentation Center:
- Topic 105426
<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=105426>

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Universal Test Methods Introduction Unit 10 - 46

Lab Exercise

Initial Test Method lab

Advantest V93000

Digital User Training

Test Method Debugging and Hardware Perspective

ADVANTEST[®]

Agenda

- Starting a debug session
- Navigate through Test Method Code
- Monitoring response of executed C++ Code (variables & hardware)
- Switching to Hardware Perspective
- Verify and modify current hardware settings

ADVANTEST[®]

Learning Objectives

- Know the tools used to debug Test Methods
- Know the tools used to monitor and modify the hardware interactively
- Know how to interpret system settings interactively

Relevance

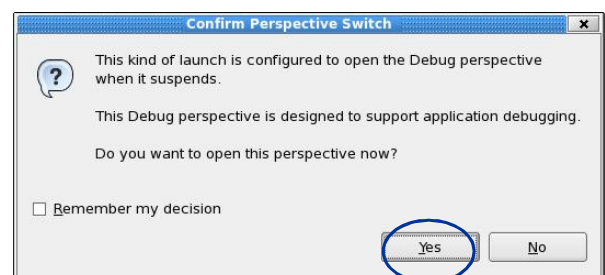
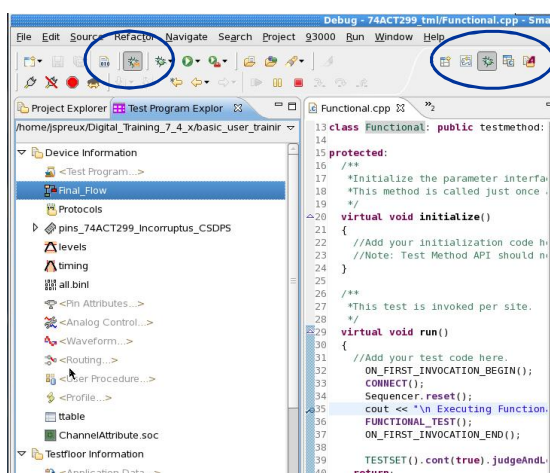
- Debugging Test Method is needed in various phases of test program development
- Hardware Perspective is a key engineering tool that helps in failure analysis

Typical Debugging Task

1. Starting a debug session
2. Set the breakpoints
3. Open the Test Method Debugger
4. Execute the Testsuite using the Test Method
5. Debug the Test Method step by step with the debug icons

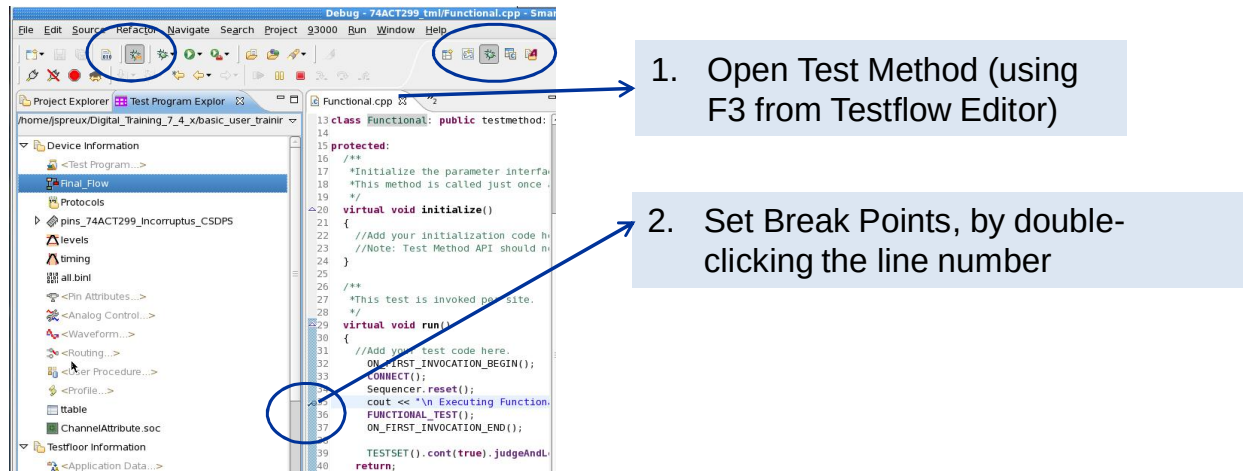
Starting a Debug Session

1. Click Test Method debug button in setup or C++ perspective



2. Select yes to open debug perspective

Starting a Debug Session



ADVANTEST®

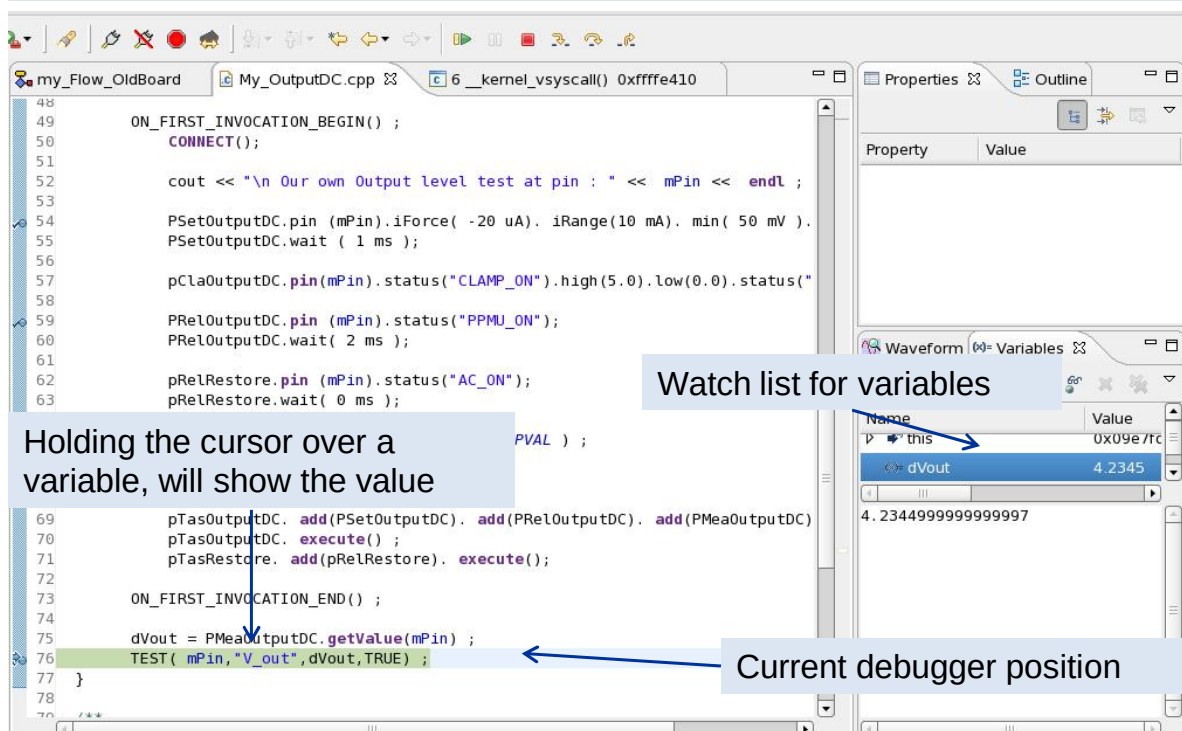
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 7

Navigating through C++ Code

Upon starting Testflow, debugger will stop at first Breakpoint



ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 8

Debugging Icons

Debug Testflow

Run last execution

Navigate sources

Step Into

Step Over

Step Return

Resume

Terminate

Note: Clicking these icons switches Eclipse (back) to the Debug Perspective

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 9

Views in Debug Perspective

Firmware View: Execute Firmware commands interactively

Connected device

Disconnected device

Monitoring, enable and disable Breakpoints

ADVANTEST

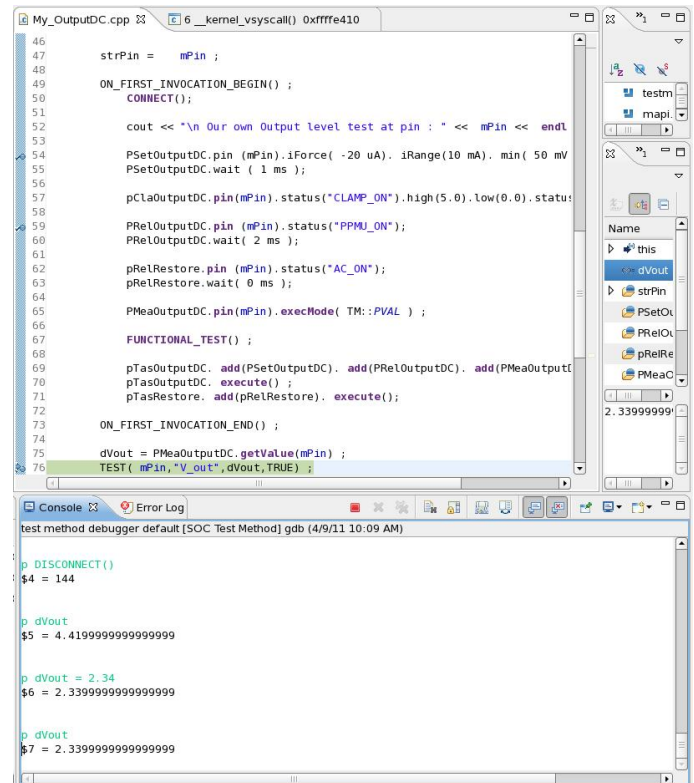
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 10

Calling Functions from the Debugger Console

- Function calls are supported via the print debugger command
- You can call any C/C++ functions. C macros are not supported though
- This can be used to build your own debug helpers – e.g. a print function for arrays and maps
- It can also be used to call existing SmarTest APIs directly



ADVANTEST

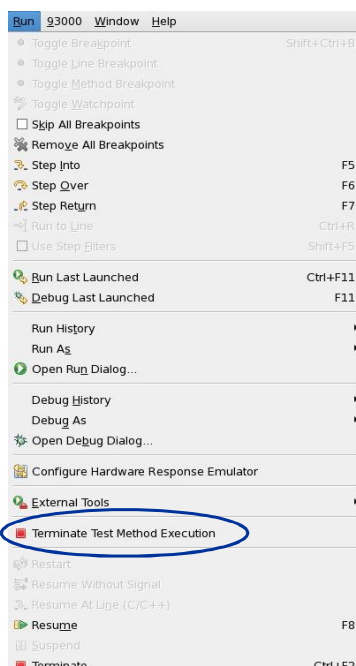
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 11

Stop Abnormal Execution

- If Test Method is running abnormally (e.g. endless loop) it can be stopped by:



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 12

Hardware Perspective Overview

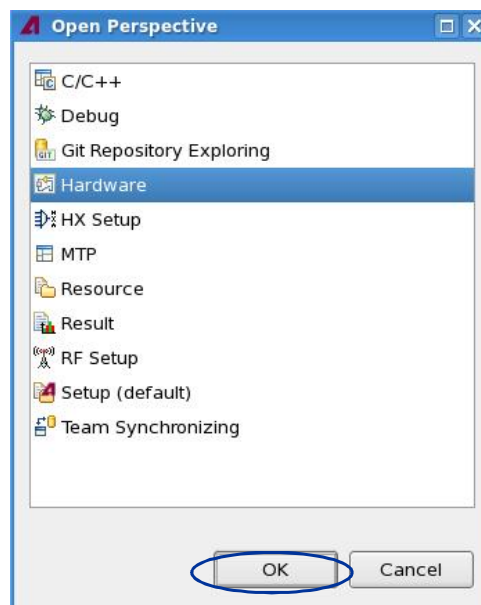
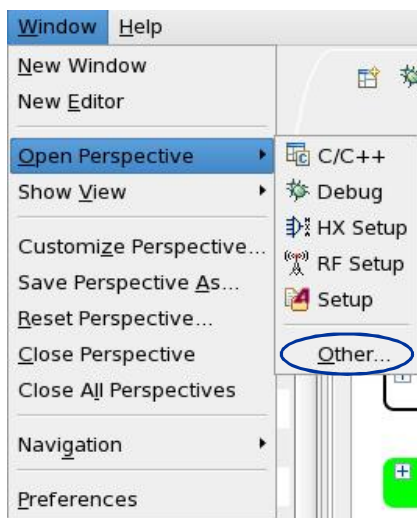
The V93000 Hardware perspective (also “hardware monitor tool”) is one of the predefined perspectives. It allows real-time debugging of your test setup and combines the following views:

- Primary View
- Pin Browser View
- Hardware View
- Utility Line View

Opening the V93000 HW Perspective

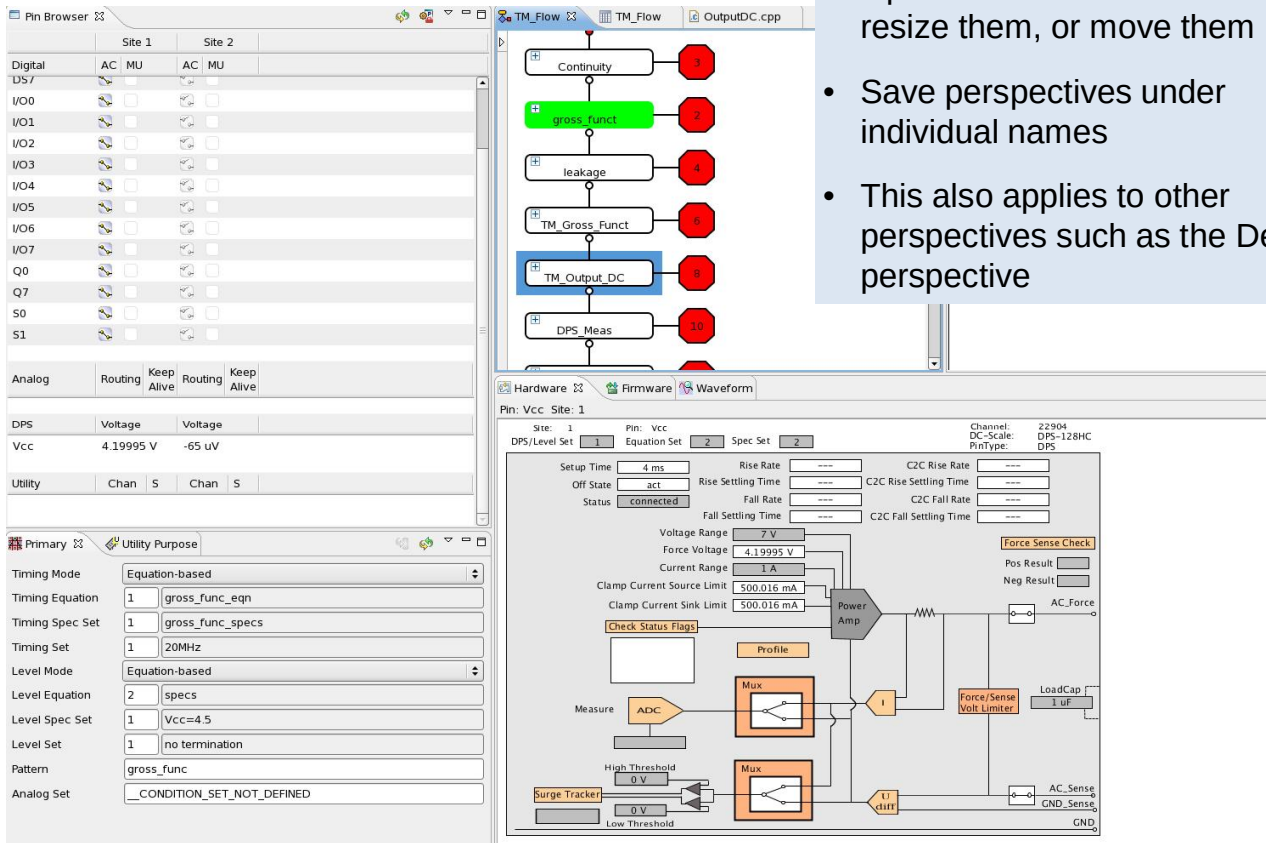
Different ways for opening the V93000 Hardware perspective:

Window > Open Perspective > other > Hardware



Hardware icon

V93000 Hardware



- Open or close different views, resize them, or move them
- Save perspectives under individual names
- This also applies to other perspectives such as the Debug perspective

ADVANTEST

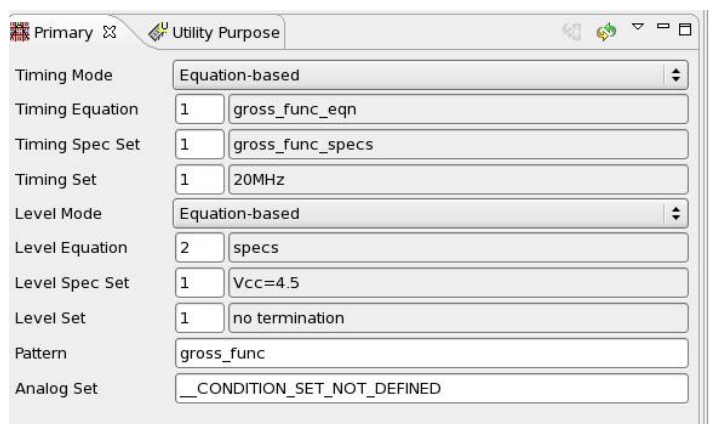
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 15

Primary View

- Displays current primary set (timing, levels, etc.)
- The fields that appear depend on the particular test setup
- Reflects primary setting of Testflow Properties section



ADVANTEST

April 2016

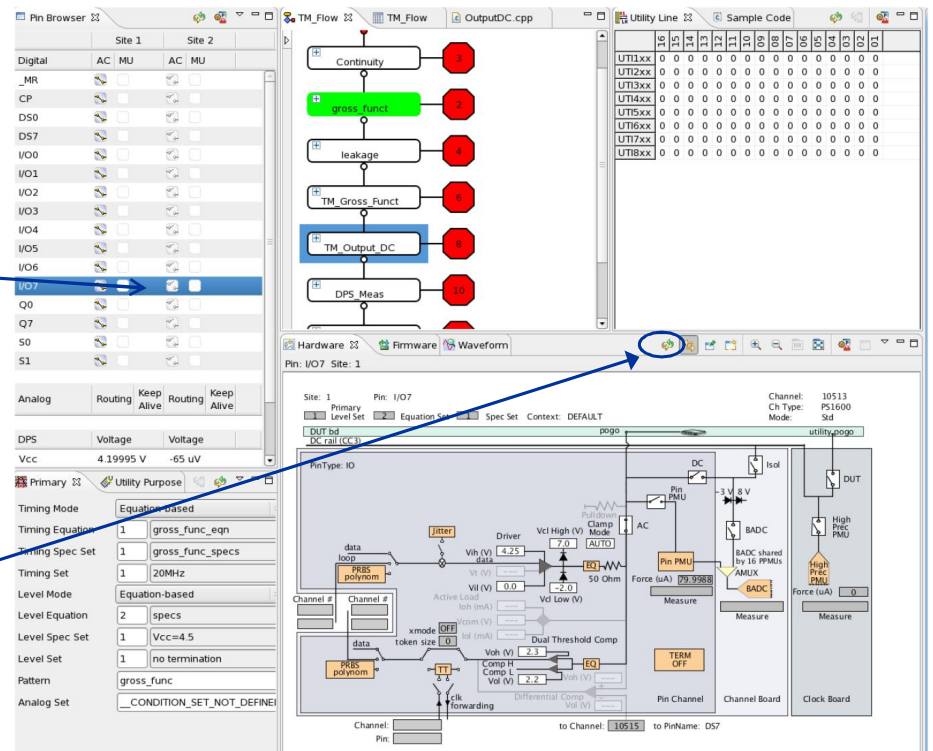
All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 16

Pin Browser Operation

- Select a pin in the Pin Browser to display a schematic diagram of the tester pin circuitry in the Hardware View

- To make sure, schematic shows current state of hardware, you have to refresh hardware information



ADVANTEST

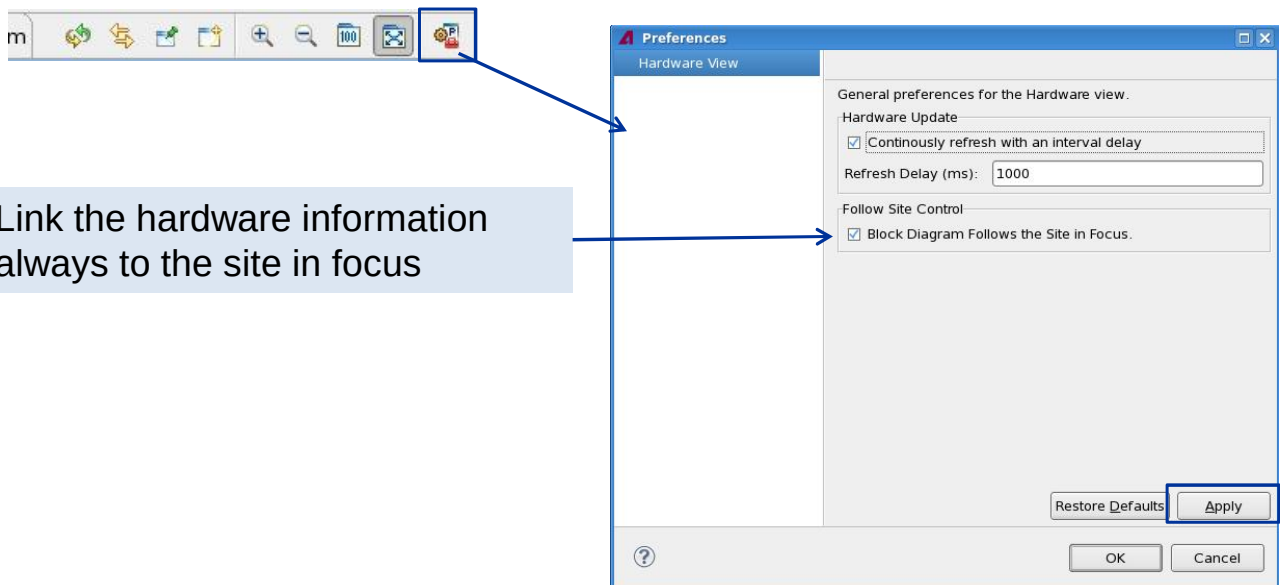
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 17

Pin Browser Preferences

You can determine the refresh settings for the Pin Browser



Link the hardware information always to the site in focus

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 18

Pin Browser

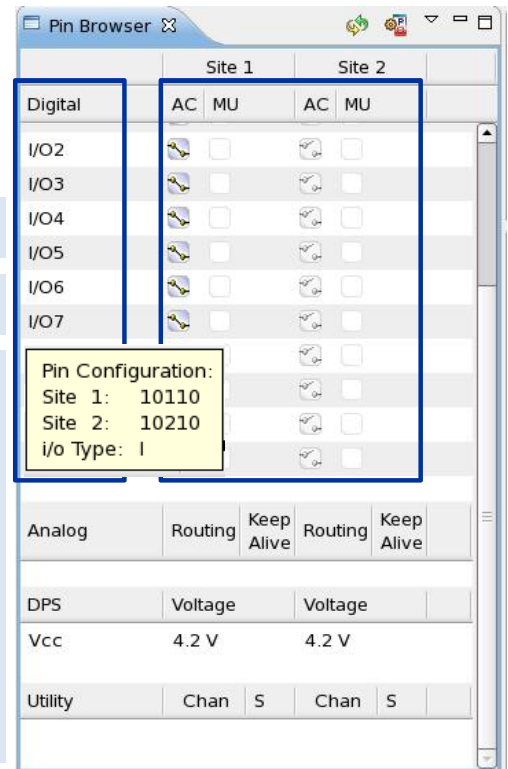
This view displays the configured pins listed under their appropriate headings: Digital, Analog, DPS, and Utility

Pins are sorted alphabetically by name

Holding cursor over pin shows information

Connection states:

-  •AC relay open
-  •AC relay closed
-  •Pin PMU relay closed
-  •HPPMU (SPMU) relay closed
-  •ADC switch enabled



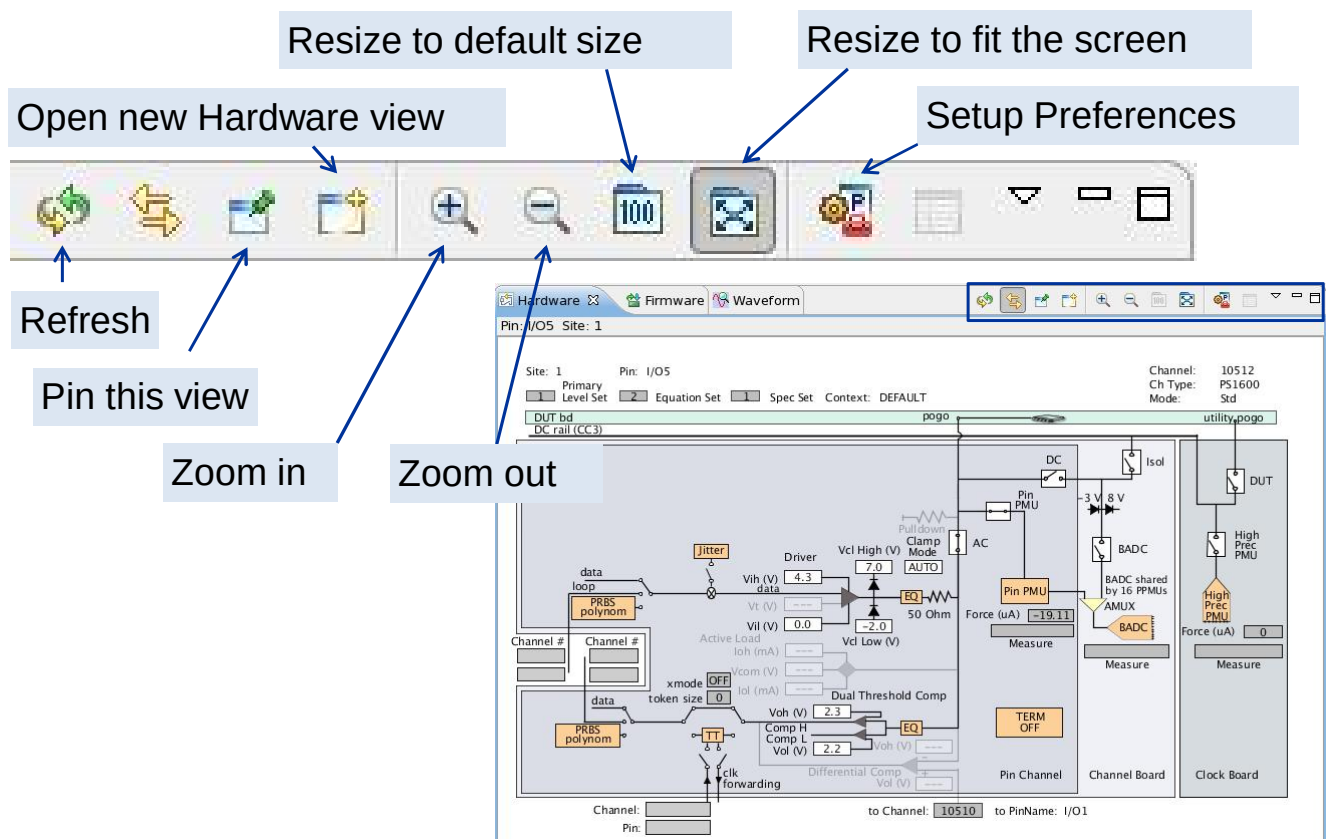
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 19

Hardware View Icons



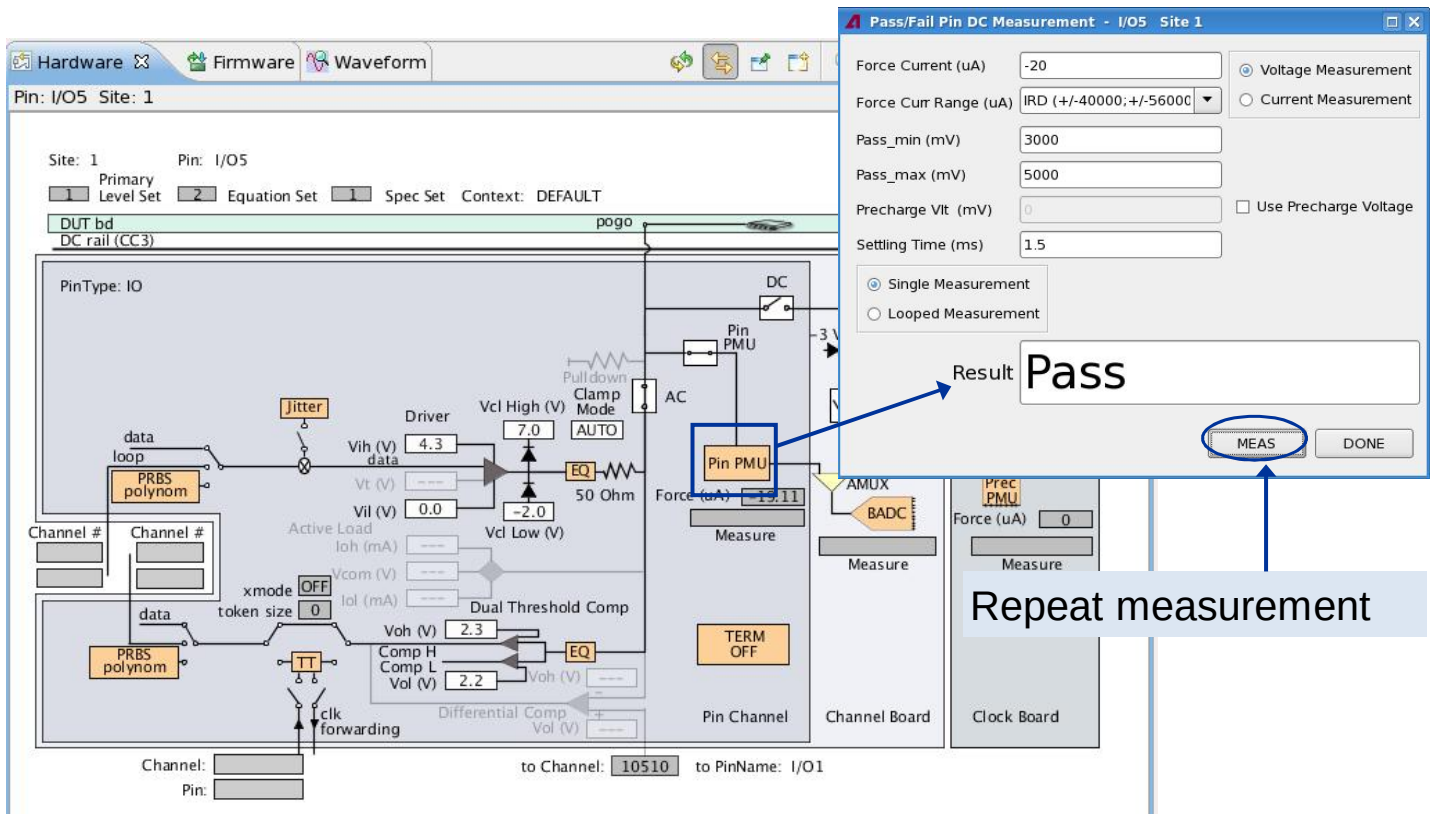
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 20

Interactive Measurement with Pin PMU



ADVANTEST®

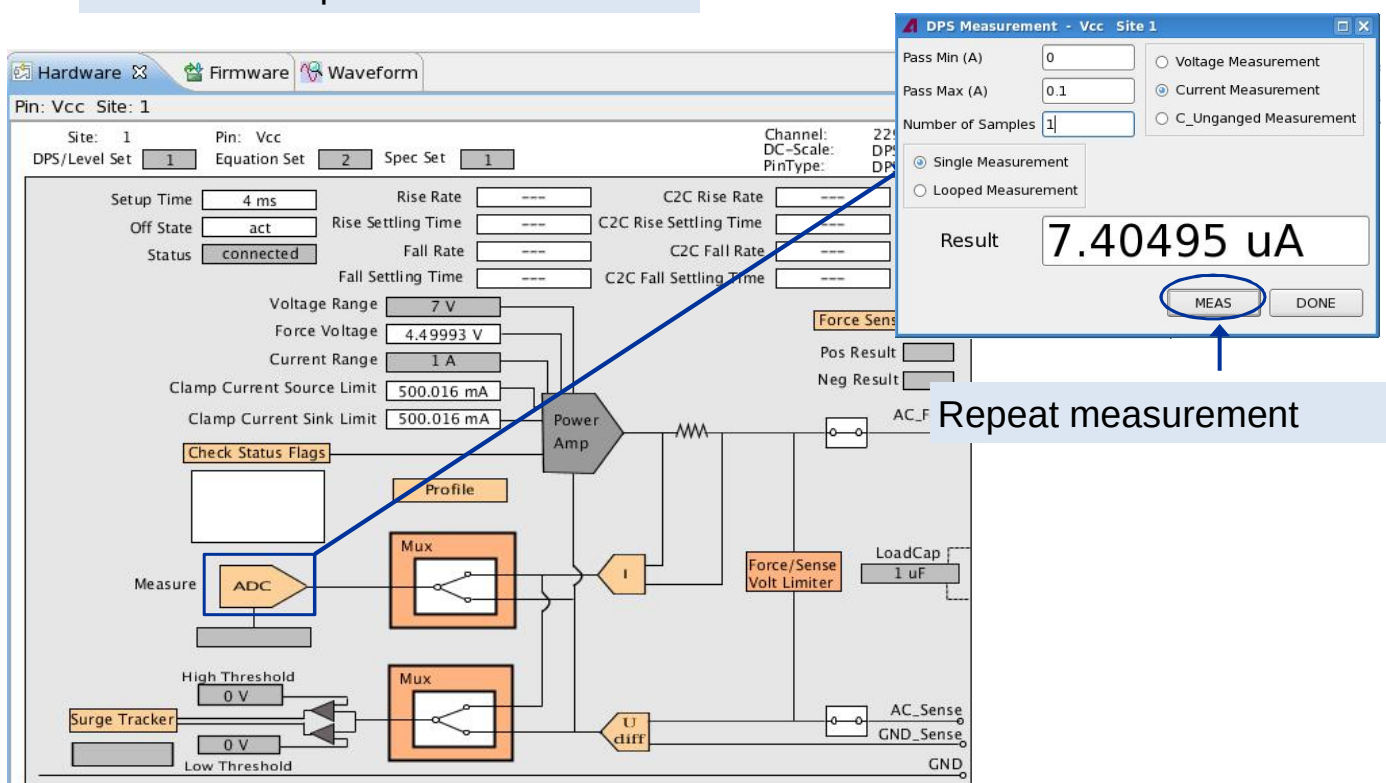
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 21

Interactive Measurement with DPS

Select a DPS pin in Pin Browser



ADVANTEST®

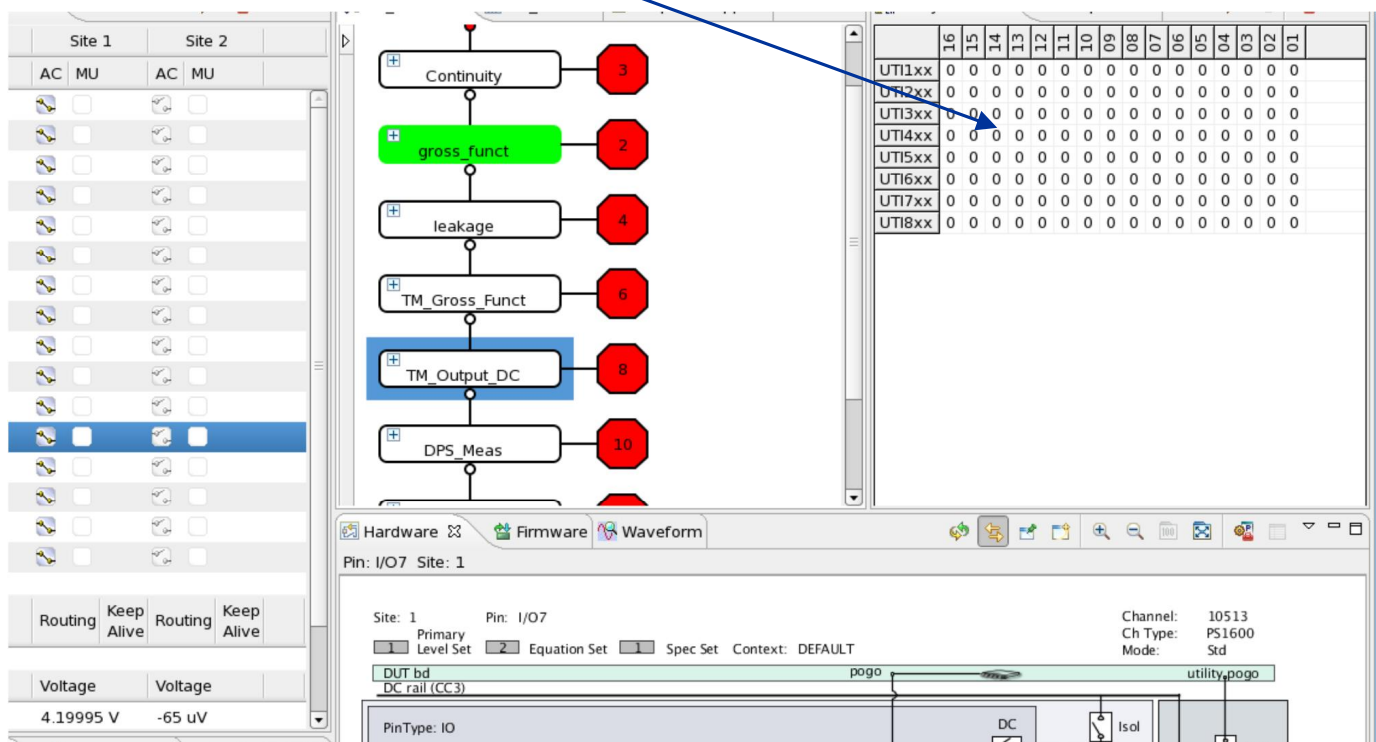
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 22

Utility Line View

Current status of utility channels can be monitored and changed



ADVANTEST

April 2016

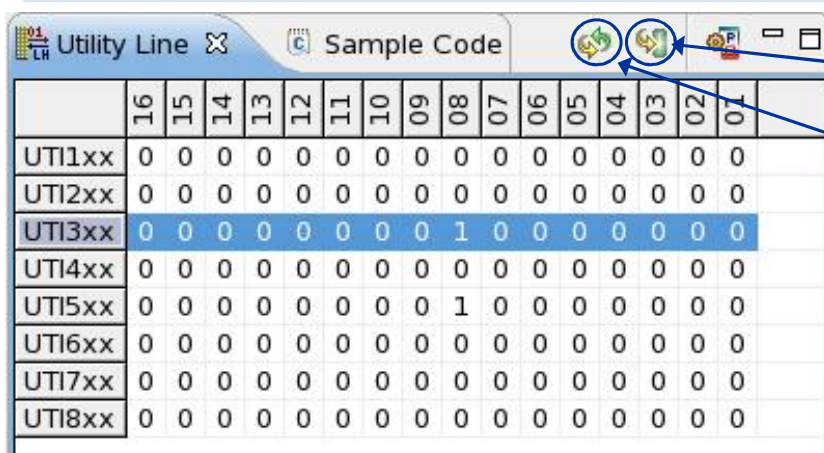
All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 23

Utility Line Write State

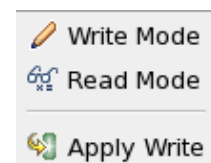
In the Utility Line view

- Select one or more utility block(s)
- Right-click and select Write Mode
- Click a single pin to toggle the write state of the pin
- Right-click and select Apply Write



Apply Write

Refresh view



ADVANTEST

April 2016

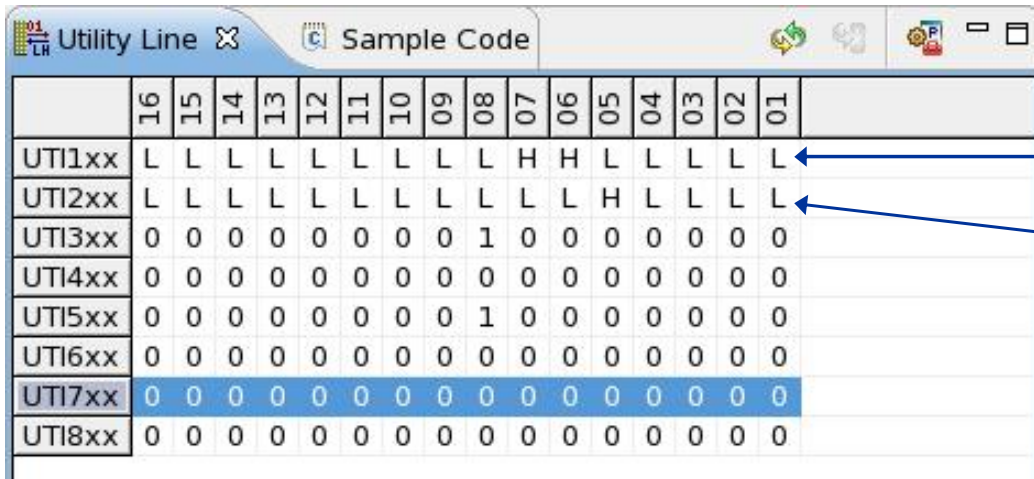
All Rights Reserved - ADVANTEST CORPORATION

TM Debugging and Hardware Perspective Unit 11 - 24

Query Read State

In the Utility Line view

- Select one or more utility block(s)
- Right-click and select Read Mode



	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01
UTI1xx	L	L	L	L	L	L	L	L	L	H	H	L	L	L	L	L
UTI2xx	L	L	L	L	L	L	L	L	L	L	L	H	L	L	L	L
UTI3xx	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
UTI4xx	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
UTI5xx	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
UTI6xx	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
UTI7xx	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
UTI8xx	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

States:

- TTL Low (L)
- TTL High (H)

ADVANTEST

Reference

V93000 Technical Documentation Center:

Topic 109172

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=109172>

ADVANTEST

Advantest V93000

Digital User Training

Test Table/TM Limits

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Test tables

ADVANTEST[®]

Learning Objectives

- Know the structure of Test table
- Know how to setup a Test Table
- Know how to use Test table with a testflow
- Know how to reference Test table in Test Methods

Relevance

- Test tables are used to provide test numbers and limits for tests
- TMLimits is a practical implementation of Test table in SmarTest programs

Test tables

- Test tables are used in production or engineering to provide test numbers and limits for tests.
- Test tables can be accessed from your test method.
- TMLimits is a specialized form of test table implementation.
- TMLimits are CSV file.

Test table typical fields

The test table (limit table) is a standard CSV file with a specified column structure.

- Test suite name
- Test name
- Test number
- Units
- Lsl
- Lsl_typ
- Usl_typ
- Usl
- Offset
- Bin_s_name
- Bin_h_name
- Bin_h_number
- Test_remarks

A test table created by SmarTest also includes additional line for Test_mode. Attached is an example of test table with Test_mode.

Test tables example

	A	B	C	D	E	F	G	H	I	J	K	L
1	# Product Eng											
2	# Revision:	1.0 - Initial release March 2016										
3	# Sample limit definitions											
4	# Transform are	userdefined values										
5	Suite name	Test name	Test number	PinName	Units	Lsl	Lsl_typ	Usl_typ	Usl	Lsl	Lsl_typ	Usl_typ
6	Test mode					FT	FT	FT	FT	QA	QA	QA
7	Scan	func	5001			0	GT	LT	0	0	GT	LT
8	JTAG	func	5002			0	GT	LT	0	0	GT	LT
9												
10	Board_ID	board_id	5033			-0.1	GT	LT	23	-0.1	GT	LT
11												
12	Continuity	Cont_dig	209	V		-0.89	GT	LT	-0.21	-0.9	GT	LT
13												
14	Leakage	lih_dig	1001	uA		-0.9	GE	LE	0.9	-1	GE	LE
15	Leakage	lil_dig	1022	uA		-0.9	GE	LE	0.9	-1	GE	LE
16												
17	MaxIdd	MaxIdd_TXALL	77028	mA		470	GT	LT	558	460	GT	LT
18	MaxIdd	MaxIdd_RXALL	77032	mA		345	GT	LT	425	325	GT	LT
19												
20	RX	Gain	87688	dB		70.25	GT	LT	79.75	70	GT	LT
21	RX	Gain_I	87689	dB		70	GT	LT	80	70	GT	LT
22	RX	Gain_Q	87690	dB		70	GT	LT	80	70	GT	LT
23	RX	RSB	87691	dBc		27.25	GT	LT	79.75	27	GT	LT
24	RX	IQ_phs_err	87692	deg		-5	GT	LT	5	-6	GT	LT
25												
26	PDADC	INL	80100	lsb		0.25	GT	LT	1.6	0	GT	LT
27	PDADC	DNL	80101	lsb		0.25	GT	LT	1.49	0	GT	LT
28	PDADC	GAIN	80102	%		96	GT	LT	104	95	GT	LT
29	PDADC	OFFSET	80103	mV		-29	GT	LT	39	-30	GT	LT

Test mode

- A test table can contain multiple versions of limits from same test, with each version being assigned to a test mode. These modes are user definable; common modes include Characterization, Final Test, Quality Assurance etc.
- Number of Test Mode is not limited by software
- Test Mode can be set by:
 - *setTestMode* member function of *TMLimits* API class
 - Or Testflow variable "TMLimit_TestMode"

Test table creation procedure

1. Load the testflow for which you want to create a test table.
2. In the Test Program Explorer, right-click the <Test Table> entry and choose New from the context menu. A Save dialog is displayed.
3. Enter a name for the new test table, choose a location, and click OK. The name of the test table appears in the Test Program Explorer.
4. Choose Save from the context menu.
5. The standard location for TMLimits test tables is in the test table subdirectory of the device directory.

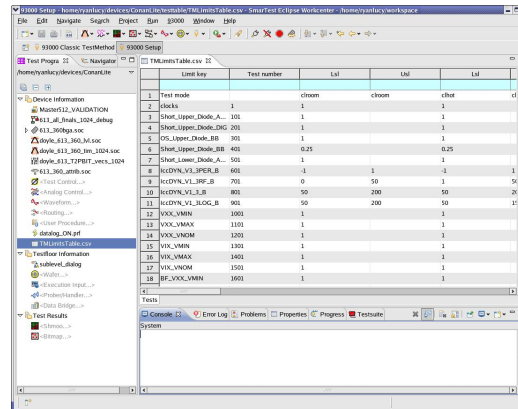
Editing Test tables

1. Once you have a test table in place, you can modify its content
2. Edit the values in the table as needed.
3. Add new rows or columns
4. Right-click the test table in the Test Program Explorer and choose Save from the context menu.
5. The standard location for TMLimits test tables is in the test table subdirectory of the device directory.

Editing Test tables

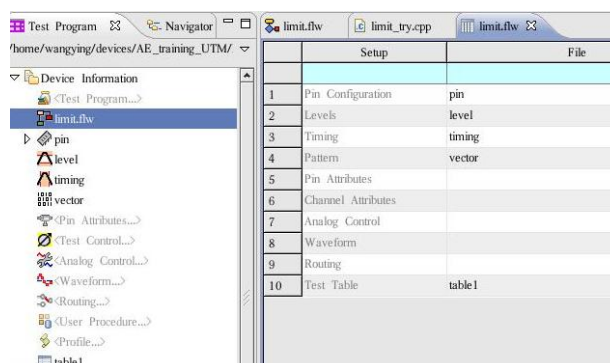
Double-clicking on the “Test Table” entry in the Test Program Explorer opens the CSV content for viewing and editing.

Modify the file as requested.



Test tables & Testflow

1. Create a test table (.csv file) to store Test Limits, test number assignments etc.
2. Open the testflow with the Test Data Editor.
3. Assign the test table as one of the setup files of the testflow.
4. In your TestMethod, use TMLimits API to get Limits from the testtable.



Test tables & Testflow

Once a test table is loaded all testflow based limits will be “grayed out” and not editable.

However they can still be used. To edit them unload the Test Table temporarily and then you can edit them in the normal manner.

Property	Value
Test Suite	Continuity_both
Comment	
Primarys	
Test Type	M
Test Method	dc_tml.DcTest.Continuity
Parameters	Continuity, 1[uA], 1[ms], PPM
pinlist	Continuity
testCurrent	1[uA]
settlingTime	1[ms]
measurementMode	PPMUpAr
polarity	BPOL
prechargeToZeroVo	OFF
output	None
Limits	
passVolt_mV	-200 < X <=2499
Test Number	
Limit Value	-200 < X <=2499
Flags	

Test limit usage examples

The TMLimits class lets you control limits and test numbering in your testflow with data from a CSV file.

You can switch between different testing modes or stages in the same testflow for different situations.

The settings in the CSV file can be edited even without access to SmarTest, for example, using Open Office Calc or a standard text editor.

Test limit usage examples

Example 1: Setting test mode

//declare the V93kLimits namespace in your test method with the statement

using namespace V93kLimits;

//Setting the test mode and performing a test

tmLimits.setTestMode("FT");

TestSet.TEST("pinName", "myTestName", tmLimits, 4);

Or

Use testflow variables "TMLimit_TestMode" to set the mode, i.e. "FT", "QA", "WS", etc...

Judgement Example

- Add the following line to your Test Method class

using namespace V93kLimits;

- Modify the Judgement section of your test method

TESTSET().cont(true).judgeAndLog_ParametricTest("pin", "tname", tmLimits, value);

Or

TestSet.judgeAndLog_ParametricTest("pin", "tname", tmLimits.getLimitObj("tname"), result);

TestSet.judgeAndLog_ParametricTest("pin", "tname", tmLimits, result);

Reference

V93000 Technical Documentation Center (TDC):

- Topic 110432

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=110432>

Advantest V93000

Digital User Training

DC Testing APIs

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- DC Test Resources
- FlexDC Basic Programming
- FlexDC Introduction
- PPMU Test Programming
- DPS128 Test Programming
- TM creation via DC sample Code
- Exception handling in UTM programming

ADVANTEST[®]

Learning Objectives

- Know how to write a DC Test Method using FlexDC
- Know how to program DPS using Test Methods

Relevance

- Correct DC programming helps to reduce test time and increase throughput in engineering and production test programs

Standard DC Test Methods

74ACT299 DC Parameters

DC Electrical Characteristics for ACT

Symbol	Parameter	V _{CC} (V)	T _A = 25°C		T _A = -40°C to +85°C		Units	Conditions
			Typ	Guaranteed Limits				
V _{IH}	Minimum HIGH Level Input Voltage	4.5	1.5	2.0	2.0		V	V _{OUT} = 0.1V or V _{CC} - 0.1V
V _{IL}	Maximum LOW Level Input Voltage	3.0	1.5	0.8	0.8		V	V _{OUT} = 0.1V or V _{CC} - 0.1V
V _{OH}	Minimum HIGH Level Output Voltage	4.5	4.49	4.4	4.4		V	I _{OUT} = -50 μA
		5.5	5.49	5.4	5.4		V	I _{OUT} = -50 μA
V _{OL}	Maximum LOW Level Output Voltage	4.5	0.0001	3.86	3.76		V	V _{IN} = V _{IL} or V _{IH} I _{OH} = -24 mA
		5.5		4.86	4.76		V	I _{OH} = -24 mA (Note 5)
		4.5	0.001	0.1	0.1		V	I _{OUT} = 50 μA
		5.5	0.001	0.1	0.1		V	I _{OUT} = 50 μA
I _{IN}	Maximum Input Leakage Current	5.5		± 0.1	± 1.0		μA	V _I = V _{CC} , GND
I _{CC1}	Maximum I _{CC} /Input	5.5	0.6		1.5		mA	V _I = V _{CC} - 2.1V
I _{OLD}	Minimum Dynamic Output Current (Note 6)	5.5			75		mA	V _{OLD} = 1.65V Max
I _{OHD}	Output Current (Note 6)	5.5			-75		mA	V _{OHD} = 3.85V Min
I _{CC}	Maximum Quiescent Supply Current	5.5		4.0	40.0		μA	V _{IN} = V _{CC} or GND
I _{OZT}	Maximum I/O Leakage Current	5.5		±0.3	±3.0		μA	V _I (OE) = V _{IL} , V _{IH} V _I = V _{CC} , GND V _O = V _{CC} , GND

Note 5: All outputs loaded; thresholds on input associated with output under test.

Note 6: Maximum test duration 2.0 ms, one output loaded at a time.

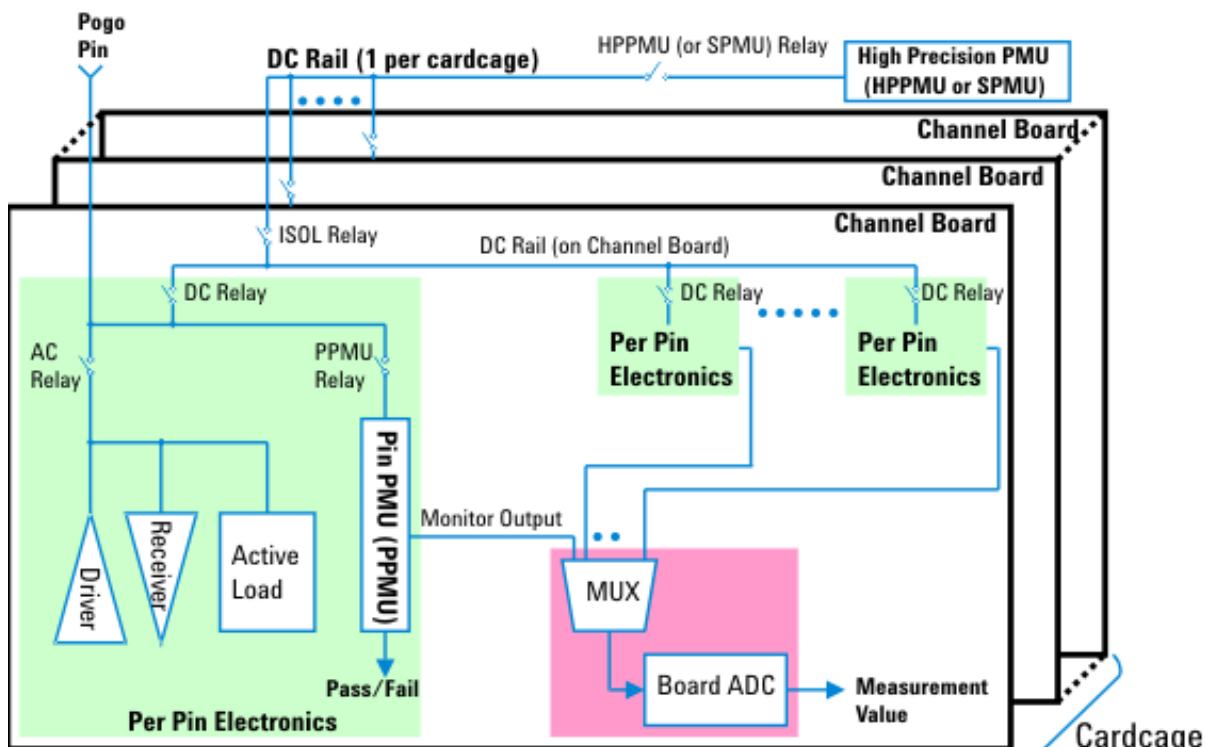
ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 5

V93000 Per Digital Pin DC Resources



ADVANTEST

March 2016

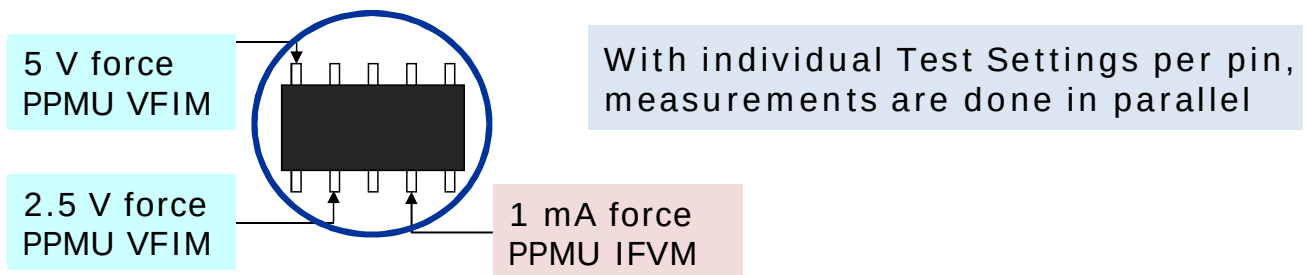
All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 6

FlexDC Purpose

- Support of different measurements on different pins in parallel
- Support to force voltages/currents to a device pin
- Support of fast multi-site measurements with different settings on different sites
- Optimizing per pin value, per pin pass fail measurements

Pin PMU example:



ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

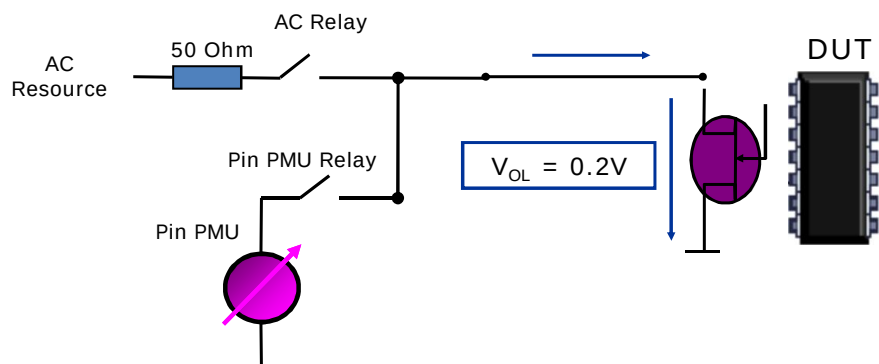
DC Testing Unit 13 - 7

DC Test Programming with PPMU

FlexDC APIs :

- PPMU_SETTING
- PPMU_RELAY
- PPMU_MEASURE
- PPMU_CLAMP
- TASK_LIST

Using these APIs, you can program individual force voltage or current value per pin, apply it to the pin and perform a measurement



ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 8

Relay Control Overview

Specifier for relay_stat	AC Relay status	Pin PMU Relay status	DC Relay status
"ALL_ON"	Close	Close	Close
"ALL_OFF"	Open	Open	Open
"AC_ON"	Close	Open	Open
"AC_OFF"	Open	Open	Open
"PPMU_ON"	Open	Close	Open
"PPMU_OFF"	Open	Open	Open
"DC_ON"	Open	Open	Close
"DC_OFF"	Open	Open	Open
"ACDC_ON"	Close	Open	Close
"ACDC_OFF"	Open	Open	Open
"DCPM_ON"	Open	Close	Close
"DCPM_OFF"	Open	Open	Open
"ACPM_ON"	Close	Close	Open
"ACPM_OFF"	Open	Open	Open
"OAC_ON"	Close	No change	No change
"OAC_OFF"	Open	No change	No change
"OPM_ON"	No change	Close	No change
"OPM_OFF"	No change	Open	No change
"ODC_ON"	No change	No change	Close
"ODC_OFF"	No change	No change	Open

Test Method Code

```
PPMU_RELAY relay1;
relay1.pin("pin1").status("PPMU_ON");
```

ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 9

Single-site PPMU FlexDC APIs Example

```
PPMU_SETTING pSetSample;
PPMU_RELAY   pRelSample, pRelRestore;
PPMU_MEASURE pMeaSample;
TASK_LIST    pTasSample;

pSetSample.pin("pin1,pin2").iRange(100 uA);
pSetSample.pin("pin1").iForce(10 uA).min(50 mV).max(100 mV);
pSetSample.pin("pin2").vForce(1 mV).min(10 uA).max(20 uA);

pRelSample.pin("pin1,pin2").status("PPMU_ON");
pRelSample.wait(1.3 ms);
pRelRestore.pin("pin1,pin2").status("AC_ON");
pRelRestore.wait(0 ms);

pMeaSample.pin("pin1,pin2").execMode(TM:PVAL);

pTasSample.add(pRelSample).add(pSetSample).add(pMeaSample)
            .add(pRelRestore);

pTasSample.execute();

DOUBLE dMeasure1 = pMeaSample.getValue("pin1");
DOUBLE dMeasure2 = pMeaSample.getValue("pin2");

TEST("pin_1", "Testname1", dMeasure1);
TEST("pin_2", "Testname2", dMeasure2);
```

Classes instantiation
Define objects

Settings
Measure mode (VFIM, IFVM)
Force value, Limits (min, max)

Relay Setup
AC relay, DC relay, PPMU relay

Measurement Setup
Pass/Fail, value measurement

Execution
Define execution order of
Setups and execute

Result retrieval
Pass/Fail, value

Judgment
TEST API for judge and datalog

ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 10

Multi-site FlexDC APIs Example

```
PPMU_SETTING pSetMultiSite;
PPMU_RELAY   pRelMultiSite, pRelRestore;
PPMU_MEASURE pMeaMultiSite;
TASK_LIST    pTasMultiSite;
```

Define FlexDC objects

```
ON_FIRST_INVOCATION_BEGIN();
```

Code entered once for all number of sites

```
pSetMultiSite.pin("pin_1").vForce(1.5 V).iRange(10 mA)
    .min(0 mA).max(40 mA);
pSetMultiSite.pin("pin_2").iForce(1 mA)
    .min(50 mV).max(100 mV);
```

Set PPMU values

```
pRelMultiSite.pin("pin_1, pin_2").status("PPMU_ON");
pRelMultiSite.wait(1 ms);
pRelRestore.pin("pin_1, pin_2").status("AC_ON");
pRelRestore.wait(0 ms);
```

Set Relays to switch

```
pMeaMultiSite.pin("pin_1, pin_2").execMode(TM::PVAL);
```

Set Measure Mode

```
pTasMultiSite.add(pRelMultiSite).add(pSetMultiSite)
    .add(pMeaMultiSite).add(pRelRestore).execute();
```

Set Task and Execute

```
ON_FIRST_INVOCATION_END();
```

From here, code section is executed per site

```
DOUBLE dI_meas = pMeaMultiSite.getValue("pin_1");
DOUBLE dV_meas = pMeaMultiSite.getValue("pin_2");
```

Get Results

```
TEST("pin_1", "I_LowValue", dI_meas_uA);
TEST("pin_2", "V_LowValue", dV_meas);
```

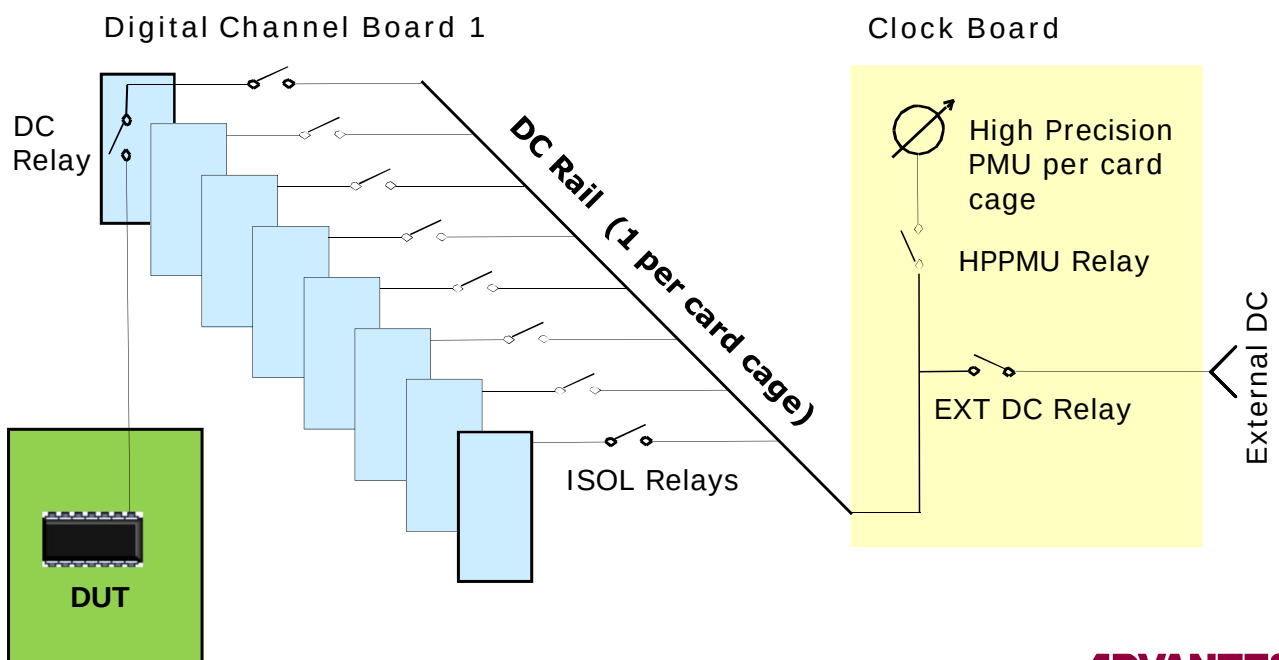
Judge and Log Values per site

After this test "pin_1" and "pin_2" are connected to AC-source by "pRelRestore" object, PPMU of "pin_1" remains set to force 1.5V

ADVANTEST

DC Test Programming with HPPMU/SPMU

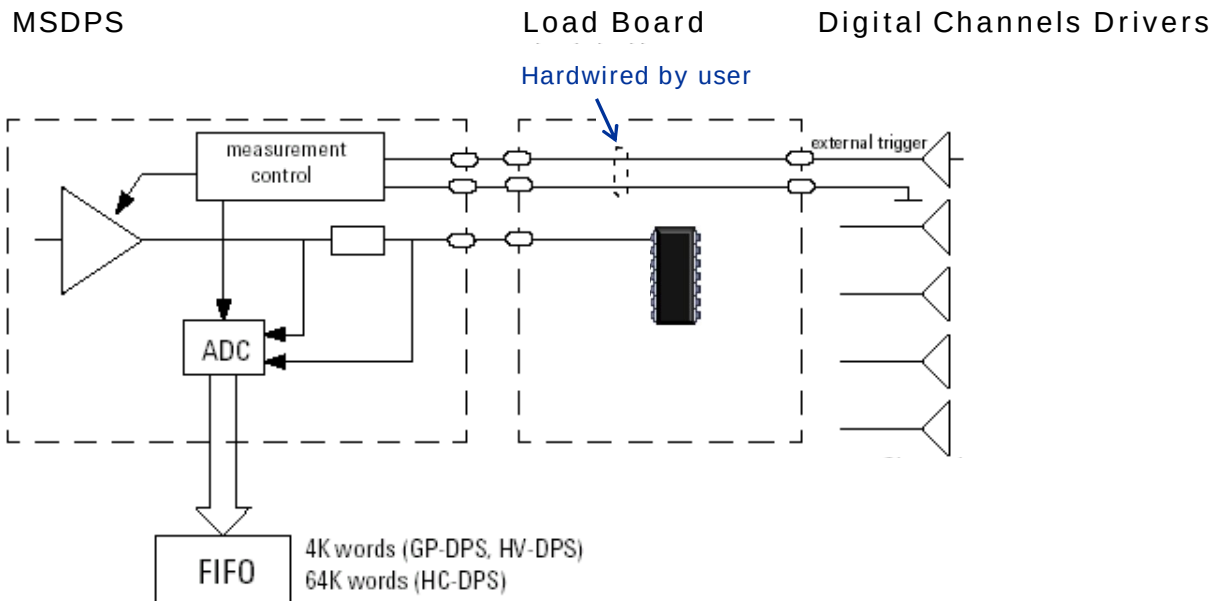
- HPPMU can be used through digital pin over DC rail
- One HPPMU available per Card cage



ADVANTEST

Hardware-Triggered Measurement Setup

- DPS Slot Power Supplies (e.g. MS-DPS) can obtain multiple results
- Via the trigger input line it is possible to do precisely triggered measurements



MS-DPS Software-Triggered Measurement

```
DPS_TASK    dpsTasSample;
```

```
ON_FIRST_INVOCATION_BEGIN();
```

```
dpsTasSample.pin("Vcc").min(0 A).max(0.1 A);
dpsTasSample.pin("Vee").min(0 A).max(0.2 A);
```

```
dpsTasSample.execMode(TM::PVAL);
dpsTasSample.trigMode(TM::INTERNAL);
```

Use internal software-driven trigger

```
dpsTasSample.wait(5 ms);
```

```
dpsTasSample.execute();
```

This will perform the measurement

```
ON_FIRST_INVOCATION_END();
```

```
DOUBLE dIddqVcc = dpsTasSample.getValue("Vcc");
DOUBLE dIddyVee = dpsTasSample.getValue("Vee");
```

```
TEST("Vcc", "Iddq", dIddqVcc);
TEST("Vee", "Iddq", dIddyVee);
```

MS-DPS Hardware-Triggered Measurement

```

DPS_TASK      dpsTasTriggered;
ARRAY_D       valueArrayVcc;
INT           i;
static LIMIT  limit;
MULTI_LIMITS  mLimits_dps;

ON_FIRST_INVOCATION_BEGIN();

// Create a Limit for each measurement
limit.unit("A") ;
limit.low(TM::GE, -4 uA);
limit.high(TM::LE, 175 uA);

// Make sure that your pattern issues 8 triggers,
// otherwise you will get an error message
mLimits_dps.add(limit, 8);

dpsTasTriggered.pin("Vcc").limits(mLimits_dps);

dpsTasTriggered.trigMode(TM::EXTERNAL).execMode(TM::PVAL);

dpsTask.execute();

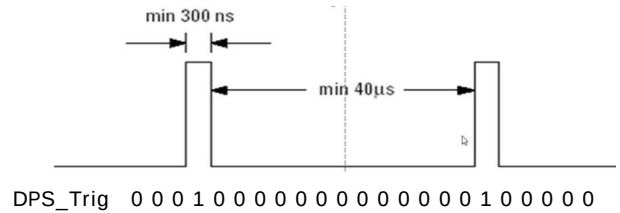
ON_FIRST_INVOCATION_END();

valueArrayVcc = dpsTasTriggered.getWaveform("Vcc");

for(i=0; i<valueArrayVcc.size(); i++){
    TEST("Vcc", "extTrigDps", limit, valueArrayVcc[i]);
}

```

Expected trigger input signal
(in this example 8 pulses required)



Number of samples = 8

Trigger signal required for measurement
(TM::EXTERNAL)

Run functional test with trigger

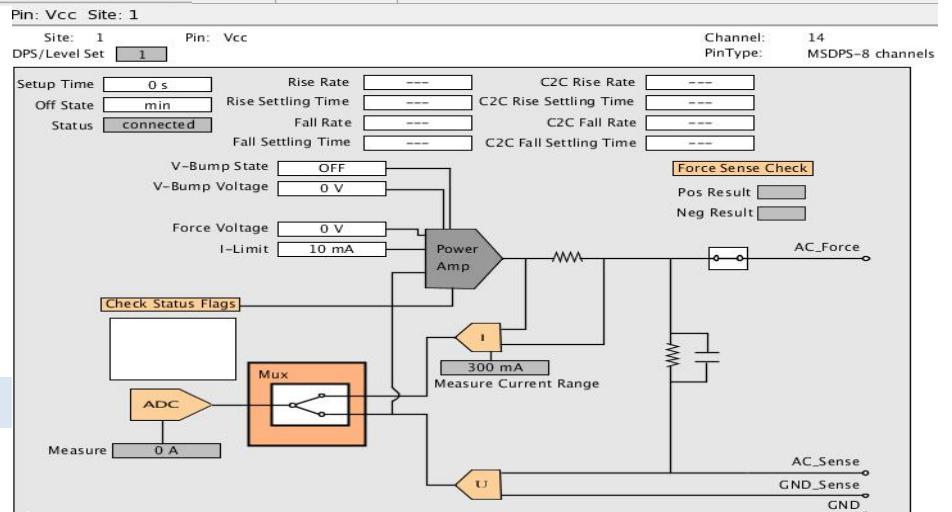
Retrieve measured values into array

TM creation via DC sample Code

1. Open the hardware view of the related pin
2. Setup a measurement using the hardware view
3. Execute the test
4. Copy and paste the generated Sample Code to your test method
5. Modify the Test Method code as wanted

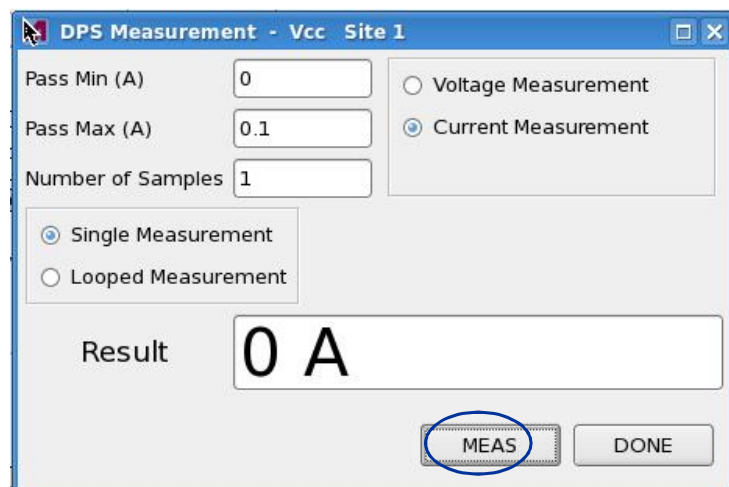
1. DPS measurement @pin Vcc

Analog	Routing	Keep Alive	Routing	Keep Alive	
DPS	Voltage	Voltage			
Vcc	0 V	0 V			
Utility	Chan	S	Chan	S	

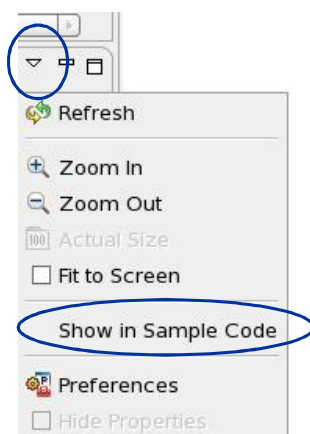


Measurement execution

Measurement execution



Activate Sample Code



```
Pin: Vcc Site: 1
//DPS Current Measurement
DPS_TASK dpsTask;
ON_FIRST_INVOCATION_BEGIN();
    dpsTask.pin("Vcc").min(0 A).max(0.1 A).measurementMode(TM::MEASURE_CURRENT);
    dpsTask.samples(1).execMode(TM::PVAL);
    dpsTask.execute();
ON_FIRST_INVOCATION_END();

double measuredValue = dpsTask.getValue("Vcc");
```

DC Measurements with DPS128

DPS128 implements two instrument operation modes as defined for the DC Scale family of cards:

- DPS instrument mode (DPS)
- PMU instrument mode (SIG)

In DPS instrument mode you can use DC Scale DPS128 cards like a standard DPS, to supply devices with power during the execution of a test flow. Each channel can source and sink current at a programmed voltage or via clamp circuitry, and each channel can measure the voltages and currents supplied.

DC Measurements with DPS128

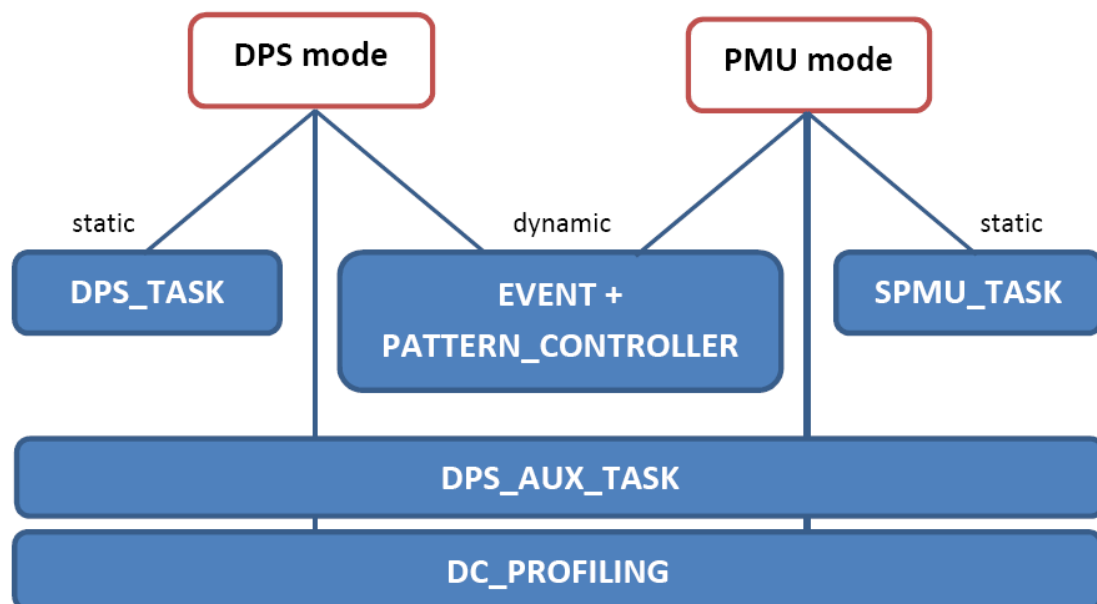
Static “code based” operation:

- Sequencers are disabled, i.e. not taking part in any pattern run
- No triggered actions like vBump (DC Scale is sequencer triggered)
- Asynchronous measurements - “code based”

Dynamic “pattern based” operation:

- Sequencers are active, i.e. taking part in pattern run
- Pattern synchronous actions (force/measure) possible - “pattern based”

DPS 128 Operating modes



Defining Instrument mode

Digital Pins							
							CONTEXT: DEFAULT
	Name	No	Type	Mode	HW	Comment	Tester Channel
							Site 1
1	Signal		dc	SIG	DCS-DPS128HC		20401
2	Signal_gang		dc	SIG	DCS-DPS128HC		20405-20416

DPS Pins						
	Name	Pol	C[uF]	HW	Comment	Tester Channel
						Site 1
1	DPS	POS		DCS-DPS128HC		20501
2	DPS_gang	POS		DCS-DPS128HC		20505-20516

ADVANTEST

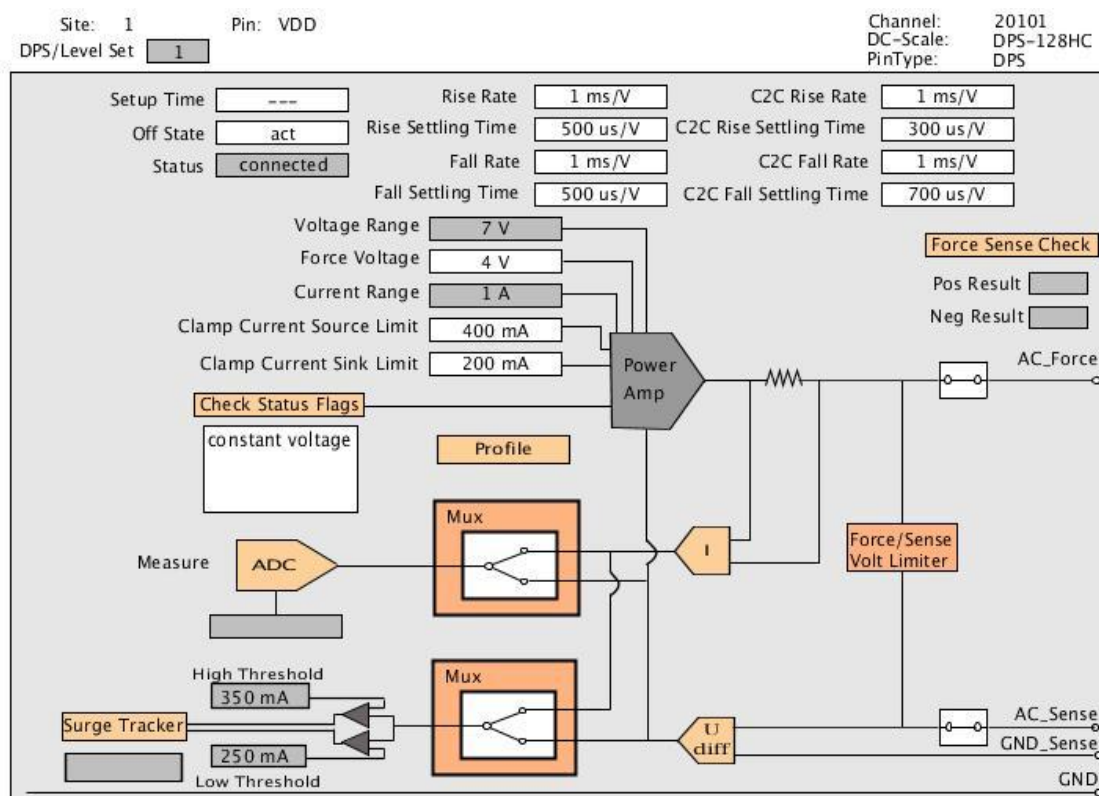
March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 23

Hardware view DPS128 in DPS mode

DPS128



ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 24

DPS Programming examples (static)

Source 10 mA to the device under test and measure expected device output voltage is in the range of [1.25 V, 1.75 V].

As DPS (setup done in level setup)

```
DPSPINS VDD
vout_frc_rng    = 3          # [V]
vout            = 2.5        # [V] clamp voltage
                                # set to a value the DUT will never reach
iout_clamp_rng  = 25         # [mA]
ilimit          = 10         # [mA]
t_ms           = 1          # [ms]
offcurr         = act        # disconnect state
disable_const_curr_check # suppress warning message for current force
```

As PMU (setup in test method code)

```
SPMU_TASK ifvm;
ifvm.pin(sigPin).mode("IFVM").relay("NT")
    .vForceRange(3 V).clamp(2.5 V) // the clamp voltage
    .preCharge(1.5 V)              // V DUT expected, avoid glitches
    .min(1.25 V).max(1.75 V)      // measurement limits
    .iForceRange(25 mA).iForce(10 mA); // the force current
ifvm.execMode(TM::PVAL).execute();
```

ADVANTEST

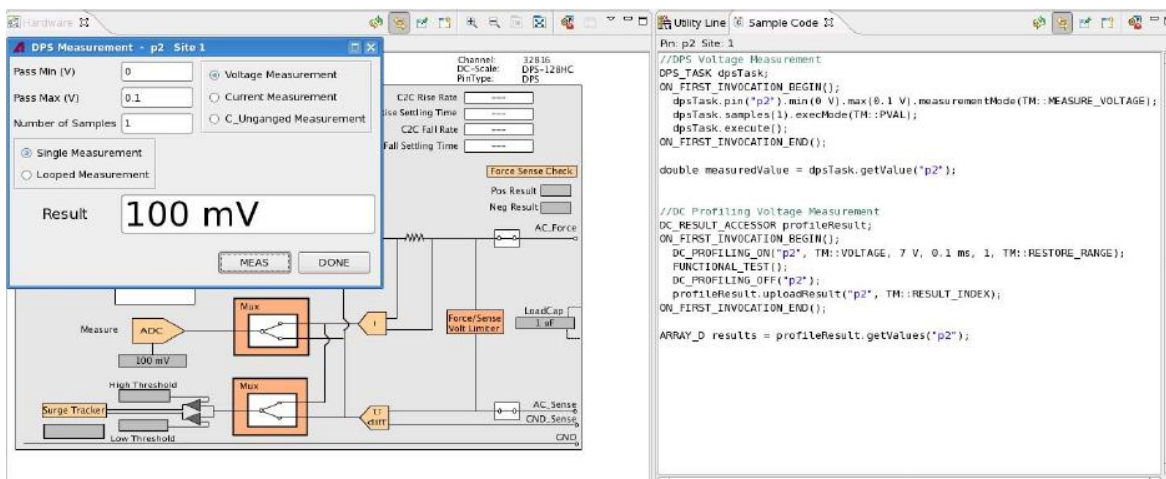
March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 25

DPS Programming examples (static)

Create a measurement in the Hardware view and use the sample code view to generate the test method code and adapt it to your needs.



ADVANTEST

March 2016

All Rights Reserved - ADVANTEST CORPORATION 2016

DC Testing Unit 13 - 26

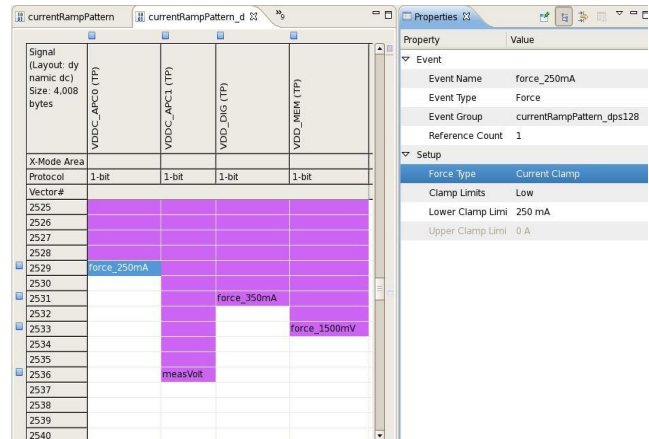
Features Overview – DC Dynamic

Besides the “code-based” DPS128 supports the “pattern-based” DC dynamic execution for the following actions:

Pattern controlled / vector synchronized DC events

- Force / measure voltage and current
- AWG and DGT like operation for voltage and current
- Measurement range down and restore
- Fast pass / fail judgment, contributes to global pass / fail
- Complete SmartTest integration
- “SmartCalc” and Concurrency support

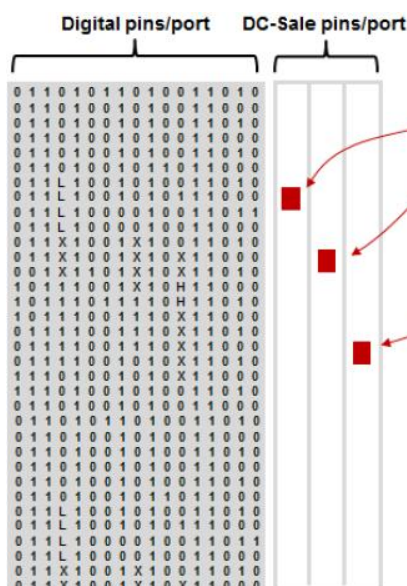
- Setup via Pattern Editor or Testmethod API
- Debug capability via standard result tools, i.e. Error Map, Pattern Editor
- Result visualization with Signal Analyzer



à Common use model for all DC Scale type of cards

ADVANTEST

Dynamic DC Events in Digital Pattern



DC Event

The following DC tasks can be assigned:

- Set Measurement Range
- Force Voltage
- Clamp Current (Force Current)
- Current/Voltage Measurement
- Restore Force Range

Digital Channel:

During the pattern execution each vector is referring to a physical waveform that is sent to the DUT. The physical waveform can drive 0 or 1 or compare the logical level of device output.

DC Scale channel:

During the pattern execution dc events can be executed at certain vector position. A DC event can set a measurement range, force voltage/current or perform voltage/current measurements. The minimum anchor distance between consecutive DC events is 32 vectors.

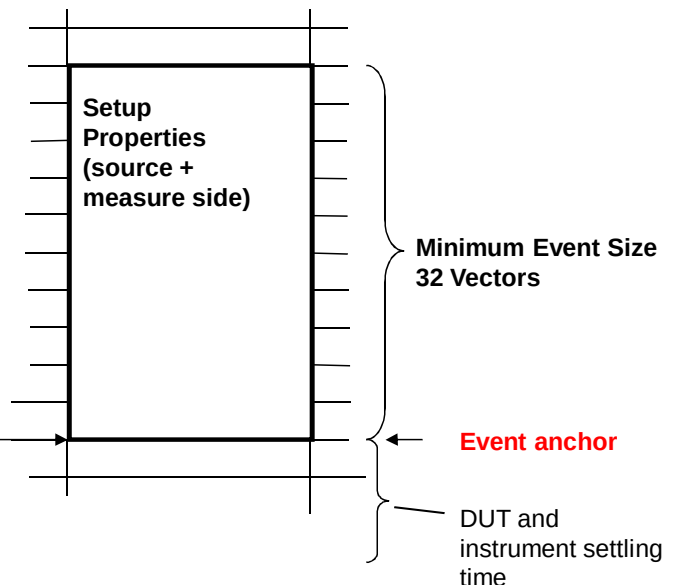
ADVANTEST

Dynamic DC: DC Event

The concept of dynamic testing revolves around “*DC Events*”, consisting of:

- Test Data (Currently only dcsets), which defines the *DC Event* settings - setups and actions. Test Data definitions are stored in the level file.
- Test Points which define when the *DC Event* will be executed within the pattern and which Test Data will be used. Test Points are stored in the vector file.

The vector where DC event is attached to.



The DC Event is embedded into the pattern and has a fixed length of 32 vectors. At the end of the block the actual value is measured. The Test Point is also valid for force voltage or set measurement range events. This event anchor is called the Test Point.

ADVANTEST

DC Events in Pattern

Signal (Layout: dynamic dc) Size: 4,008 bytes	VDDC_APC0 (TP)	VDDC_APC1 (TP)	VDD_DIG (TP)	VDD_MEM (TP)
X-Mode Area				
Protocol	1-bit	1-bit	1-bit	1-bit
Vector#				
2525				
2526				
2527				
2528				
2529	force_250mA			
2530				
2531			force_350mA	
2532				
2533				force_1500mV
2534				
2535				
2536		measVolt		
2537				
2538				
2539				
2540				

Property	Value
Event	
Event Name	force_250mA
Event Type	Force
Event Group	currentRampPattern_dps128
Reference Count	1
Setup	
Force Type	Current Clamp
Clamp Limits	Low
Lower Clamp Limi	250 mA
Upper Clamp Limi	0 A

ADVANTEST

DC Event Programming in test method Editor

```
// test data
EVENT_VM measVolt("measVolt");
measVolt.limitLow(4.9 V).limitHigh(5.1 V).averages(8);

PATTERN_CONTROLLER controller;
controller.label("dcscale_label", "dps128_port")
    .vecEvents("dps128_pins")
    .clear()
    .insert(200, measVolt) // (test point, test data)
    .setup();
```

Units in Test Methods Specs

The parameter type SpecValue is defined to represent a value with unit. Its value is in the format of "value[unit]".

The pre-defined categories and units include:

- Voltages: V, mV, uV, nV
- Current: A, mA, uA, nA
- Time: s, ms, us, ns, ps, fs
- Frequency: THz, GHz, MHz, kHz, Hz
- Bit width: UI

Coding example:

```
addParameter("forceValue",
    "SpecValue",
    &forceValue)
    .setDefault("3800[mV]")
```

Exception handling in UTM code

Via Testflow variable (default)

Tm_crash_as_fatal = off

All sites of the current Testsuite (all sites that are to be executed/already executed by the Testsuite in which the issue happens) are sent to otherwise bin.

All other/waiting sites continue their normal flow (after a restart of the TM-process). Lot testing continues with the next insertion.

Tm_crash_as_fatal = on

All sites that have not been binned yet will be aborted and sent to otherwise bin. Lot testing is aborted and needs user interaction to restart production.

Exception handling in UTM code

User controlled: user registered exception handler

```
TM::EXIT_TYPE myExceptionHandler(const EXCEPTION_INFO
info, void *args){
    cerr << info.getMessage() << endl;
    return TM::ABORT_FLOW();
}
```

```
Somewhere in your flow you register your method:
void run(){
    SET_EXCEPTION_HANDLER(TM:ALL, myExceptionHandler, 0);
}
```


Type of user controlled Exception handling in UTM code

Error Handler Type (TM::EXIT_TYPE)	Description
ABORT_FLOW	All sites that have not been binned yet will be aborted and binned to otherwise bin. Lot testing will be aborted (testflow execution ends after current flow execution). It needs user-interaction to restart the next lot.
EXIT_FLOW	All sites of the current TestSuite (all sites that are to be executed by the current Testsuite) are sent to otherwise bin. All other sites continue their normal flow logic. Lot testing continues (no termination at end of current testflow execution).
RETURN_TO_FLOW	The test method code execution is terminated only for the current test site and Testsuite is set to fail (continues fail branch in case of run&branch TestSuite-node) for the current site only. Test Method code will be called for all remaining sites without any more special handling.
RETRY	The test method program execution is retried only for the current test site after the current result is logged as unreliable.
INTERACTIVE	A dialog box opens. You can select one of the above error handler actions in the dialog box interactively.
THROUGH	The test method program execution is continued even if an error occurs. Note that this is valid only for GPIB and Computation API (GPIB_XXXX and DSP_XXXX).

Reference

V93000 Technical Documentation Center:

Topic 107460

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=107460>

Advantest V93000

Digital User Training

Data Logging

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Data Logging Overview
- EDF (Event Datalog Format)
- EDF Setup
- Report Dialog Window
- Examples
- Datalog Profiles
- Data Formatter

ADVANTEST[®]

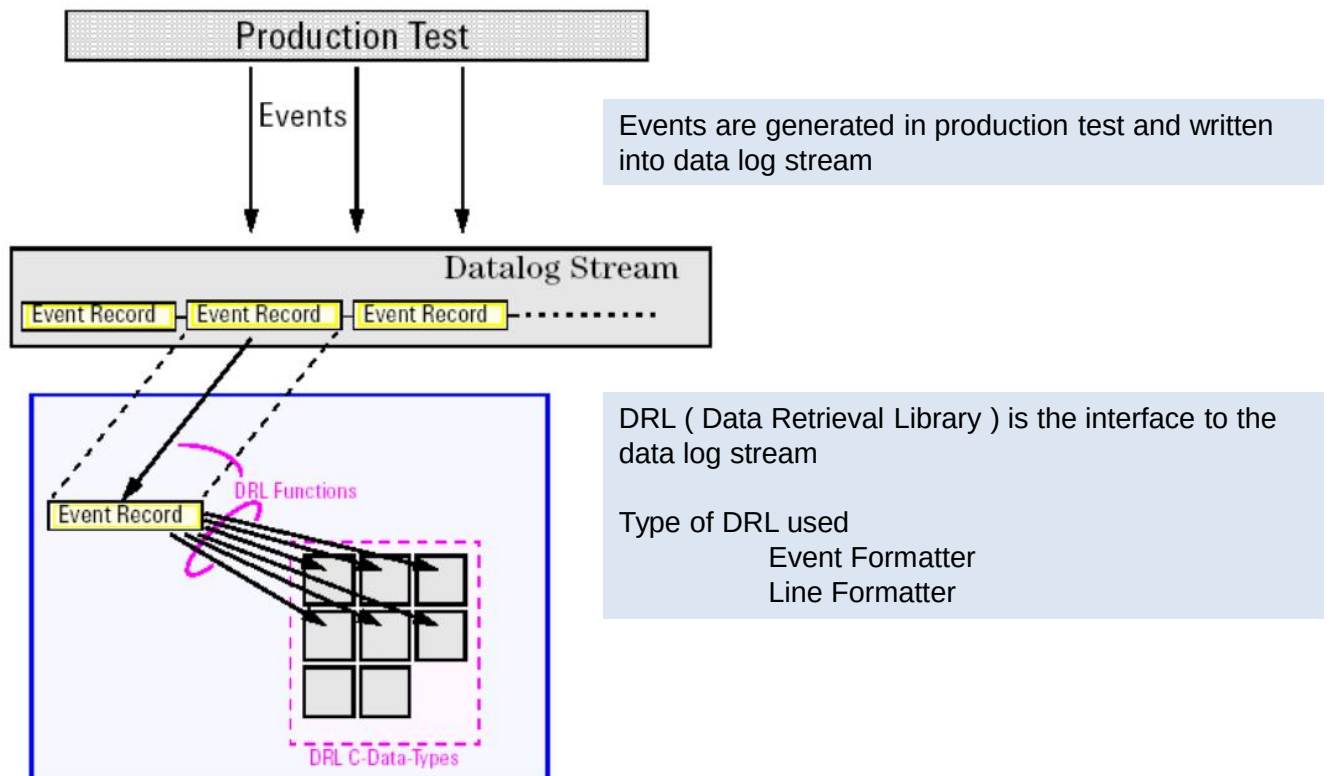
Learning Objectives

- Know the concept of datalogging in SmarTest
- Know how to activate the EventFormatter
- Know how to set datalogging flags

Relevance

- It is important to understand how to retrieve the correct and needed data

Event Data Logging Overview



ADVANTEST

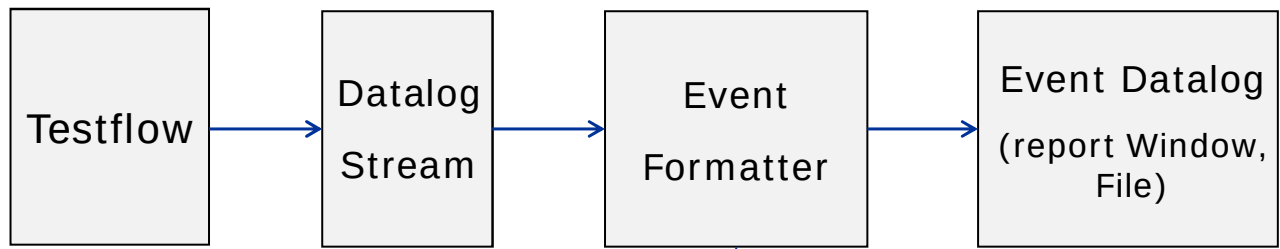
Event Datalog Format (EDF)

EDF Overview

- Event Driven
- Integrated Event Formatter
- Application Programming Interface (DRL-Data Retrieval Library)
- Datalog Data Available in Binary and ASCII Data format

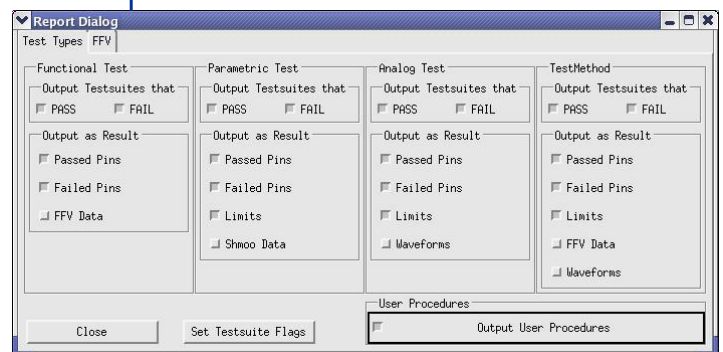
ADVANTEST

EDF Testsuite Flags and Datalog Flags



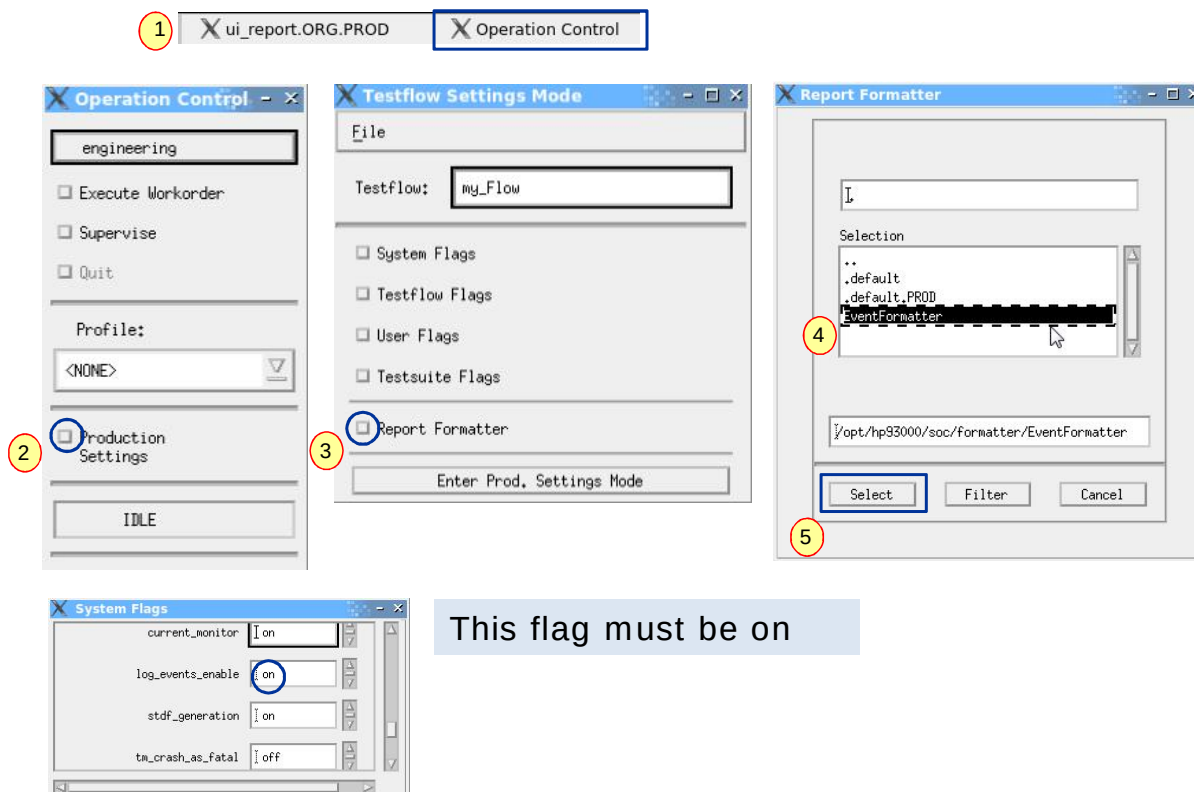
Testsuite flags specify the data written to the Datalog stream

Datalog flags specify the data written to the Datalog (filter settings)



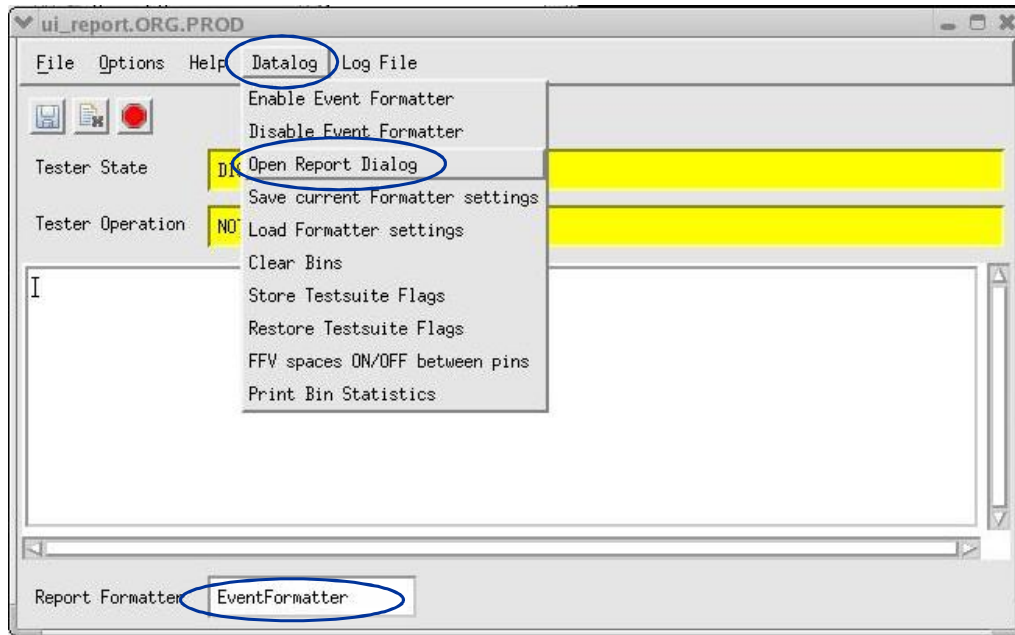
ADVANTEST

Activate the Event Formatter



ADVANTEST

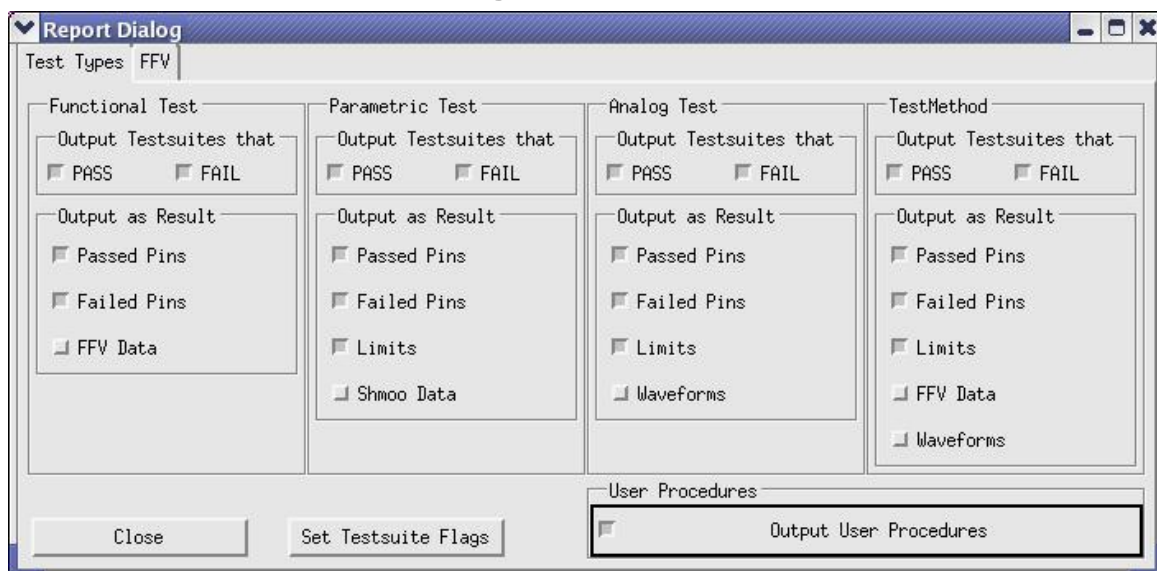
EventFormatter ui_report Datalog Menu



Note: The Report Formatter is now EventFormatter (not .default or .default.PROD)

ADVANTEST

EDF Report Dialog Window

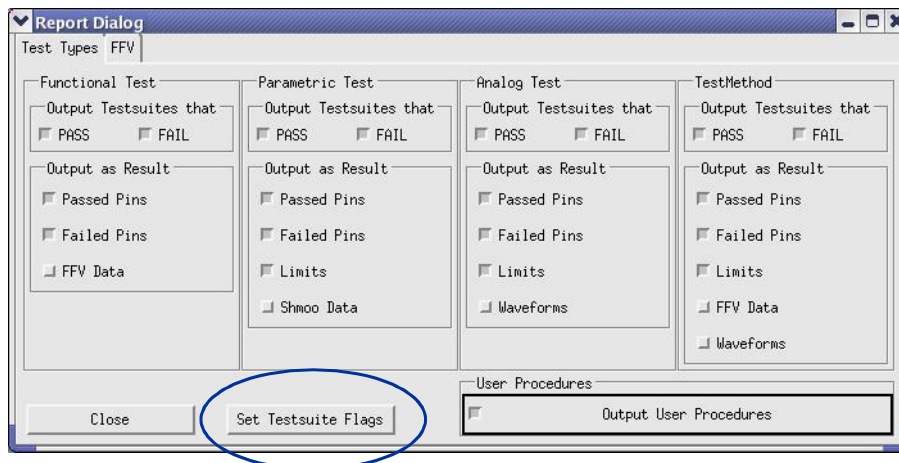


From the Report Dialog window, you can:

- Set log flags for a Testsuite
- Set log flags for pin results
- Set functional test display information
- Set flag for Test Limit display
- Collect shmoo data and waveforms

ADVANTEST

EDF Testsuite Flags and Datalog Flags



Sets the Testsuite Flags according to the datalog flags

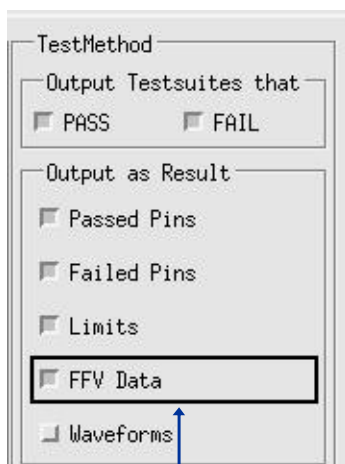
ADVANTEST

April 2016

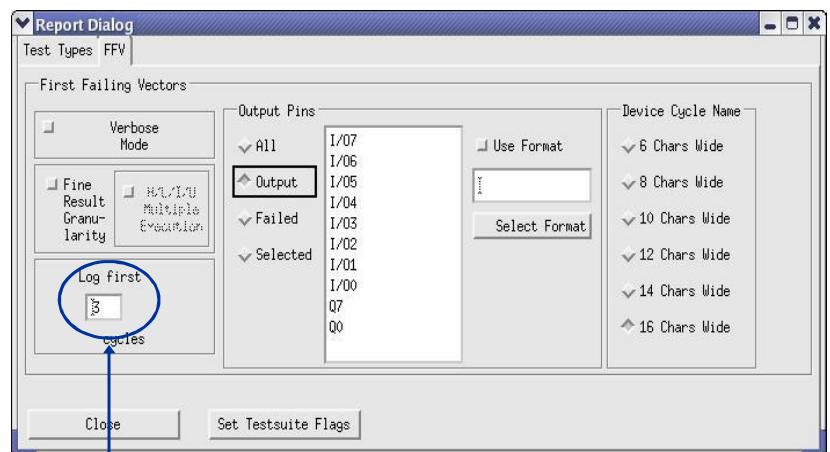
All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 11

Logging FFV Data in the Report Dialog



- If the test fails, FFV (First Failing Vector), data are logged
- Nothing is logged if the test passes



- Logs vector data of the first 3 failing cycles

ADVANTEST

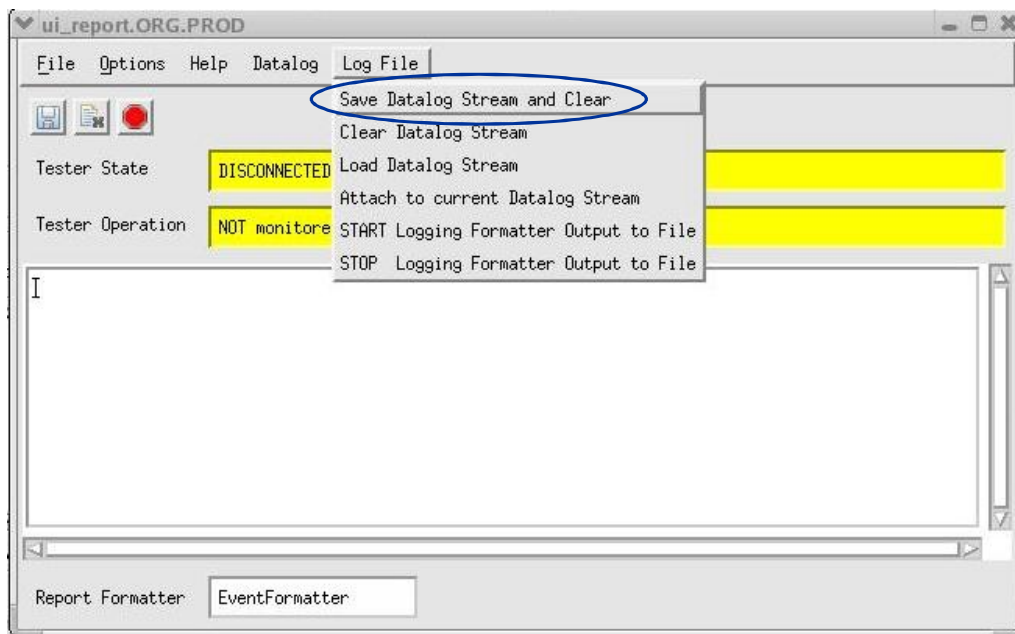
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 12

EventFormatter ui_report Log File Menu

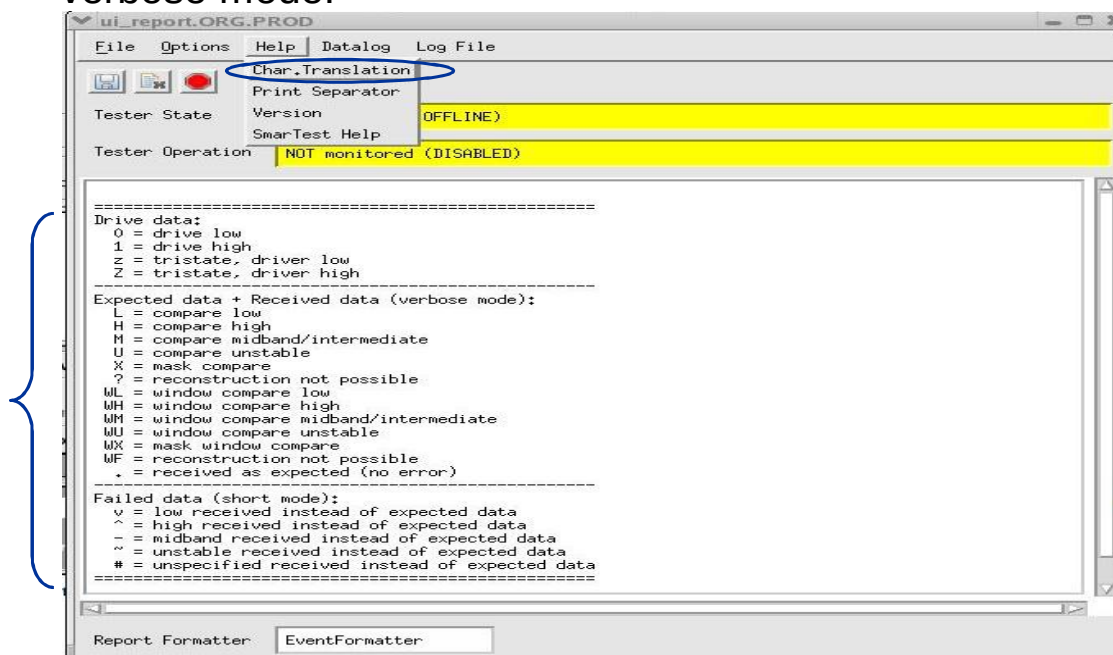
- The EDF datalog stream can be saved from the Log File menu



ADVANTEST

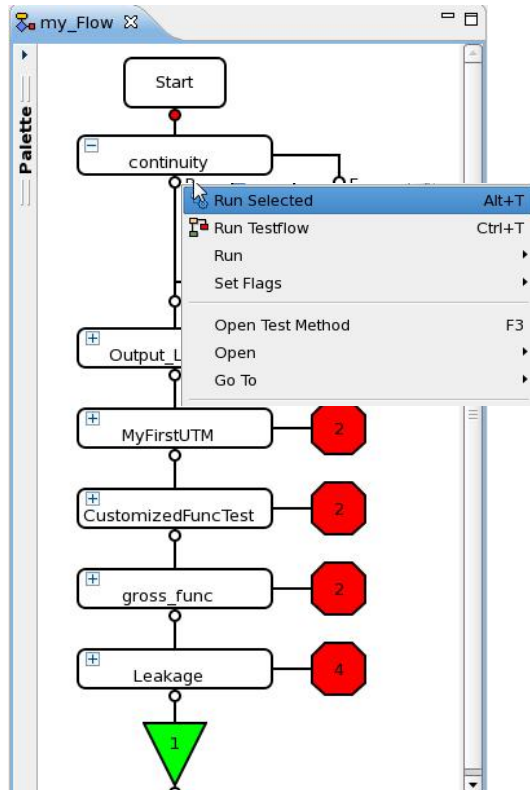
FFV Datalog Data Characters

- This table explains the FFV character data used in the EDF format verbose mode.



ADVANTEST

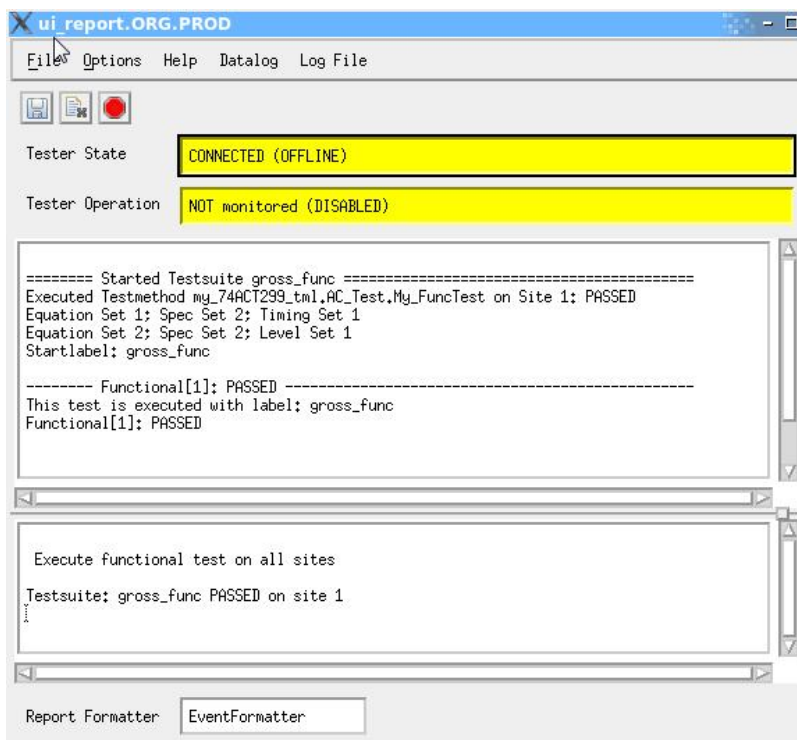
EDF Execution



1. To execute the complete Testflow select Run Testflow
2. To execute e.g. one single Testsuite select Run Selected

ADVANTEST

Example ui_report Window



Upon selecting EventFormatter and execution of Test, the ui_report window splits into 2 sections

- The top section contains Event Datalog information
- The bottom section contains Production Report information

ADVANTEST

Parametric Test Event Datalog Example

```
===== Started Testsuite Continuity =====
Equation Set 1; Spec Set 1; Timing Set 1
Equation Set 2; Spec Set 2; Level Set 1
Startlabel: gross_func

----- passVolt_mV[1]: PASSED -----
passVolt_mV[1]: PASSED
Pass/Fail Limits: [-800.000000 mV, -200.000000 mV]
----- Pin Results -----
CP PASSED -656.139000 mV
DS0 PASSED -637.797000 mV
DS7 PASSED -648.532000 mV
I/00 PASSED -402.563000 mV
I/01 PASSED -516.875000 mV
I/02 PASSED -403.170000 mV
I/03 PASSED -406.424000 mV
I/04 PASSED -402.198000 mV
I/05 PASSED -406.628000 mV
I/06 PASSED -404.969000 mV
I/07 PASSED -407.987000 mV
Q0 PASSED -535.891000 mV
Q7 PASSED -423.728000 mV
S0 PASSED -647.147000 mV
S1 PASSED -645.773000 mV
_MR PASSED -645.772000 mV
```

Test conditions

Test Limits

Per pin PASS/FAIL results

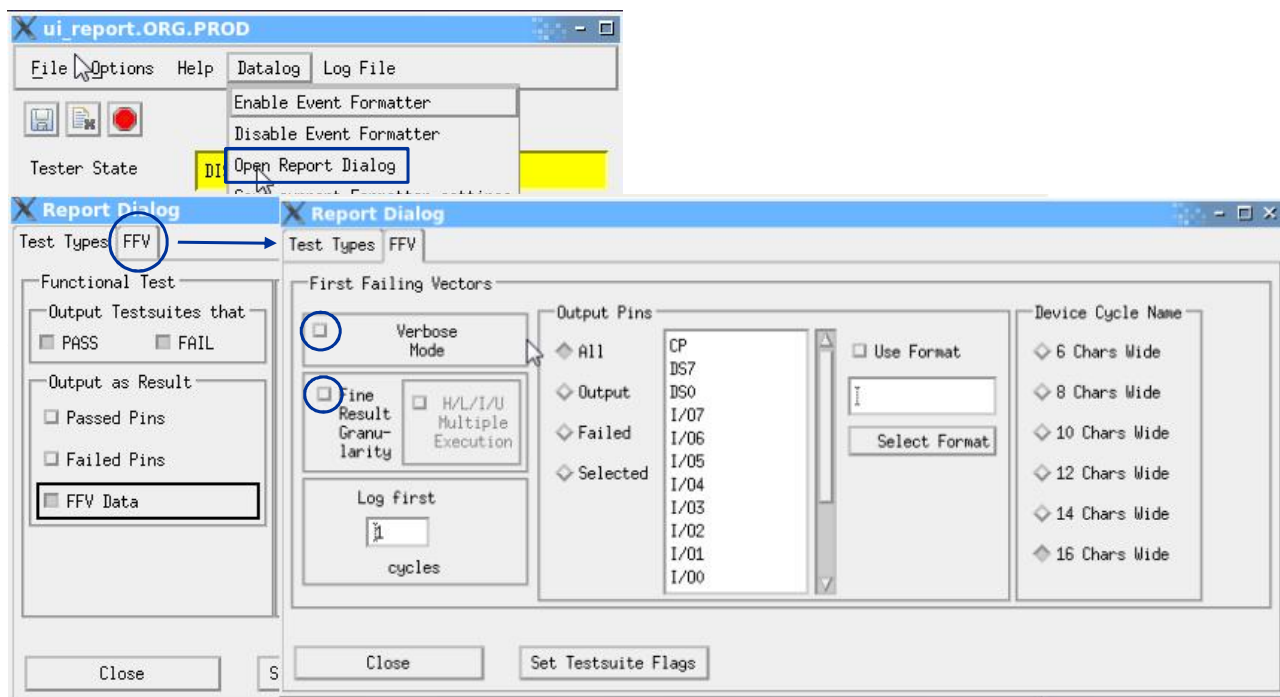
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 17

Verbose Mode/Fine Result Granularity



1. Select FFV Data

2. Select "FRG"

3. Option: Select Verbose

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 18

EDL Verbose Example

Vector Results										
Vector Label: funct_first										
I	I	I	I	I	I	I	I			
/	/	/	/	/	/	/	/			
0	0	0	0	0	0	0	0	Q	Q	
7	6	5	4	3	2	1	0	7	0	
		#			#	#	#	#		
FAILED	L	L	L	L	L	H	L	H	L	L
C0000000004	.	.	.	.	.	.	.	.	L	.
V0000000004	.	.	.	.	.	.	.	.	L	.
FAILED	L	L	H	L	L	L	L	L	L	L
C0000000010	.	.	.	.	L	.	.	.	.	.
V0000000010	.	.	.	.	L	.	.	.	.	.
FAILED	H	H	L	L	L	L	L	L	L	L
C0000000012	.	.	.	.	.	.	.	.	.	H
V0000000012	.	.	.	.	.	.	.	.	.	H
FAILED	L	L	L	L	H	L	L	L	L	L
C0000000016	.	.	.	.	.	.	H	.	.	.
V0000000016	.	.	.	.	.	.	H	.	.	.
FAILED	L	L	H	L	H	L	L	L	L	L
C0000000026	.	.	.	.	.	.	.	H	.	.
V0000000026	.	.	.	.	.	.	.	H	.	.

Pin List

failing pins

Expected Data
(device cycle name)

Received Data

Vector Number Cycle Number

Fine Result Granularity EDL Short Example

```

----- Vector Results -----
Vector Label: gross_func_fail

pass/fail status      I I I I I I I I
DD/////////          -
CSS000000000QQSSM
P70765432107010R
#####              # = failing pins

Failing cycle receive edge
V000000000005 C00000000005 P 100LLLLLHLLLL011
V000000000006 C00000000006 F 100LLLLv^LLLL111
r01          .....v^.....
V000000000007 C00000000007 F 100LLLv^LLLLL011
r01          .....v^.....
V000000000008 C00000000008 F 100LLvL^LLLLL001
r01          .....v.^.....
V000000000009 C00000000009 F 100LvL^LLLLLL011
r01          .....v.^.....
Vector Number → V00000000010 C00000000010 F 100vL^LLLLLvL011
r01          ...v.^.....v....

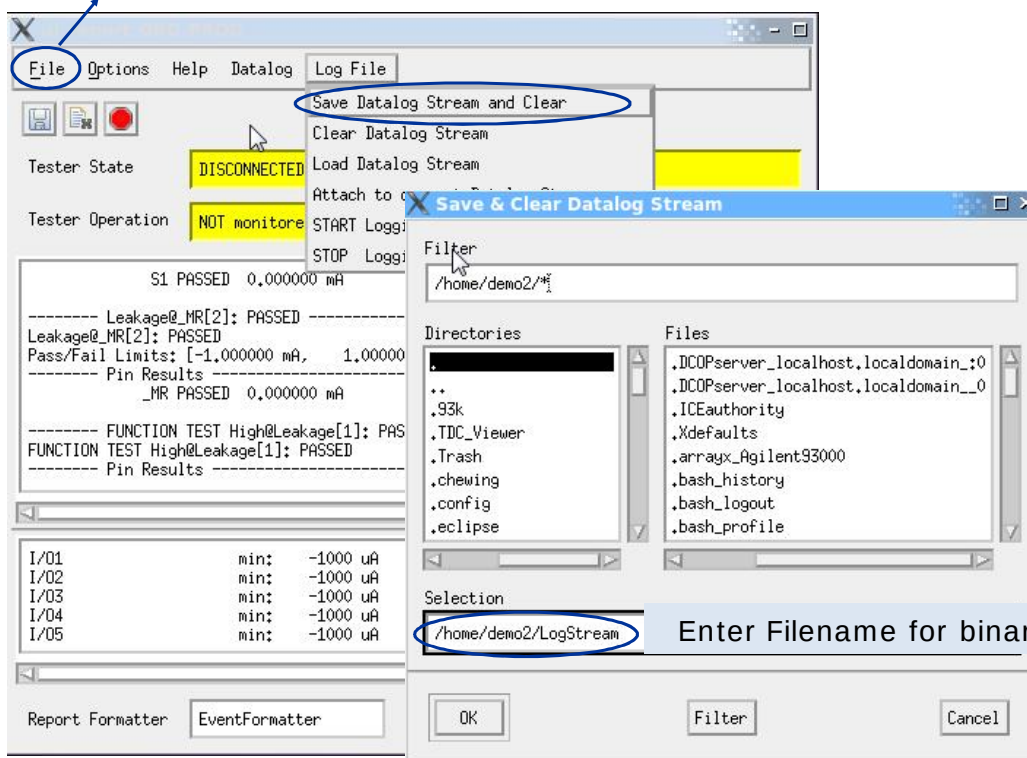
Cycle Number → ===== Ended Testsuite gross_func =====

Failure Symbols: v, ^, -, ~, #

```

Saving the Datalog Stream in EDF

Content of Report section saved in ASCII format via File > Save > ...



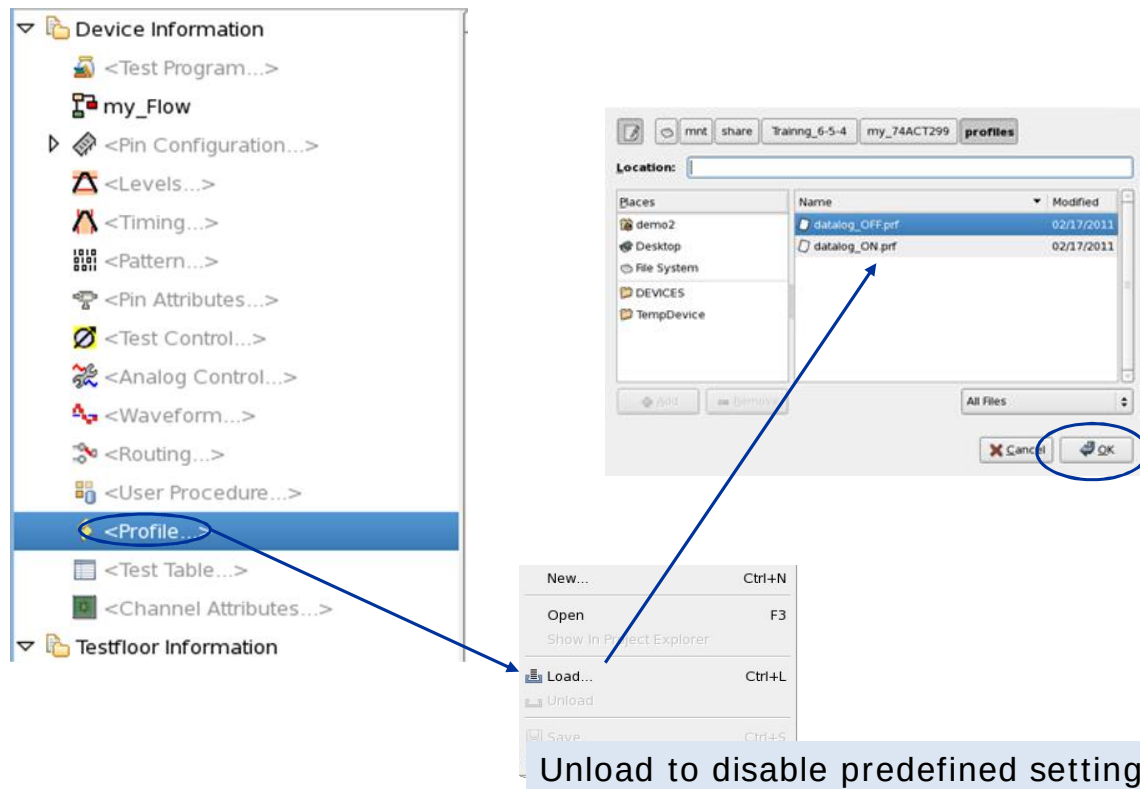
ADVANTEST

Datalog Profiles

- are logical containers for specific data logging rules that determine which data is logged within the test system and how
- provide a very easy and powerful way to control data logging
- Two default data log profiles are delivered with SmarTest: datalog_OFF.prf and datalog_ON.prf
- You can create new data log profiles, which suit your requirements

ADVANTEST

Loading Predefined Datalog profile



ADVANTEST

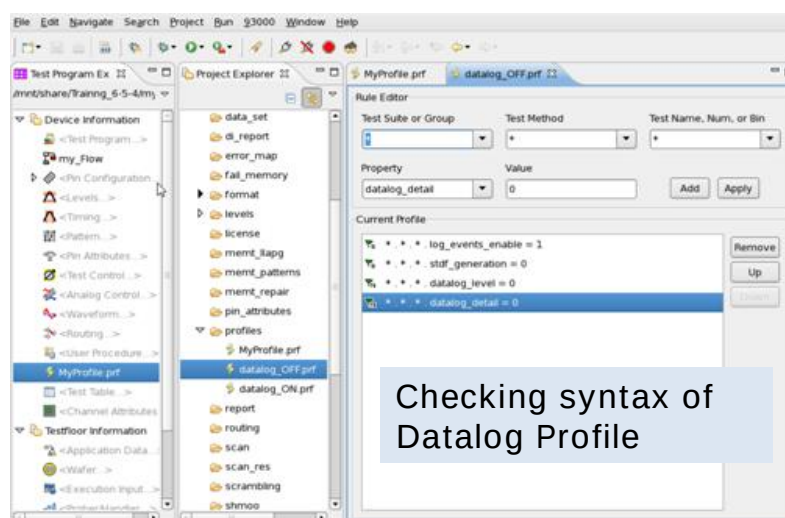
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 23

Change or Check the Datalog Profile

- Rule Editor opened on double-click of Datalog Profile in Test Program Explorer or Project Explorer



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 24

Syntax Sample of Datalog Profile

Example datalog profile

Profile	Explanation
..*.log_events_enable = 1	sets the system flag log_events_enable to on
..*.datalog_level = 3	datalog_level is a convenience property, see table system properties below
..*.datalog_detail = 2	datalog_detail is a convenience property, see the table in System properties below

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 25

Verify Loaded Datalog Profile

In Testsuite Flags page you can globally verify and change settings of the Testsuite Flags

The screenshot shows the Advantest software interface. On the left, a sidebar contains several options: 'engineering', 'Execute Workorder', 'Supervise', 'Quit', 'Profile:', 'datalog_ON.prf', 'Testflow: my_Flow', 'Production Settings' (circled), 'IDLE', and 'Testsuite Flags' (circled). The 'Testsuite Flags' option is highlighted. The main window displays the 'Test Suite Flags' page. It has a menu bar with 'Edit' and 'Select'. Below the menu bar, there are two input fields: 'Selected Group' and 'Selected Testfunction'. To the right of these fields is a 'global_col_set' button. Below the input fields is an 'undo' button. The main area of the window is a table with various flags and their values. The flags are listed in the first column, and the values are in the subsequent columns. The flags include: 'fail value', 'pass value', 'output on fail', 'output on pass', 'hold on fail', 'hold', 'set fail', 'set pass', 'bypass', 'CustomizedFuncT', 'Leakage', 'MyFirstUTM', 'Output_Levels', 'continuity', and 'gross_func'. The values are represented by checkboxes or text. The table is organized into several groups of flags.

Flag	Value
fail value	
pass value	
output on fail	
output on pass	
hold on fail	
hold	
set fail	
set pass	
bypass	
CustomizedFuncT	
Leakage	
MyFirstUTM	
Output_Levels	
continuity	
gross_func	

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Data Logging Unit 14 - 26

Data Formatter

- Data Formatter starts automatically when starting SmarTest
- If the Data Formatter is setup appropriately in the global.conf file, then it will start automatically when a test program is loaded
 - Observe the Datalog Formatter messages in the ui_report window
 - If formatter.stdf.main.generateStdFFromTestflowEditor is true, then STDF data will be collected when the test program is run from the Flow Sequence Editor
- There are many file options in the formatter configuration files (Global.conf & formatter.stdf.main.conf)
- The dataformatter can either read a current EDL datalog stream generated by the test system during production test or a saved EDL datalog stream.

Data Formatter Files

Data Formatter Program

/opt/hp93000/soc/formatter/bin/dataformatter

Tester Level Data Formatter Configuration Files

/etc/opt/hp93000/soc_common/datalog/global.conf

/etc/opt/hp93000/soc_common/datalog/formatter.stdf.main.conf

Device Specific Configuration File

- Create a “datalog” directory within the device directory
- Copy the formatter.stdf.main.conf file from the directory above to the device specific directory and modify it to suit your needs

Viewing STDF as Text

/opt/hp93000/soc/formatter/bin/STDFreader

Configuration Files

formatter.stdf.generateStdF=true

Enable/disables STDF generation

formatter.stdf.detailedFileLocation=\$HOME/stdf

Specifies STDF file directory

formatter.stdf.summaryFileLocation=\$HOME/stdf/summary

Specifies STDF summary file directory

formatter.stdf.testFlowEditorFileLocation=\$HOME/stdf/manual

Specifies location for files generated from manual testflow execution

Reference

V93000 Technical Documentation Center:

Topic 105113

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=105113>

Advantest V93000

Digital User Training

SmartRDI - Introduction

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- RDI introduction
- RDI Structure

ADVANTEST[®]

Learning Objectives

- Introduction to SmartRDI
- Understand SmartRDI structure

Relevance

SmartRDI allows fast and rapid development of digital, DC and other applications.

SmartRDI can be used alone or together with legacy SmarTest APIs.

What is SmartRDI

Smart Rapid **D**evelopment **I**nterface is a API set

- **Easy to use programming interface for test engineers**
 - C++ code in SmarTest IDE
 - Simple and self-explained APIs
 - Common look&feel for all Test Type
 - Test oriented instead of Instrument oriented programming
 - Fast learning curve
- **Automated generation of Sequencer Based Setups**
 - Sequencer-based execution is optimised for test time and repeatability.
 - Automated pattern & burst generation
- **Easy Debugging**
 - Production code is the same as engineering code
 - Step by step execution of auto generated sequencer objects supported

ADVANTEST[®]

April 2016

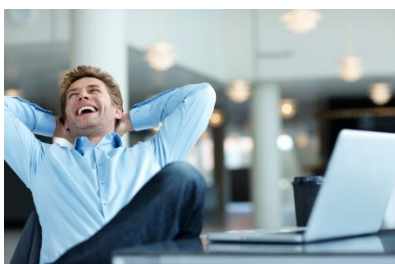
All Rights Reserved - ADVANTEST CORPORATION

RDI Introduction Unit 15 - 5

Test Oriented Programming Model

- run "Mode" Pattern
- send -20dBm 850MHz GSM signal to "LNA" pin
- wait 2 ms
- capture 1024 pts with 1M FS at "BB" pin
- force voltage 3 V in VDD pin
- measure voltage at "Vref" pins
- capture the 100 samples in DPO pin

```
RDI_BEGIN();  
    rdi.func().label("mode").execute();  
    rdi.rfs().pin("LNA").signal(TA::MOD).wave(gsm)  
        .freq(850e6).power(-20).execute();  
  
    rdi.wait(2 ms).  
    rdi.dgt("bb").pin(BB).sample(1024).FS(1e6).execute();  
    rdi.dc().pin("vdd").vForce(3 V).execute();  
    rdi.dc("ref").pin("VREF").vMeas().execute();  
    rdi.digCap("dcap").pin("DPO").sample(100).execute();  
RDI_END();
```



Program like writing a test plan

ADVANTEST[®]

April 2016

All Rights Reserved - ADVANTEST CORPORATION

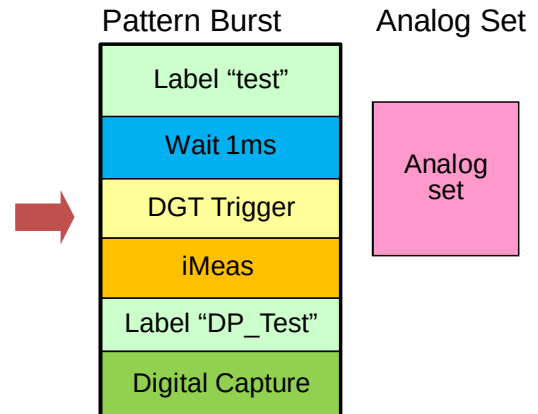
RDI Introduction Unit 15 - 6

Automated Setup Generation by SmartRDI

User Code

```
ON_FIRST_INVOCATION_BEGIN();  
RDI_BEGIN( );  
  rdi.func().label("test").execute();  
  rdi.wait(1 ms);  
  rdi.dgt("cap").pin("B").sample(1024).Fs(10M).execute();  
  rdi.dc("idd").pin(vdd).iMeas().execute();  
  rdi.func().label("DP_Test").execute();  
  rdi.digCap("cap").pin(GPO).sample(1000).execute();  
RDI_END();  
ON_FIRST_INVOCATION_END();
```

V93000 Setup



Bursted Setup; Auto-Generation by Code!

ADVANTEST[®]

SmartRDI User Interface

- **Digital**: execute a functional pattern or create the corresponding label.
`rdi.func().label("PATT1").execute();` **//Sequencer**
- **DC**: perform a voltage measurement at a specific digital pin or create the corresponding label.
`rdi.dc().pin("IO1").vMeas().execute();` **// PPMU**
- **DC**: perform a current measurement at a specific DPS pin or create the corresponding label.
`rdi.dc().pin("VDD").iMeas().execute();` **// DPS**

Standard code for Digital, DC, Mixed Signal and RF.



Readability improved.

ADVANTEST[®]

How to set RDI environment

```
2 #include "testmethod.hpp"
3 #include "rdi.hpp" (2)
4
5 //for test method API interfaces
6 #include "mapi.hpp"
7 using namespace std;
8
9 /**
10  * Test method class.
11  *
12  * For each testsuite using this test method, one object of this
13  * class is created.
14  */
15 class RDI_virgin: public testmethod::TestMethod {
16
17 protected:
18 /**
19  *Initialize the parameter interface to the testflow.
20  *This method is called just once after a testsuite is created.
21  */
22 virtual void initialize()
23 {
24     //Add your initialization code here
25     //Note: Test Method API should not be used in this method!
26 }
27
28 /**
29  *This test is invoked per site.
30  */
31 virtual void run()
32 {
33     //Add your test code here.
34     RDI_INIT() (3)
35
36     //rdi code
37
38 }
```

1/ Create an universal Test Method Class



2/ Add the include "rdi.hpp"



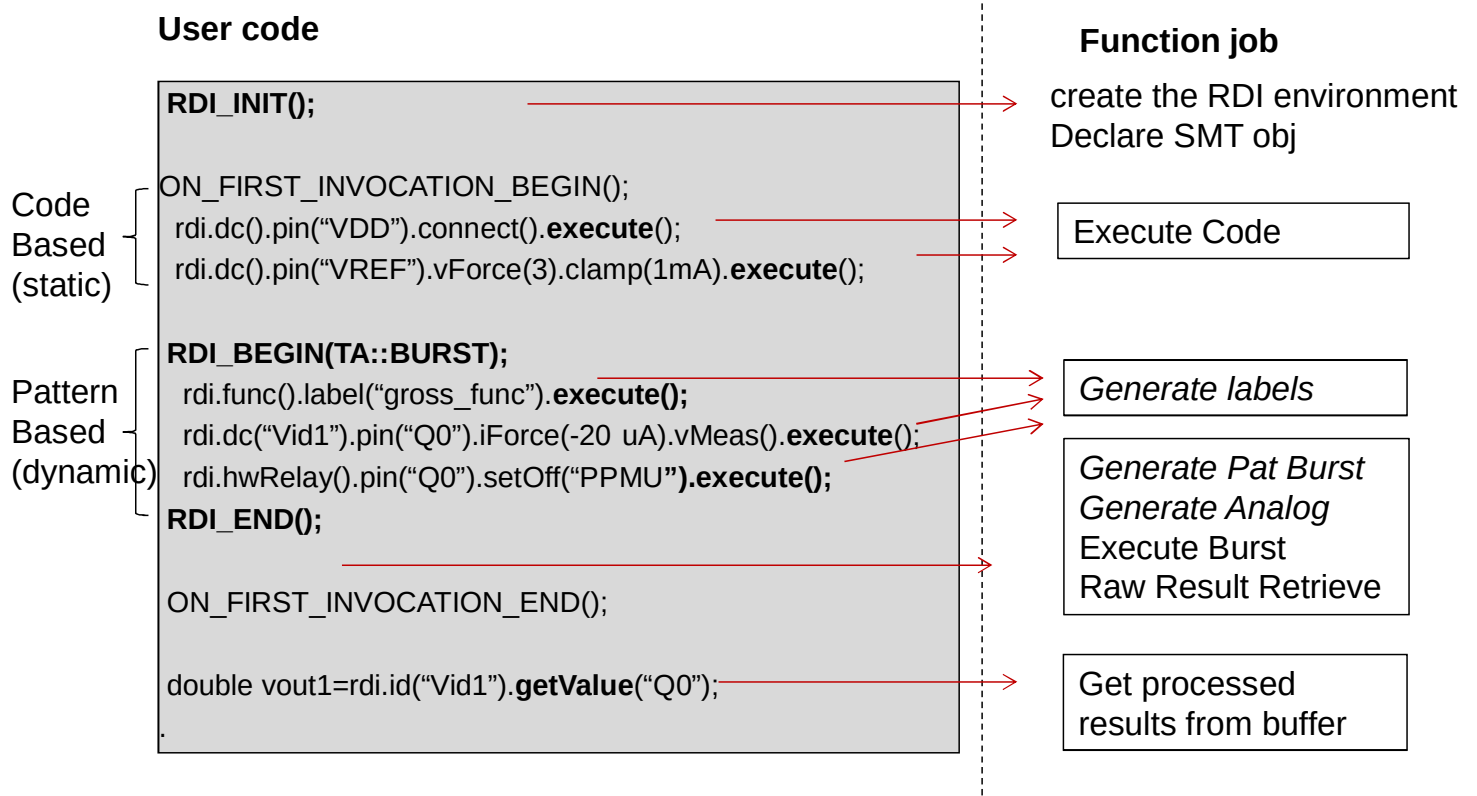
3/ Add the RDI_INIT()



You can now use RDI in your Test Method

ADVANTEST

RDI User code Structure



ADVANTEST

RDI User code Structure

User code

```
RDI_INIT();

ON_FIRST_INVOCATION_BEGIN();
rdi.setPatBurst("Burst_Name");
RDI_BEGIN(TA::BURST);
    rdi.func().label("gross_func").execute();
    rdi.dc("Vid1").pin("Q0").iForce(-20 uA).vMeas().execute();
    rdi.hwRelay().pin("Q0").setOff("PPMU").execute();
RDI_END();

ON_FIRST_INVOCATION_END();

double vout1=rdi.id("Vid1").getValue("Q0");
.
```

Generated Burst

Signal	Burst_Name (Instructions)
Call#	
0	CALL gross_func
1	CALL _ppmu_Q0_if-20u_msa_vm_v0
2	CALL _rly_Q0_OPM_OFF_v0
3	BEND

Note : It is possible to set the burst name with the command :
rdi.setPatBurst("Burst_Name");

ADVANTEST

RDI_BEGIN(TA:BURST)/RDI_END()

- **First Run after the build :**

- Ø Between the RDI_BEGIN/END

RDI generates the different labels corresponding to the actions

- Ø At the RDI_END line

RDI generates and executes the corresponding Burst.

- **Following Runs :**

- Ø Between the RDI_BEGIN/END

Do nothing

- Ø At the RDI_END line

RDI executes the corresponding Burst.

ADVANTEST

Note for RDI_BEGIN()/RDI_END()

- The code inside **RDI_BEGIN()/RDI_END()** block will only be executed in the **1st** run after build.
- In the following runs, the code inside RDI_BEGIN()/RDI_END() block will be **skipped**. So please do **NOT** put any code which you would like to run in **EACH** execution into RDI_BEGIN/RDI_END().
- Different Run modes are available. **BURST** mode, **SINGLE** mode and **PROD** mode give the possibility to generate the labels or not, to assembly them into a burst or not.
More info in ***Debugging with RDI***.
- The default mode is the Burst mode :
Generation and Burst execution.

RDI with other SMT APIs

- RDI APIs are one group of SMT APIs. They **CAN** work well with other SMT commands and setups.

Example:

```
ON_FIRST_INVOCATION_BEGIN();  
CONNECT();  
RDI_BEGIN();  
    rdi.func().label("test").execute();  
RDI_END();  
Primary.getSpecification().change("Tp@port1", 10);  
RDI_BEGIN(TA::BURST);  
    rdi.func().label("gross_func").execute();  
    rdi.dc("Vid1").pin("Q0").iForce(-20 uA).vMeas().execute();  
    rdi.hwRelay().pin("Q0").setOff("PPMU").execute();  
RDI_END();  
ON_FIRST_INVOCATION_END();
```

- If the other SMT commands need to run in each execution. Put them out of the **RDI_BEGIN()/RDI_END()** block.

Result Retrieval : Structure

```
RDI_INIT();

ON_FIRST_INVOCATION_BEGIN();
rdi.dc().pin("VDD").connect().execute();

RDI_BEGIN(TA::BURST);
rdi.func().label("test").execute();
rdi.dc(VDD_meas_1).pin("VDD").iMeas().execute();
RDI_END();

ON_FIRST_INVOCATION_END();

double i= rdi.id(VDD_meas_1).getValue("VDD");
```

1/ Define an **id** when doing a measurement.

2/ Retrieve your measurement by using the **id**

If there is **NO** id in the RDI command, RDI will **NOT** do result retrieve and buffering for that command.

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

RDI Introduction Unit 15 - 15

Without RDI: Pattern Controller

```
PPMU_SETTING setting;
PPMU_RELAY relay_on, relay_off;
PPMU_MEASURE meas1;
PPMU_CLAMP clamp_on, clamp_off;
PATTERN_CONTROLLER controller;
DC_RESULT_ACCESSOR meas1_result
```

2) Definition of Pattern

Controller : Setups will be inserted into a sub pattern.

That Sub pattern must already exist.

3) Execution of the pattern which must call the JSUB.

With RDI



```
Rdi.dc("Vid1").pin("Q0").iRange(100 uA).iForce(-20 uA).vMeas().execute();
Rdi.hwRelay().pin("Q0").setOff("PPMU").execute();
```

1) API declarations and definitions

```
ON_FIRST_INVOCATION_BEGIN();

//Setup for PPMU
setting.pin("Q0").iRange(100uA).min(0V).max(4.3 V).iForce(-20 uA);

//Relay Setups
relay_on.pin("Q0").status("PPMU_ON");
relay_on.wait(1.3 ms);
relay_off.pin("Q0").status("PPMU_OFF");

//Measurement Setups
meas1.pin("Q0").execMode(TM::PVAL);

//Set up the controller
controller.label("Q0_PPMU_meas").add(setting).add(clamp_on)
.add(relay_on).add(clamp_off).add(meas1).add(relay_off);

Primary.label("Pattern_controller_trial");
FUNCTIONNAL_TEST();
ON_FIRST_INVOCATION_END();
```

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

RDI Introduction Unit 15 - 16

Tool function: Convert to string

rdi.itos(int)

↳ Convert create string from an integer value

rdi.itos_hex(int)

↳ Convert create string (hexadecimal) from an integer value

string rdi.dtos(double)

↳ Convert create string from a double value

```
double d =100.001;  
string str_d=rdi.dtos(d);  
  
double i =26;  
string str_i=rdi.itos(i);
```

:

ADVANTEST[®]

Back Up

ADVANTEST[®]

FlexDC coding vs SmartRDI coding

Traditional API code

```
/** PPMU **/  
PPMU_SETTING ppmuSource;  
ppmuSource.pin(ppmuSourcePins).vForce(5 V).iRange(40 mA).min(0  
mA).max(40 mA);  
ppmuSource.wait(1 ms);  
  
PPMU_RELAY ppmuConnectSource;  
ppmuConnectSource.pin(ppmuSourcePins).status("PPMU_ON");  
ppmuConnectSource.wait(0.3 ms);  
  
TASK_LIST ppmuSourceTask;  
ppmuSourceTask.add(ppmuSource).add(ppmuConnectSource);  
  
/** BADC **/  
  
badcMeasTask.pin(badcMeasPins).mode("VM_HIZ").min(badcLowLimit  
V).max(badcHighLimit V).relay("NTMBB");  
badcMeasTask.execMode("PVAL");  
  
/***** EXECUTION SEQUENCE*****/  
ppmuSourceTask.execute(); // execute ppmu force  
badcMeasTask..execute(); // execute badc force
```

SmartRDI code

```
rdi.dc().pin(pmuPins).vForce(5).iRange(40 mA).execute();  
rdi.dc().pin(badcPins,TA::BADC).vMeas().execute();
```

ADVANTEST[®]

ADVANTEST[®]

Advantest V93000

Digital User Training

SmartRDI – Digital and DC Tests

ADVANTEST[®]

Agenda

- Using SmartRDI for Digital Tests
- Using SmartRDI to generate patterns
- Using SmartRDI for DC Tests
- Debugging with SmartRDI
- RDI Lab

ADVANTEST[®]

Learning Objectives

- Know how to use SmartRDI in Digital Tests
- Digital Capture with RDI
- Generate patterns : first steps with smartVec()
- Know to use SmartRDI in DC Tests
- Know how to use SmartRDI when debugging

Relevance

SmartRDI allows fast and rapid development of digital, DC and other applications.

SmartRDI can be used alone or together with legacy SmartTest APIs.

Using RDI for Digital tests

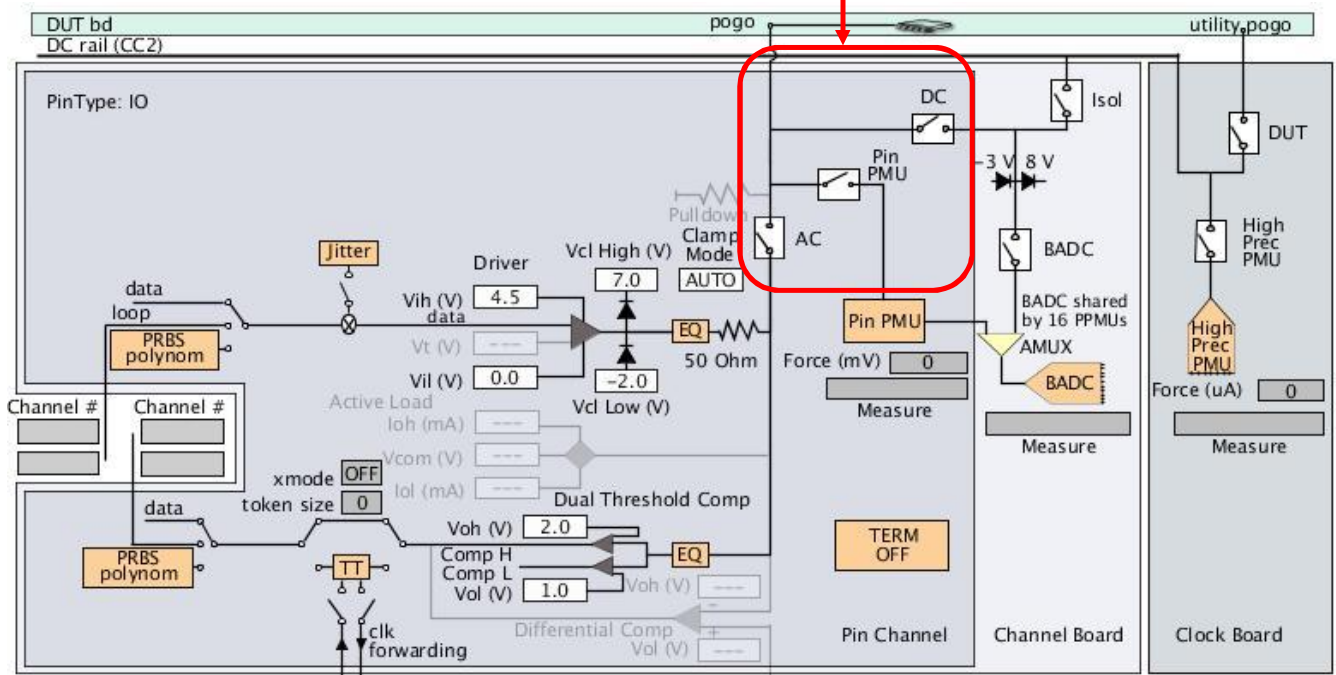
Functional Test/Pattern execution

```
ON_FIRST_INVOCATION_BEGIN();  
  RDI_BEGIN();  
    rdi.func().label("pattern1").execute();  
    rdi.func( "f1" ).label("pattern2").execute();  
  RDI_END();  
ON_FIRST_INVOCATION_END();  
  
int result1 = rdi.id( "f1" ).getPassFail();  
Int result2 = rdi.getBurstPassFail();
```

- **func()**: the functional test class
- **label()**: specify the label name
- **getPassFail()**: get the pass/fail result of a certain label
- **getBurstPassFail()**: get the pass/fail result of the latest burst

Relay control

Pin Electronic relays: AC,DC,PPMU



ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 7

Relay in Digital Channel

```
ON_FIRST_INVOCATION_BEGIN();
RDI_BEGIN();
rdi.hwRelay().pin("pinA").setOn("DC,AC").setOff("PPMU").execute();
rdi.hwRelay().pin("pinB").setOff("ALL").execute();
RDI_END();
ON_FIRST_INVOCATION_END();
```

- **setOn**(string relay) to set relay on
- **setOff**(string relay) to set relay off
- The relay candidates are "**AC**", "**DC**", "**PPMU**" and "**ALL**".
- "**ALL**" = "AC,DC,PPMU"
Support multiple targets, separated by ",". E.g. "DC, PPMU"

ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 8

Other features

- Timing Set and Level Set Change

```
rdi.level().set(int);
```

```
rdi.timing().set(int);
```

It will set CLEV or CTIM in pattern

- Wait time

```
rdi.wait(double);
```

```
rdi.waitCycle(int);
```

- Digital Capture

```
rdi.digCap().pin(" ").execute();
```

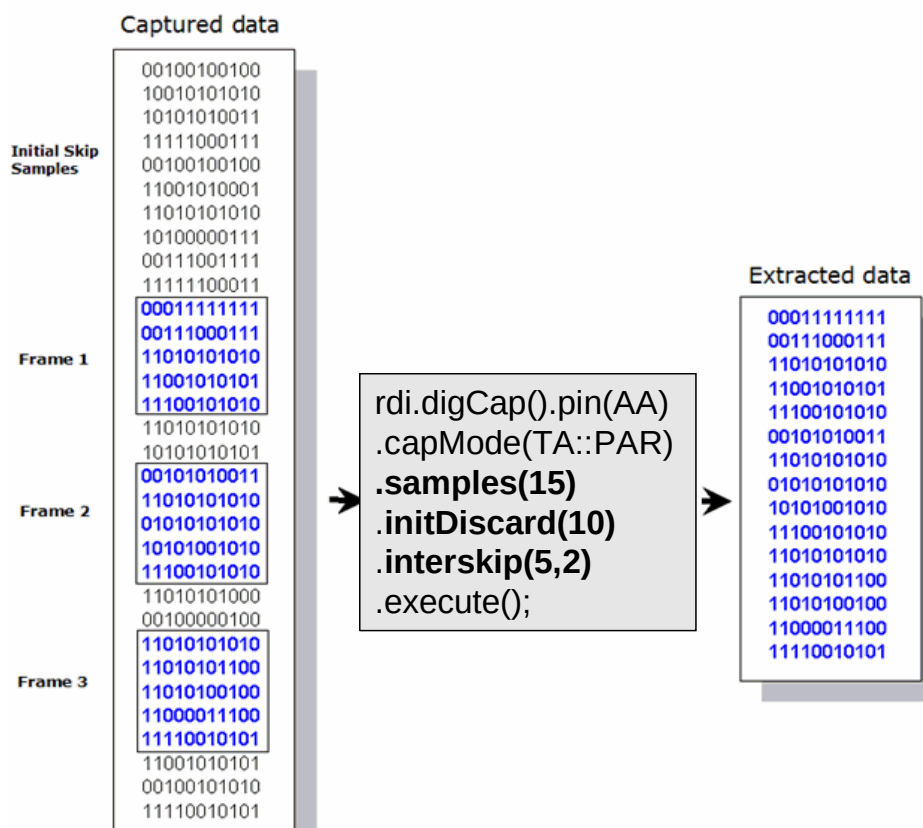
ADVANTEST[®]

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 9

Digital Capture Overview



ADVANTEST[®]

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 10

Digital Capture Capture Mode

- **capMode(TA::SER)**

- Transfer the captured data of only one pin
- **bitPerWord()** sets the bit number of each vector to transfer;
default = 32 bit-per-word.
- Use **getVector()** to get the result vector array
- Use **getVectorBit()** to get an extracted bit array

- **capMode(TA::PAR)**

- Transfer the captured data of more than one pin
- 1 sample per word transfer mode
- With **pin("A,B,C")**, "A" is MSB and "C" is LSB.
- To retrieve the captured result, only **getVector()** is usable.

- Without specifying the **capMode()**, the **default** capture mode is:
 - For one capture pin, default mode is SER and 32bit per word
 - For multiple capture pins, default mode is PAR.

ADVANTEST[®]

Digital Capture API Summary

API	
samples(int)	Number of samples to capture
initDiscard(int)	Initial discard
interSkip(int m,int n)	skip n samples for each m captured samples
capMode(enum)	Capture Mode (PAR or SER)
bitPerWord(int)	Bit Per Word in SER mode
label(string)	Name of an existing label to add digCap
addVecRange(int,int)	the vec range to add digcap in an existing label

ADVANTEST[®]

```
ON_FIRST_INVOCATION_BEGIN();
  RDI_BEGIN();
  rdi.digCap("cap").pin("DO").samples(100).initDiscard(10).interSkip(2,1).execute();
  RDI_END( );
ON_FIRST_INVOCATION_END();

ARRAY_I vec = rdi.id("cap").getVector("DO");
```

- RDI will **create a label** and the **vector variable** to capture samples with initial discard and inter-discard settings.
- RDI will auto check the **X-mode** in the wavetable.
- RDI will auto check the **capture wave index** in the wavetable and apply it in the generated pattern.

Using RDI to generate patterns

Generate Pattern

RDI allows patterns generation :

- Generate pattern with direct vector access
- Generate pattern with logical data
- Generate pattern with burst data

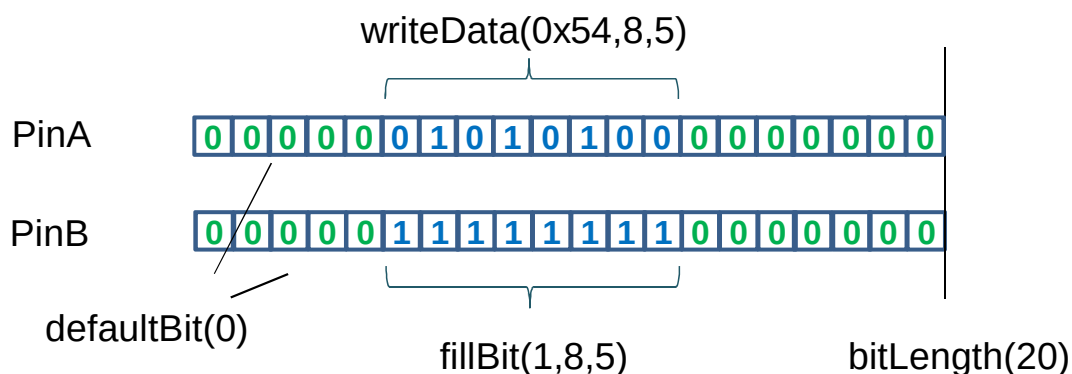
The **smartVec()** class offers different features.

The objective of this part is to give an idea of smartVec capability. An dedicated RDI training is available.

Generate Pattern from Data : Example

RDI allows patterns generation and execution :

```
rdi.smartVec().pin(pinA).writeData(0x54,8,5).defaultBit(0)
                .pin(pinB).fill(1,8,5).defaultBit(0)
                .bitLength(20).execute();
```



Vector Generation – API Summary

- **Vector Layer** : direct control of each vector in the generated pattern.
- **Data Block Layer and Bit Stream Layer** : The pattern content may come from register data, or a bit stream.

	Vector Layer	Bit Stream Layer	Data Block Layer
write...	writeVec(), writePat()	writeBit()	writeData()
default...	defaultVec()	defaultBit()	
fill...	fillVec()	fillBit()	
expect...		expectBit()	expectData()
read...		readBit()	readData()
xMode	No	Yes	Yes
pMode	No	Yes	Yes

ADVANTEST[®]

BurstLabel: Multiple Commands in **One** Label

```
RDI_BEGIN();  
...  
rdi.smartVec().burstLabel("label").begin();  
    rdi.smartVec()...  
    rdi.smartVec()...  
    rdi.wait(...) // can also support wait() in burstLabel()  
    rdi.smartVec()...  
rdi.smartVec().burstLabel().end();  
...  
RDI_END();
```

All smartVec calls are combined to **ONE** single pattern label.

The created label will contain a md5 string postfix for the “content” to ensure different pattern names for different contents.

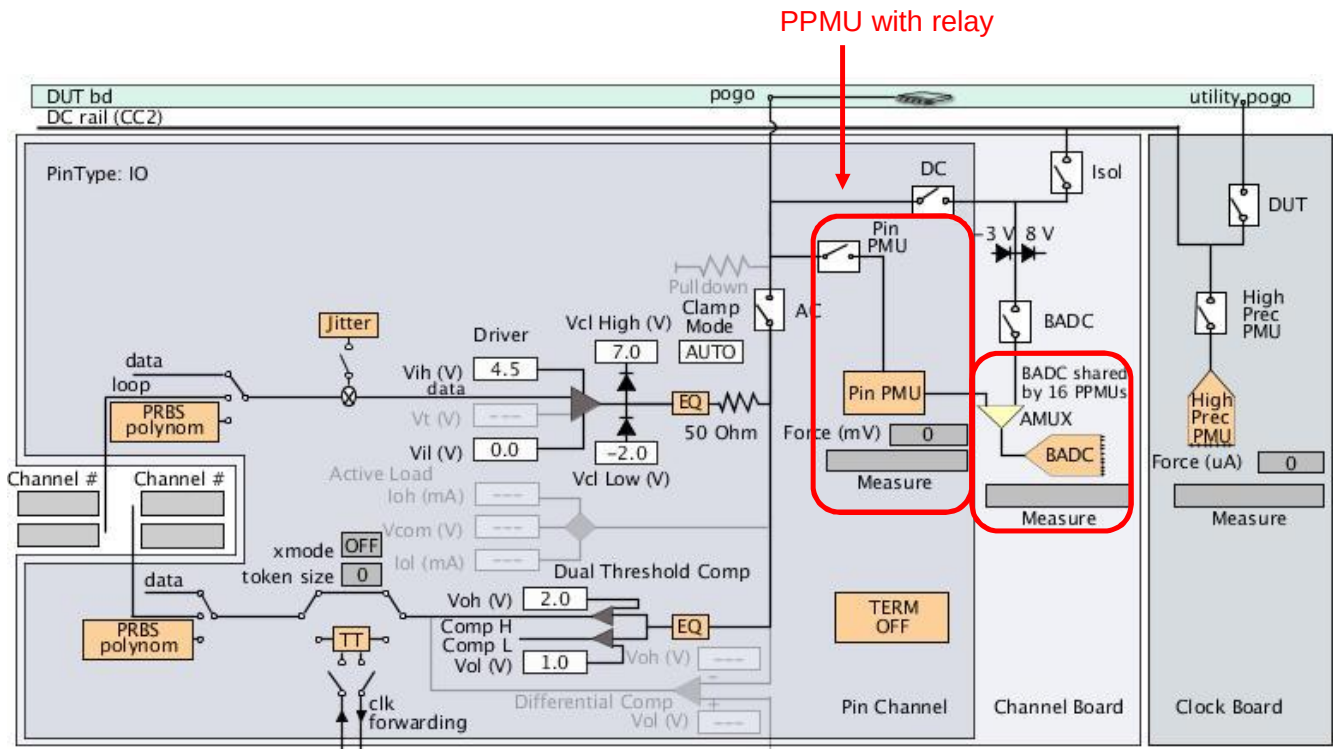
ADVANTEST[®]

Using RDI for DC tests

DC Programming with SmartRDI

- DC programming involves measurement via PPMU or BADC
- How to select PPMU or BADC
- PPMU : How to define measurement mode
- BADC : How to set a measurement
- DPS128 : RDI handles dynamic use

DC Resources: PPMU and BADC



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 21

DC Resources PPMU and BADC

- **Resource selection**

With the pin name, select the instrument :

```
rdi.dc().pin("pinA",TA::PPMU).xxx().execute(); // PPMU
rdi.dc().pin("pinA",TA::BADC).xxx().execute(); // BADC
rdi.dc().pin("pinA").xxx().execute();           // Default = PPMU
```

Digital channels provide the options `TA::PPMU`, `TA::BADC` within the `pin()` parameter.

PPMU is the default selection for digital channels.

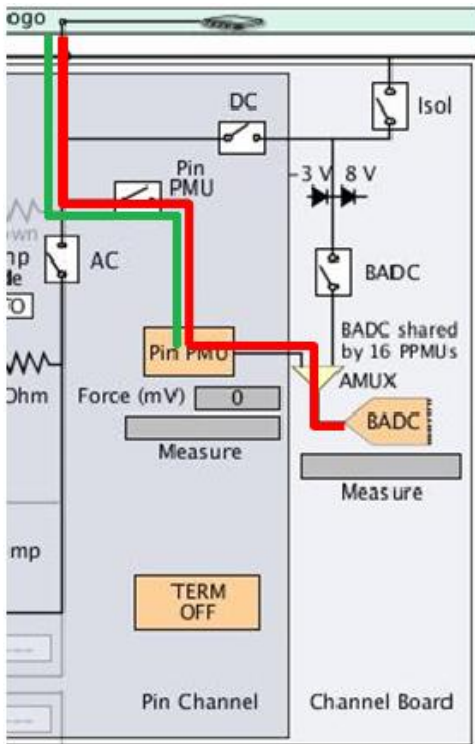
ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 22

PPMU measurement : Measurement mode selection



Using PPMU, two measurements mode are available :

- **SA mode** : Get value by Successive Approximation

```
rdi.dc().pin("pinA",TA::PPMU).xxx().valueMode(TA::SA).execute();  
rdi.dc().pin("pinA",TA::PPMU).xxx().execute();
```

- **BADC mode** : Get value by Board ADC

```
rdi.dc().pin("pinA",TA::PPMU).xxx().valueMode(TA::BADC).execute();
```

ADVANTEST

PPMU measurement : Measurement mode selection (2)

Important : 16 pins shared one BADC.

- **valueMode(TA::BADC)** :

The AMUX will be switched in the pattern, a measWait() might be required to ensure the settling time.

If a BADC is shared by pins or sites the measurement will be serialized automatically.

- **valueMode(TA::SA)** :

This mode works in parallel for any pin and site combination, therefore SA is the default mode.

The **accuracy** of the **BADC** is better than **SA**.

For a single pin measurement the BADC option is much **faster**.

SA value mode : 300 us

BADC value mode: 25 us + 4 us * Average

ADVANTEST

PPMU measurement : Relay action (2)

Relay actions can be defined with the .relay() function:

```
rdi.dc().pin("pinA",TA::PPMU).vMeas().relay(TA:: ppmuRly_xxx,TA:: ppmuRly_xxx).execute();
```

.relay(TA::ppmuRly_xxx,TA::ppmuRly_xxx)



Relay action before the measurement



Relay action after the measurement

Options:

<i>ppmuRly_onPPMU_offACDC</i>	<i>ppmuRly_onPPMU</i> (keep others)
<i>ppmuRly_onAC_offDCPPMU</i>	<i>ppmuRly_onAC</i> (keep others)
<i>ppmuRly_onDC_offACPPMU</i>	<i>ppmuRly_onDC</i> (keep others)
<i>ppmuRly_onACPPMU_offDC</i>	<i>ppmuRly_onALL</i>
<i>ppmuRly_onACDC_offPPMU</i>	<i>ppmuRly_offALL</i>
<i>ppmuRly_onDCPPMU_offAC</i>	<i>ppmuRly_NO</i>

Default: rly_before : TA::ppmuRly_onPPMU_offACDC,
rly_after : TA::ppmuRly_NO

ADVANTEST®

PPMU: API Summary

The parameters available for PPMU programming are :

API	Option	Notes
iForce(double)		Force current
vForce(double)		Force voltage
iMeas()		Current measurement
vMeas()		Voltage measurement
valueMode(enum)	BADC:SA	Method of value measurement
relay(enum,enum)	12 options	Relay before and after PPMU event
clamp(double,double)		Voltage clamp (low, high)
iRange(double)		Current range
measWait(double)		Wait before measurement
average(int)		Number of samples for averaging
getValue(string)		Get measurement result

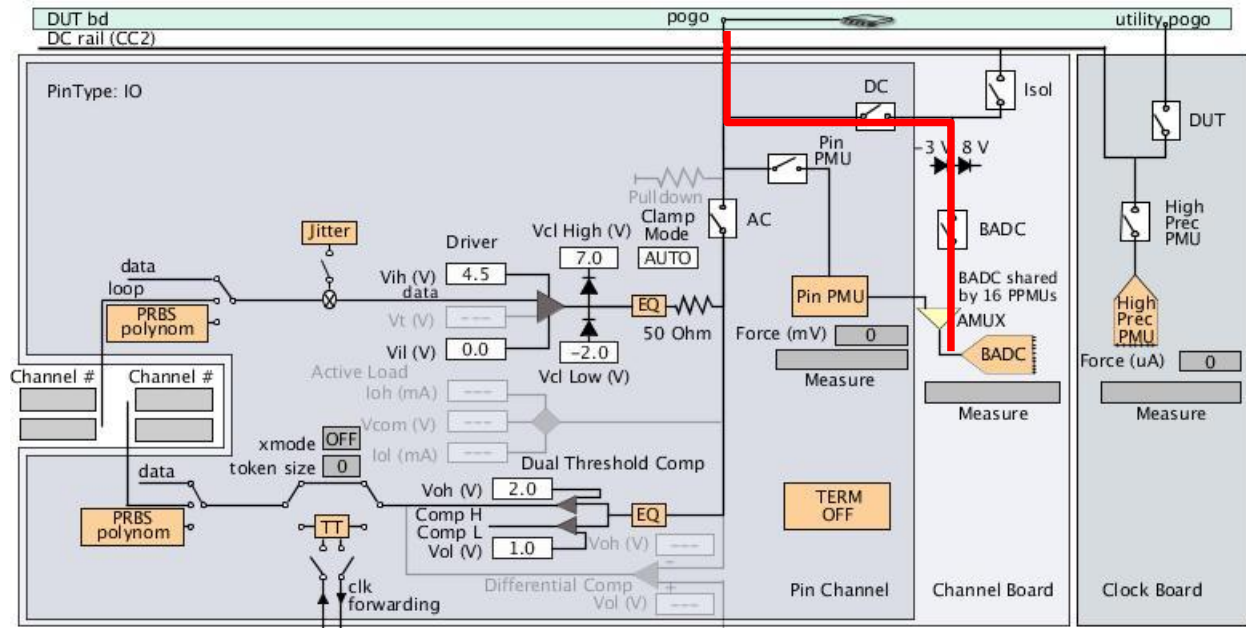
Example:

```
rdi.dc().pin("pinA",TA::PPMU).vForce( 2 V).iMeas().measWait (1.3 ms).valueMode(TA:SA).execute();
```

ADVANTEST®

Pure BADC measurement

```
rdi.dc().pin("pinA",TA::BADC).vMeas().execute(); // pure BADC, vMeas only
```



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 27

Pure BADC measurement

Important : 16 pins shared one BADC.

Pattern-based mode characteristics:

- **1 pin** of a 16 pin BADC group can be used inside one execute().
- The AMUX+BADC relays are switched automatically.
- .relay() option
TA::badcRly_onDC_MEAS_offDC (Default)
TA::badcRly_NO

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 28

Pure BADC: API Summary

The parameters available for Pure BADC programming are :

API	Option	Notes
vMeas()		Voltage measurement
measWait(double)		Wait between relay/mux and meas
average(int)		Averaging to provide better result
relay(enum)	2 options	Relay control

Example:

```
rdi.dc().pin("pinA",TA::BADC).measWait (1.3 ms).vMeas(). execute();
```

Note for Result Retrieval

- If there is **NO** id in the RDI command, RDI will **NOT** do result retrieve and buffering for that command.
- When the measurement is on a pin, but the pin is omitted in **rdi.id().get...()** the **default** result pin is

1. the **only** pin in the rdi...execute() command

```
RDI_BEGIN();  
rdi.dc("i").pin("VDD").iMeas().execute();  
RDI_END();  
double mv = rdi.id("i").getValue(); => VDD pin
```

2. the **1st** pin of multiple pins in the rdi...execute() command and with warning

```
RDI_BEGIN();  
rdi.dc("i").pin("VDD,VCC").iMeas().execute();  
RDI_END();  
double mv = rdi.id("i").getValue(); => VDD pin
```

DC Scale with SmartRDI

- RDI handles DC Scale **setups** and **measurements** dynamically.
- Commands used in the BEGIN/END block will lead to DC events generation
- DPS128 (DPS or SIG mode) can easily be set and used in pattern because of **RDI autoburst generation**.

- For instance, with **Vcc** a **DPS128** resource :

```
RDI_BEGIN();  
rdi.func().label("gross_func").execute();  
rdi.dc("DPS_imeas").pin("Vcc").iMeasRange(25 mA).iMeas().execute();  
RDI_END();
```

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 31

DC-Scale with SmartRDI

The screenshot displays the SmartRDI interface with a test pattern on the left and a properties panel on the right. A red arrow points from the 'CALL _dcs_mr_Vcc_imr_0.025_v0' instruction in the pattern to the 'Event' section of the properties panel. Another red arrow points from the 'EV_IMR_2_Vcc_0.025000_' event in the pattern to the 'Setup' section of the properties panel.

Signal	Burst (Instructions)
Call#	
0	CALL gross_func
1	CALL _dcs_mr_Vcc_imr_0.025_v0
2	CALL _dcs_mr_Vcc_imr_0.025_v0

Signal	Size: 3 bytes	Vcc (TP)
X-Mode Area		
Protocol		1-bit
Vector#	Instruction	...
17		
18		
19		
20		
21		
22		
23		
24		
25		
26		
27		
28		
29		
30		
31		
	STOP	

Property	Value
Event	
Event Name	__EV_IMR_2_Vcc_0.025000__
Event Type	Set Measurement Range
Event Group	_dcs_mr_Vcc_imr_0.025_v0
Reference Count	1
Setup	
Range Type	Current
Range min. Value	0 A
Range max. Value	25 mA
Range Accuracy	Standard

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 32

Debugging with RDI

Run Modes in RDI_BEGIN(enum)

Burst Mode (TA::BURST) : Target mode , DEFAULT

- A sequencer-controlled setup will be generated in the 1st run after build.
- The execution command will run at RDI_END() in the following runs.

• **Single Mode (TA::SINGLE) : Only for single step debugging**

- Each execute() will run the setups/label for this step

• **Production Mode (TA::PROD) : Non-generation mode,**

- Works similarly as **TA::BURST**, but **NO** setup generation.
- RDI will just follow the naming rules to run the needed setups.
- The user still needs to keep alignment between the code and the setup.
The result retrieve information will still be based on the code.

Easy Debug - Single Step Debug

```
RDI_BEGIN(TA::BURST);
rdi.hwRelay().pin("IN3").setOn("PPMU").execute();
rdi.wait(1 ms);
rdi.dc("resID1").pin("OUT1").vMeas().execute();
rdi.dc("resID2").pin("IN3").iMeas().execute();
rdi.hwRelay().pin("IN3").setOff("PPMU").execute();
RDI_END();
```

Switch on relay

Wait 1ms

vMeas.

iMeas.

Switch off Relay



```
RDI_BEGIN(TA::SINGLE);
rdi.hwRelay().pin("IN3").setOn("PPMU").execute();
rdi.wait(1 ms);
rdi.dc("resID1").pin("OUT1").vMeas().execute();
rdi.dc("resID2").pin("IN3").iMeas().execute();
rdi.hwRelay().pin("IN3").setOff("PPMU").execute();
RDI_END();
```

Switch on relay

Wait 1ms

vMeas.

iMeas.

Switch off Relay

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 35

RDI_INFO_LEVEL Flag

- RDI supports an user flag called **RDI_INFO_LEVEL**

	Name	Type	Initial Value	Current Value
1	RDI_INFO_LEVEL	USERFLAG	0	0

- When **RDI_INFO_LEVEL=1**, the SMT commands behind RDI will appear in the UI report window.

	Name ▾	Type	Initial Value	Current Value
1	RDI_INFO_LEVEL	USERFLAG	1	1

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

SmartRDI – Digital and DC Tests Unit 16 - 36

RDI_INFO_LEVEL Flag (2)

```
RDI_BEGIN();  
rdi.dc().pin("Q0").iForce(-20 uA).vMeas().execute();  
rdi.hwRelay().pin("Q0").setOff("PPMU").execute();  
RDI_END();
```

1. API setup
2. Labels generation
3. Burst generation and execution

```
*****  
SmartRDI version : 2.4.2.0  
*****  
rdi:[1] FLUSH(TM::APRM); // initial  
rdi:[1] pmuMeas .pin("Q0").execMode(PVAL).numberOfSamples(1);  
rdi:[1] pmuMeas .pin("Q0").measurementType(VMUM);  
rdi:[1] pmuMeas .wait(0.0003);  
rdi:[1] pmuSet .pin("Q0").iForce(-2e-05) .iRange(0.02 mA);  
rdi:[1] pmuSet .wait(4e-06);  
rdi:[1] pmuRlyOn .pin("Q0").status(PPMU_ON);  
rdi:[1] pmuSet .pin("Q0") .min(-2);  
rdi:[1] pmuSet .pin("Q0") .max(6.5);  
rdi:[1] pc .label("_ppmu_Q0_if-20u_msa_vm_v0_sub",@).add(pmuSet);  
rdi:[1] pc .label("_ppmu_Q0_if-20u_msa_vm_v0_sub",@).add(pmuRlyOn);  
rdi:[1] pc .label("_ppmu_Q0_if-20u_msa_vm_v0_sub",@).add(pmuMeas);  
rdi:[1] pc .setup();  
rdi:[1] -----  
rdi:[1] ppmu_relay1.pin("Q0").status(OPM_OFF);  
rdi:[1] pc.label("_rly_Q0_OPM_OFF_v0_sub",@).add(ppmu_relay1);  
rdi:[1] pc.setup();  
rdi:[1] -----  
rdi:[1] -----  
rdi:[1] FLUSH(); //to delete burst for regen  
rdi:[1] Create pattern burst (_rdi_burst_DC_Measures_bk1) at port : @  
rdi:[1] Primary.label("_rdi_burst_DC_Measures_bk1");  
rdi:[1] SMC_FUNCTIONAL_TEST();  
rdi:[1] -----  
rdi:[1] DC_RESULT_ACCESSOR ra();  
rdi:[1] ra.uploadResult("Q0", TM::RESULT_INDEX);  
rdi:[1] ra.getValues(Q0);  
rdi:[1] rdi_obj(1), get result from pin Q0: 0  
rdi:[1] -----  
rdi:[1] =====
```

site

ADVANTEST

Reference

- V93000 Technical Documentation Center:
- Topic 142069
<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=142069>

ADVANTEST

Lab Exercise

RDI DC and Digital Test Method lab

Advantest V93000

Digital User Training

Digital Debugging

ADVANTEST[®]

Agenda

- Result Analysis Tools Overview
- Functional fail located via Error Map
- Functional fail & response analyzed via Timing Diagram
- Functional fail located via Pattern Debug tool
- Debugging a functional fail

ADVANTEST[®]

Learning Objectives

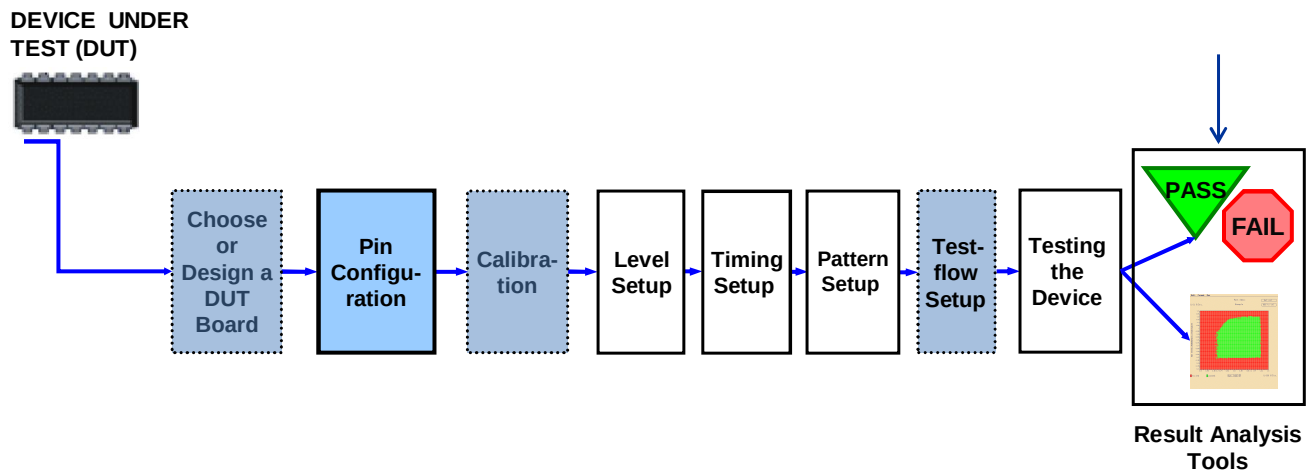
- Know the tools used to debug digital tests:
 - Error Map
 - Timing Diagram
 - Pattern Debug Tool
- Know how to interpret errors in functional test using the debug tools
- Know how to “align” the debug tools
- Know how to prepare a functional test for a debug session

Relevance

- Debugging is needed in many test stages: Wafer Test, Final Test, QA Test; especially during development of a device

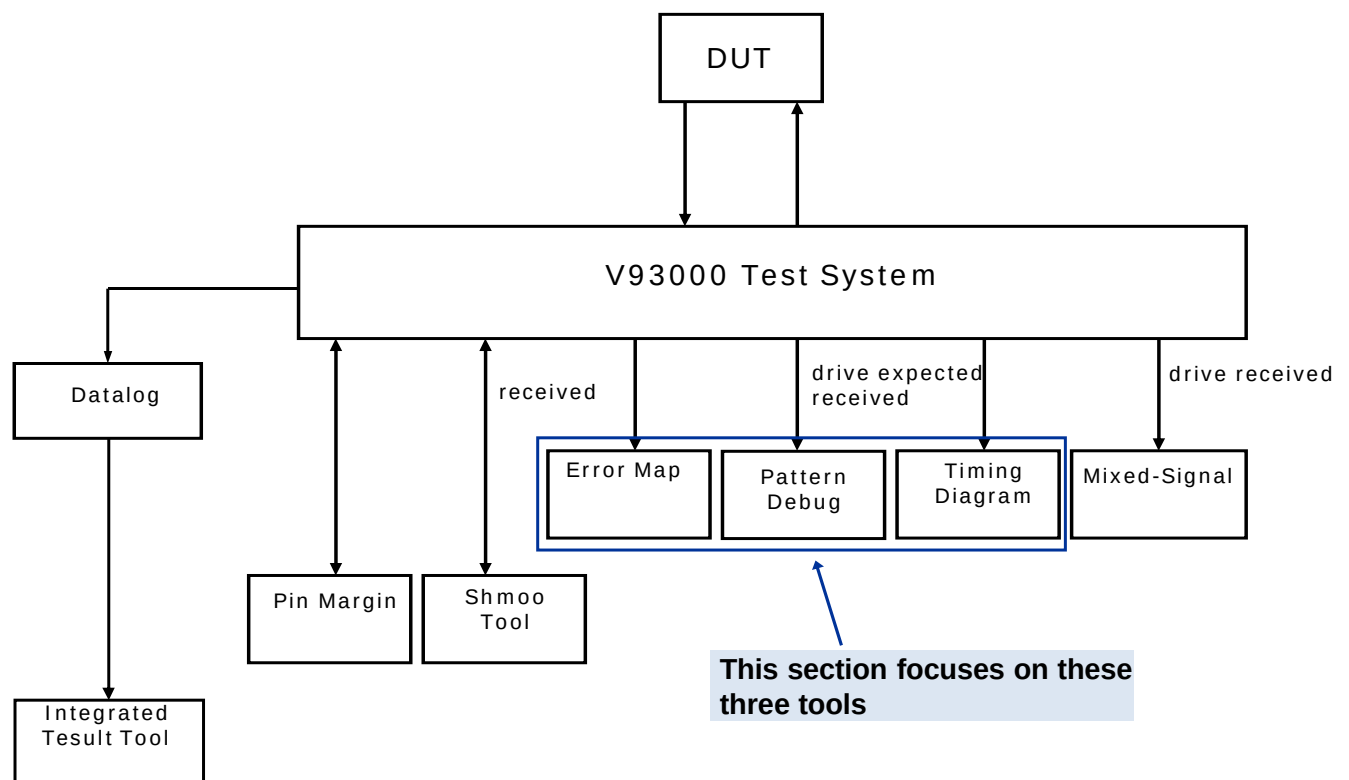
Test Program Flow Chart

Debug the Results



ADVANTEST

Results Analysis Tools



ADVANTEST

Key Resources of Test Result Processing

All result tools are based on functional test results

Reporting

- Error Map:
displays the records information about failing test cycles or strobes
- Timing Diagram:
shows the actual waveforms that are sent to the device, and the response from the DUT's outputs
- Pattern Editor / Pattern Debug Tool:
is used for displaying, editing, searching, and debugging patterns
- Test Method APIs: Users could customize the result report format

Error Map size

Internal Error Map size (inside test processor):

- Smart Scale channel cards: 65536 bits

External Error Map size (RAM):

- Smart Scale channel cards: memory pooling between channels possible

Error Map Recording Modes

Cycle oriented mode

- Logs data for each (passed or failed) test cycle
- One test run provides acquired data for up to 8K cycles in the internal error map, depending on the result granularity (see Result Mode)
- To acquire data for a different cycle block, you need to run the test again after re-focusing the detailed Error Map on the desired block in the overview Error Map
- Upload of data is faster than with failure oriented mode

Failure oriented mode

- Logs only failed test cycles
- Can possibly acquire data for a complete test in one run
- The number of errors which can be acquired is dependent on how the errors are distributed
- The upload of data is slower than with cycle oriented mode

ADVANTEST

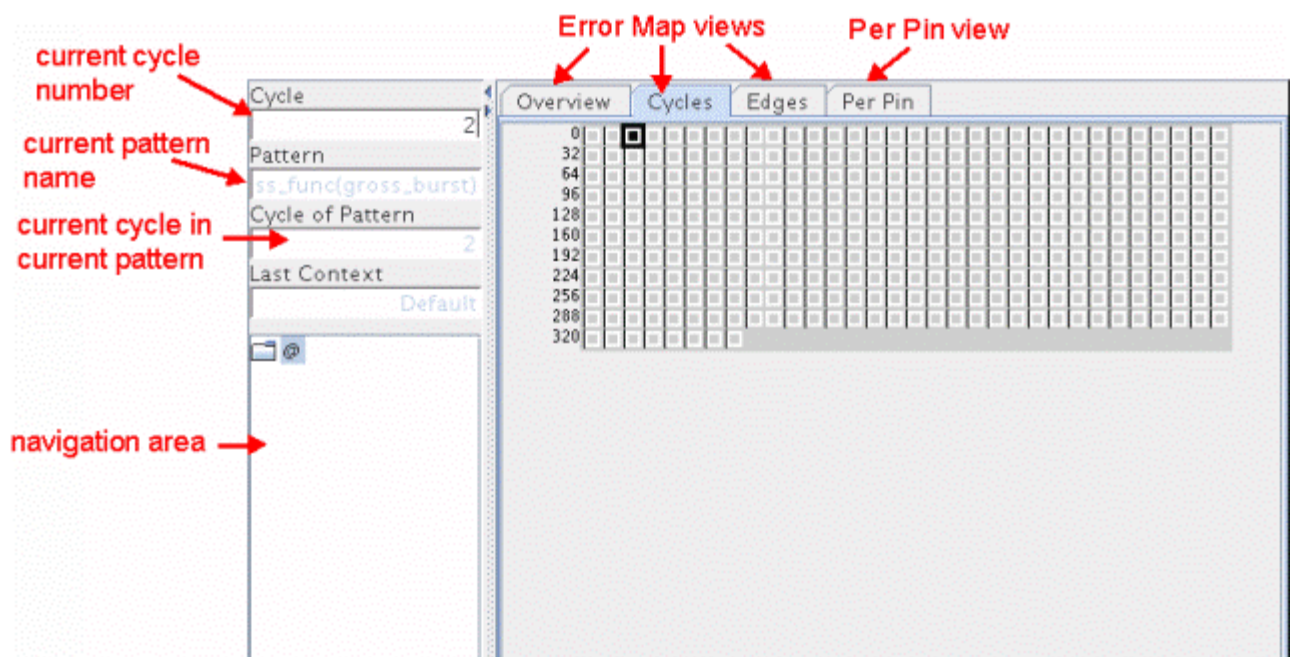
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Digital Debugging Unit 17 - 9

Opening the Error Map

V93000 -> Results -> Error Map



ADVANTEST

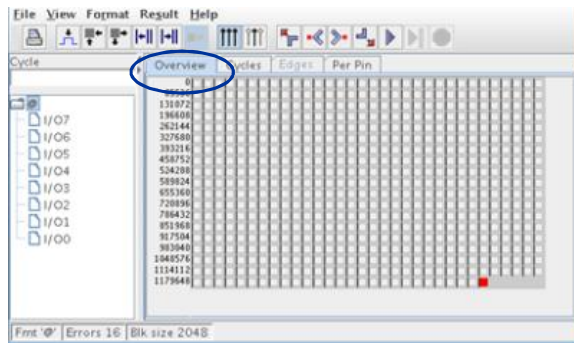
April 2016

All Rights Reserved - ADVANTEST CORPORATION

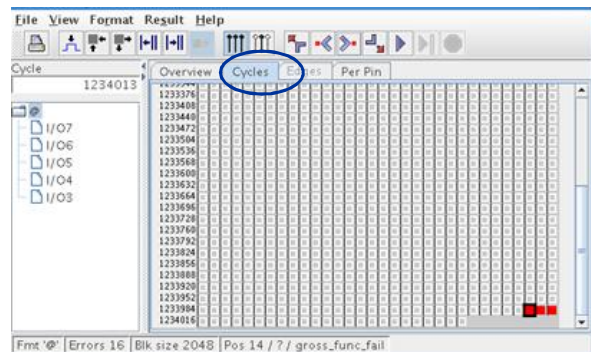
Digital Debugging Unit 17 - 10

Error Map: Overview and Cycles View

Overview Error Map



Cycle Error Map



To analyze failing cycles in detail

1. Click on the desired cycle block in the overview Error Map
2. Click on the  button to re-execute the test
3. Click on the Cycles tab
4. For analyzing instable vectors, re-execute test; this will update Error Map

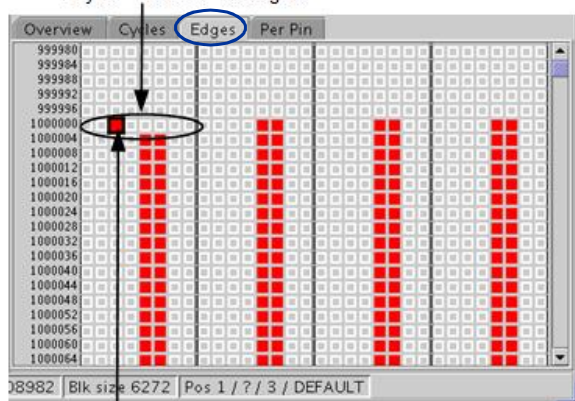
ADVANTEST

Error Map: Edge View and Per Pin View

Edge Error Map

- Gives information of failing strobe within failing cycle
- The error map can store and show the test results of all receive edges

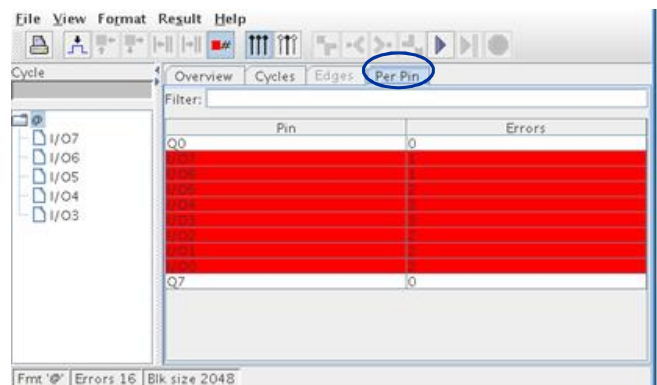
1 cycle = 8 cells = 8 edges



Eight strobe-cells
represent one cycle

Per Pin Error Map

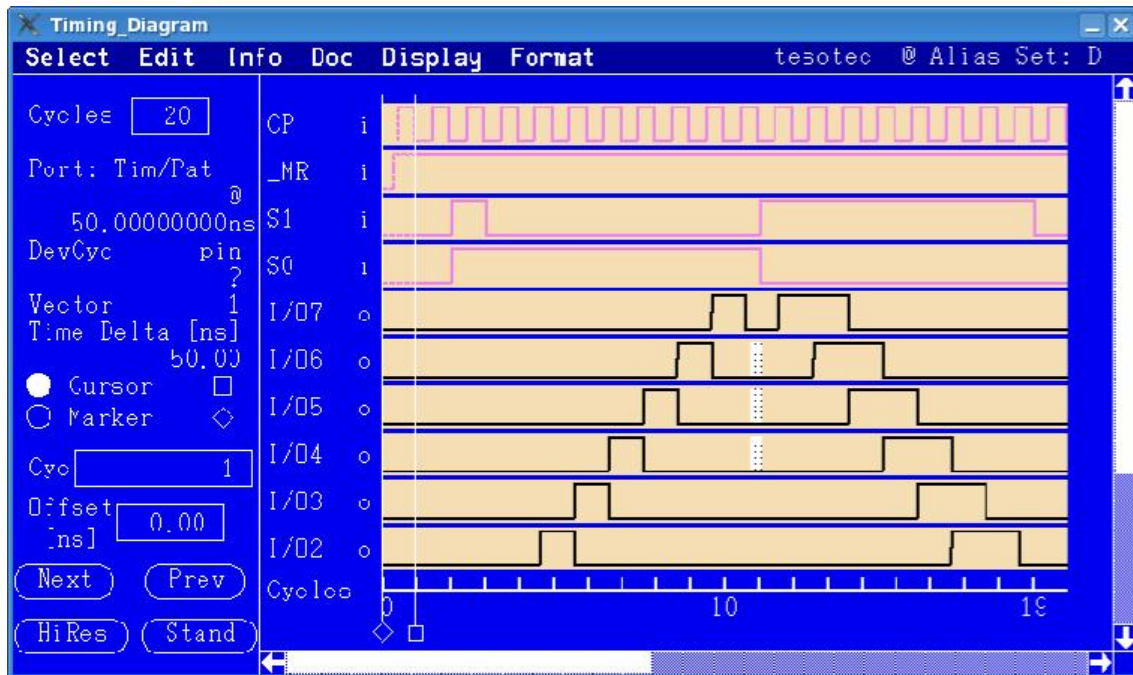
- Gives information of failing pins along with number of fails per pin



ADVANTEST

Opening the Timing Diagram

V93000 -> Results -> Timing Diagram



ADVANTEST

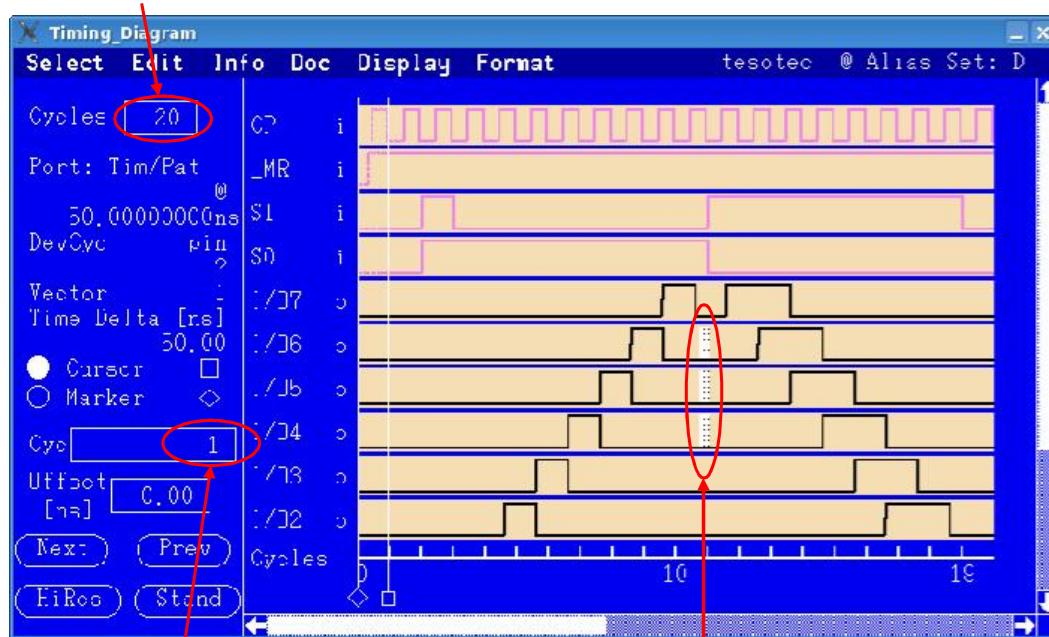
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Digital Debugging Unit 17 - 13

Using the Timing Diagram

Number of cycles shown



Drive
waveforms

Receive
waveforms

Selected cycles

Failing cycles are shaded

ADVANTEST

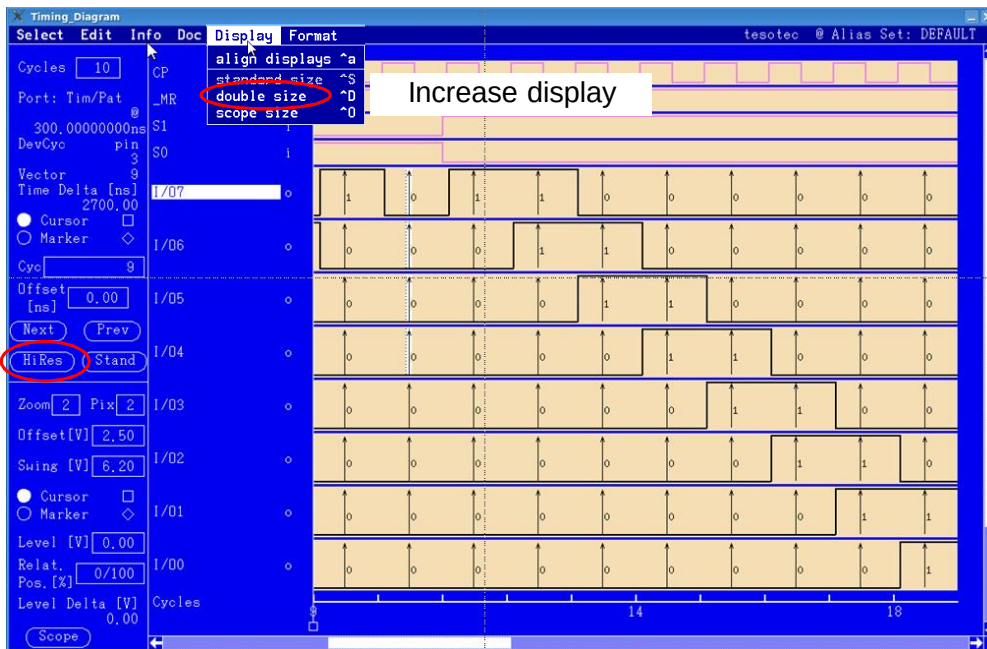
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Digital Debugging Unit 17 - 14

Timing Diagram: High Resolution Mode

- To activate high resolution mode, click HiRes button or 'Select à high res'
- Then functional tests with various edge settings are used to show detailed timing transitions for the chosen level threshold

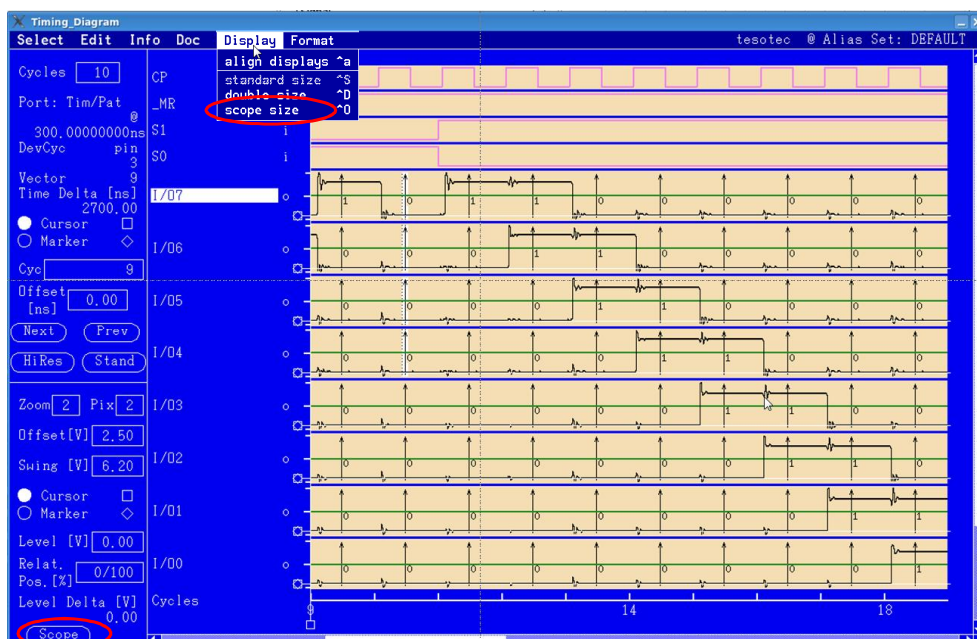


Fixed level
Changed timing

ADVANTEST

Timing Diagram: SoftScope Mode

- To enable SoftScope, change display to scope size
- Start SoftScope by clicking "Scope"
- Then, timing and voltage are varied, producing an oscilloscope-like picture

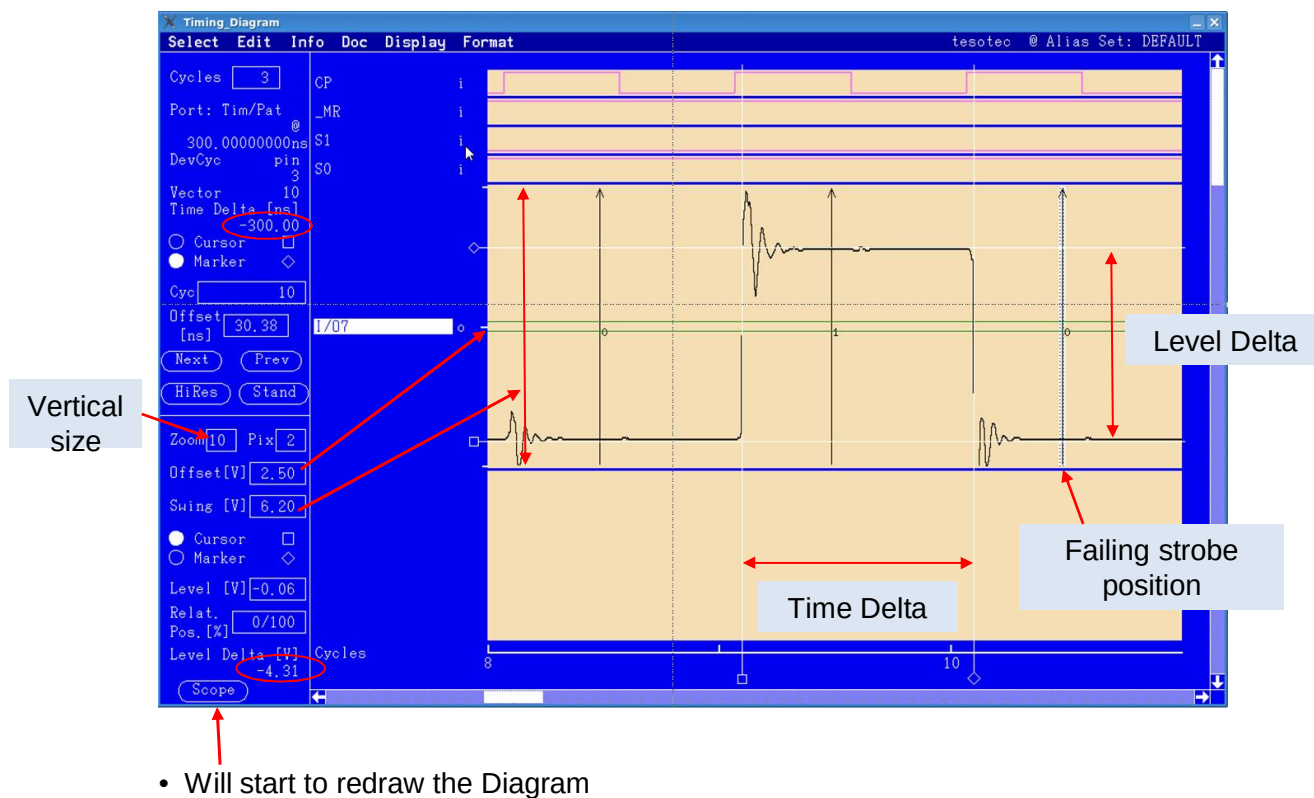


Changed level
Changed timing

Sampling Scope
per Pin!

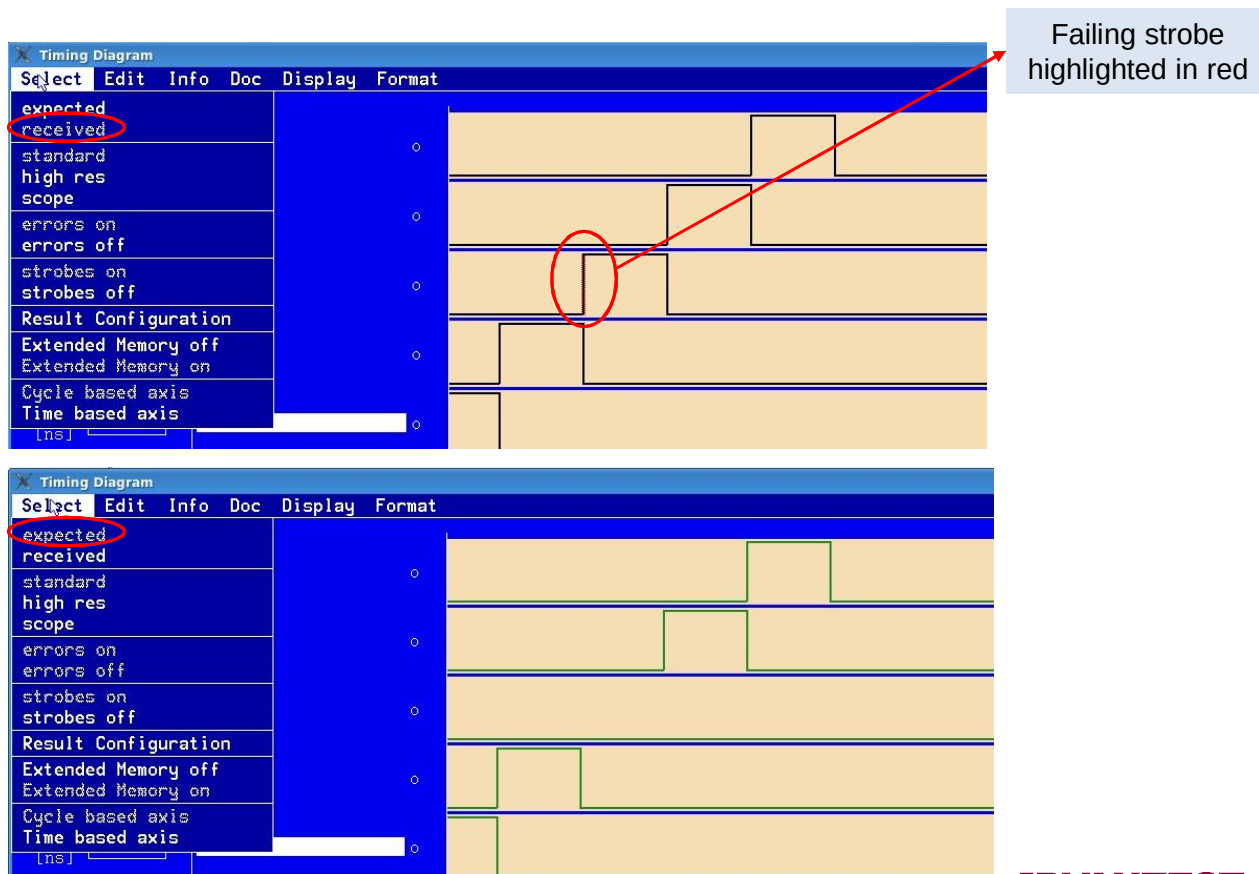
ADVANTEST

Timing Diagram: SoftScope View



ADVANTEST

Timing Diagram: Received vs Expected

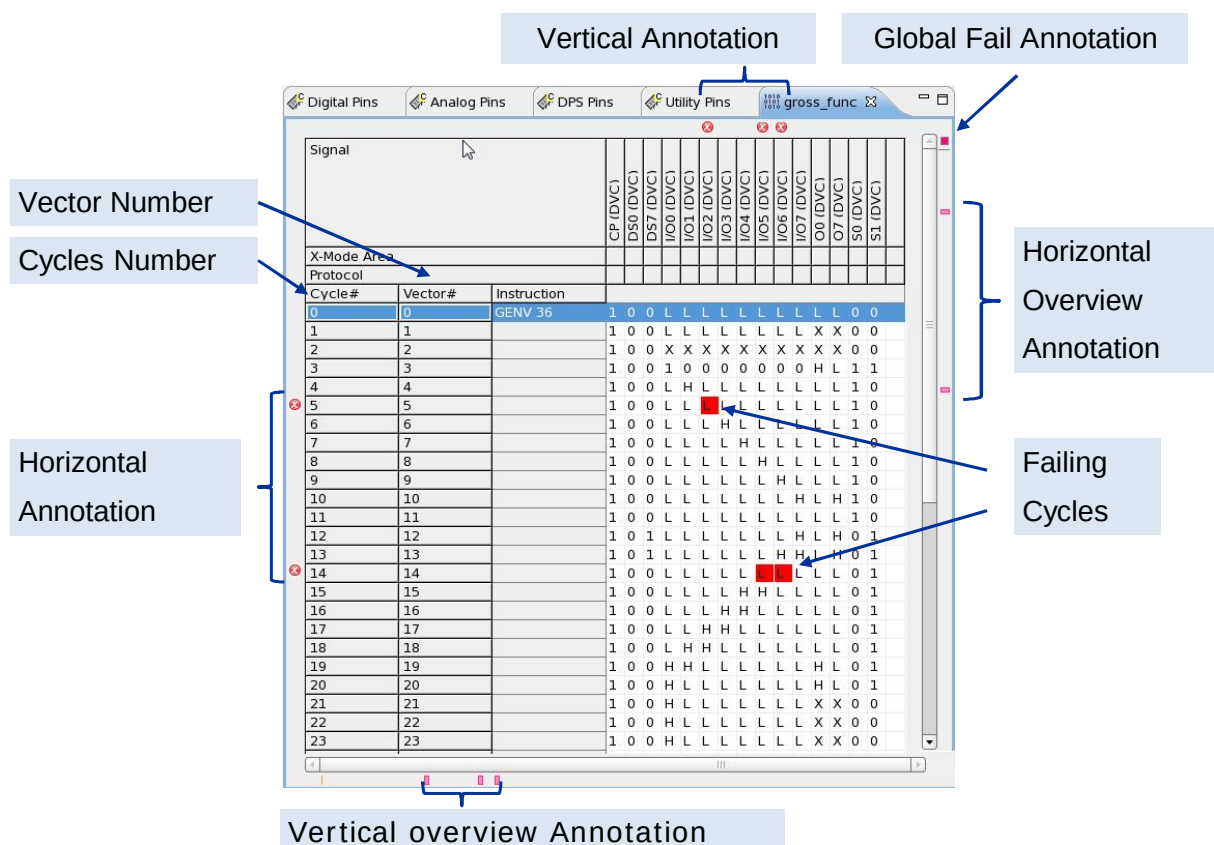


ADVANTEST

Pattern Editor as Debug Tool, ErrorMap and stimuliSource

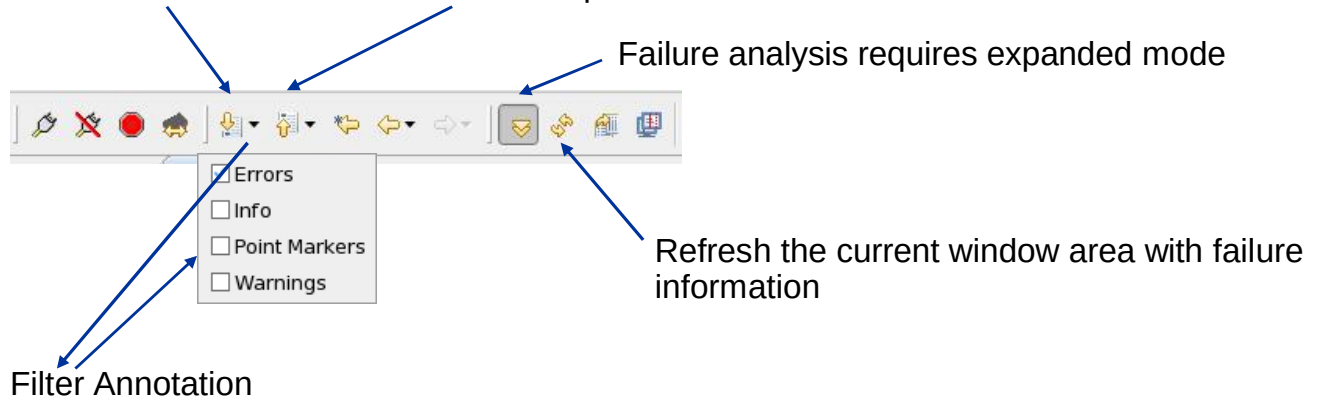
- Pattern Explorer must display the Pattern in the expanded mode
- Errors are highlighted in red
- Horizontal annotations indicate failing cycles
- Vertical annotations indicate failing signals (pins/pin groups)

Result Analysis/Displaying Failure Data



Navigate through Failure Data

Move forward to next and backward to previous annotation



ADVANTEST

Debugging a Functional Fail

1. Open Failing Pattern via failing Testsuite in Flow Sequence Editor or Flow Data Editor
2. Use Pattern Editor as Pattern Debug Tool
3. Open Timing Diagram and Error Map for detailed analysis
4. Align all three Debug Tools

ADVANTEST

1. Open Failing Pattern via Failing Testsuite

Double-clicking failing Testsuite will open Primaries

The screenshot displays the Advantest software interface. On the left, a 'Palette' shows a flowchart starting with 'Start', followed by 'Continuity' (green), 'leakage' (green), 'user_gross_func' (red), 'USer DC test' (white), and 'USer DPS Meas' (white). The 'user_gross_func' block is highlighted. On the right, the 'Properties' window is open, showing the 'Test Suite' 'user_gross_func'. The 'Pattern' field is circled and labeled 'func_t_first'. A text box next to it says: "F3" will open Pattern Editor with current pattern.

Property	Value
Test Suite	user_gross_func
Comment	
Primaries	...
Timing	1, 1, 1
Level	2, 2, 1
Analog	
Pattern	func_t_first
Context	DEFAULT
Test Type	M
Test Method	my_user_project.user_digital_class.my_functional
Parameters	dc
input_string	dc
Limits	...
Flags	...
Site Control	...
Results (Site in Focus)	Fail, , 0

ADVANTEST

2. Use Pattern Editor as Pattern Debug Tool

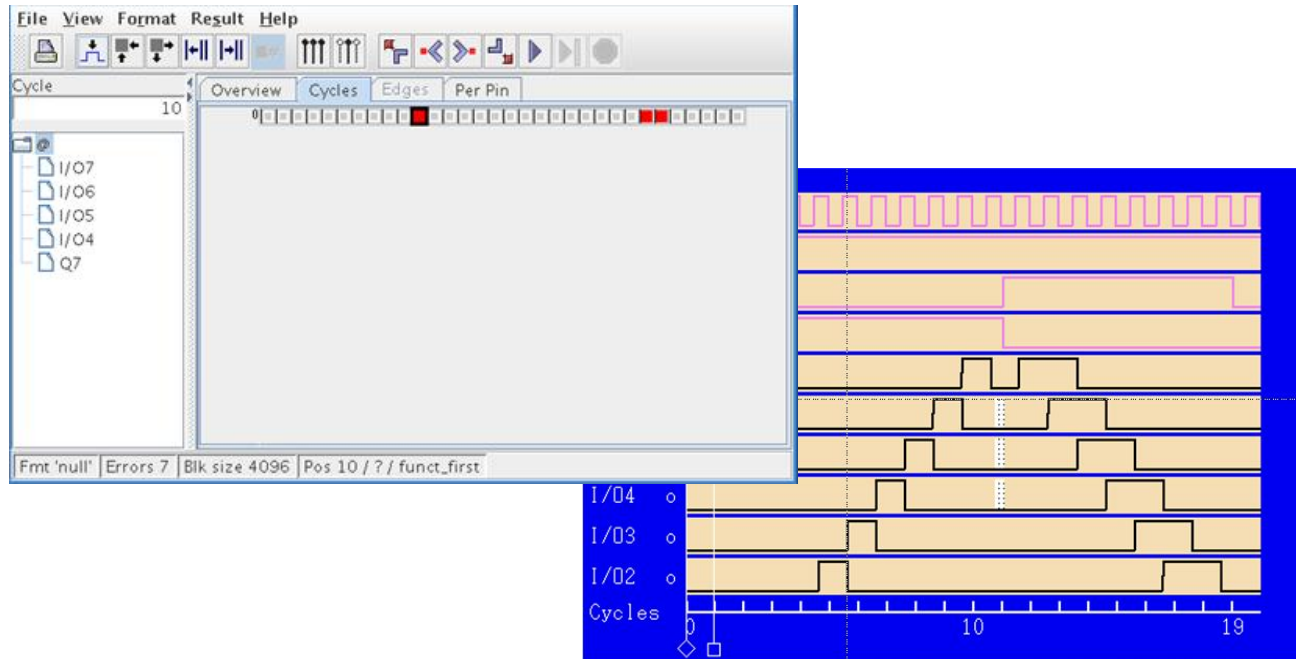
Switch to expanded mode for debugging pattern

The screenshot displays the Advantest software interface in expanded mode. The 'Signal' window is open, showing a table of signals and their values. The table has columns for GP (SC), DS0 (SC), DS7 (SC), IO0 (SC), IO1 (SC), IO2 (SC), IO3 (SC), IO4 (SC), IO5 (SC), IO6 (SC), IO7 (SC), O0 (SC), and O7 (SC). The 'X-Mode Area' is highlighted, showing a table of signals and their values. The 'Protocol' is set to 'Vector#'. The 'Cycle#' and 'Vector#' columns are visible. The table shows data for cycles 0 through 16. The 'Signal' window is also visible, showing a table of signals and their values.

Signal	GP (SC)	DS0 (SC)	DS7 (SC)	IO0 (SC)	IO1 (SC)	IO2 (SC)	IO3 (SC)	IO4 (SC)	IO5 (SC)	IO6 (SC)	IO7 (SC)	O0 (SC)	O7 (SC)
X-Mode Area	1	1	1	1	1	1	1	1	1	1	1	1	1
Protocol													
Cycle#	Vector#												
0	0	1	0	0	0	0	0	0	0	0	0	X	X
1	1	1	0	0	X	X	X	X	X	X	X	X	X
2	2	1	0	0	1	0	0	0	0	0	0	H	L
3	3	1	0	0	L	H	L	L	L	L	L	L	L
4	4	1	0	0	L	L	H	L	L	L	L	L	L
5	5	1	0	0	L	L	L	H	L	L	L	L	L
6	6	1	0	0	L	L	L	L	H	L	L	L	L
7	7	1	0	0	L	L	L	L	L	H	L	L	L
8	8	1	0	0	L	L	L	L	L	L	H	L	L
9	9	1	0	0	L	L	L	L	L	L	L	H	H
10	10	1	0	0	L	L	L	L	H	H	L	L	L
11	11	1	0	1	L	L	L	L	L	L	H	L	H
12	12	1	0	1	L	L	L	L	L	L	H	H	L
13	13	1	0	0	L	L	L	L	H	H	L	L	L
14	14	1	0	0	L	L	L	L	H	H	L	L	L
15	15	1	0	0	L	L	L	H	H	L	L	L	L
16	16	1	0	0	L	L	H	H	L	L	L	L	L

ADVANTEST

3. Open Timing Diagram and Error Map for Detailed Analysis



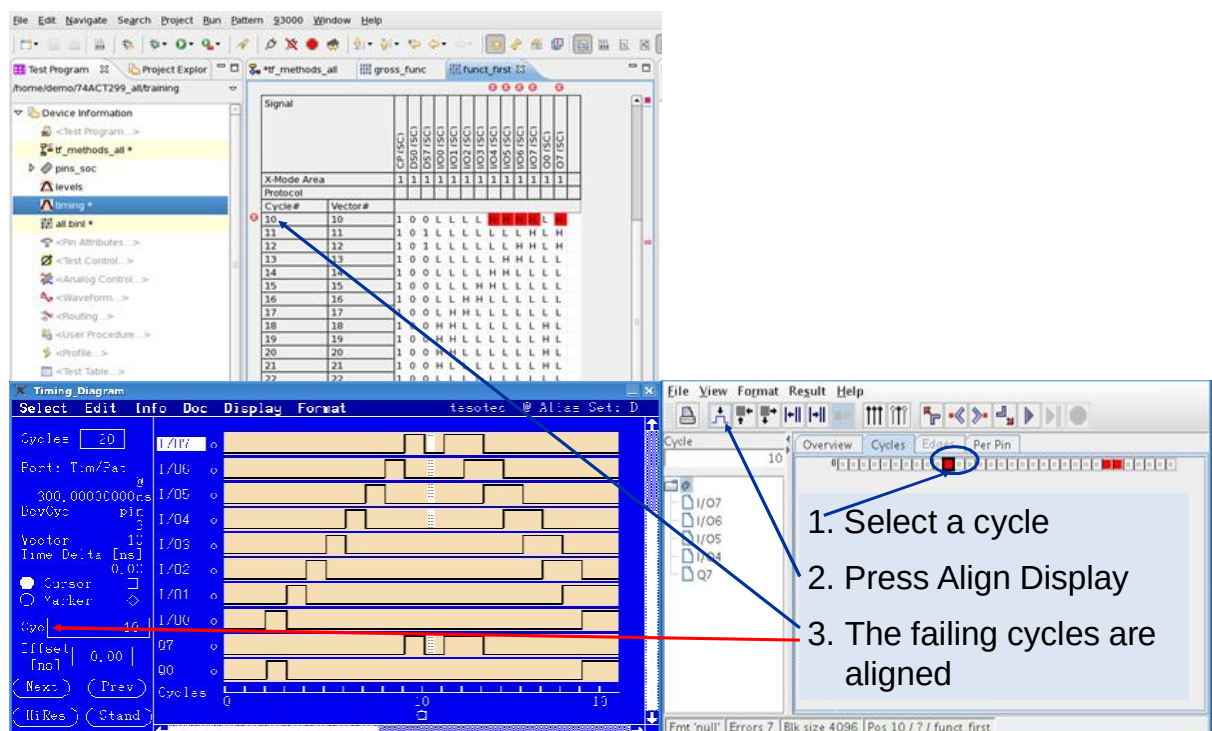
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Digital Debugging Unit 17 - 25

4. Align all three Debug Tools



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Digital Debugging Unit 17 - 26

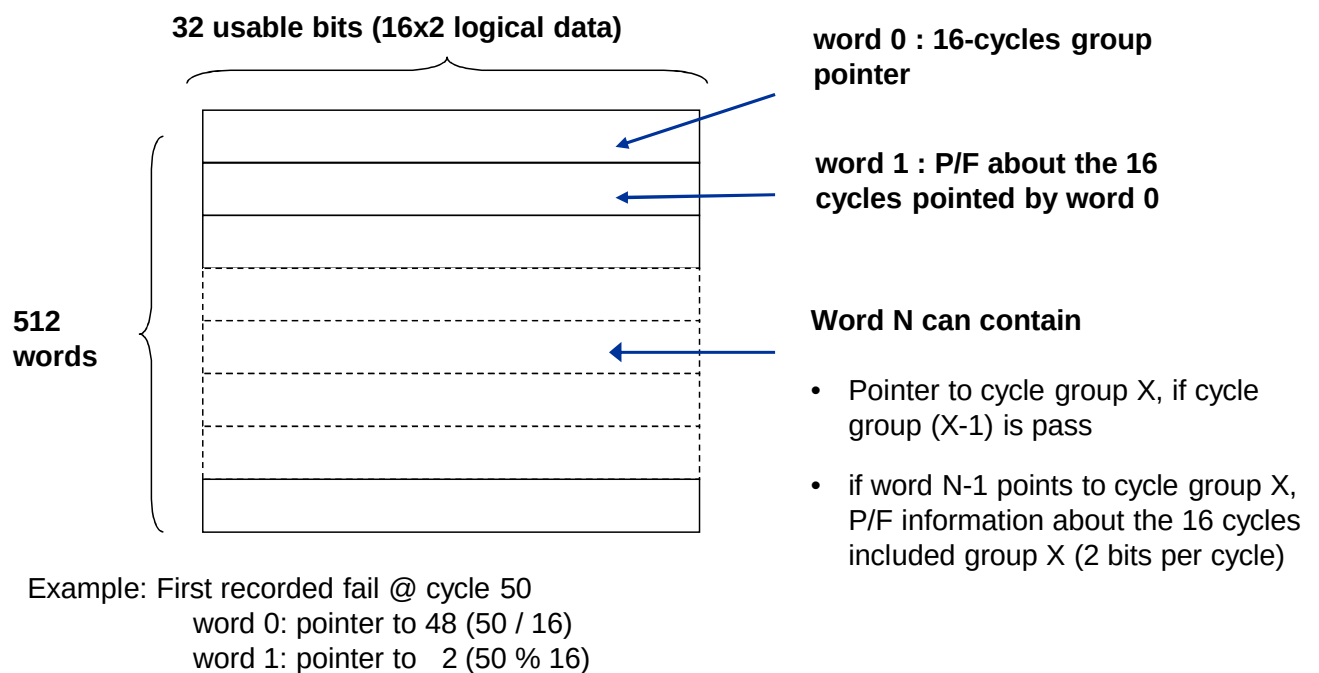
Reference

V93000 Technical Documentation Center:

Topic 108741

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=108741>

Layout of the Error Map: Failure Mode



Advantest V93000

Digital User Training

Device Characterization

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- AC and DC Characterization Parameters
- Tester Resources for Characterization
- Characterization Test Methods

ADVANTEST[®]

Learning Objectives

- Know the typical characteristic parameters of digital devices
- Know how to extract characterization parameters using SmarTest tools

Relevance

- Extracting device parameter especially in first silicon is a typical product engineer or test engineer task

Device Characterization Test Examples

DC Characterization

- VOH/VOL
- VIH/VIL
- Leakage
- IDDQ

AC Characterization

- Setup Time
- Hold Time
- Propagation delay

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 5

DC Parameter Review Datasheet

DC Electrical Characteristics for 74ACT299

Symbol	Parameter	V _{CC} (V)	T _A = 25°C		T _A = -40°C to +85°C		Units	Conditions
			Typ	Guaranteed Limits				
V _{IH}	Minimum HIGH Level Input Voltage	4.5 5.5	1.5 1.5	2.0 2.0	2.0 2.0		V	V _{OUT} = 0.1V or V _{CC} - 0.1V
V _{IL}	Maximum LOW Level Input Voltage	3.0 4.5	1.5 1.5	0.8 0.8	0.8 0.8		V	V _{OUT} = 0.1V or V _{CC} - 0.1V
V _{OH}	Minimum HIGH Level	4.5 5.5	4.49 5.49	4.4 5.4	4.4 5.4		V	I _{OUT} = -50 μA
		4.5 5.5	0.0001	3.86 4.86	3.76 4.76		V	V _{IN} = V _{IL} or V _{IH} I _{OH} = -24 mA I _{OH} = -24 mA (Note 5)
		4.5 5.5	0.001 0.001	0.1 0.1	0.1 0.1		V	I _{OUT} = 50 μA
		4.5 5.5		0.36 0.36	0.44 0.44		V	V _{IN} = V _{IL} or V _{IH} I _{OL} = 24 mA I _{OL} = 24 mA (Note 5)
V _{OL}	Maximum LOW Level Output Voltage	4.5 5.5	0.001 0.001	0.1 0.1	0.1 0.1		V	I _{OUT} = 50 μA
I _{IN}	Maximum Input Leakage Current	5.5		± 0.1	± 1.0		μA	V _I = V _{CC} , GND
I _{CC1}	Maximum I _{CC} /Input	5.5	0.6		1.5		mA	V _I = V _{CC} - 2.1V
I _{OLD}	Minimum Dynamic	5.5			75		mA	V _{OLD} = 1.65V Max
I _{OHD}	Output Current (Note 6)	5.5			-75		mA	V _{OHD} = 3.85V Min
I _{CC}	Maximum Quiescent Supply Current	5.5		4.0	40.0		μA	V _{IN} = V _{CC} or GND
I _{OZT}	Maximum I/O Leakage Current	5.5		± 0.3	± 3.0		μA	V _I (OE) = V _{IL} , V _{IH} V _I = V _{CC} , GND V _O = V _{CC} , GND

Note 5: All outputs loaded; thresholds on input associated with output under test.

Note 6: Maximum test duration 2.0 ms, one output loaded at a time.

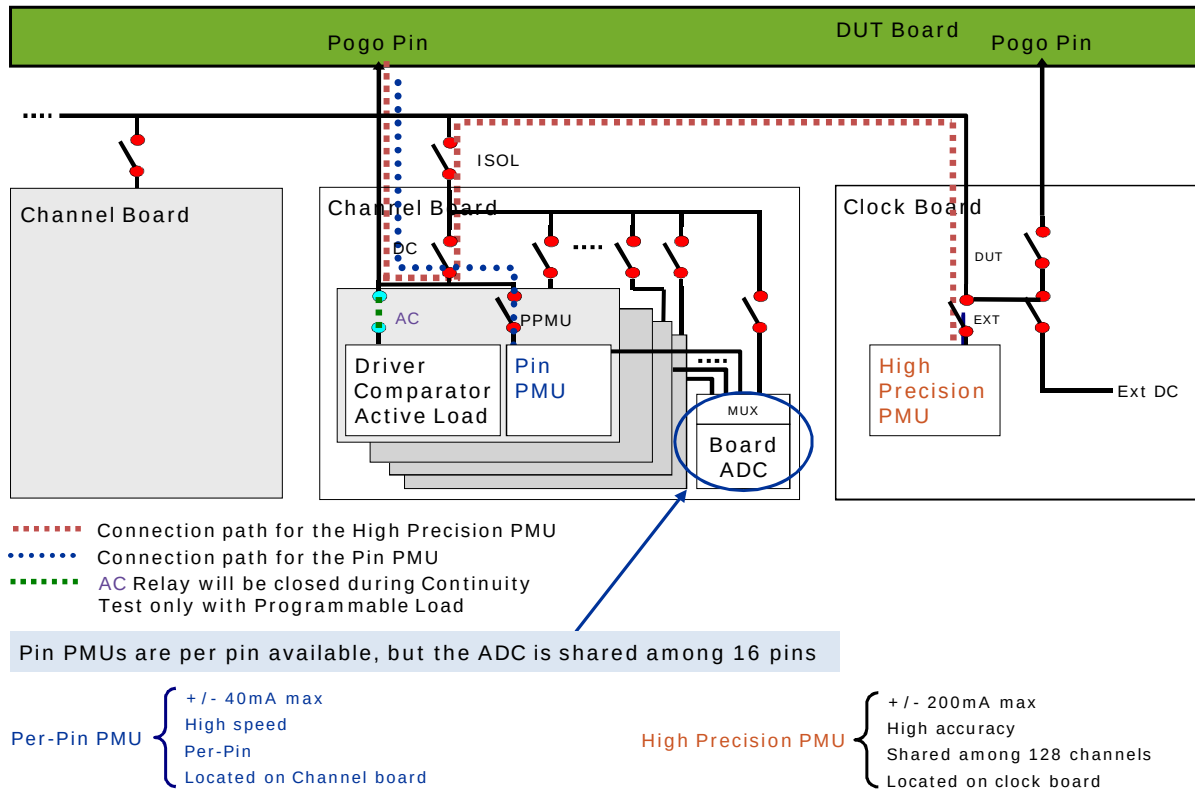
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 6

Tester Resources



ADVANTEST

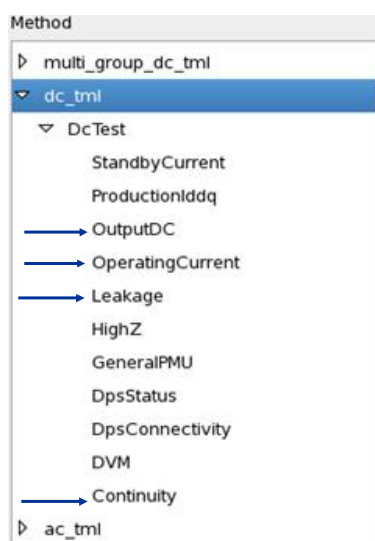
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 7

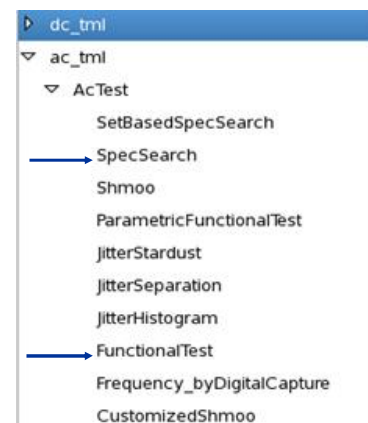
Built-in Characterization Test Method Libraries

DC Test Method Library



- Leakage
- VOH/VOL
- IDD: operating, quiescent, standby

AC Test Method Library



- Frequency
- Functional
- SpecSearch

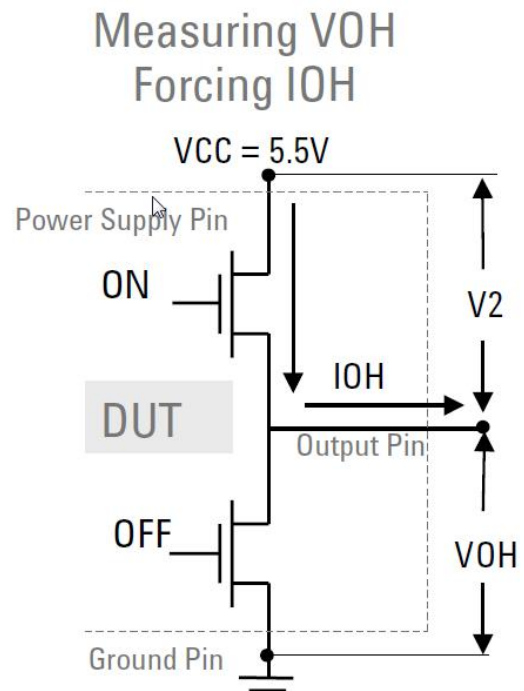
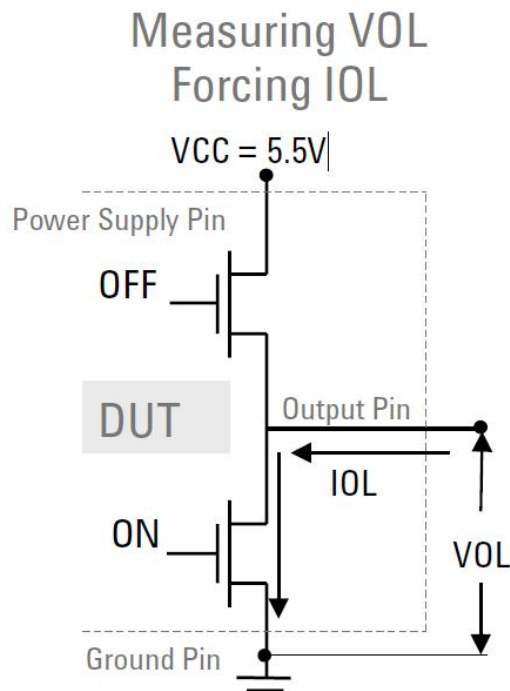
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 8

Output DC Voltage Measurement



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 9

Output DC Vector Range

Label "gross_func" (avc file)

```

FORMAT CP _MR mode ser_in ser_out io_pins ;
R1 std 1 0 00 00 LL LLLLLLLL reset ;
R1 std 1 1 00 00 XX LLLLLLLL hold ;
R1 std 1 1 00 00 XX XXXXXXXX hold (dummy) ;
R1 std 1 1 11 00 HL 10000000 parallel load ;
R1 std 1 1 01 00 LL LHLLLLLL shift right ;
R1 std 1 1 01 00 LL LLHLLLLL shift right ;
R1 std 1 1 01 00 LL LLLHLLLL shift right ;
R1 std 1 1 01 00 LL LLLLHLLL shift right ;
R1 std 1 1 01 00 LL LLLLLHLL shift right ;
R1 std 1 1 01 00 LL LLLLLLHL shift right ;
R1 std 1 1 01 00 LH LLLLLLLH shift right ;
R1 std 1 1 01 00 LL LLLLLLLL shift right ;
R1 std 1 1 10 01 LH LLLLLLLH shift in left ;
R1 std 1 1 10 01 LH LLLLLLLH shift in left ;
R1 std 1 1 10 00 LL LLLLLHLL shift left ;
R1 std 1 1 10 00 LL LLLHLLLL shift left ;
R1 std 1 1 10 00 LL LLHLLLLL shift left ;
R1 std 1 1 10 00 LL LHLLLLLL shift left ;
R1 std 1 1 10 00 HL HLLLLLLL shift left ;
R1 std 1 1 10 00 HL HLLLLLLL shift left ;
R10 std 1 1 00 00 XX HLLLLLLL hold ;
    
```

Output DC vector range 10 - 20:
DC measurements at 11 different vectors

...

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 10

VOH/VOL Test Output DC Test

Property	Value
Test Suite	Std_Output_Levels
Comment	
Primaries	...
Timing	1, 1, 1
Level	2, 2, 1
Analog	
Pattern	gross_func
Context	DEFAULT
Test Type	M
Test Method	dc_tml.DcTest.OutputDC
Parameters	all_out, PPMU, BOTH, 24[mA], -24[mA], 2[ms], 2[m
pinlist	all_out
mode	PPMU
measuredLevel	BOTH
forceCurrentLow	24[mA]
forceCurrentHigh	-24[mA]
settlingTimeLow	2[ms]
settlingTimeHigh	2[ms]
spmClampVolta	0[V]
spmClampVolta	0[V]
vectorRange	30
testName	(OutputDC_LowLevel[V],OutputDC_HighLevel[V])
output	ReportUI
Limits	...
Flags	...
Site Control	...
Results (Site in Focus)	...

ui_report.ORG.PROD		
File	Options	Help Mode Datalog
Tester State	CONNECTED	
Tester Operation	NOT monitored (DISABLED)	
Execute 'OutputDC':		
Vout (I/00)	L = 319,uV	P
Vout (I/00)	H = 5,50 V	P
Vout (I/01)	L = 201,uV	P
Vout (I/01)	H = 5,50 V	P
Vout (I/02)	L = 78,0uV	P
Vout (I/02)	H = 5,50 V	P
Vout (I/03)	L = 552,uV	P
Vout (I/03)	H = 5,50 V	P
Vout (I/04)	L = 190,uV	P
Vout (I/04)	H = 5,50 V	P
Vout (I/05)	L = 307,uV	P
Vout (I/05)	H = 5,50 V	P
Vout (I/06)	L = 361,uV	P
Vout (I/06)	H = 5,50 V	P
Vout (I/07)	L = -106,uV	P
Vout (I/07)	H = 5,50 V	P
Vout (Q7)	L = 727,uV	P
Vout (Q7)	H = 5,50 V	P
Vout (Q0)	L = 66,0uV	P
Vout (Q0)	H = 5,50 V	P
Total result: PASSED		
Report Formatter	.default.PROD	

ADVANTEST

April 2016

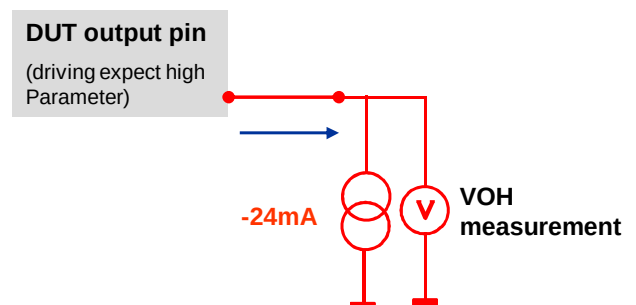
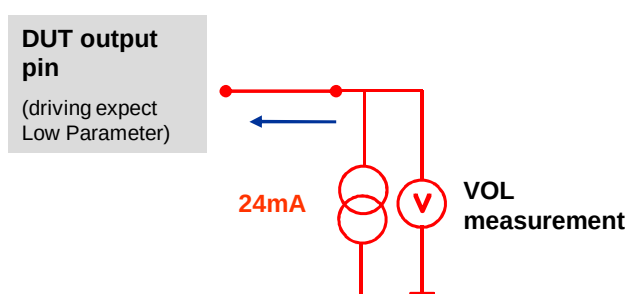
All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 11

VOH/VOL Test PMU Option

1. Execute test vectors; Scan for "H" or "L" state in a pin-by-pin basis in specified 'vec range' field
2. Connect PMU, Force current @spec value and measure voltage
3. Repeat for remaining pins
4. Compare to pass values

Force Current, Measure Voltage



ADVANTEST

April 2016

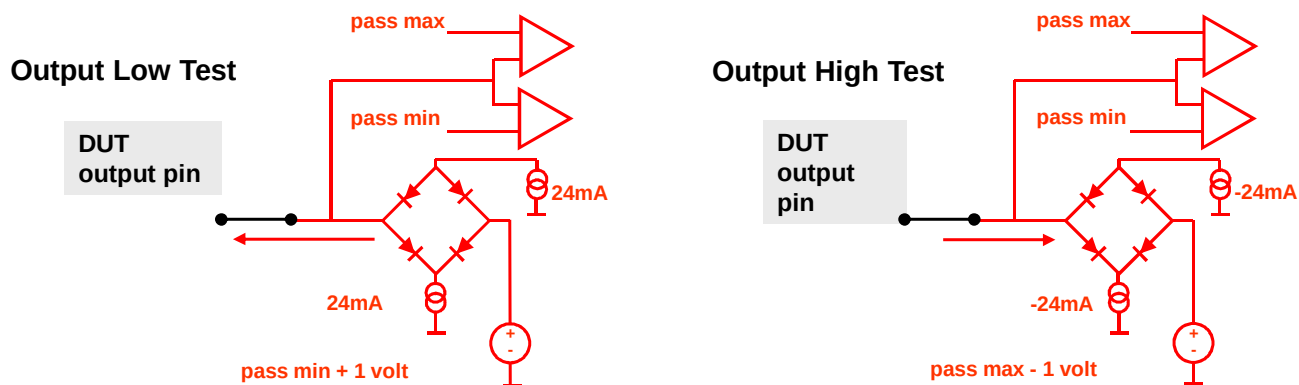
All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 12

VOH/VOL Test

Programmable Load Option

1. Set Comparator VOL to pass min, setup active load to force low value
2. Execute Functional test
3. Set Comparator VOH to pass max, setup active load to force low value
4. Execute Functional test
5. Fails if either execution fails, otherwise passes
6. Execute binary search if min/max values are required

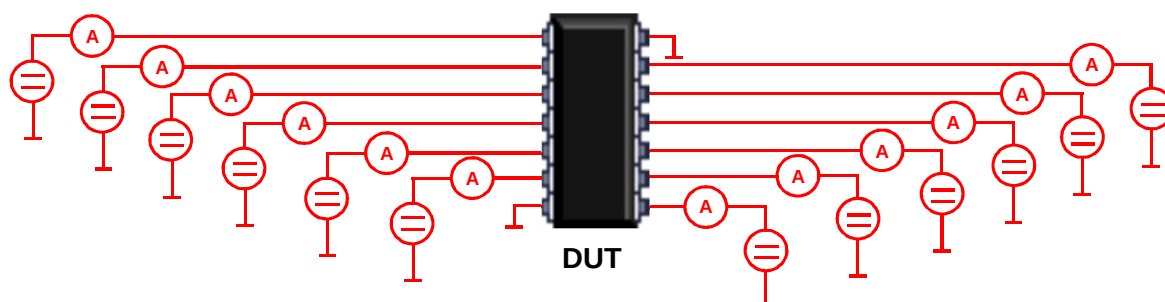


ADVANTEST

Leakage Test

1. Connect Device power supplies
2. Connect PPMUs in parallel (*)
3. Wait for settling time, force specified voltage, measure current
4. Compare values to limits

Force Voltage, Measure Current

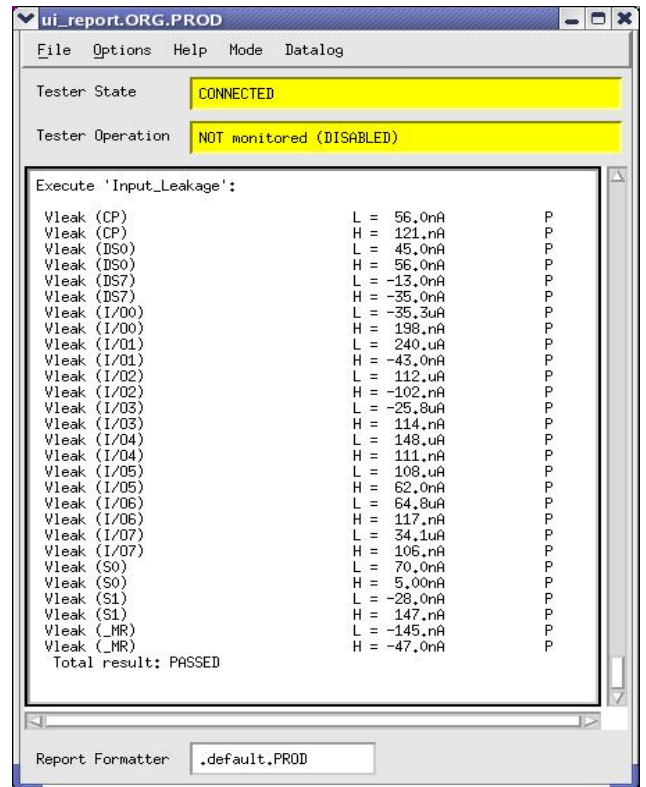
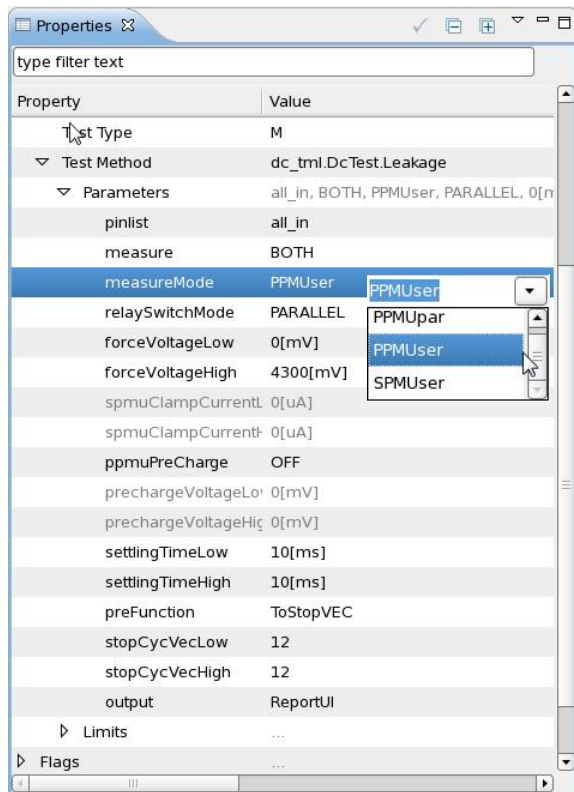


(*) alternatively to this Parallel Measure option it is possible to:

- Serial Option: connect one pin at a time to PMU and measure, loop across pins
- Ganged Option: connect PMU to all pins (shorted among them), measure the sum

ADVANTEST

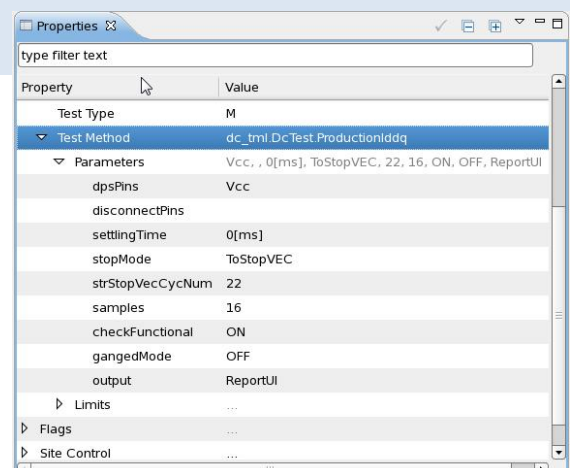
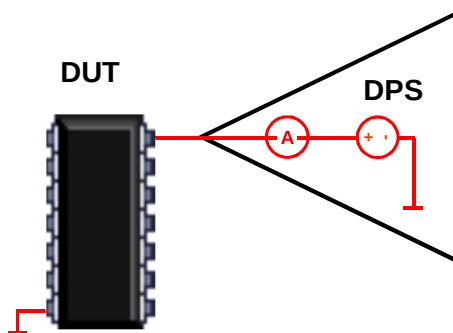
Input Leakage Test



ADVANTEST

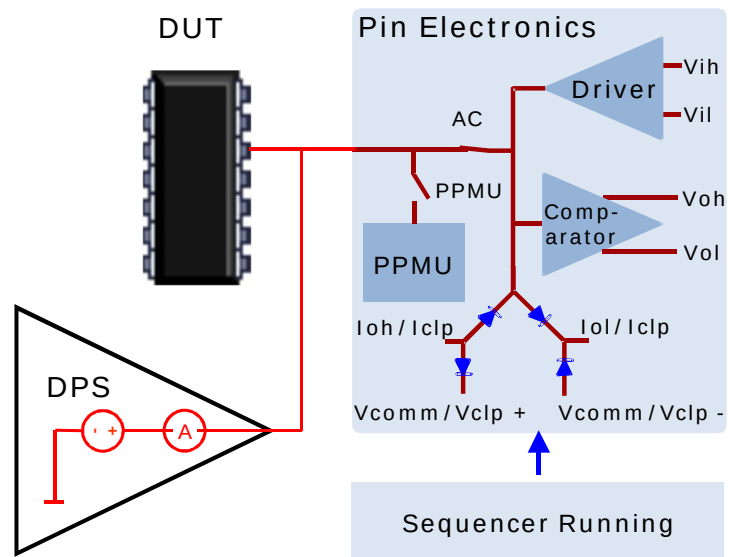
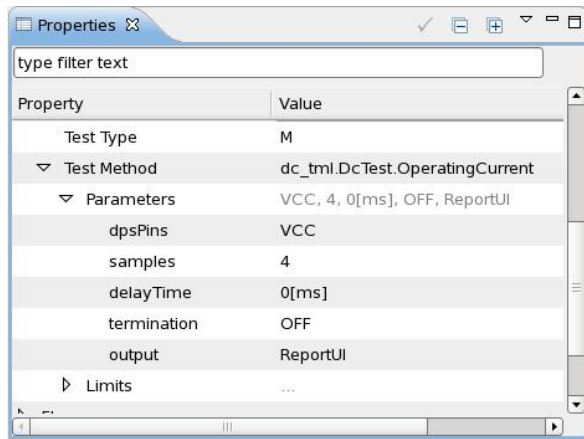
IDDQ Production Test

1. Prepare a specific IDDQ pattern (typically the pattern comes out from an ATPG tool)
2. Execute Functional test
3. Stop at vector locations (or cycles) specified
4. Wait for settling time, measure current
5. Repeat until all stop locations are tested
6. Compare values to limits



ADVANTEST

Operating Current Test

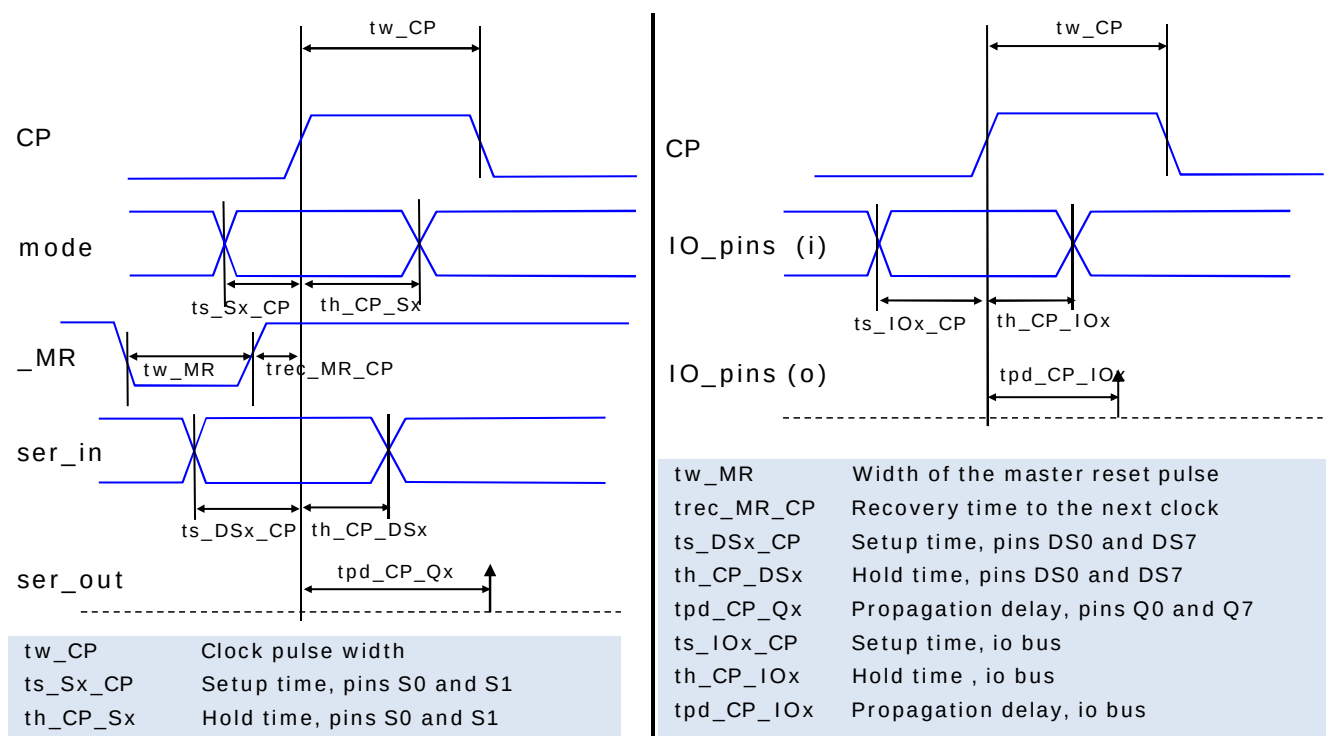


- It measures the current supplied to the DUT during a functional test
- It uses the internal DPS current measurement unit(s)
- Accuracy and resolution depend on that of the installed DPS

ADVANTEST

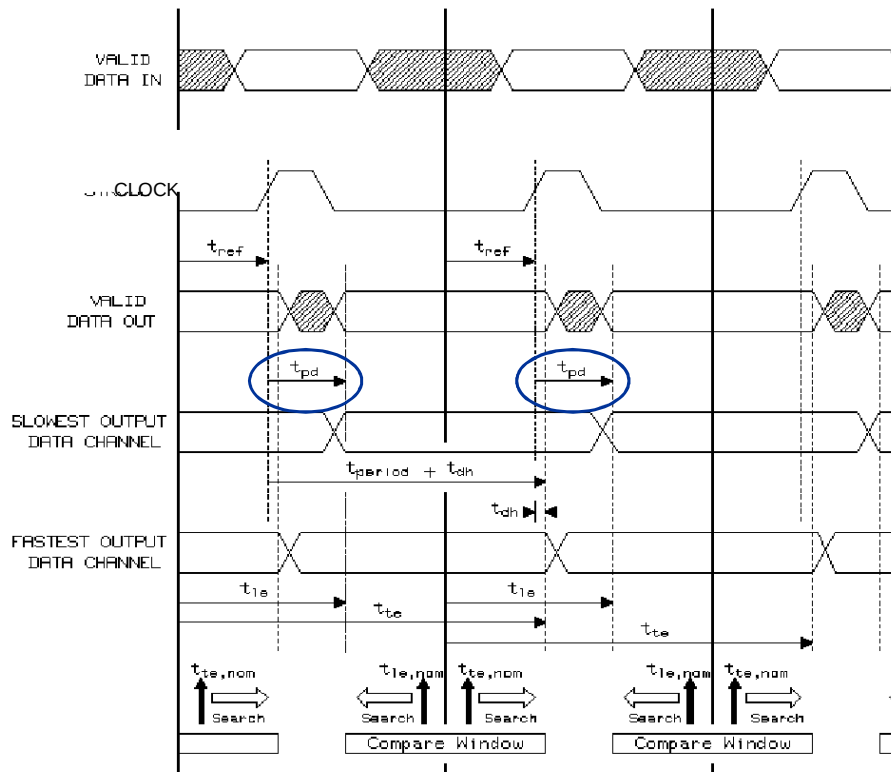
AC Parameter Review Datasheet

AC Electrical Characteristics for 74ACT299



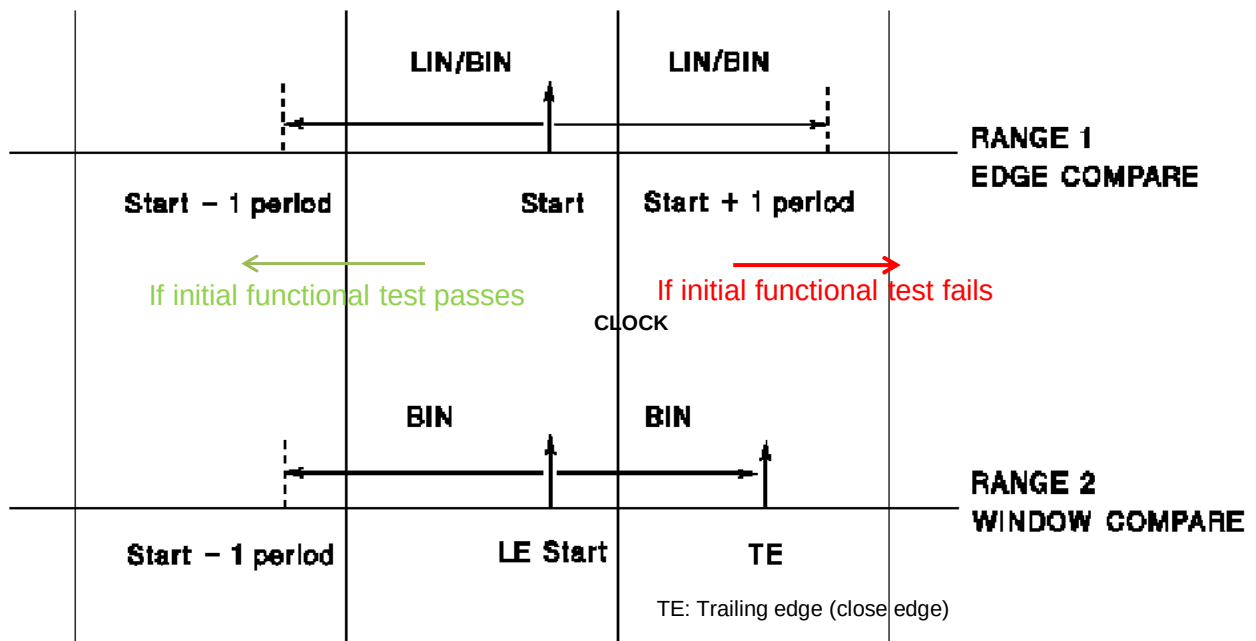
ADVANTEST

Propagation Delay Test (1)



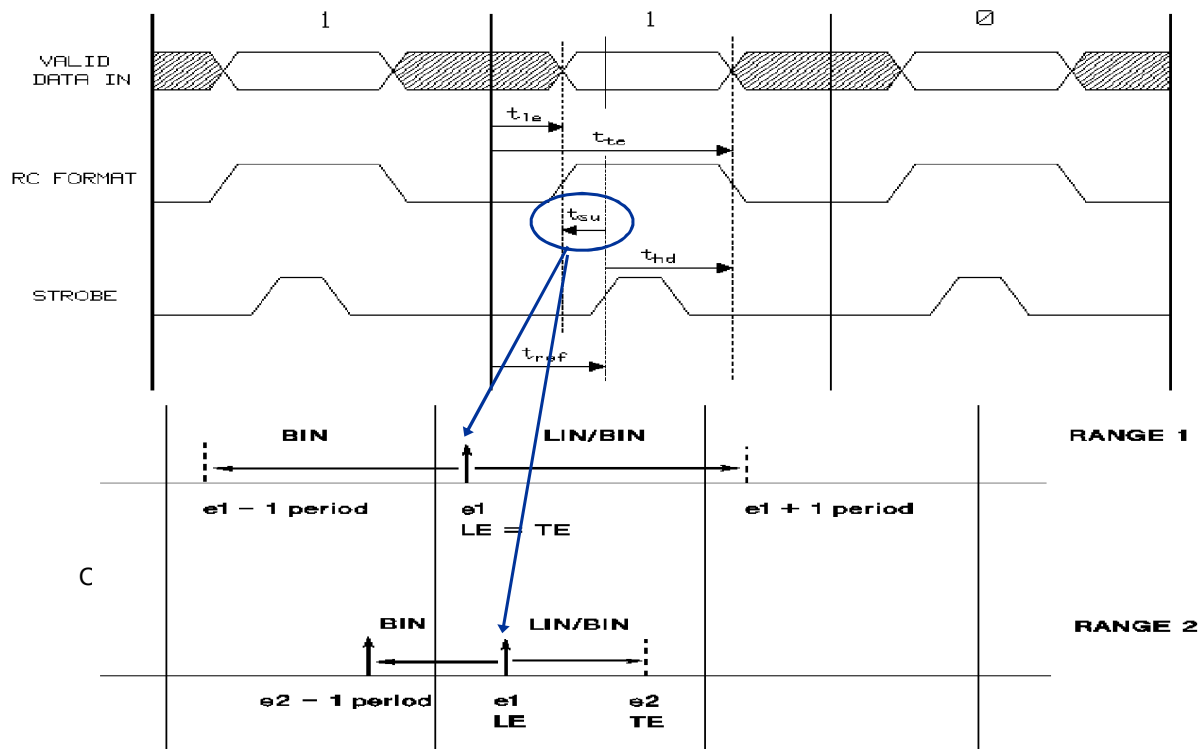
ADVANTEST

Propagation Delay Test (2)



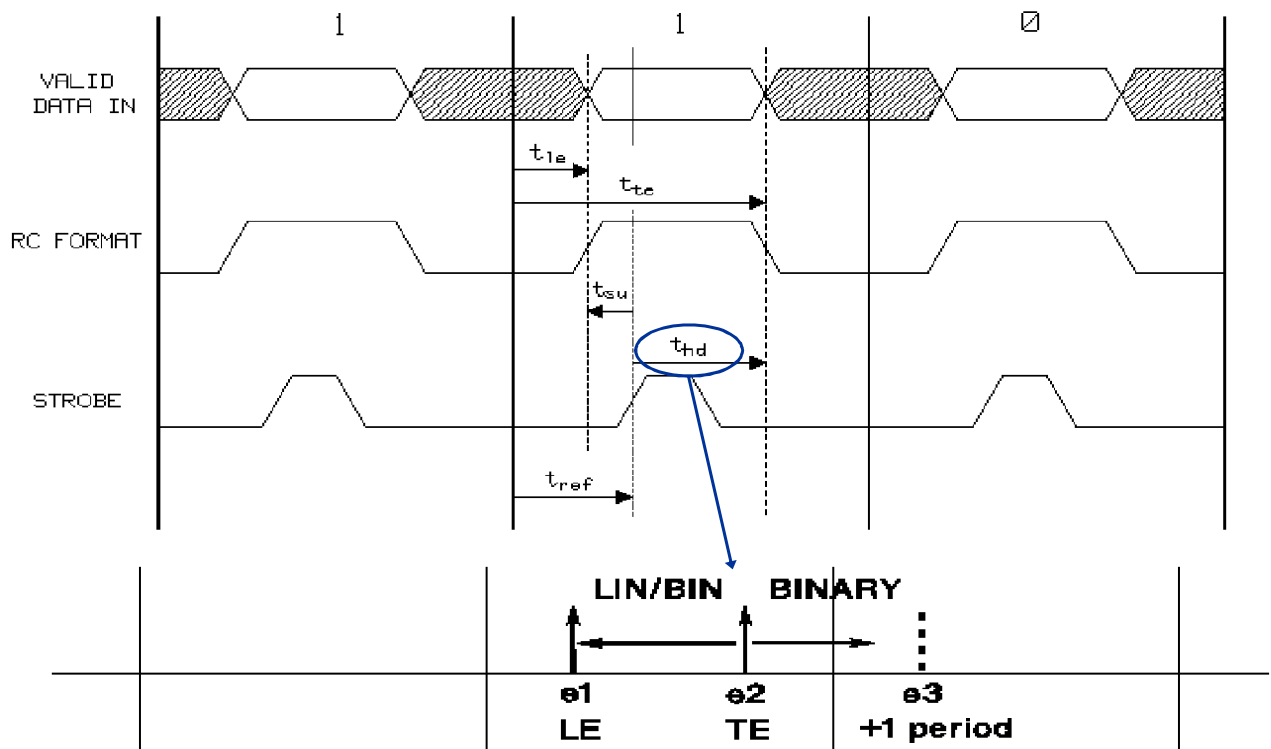
ADVANTEST

Setup Time Test



ADVANTEST

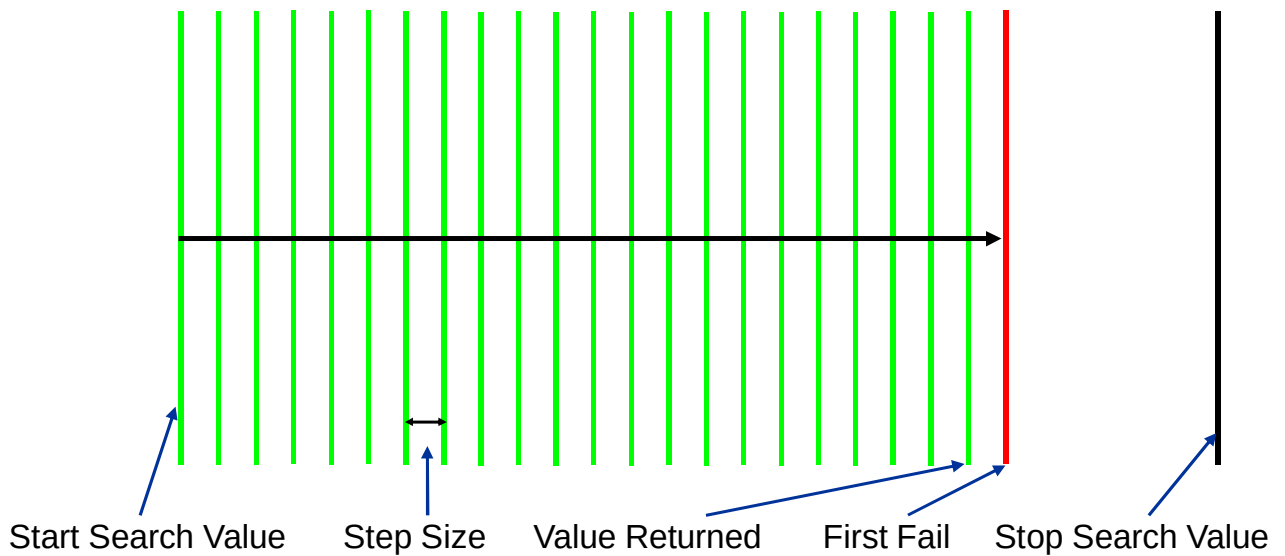
Hold Time Test



ADVANTEST

Quick Review of Searches

Linear Search



ADVANTEST[®]

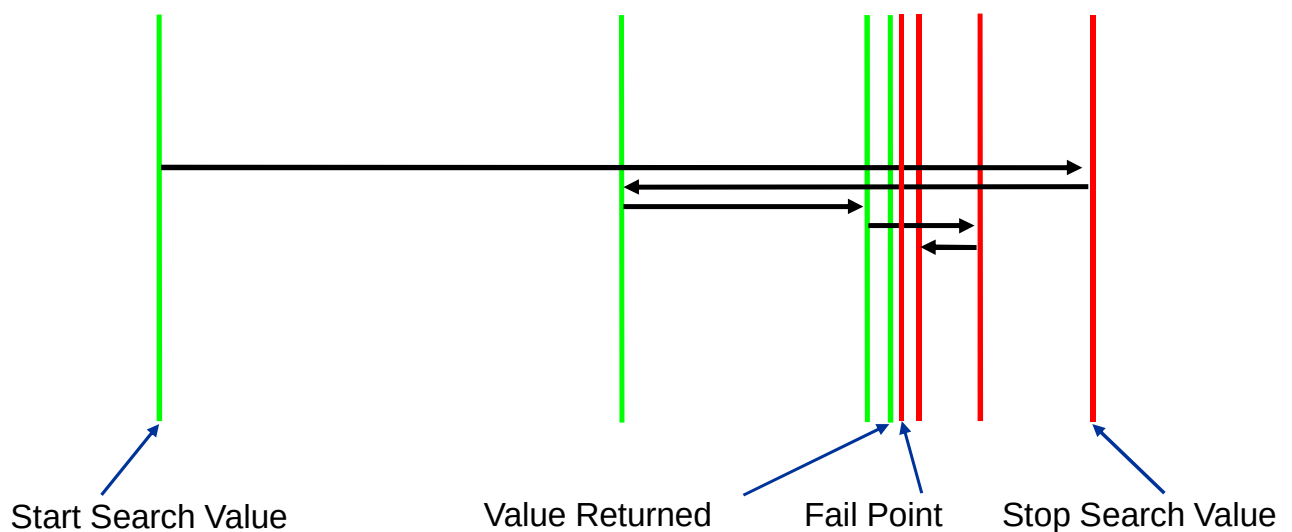
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 23

Quick Review of Searches

Binary Search



ADVANTEST[®]

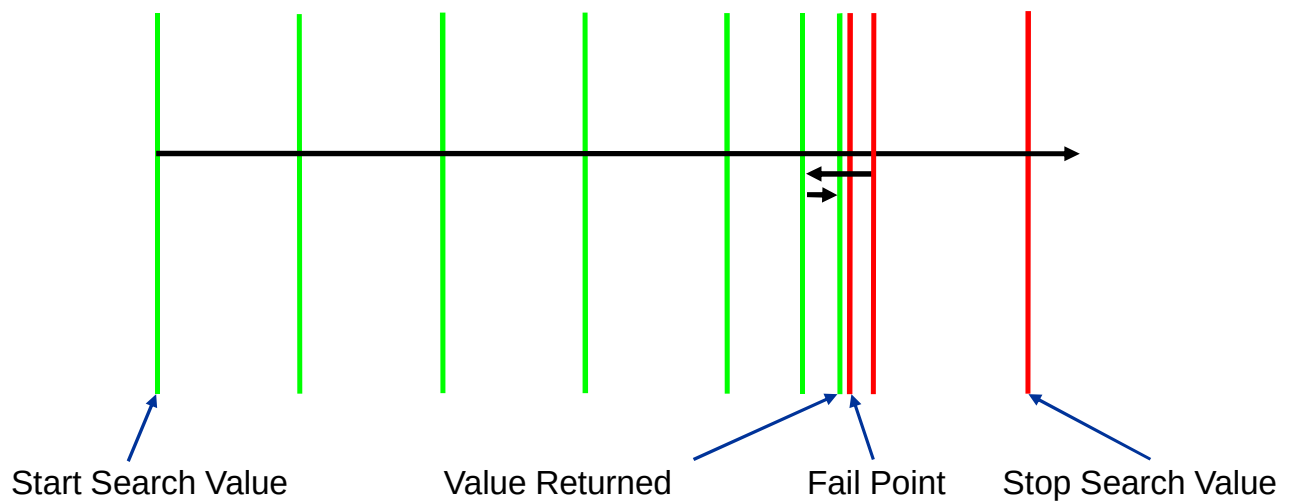
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 24

Quick Review of Searches

Linear/Binary Search



ADVANTEST

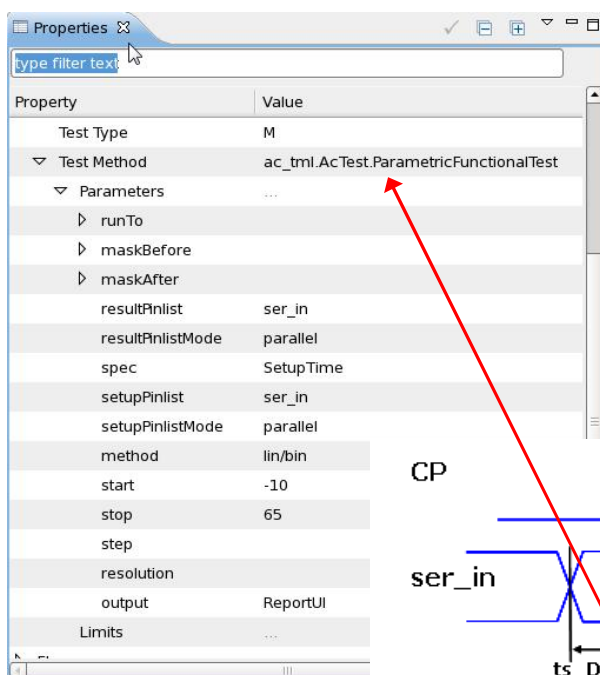
April 2016

All Rights Reserved - ADVANTEST CORPORATION

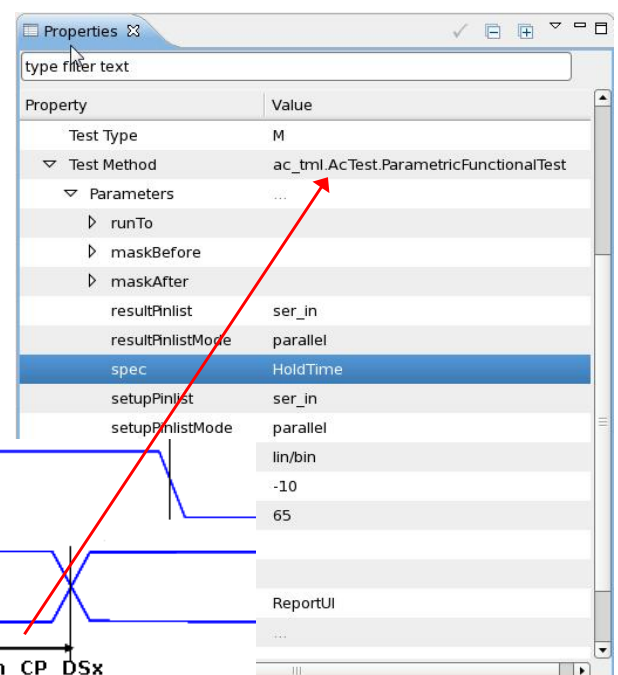
Device Characterization Unit 18 - 25

Setup/Hold Time Test

Setup Time Test



Hold Time Test



CP

ser_in

$t_{s_DSx_CP}$ $t_{h_CP_DSx}$

ADVANTEST

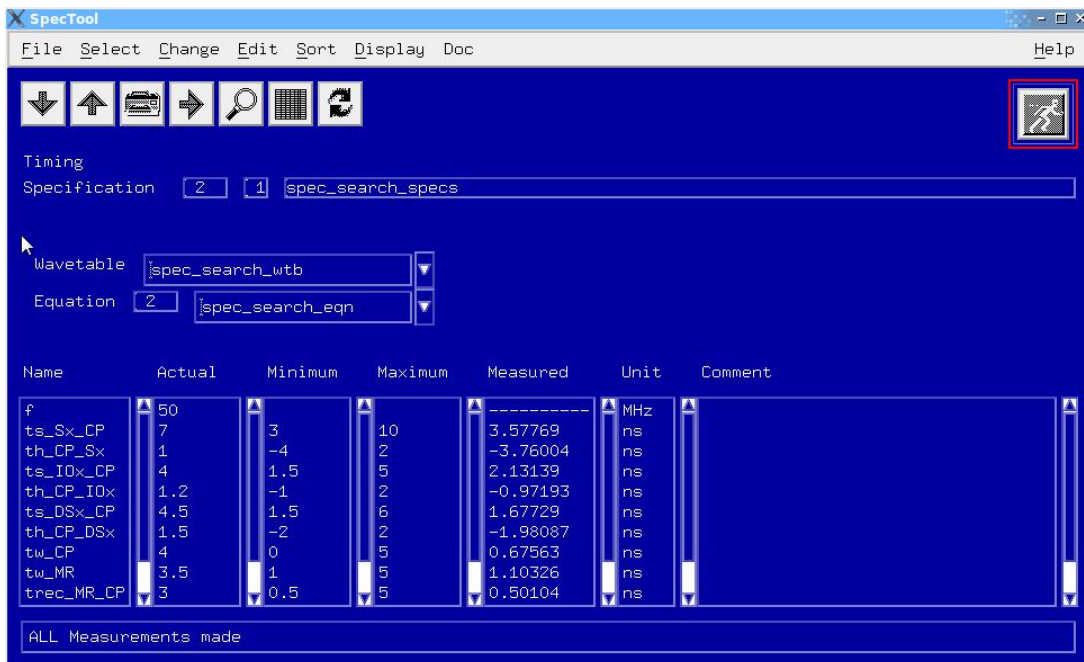
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Device Characterization Unit 18 - 26

SpecTool Used for Characterization

- Define Min.- Max.- range of spec-variable
- Select spec-variable Start search of real value



ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):
<https://www.advantest.com/V93000/help/index.jsp>

ADVANTEST

Advantest V93000

Digital User Training

Shmoo Tool

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Shmoo introduction
- Types of Shmoo
- Using the Shmoo Tool

ADVANTEST[®]

Learning Objectives

- Know the type of Shmoo Tool
- Know how to prepare a Testsuite prior to start the Shmoo Tool
- Know how to read a two-dimensional Shmoo

Relevance

- Shmoo-ing relevant parameters against each other allows to see the devices' limits and margins

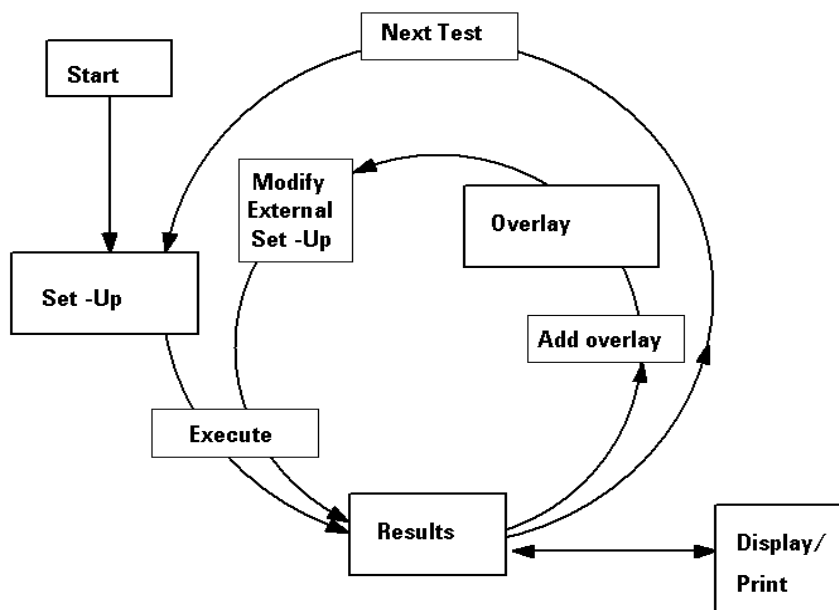
Shmoo Tool Introduction

Interactive Engineering Tool for supporting Debugging and Characterization tasks:

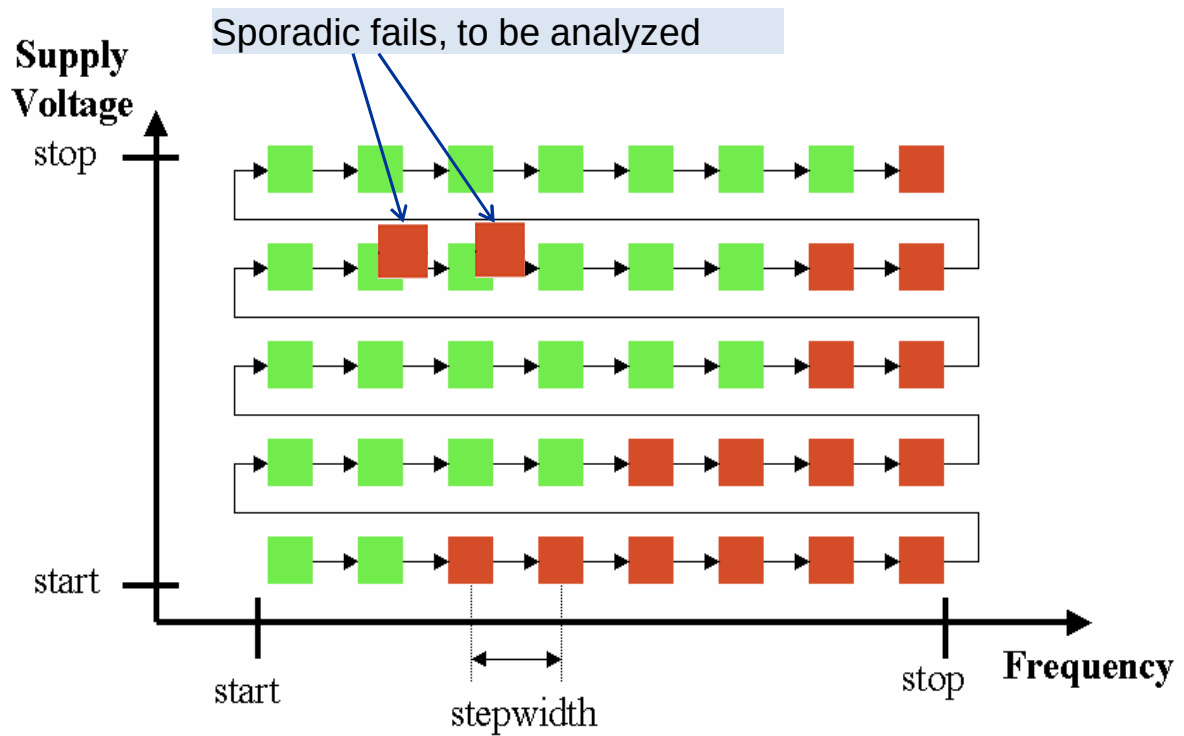
- Find holes
- Shmoo Vdd versus Frequency
- Set final specifications
- Detailed Device Analysis

Shmoo Operation: Diagram Flow

- Run the Testsuite to characterize first
- Then do the following:

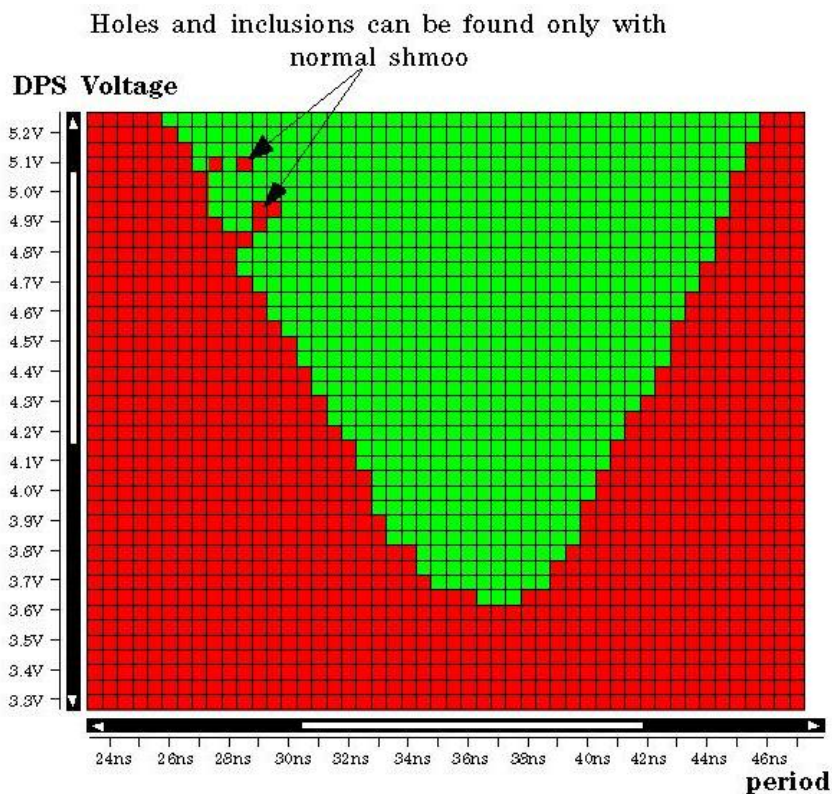


Shmoo Principals



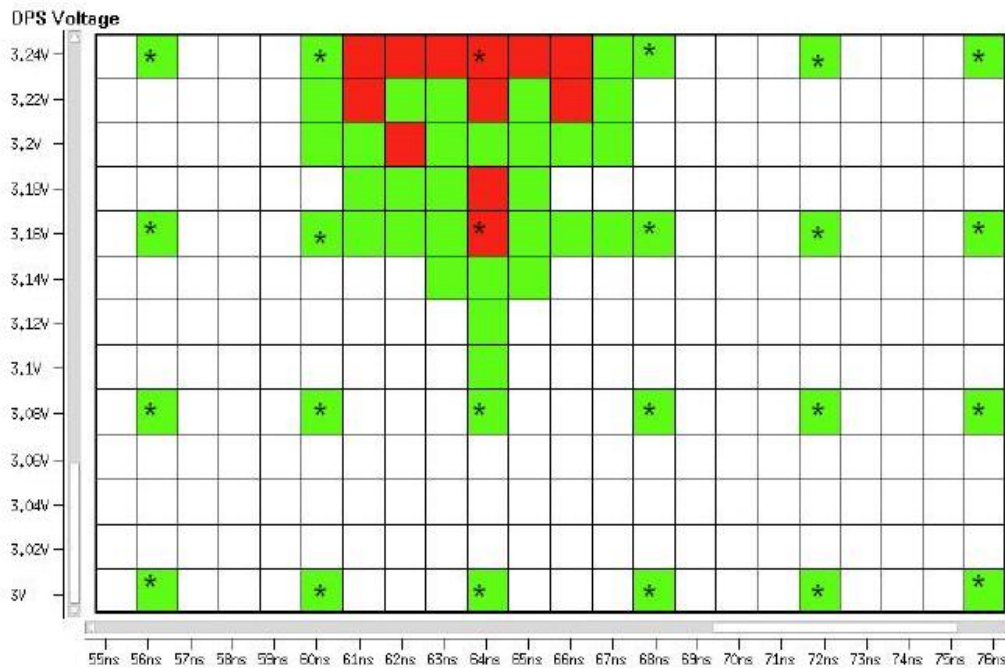
ADVANTEST

Normal Shmoo



ADVANTEST

Fast Shmoo



This type of plot is perfectly satisfactory when the pass/fail transition represents a smooth envelope, but it can miss anomalies in this envelope, such as holes and inclusions

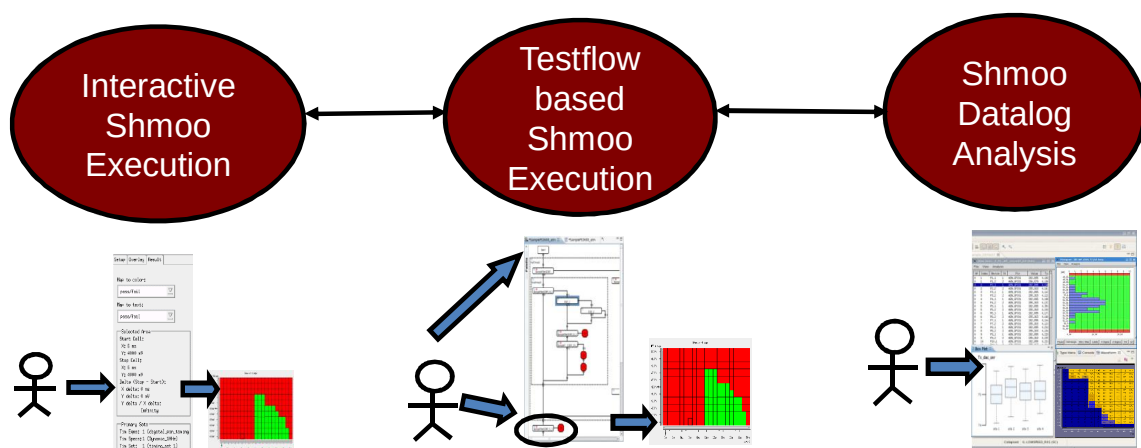
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Shmoo Tool Unit 19 - 9

Shmoo applications



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Shmoo Tool Unit 19 - 10

Typical Shmoo workflow

Start Shmoo tool

- Configure the tool for the particular task

Setup Primaries and Parameters via Properties page

- Initiate a re-run of a test execution
- Assign the resources to the required axis
- Select test to be performed and measurements to be displayed

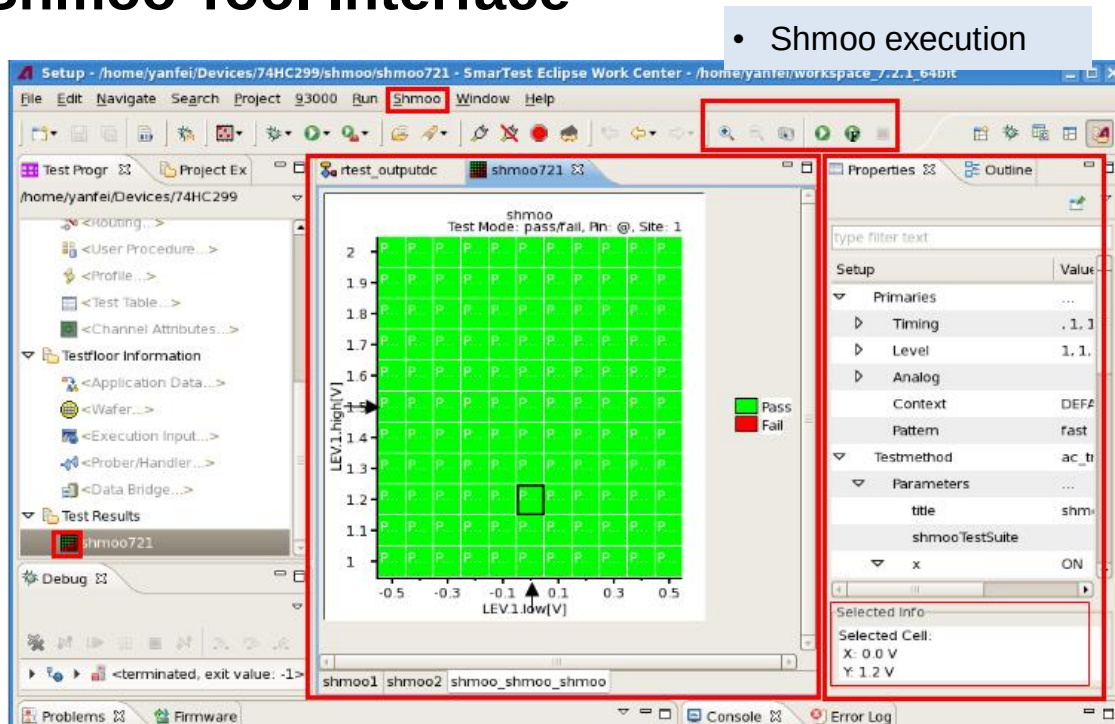
Execute Shmoo

- Output display resulting from the Shmoo Tool execution

Verify Results

- Select result(s) to display from options defined on the setup page
- Zoom in and display detailed results

Shmoo Tool Interface



• Access via Test Result
in TPE

• Shmoo view

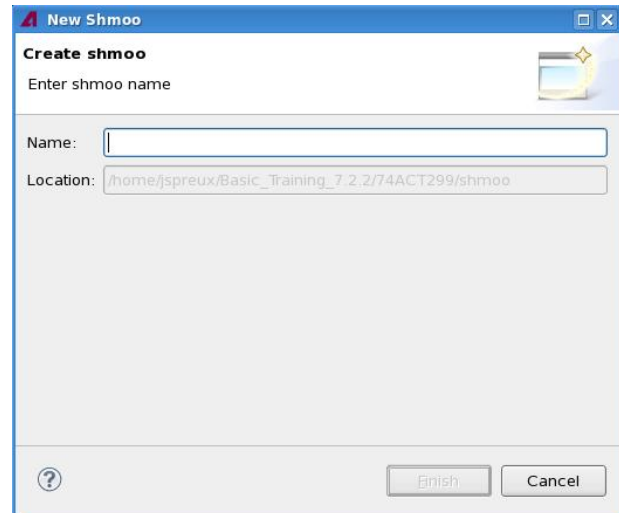
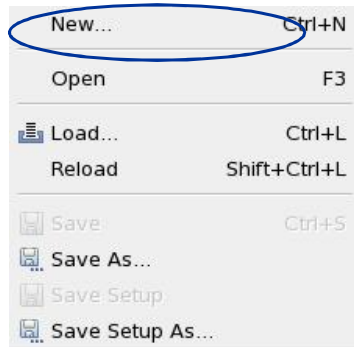
• Properties page

Interactive usage

- TPE –right click Test results-Shmoo



- Enter name



- Access via Test Result in TPE

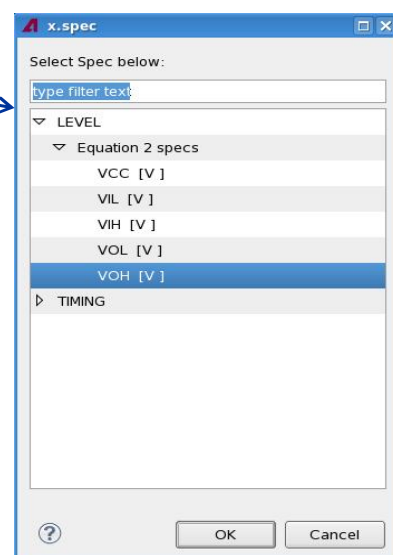
- Shmoo view

ADVANTEST

Interactive usage

- Enter primaries and shmoo parameters for x and y axis

Setup	Value
Testmethod	ac_tml.AcTest.Shmoo
Parameters	...
title	shmoo
shmooTestSuite	TM_gross_func
x	ON
parameterType	SpecVariable
spec	LEV.2.VOH[V]
start	2
stop	3.3
step	#100
scale	Linear
setupPins	@
y	ON
parameterType	SpecVariable
spec	LEV.2.VCC[V]
start	3
stop	4.5
step	#100
scale	Linear
setupPins	@
isYAxisFirst	OFF
testName	Shmoo
testMode	pass/fail

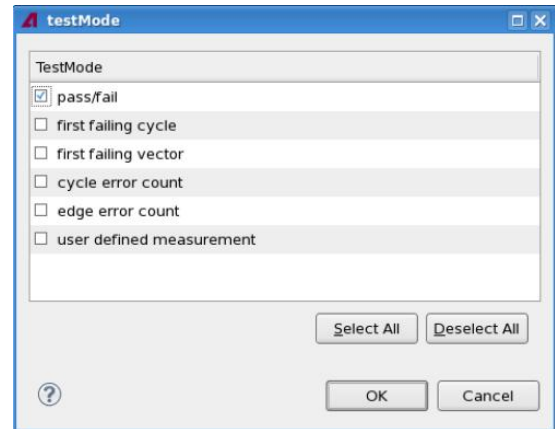


ADVANTEST

Interactive usage

Select Test mode

Setup	Value
Testmethod	ac_tml.AcTest.Shmoo
Parameters	...
title	shmoo
shmooTestSuite	TM_gross_funct
x	ON
parameterType	SpecVariable
spec	LEV2.VOH[V]
start	2
stop	3.3
step	#100
scale	Linear
setupPins	@
y	ON
parameterType	SpecVariable
spec	LEV2.VCC[V]
start	3
stop	4.5
step	#100
scale	Linear
setupPins	@
isYAxisFirst	OFF
testName	Shmoo
testMode	pass/fail



ADVANTEST

Interactive Usage – Test modes

Pass/Fail: (default) Displays either PASS or FAIL for each (x, y) pair in the shmoo results plots. It can be used both for functional test and any parametric test

First failing cycle (used only for functional tests): Displays the first failing cycle number for each (x, y) pair in the shmoo results plots.

First failing vector (used only for functional tests): Displays the first failing vector number for each (x, y) pair in the shmoo results plots.

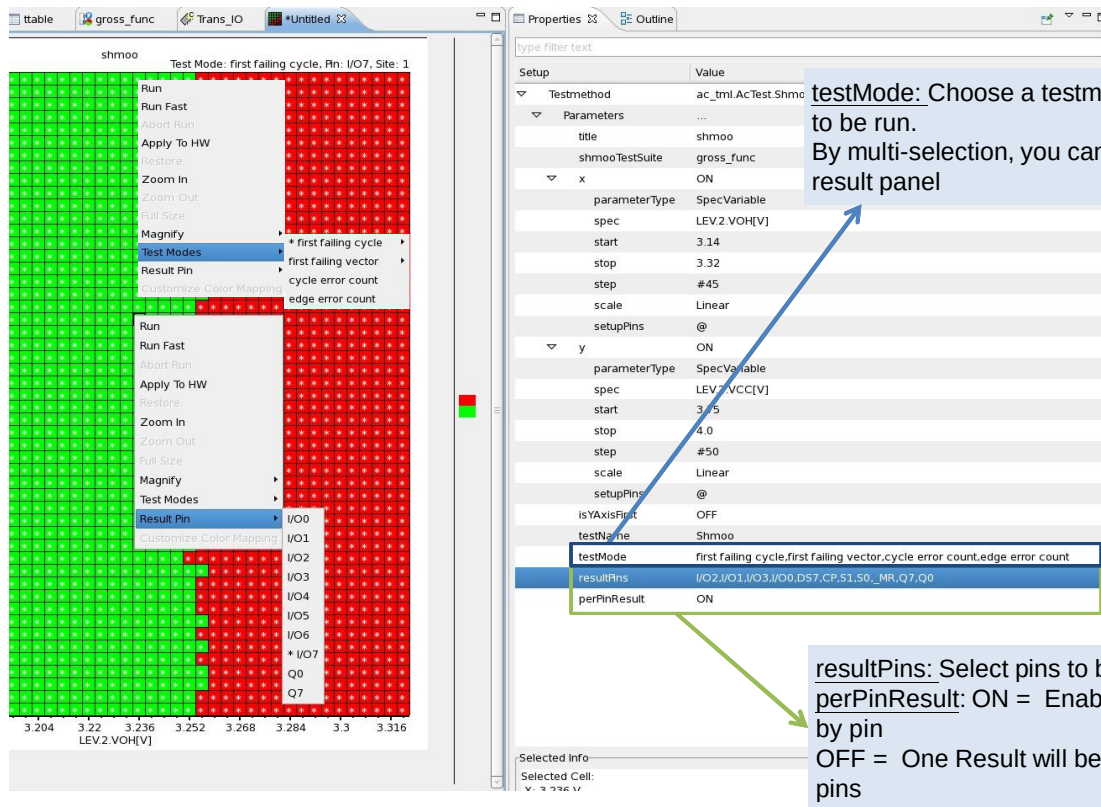
Cycle error count (used only for functional tests): Displays the number of the total failed cycles for each(x, y) pair in the shmoo results plots.

Edge error count (used only for functional tests): Displays the number of the total failed edges for each (x, y) pair in the shmoo results plots.

User defined measurement (used for shmoo over testsuite).

ADVANTEST

Interactive usage – select modes and pins



The screenshot shows the Advantest Shmoo tool interface. On the left is a shmoo plot with a green area and a red area. A context menu is open over the plot, showing options like 'Run', 'Run Fast', 'Abort Run', 'Apply To HW', 'Restore', 'Zoom In', 'Zoom Out', 'Full Size', 'Magnify', 'Test Modes', 'Result Pin', and 'Customize Color Mapping'. The 'Test Modes' option is selected, showing a list of test modes: 'first failing cycle', 'first failing vector', 'cycle error count', and 'edge error count'. The 'Result Pin' option is also selected, showing a list of pins: 'I/O0', 'I/O1', 'I/O2', 'I/O3', 'I/O4', 'I/O5', 'I/O6', 'I/O7', 'Q0', and 'Q7'. On the right is the Properties panel, showing the 'Setup' section with parameters like 'Testmethod', 'Parameters', 'x', 'y', 'parameterType', 'spec', 'start', 'stop', 'step', 'scale', 'setupPins', 'isYAxisFirst', and 'testName'. The 'testMode' parameter is highlighted, showing its value as 'first failing cycle, first failing vector, cycle error count, edge error count'. The 'resultPins' parameter is also highlighted, showing its value as 'I/O2, I/O1, I/O3, I/O0, D57, CP, S1, S0, MR, Q7, Q0'. The 'perPinResult' parameter is set to 'ON'.

testMode: Choose a testmode a multiple test modes to be run.
By multi-selection, you can view all test mode result in result panel

resultPins: Select pins to be run
perPinResult: ON = Enable to display the result pin by pin
OFF = One Result will be displayed for all selected pins

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Shmoo Tool Unit 19 - 17

Shmoo execution

- shmoo icons



F: run fast execution

S: run shmoo

: Snapshot

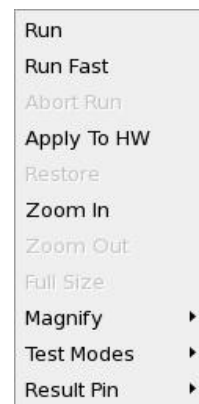
Apply to HW:

Apply shmoo cell settings (edges, levels,...) to the current primaries.

- shmoo menu



- shmoo context menu



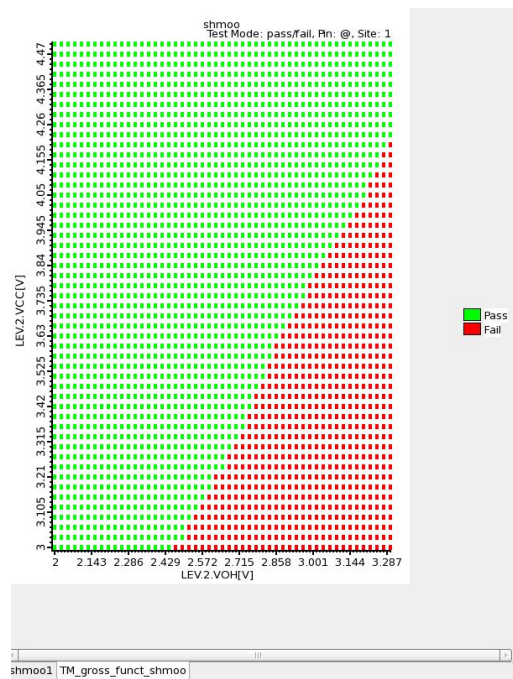
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Shmoo Tool Unit 19 - 18

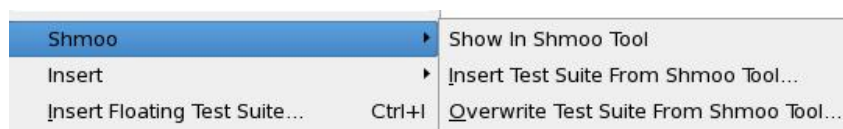
Shmoo Run fast



- Access via Test Result in TPE

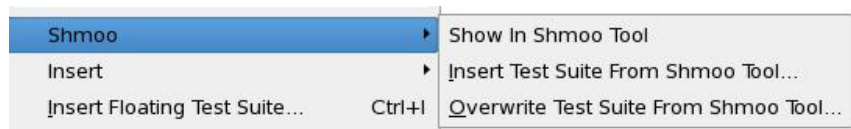
Shmoo from Testflow

- Load and open testflow in flow sequence editor
- Execute a testsuite
- Right Click and select Shmoo-> Show in Shmoo Tool
- Setup shmoo parameters and execute shmoo

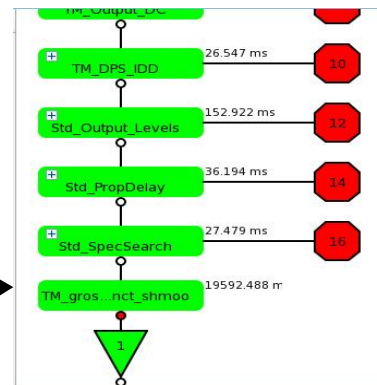


Shmoo in Testflow

- Load and open testflow in flow sequence editor
- Right Click and select Shmoo-> Insert Test suite from Shmoo Tool
- Save and execute the testflow with the shmoo testsuite



- Shmoo testsuite



Available types of Shmoo Test

§ Functional Test Shmoos

Standard Shmoo executing a functional test for each X/Y setting.
Pattern is taken from primary settings.
“shmooTestSuite” parameter is unused.

§ Shmoo tests over an existing testsuite

For each X/Y parameter setting the selected testsuite is executed.
Testsuite is selected through “shmooTestSuite” parameter
User-defined testmethod is called, its TM parameters can be as shmoo X or Y axis.

§ A customized shmoo test.

To adjust the shmoo behavior to your needs, a customized shmoo test method based on a provided customization template can be created.
Only for advanced users when the shmoo testmethod cannot meet your requirements.

§ A fail-bit-count Shmoo

For each X/Y parameter setting the number of fail bits is shown in the shmoo.
This is available only for MTP.

Shmoo Over Testsuite

ShmooTestSuite

- Name of an existing test suite to be shmoo-ed over.
- This parameter is only used for shmoo over test suite.
- For User-defined testmethods , the testmethods' parameters can be used as the shmoo's X or Y parameter.

Primary setups

- There's no primary setup for shmoo-ed test suite.
- Primary setup are the ones from test suite itself.

Result Type: user defined Measurement

- When shmooing an existing test suite, the test results of this test suite are displayed on the shmoo plot if they are logged with the **judgeAndLog** API. To view the results that are not logged with the judgeAndLog API on shmoo plots, modify the test method code of the shmooed suite with the **SHMOO_RESULT** API to set up these results.
- An example file is in attached. For detailed syntax you can refer TDC topic 136923

Reference

V93000 Technical Documentation Center:

Topic 136156

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=136152>

Lab Exercise

- Digital Shmoo

Advantest V93000

Digital User Training

Test Program Execution in Production

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- The Test Program
- The Application Model File
- Executing Test Program
- Test Cell Control Tool (TCCT)
- Working with TCCT

ADVANTEST[®]

Learning Objectives

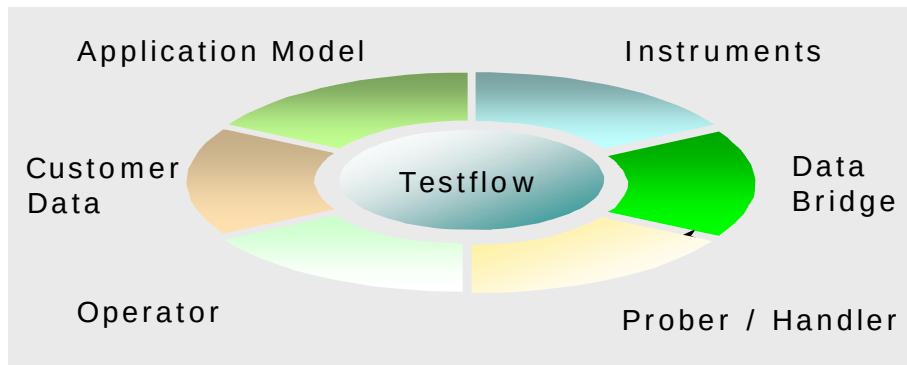
- Know the difference between Testflow and Test Program
- Know how to setup a Test Program
- Know how to execute a Test Program
- Know how to use the Test Cell Control Tool

Relevance

- In Production, a Testflow is executed a various number of times
- Before, in between and after the Testflow executions, different actions need to take place
- Controlling testflow execution using the Application Model
- Controlling testflow execution using the Test Cell Control Tool

Test Program

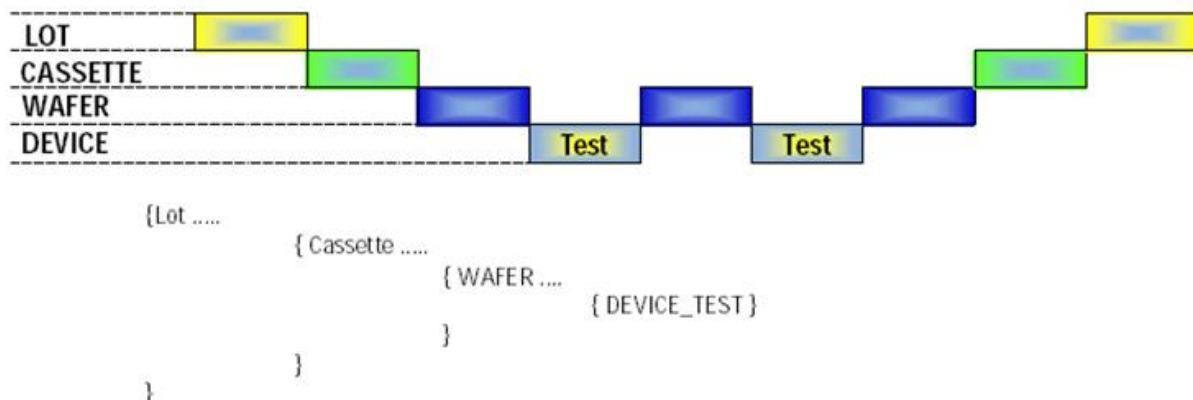
- Test Program is a combination of a Testflow and test environment data
- A test program does not need all the individual components, but must have at least a **Testflow** and an **application model** file



ADVANTEST

Application Model File

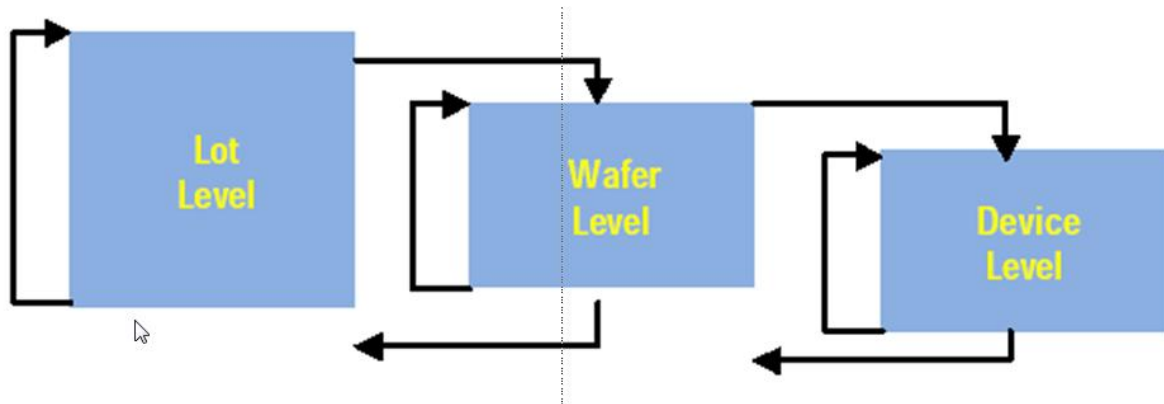
- Provides a hierarchical structure for the test process
- Each level "commands" the tester what to do
- This file is written in a dedicated high-level description language
- The application model file's structure reflects that of the application mode



ADVANTEST

Different Levels for Testing Wafers

- The last level in the application model is always the Device Test level (Testflow execution)



ADVANTEST

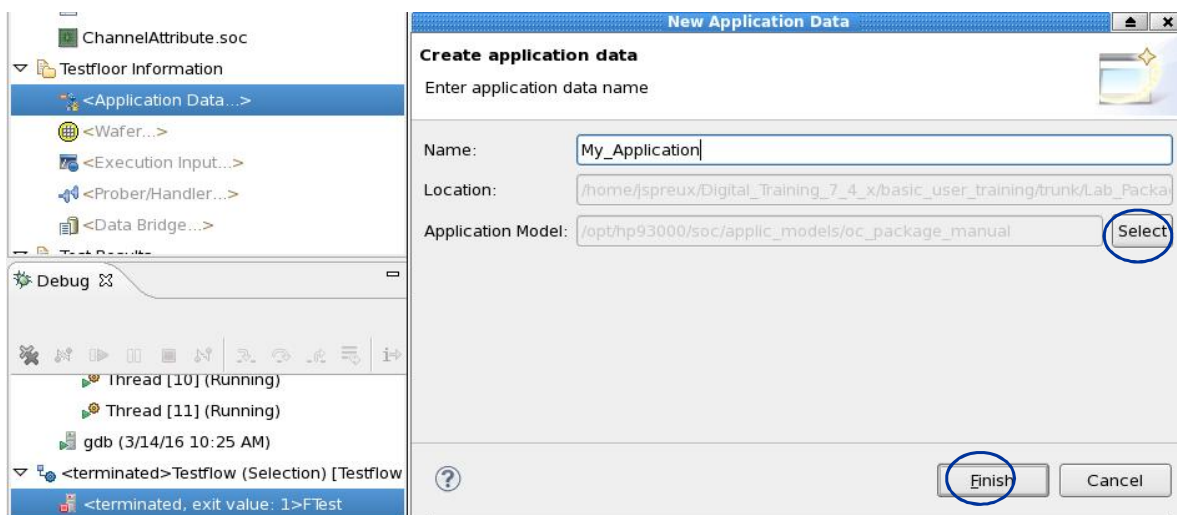
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TP Execution in Production - Unit 20 - 7

Defining an Application Model File

- In Test Program Explorer right-click <Application Data...> > New...
- This opens the “New Application Data” window
- Select** Application Model and **Finish**



ADVANTEST

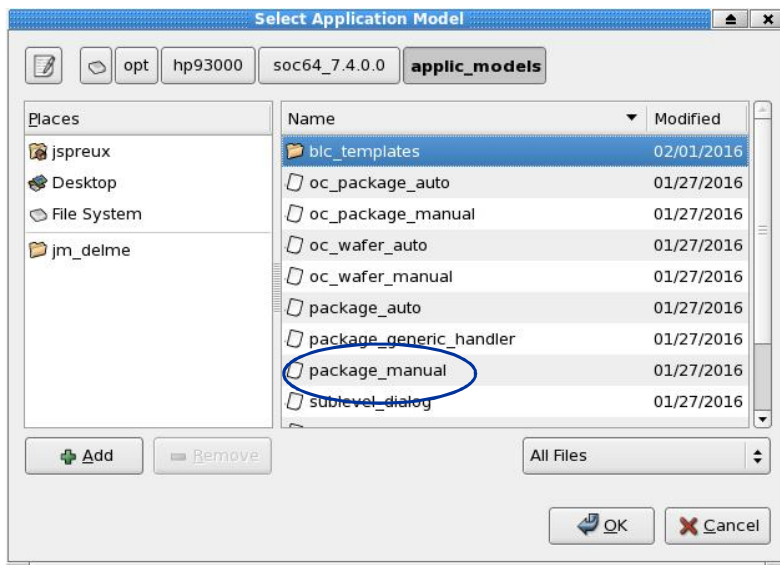
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TP Execution in Production - Unit 20 - 8

List of Standard Application Model Files

- Standard Application Model Files are located in **/opt/hp93000/soc/applic_models**



Select Application Model

Close window

ADVANTEST

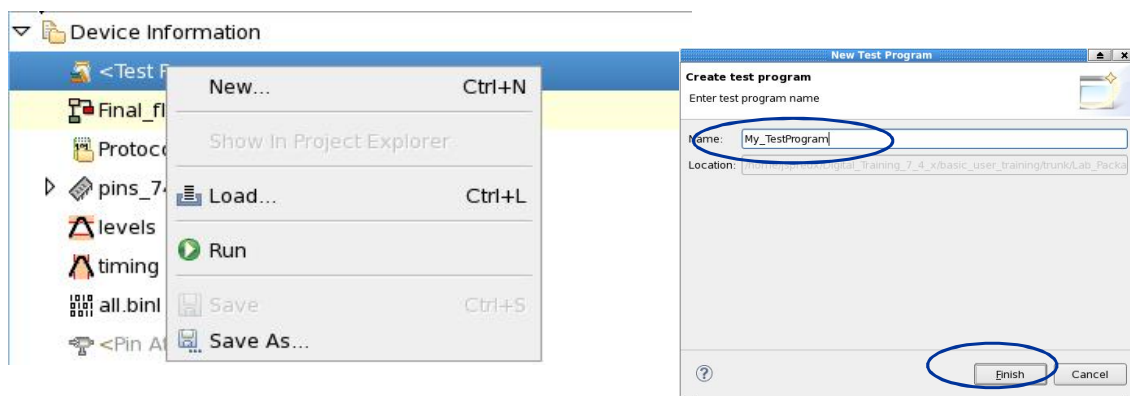
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TP Execution in Production - Unit 20 - 9

Defining a Test Program

- In Test Program Explorer, right-click <Test Program...> > New...
- This opens the “New Test Program” window
- Define the Name
- Select **Finish**



ADVANTEST

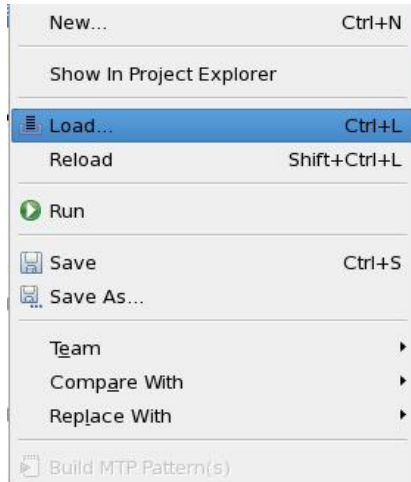
April 2016

All Rights Reserved - ADVANTEST CORPORATION

TP Execution in Production - Unit 20 - 10

Executing a Test Program

- In Test Program Explorer, right-click <Test Program>
- Select Run
- This will open and execute the associated Application Model



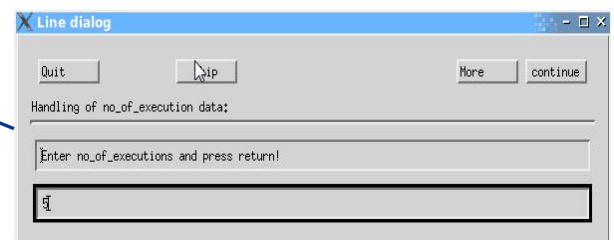
ADVANTEST

Test Program Controlled by Application Model

- Upon starting the Test Program a dialog pops up
- This dialog is caused by an entry in Application Model File

Application Model File “sublevel_dialog”

```
hp93000,applicmodel,0.1
-- standard application model for interactive testflow execution (GPE exec)
{ no_of_execution:
  LINE_DIALOG
  SUBLEVEL_COUNT no_of_executions = DIALOG_INPUT;
  { DEVICE device:
    {
      DEVICE_TEST
    }
  }
}
{ ON_PAUSE pause:
  -- LINE_DIALOG
  -- reason_paused = DIALOG_INPUT;
}
```

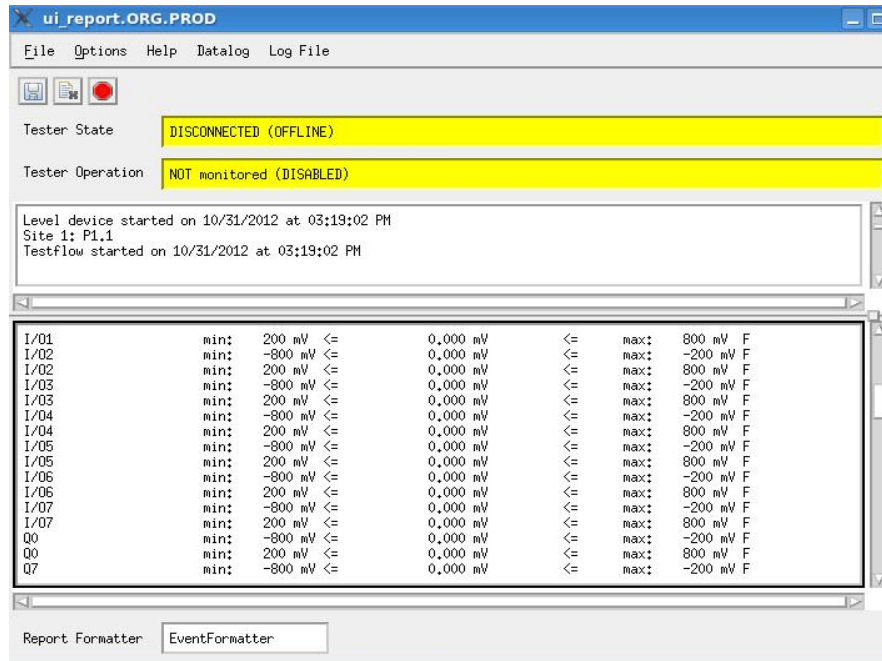


Causes execution of Testflow

ADVANTEST

Check Result of Test Program Execution

- After Testflow execution, see the results in the report window



ADVANTEST

TEST Cell Control Tool (TCCT)

Test Cell Control Tool(TCCT) is designed for production engineering.

TCCT allows:

- debugging the testflow during the production testing.
- product ramp-up in design validation
- product qualification
- manufacturing validation
- supports HAND test,
- PROBER test and
- HANDLER test.

ADVANTEST

TCCT Working model

Load Testflow

- Selects and loads the testflow to execute. The setup files used by the testflow will also be loaded. This equals loading the testflow and loading all its setup files in the EWC. If you already loaded the testflow and the setups in EWC, you still need to click the Load Testflow button in TCCT. TCCT will check whether the testflow has been loaded. If yes, the testflow information will be updated in TCCT.

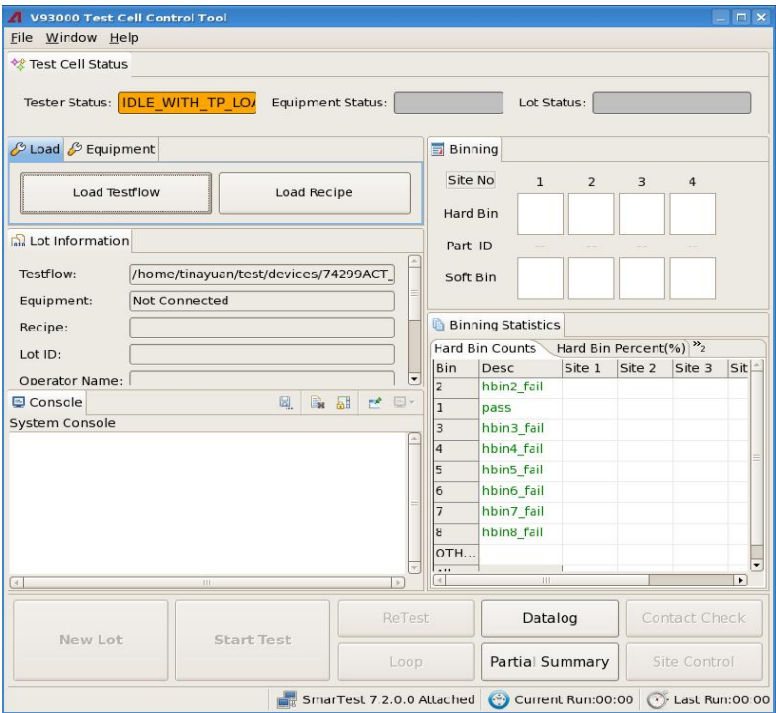
Equipment setting

- Selects the test type and the equipment to use. The test type can be HAND, PROBER or HANDLER. If you select PROBER or HANDLER, then you need to select the configuration file for the equipment.
- The available configuration files are listed in the file `/opt/hp93000/tcct/etc/equipment.xml`. You can edit this file and add the configuration files you want to use.

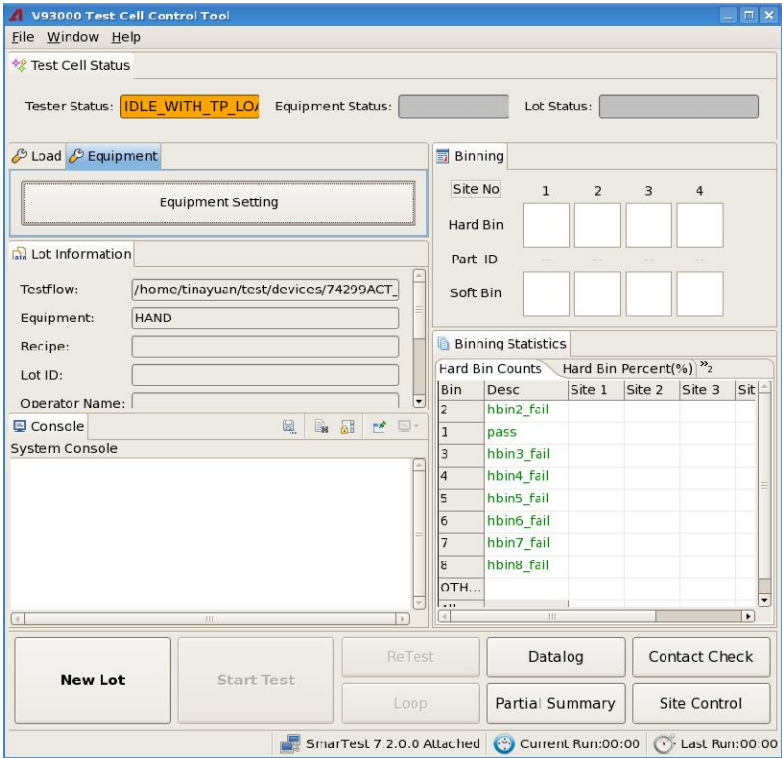
Working with TCCT

- To start the TCCT
 - enter the following command in a shell:
`/opt/hp93000/tcct/bin/tcct`
 - It will attach to the running SmarTest session automatically. If no SmarTest session is running, it will attach to the SmarTest session when you start it. If you have more than one SmarTest session running, a dialog will pop up asking you to select the session to work with. Select the session to use and click OK.

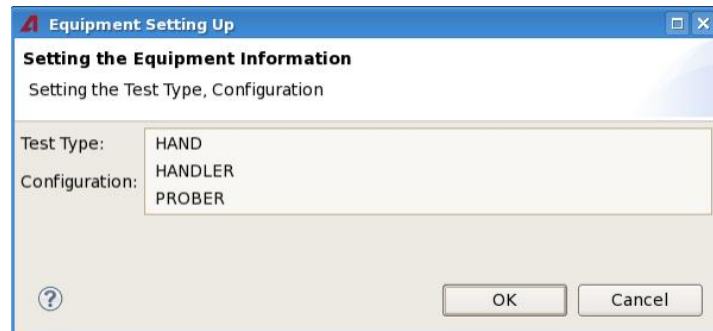
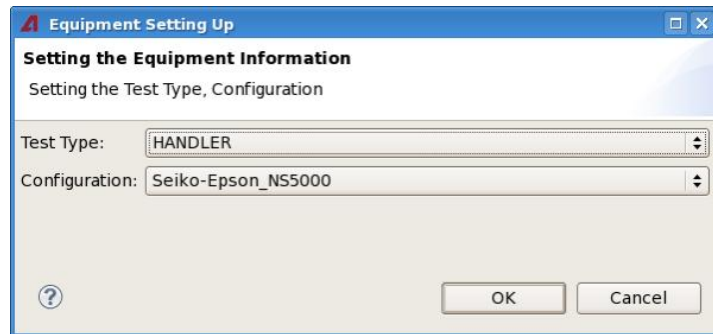
TCCT GUI



TCCT Equipment Settings



TCCT Equipment Settings



ADVANTEST

TCCT GUI

When the testflow is loaded and the equipment settings are done, the New Lot button will be enabled. Clicking the New Lot button will start the datalog for the test results of the following executions. Testing can begin now.

Datalog Setup

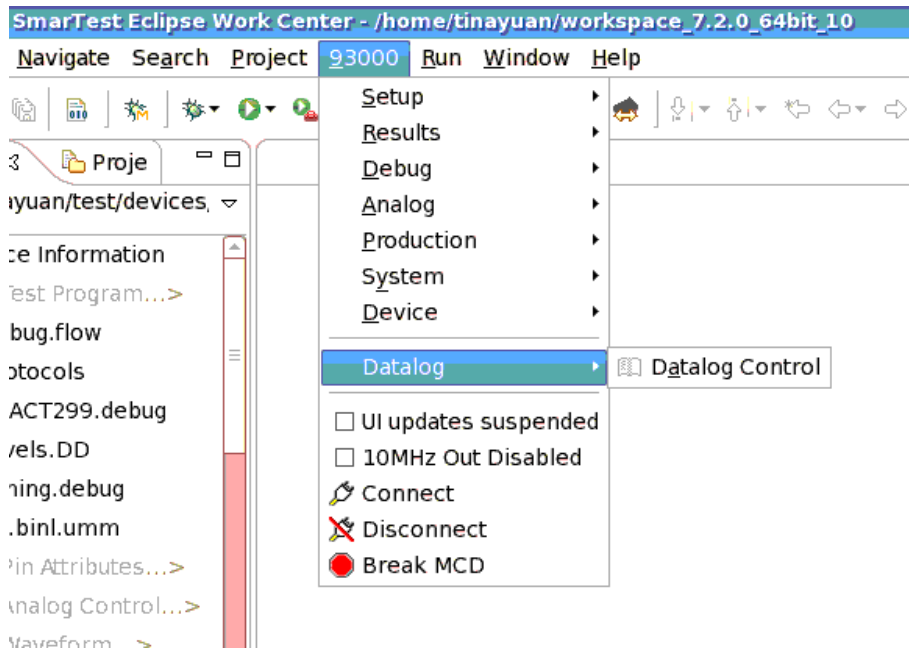
The Datalog setup button will bring up dialog window for detailed configuration of datalog file settings and generation options. Detailed description of the capabilities are on the next slide

Contact Check

When the testflow is loaded and equipment settings are done, the Contact Check button will be enabled. Clicking the button will execute the testflow. The binning information will be displayed in TCCT. But the test result will not be included in the data log file.

ADVANTEST

Datalog control



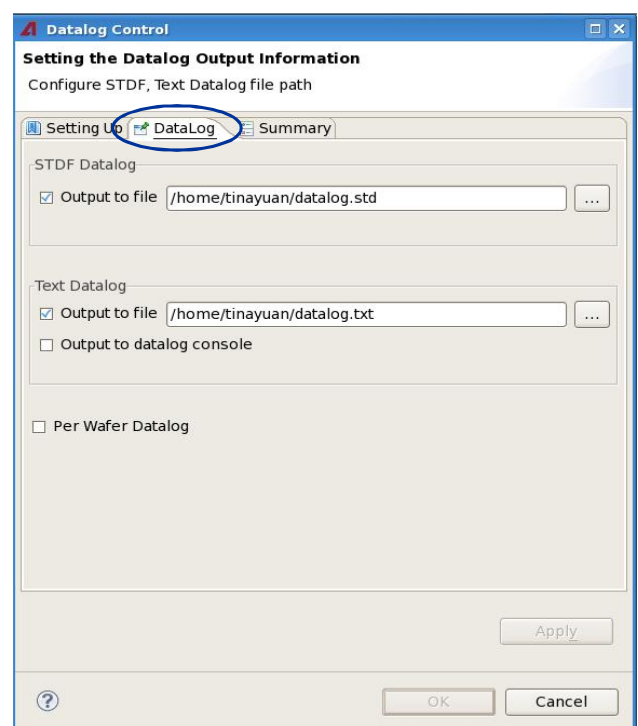
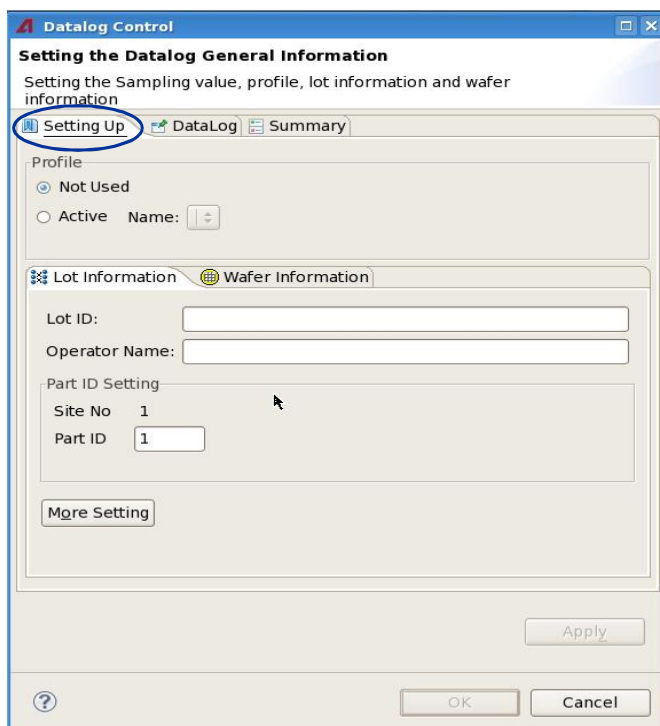
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TP Execution in Production - Unit 20 - 21

Datalog Control



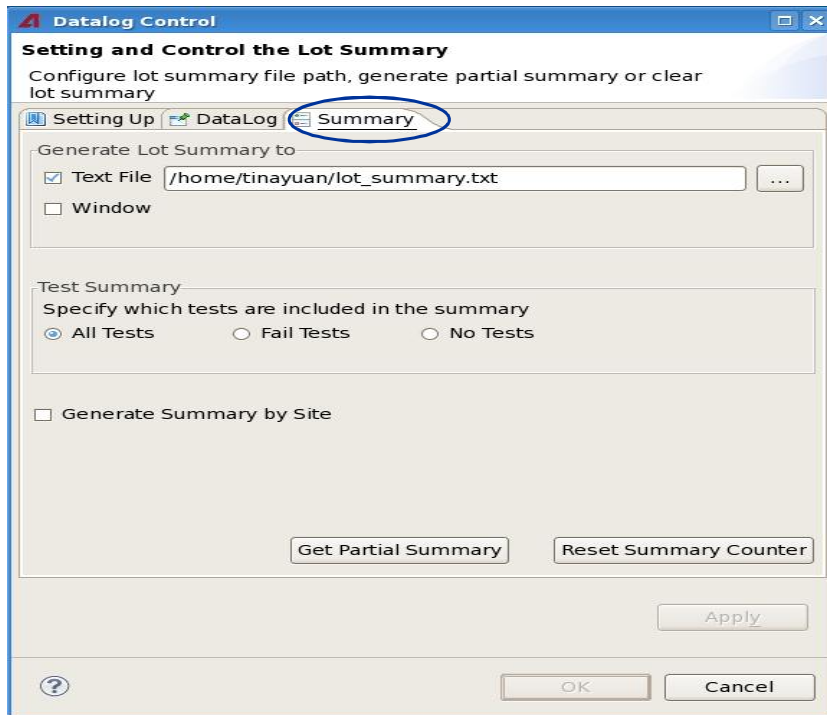
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

TP Execution in Production - Unit 20 - 22

Datalog Control



Test Execution

Start Test

When a lot is started, this button will be enabled. Clicking this button will execute the loaded testflow. In hand test, it only execute the testflow once. In handler/prober test, it will test all the devices in this lot if you don't pause the test.

Resume

Resumes the test of the lot that was paused. It will be enabled after you click Pause Test in handler/prober test.

Test Execution

ReTest

- When the loaded testflow has been executed and the device has not been binned out, this button will be enabled. In prober/handler test, you need to pause the test before doing retest. Clicking the button will execute the loaded testflow on the device again. The Binning area will be updated. The pass/fail counts and yield are updated according to the result of retest. The result of the retest will also be included in the data log file.

Loop

- When the testflow is loaded, equipment settings are done and the testflow has been executed, the Loop button is enabled. Clicking the button will bring up the Loop Setting dialog. Enter the loop times and the delay time in the dialog and click OK. The testflow will be continuously executed for the specified times. The delay time will be waited between two neighboring executions of the testflow.

Test Execution

Site Control

- Controls the sites to be executed in TCCT. Clicking the button will bring up the Site Control window where you can select the sites to be enabled.

Partial Summary

- Generates the partial lot summary for the tested devices until now if the summary has not been cleared yet. It has the same functionality with the Generate Partial Summary button in the Datalog Control window.

- **End Lot**

- Ends the testing of the current lot. Clicking the button will generate the STDF file and lot summary file for the lot. This button will only be available after new lot has started.

Reference

V93000 Technical Documentation Center (TDC):

- Topic 102282
<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=102282>
- Topic 132026
<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=132026>

Advantest V93000

Digital User Training

Integrated Result Analysis Tool

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Integrated Result Analysis Tool Overview
- Operating principle
- Graph types
- Data Analysis workflows

ADVANTEST[®]

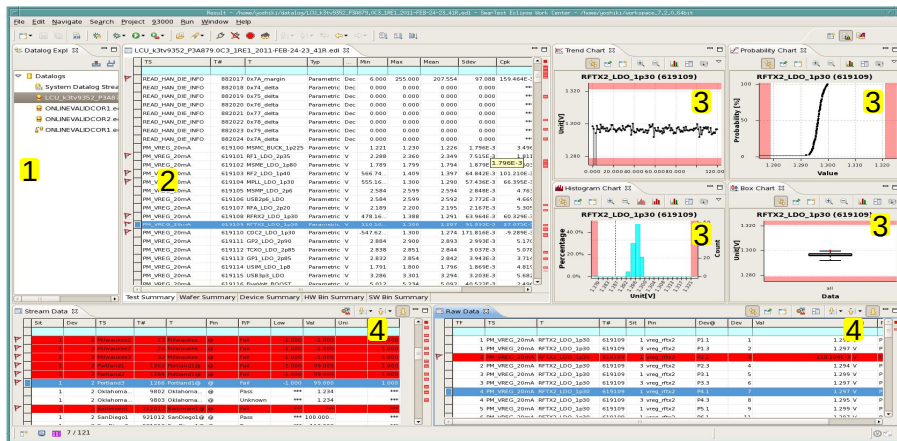
Learning Objectives

- Know how to prepare a Test data prior to using the Integrated Result Analysis Tool
- Know how to use the Integrated Result Analysis Tool along with the Datalog stream
- Know how to extract statistical parameters using the Integrated Result Analysis Tool

Relevance

- The Integrated Result Analysis Tool is a versatile tool used in correlation, characterization and debug scenarios

V93000 Integrated Result Analysis tool



- 1: Data Log Explorer
- 2: Data navigation view
- 3: Result views (as graphics)
- 4: Detail data views (as table)

ADVANTEST

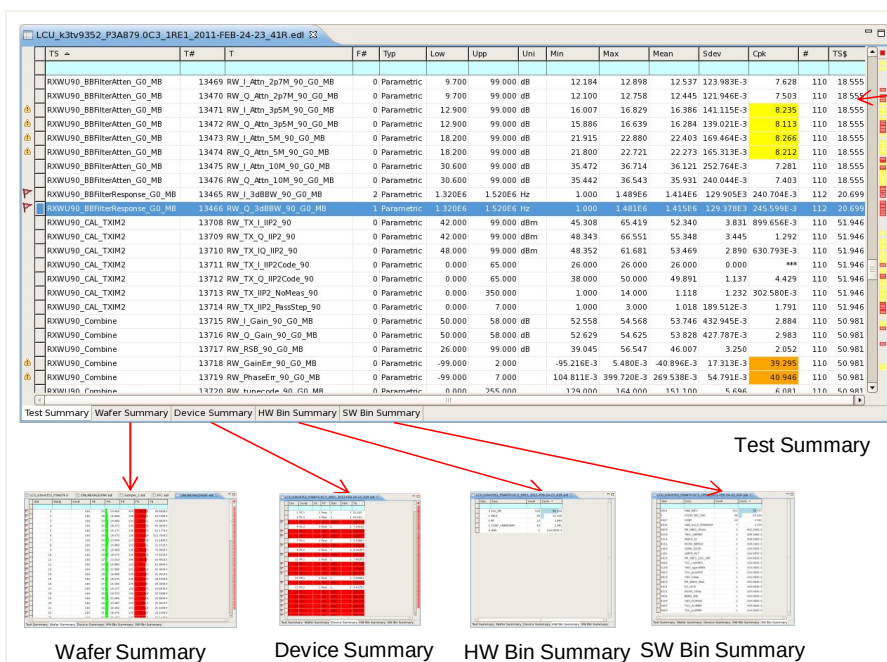
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Integrated Result Tool Unit 21- 5

Data Navigation View

Data navigation view shows statistical summary of selected data log in tabular form. Test Summary tab is main entry point for further interactive data analysis. Data navigation view is updated during testflow execution to enable real-time monitoring of results.



Features

- Multi-tab window
- Test Summary tab to show statistical test summary
- Additional summary tabs (Wafer/Device/HW Bin/SW Bin)
- Dynamically updated statistical summary while test execution is in progress

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

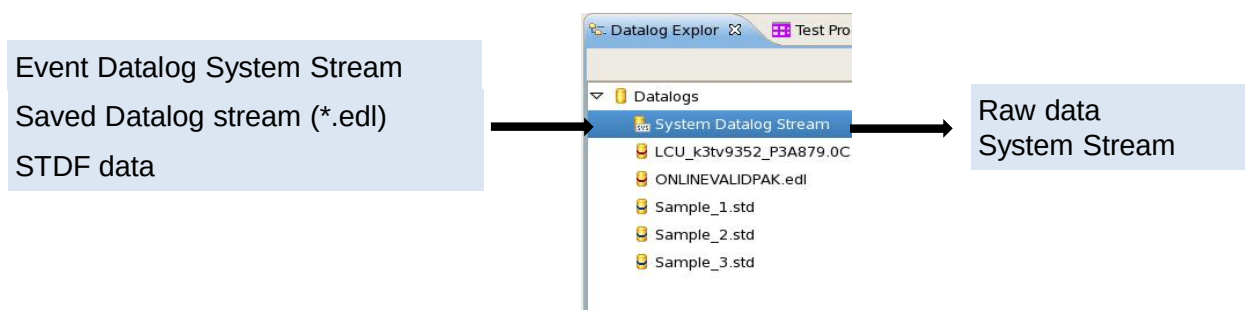
Integrated Result Tool Unit 21- 6

Tools main functionalities

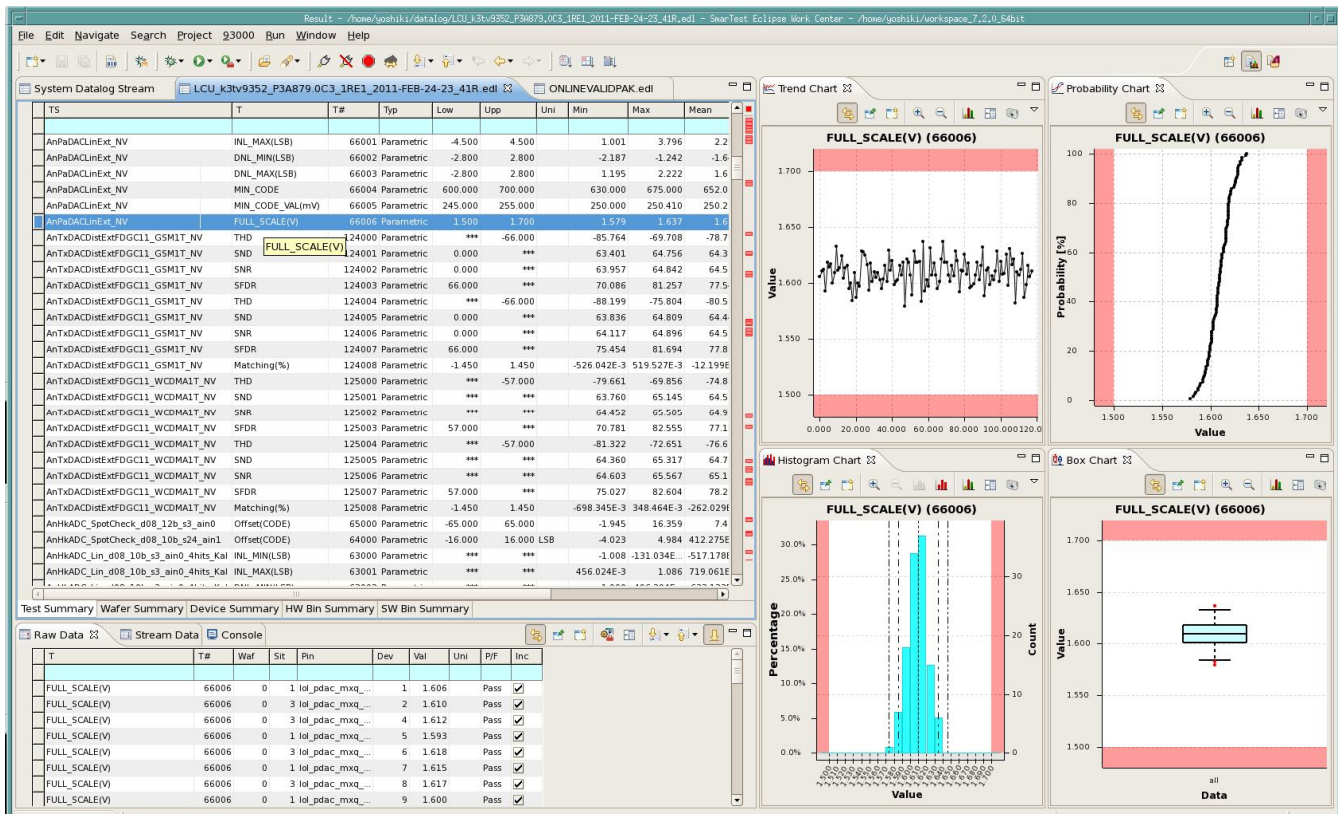
- Statistical analysis of test results (parametric and functional) as well as test times
- Online and offline data log access, multiple data sources (STDF and EDL)
- Real time display of processed results
- Textual, tabular and graphical displays
- Fast filtering of the data population
- Tools for comparison and correlation
- Easy data exchange with various applications (by text, .png, csv, etc.)

Integrated Result Tool Operating principle

- Statistical and graphical analysis on real-time or saved Event Data Log (EDL) test results possible



Analyzing stream Data



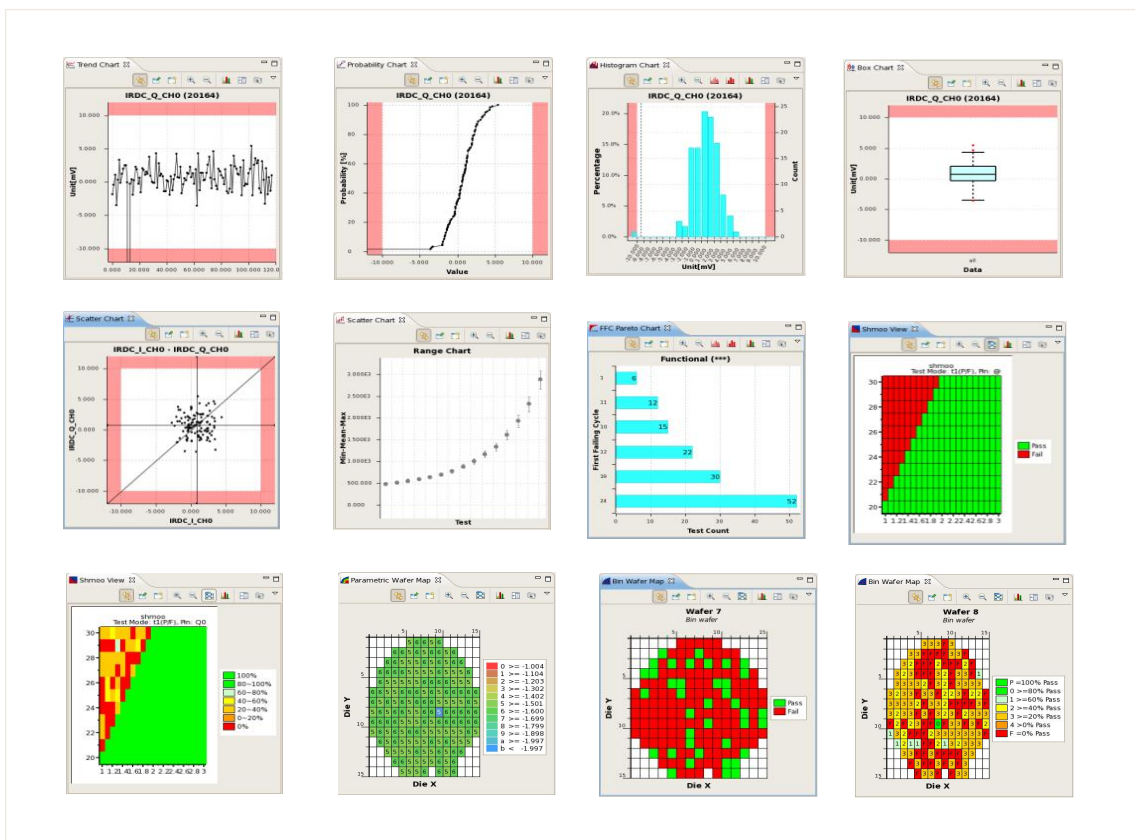
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Integrated Result Tool Unit 21- 9

Provided Graph types



ADVANTEST

April 2016

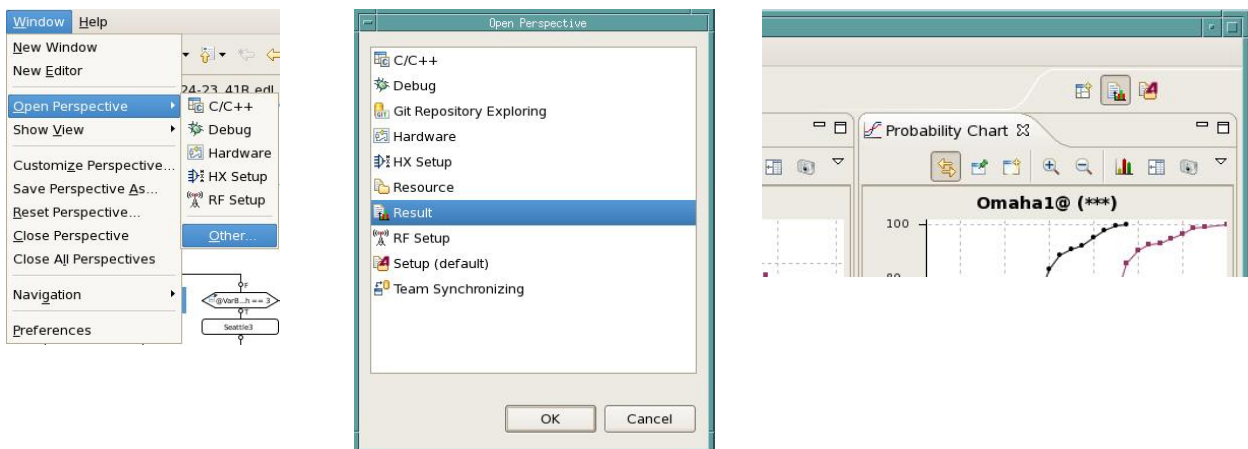
All Rights Reserved - ADVANTEST CORPORATION

Integrated Result Tool Unit 21- 10

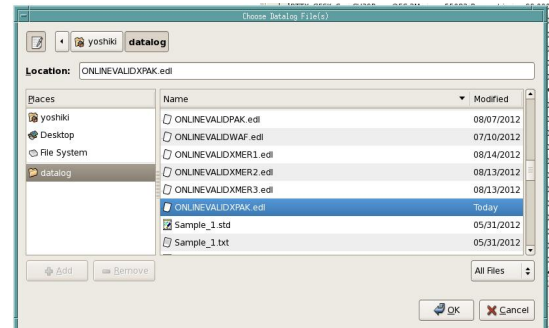
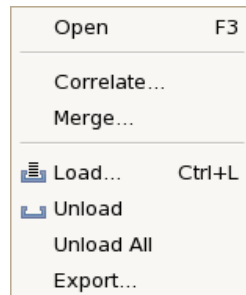
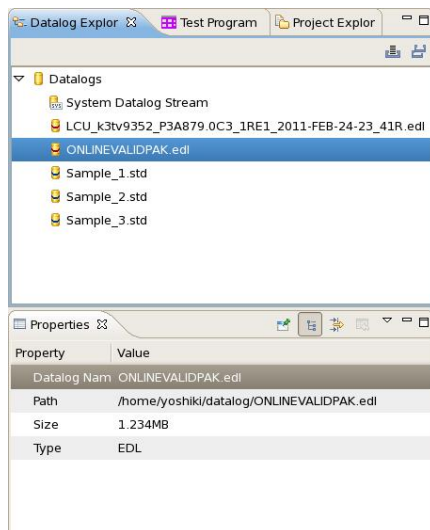
Typical Data analysis workflow

- Switch to Result perspective
- Load desired data log file
- Open data navigation view
- Identify tests which returned unexpected results
- Jump to those tests
- Sort, filter, configure table appearance
- View test results on result view
- Exclude outliers
- View statistical test results by site or by pin
- Open additional result view for visual comparison
- Save graph to file

Switch to Result perspective



Load datalog



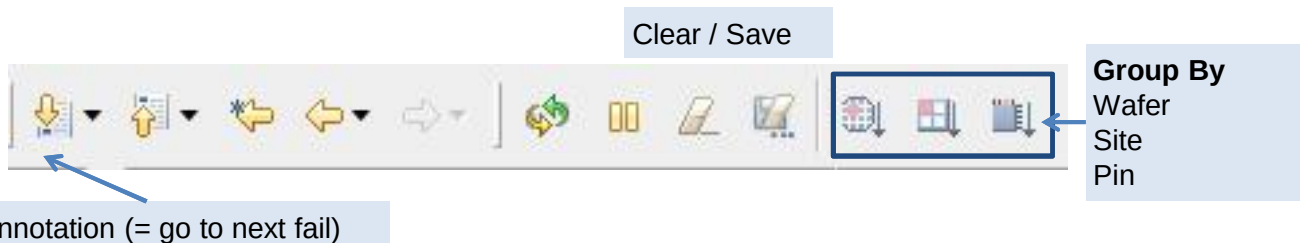
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Integrated Result Tool Unit 21- 13

Data Navigation View



Next Annotation (= go to next fail)

Test Suite	Test	Test#	Type	Lower	Upper	Unit	Min	Max	Mean	Sdev	Cpk
Continuity	passVolt_mV	***	Parametric	-800.000	-200.000	mV	-659.078	-296.330	-473.857	149.981	0.608650
Continuity	passVolt_mV	1	Parametric	200.000	800.000	mV	562.989	626.069	591.725	25.5115	2.72132
gross_funcnt	Functional	***	Functional	***	***		***	***	***	***	***
leakage	passCurrentLow_uA	***	Parametric	-100.000	100.000		-0.0500000	0.0180000	-0.0160888	0.0126821	2627.95
leakage	FUNCTION TEST Low	***	Parametric	***	***		0	0	0	0	***
leakage	passCurrentHigh_uA	***	Parametric	-100.000	100.000		-0.0490000	0.0230000	-0.0162360	0.0125904	2647.09
leakage	FUNCTION TEST High	***	Parametric	***	***		0	0	0	0	***
TM_gross_funcnt	Functional	***	Functional	***	***		***	***	***	***	***
TM_Output_DC	Vouth	***	Parametric	4.10000	5.00000		4.49938	4.49996	4.49967	143.664E-6	927.323
TM_DPS_IDD	ldd	***	Parametric	0	10.0000	uA	0.733914	0.992491	0.784753	0.0558989	4.67960
Std_Output_Levels	OutputDC_LowLevel	***	Parametric	0	0.100000		0.0474830	0.0634700	0.0570081	0.00389883	3.67562
Std_Output_Levels	OutputDC_HighLevel	***	Parametric	4.00000	5.00000		4.25031	4.25681	4.25345	0.00179770	46.9946
Std_Output_Levels	FUNCTIONAL TEST	***	Parametric	***	***		0	0	0	0	***
Std_PropDelay	Parametric_SpecSearch	***	Parametric	3.00000	20.0000	ns	6.00000	6.29000	6.14260	0.132965	7.87825
Std_PropDelay	Parametric_SpecSearch	1	Parametric	3.00000	20.0000	ns	5.99000	6.28000	6.13353	0.131986	7.91380
Std_SpecSearch	SpecSearch_Test	***	Parametric	***	***		6.00000	6.29000	6.14353	0.132159	***

Display Statistics

Fail Annotation

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Integrated Result Tool Unit 21- 14

Detail Data View

TM Flow ContinuityUtil.hpp System Data Log Stream

Test Suite	Test	Test#	Type	Pattern	Pin	Lower	Upper	Unit	Min	Max	Mean	Sdev	Cpk	Cpl	Cpu	Device#	Fail#	Time	TS Time
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O1		***	***	uA	2.69000	2.78300	2.73698	0.0462520	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O2		***	***	uA	1.59900	2.80800	2.65580	0.209460	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O3		***	***	uA	2.80800	2.86000	2.83362	0.0255299	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O4		***	***	uA	2.69200	2.81300	2.78985	0.0268375	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O5		***	***	uA	2.67100	2.76300	2.71702	0.0452673	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O6		***	***	uA	2.71000	2.73700	2.73150	0.00562193	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O7		***	***	uA	2.75500	2.80800	2.78135	0.0263456	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O8		***	***	uA	2.76400	2.82900	2.79689	0.0322731	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O9		***	***	uA	2.74900	2.79300	2.77106	0.0220517	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O10		***	***	uA	2.80900	2.84600	2.83444	0.00905463	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O11		***	***	uA	2.72300	2.81800	2.77002	0.0462968	***	***	***	100	0	0.0488100	65.0504
Leakage	passCurrentHigh_uA	***	Parametric	gross_func_leak_I/O12		***	***	uA	0.0440000	2.70800	1.37753	1.33217	***	***	***	100	0	0.0488100	65.0504
Leakage	FUNCTION TEST High	***	Parametric	gross_func_leak_@		0	0		0	0	0	0	***	***	***	100	2	0.0285700	65.0504

Test Summary Wafer Summary Device Summary HW Bin Summary SW Bin Summary

Raw Data Stream Data

Test	Test#	Pattern	Wafer	Site	Pin	Device	Value	Unit	P/F	Time	TS Time	Print To Data Log
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.036...	69.4010	✓
passCurre...	***	gross_fu...	0	2	Q7	0	2.79300	uA	Pass	0.041...	69.4010	✓
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.009...	68.7960	✓
passCurre...	***	gross_fu...	0	2	Q7	0	2.79300	uA	Pass	0.033...	68.7960	✓
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.037...	68.3970	✓
passCurre...	***	gross_fu...	0	2	Q7	0	2.79300	uA	Pass	0.037...	68.3970	✓
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.039...	67.3980	✓
passCurre...	***	gross_fu...	0	2	Q7	0	2.79300	uA	Pass	0.043...	67.3980	✓
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.026...	68.8190	✓
passCurre...	***	gross_fu...	0	2	Q7	0	2.79300	uA	Pass	0.037...	68.8190	✓
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.037...	67.3600	✓
passCurre...	***	gross_fu...	0	2	Q7	0	2.79300	uA	Pass	0.038...	67.3600	✓
passCurre...	***	gross_fu...	0	1	Q7	0	2.74900	uA	Pass	0.038...	66.3800	✓

Example:
Data Navigation View: "Group By Pin":
100 execution of pin Q7 summarized in c

Raw Data: displays 100 individual
for pin Q7 (= selected row)

Example:

Data Navigation View: "Group By Pin":
100 execution of pin Q7 summarized in one row

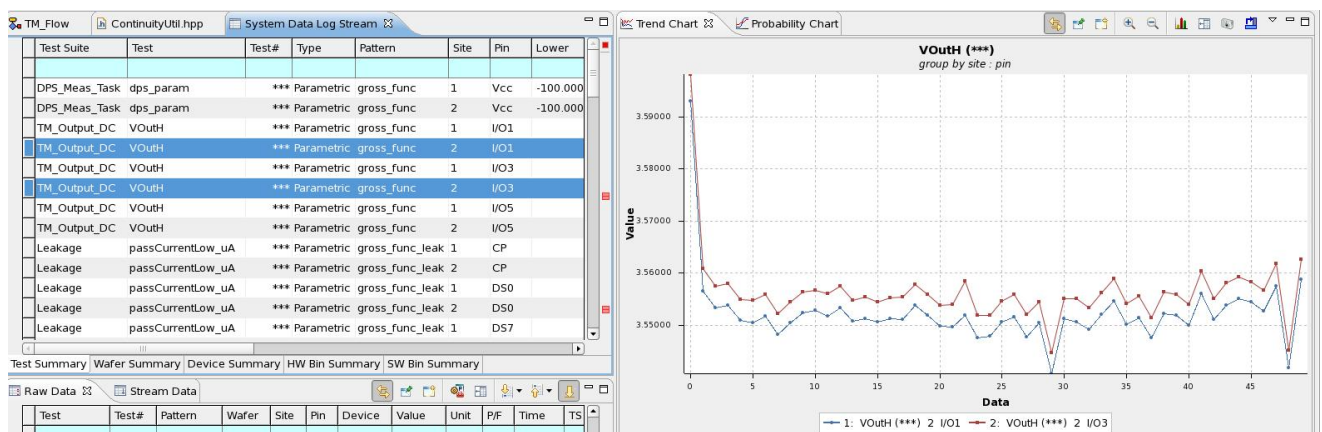
Raw Data: displays 100 individual records for pin Q7 (= selected row)

Stream Data: displays all individual records of the datalog

ADVANTEST

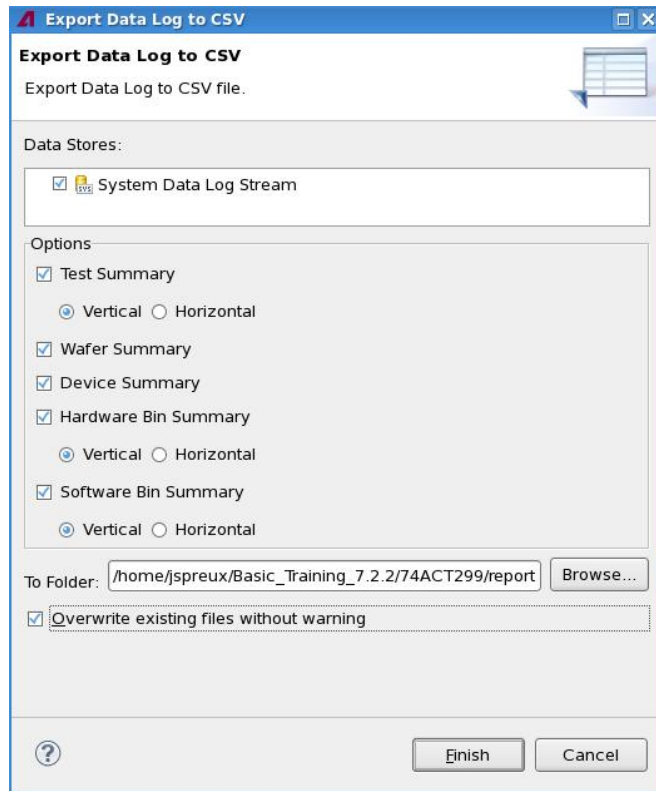
Graph View

- Select data that have to be displayed in the opened graphs.
- **Multi selection** = Display data on the same graph.
- All result views are still **dynamically updated** when new data arrives (in online).
- Possible to **exclude** data points.



ADVANTEST

Export Datalog stream



ADVANTEST

Data Analysis flow (2)

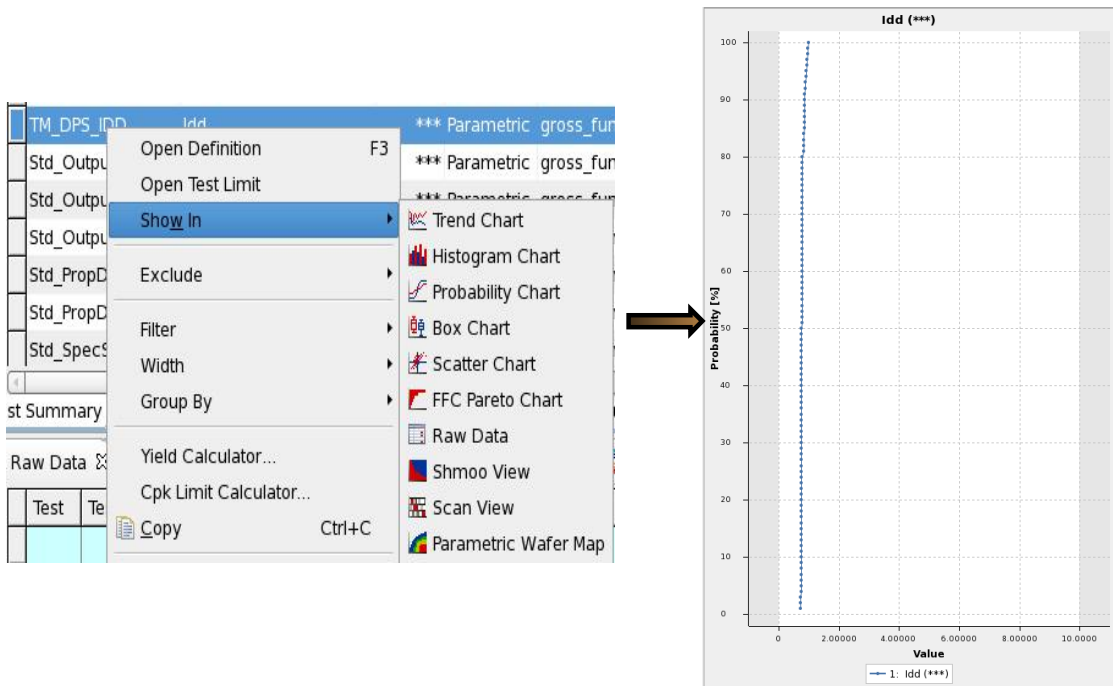
- Load saved data log stream
- Correlate offline data log stream
- Analyze correlation summary

ADVANTEST

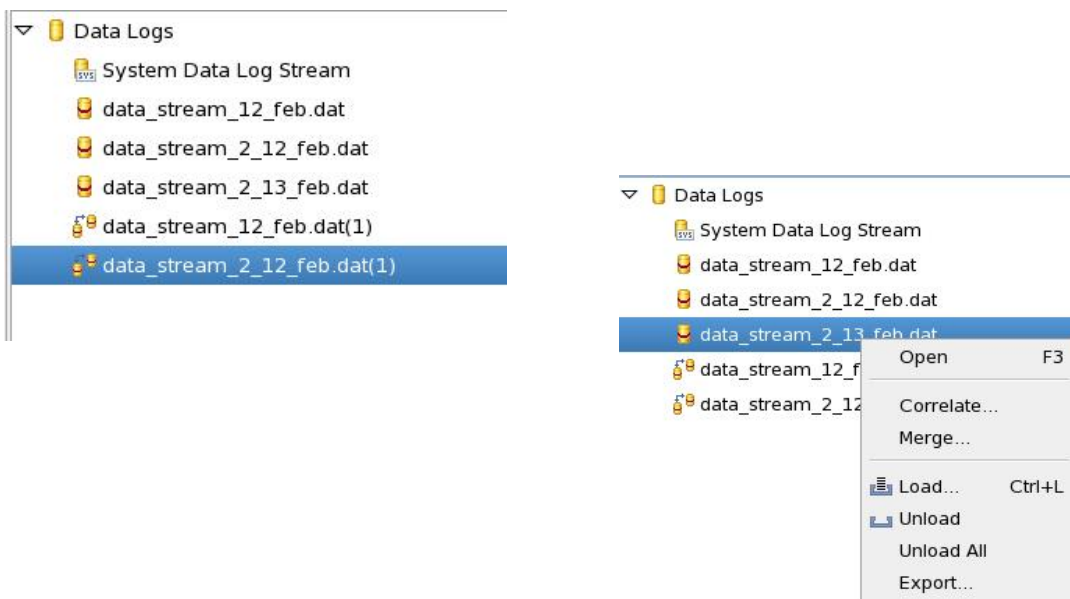
Load Datalog stream

Data Logs							
System Data Log Stream							
data_stream_2_12_feb.dat							
Continuity	passVolt_mV	*** Parametric	gross_func	-800.000	-200		
Continuity	passVolt_mV	1 Parametric	gross_func	200.000	800		
gross_func	Functional	*** Functional	gross_func	***			
leakage	passCurrentLow_uA	*** Parametric	gross_func_leak	-100.000	100		
leakage	FUNCTION TEST Low	*** Parametric	gross_func_leak	***			
leakage	passCurrentHigh_uA	*** Parametric	gross_func_leak	-100.000	100		
leakage	FUNCTION TEST High	*** Parametric	gross_func_leak	***			
TM_gross_func	Functional	*** Functional	funcnt_first	***			
TM_Output_DC	Vouth	*** Parametric	gross_func	4.10000	5.0		
TM_DPS_IDD	Idd	*** Parametric	gross_func	0	10.		
Std_Output_Levels	OutputDC_LowLevel	*** Parametric	gross_func	0	0.10		
Std_Output_Levels	OutputDC_HighLevel	*** Parametric	gross_func	4.00000	5.0		
Std_Output_Levels	FUNCTIONAL TEST	*** Parametric	gross_func	***			
Std_PropDelay	Parametric_SpecSearch	*** Parametric	gross_func	3.00000	20.		
Std_PropDelay	Parametric_SpecSearch	1 Parametric	gross_func	3.00000	20.		
Std_SpecSearch	SpecSearch_Test	*** Parametric	gross_func	***			

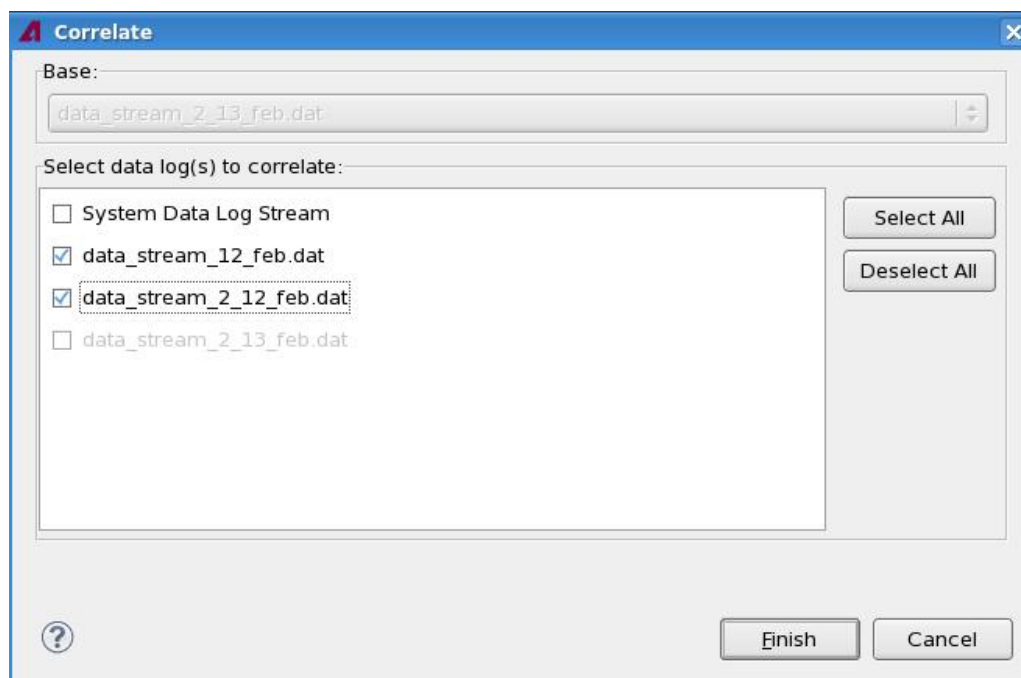
View offline result with charts



Correlate Data log stream



Select stream to correlate



View correlation summary

System		
	Legend File	Path
A	* data_stream_2_12_feb.dat	/home/jspreux/Basic_Training_7.2.2/74ACT299/report/data_stream_2_
B	data_stream_12_feb.dat	/home/jspreux/Basic_Training_7.2.2/74ACT299/report/data_stream_12
C	data_stream_2_13_feb.dat	/home/jspreux/Basic_Training_7.2.2/74ACT299/report/data_stream_2_

Test Summary Correlation Information:

Data log [A] summary information

TS=Continuity, TM=dc_tml.DcTest.Continuity, T=passVolt_mV, TN=0
TS=Continuity, TM=dc_tml.DcTest.Continuity, T=passVolt_mV, TN=1
TS=gross_funct, TM=ac_tml.AcTest.FunctionalTest, T=Functional, TN=0
TS=leakage, TM=dc_tml.DcTest.Leakage, T=passCurrentLow_uA, TN=0
TS=leakage, TM=dc_tml.DcTest.Leakage, T=FUNCTION TEST Low, TN=0
TS=leakage, TM=dc_tml.DcTest.Leakage, T=passCurrentHigh_uA, TN=0
TS=leakage, TM=dc_tml.DcTest.Leakage, T=FUNCTION TEST High, TN=0
TS=TM_gross_funct, TM=74ACT299_tml.Functional, T=Functional, TN=0
TS=TM_Output_DC, TM=74ACT299_tml.OutputDC, T=Vouth, TN=0
TS=TM_DPS_IDD, TM=74ACT299_tml.DPS_Meas, T=Idd, TN=0
TS=Std_Output_Levels, TM=dc_tml.DcTest.OutputDC, T=OutputDC_LowLevel, TN=0
TS=Std_Output_Levels, TM=dc_tml.DcTest.OutputDC, T=OutputDC_HighLevel, TN=0
TS=Std_Output_Levels, TM=dc_tml.DcTest.OutputDC, T=FUNCTIONAL TEST, TN=0
TS=Std_PropDelay, TM=ac_tml.AcTest.ParametricFunctionalTest, T=Parametric_SpecSearch, TN=0
TS=Std_PropDelay, TM=ac_tml.AcTest.ParametricFunctionalTest, T=Parametric_SpecSearch, TN=1
TS=Std_SpecSearch, TM=ac_tml.AcTest.SpecSearch, T=SpecSearch_Test, TN=0

Reference

V93000 Technical Documentation Center (TDC):

Topic 136257

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=136257>

Lab Exercise

- Integrated Result Analysis Tool

Advantest V93000

Digital User Training

Pin Margin Tool

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Passive and Active Result Tools
- Pin Margin Tool (Active)

ADVANTEST[®]

Learning Objectives

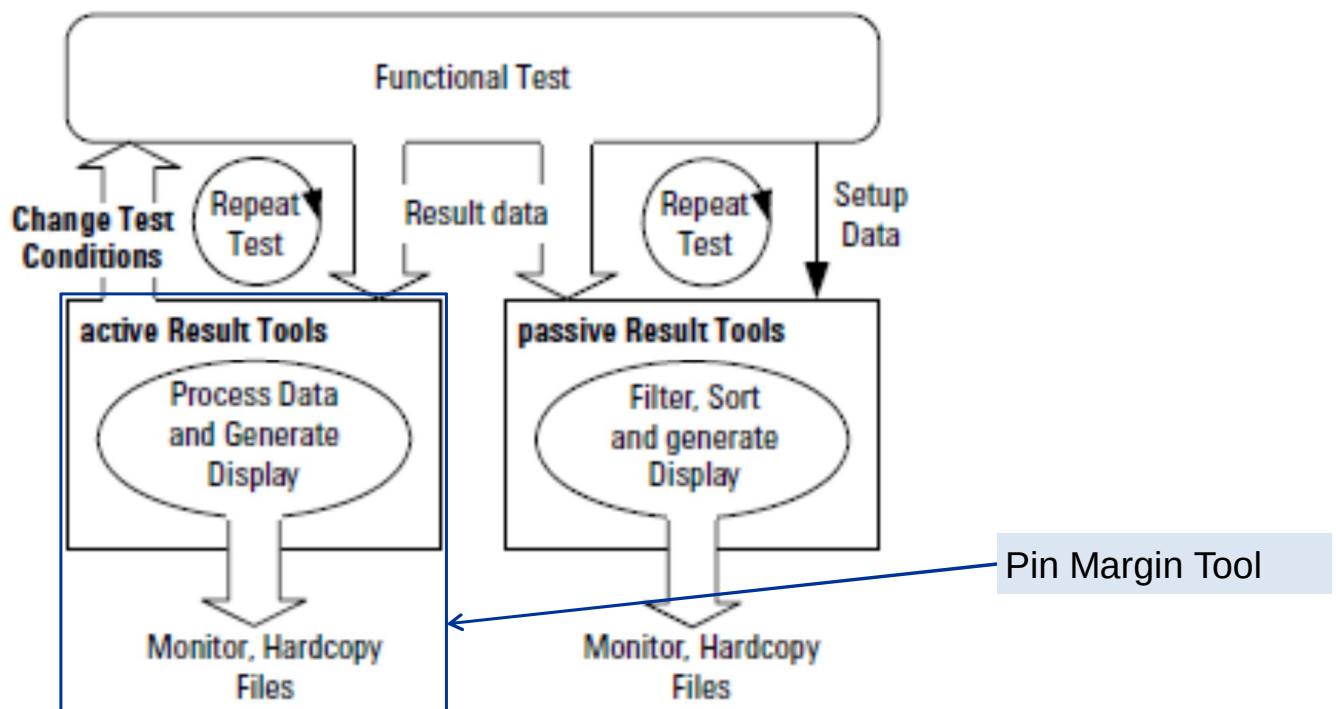
- Know the Pin Margin Tool
- Know how to prepare a Testsuite prior to start the Pin Margin Tool
- Know how to read multiple parameters

Relevance

- Pin margin test allows to check for parameter marginality

Debug and Analysis

Passive & Active Result Tools



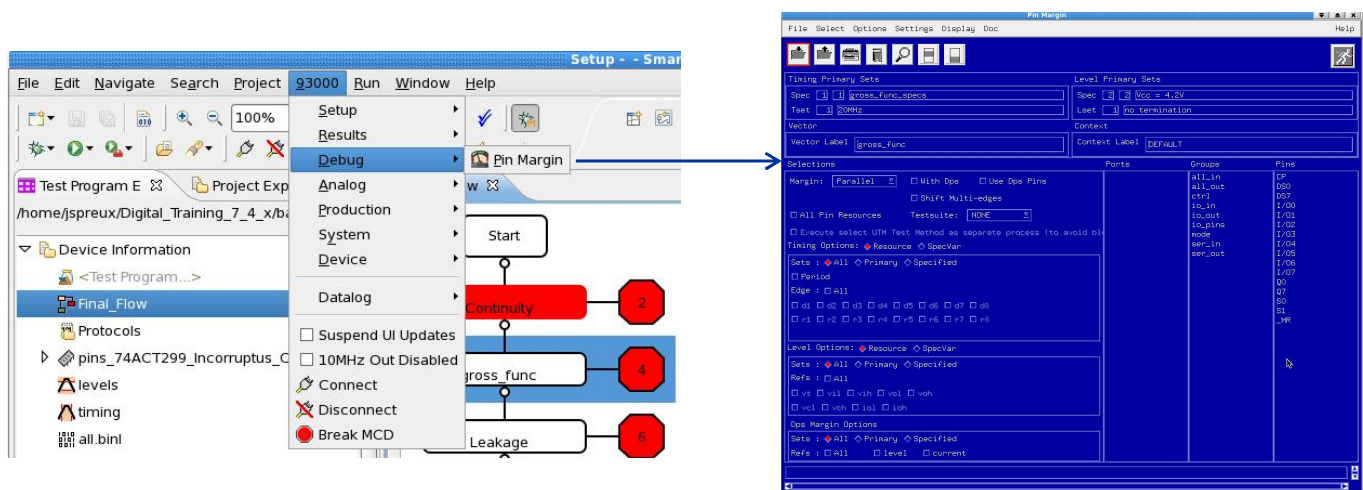
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Pin Margin Tool Unit 22 - 5

Access



- Measures the parameter margins of your DUT/test
- These margins are measured by executing one dimensional shmoos on a normally passing functional test

ADVANTEST

April 2016

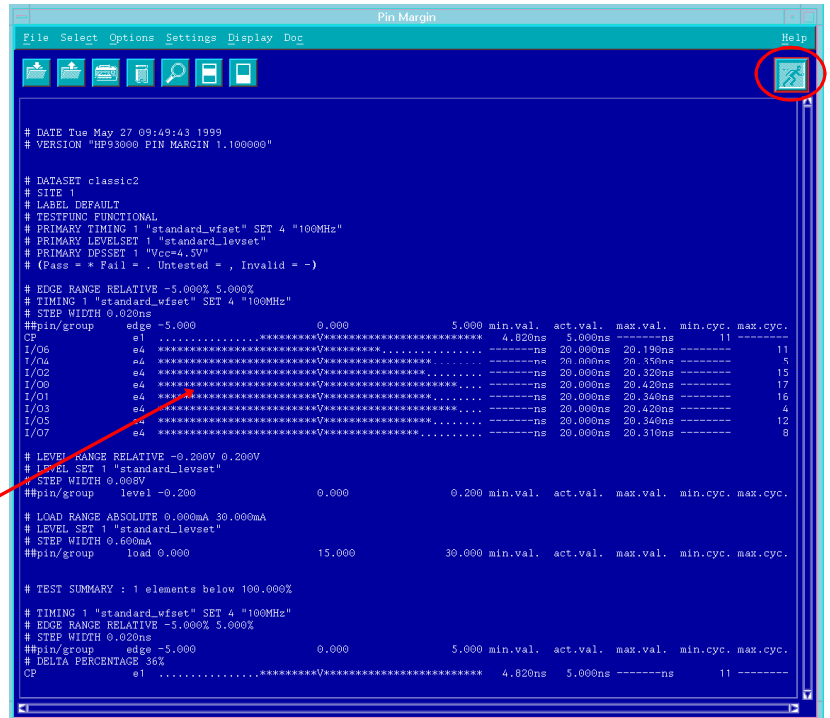
All Rights Reserved - ADVANTEST CORPORATION

Pin Margin Tool Unit 22 - 6

Setup, Execution, and Results

1. Launch Pin Margin Tool
2. Execute Testsuite to load primary sets
3. Set parameters in Pin Margin window
4. Execute

Results appear here



```
# DATE Tue May 27 09:49:43 1999
# VERSION "HP93000 PIN MARGIN 1.100000"

# DATASET classic2
# SITE 1
# LABEL DEFAULT
# TESTFUNC FUNCTIONAL
# PRIMARY TIMING 1 "standard_wfset" SET 4 "100MHz"
# PRIMARY LEVELSET 1 "standard_levset"
# PRIMARY DPSET 1 "Vcc=4.5V"
# (Pass = * Fail = . Untested = , Invalid = -)

# EDGE RANGE RELATIVE -5.000% 5.000%
# TIMING 1 "standard_wfset" SET 4 "100MHz"
# STEP WIDTH 0.020ns
##pin/group edge -5.000 0.000 5.000 min.val. act.val. max.val. min.cyc. max.cyc.
CP e1 .....V..... 4.820ns 5.000ns -----ns 11 -----
I/O6 e4 .....V..... 20.000ns 20.190ns -----ns 11 -----
I/O4 e4 .....V..... 20.000ns 20.350ns -----ns 5 -----
I/O2 e4 .....V..... 20.000ns 20.320ns -----ns 15 -----
I/O0 e4 .....V..... 20.000ns 20.420ns -----ns 17 -----
I/O1 e4 .....V..... 20.000ns 20.340ns -----ns 16 -----
I/O3 e4 .....V..... 20.000ns 20.420ns -----ns 4 -----
I/O5 e4 .....V..... 20.000ns 20.340ns -----ns 12 -----
I/O7 e4 .....V..... 20.000ns 20.310ns -----ns 8 -----

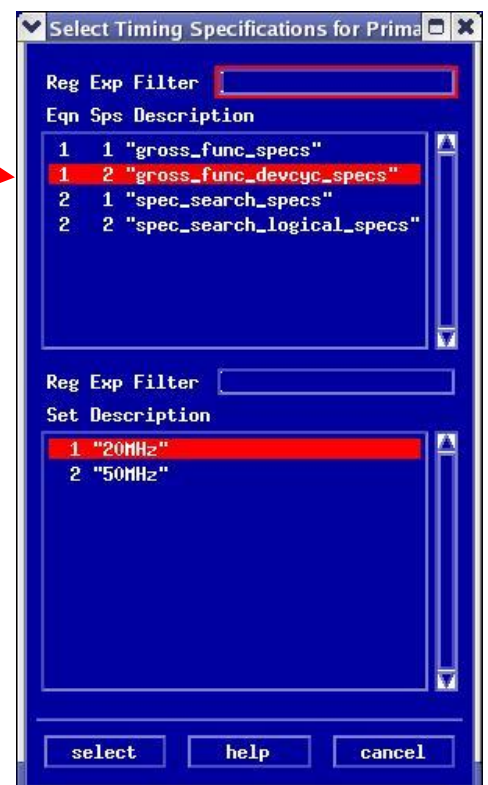
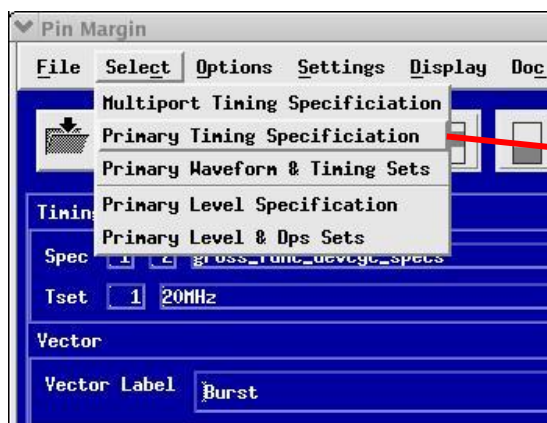
# LEVEL RANGE RELATIVE -0.200V 0.200V
# LEVEL SET 1 "standard_levset"
# STEP WIDTH 0.008V
##pin/group level -0.200 0.000 0.200 min.val. act.val. max.val. min.cyc. max.cyc.

# LOAD RANGE ABSOLUTE 0.000mA 30.000mA
# LEVEL SET 1 "standard_levset"
# STEP WIDTH 0.600mA
##pin/group load 0.000 15.000 30.000 min.val. act.val. max.val. min.cyc. max.cyc.

# TEST SUMMARY : 1 elements below 100.000%
# TIMING 1 "standard_wfset" SET 4 "100MHz"
# EDGE RANGE RELATIVE -5.000% 5.000%
# STEP WIDTH 0.020ns
##pin/group edge -5.000 0.000 5.000 min.val. act.val. max.val. min.cyc. max.cyc.
# DELTA PERCENTAGE 36%
CP e1 .....V..... 4.820ns 5.000ns -----ns 11 -----
```

ADVANTEST

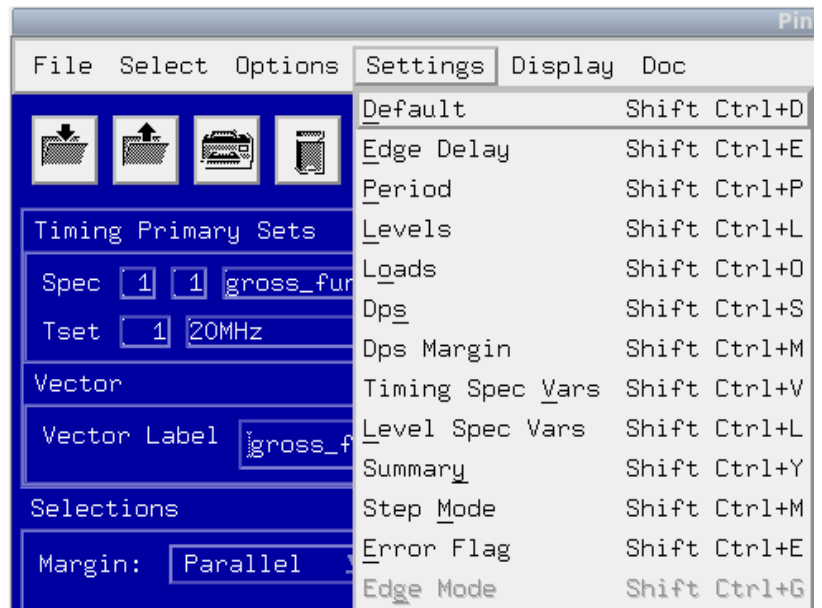
Selecting Primary Sets



ADVANTEST

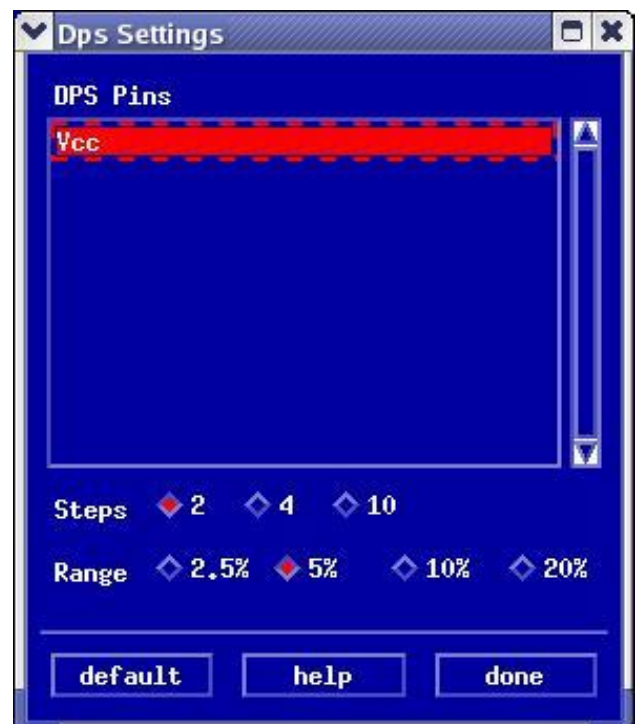
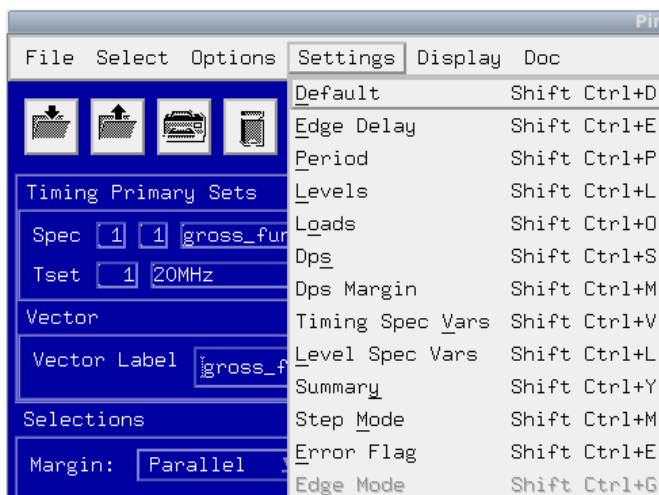
Selectable Parameters

- Tester period
- Edge delays
- Driver levels
- Receive thresholds
- Clamp voltages
- Load currents
- Termination voltages
- DPS voltages
- DPS currents
- Spec variables



ADVANTEST

Setup Example: DPS Pins



ADVANTEST

Tester Resources vs. Spec Variables

Selections	Ports	Groups	Pins
Margin: Parallel ☐ With Dps ☐ Use Dps Pins		all_in	CP
☐ Differential Mode ☐ Shift Multi-edges		all_out	DS0
☒ All Pin Resources		ctrl	DS7
Timing Options: ☒ Resource ☐ SpecVar		io_in	I/00
Sets : ☒ All ☐ Primary ☐ Specified		io_out	I/01
☐ Period		io_pins	I/02
Edge : ☒ All		node	I/03
☒ d1 ☒ d2 ☒ d3 ☐ d4 ☐ d5 ☐ d6 ☐ d7 ☐ d8		ser_in	I/04
☒ r1 ☐ r2 ☐ r3 ☐ r4 ☐ r5 ☐ r6 ☐ r7 ☐ r8		ser_out	I/05
			I/06
			I/07
			Q0
			Q7
			S0
			S1
			...

Selections	Ports	Groups	Pins
Margin: Parallel ☐ With Dps ☐ Use Dps Pins		all_in	CP
☐ Differential Mode ☐ Shift Multi-edges		all_out	DS0
☒ All Pin Resources		ctrl	DS7
Timing Options: ☐ Resource ☒ SpecVar		io_in	I/00
☐ All Specs		io_out	I/01
th_CP_I0x		io_pins	I/02
th_CP_Sx		node	I/03
tpd_CP_I0x		ser_in	I/04
tpd_CP_Qx		ser_out	I/05
trec_MR_CP			I/06
			I/07
			Q0
			Q7

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Pin Margin Tool Unit 22 - 11

Execution Options

Using the GUI

1. Set up the parameters
2. Click on  to run the test

Using a Control File

1. Start SmarTest, (load the setups if not already defined in the control file)
2. From command line prompt, type:
`/opt/hp93000/soc/pws/bin/pm -c <control file> -o <report file>`

ADVANTEST

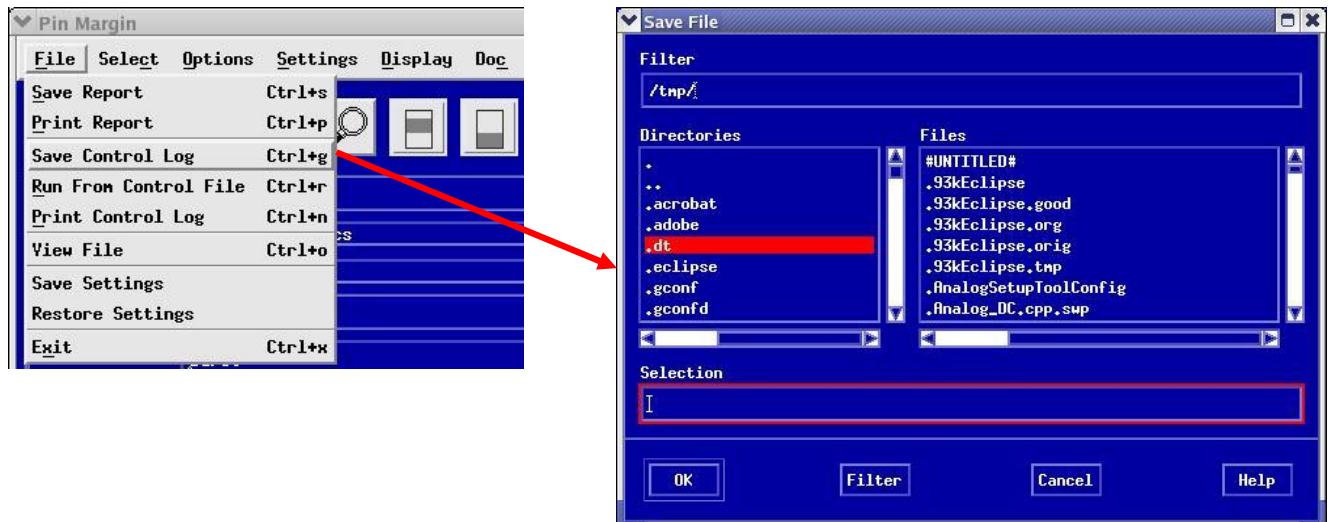
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Pin Margin Tool Unit 22 - 12

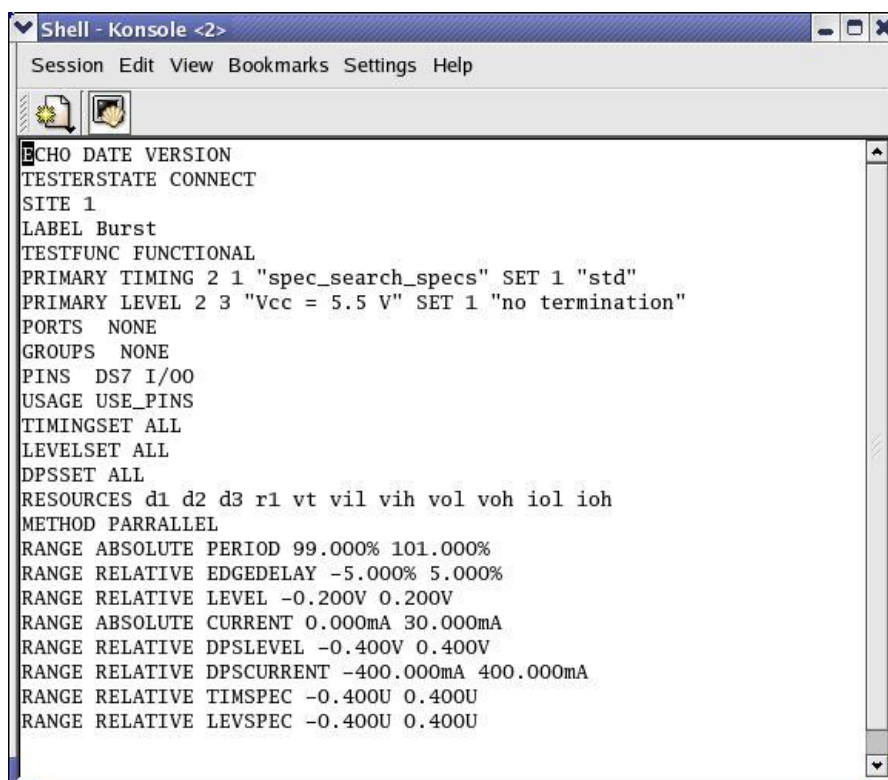
Control Log File

- All steps and settings can be saved as a control log file to repeat a specific sequence of tests



ADVANTEST

Control Log File Contents



ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):

Topic 14883

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=14883>

Lab Exercise

- Pin Margin Tool

Advantest V93000

Digital User Training

X-Mode Introduction

ADVANTEST[®]

Agenda

- Overview
- Concept of X-Modes
- Usage of X-Mode
- Higher data speed
- Vector compression
- X1 Example
- X2 Example

ADVANTEST[®]

Learning Objectives

- Understand the concept of X-mode, and its usage
- Understand the basic terms related to x-mode

Relevance

Reason for using X-modes:

- Reduce the vector memory
- Increase the operating frequency

Overview

What is X-modes?

- X-Modes is the process of combining several device cycles into a single tester cycle
- The benefit can be either faster data rates or higher data compression

X-Modes concept

Reasons to Use X-Modes

Enables Vector compression

Your designers have given you 60 MV of scan vectors, plus 10 MV of functional vectors

- You may “only” have 54 MV (million vector) pin cards
- Each vector can hold 3 bits of data with X3 wavetable
- $54 * 3 = 162$ million bits of data on a 54 MV Pin Scale card

High data rates

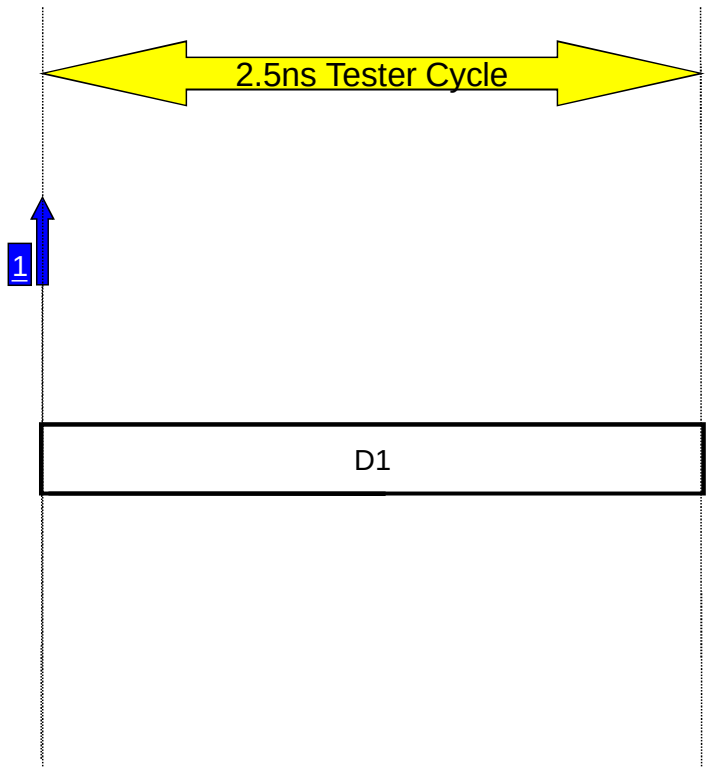
- You need to test at high-speed, e.g. at 3.6 Gbit/s
- Pin Scale 3600 executes one vector every 2.2 ns
- If each vector contains one bit of data, this equals “only” 450 Mbit/s

Normal Data (DNRZ) (Using PS1600 as example)

Define Tester Period
e.g. 2.5ns (400 MHz Base Rate)

Define Drive Edge Timing
e.g. 0 ns

Define Drive Actions
e.g. DNRZ Drive Data



Data Rate = $1/2.5\text{ns} = 400\text{Mbps}$

Only 1 Drive Edges used!!

ADVANTEST

High Speed Data, using X-Modes

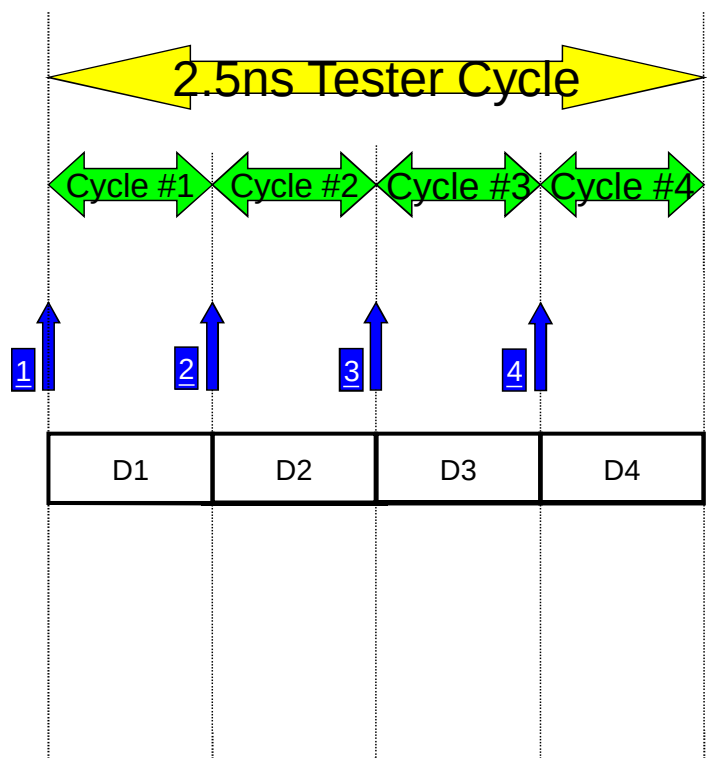
E.g. Use X4 Mode to generate a
1600Mbps on a single pin

Define Tester Period
e.g. 2.5ns (400 MHz Base Rate)

Define X-Mode
e.g. X4 Mode, 0.625ns,
1600Mbps

Define Drive Edge Timing
e.g. 0ns & 0.625ns ...

Define Drive Actions
e.g. DNRZ Drive Data



4x Speed by using 4 drive
Edges!!

ADVANTEST

How about using 8-Edges?

Refer to TDC topic 126230:

Driver	
AC performance ¹⁾	
Maximum transition time (10 to 90%)	
at 1.0 V swing	0.6 ns
at 1.8 V swing	0.7 ns
at 3.0 V swing	0.8 ns
Minimum pulse width	
at 1.0 V swing	0.7 ns
at 1.8 V swing	0.8 ns
at 3.0 V swing	0.9 ns
¹⁾ Verification condition: half the noted voltage into 50 Ohm	

For 1V swing, minimum transition time = 0.6 ns.

From previous slides, edge to edge is separated by 0.625 ns.

If we increase to 8 edges per period, AC output swing will be less than 1V.

In summary:

- At 1.0V swing -> around 1600 Mbps
- At 3.0V swing -> around 1200 Mbps

Enable Vector Compression

What is Vector

Signal			10101 (DVC)	10102 (DVC)
Size: 24 bytes				
X-Mode Area				
Protocol				
Vector#	Instruction	Comment		
0			0	L
1			1	H
2			0	L
3			1	H
4			0	L
5			1	H
6			0	L
7			1	H
8			0	L
9			1	H
10			0	L
11			1	H
12			0	L
13			1	H
14			0	L
15			1	H
16			0	L
17			1	H
18			0	L
19			1	H
STOP				

An array of physical index, defined in wavetable

Number of configurable drive edges per pin = 8

Number of configurable receive edges per pin = 8

Number of physical index = 256

X-mode -> Maximize configurable resource

```
WAVETBL X1_WTB
PINS 10101_10102
0 "d1:0" 0
1 "d1:1" 1
2 "r1:L" L
3 "r1:H" H
4 "r1:X" X
brk "
```

ADVANTEST

Example: X1 Pattern

```
EQNSSET 3 "x1_timing"
SPECS
Freq [MHz]
E6
EQUATIONS
t = 1000/Freq
Freq_Hz = Freq*1000000
TIMINGSET 1
period= fract(1000,Freq_Hz,E6)
PINS 10101 10102
d1 = 0
r1 = t/2
```

Name	Actual	Minimum	Maximum	Measured	Unit	Comment
Freq	19.23				MHz	
E6	100000					

Upload from file done!

19.23MHz Frequency

1 Drive and 1 Receive Edge

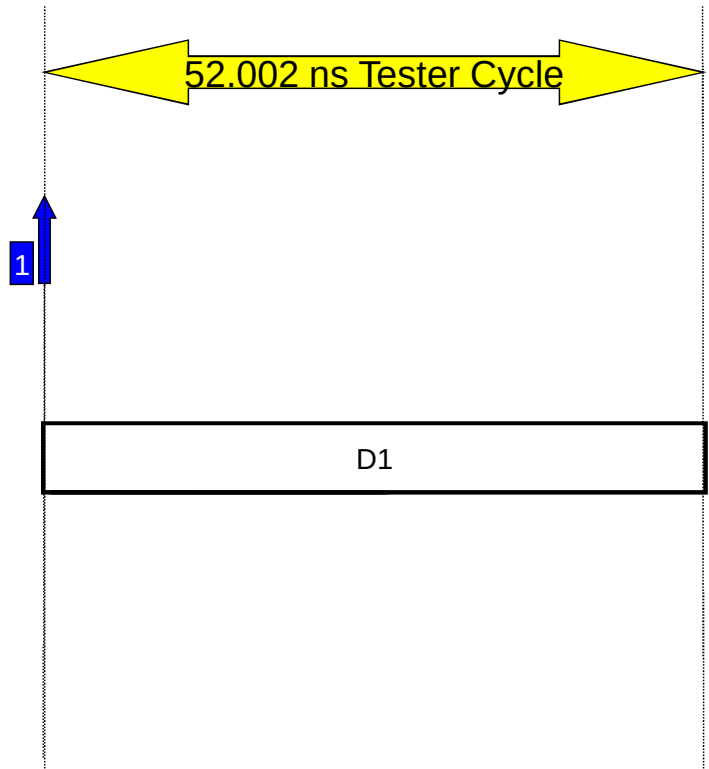
ADVANTEST

X1 Example

Define Tester Period
e.g. 52.002ns (19.23 MHz)

Define Drive Edge Timing
e.g. 0 ns

Define Drive Actions
e.g. DNRZ Drive Data



Clock Rate = 19.23MHz

Only 1 Drive Edges used!!

ADVANTEST

High Speed Data, using X-Modes

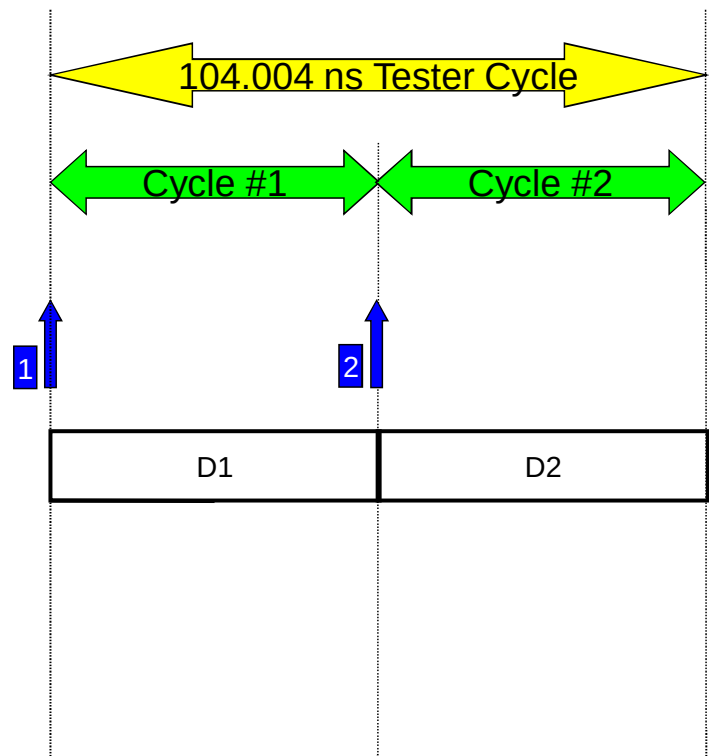
E.g. Use X4 Mode to generate a
1600Mbps on a single pin

Define Tester Period
e.g. 104.004ns (9.615 MHz)

Define X-Mode
e.g. X2 Mode

Define Drive Edge Timing
e.g. 0ns & 52.002 ns ...

Define Drive Actions
e.g. DNRZ Drive Data



2 data in 104.004ns

Data Rate = $2/104.004$

= 19.23MHz!

2 Edges have been used

ADVANTEST

X2 Mode Wavetable

```
WAVETBL X2_WTB
PINS 10101-
0 "d1:0 d2:0" 00
1 "d1:0 d2:1" 01
2 "d1:1 d2:0" 10
3 "d1:1 d2:1" 11
brk ""

STATEMAP
#physSeq    statechars    x-mode    selector
0           00           x2        ts1
1           01           x2        ts1
2           10           x2        ts1
3           11           x2        ts1

PINS 10102
0 "d1:Z r1:X r2:X" XX
1 "d1:Z r1:X r2:L" XL
2 "d1:Z r1:X r2:H" XH
3 "d1:Z r1:L r2:X" LX
4 "d1:Z r1:L r2:L" LL
5 "d1:Z r1:L r2:H" LH
6 "d1:Z r1:H r2:X" HX
7 "d1:Z r1:H r2:L" HL
8 "d1:Z r1:H r2:H" HH
brk ""

STATEMAP
#physSeq    statechars    x-mode    selector
0           XX           x2        ts1
1           XL           x2        ts1
2           XH           x2        ts1
3           LX           x2        ts1
4           LL           x2        ts1
5           LH           x2        ts1
6           HX           x2        ts1
7           HL           x2        ts1
8           HH           x2        ts1
```

Separate Drive and Receive pins

Created all possible drive and receive combination

Added STATEMAP for ease of pattern viewing (see later slides)

ADVANTEST

X2-Mode Timing Setup

```
EQNSET 4 "x2_timing"

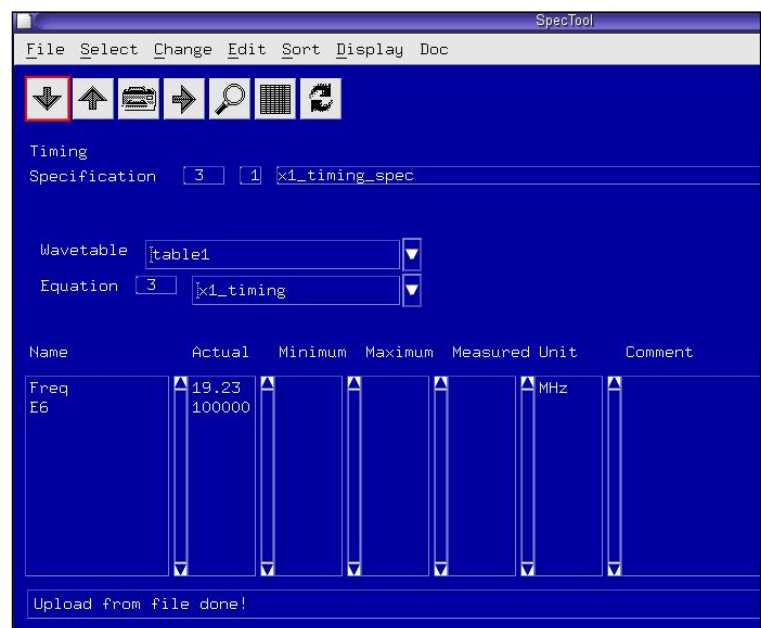
SPECS
Freq [MHz]
E6

EQUATIONS

t = 1000/Freq
Freq_Hz = Freq*1000000

TIMINGSET 1
period= fract(2000,Freq_Hz,E6)

PINS 10101 10102
d1 = 0
d2 = t
r1 = (1*t)/2
r2 = (3*t)/2
```



No change in Spec Variable, 2x period has been handled through “period = fract (2000,Freq_Hz,E6)”.

This make the spec_search task easier to understand

Added in newly added Edges “d2” and “r2” timing definition

ADVANTEST

X2-Mode Pattern (vs X1-Mode Pattern)

X1-pattern

Signal Size: 24 bytes

Vector#	Comment	10101 (DVC)	10102 (DVC)
0		0 L	
1		1 H	
2		0 L	
3		1 H	
4		0 L	
5		1 H	
6		0 L	
7		1 H	
8		0 L	
9		1 H	
10		0 L	
11		1 H	
12		0 L	
13		1 H	
14		0 L	
15		1 H	
16		0 L	
17		1 H	
18		0 L	
19		1 H	

***testflow X2-pattern**

Signal Size: 16 bytes

Vector#	Instruction	Comment	10101 (DVC)	10102 (DVC)
0			01 LH	
1			01 LH	
2			01 LH	
3			01 LH	
4			01 LH	
5			01 LH	
6			01 LH	
7			01 LH	
8			01 LH	
9			01 LH	
	STOP			

Vector Memory Usage reduced by half!!
(From 20 Vectors to 10 Vectors)

ADVANTEST

Summary

- X-Modes is the process of combining several device cycles into a single tester cycle
- The benefit can be either higher data rates or vector compression

ADVANTEST

Reference

V93000 Technical Documentation Center (TDC):

Topic 100339

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=100339>

Advantest V93000

Digital User Training

Multiport

ADVANTEST[®]

Learning Objectives

- Multiport concept and application
- Multiple clock domain application

ADVANTEST[®]

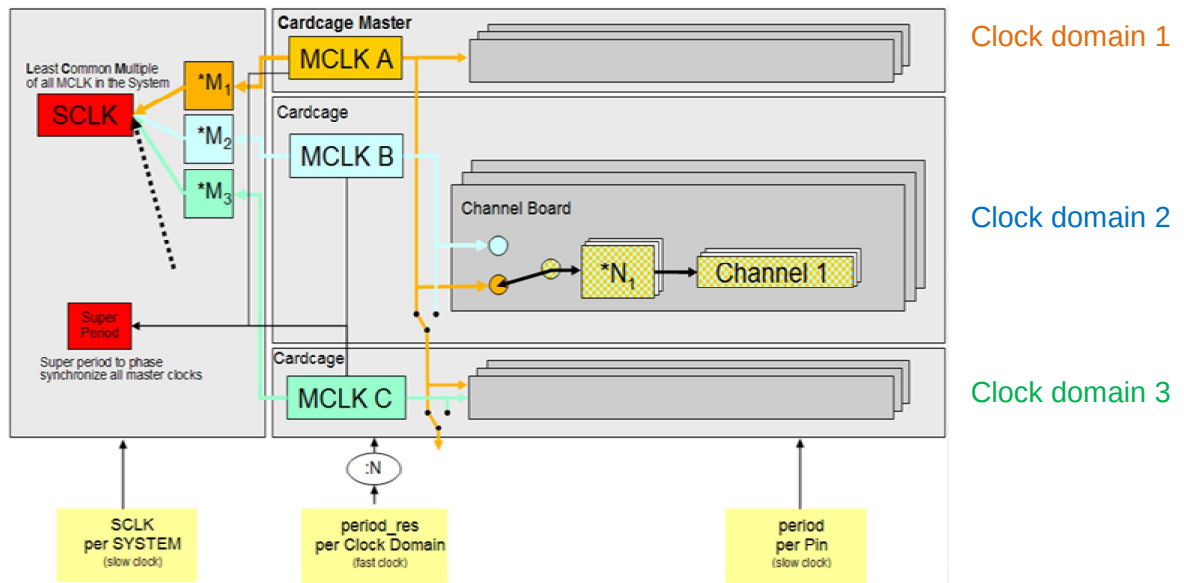
Prerequisites

- Basic knowledge of V93000
- Basic project development experience of V93000 - Digital / DC / Mixed-signal, Offline & Online
- V93000 SOC Basic User Training Part 1

Agenda

- Multiport Definition
- Main Applications of Multiport
 - Fractional Bus Test: Multiport vs. Multiple Clock Domain
- Multiport Setup

Pin Scale Clocking System



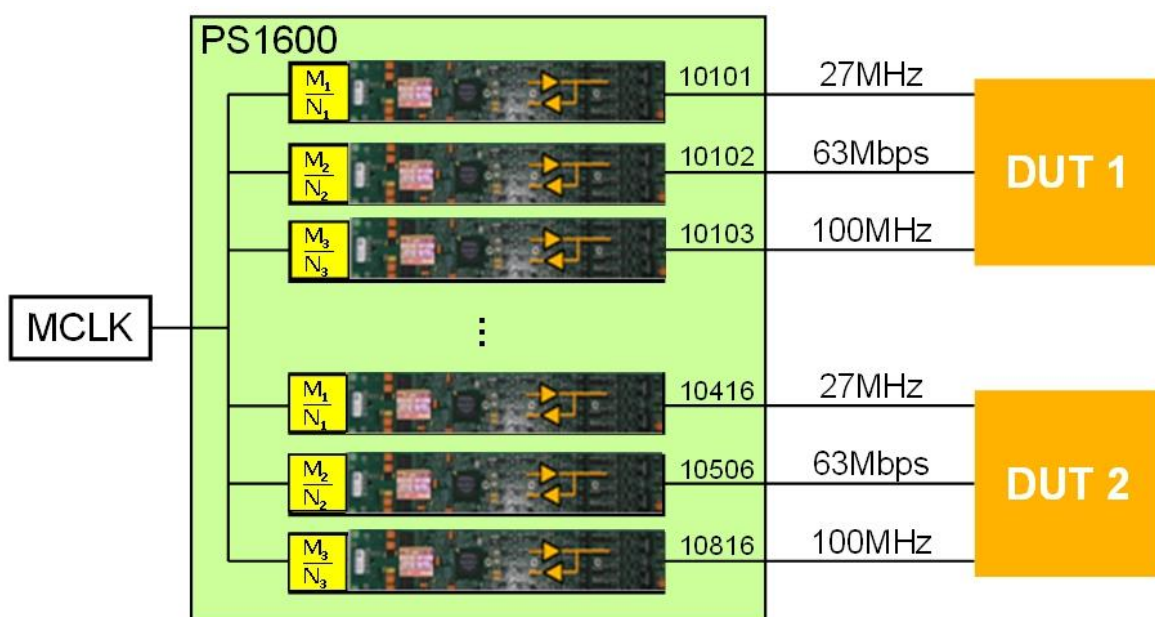
SCLK: The system clock (SCLK) is the reference clock which coordinates the operation of the test system channels and is also responsible to start the pattern generation of the individual channels.

MCLK: Master Clock: local clock - one per card cage.

Super Period: internal clock used to ensure that all master clocks are started synchronously so that a certain phase relationship can be assured.

ADVANTEST

Smart Scale Clocking System – Clock Domain Per Pin



Every SmartScale channel has it own PLL, combined with the local master clock è **Independent clock domain per pin**

ADVANTEST

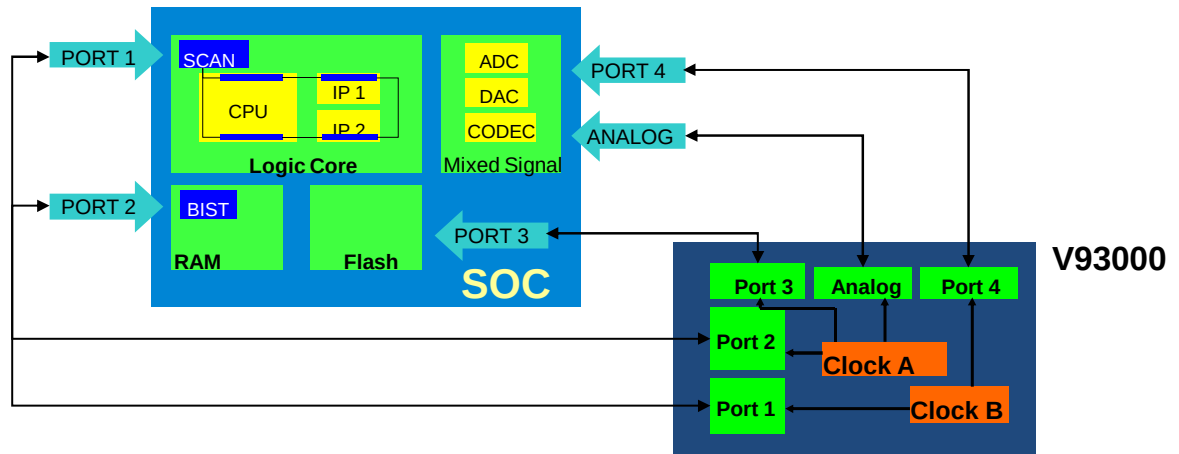
Multiport Definition

Port

- The segmentation of a test program into pin groups corresponding to device cores and/or common clock settings

Multiport

- Segmentation of the device pins into test execution of various timing and/or various sequencer settings for parallel test operation



Test Processor-per-Pin Architecture

ADVANTEST

Fractional Bus Test

Devices with ports running at different speeds (for example fractional buses), can utilize multiport to run buses with different periods at their natural speeds

Port	Speed
REF (reference clock)	26MHz
DIG (all other digital pins)	10MHz

• Multiple Continuous Clock Setup

- § For testing of devices that require constantly running clocks or signals of a clocking action during the serial execution of multiple Testsuites

ADVANTEST

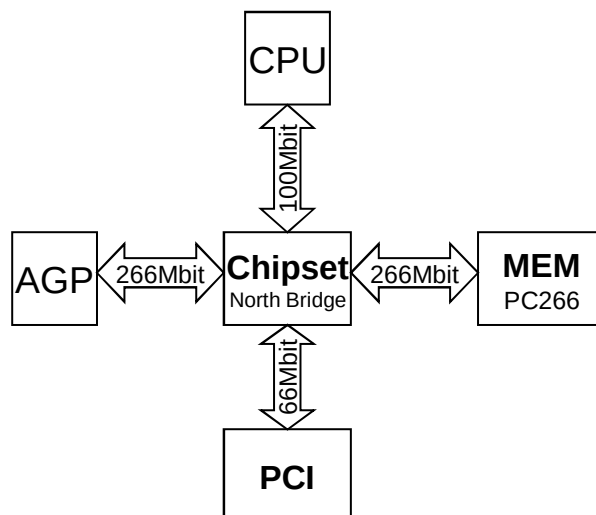
Fractional Bus Test Example -1

PC Chipset

V93000 Test System - Non Multiport

Notes:

- All pins run at same period: 7.5ns
- CPU needs X3 mode, then X1/4 mode
- PCI needs X1/2 mode



Non Multiport	CPU	AGP	MEMORY	PCI
Frequency	100MHz	266MHz	266MHz	66MHz
Required Period	10ns	3.75ns	3.75ns	15ns
Actual Period	7.5ns			
Xmode	first X3 mode, then X1/4mode	X2	X2	X1/2
Clock Domain (Period resolution)	7.5ns			

ADVANTEST

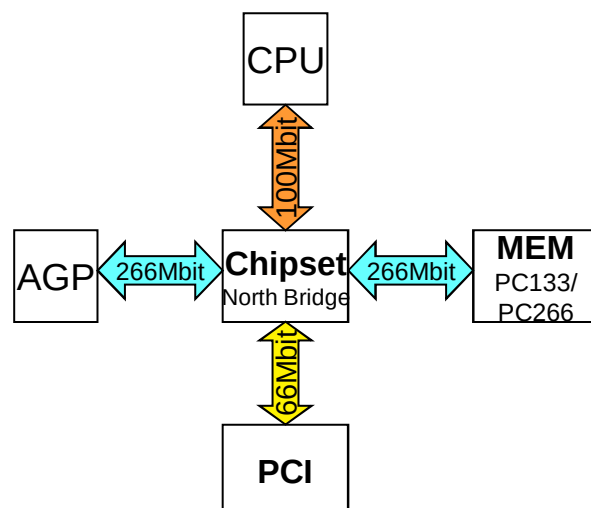
Fractional Bus Test Example -2

PC Chipset

V93000 Test System - Multiport

Notes:

- Different ports run at different speeds
- CPU only needs X3 mode
- PCI does not need X1/2 mode



Multiport	CPU	AGP	MEMORY	PCI
Frequency	100MHz	266MHz	266MHz	66MHz
Required Period	10ns	3.75ns	3.75ns	15ns
Actual Port Period	30ns	7.5ns	7.5ns	15ns
Xmode	X3	X2	X2	X1
Clock Domain (Period resolution)	7.5ns			

ADVANTEST

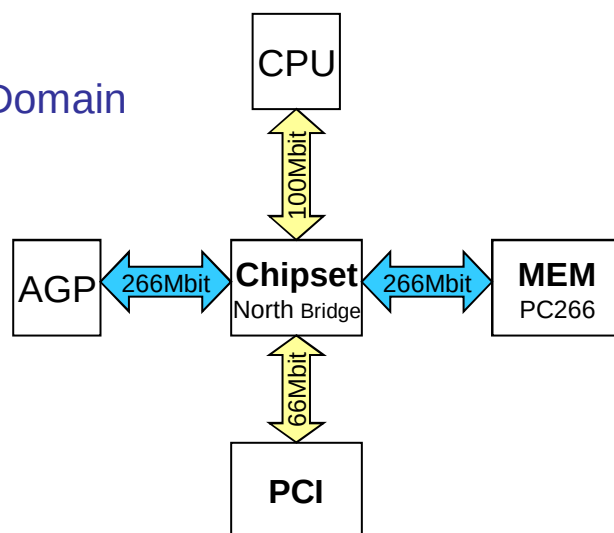
Fractional Bus Test Example -3

PC Chipset

V93000 Test System - Multiple Clock Domain

Notes:

- Different ports run at different speeds
- Different ports run at different domains (period resolutions)
- CPU does not need X3 mode



Multi Clock Domain	CPU	AGP	MEMORY	PCI
Frequency	100MHz	266MHz	266MHz	66MHz
Required Period	10ns	3.75ns	3.75ns	15ns
Actual Port Period	10ns	3.75ns	3.75ns	15ns
Xmode	X1	X1	X1	X1
Clock Domain	5ns	3.75ns	3.75ns	5ns

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 11

Multiple Clock Domain

Background of Multiple clock domain

- Default setup:

$$\text{portA} = x * \text{period_res} \text{ and } \text{portB} = x * \text{period_res}$$

$$\text{portA} = 3 * 5\text{ns} = 15\text{ns} \quad \text{portB} = 3 * 5\text{ns} = 15\text{ns}$$

- Multiport with a single clock domain (access to N):

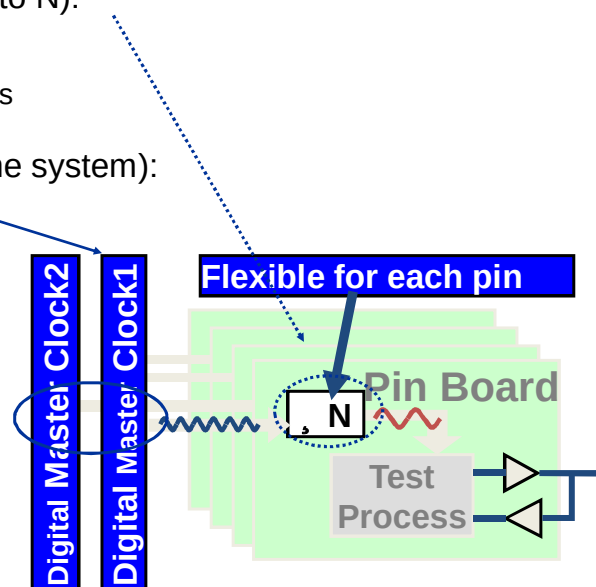
$$\text{portA} = x * \text{period_res} \text{ and } \text{portB} = y * \text{period_res}$$

$$\text{portA} = 3 * 5\text{ns} = 15\text{ns} \quad \text{portB} = 4 * 5\text{ns} = 20\text{ns}$$

- Multiple Clock domains (multiple MCLK on the system):

$$\text{portA} = x * \text{period_res1} \text{ and } \text{portB} = y * \text{period_res2}$$

$$\text{portA} = 3 * 5\text{ns} = 15\text{ns} \quad \text{portB} = 4 * 6\text{ns} = 24\text{ns}$$



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 12

Multiport vs. Multiple Clock Domain

- Multiport does not impact load board design and it is available on V93000 Pin Scale system and SmartScale system
- Multiple Clock Domains are only available on V93000 Pin Scale and Smart Scale system. It is a hardware option and it requires multiport function
- Proper assignment of tester clock domains (HW) in a test program (SW) can only be done with good load board design*

* SmartScale provides a fractional PLL per pin and thus, has a clock domain per pin. Compared to Pin Scale system, you don't need to follow strict load board assignments anymore.

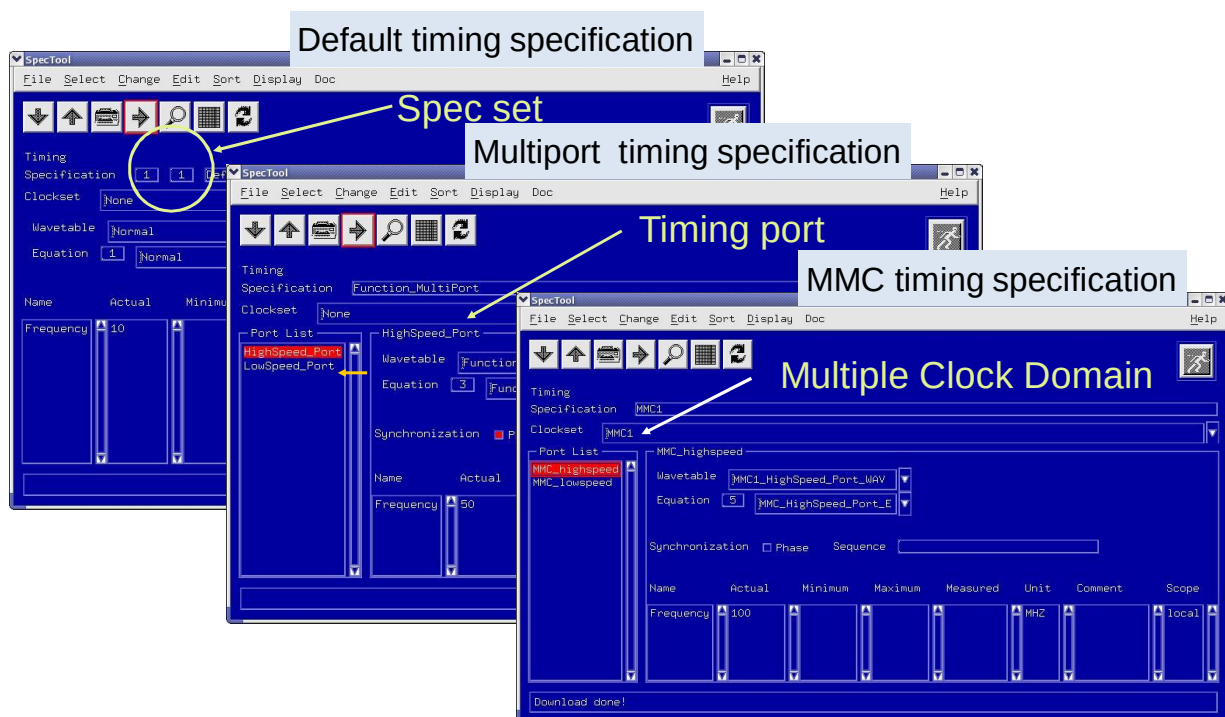
ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 13

Setup of Multiport and MCLK



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 14

Port Properties

Ports organize the device into groups of pins that are tested independently and can be driven by separate sequencer programs

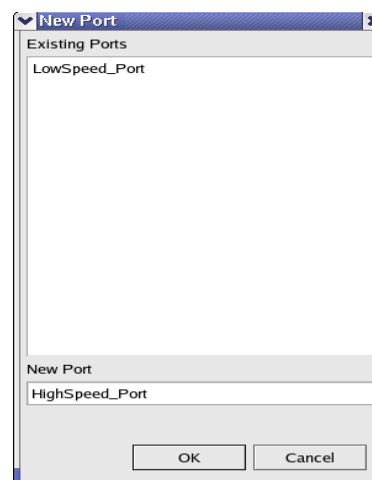
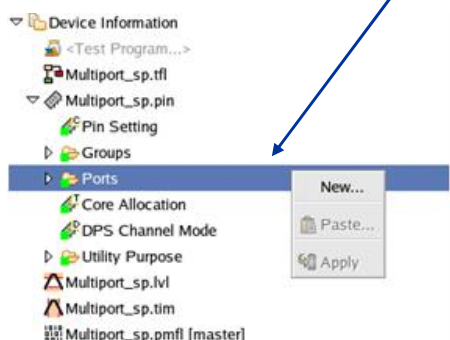
Properties of port:

- A port must not have the same name as a pin or a pin group
- Expression-based port definitions are not supported
- Ports are automatically pin groups, and you can use ports as pin groups in expression based pin group definitions
- Pins connected to non-digital instruments such as analog and DPS channels can not be assigned to ports
- The maximum number of ports is 64(63 user defined plus the @ port)

Port Assignment (1)

Right click *Ports* from the *Pin Setting* of Test Program Explorer

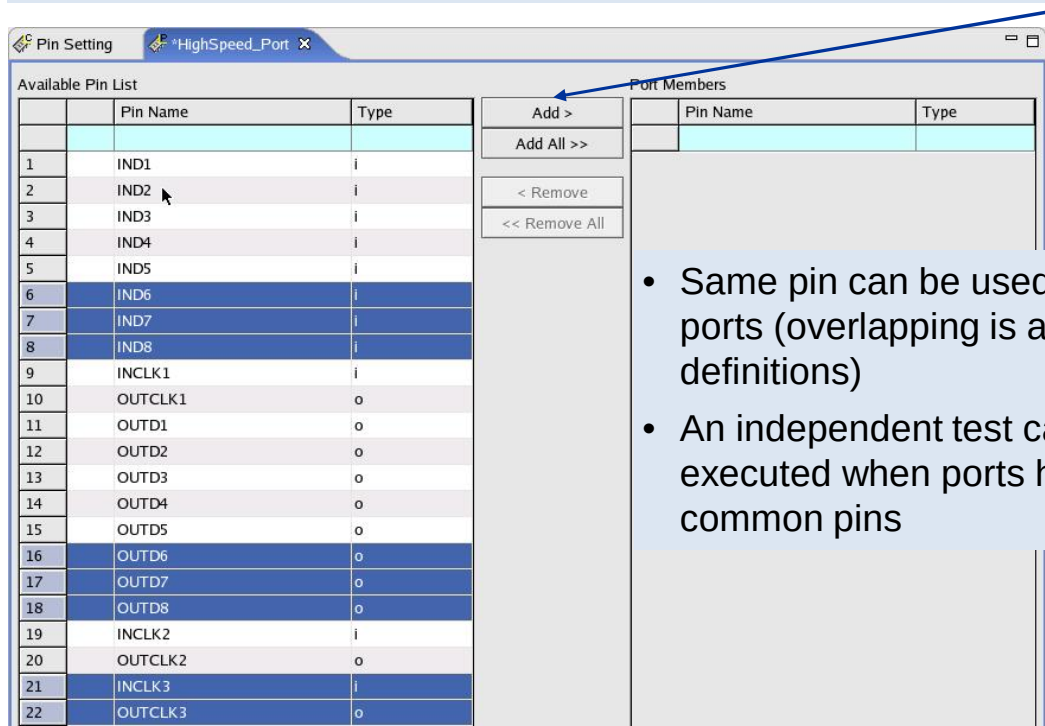
Click New to define port



Enter the name of the port

Port Assignment (2)

Select the pins to be included in current port and click *Add>*



- Same pin can be used in multiple ports (overlapping is allowed in port definitions)
- An independent test can only be executed when ports have no common pins

ADVANTEST

Port vs. Group

Firmware command DFPT is used to define ports, DFGRP is used to define group

hp93000,config,0.1

DFPN 10101,"1",(IND1)

...

DFPT (IND6,IND7,IND8,OUTD6,OUTD7,OUTD8,INCLK3,OUTCLK3),(HighSpeed_Port)

DFGRP 1,(IND1,IND2,IND3,IND4,IND5,IND6,IND7,IND8),(input_data_pins)

Port is a superset of group

ADVANTEST

Single-Port vs. Multiport Timing Setup

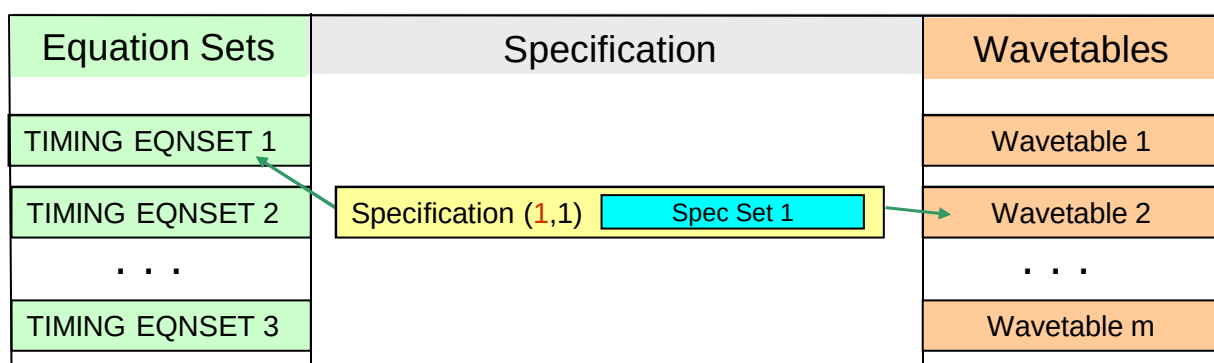
Same

- Syntax for equation sets definition
- Syntax for wavetables definition

Difference

- Multiport setup only works with equation-based timing
- Port definition needs to be added in equation set and wavetable
- Multiport Specification

Equation-Based Timing Setup Single-Port

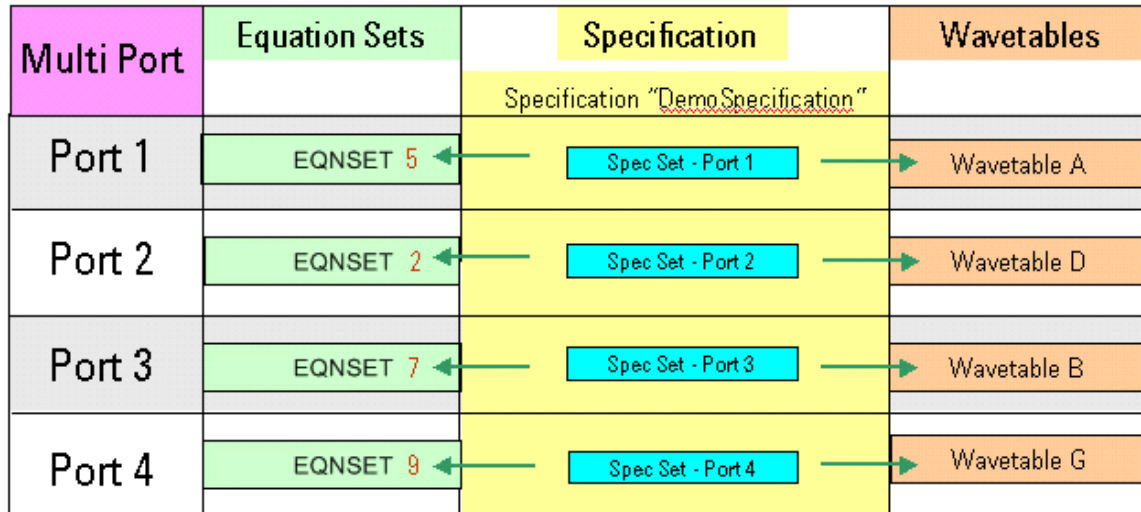


- Define wavetables
- Define equation sets (including timing sets)
- Define specifications to combine a wavetable and an equation set and to specify values for the spec variables declared in the equation set

Equation-Based Timing Setup

Multiport

- The specification combines equation sets and wavetables for the port setups to an overall device setup



Multiport Timing Setup Procedure

1. Definition of port timing in equation sets
2. Definition of port timing action in the wavetables
3. Definition of specifications
4. Definition of port sequences
 - Assignment of an equation set to every port in the port sequence
 - Assignment of a wavetable to every port of the port sequence
 - Definition of a spec set for every port of the port sequence and spec scope
 - Phase and sequence synchronization group definition when needed and assignment of clock domains when multiple clocks are used

Multiport Setup

Equation

EQNSET 2 "Function1_LowSpeed_Port_EQN"

#wavetable "Function1_LowSpeed_Port_WAV"

DEFINES LowSpeed_Port

SPECS # starts the declaration of the specs

Frequency [MHZ] # NOTE: no checks are performed on the base of the unit

EQUATIONS

Tcyc=1000/Frequency

TIMINGSET 1 "functional"

period=Tcyc # Note all resource values are in ns

PINS IND1 IND2 IND3 IND4 IND5

d1 = 0

PINS INCLK1 INCLK2

d1 = 0

d2 = 0.5*Tcyc

Port assignment, optional if the definition is for @ or all ports

These equation definitions include all pins of the ports listed in the DEFINES statement

ADVANTEST

Multiport Setup

Wavetables

WAVETBL "Function1_LowSpeed_Port"

DEFINES LowSpeed_Port

PINS IND1 IND2 IND3 IND4 IND5

0 "d1:0 " 0

1 "d1:1 " 1

PINS INCLK1 INCLK2

0 "d1:0 " 0

1 "d1:1 d2:0" 1

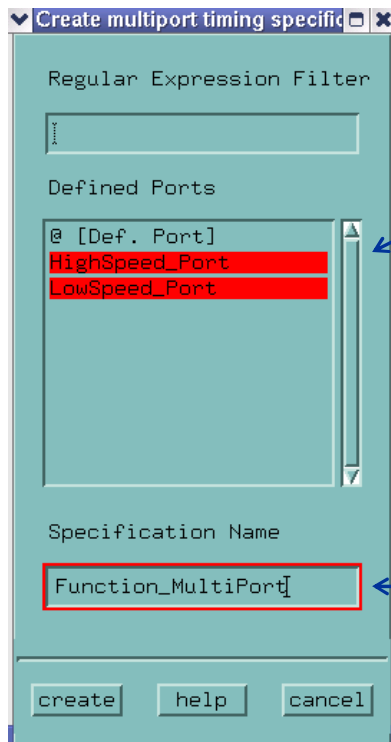
Port assignment, optional if the definition is for @ or all ports

These waveform definitions include all pins of the ports listed in the DEFINES statement

ADVANTEST

Multiport Setup

Multiport Specification



- Port assignment
- Selection of ports
- Select the ports required for this specification setting
- Overlapping ports are not allowed in this setting software checking

•Specification Name

ADVANTEST

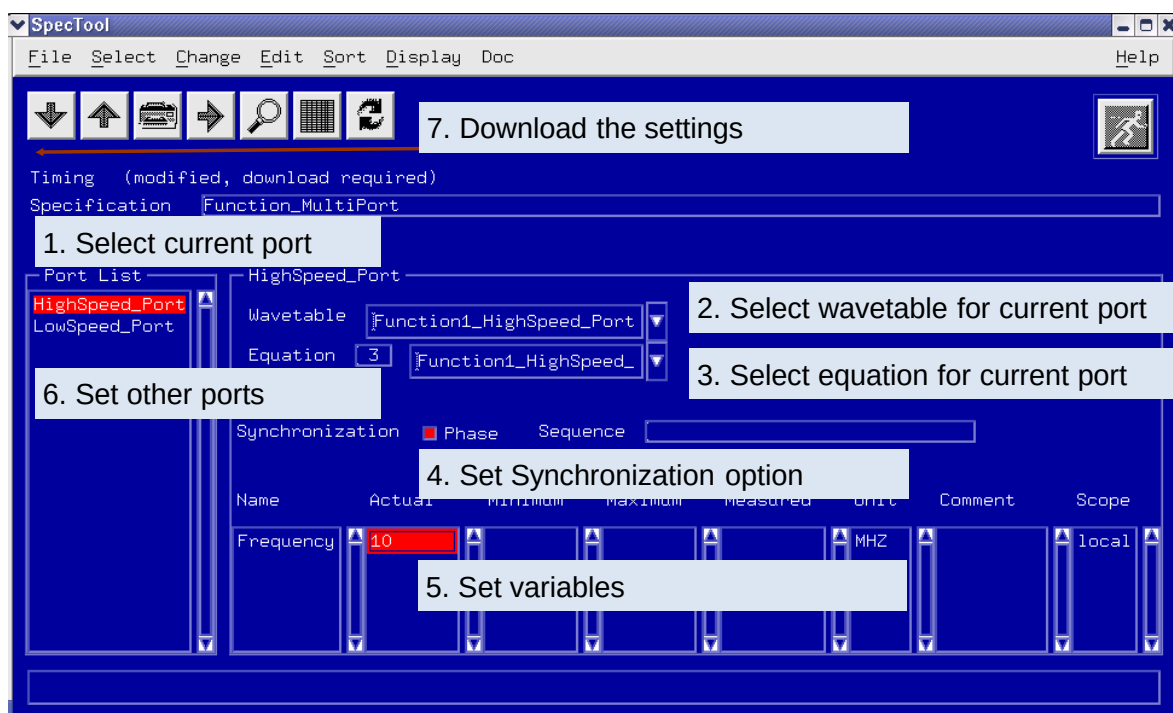
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 25

Multiport Setup

Multiport Specification



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 26

Period Resolution Setting in Multiport

The period resolution can be defined

- By SmarTest automatically based on
 - the order of the ports in the specification
- By user with
 - the definition of **period_res** statement in an equation
 - an override of the clock resolution in the digital clock domain settings

Spec Variable Scope

“Frequency” is local at this port, value changed will not affect other ports value

“Frequency” is global at this port, value changed will affect other ports value

Spec	HighSpeed_port	LowSpeed_port	PORT 1	PORT 2
Frequency	local	local	global	global
Frequency1	global	global	global	global
Frequency2	global	global	global	global
Frequency3	local	local	local	local
Frequency4	local	local	local	local
Frequency5	local	local	local	global

Spec Values Change at Run Time

Firmware Command SVLR

Changing the value of a spec variable using the SVLR command for a single port:

SVLR TIM,, "Function_Multiport", "Frequency@HighSpeed_Port", 20

Based on the variable scope option, change can be locally or globally

A global spec variable setting independent of the scope is:

SVLR TIM ,,"Function_Multiport","Frequency",20

Provided that the spec name “Frequency” is used for other ports in the specification, this setting will be global

Phase and Sequencer Synchronization

In Single-Port

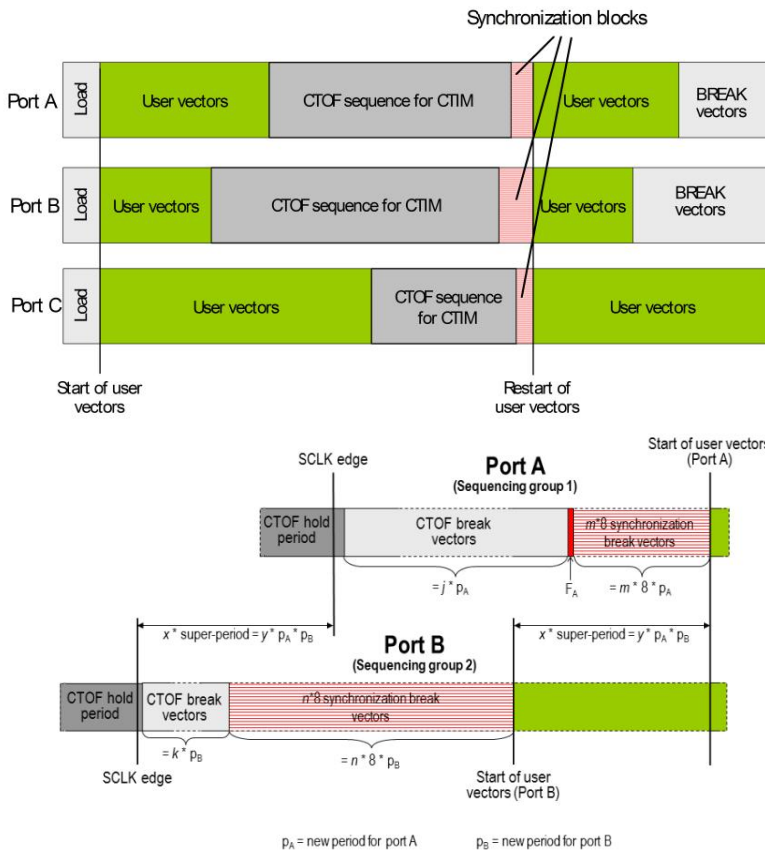
- All driving data is phase and sequencer synchronized
- Free running clocks are phase synchronized to sequencer data

In Multiport

- Synchronization is optional. There are three types of synchronization

- Phase
 - Sequence
 - Sequencer groups
- ┆ → Timing control
- Vector control

Phase and Sequencer Synchronization



Sequencer Synchronization

Port A, B, C in the same sequencing group =
they are started simultaneously after CTIM (timing change) and at a start of a functional test

Phase Synchronization

Port A, B belong to different sequencing groups
They only need to be synchronous with some clock pulse
= partial synchronization

i, k, m, n, x, y are integers.

ADVANTEST

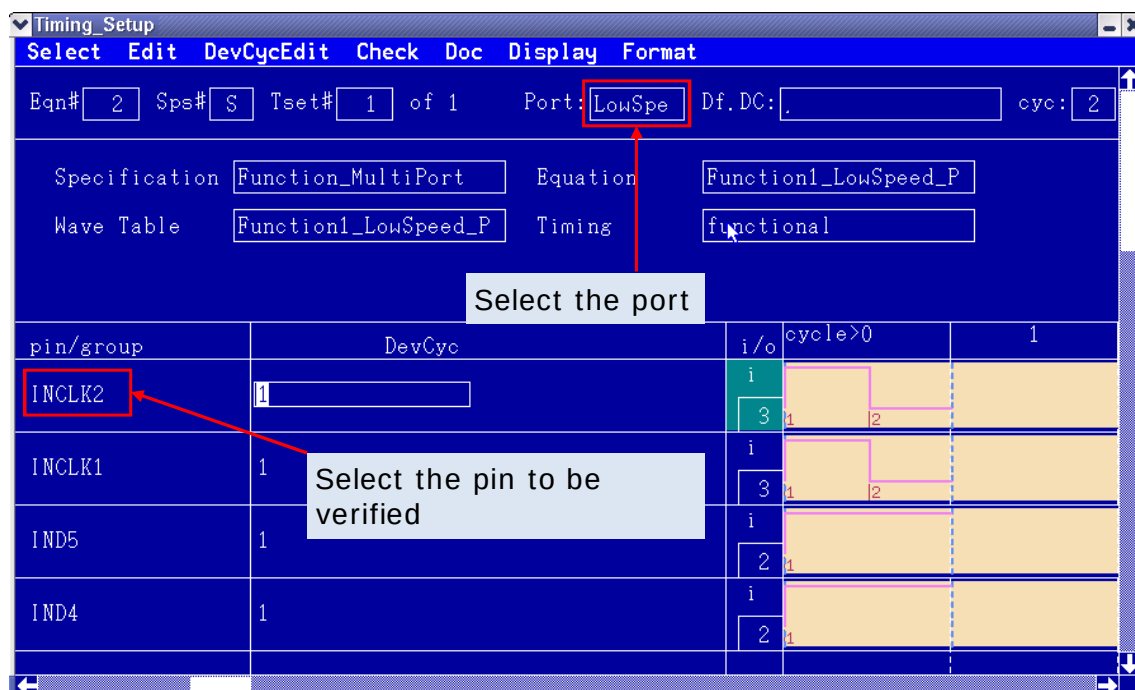
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 31

Multiport Setup

Timing Setup Verification



ADVANTEST

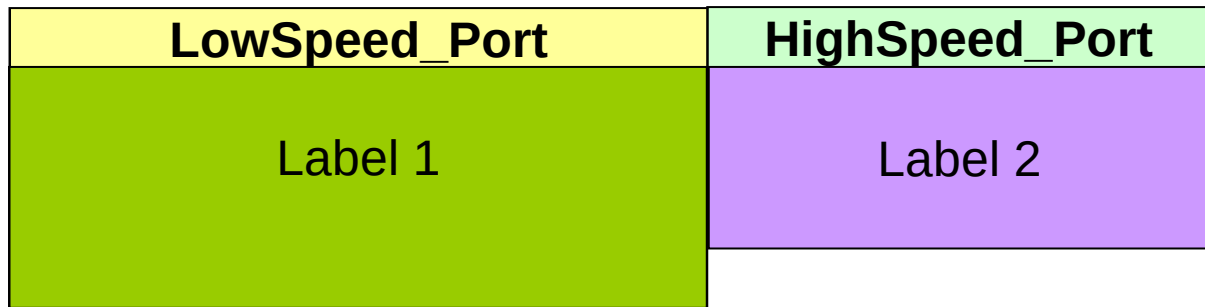
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Multi Port - Unit 24 - 32

Multiport Setup

Pattern / Vector Ports Feature



- Faster debugging by allowing selection of patterns to be loaded
- Pattern files can be stored anywhere in the file system
- Supports Multiport Bursts

Create Vector for Port

- Port vectors can be created, modified, and viewed in pattern tool in the same way as single-port vectors
- All single-port vector capabilities apply to port vectors
- Need to specify the port
- The Multiport Burst is of type “MPBurst”

Reference

V93000 Technical Documentation Center (TDC):

Topic 37515

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=37515>

Advantest V93000

Digital User Training

Test Data Conversion with SmarTest Data Link

ADVANTEST[®]

Agenda

- Learning Objectives
- Relevance
- Design Test Data Formats and Pins, Timing, Levels and Patterns
- SmarTest Data Link Overview
- Conversion of a WGL File
- Conversion of a STIL File
- Conversion of a VCD File

ADVANTEST[®]

Learning Objectives

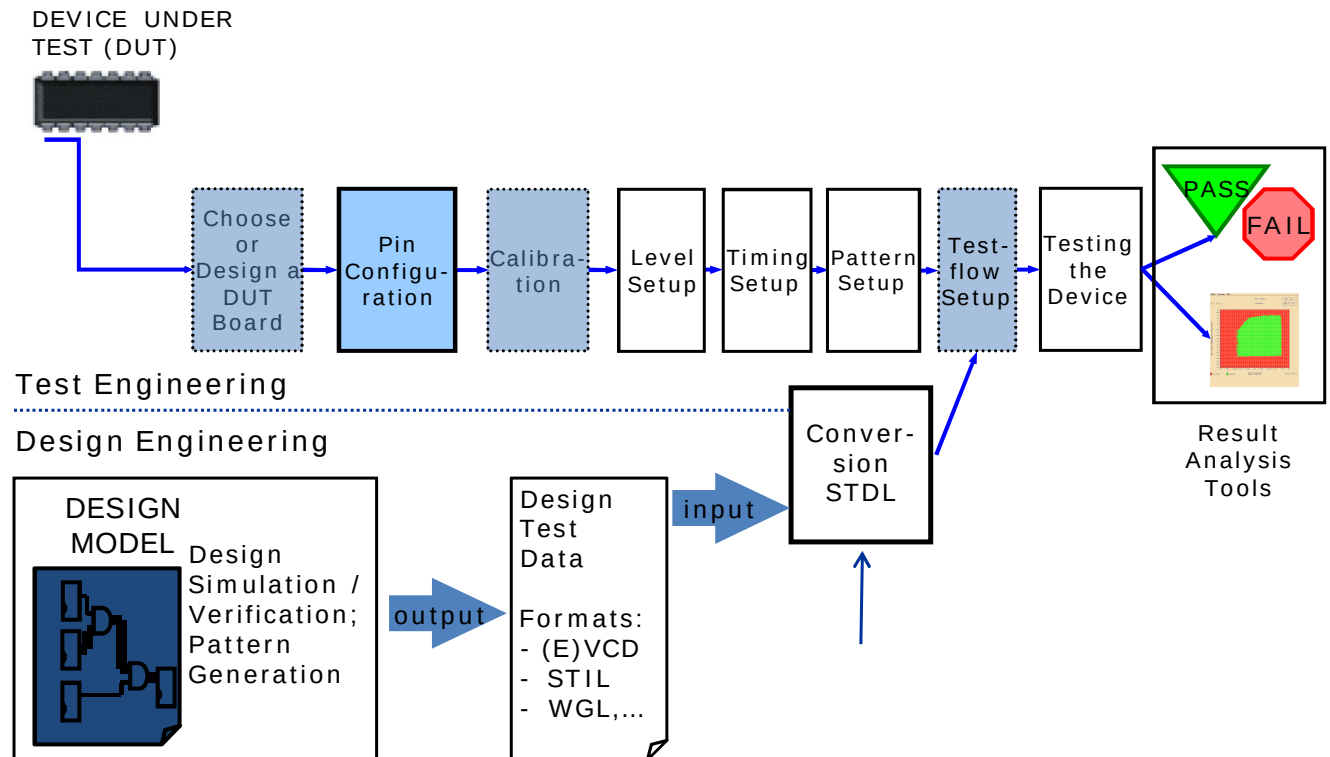
- Know the design test data formats STIL, WGL, and VCD
- Know the conversion capabilities of SmarTest Data Link (STDL)
- Know how to use SmarTest Data Link (STDL)
- Know how to cyclize test data in the event driven format VCD

Relevance

- In most cases (digital) test data for semiconductor devices is generated in conjunction with verifying device designs
- For example, test data is the output of simulating the design model of a device (e.g. functional tests) or it is the result of running a pattern generator (e.g. scan tests)
- SmarTest Data Link (STDL) converts this data to V93000 data, so that there is no need to manually configure pins, levels, timings, patterns...

Flow Chart

Test Data Conversion



ADVANTEST

Terms & Definitions

Design Test Data

- Test data that is produced by EDA (Electronic Design Automation) tools, for example via design simulation or automatic test pattern generation.
- Common formats:

- **STIL** (Standard Test Interface Language)
 - Widespread format that is used for cyclized data like scan test
 - Allows to describe pins/pin groups, ports, channels, levels, waveforms, patterns, etc.
- § **VCD** (Value Change Dump)
 - Widespread event driven format that is used to store values at DUT IOs toggling during simulation
 - In most cases it just contains timing values and signals
 - Part of Verilog
- § **WGL** (Waveform Generation Language)
 - Format that in most cases just contains waveforms, timing and signals/pins

ADVANTEST

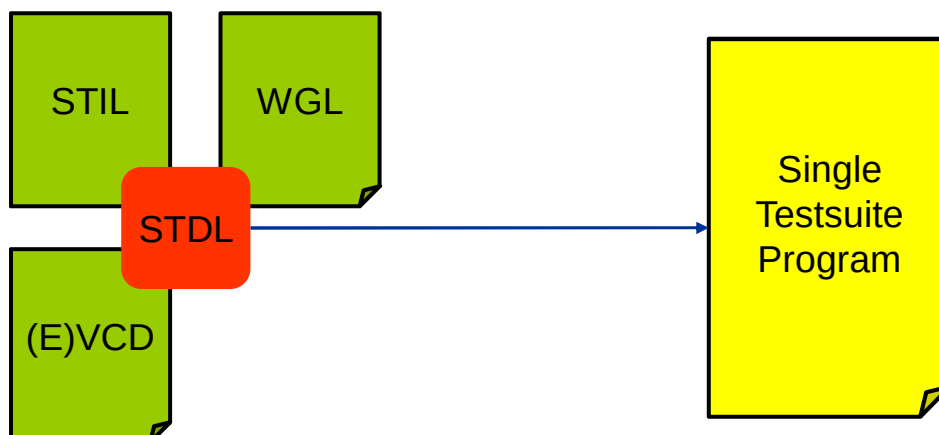
Terms & Definitions

STDL (SmarTest Data Link)

- Advantest's conversion tool to produce setup files from design test data in STIL, WGL and VCD format
- It is separated from SmarTest **with dedicated installation files and licenses**

SmarTest Data Link From Design to V93000

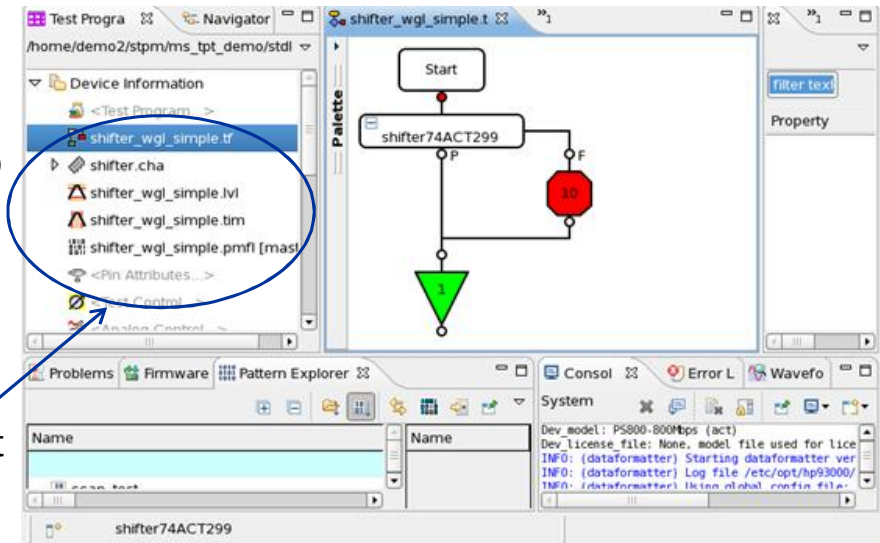
- STDL is a dedicated toolset that is run from the command line
- Input is design test data in the widespread formats STIL, WGL, (E)VCD and setup files that control the conversion process
- Output is a valid SmarTest device directory that contains
 - A single Testflow
 - A single Testsuite that refers to the converted design data



STDL Output

Single Testsuite Program

- Output is a valid V93000 device directory
- Loaded into SmarTest in order to check that the conversion was set up correctly
- Contains a single Testflow, and a single Testsuite, that refers to the converted data including pin configuration, levels and timing setup and patterns



ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 9

List of STDL Binary Names

Core tools

- | | |
|-------------------|--|
| stilreader | Conversion STIL, WGL & TDL to V93000 binary |
| vcdstil | Conversion (E)VCD to STIL |
| 93kbench* | Generation of a Verilog test bench from a V93000 Testsuite |

Viewers

- | | |
|--------------|--|
| waver | Waveform viewer for (E)VCD, STIL and WGL files |
| ste | Viewer and editor for STIL waveforms |

***will not be discussed in this training**

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 10

Usage of STDL Tools

- All the tools of SmartTest Data Link are used similarly; here for example **stilreader** for conversion of STIL/WGL/TDL files

```
> stilreader -help
```

```
Running STIL Reader (Version 2.2.1 [linux x86_64] 15-Sep-2011).  
(c) Copyright 2000-2011 Verigy Ltd.
```

```
Usage: stilreader-2.2 [-help] [-version] [-timestamp] [-licwait] [-debug] [-template] [-setup <str>]  
[-input_file_type <str>] [-target <str>] [-workdir <str>] [-project_name <str>] [-unify_periods] [-pa  
ttern_comment <str>] [-label_suffix <str>] [input file (STIL, WGL or TDL)...]  
-help                display this message  
-version             show version number and exit  
-timestamp           enable log timestamp  
-licwait             wait for the license, if not available  
-debug              generate intermediate file(s) to simplify debug ( application depende  
nt )  
-template            generate setup template  
-setup:<str>          path to setup file  
-input_file_type:<str> input file type (STIL|WGL|TDL)  
-target:<str>         target ATE type  
-workdir:<str>        output directory  
-project_name:<str>   project name  
-unify_periods       unify periods  
-pattern_comment:<str> pattern comment  
-label_suffix:<str>  pattern label suffix
```

- **-help** explains all the options for starting the tool command line

- **-template** generates a template of a setup file including explanations

- Use this to learn about features

```
> stilreader -template
```

```
The tool will generate template setup file in './stilreader_setup_template.setup' and silently exit.
```

```
> stilreader -setup setup/vcdstil93k_simple.setup
```

- Most STDL tools are called **<toolname> -setup:<setupfile>**
- More help is in the [TDC](#) or under **/opt/93000/stdl/doc/**

ADVANTEST[®]

STDL Examples

The following slides are about:

- Converting a WGL file to V93000 data
- Converting a STIL file to V93000 data
- Converting a VCD file to V93000 data

The conversion of the VCD file includes cyclization: conversion from the event driven format VCD (=Value Change Dump) to a cyclized format (STIL) as an intermediate step

ADVANTEST[®]

WGL Pattern Conversion Example

Conversion command

stilreader -setup:setup/wgl_simple.setup

- Converts WGL pattern "shifter74ACT299.wgl" given in subdirectory **/wgl/**
- WGL file path and all other setup data controlling the conversion process is given in file **/setup/wgl_simple.setup**

ADVANTEST[®]

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 13

WGL Pattern Conversion Setup File

Defines

- Path to input WGL pattern file
- Re-naming of signal names
- Dedicated STIL file "channels.stil" is linked into conversion to correctly assign channels
- Other data: Default levels, target tester, path of new device directory, etc.

```
#####
# WGL Reader Configuration
#####

input_file = 'wgl/shifter74ACT299.wgl'

#####
# V93000 Generator Configuration
#####

# Rename signals. (Optional)
# Define a mapping between simulation signals and target
rename_signals = { 'DS0' : 'D0', 'DS7' : 'D7', 'IO0' : 'IO[0]',
                  'IO1' : 'IO[1]', 'IO2' : 'IO[2]',
                  'IO3' : 'IO[3]', 'IO4' : 'IO[4]',
                  'IO5' : 'IO[5]', 'IO6' : 'IO[6]',
                  'IO7' : 'IO[7]', '_MR' : 'MR' }

# Files containing additional STIL blocks. (Optional)
external_data_files = [ 'stil/channels.stil' ]
default_definitions = [ 'stil/defaults.stil' ]

target = '93000DD'
workdir = './wgl_simple_dev'
project_name = 'shifter_wgl_simple'

#####
# 93000 DD flags
#####

smartest_version = "6.5.4"
use_unified_memory = 1
xmode_factor = 1
create_statemapping = 1
xmode_based_statemapping = 1
testmethod_mode = 'utm'
```

setup/wgl_simple.setup

```
STIL 1.0 {
  Design D22;
}

Environment DefaultChannels {
  NameMaps "93kChannel:shifter" {
    Signals {
      CP      "10302,Pin 12";
      D7      "10308,Pin 18";
      D0      "10301,Pin 11";
      IO[0]   "10314,Pin 7";
      IO[1]   "10303,Pin 13";
      IO[2]   "10313,Pin 6";
      IO[3]   "10304,Pin 14";
      IO[4]   "10312,Pin 5";
      IO[5]   "10305,Pin 15";
      IO[6]   "10311,Pin 4";
      IO[7]   "10306,Pin 16";
      Q0      "10315,Pin 8";
      Q7      "10307,Pin 17";
      S0      "10310,Pin 1";
      S1      "10309,Pin 19";
      MR      "10316,Pin 9";
      Vcc     "11,Pin 20";
    }
  }
}
```

stil/channels.stil

ADVANTEST[®]

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 14

Multiple WGL Files Conversion with STDL

Assume, that subdirectory **/wgl/** contains more WGL files, e.g.

- shifter74ACT299_1.wgl
- shifter74ACT299_2.wgl
- shifter74ACT299_3.wgl

These files can be converted with a single setup file as follows

- **stilreader -setup:setup/wgl_simple.setup -workdir:wgl_dev_1 -project_name:wgl_1 wgl/shifter74ACT299_1.wgl**
- **stilreader -setup:setup/wgl_simple.setup -workdir:wgl_dev_2 -project_name:wgl_2 wgl/shifter74ACT299_2.wgl**
- **stilreader -setup:setup/wgl_simple.setup -workdir:wgl_dev_3 -project_name:wgl_3 wgl/shifter74ACT299_3.wgl**

Simple script can call STDL multiple times to convert multiple files

STIL Pattern Conversion Example

Conversion command

stilreader -setup:setup/stil_simple.setup

According to setup file “scan_full_spec.setup”, this command converts the STIL pattern “scan_full_spec.stil” given in subdirectory **/stil/**

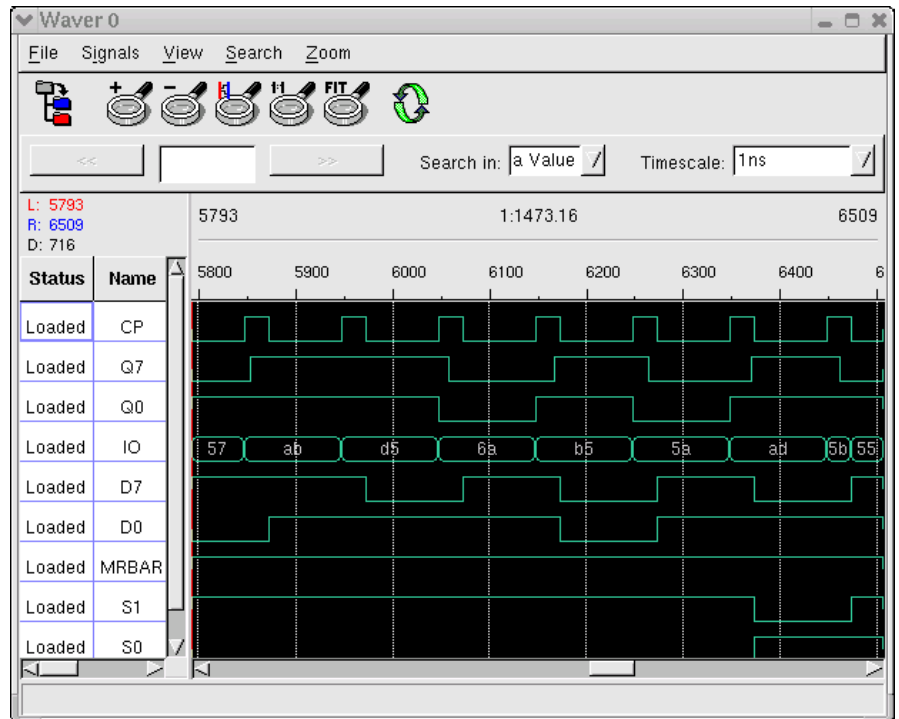
STIL allows to define much more beyond pins, patterns and timing:

- Specs
- Levels
- Channel assignments
- Pin groups
- Ports with X-mode factors
- Scan structures
- ...

Note: See files (e.g. “scan_full_spec.stil”) in the lab exercise for more information!

VCD Waveform Viewed in „waver”

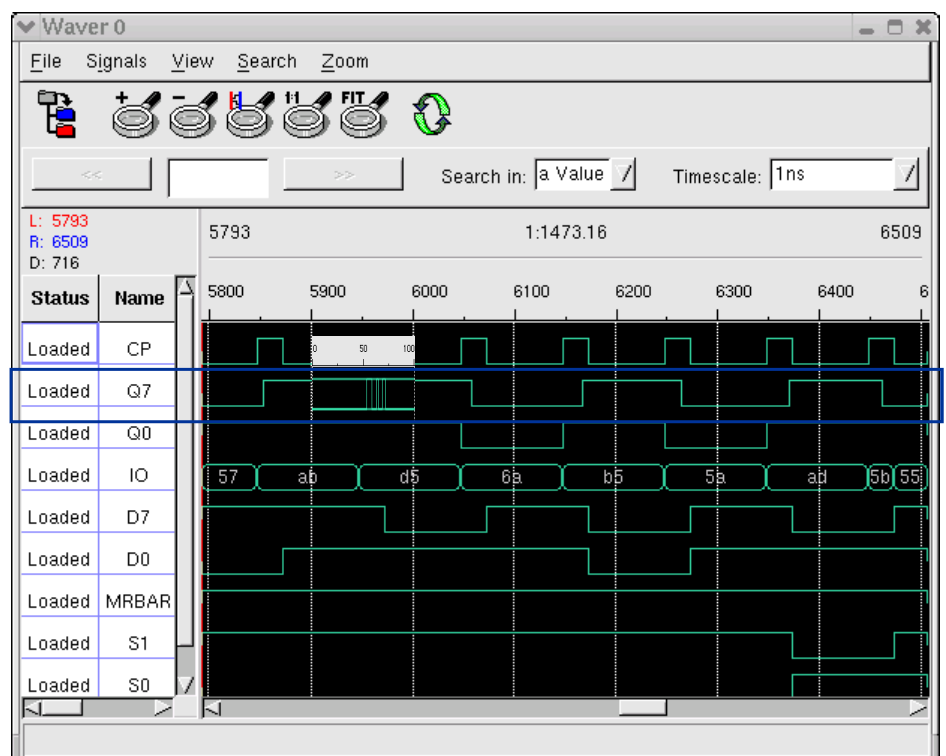
- SmarTest Data Link includes **waver**, a viewer for pattern files
- Ideally all signal changes are well aligned to a certain period
- In practice, often edges are not aligned, then checking edges in **waver** can help to setup cyclization in the right way



ADVANTEST

VCD Conversion: Edge Alignment and Cyclization

- In the example the edges of changing values of output “Q7” are not aligned
- This is clearly shown in the artificially created diagram that overlays several edges of “Q7” for the 100ns clock period
- During cyclization these edges must be aligned



ADVANTEST

Steps of VCD Conversion

In the following, an example for VCD conversion is discussed that is based on these steps:

- 1.Auto-generate a CSV file that lists all IO pins
- 2.Add and modify data in the list of pins
- 3.Auto-generate a CSV file for timing/waveforms with edited pin list
- 4.Modify timing/waveforms
- 5.Run conversion to cyclize and generate a STIL file
- 6.Review cyclization: Compare converted STIL data with VCD data
- 7.Convert STIL file to V93000 binary data

VCD Conversion CSV Setup Files

For VCD file conversion STDL needs a setup file and additional CSV files defining pins, timing, groups, ports etc.

CSV files are optional except for the files containing pins and timing information

The pin CSV setup file lists all pins and their properties:

- pin direction (in/out/inout)
- for bidirectional pins the signals that control the direction

STDL can analyze signals given in a VCD file and store the result in a CSV file in the appropriate format

VCD Conversion

1. Auto-Generate Pin List

Example command to generate a list of pins in CSV format

`vcdstil -work_mode:gen_pins -setup:setup/vcdstil_simple_gen_pins.setup`

The setup file points to the input VCD file from that the pin list should be extracted and to the output file for the pin list, which is "setup/shifter_autogen_pins.csv"

```
#####
# VCDSTIL Configuration
#####
#####
# External Files
pins_file = './setup/shifter_autogen_pins.csv' # pins definitions file (mandatory)
#####
# Conversion Attributes
input_files = ['vcd/syncshifter.vcd'] # input vcd/evcd files list (mandatory)
#####
# Pin Names Extraction
keep_design_hierarchy = 0 # keep full design path in pin names
select_design_modules = [] # process pins in the selected modules only
# (default = select all)

setup/vcdstil_simple_gen_pins.setup
```

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 21

VCD Conversion

2. Edit Pin List Data

Pin list generated by STDL

	A	B	C	D	E
1	<VS Pins>	1			
2	Pin	Type	I/O Controls	State Map	SSP
3	CP	in			0
4	D0	in			0
5	D7	in			0
6	IO[7]	in			0
7	IO[6]	in			0
8	IO[5]	in			0
9	IO[4]	in			0
10	IO[3]	in			0
11	IO[2]	in			0
12	IO[1]	in			0
13	IO[0]	in			0
14	MRBAR	in			0
15	OE1bar	in			0
16	OE2bar	in			0
17	Q0	in			0
18	Q7	in			0
19	S0	in			0
20	S1	in			0
21	Vcc	in			0
22	d_le[7]	in			0
23	d_le[6]	in			0
24	d_le[5]	in			0
25	d_le[4]	in			0

- Fix the pin direction ("Type")
- Remove unused pins

Edited pin list

	A	B	C	D	E
1	<VS Pins>	1			
2	Pin	Type	I/O Controls	State Map	SSP
3	CP	in			0
4	D0	in			0
5	D7	in			0
6	IO[7]	inout	in:oe=0,out:oe=1		0
7	IO[6]	inout	in:oe=0,out:oe=1		0
8	IO[5]	inout	in:oe=0,out:oe=1		0
9	IO[4]	inout	in:oe=0,out:oe=1		0
10	IO[3]	inout	in:oe=0,out:oe=1		0
11	IO[2]	inout	in:oe=0,out:oe=1		0
12	IO[1]	inout	in:oe=0,out:oe=1		0
13	IO[0]	inout	in:oe=0,out:oe=1		0
14	MRBAR	in			0
15	OE1bar	in			0
16	OE2bar	in			0
17	Q0	out			0
18	Q7	out			0
19	S0	in			0
20	S1	in			0
21	<VS Pins>				
22					
23					
24					
25					

- For bidirectional pins, define control for input/output; here an IO pin is an input, if signal "oe" is "0", and output if "oe=1"

ADVANTEST

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 22

VCD Conversion

Timing Modifications

Almost always it is not sufficient to directly convert the timing in the VCD file into a timing for the ATE without modifications.

Therefore, for VCD file conversion it is mandatory to provide for STDL a CSV setup file for timing and waveforms. The rows of this file can be considered as translation rules for the timing.

STDL can help to come up with a CSV setup file for timing:

It can analyze the timing of events/edges given in a VCD file and dump the results in such a CSV file.

In order to auto-generate a CSV file for timing, the user has to set correctly the clock period in the setup file. Often it is the period of the clock used during simulation when generating the VCD file.

The resulting CSV file is then a good starting point to generate a proper timing setup file for conversion.

VCD Conversion

3. Auto-Generate Timing

Example command to generate a timing in CSV format:

vcdstil -work_mode:gen_timing -setup:setup/vcdstil_simple_gen_tim.setup

```
#####
# VCDSTIL Configuration
#####

#####
# External Files
pins_file   = './setup/shifter_pins.csv'   # pins definitions file (mandatory)
timing_file  = './setup/shifter_autogen_tim.csv, # timing definitions file (mandatory)

#####
# Conversion Attributes
input_files = ['vcd/syncshifter.vcd']      # input vcd/evcd files list (mandatory)
start_time  = ""                          # extraction start time ( default = simulation start )
end_time    = ""                          # extraction end time  ( default = simulation end )
cycle_width = '100ns'                     # cycle width
active_pins  = ['D0', 'D7', 'S0', 'S1', 'MRBAR', 'CP', 'Q?', 'IO'] # active pins

#####
# Pin Names Extraction
keep_design_hierarchy = 0 # keep full design path in pin names
select_design_modules = [] # process pins in the selected modules only (default = select all)

#####
# Globals
time_unit = 'ns' # default equations time unit [ fs | ps | ns | us | ms | s ]

setup/vcdstil_simple_gen_tim.setup
```

Cyclization or

How VCD Timing Is Converted to STIL

1. **vcdstil** tool reads event driven data in VCD file
2. For each active pin the whole sequence of value changes at the pin is split into VCD cycles; the period is given by the parameter „cycle_width“ of the setup file
3. For each pin and for each cycle of that pin:
 - Ø According to the data in the pin list CSV file, **vcdstil** determines if the pin is working in that cycle as an input, output or bidirectional pin
 - Ø From the top to the bottom, **vcdstil** reads the rows of the timing CSV file that are related to the pin - until the currently processed VCD cycle matches the conditions of a row:
 - If such a row is found, the VCD cycle is translated to a STIL waveform as described in the row; Waveform data is stored
 - If no matching row can be found, then **vcdstil** throws an error and ends conversion
4. If no error occurred, the STIL waveforms and the corresponding vector data referring to the waveforms are stored in a STIL file

Matching VCD Cycles to CSV Timing Data

- If the condition given in the „Condition“ column matches data of the VCD cycle, **vcdstil** generates a STIL waveform as defined in the „Waveform“ column
- An entry in the „Waveform“ column can refer to multiple STIL waveforms; it can specifically address only inputs or only outputs; this is checked when matching VCD cycles

	A	B	C	D
1	<VS Timing>		1	
2	Pin	WFC	Waveform	Condition
3	D0	D	0ns:D	waveform
4	D0	U	0ns:U	waveform
5	D7	D	0ns:D	waveform
6	D7	U	0ns:U	waveform
7	S0	D	0ns:D	waveform
8	S0	U	0ns:U	waveform
9	S1	D	0ns:D	waveform
10	S1	U	0ns:U	waveform
11	MRBAR	D	0ns:D	waveform
12	MRBAR	U	0ns:U	waveform
13	CP	D	0ns:D; 50ns:U; 75ns:D	waveform
14	Q0	L	0ns:Z; 0ns:L	waveform
15	Q0	H	50ns:Z; 50ns:H	waveform
16	Q0	I	50ns:Z; 50ns:L	waveform
68	IO[0]	L	0ns:Z; 0ns:L	waveform
69	IO[0]	H	50ns:Z; 50ns:H	waveform
70	IO[0]	I	50ns:Z; 50ns:L	waveform
71	IO[0]	U	0ns:U	waveform
72	IO[0]	D	0ns:D	waveform
73	</VS Timing>			

- If an entry in the „Waveform“ column defines a single waveform, then a user defined waveform character (WFC) can be provided
- Head and tail lines are mandatory for the setup file

Resulting Auto-Generated Timing

Automatically generated CSV file with the timing and waveforms from the analysis of the VCD file

Condition “waveform”

- For conversion the waveform in the VCD file must perfectly match the waveform definition in the row

Waveform definitions

- “0ns:D” means drive “0” at 0 ns
- “0ns:U” means drive “1” at 0 ns
- “50ns:Z; 50ns:H” switches of driver and expects “1” at 50 ns

	A	B	C	D
1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	D0	D	0ns:D	waveform
4	D0	U	0ns:U	waveform
5	D7	D	0ns:D	waveform
6	D7	U	0ns:U	waveform
7	S0	D	0ns:D	waveform
8	S0	U	0ns:U	waveform
9	S1	D	0ns:D	waveform
10	S1	U	0ns:U	waveform
11	MRBAR	D	0ns:D	waveform
12	MRBAR	U	0ns:U	waveform
13	CP	D	0ns:D; 50ns:U; 75ns:D	waveform
14	Q0	L	0ns:Z; 0ns:L	waveform
15	Q0	H	50ns:Z; 50ns:H	waveform
16	Q0	I	50ns:Z; 50ns:L	waveform
17	Q7	L	0ns:Z; 0ns:L	waveform
18	Q7	H	63ns:Z; 63ns:H	waveform
19	Q7	I	56ns:Z; 56ns:L	waveform
20	Q7	h	54ns:Z; 54ns:H	waveform
21	Q7		0 63ns:Z; 63ns:L	waveform
22	Q7		1 56ns:Z; 56ns:H	waveform
23	Q7		2 52ns:Z; 52ns:L	waveform
24	Q7		3 55ns:Z; 55ns:L	waveform
25	Q7		4 53ns:Z; 53ns:H	waveform
26	Q7		5 55ns:Z; 55ns:H	waveform
27	IO[7]	L	0ns:Z; 0ns:L	waveform
28	IO[7]	H	50ns:Z; 50ns:H	waveform
29	IO[7]	I	50ns:Z; 50ns:L	waveform
30	IO[7]	U	0ns:U	waveform
31	IO[7]	D	0ns:D	waveform
32	IO[7]	h	0ns:Z; 0ns:H	waveform
33	IO[6]	I	0ns:Z; 0ns:L	waveform

ADVANTEST®

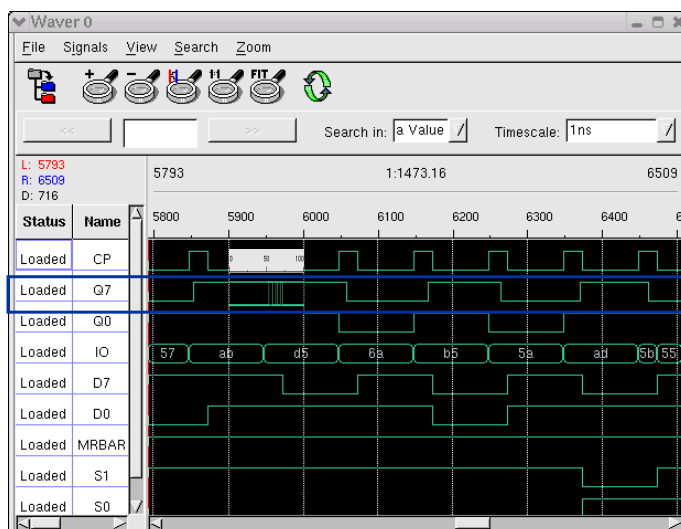
April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 27

Automatically Generated Timing for “Q7”

- In this example the edges of output “Q7” are not aligned
- STDL generates for each edge with a specific timing in the VCD file a line with the corresponding waveform in the new timing CSV file



18	Q7	H	63ns:Z; 63ns:H	waveform
19	Q7	I	56ns:Z; 56ns:L	waveform
20	Q7	h	54ns:Z; 54ns:H	waveform
21	Q7		0 63ns:Z; 63ns:L	waveform
22	Q7		1 56ns:Z; 56ns:H	waveform
23	Q7		2 52ns:Z; 52ns:L	waveform
24	Q7		3 55ns:Z; 55ns:L	waveform
25	Q7		4 53ns:Z; 53ns:H	waveform
26	Q7		5 55ns:Z; 55ns:H	waveform

- Many lines for a certain pin in the automatically generated CSV always indicate, that the edges of this pin might be not well aligned

ADVANTEST®

April 2016

All Rights Reserved - ADVANTEST CORPORATION

Smart Test Data Link Unit 25 - 28

VCD Conversion

4. Edit Timing Data

Reasons for cleaning up the generated timing data

- Timing should be defined so that the ATE will perform tests according to the spec and not according to simulation results
- Less edges are used on the ATE if “moving” edges of pins like output “Q7” are retimed and mapped to a single edge
- Mapping of timing can be made more robust; then similar VCD files can be converted with one timing CSV file
- Often, entries can be simplified and the total number of entries can be reduced

ADVANTEST

VCD Conversion

4. Edit Timing Data

	A	B	C	D
1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	D0	D	0ns:D	waveform
4	D0	U	0ns:U	waveform
5	D7	D	0ns:D	waveform
6	D7	U	0ns:U	waveform
7	S0	D	0ns:D	waveform
8	S0	U	0ns:U	waveform
9	S1	D	0ns:D	waveform
10	S1	U	0ns:U	waveform
11	MRBAR	D	0ns:D	waveform
12	MRBAR	U	0ns:U	waveform
13	CP	D	0ns:D; 50ns:U; 75ns:D	waveform
14	Q0	L	0ns:Z; 0ns:L	waveform
15	Q0	H	50ns:Z; 50ns:H	waveform
16	Q0	I	50ns:Z; 50ns:L	waveform
17	Q7	L	0ns:Z; 0ns:L	waveform
18	Q7	H	63ns:Z; 63ns:H	waveform
19	Q7	I	56ns:Z; 56ns:L	waveform
20	Q7	h	54ns:Z; 54ns:H	waveform
21	Q7		0 63ns:Z; 63ns:L	waveform
22	Q7		1 56ns:Z; 56ns:H	waveform
23	Q7		2 52ns:Z; 52ns:L	waveform
24	Q7		3 55ns:Z; 55ns:L	waveform
25	Q7		4 53ns:Z; 53ns:H	waveform
26	Q7		5 55ns:Z; 55ns:H	waveform
27	IO[7]	L	0ns:Z; 0ns:L	waveform
28	IO[7]	H	50ns:Z; 50ns:H	waveform
29	IO[7]	I	50ns:Z; 50ns:L	waveform
30	IO[7]	U	0ns:U	waveform
31	IO[7]	D	0ns:D	waveform
32	IO[7]	h	0ns:Z; 0ns:H	waveform
33	IO[6]	I	0ns:Z; 0ns:L	waveform

- The number of lines have been reduced, in particular for pin “Q7”
- Conditions have been changed for all lines to “true”
- The enhanced waveform descriptions are always used if applicable

	A	B	C	D
1	<VS Timing>	1		
2	Pin	WFC	Waveform	Condition
3	D0		0ns:F@ (50ns)	TRUE
4	D7		0ns:F@ (50ns)	TRUE
5	S0		0ns:F@ (50ns)	TRUE
6	S1		0ns:F@ (50ns)	TRUE
7	MRBAR		0ns:F@ (50ns)	TRUE
8	CP		50ns:F@ (60ns); 75ns:D	TRUE
9	Q0		99ns:R@ (80ns)	TRUE
10	Q7		99ns:R@ (80ns)	TRUE
11	IO[7]		0ns:F@ (50ns)	TRUE
12	IO[7]		0ns:Z; 99ns:R@ (80ns)	TRUE
13	IO[7]		0ns:Z	TRUE
14	IO[6]		0ns:F@ (50ns)	TRUE
15	IO[6]		0ns:Z; 99ns:R@ (80ns)	TRUE
16	IO[6]		0ns:Z	TRUE
17	IO[5]		0ns:F@ (50ns)	TRUE

ADVANTEST

Waveform Definitions for Input Pins

Input pin settings in timing CSV file

- 0ns:F@(50ns)

For each tester cycle, a drive edge will be placed at 0 ns in the tester period, forcing the value given at 50 ns in the VCD file

“F” means “force”

It equals the following two waveform definitions:

- 0ns:D@(50ns)
- 0ns:U@(50ns)

in case a “0” is sampled

in case a “1” is sampled

For the clock “CP” it is defined:

- 50ns:F@(60ns); 75ns:D

Clock value is sampled at 60 ns and driven at 50 ns

At 75 ns the clock always returns to “0”

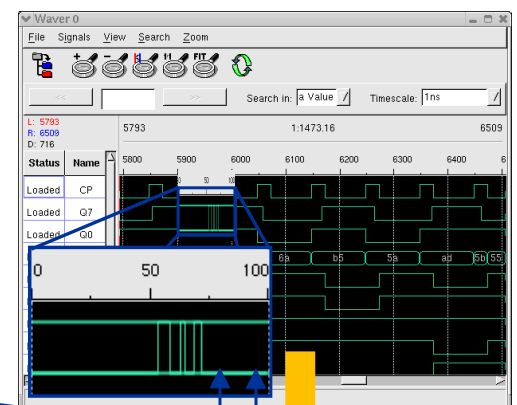
	A	B	C	D
1	<VS Tim	1		
2	Pin	WFC	Waveform	Condition
3	D0		0ns:F@(50ns)	TRUE
4	D7		0ns:F@(50ns)	TRUE
5	S0		0ns:F@(50ns)	TRUE
6	S1		0ns:F@(50ns)	TRUE
7	MRBAR		0ns:F@(50ns)	TRUE
8	CP		50ns:F@(60ns); 75ns:D	TRUE
9	Q0		99ns:R@(80ns)	TRUE
10	Q7		99ns:R@(80ns)	TRUE
11	IO[7]		0ns:F@(50ns)	TRUE
12	IO[7]		99ns:R@(80ns)	TRUE
13	IO[7]		0ns:Z	TRUE
14	IO[6]		0ns:F@(50ns)	TRUE
15	IO[6]		99ns:R@(80ns)	TRUE
16	IO[6]		0ns:Z	TRUE
17	IO[5]		0ns:F@(50ns)	TRUE

ADVANTEST

Output Data of “Q7” Aligning Edges

Assume that the DUT specification demands stable output values at 99 ns. Then edges of the output “Q7” are aligned by defining a waveform in the CSV file:

- Expected values are captured at the save sample point 80 ns:
99ns:R@(80ns)
- The V93000 edge is re-timed according to the specification, here:
99ns:R@(80ns)



	A	B	C	D
1	<VS Tim	1		
2	Pin	WFC	Waveform	Condition
3	D0		0ns:F@(50ns)	TRUE
4	D7		0ns:F@(50ns)	TRUE
5	S0		0ns:F@(50ns)	TRUE
6	S1		0ns:F@(50ns)	TRUE
7	MRBAR		0ns:F@(50ns)	TRUE
8	CP		50ns:F@(60ns); 75ns:D	TRUE
9	Q0		99ns:R@(80ns)	TRUE
10	Q7		99ns:R@(80ns)	TRUE
17	IO[5]		0ns:F@(50ns)	TRUE

ADVANTEST

Waveform Definitions for Output Pins

Output pin settings in timing CSV file:

- 99ns:R@(80ns)

For each tester cycle, a receive edge for the V93000 will be placed at 99 ns in the tester period, expecting the value given at 80 ns in the VCD file

“R” means “receive”

It equals the following two waveform definitions:

- 99ns:L@(80ns)
- 99ns:H@(80ns)

In case a “0” is sampled

In case a “1” is sampled

With this setting, the various edges for comparing values at pin “Q7” are addressed

	A	B	C	D
1	<VS Tim	1		
2	Pin	WFC	Waveform	Condition
3	D0		0ns:F@(50ns)	TRUE
4	D7		0ns:F@(50ns)	TRUE
5	S0		0ns:F@(50ns)	TRUE
6	S1		0ns:F@(50ns)	TRUE
7	MRBAR		0ns:F@(50ns)	TRUE
8	CP		50ns:F@(60ns); 75ns:D	TRUE
9	Q0		99ns:R@(80ns)	TRUE
10	Q7		99ns:R@(80ns)	TRUE
11	IO[7]		0ns:F@(50ns)	TRUE
12	IO[7]		0ns:Z; 99ns:R@(80ns)	TRUE
13	IO[7]		0ns:Z	TRUE
14	IO[6]		0ns:F@(50ns)	TRUE
15	IO[6]		0ns:Z; 99ns:R@(80ns)	TRUE
16	IO[6]		0ns:Z	TRUE
17	IO[5]		0ns:F@(50ns)	TRUE

ADVANTEST

Waveform Definitions for Bidir Pins

Bidirectional pins settings in timing CSV file

- 0ns:F@(50ns)

	A	B	C	D
1	<VS Tim	1		
2	Pin	WFC	Waveform	Condition
3	D0		0ns:F@(50ns)	TRUE
4	D7		0ns:F@(50ns)	TRUE
5	S0		0ns:F@(50ns)	TRUE
6	S1		0ns:F@(50ns)	TRUE
7	MRBAR		0ns:F@(50ns)	TRUE
8	CP		50ns:F@(60ns); 75ns:D	TRUE
9	Q0		99ns:R@(80ns)	TRUE
10	Q7		99ns:R@(80ns)	TRUE
11	IO[7]		0ns:F@(50ns)	TRUE
12	IO[7]		0ns:Z; 99ns:R@(80ns)	TRUE
13	IO[7]		0ns:Z	TRUE
14	IO[6]		0ns:F@(50ns)	TRUE
15	IO[6]		0ns:Z; 99ns:R@(80ns)	TRUE
16	IO[6]		0ns:Z	TRUE

For each tester cycle, for that the bidir pin is working as an input, a tester drive edge will be placed at 0 ns in the tester period, driving the value given at 50 ns in the VCD file

If the bidir pin is working as an output, this line is skipped

- 0ns:Z; 99ns:R@(80ns)

If the bidir pins is working in a cycle as an output, the driver will switched off at 0 ns and a tester receive strobe will be placed at 99 ns, expecting the value given at 80 ns in the VCD file

If pins are working as input, STD L will not reach this line, because the line above would be used for conversion first

- 0ns:Z

If both lines above are skipped (for example, because the pin direction is unknown), then the pin driver is simply switched off

ADVANTEST

VCD Conversion

5. Conversion VCD to STIL

Example command to convert VCD to STIL:

vcdstil -setup:setup/vcdstil_simple.setup

```
#####
# VCDSTIL Configuration
#####

#####
# External Files
pins_file   = './setup/shifter_pins.csv' # pins definitions file (mandatory)
timing_file  = './setup/shifter_timing.csv' # timing definitions file (mandatory)

#####
# Conversion Attributes
input_files = ['vcd/syncshifter.vcd'] # input vcd/evcd files list (mandatory)
stil_out_file = 'tmp_stil/vcdstil_simple.stil' # output STIL file
start_time  = "" # extraction start time ( default = simulation start )
end_time    = "" # extraction end time ( default = simulation end )
cycle_width = '100ns' # cycle width
active_pins  = ['D0', 'D7', 'S0', 'S1', 'MRBAR', 'CP', 'Q?', 'IO'] # active pins

#####
# Pin Names Extraction
keep_design_hierarchy = 0 # keep full design path in pin names
select_design_modules = [] # process pins in the selected modules only
                        # (default = select all)

#####
# Globals
time_unit      = 'ns' # default equations time unit [ fs | ps | ns | us | ms | s ]
stil_file_layout = 'all_in_one'

setup/vcdstil_simple.setup
```

ADVANTEST[®]

VCD Pattern Conversion: Setup File

If no specific work mode is enabled, the setup file „vcdstil_simple.setup“ controls these two steps:

- Cyclization: Reading and analyzing the VCD file
- Output: Writing the resulting STIL file after conversion

Among others, in the file is set:

- Paths to auxiliary CSV setup files defining pins and timing
- Path to VCD input and STIL output
- Tester period and time unit
- List of „active“ pins, only these are considered for conversion

The setup files given in the VCD example are simple and short because these make use of default settings

In order to find appropriate settings for complex tasks generate a setup file template:
vcdstil -template

ADVANTEST[®]

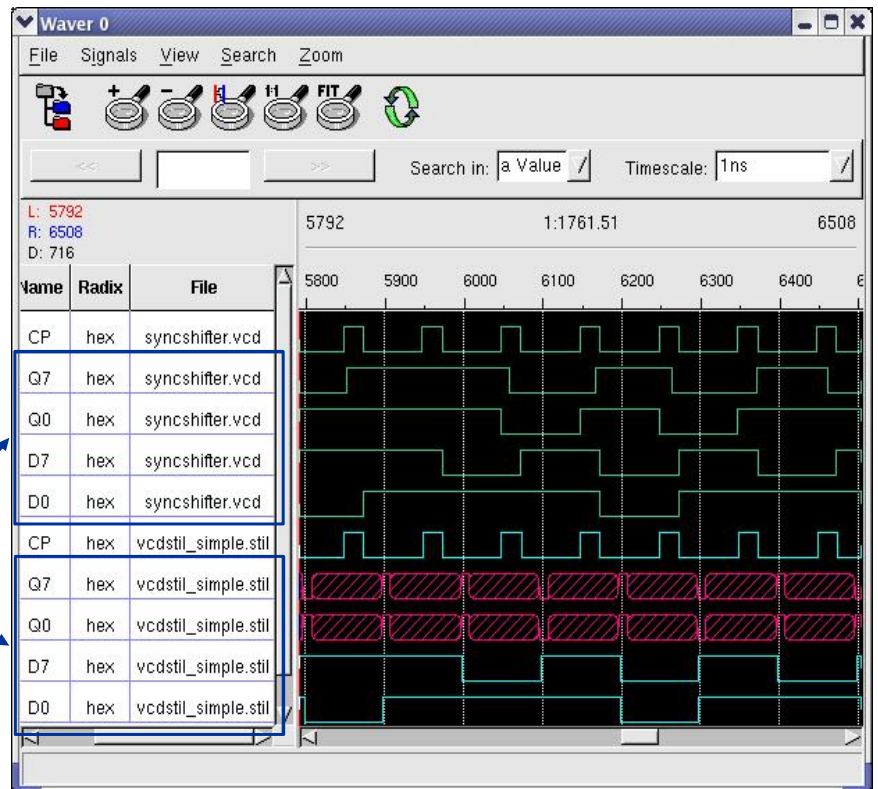
VCD Conversion

6. Review Cyclized Data

waver allows to display signals of various sources and formats in a single window:

- VCD
- EVCD
- STIL
- WGL

In this example **waver** shows signals before (VCD) and after (STIL) cyclization and conversion



ADVANTEST

VCD Conversion

7. Conversion of STIL File

Example command to convert VCD to STIL:

stilreader -setup:setup/vcdstl93k_simple.setup

Again, **stilreader** is used with a similar setup file as above

```
#####
# V93000 Generator Configuration
#####

target = '93000DD'
input_file = './tmp_stil/vcdstl_simple.stil'
workdir = './vcdstl93k_simple_dev'
project_name = 'shifter_vcd_simple'
rename_signals = { 'MRBAR' : 'MR' }

# Files containing additional STIL blocks. (Optional)
external_data_files = [ 'stil/channels.stil' ]
default_definitions = [ 'stil/defaults.stil' ]

#####
# 93000 DD flags
#####

smartest_version = "6.5.4"
use_unified_memory = 1
xmode_factor = 1
create_statemapping = 1
xmode_based_statemapping = 1
testmethod_mode = 'utm'

setup/vcdstl93k_simple.setup
```

ADVANTEST

Summary: Conversion of a VCD Pattern

In summary, with valid setup files only two commands are needed:

vcdstil –setup:setup/vcdstil_simple.setup

stilreader –setup:setup/vcdstil93k_simple.setup

The first step performs cyclization controlled by the setup file “vcdstil_simple.setup”: the VCD pattern is cyclized and converted into STIL format as an intermediate format

The second step is the conversion from STIL to V93000 controlled by the setup file “vcdstil93k_simple.setup”; this step is about optimizing the test pattern for the V93000

Reference

V93000 Technical Documentation Center (TDC):

Topic 105114

<https://www.advantest.com/V93000/help/index.jsp?tab=search&searchWord=105114>